Rockstar Developer

အိမောင်

Fairway Web

10x Programmer ခေါ် ဆယ်ဆပိုတော်သူတစ်ဦး
ဖြစ်လာအောင် အကူအညီပေးမယ့်စာအုပ်

[rsdbook.com]

© Copyright 2016, **Ei Maung**, Fairway Technology.



License - CC-BY-NC-SA

This document, “Rockstar Developer by Ei Maung ”is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License.

This document is free to share, copy, distribute and transmit. And, also allow to remix or adapt to this document. But, you must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work). And, you may not use this work for commercial purposes. If you alter, transform, or build upon this work, you may distribute the resulting work only under the same or similar license to this one.

For any reuse or distribution, you must make clear to others the license terms of this work.

THE WORK IS PROTECTED BY COPYRIGHT AND/OR OTHER APPLICABLE LAW. ANY USE OF THE WORK OTHER THAN AS AUTHORIZED UNDER THIS LICENSE OR COPYRIGHT LAW IS PROHIBITED -
http://creativecommons.org/licenses/by-nc-sa/3.0/legalcode

မာတိကာ

အမှာစာများ ၃

မိတ်ဆက် ၉

အပိုင်း (၁) – Key Concepts

အခန်း (၁) – Development Processes Management ၁၄

အခန်း (၂) – Test Driven Development ၂၇

အခန်း (၃) – SOLID Design Principles ၄၆

အခန်း (၄) – Service Oriented Architecture ၇၄

အပိုင်း (၂) – Project Management

အခန်း (၅) – Version Control System ၈၉

အခန်း (၆) – Issue Tracking System ၁၂၄

အခန်း (၇) – Team Collaboration ၁၄၀

အခန်း (၈) – Development Wrokflow Automation ၁၆၀

အပိုင်း(၃) – Modern Development Stack

အခန်း (၉) – BackboneJS (Client-side MVC) ၁၉၃

အခန်း (၁၀) – NodeJS (Server-side JavaScript) ၂၂၄

အခန်း (၁၁) – MongoDB (NoSQL Database) ၂၅၈

အခန်း (၁၂) – ExpressJS (REST Framework) ၂၉၄

အပိုင်း (၄) – Building an Issue Tracking System

အခန်း (၁၃) – Building Back-end Service ၃၁၉

အခန်း (၁၄) – Building Client (HTML Templates) ၃၅၀

အခန်း (၁၅) – Building Client (Web App) ၃၈၁

အခန်း (၁၆) – Building Client (Hybrid Mobile App) ၄၁၀

အပိုင်း (၅) – Deployment and Scaling

အခန်း (၁၇) – Deployment with Docker ၄၁၈

အခန်း (၁၈) – Load Balancing with Nginx ၄၃၃

အခန်း (၁၉) – MongoDB Replication and Sharded Cluster ၄၄၈

အခန်း (၂၀) – Common Server Architectures ၄၆၅

နိဂုံးချုပ် ၄၇၉

ကျေးဇူးတင်လွှာ ၄၈၀

ဆရာဦးထွန်းခိုင်၏အမှာစာ

နည်းပညာပြောင်းလဲတိုးတက်မှုတွေက အရှိန်အဟုန် မြင့်လွန်းလှသည်။ ကျွန်တော်တို့ ကွန်ပျူတာနှင့် စတင်ထိတွေ့ရပြီး Programming ဆိုတာဘာလဲဆိုတာကို မြည်းစမ်းခဲ့ရတဲ့ (၁၉၈၉) လောက်ကနေ ယနေ့အချိန်အထိမှာ အများကြီး အများကြီးကို ပြောင်းလဲကုန်ပြီ။

ကိုယ်တွေ့ခေတ်က အင်တာနက်ဆိုတာလည်းမရှိသေး။ စာအုပ်စာတမ်းဆိုတာကလည်း အလွန်ကိုရှား။ Programming ကို စလေ့လာတော့မယ်ဆိုတော့ MCC မှာ သင်တန်းမတက်ခင်၊ ဗြိတိသျှသံရုံးက စာကြည့်တိုက်ကိုသွားပြီး Programming နဲ့ ဆိုင်မယ့် စာအုပ်သွားရှာတော့ Low Level Programming ဆိုတဲ့စာအုပ်လေးတွေ့ရော။ အင်း၊ ငါလည်းအခုမှလေ့လာမှာ ဆိုတော့ Low Level ပဲ စဖတ်ကြည့်တာပေါ့ဆိုပြီး ငှားလာခဲ့တယ်။ ညဦးပိုင်းကနေ စဖတ်တာ ၂ နာရီ ၃ နာရီလောက် ကြိုးစားဖတ်ပါသော်လည်း တစ်လုံးတစ်ပါဒမှ နားမလည်။ စိတ်အားငယ်ရပြီ။ Low Level တောင် ဒီလောက်ခက်နေရင် High Level ဆိုဘယ့်နှယ်လုပ်ကြမတုန်း။ အခုပြန်ပြောပြတော့တာ ပြုံးရရယ်ရပေမဲ့ အဲဒီတုန်းကတော့မပြုံးနိုင်ခဲ့။ ကူညီ သင်ပြပေးမဲ့သူလည်းနည်း၊ စာအုပ်စာတမ်းကရှား၊ ကွန်ပျူတာစာအုပ်ဝယ်လို့ရတာ ဆိုလို့ မဂိုလမ်းနဲ့ ကုန်သည်လမ်းထောင့်က ရွှေမြိုင်ဆိုတဲ့မိတ္တူကူးဆိုင်လေးပဲရှိတယ်။

(၁၉၈၉) လောက်ကနေ အခုချိန်အထိကို Software Development နဲ့ ပတ်သက်လို့ Language မျိုးစုံ BASIC, Pascal , COBOL, C, C+, VB, Standalone, Client-Server, Three-Tier, Web စသည်ဖြင့် ပုံစံမျိုးစုံ ကြုံတွေ့ ကျင်လည်ခဲ့ရပြီ။ Sun Microsystem က John Gage (၁၉၈၄) မှာ ပြောခဲ့တဲ့ "Network is the computer" ဆိုတဲ့ စကားလေးက အခုချိန်မှ တကယ်အကောင်အထည်ပေါ်လာခဲ့တာ။ Cloud Computing လို နည်းပညာတွေ၊ Mobile နည်းပညာတွေ၊ Internet of Things တို့လို နည်းပညာတွေနဲ့ နေရာတကာ ကွန်ပျူတာဖြစ်ကုန်ကြပြီ။ ကွန်ပျူတာဆိုတာ ကျွန်တော်တို့ခေတ်က လို Monitor နဲ့ Keyboard, Mouse တပ်ထားထဲ့ CPU ပုံးပါတဲ့ အရာတစ်မျိုးတည်း မဟုတ်တော့။ လက်ကိုင်ဖုန်း၊ နာရီကအစ ကိုယ်မောင်းတဲ့ ကားထဲမှာလည်း ကွန်ပျူတာပါလာပြီ။ ဒီလိုအခြေနေမှာ Software Development ရဲ့ အခန်းကဏ္ဍကလည်း အရမ်းကို စိတ်ဝင်စားဖွယ်ဖြစ်လာပြီပေါ့။

ဒီလိုအချိန်မှာ ကိုအိမောင်ရဲ့ အခုစာအုပ်လေးက မြန်မာပြည်က Software Development ကို စိတ်ပါဝင်စားတဲ့ လူငယ်များအတွက် တစ်ဖက်တစ်လမ်းက အထောက်ကူဖြစ်မယ်ဆိုတာ အသေအချာပါပဲ။ အရင်ထွက်ထားတဲ့ စာအုပ်နှစ်အုပ်မှာကတည်းက စာရေးသူရဲ့စေတနာကို မြင်ရခံစားခဲ့ရတယ်။ အခုစာအုပ်လေးမှာလည်း Background Theory များနဲ့အတူ၊ ပြောင်းလဲလာတဲ့ယနေ့ခေတ်မှာ လက်တွေ့အသုံးဝင်မယ့် နည်းပညာများအကြောင်းကို သိုင်းဝိုင်းပြီး တင်ပြထားတာ စိတ်ဝင်စားစရာပါ။

ကိုအိမောင်အနေနဲ့လည်း လူငယ်တွေအတွက်လိုအပ်နေတဲ့ကွက်လပ်လေးတွေကို ဆက်ပြီးဖြည့်သွားနိုင်ပါစေလို့ဆန္ဒပြုရင်း ...

ထွန်းခိုင်
Managing Director (Enterprise Solutions)
Vice President 2 (MCPA)

ကိုသာထက်၏အမှာစာ

ကိုအိမောင်ကအမှာစာရေးပေးပါဟုစာမူကြမ်းပေးဖတ်သဖြင့် Rockstar Developer စာမူကိုအသေအချာဖတ်ပြီး၊ ရှုထောင့် စုံအောင် စဉ်းစားကြည့်မိသည်။

ဂီတလောကတွင် Rockstar ဟူသည် ထူးခြားထင်ရှား ကျော်ကြားကာ၊ လူအများ အားကျနှစ်ချိုက်သူကို ဆိုလိုသကဲ့သို့ Software Developer လောကတွင်လည်း ထို့နည်းတူပင် ဖြစ်သည်။ ဤစာအုပ်၏ရည်ရွယ်ရင်းမှာ စိတ်ပါဝင်စားသူတိုင်း Rockstar ဖြစ်နိုင်စေရန် တတ်သိသင့်သည်များကို ပြုစုတင်ပြထားသည်ဟု ရှုမြင်မိသည်။

ဤစာအုပ်တွင် သီဝရီ (Theory) နှင့် လက်တွေ့ကိရိယာ (Tools) များကို မျှတစွာ တင်ပြထားသည်ကိုတွေ့ရသည်။

အခြေခံ အတွေးပုံစံသီဝရီများဖြစ်သည့် TDD, SOLID, Database Scaling နှင့် အသုံးများသည့် Server Architecture များကို တင်ပြထားသလို ခေတ်သစ် Development အဆင့်များဖြစ်သည့် MEAN Stack နှင့် နောက်ဆုံးခေတ်မီ Docker ကဲ့သို့ Container အခြေပြု ဖြန့်ကျက်နည်းကိုလည်း ရှင်းပြထားသည်။ ထို့ပြင် စနစ်ကျသော Developer များ၏ အလေ့အကျင့်ကောင်းများကိုလည်း Project Management ကဏ္ဍတွင် လက်တွေ့အသုံးချနိုင်ရန် ရေးသားထားသည်။

အသစ်တက်သော လေ့လာသူအများအတွက် ဖတ်ရှုရန် အနည်းငယ်ခက်ခဲနိုင်သော်လည်း၊ အတွေ့အကြုံ အတန်အသင့်ရှိနှင့်ပြီးသော Developer များအဖို့ သိကောင်းစရာများစွာ ပါဝင်လေသည်။

အပိုင်း (၅) ပိုင်းနှင့် တင်ပြထားသည်ကိုဖတ်ရသည်မှာ မဝမလင် ဖြစ်သည်။ ကဏ္ဍတစ်ခုချင်းစီတွင် အဆင့်မြင့် ခေါင်းစဉ်နှင့် ခက်ခဲသောအကြောင်းရာများကို နားလည်လွယ်ကူစေရန် ကျစ်ကျစ်လစ်လစ် ရေးသားထားရာ၊ ယခုထက် ပိုမိုကျယ်ပြန့်သော စာအုပ်တစ်အုပ် ထပ်မံရေးသားပါမှ ပြည့်စုံခြင်းသို့ ရောက်ပေမည်။

မည်သို့ပင်ဆိုစေ လေ့လာသူများအနေဖြင့် အခန်းတစ်ခုခြင်းစီမှပေးသော အခြေခံကောင်းဖြင့် မိမိဘာသာ ကျယ်ကျယ် ပြန့်ပြန့်လေ့လာရန် လုံလောက်သောပညာကို ယခုစာအုပ်မှ ပေးစွမ်းနိုင်မည်မှာ အသေအချာပင်ဖြစ်လေသည်။

သာထက်
CEO (Zwenexsys International Limited)

စာရေးသူ၏အမှာ

မင်္ဂလာပါခင်ဗျာ။ Software Development နဲ့ ကွန်ပျူတာနည်းပညာကို လေ့လာနေသူများ အထောက်အကူဖြစ်စေဖို့ ဆိုတဲ့ ရည်ရွယ်ချက်နဲ့၊ ကျွန်တော်လေ့လာသိရှိထားတဲ့နည်းပညာတွေကို ကဏ္ဍအလိုက်ရေးသားပြုစုလာခဲ့တာ အခုဆိုရင် တတိယမြောက်စာအုပ်ကို ရောက်ရှိလာခဲ့ပြီဖြစ်ပါတယ်။

Rockstar Developer လို့ အမည်ပေးထားတဲ့ ဒီစာအုပ်ဟာ ကျွန်တော်ရေးမှာရေးခဲ့တဲ့ စာအုပ်နှစ်အုပ်နဲ့ ဆက်စပ်နေပါတယ်။ ကျွန်တော့်ရဲ့ ပထမဆုံးစာအုပ်ဖြစ်တဲ့ **Professional Web Developer** မှာ Web Standards, jQuery, HTML5 စတဲ့ အခြေခံနည်းပညာများနဲ့အတူ PHP, MySQL, MVC, Web Application Security စတဲ့ Web Developer တစ်ဦး သိဖို့လိုအပ်တဲ့ နည်းပညာတွေကို စုစည်းဖော်ပြထားပါတယ်။ ဒုတိယစာအုပ်ဖြစ်တဲ့ **Ubuntu – သင့်အတွက် Linux** မှာတော့ Web Developer အပါအဝင် Software Developer တိုင်းလိုလိုသိရှိထားသင့်တဲ့ Linux Operating System တွေထဲက အထင်ရှားဆုံးတစ်ခုဖြစ်တဲ့ Ubuntu အကြောင်းကို ဖော်ပြထားပါတယ်။

ဒီစာအုပ်မှာတော့ ပြုပြင်ထိမ်းသိမ်းရလွယ်ကူတဲ့ Code တွေရေးသားရာမှာ အထောက်အကူဖြစ်စေနိုင်တဲ့ နည်းစနစ်တွေ၊ အဖွဲ့လိုက်ဆောင်ရွက်ရာမှာ အထောက်အကူဖြစ်စေနိုင်တဲ့ Project Management ပိုင်းဆိုင်ရာ သိမှတ်ဖွယ်ရာတွေ၊ NodeJS, MongoDB စတဲ့ Software Development နည်းပညာတွေနဲ့ Server Architecture ပိုင်းဆိုင်ရာ သိမှတ်ဖွယ်ရာတွေကို စုစည်းဖော်ပြပေးသွားမှာ ဖြစ်ပါတယ်။

ဒီစာအုပ်မှာဖော်ပြမယ့် နည်းပညာများက Web Development ပိုင်းကို အသားပေးတဲ့ နည်းပညာများ ဖြစ်ပါတယ်။ Business Solution, Web App, Mobile App, Game စသဖြင့် Software အမျိုးအစားအမျိုးမျိုးရှိပေမယ့် ကနေ့ခေတ်မှာ ဘယ်လို Software အမျိုးအစားမဆို Web Technology များနဲ့ ကင်းကွာလို့ မရတော့ပါဘူး။ မည်သည့် Software မဆို Web Technology ကို Back–end အဖြစ်သော်လည်းကောင်း၊ ကြားခံဆက်သွယ်ရေး နည်းပညာအနေနဲ့သော် လည်းကောင်း အသုံးပြုကြရပါတယ်။ ဒါကြောင့် ဒီစာအုပ်မှာ ဖော်ပြမယ့် နည်းပညာများဟာ Web Developer များအတွက်သာမက၊ Mobile App Developer များ၊ Business Solution Developer များနဲ့ Software Engineer များ အားလုံးအတွက် အသုံးဝင်ပါလိမ့်မယ်။

ဒီစာအုပ်ကိုဖတ်ရှုသူဟာ အခြေခံ Software Development နဲ့ Web Development ဆိုင်ရာနည်းပညာများကို သိရှိထားပြီးဖြစ်ဖို့လိုအပ်ပါတယ်။ အချို့အခြေခံတွေကို အသေးစိတ် ပြန်ရှင်းတော့မှာမဟုတ်ပါဘူး။ ဒါအပြင်၊ Command Line Tool အမြောက်အမြားကို ထည့်သွင်းဖော်ပြမှာဖြစ်ပြီး၊ Command လက်တွေ့နမူနာတွေမှာ Ubuntu Linux ကို နမူနာထားပြီး ဖော်ပြသွားမှာပါ။ ဒါကြောင့် ကျွန်တော်ရဲ့ ပထမစာအုပ်နှစ်အုပ်ဖြစ်တဲ့ **Professional Web Developer** နဲ့ **Ubuntu – သင့်အတွက် Linux** တို့ကို ကြိုတင်ဖတ်ရှုထားမယ်ဆိုရင် စာဖတ်သူအတွက် ပိုမိုထိရောက်အကျိုးရှိစေမှာဖြစ် ပါတယ်။ **Professional Web Developer** နဲ့ **Ubuntu – သင့်အတွက် Linux** တို့ကို အောက်ပါလိပ်စာများမှာ အခမဲ့ Download ရယူနိုင်ပါတယ်။

- Professional Web Developer – pwdbook.com
- Ubuntu – သင့်အတွက် Linux – u4ubook.com

ချစ်ဇနီးနှင်းဝေလွင်ရဲ့ပံ့ပိုးမှုတွေနဲ့

ချစ်သမီးလေးစုရတနာမောင်ပေးတဲ့အားအင်တွေကြောင့်

ဒီစာအုပ်ဖြစ်ပေါ်လာခဲ့တာပါ

မိတ်ဆက်

10x Programmer

Programmer ကောင်းတစ်ယောက်ဟာ
သာမာန် Programmer တစ်ယောက်ထက်
ဆယ်ဆအလုပ်ပိုတွင်ပါတယ်။

မိတ်ဆက်

> ❝ Any fool can write code that a computer can understand. Good programmers write code that humans can understand. ❞ – *Martin Fowler*

ပရိဂရမ်ဆိုတာကတော့ ဘယ် Programmer မဆိုရေးနိုင်ပါတယ်။ Programmer ကောင်း တစ်ယောက်မှသာ ရှင်းလင်း စနစ်ကျပြီး နားလည်လွယ်တဲ့ Code ကို ရေးနိုင်တာပါ။ နားလည်လွယ်တဲ့ Code ဖြစ်မှသာ အမှား နည်းမှာ ဖြစ်ပါတယ်။ ရေရှည်ပြုပြင်ထိမ်းသိမ်းရာမှာ အချိန်ကုန် သက်သာမှာဖြစ်ပါတယ်။ လိုအပ်တဲ့အဆင့်မြှင့် တင်မှုတွေကို အလွယ်တစ်ကူ ဆောင်ရွက်နိုင်စေမှာဖြစ်ပါတယ်။

အမှားတစ်စက်မှ မရှိတဲ့ Bug Free Program ဆိုတာ လက်တွေ့မှာ မရှိပါဘူး။ Bug ဆိုတာမျိုးက ရုတ်တရက် သတိမပြုမိပဲ အကြောင်းတိုက်ဆိုင်တော့မှ ထပြဿနာရှာတက်တဲ့ Error မျိုးပါ။ နားလည်လွယ်အောင် ရေးသား ထားတဲ့ Code မှသာ Bug တွေ ရှိလာတဲ့အခါ အလွယ်တစ်ကူ တွေ့ရှိပြင်ဆင်နိုင်မှာဖြစ်ပါတယ်။ ပြီးတော့ ပရို ဂရမ်တစ်ခုဆိုတာ တစ်ယောက်တည်းရေးလို့ မရပါဘူး။ ရတယ်ပဲထား၊ အဲ့ဒီ Programmer မရှိတော့တဲ့အခါ (ဥပမာ – မနက်ဖြန် ကားတိုက်ခံရလို့သေသွားတဲ့အခါ) ပရိဂရမ်ကို လွှဲပြစ်ရမယ်ဆိုရင်မဟုတ်သေးပါဘူး။ ပေါက်ကရပြောတာ မဟုတ်ပါဘူး၊ ဒီအခြေအနေအတွက် Bus Factor ဆိုတဲ့ အသုံးအနှုံးတစ်ခု အမှန်တစ်ကယ် ရှိပါတယ်။ Software Project တစ်ခုရဲ့ အောင်မြင်မှုအလားအလာကို Bus Factor နဲ့လည်း တိုင်းတာတက်ကြပါ တယ်။ Software Project တစ်ခုအောင်မြင်ဖို့ အများနဲ့ ပူးပေါင်းဆောင်ရွက်ရတက် ပါတယ်။ မဟုတ်ရင်လည်း၊ အခြား Programmer က အလွယ်တစ်ကူ လက်ဆင့်ကမ်း ဆောင်ရွက်နိုင်ဖို့ လိုပါတယ်။ နားလည်လွယ်အောင် ရေးထားတဲ့ Code မှသာ အများနဲ့ ပူးပေါင်း ဆောင်ရွက်နိုင်မှာဖြစ်သလို၊ လိုအပ်ရင်လည်း နောက် Programmer တစ်ယောက်က လက်ဆင့်ကမ်း ဆောင်ရွက်သွားနိုင်မှာဖြစ်ပါတယ်။

10x Programmer ဆိုတဲ့ အသုံးအနှုံးတစ်ခုလည်းရှိပါတယ်။ Programmer ကောင်းတစ်ယောက်ဟာ သာမာန် Programmer ထက် ဆယ်ဆအလုပ်ပိုတွင်တယ်ဆိုတဲ့ အဓိပ္ပါယ်ဖြစ်ပါတယ်။ 10x Programmer တစ်ယောက်ရဲ့ အရည်အချင်းတွေ ထဲမှာ နည်းပညာကျွမ်းကျင်မှုအပြင်၊ အဖွဲ့လိုက်ပူးပေါင်းဆောင်ရွက်နိုင်မှု၊ စူးစမ်းလိုစိတ် ပြင်းပြမှု၊ အသေးစိတ်ဂရု ပြုတက်မှု၊ သင်လွယ်တက်လွယ်မှု စသဖြင့် အများကြီးရှိပါတယ်။ အဲ့ဒီထဲကမှ စနစ် ကျပြီးနားလည်လွယ်တဲ့ Code တွေ ရေးသားနိုင်ခြင်းကလည်း အဓိကကျတဲ့ အရည်အချင်းတစ်ရပ်ပဲ ဖြစ်ပါ တယ်။

ရှုပ်ထွေးပြီးနားလည်ရခက်တဲ့ Code တွေဟာ အမှားများပါတယ်။ ပြင်ဆင်ရခက်ပါတယ်။ Maintain လုပ်ရခက် ပါတယ်။ အခြေခံမကောင်းခဲ့တဲ့အတွက်၊ နောက်ပိုင်းမှာ လုပ်ဆောင်ချက်သစ်တွေနဲ့ အဆင့်မြှင့်ဖို့ လိုအပ်လာတဲ့ အခါ အဆင်မပြေတော့ပဲ ပရိဂရမ်ကို အစအဆုံးပြန်ရေးရတော့မလို ဖြစ်လာတက်ပါတယ်။ ဒါကြောင့် Programmer ကောင်းတစ်ယောက်ဟာ သာမာန် Programmer ထက် ဆယ်ဆပို အလုပ်တွင်တယ်ပြောတာပါ။

ရေးလိုက်တဲ့ Code ရဲ့ အရည်အသွေးဟာ ပရိုဂရမ်ရဲ့ အနာဂါတ်ကိုပါ သက်ရောက်စေမှာမို့ ဖြစ်ပါတယ်။

တစ်ချို့ Programmer တွေ ရှိပါတယ်။ Code တစ်ချို့ ရေးကြည့်လိုက်၊ စမ်းကြည့်လို့ အလုပ်လုပ်တယ်ဆိုရင် နောက် Code တစ်ချို့ ထပ်ရေးကြည့်လိုက်၊ လိုအပ်တဲ့အခါ အင်တာနက်မှာရှာပြီး တွေ့တဲ့ Code တစ်ချို့ ထည့်သုံးလိုက်နဲ့ ပရိုဂရမ်ကတော့ အလုပ်လုပ်သွားပါတယ်။ အသေအချာ သုံးသပ်ပြင်ဆင်ထားတဲ့ အစီအစဉ်ဆိုတာ မရှိပါဘူး။ အနာဂါတ် အလားအလာဆိုတာလည်း ထည့်စဉ်းစားမထားပါဘူး။ နောက်ဆုံးမှာ ပရိုဂရမ်ကတော့ အလုပ်လုပ်နေပါတယ်။ ဒါပေမယ့်၊ ရေးသားထားတဲ့ Code တွေရဲ့ အဓိပ္ပါယ်ကို Programmer ကိုယ်တိုင် သေချာနားလည်ဘူးဆိုတဲ့ အခြေအနေမျိုးတွေ ဖြစ်နေတက်ပါတယ်။ ဒီအခြေအနေကို Code by coincidence လို့ခေါ်ပါတယ်။

တစ်ခါတစ်ရံမှာ Programmer ကောင်း တွေတောင်မှ "နောက်ပြဿနာ နောက်မှရှင်းမယ်" လို့တွေးပြီး ဒီလိုရေးကြတာတွေ ရှိပါတယ်။ Project Deadline သိပ်ကပ်နေလို့လည်း ဖြစ်နိုင်ပါတယ်။ တစ်ခြားအခြေအနေတစ်ခုခုကြောင့်လည်း ဖြစ်နိုင်ပါတယ်။ ဒီလို "နောက်ပြဿနာ နောက်မှရှင်းမယ်" ဆိုတဲ့ အပြုအမူမျိုးကို Technical Debt လို့ခေါ်ပါတယ်။ နည်းပညာကြွေးတွေပါ။ အကြွေးတွေများ လာတဲ့အခါ ဆပ်လို့မနိုင်ဖြစ်လာတက်ပါတယ်။ ရှေ့ကအကြွေးရှင်းရင်းနဲ့ အကြွေးသစ်တွေ တင်လာတက်ပါတယ်။ ဒီလိုနဲ့ ရှုပ်ထွေးပြီးနားလည်ရခက်တဲ့ Code တွေနဲ့ ပြည့်နှက်နေတဲ့ ပရိုဂရမ်တွေ ဖြစ်ပေါ်လာကြပါတယ်။ Bug တစ်ခုရှိလာတဲ့အခါ၊ ဘယ်နားမှာ ဘယ်လိုသွားရှင်းရမလဲဆိုတဲ့ ကိစ္စမျိုးတွေအတွက် အချိန်တွေ ကုန်ရပါတယ်။ တန်ဖိုးလေးတစ်ခုလောက် ပြင် လိုက်ယုံနဲ့ ပြီးသွားရမယ့် ပြဿနာကို ဘယ်နားသွားရှာရမှန်း မသိတာနဲ့ နာရီပေါင်းများစွာ၊ ရက်ပေါင်းများစွာ အချိန်တွေ ကုန်သွားတာမျိုး ဖြစ်တက်ပါတယ်။

ဒီပရိုဂရမ်ကို လိုအပ်ချက်အရ အဆင့်မြှင့်ဖို့အတွက် ဆက်လက်ဆောင်ရွက်ရသူမှာ လည်း အခက်တွေ့ရမှာဖြစ်ပါတယ်။ လက်ရှိရေးထားတဲ့ Code က ရှုပ်ထွေးလွန်းလို့ သူလည်းနားမလည်ပါဘူး။ ပိုကောင်း အောင် ပြင်လိုက်မှ လုံးဝအလုပ်မလုပ်တော့တာမျိုး ဖြစ်သွားတက်ပါတယ်။ ဘာကြောင့် ဒီလိုဖြစ်မှန်းလည်း မသိပါဘူး။ ဒါနဲ့ပဲ နောက်ထပ်ရှုပ်ထွေးတဲ့ Code တွေ ထပ်ထည့်ပြီး အလုပ်မလုပ်လုပ်အောင် ဇွတ်လုပ်ရပါတော့တယ်။ ဒီအခြေအနေ ကိုတော့ Accidents of implementation လို့ခေါ်ပါတယ်။

ကျွန်တော်တို့တွေ ဝါသနာပါလွန်းလို့ လုပ်ခဲ့တဲ့ Programmer အလုပ်ဟာလည်း ဒီလိုအခြေအနေမှာ စိတ်ပျက်စရာတွေ ပြည့်နှက်နေတဲ့အလုပ် တစ်ခုဖြစ်လာပါတယ်။ တစ်ချိန်က ခေါင်းမော့ရင်ကော့ပြီး တက်ကြွရွှင်လန်းခဲ့တဲ့ Programmer တွေလည်း အလုပ်သွားရမှာကိုတောင် ကြောက်ရွံ့လာပါတယ်။ Code ဖိုင်ကိုဖွင့်ကြည့်ရမှာကို စိုးရွံ့လာပါတယ်။ တစ်ယောက် ယောက်က ပရိုဂရမ်မှာရှိနေတဲ့ Bug တစ်ခုအကြောင်း ပြောလာမှာကို စိုးထိတ်နေရပါတယ်။ ကိုယ့်ကိုယ်ကို ယုံကြည်မှုတွေ ပျောက်ဆုံးလာပါတယ်။ "မန်နေဂျာစိတ်ပြောင်းပြီး ဒီ Project ကြီးကို ရပ်လိုက်ပါစေ" လို့ အကြိမ်ကြိမ် ဆုတောင်းမိလာ ပါတယ်။ ပရိုဂရမ်ကဘယ်လိုမှအလုပ်မဖြစ်လို့ ရပ်လိုက်ရတဲ့အခါ လူမြင်ကောင်းအောင် စိတ်မကောင်းချင်ဟန် ဆောင်ထားပေမယ့် တိတ်တိတ်လေး ဝမ်းသာနေမိတက်ပါတယ်။

စနစ်တစ်ကျ သပ်ရပ်အောင်ရေးထားတဲ့ Code တွေမှာတော့ ဒီ Code ကို ထိုင်ကြည့်နေရတာကိုက အနုပညာမြောက် ပန်းချီကားတစ်ချပ်ကို တစ်စိမ့်စိမ့်ထိုင်ကြည့်နေရသလို အရသာရှိပါတယ်။ စိတ်ခါတ်တွေ တက်ကြွပါ

တယ်။ အရာရာကလည်း ရိုးရှင်းလွယ်ကူနေတက်ပါတယ်။ Bug တွေရှိလာရင် စိတ်မညစ်ပါဘူး၊ ဝမ်းတောင်သာပါသေးတယ်။ မိမိကိုယ်ကိုနဲ့ Project အပေါ် ယုံကြည်မှုအပြည့်အဝရှိတဲ့အတွက် Bug Fix လုပ်တယ်ဆိုတဲ့အလုပ်ကိုက စိတ်ဝင်စားစရာအလုပ်တစ်ခု ဖြစ်လာပါတယ်။ ဒီ Code ကို ဆက်လက်တာဝန်ယူရမယ့်သူရဲ့ အသိအမှတ်ပြုမှုကိုခံရပါတယ်။ အခြေခံကောင်းတဲ့ Code ကို ဆက်ခံရတယ်ဆိုပြီး အကြိမ်ကြိမ် ချီးမွမ်းမှုကို ခံရပါလိမ့်မယ်။

စနစ်ကျသပ်ရပ်တဲ့ Code တွေရေးနိုင်ဖို့ Language ကျွမ်းကျင်မှုသက်သက်နဲ့ မလုံလောက်ပါဘူး။ Process Management, Source Code Management, Design Principles နဲ့ Design Architecture တို့လို Software Engineering ဆိုင်ရာ ဆက်စပ်နည်းပညာဗဟုသုတတွေ ရှိထားဖို့လိုအပ်ပါတယ်။ ဒီဗဟုသုတွေ ပြည့်စုံတဲ့ 10x Programmer တွေမှသာ စနစ်ကျသပ်ရပ်တဲ့ Code တွေရေးနိုင်မှာဖြစ်ပါတယ်။ အဖွဲ့လိုက် ပူးပေါင်းဆောင်ရွက်တက်မှာ ဖြစ်ပါတယ်။ အရည်အသွေးကောင်းပြီး Scalable ဖြစ်တဲ့ပရိဂရမ်တွေရေးနိုင်မှာဖြစ်ပါတယ်။ 10x Programmer ကို Rockstar Developer လို့လည်းခေါ်ပါတယ်။ ဒီစာအုပ်က စာဖတ်သူကို 10x Programmer သို့မဟုတ် Rockstar Developer တစ်ယောက်ဖြစ်လာအောင် အထောက်အကူပြုလိမ့်မယ်လို့ မျှော်လင့်ပါတယ်။

အဓိဂ်း(၁)

Key Concepts

လက်တွေ့အသုံးပြုမယ့်သူက သူလိုချင်တာ ဒီလိုမဟုတ်သေးဘူး
ဆိုတာကို အတော်လေးခရီးရောက်ပြီးမှ သိရှိသတိပြုမိလာခြင်းဟာ
Software Project တွေ မအောင်မြင်ရခြင်းရဲ့ အဓိက
အကြောင်းရင်းတစ်ခုဖြစ်ပါတယ်။

အခန်း(၁) – Development Processes Management

"ဗျာ... နောက်ခံကို အပြာရောင်ပြောင်းချင်တယ်ဟုတ်လား။ အခုမှပြောရသလားဗျာ။ ကျွန်တော်က အနီရောင်နဲ့ လုပ်ထားတာ၊ ပြီးတောင်ပြီးတော့မယ်။"

"ဟုတ်ပါတယ်။ ကျွန်တော်တို့ကလည်း၊ အနီရောင်ဆိုရင် ကောင်းမယ်ထင်လို့ ရွေးခဲ့တာပါ။ ဒါပေမယ့် ခင်ဗျားအခုပြတဲ့ နောက်ဆုံးရလဒ်ကို ကြည့်လိုက်မှ၊ ကျွန်တော်တို့လိုချင်တဲ့ပုံစံ မဟုတ်မှန်းကိုသိရလို့ပါ။"

"အခုက အားလုံးပြီးသလောက်ဖြစ်နေပြီ။ ပြောင်းဖို့ အဆင်မပြေနိုင်တော့ဘူး။"

"စိတ်တော့မရှိပါနဲ့။ နားမလည်လို့ မေးပါရစေ။ နောက်ခံအရောင်လေးပဲ ပြောင်းချင်တာ၊ ဘာလို့ အဆင်မပြေတာလဲ။ ကျန်တာတွေကို ပြင်ဖို့မလိုဘူးလေ။"

"ကျွန်တော်က စာလုံးအရောင်၊ အနေအထားနဲ့ နေရာအထားအသို အားလုံးကို နောက်ခံအနီဆိုတဲ့ အချက်ပေါ်မှာအခြေခံပြီး လိုက်ဖက်အောင် အမျိုးမျိုးစမ်းသပ်၊ သေးချာစီစစ်ပြီးမှ ရွေးခဲ့ရတာပါ။ အခု နောက်ခံအရောင်ပြောင်းချင်တယ်ဆိုလို အရောင်လေးတင် ကွက်ပြောင်းလို့မရပါဘူး။ စာလုံးအရောင်၊ အနေအထားနဲ့ နေရာအထားအသို အားလုံးကို ပြန်လုပ်ရတော့မှာပါ။ ပြောင်းချင်တာက နောက်ခံဆိုပေမယ့် အစအဆုံးပြန်လုပ်ရတဲ့သဘောမို့လို့ပါ။"

"ခင်ဗျားလုပ်ငန်းအသေးစိတ်တော့ ကျွန်တော်တို့လည်း နားမလည်ပါဘူး။ ဒါပေမယ့် ကျွန်တော်တို့ အနေနဲ့ကတော့ နောက်ခံအပြာမှမရရင် အဆင်မပြေပါဘူး။"

"အားနာနာနဲ့ပဲ ပြောပါရစေ။ ကျွန်တော်တို့ စသေဘာတူခဲ့တုန်းက အနီနဲ့သဘောတူခဲ့တာ။ ဒါကြောင့် အနီနဲ့ပဲ လုပ်ပေးနိုင်ပါမယ်။ အပြာနဲ့ထပ်လုပ်ပေးစေချင်ရင်တော့ ဝန်ဆောင်ခကျသင့်ငွေ ထပ်ပေးရပါ ယ်။"

"ဘာဗျ။ ကျသင့်ငွေထပ်ပေးရမယ် ဟုတ်လား။ ဒါတော့ မဟုတ်သေးဘူးထင်တယ်။ ခင်ဗျားကို ကျွန်တော်တို့က တန်ရာတန်ကြေးနဲ့သဘောတူထားပြီးသားပဲ။ အဆင်ပြေအောင်လုပ်ပေးရမှာပေါ့။"

"ဖြစ်နိုင်တဲ့ကိစ္စဆိုရင် လုပ်ပေးချင်ပါတယ်။ အခုဟာက အစအဆုံးနီးပါ ပြန်လုပ်ရမှာဆိုတော့ ဘယ်လိုမှ မဖြစ်နိုင်လို့ပါ။ တန်ရာတန်ကြေးနဲ့ သဘောတူထားပေမယ့် နောက်ခံအနီနဲ့ လုပ်မယ်ဆိုတာကို ခင်ဗျား တို့ ဘက်ကလည်း သဘောတူခဲ့တာပဲမဟုတ်လား။"

> **"အစကသဘောတူခဲ့တာမှန်ပါတယ်။ ကျွန်တော်တို့ကလည်းရလဒ်ကို မမှန်းဆတက်တော့ ကောင်းမယ် ထင်လို့ သဘောတူခဲ့တာပါ။ အခုလက်တွေ့မြင်ရတော့မှ အဆင်မပြေမှန်းကို သိရတာလေဗျာ။"**

ဒါဟာ Software Project အများစုမှာ ကြုံတွေ့ရတက်တဲ့ အခြေအနေ တစ်ရပ်ပဲဖြစ်ပါတယ်။ Software ကို အမှန်တစ်ကယ် အသုံးပြုကြမယ့် User သို့မဟုတ် Customer များဟာ နည်းပညာနားမလည်ကြပါဘူး။ သူတို့လုပ်ငန်းရဲ့ သဘောသဘာဝနဲ့ ရှိနေတဲ့ပြဿနာကိုနားလည်ကြပေမယ့်၊ အဲ့ဒီပြဿနာကို နည်းပညာနဲ့ဖြေရှင်းရမယ့် Software Solution ဟာ ဘယ်လိုပုံစံဖြစ်သင့်ပြီး ဘယ်လိုအလုပ်လုပ်သင့်တယ် ဆိုတာကိုတော့ မမှန်းဆတက်ကြပါဘူး။ နောက်ဆုံး Software ကို မြင်ရတော့မှသာ ဒါဟုတ်တယ်၊ မဟုတ်ဘူး အမှန်တစ်ကယ်သိရှိကြရတာပါ။

Software Developer အနေနဲ့ ကလည်း မျက်မြင်တွေ့နေရတဲ့ ပြဿနာကိုသာသိပြီး၊ နောက်ကွယ်က လုပ်ငန်းသဘော သဘာဝ အသေးစိတ်နဲ့ အခြားဆက်စပ် အချက်အလက်တွေကို အတွင်းကျကျ မသိရှိတက်ကြပါဘူး။ ဘယ်လောက်ပဲ အချက်အလက်တွေ ရနိုင်သမျှရအောင်စုစည်းပြီး လေ့လာပေမယ့် မှန်းဆလို့မရတဲ့အခြေအနေတွေက အမြဲရှိနေတက်ပါတယ်။ ရထားတဲ့အချက် အလက်ကို အခြေခံပြီး Software ကို တည်ဆောက်ရတဲ့အခါ၊ အတော်လေးခရီးရောက်တော့မှ ဒါမဟုတ်သေးဘူး လွဲနေပြီလို့ ဆိုလာတဲ့အခါ ပြဿနာတက်တော့တာပါပဲ။



ပုံ(၁.၁) – Common Project Management Problem

Source – projectcartoon.com

အတွေ့အကြုံ တွေများလာတာနဲ့အမျှ ဘယ်လိုလိုအပ်ချက်မျိုးဆိုရင် ဘယ်လို Solution မျိုးဖြစ်သင့်တယ်ဆိုတဲ့ Sense နဲ့ အထွေထွေ ဗဟုသုတတွေ ကြွယ်ဝလာတဲ့အခါ၊ ပိုမိုမှန်ကန်တဲ့ ဆုံးဖြတ်ချက်တွေ ချနိုင်လာမှာဖြစ်တဲ့အတွက် ဒီပြဿနာမျိုးတွေ လျော့ကျလာမှာ ဖြစ်ပါတယ်။ ဒါပေမယ့် လုံးဝမရှိတော့အောင် ရှင်းပြစ်ဖို့တော့ မလွယ်ပါဘူး။

လုပ်လို့ရနိုင်တာကတော့၊ အမှန်တစ်ကယ်လိုအပ်ချက်နဲ့ လွဲနေတာရှိခဲ့ရင် စောစောသိရအောင် လုပ်ခြင်းပဲဖြစ်ပါတယ်။ စောစောသိပြီး စောစောပြင်ခွင့်ရတဲ့အခါ အချိန်လွန်မှသိရလို့ ပြဿနာဖြစ်ရပါတယ်ဆိုတာမျိုးကို ရှောင်လွဲနိုင်မှာဖြစ်ပါတယ်။ ဒီလိုသိရဖို့အတွက် Software ကို အမှန်တစ်ကယ်အသုံးပြုမယ့် Customer သို့မဟုတ် User များထံကနေ Feedback တွေကို စောစောလက်ခံရယူခြင်းပဲ ဖြစ်ပါတယ်။

လုပ်ငန်းတွေအတွက် ရည်ရွယ်ဖန်တီးတဲ့ Business Solution Software တွေမှာတင် ဒီပြဿနာက ရှိတာမဟုတ်ပါဘူး။ Web App, Mobile App တို့လို အများသုံး၊ တစ်ကိုယ်ရည်သုံး Software တွေမှာလည်း ဒီပြဿနာရှိတာပါပဲ။ ကိုယ့်မှာ စိတ်ကူးအိုင်ဒီယာရှိပေမယ့် အဲ့ဒီစိတ်ကူးအိုင်ဒီယာဟာ အများရဲ့လိုအပ်ချက်နဲ့ အမှန်တစ်ကယ်ကိုက်မကိုက်၊ လက်တွေ့အလုပ် ဖြစ်မဖြစ်ဆိုတဲ့အခြေအနေ Feedback ကို စောစောသိရလေ ကောင်းလေပဲဖြစ်ပါတယ်။

Rockstar Developer တစ်ဦးဟာ အခြေခံကောင်းပြီး ပြင်လွယ်ပြောင်းလွယ်တဲ့ Software တွေကို ဖန်တီးတက်ယုံမက User Feedback ကို စောစောရယူနိုင်တဲ့ Development Process တွေကိုလည်း သိရှိအသုံးပြုနိုင်ဖို့ လိုအပ်ပါတယ်။

1.1 – Stages of Software Development

Software Product တစ်ခုဖြစ်မြောက်လာဖို့ Code တွေချည်းထိုင်ရေးနေယုံနဲ့ မရပါဘူး။ Solution Idea နဲ့ Design Concept တွေ စဉ်းစားဖန်တီးရပါတယ်။ လိုအပ်တဲ့အချက်အလက်တွေ စုစည်းရပါတယ်။ အသုံးပြုမယ့်နည်းပညာတွေ ရွေးချယ်ရပါတယ်။ Unit Testing, Integration Testing စသဖြင့် အဆင့်ဆင့် စမ်းသပ်စီစစ်မှုတွေ ပြုလုပ်ရပါတယ်။ အသုံးပြုသူတွေ လိုအပ်တဲ့ Support ပေးဖို့ အစီအစဉ်တွေ သတ်မှတ်ရပါတယ်။

Software Product တစ်ခု တည်ဆောက်ရာမှာ ပါဝင်လေ့ရှိတဲ့အဆင့်တွေကို ဖော်ပြပေးလိုက်ပါတယ်။

1. Business Model Creation
2. Requirements Analysis
3. Data Collection
4. System Design
5. UI Prototype
6. Implementation / Coding
7. Testing
8. Continuous Integration Plan
9. Deployment / Integration
10. Technical Support

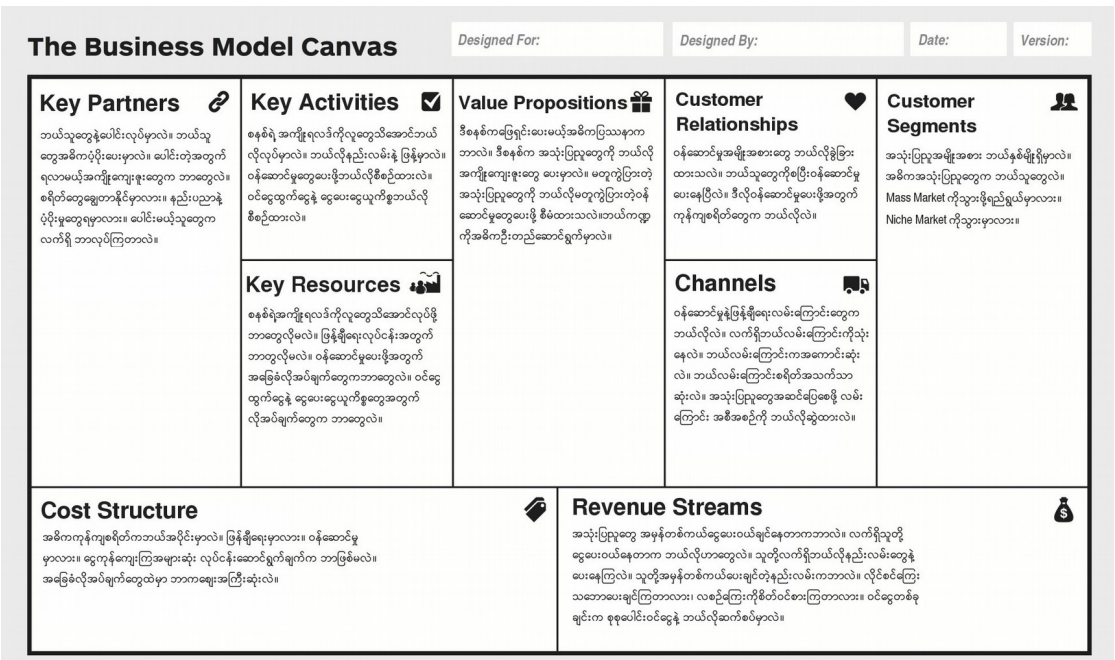
အရင်ကတော့ Software Development Process ကို Plan, Design, Code, Test ဆိုပြီး (၄) ဆင့်လောက်ပဲ ခွဲကြလေ့ရှိပါတယ်။ အခုနောက်ပိုင်းမှာတော့ Development Practice တွေ အသစ်အသစ်တိုးတက်ပါပေါက်လာတာနဲ့အတူ ပါဝင်တဲ့အဆင့်တွေလည်း ဖော်ပြခဲ့သလို ပြောင်းလဲလာခဲ့ပြီပဲဖြစ်ပါတယ်။ ဥပမာ - Test လုပ်တယ်ဆိုတဲ့

ကိစ္စကို အရင်က Software တည်ဆောက်ခြင်းလုပ်ငန်း ပြီးမှလုပ်ကြလေ့ရှိပေမယ့် အခုနောက်ပိုင်းမှာတော့ Test Driven Development (TDD) လို နည်းစနစ်မျိုးတွေနဲ့ Coding လုပ်ငန်းဆောင်ရွက်နေစဉ်မှာပဲ တစ်ခါတည်း Unit Level, Function Level မှာ Test လုပ်နိုင်လာပြီ ဖြစ်ပါတယ်။

Software Product တစ်ခုတည်ဆောက်ရာမှာ **Business Model** ကိုလည်း ထည့်သွင်းစဉ်းစားရလေ့ရှိပါတယ်။ Business Model ဆိုတဲ့အထဲမှာ Cost Structure နဲ့ Revenue Stream လို ငွေကြေးဆိုင်ရာကိစ္စတွေနဲ့ Distribution Channel တို့လို Software Development နဲ့ တိုက်ရိုက်မသက်ဆိုင်တဲ့အပိုင်းတွေ ပါဝင်ပါတယ်။ ဒါပေမယ့် ဒီ Software ကို ဘယ်နေရာမှာ အသုံးချမှာလဲ။ ဒီ Software ကပေးတဲ့အဓိက Value ကဘာလဲ။ ဒီ Software က ဖြေရှင်းပေးမယ့် ပြဿနာကဘာလဲ စတဲ့အချက်အလက်တွေဟာ Business Model မှာ ပါဝင်ရလေ့ရှိပါတယ်။

ဗဟုသုတအနေနဲ့ Business Model တစ်ခုရေးဆွဲရာမှာ အခြေခံအားဖြင့် ပါဝင်ရမယ့်အချက်တွေကို စုစည်းဖော်ပြထားတဲ့ Business Model Canvas ကို **ပုံ (၁.၂)** မှာ ထည့်သွင်းဖော်ပြပေးထားပါတယ်။

Software Developer တစ်ယောက်အနေနဲ့ Business Model တစ်ခု လုံးကို ရေးဆွဲပေးဖို့ လိုချင်မှလိုမှာပါ။ ဒါပေမယ့် တည်ဆောက်မယ့် Software ကိုတော့ Business Model နဲ့ ချိန်ညှိ ထားဖို့လိုအပ်မှာဖြစ်ပါတယ်။ ဥပမာ - Business Model က လစဉ်ကြေး (Monthly Subscription Fee) ကို အခြေခံတဲ့ Payment Model ကို သုံးထားတယ်ဆိုရင် Software မှာလဲ အဲ့ဒီကဏ္ဍအတွက် ထည့်သွင်းရေးသားပေးရတော့မှာ ဖြစ်ပါတယ်။ ဒါကြောင့် Business Model ဟာလည်း Software Development Process ရဲ့ အစိတ်အပိုင်းတစ်ရပ်ပဲ ဖြစ်ပါတယ်။ Business Model မှာ အပြောင်း အလဲရှိလာခဲ့ရင် Software ကိုလည်း သက်ရောက်မှာပဲဖြစ်ပါတယ်။



ပုံ(၁.၂) – Business Model Canvas

Business Model တစ်ခုရှိလာတဲ့အခါ ပိုပြီးတိကျတဲ့ **Requirements Specification** တစ်ခု ဆက်လက် တည်ဆောက်ဖို့ လိုအပ်ပါတယ်။ Software မှာ Feature အနေနဲ့ ပါဝင်ရမယ့် လုပ်ဆောင်ချက်တွေကဘာတွေ လဲ။ Performance, Security စတဲ့ Non-Functional ပိုင်းအနေနဲ့ပါဝင်ရမယ့် လုပ်ဆောင်ချက်တွေကဘာတွေ လည်း။ စသဖြင့် သတ်မှတ်ရပါတယ်။ Specification တစ်ခုမှာ ပါဝင်သင့်တဲ့အချက်တွေအကြောင်းကို **အခန်း (၇) – Team Collaboration** မှာ ဆက်လက်ဖော်ပြပါမယ်။

ဆက်လက်ပြီး လိုအပ်တဲ့အချက်အလက်တွေကို ကဏ္ဍအလိုက်ရယူစုစည်းရတဲ့ **Data Collection** လုပ်ငန်းကို လည်း ဆောင်ရွက်ရပါတယ်။ ရရှိလာတဲ့ အချက်အလက်များကိုအခြေခံပြီး၊ Data တွေရဲ့ ဖွဲ့စည်းပုံ၊ အပြန်အလှန် ဆက်စပ်ပုံ တို့ကို ခွဲခြမ်းစိတ်ဖြာပြီး သင့်တော်တဲ့ Data Structure နဲ့ Model တွေကို တည်ဆောက် ရပါတယ်။

ရရှိထားတဲ့ Specification နဲ့ Data Model တို့ကို အခြေခံပြီး အသေးစိတ် **System Design** ကိုလည်း ရေးဆွဲ ထားဖို့ လိုပါတယ်။ ဘယ်လို Code Structure မျိုးကိုသုံးမှာလဲ MVC လား၊ Multi-Tire Design လား။ Architecture အနေနဲ့ရော Service-Oriented Architecture သုံးမှာလား။ သုံးမယ်ဆိုရင် Communication Protocol အနေနဲ့ ဘာကိုသုံးမှာလဲ။ REST သုံးမှာလား၊ တစ်ခြား Protocol တစ်ခုကိုသုံးမှာလား။ Authentication နဲ့ Authorization တို့အတွက် ဘယ်လို Access Control Strategy ကို သုံးမှာလဲ။ Performance အတွက် Memcached လိုနည်း ပညာမျိုး သုံးမှာလား။ Scalability အတွက် Vertical Scaling ကိုရွေးမလား၊ Horizontal Scaling ကိုရွေးမလား၊ စသဖြင့် ဆုံးဖြတ်ပြင် ဆင်ထားရပါတယ်။

ပြီးတဲ့အခါ **User Interface Prototype** တွေလည်းတည်ဆောက်ရပါတယ်။ Prototype ကို Design Tool သက်သက် သုံးပြီးတည်ဆောက်တာမျိုးဖြစ်နိုင်သလို၊ Wire-framing Tool တွေကိုသုံးပြီး တည်ဆောက်တာ လည်း ဖြစ်နိုင်ပါတယ်။ ဒါမှမဟုတ် လက်တွေ့ Coding လုပ်ပြီး နမူနာရေးသားတည်ဆောက်လိုက် တာမျိုး လည်း ဖြစ်နိုင်ပါတယ်။ တစ်ခါတစ်ရံမှာ Prototype ကို ကြည့်ပြီး၊ ပိုကောင်းတဲ့အိုင်ဒီယာရလာလို့၊ ဒါမှမဟုတ်၊ လက်တွေ့မဖြစ်နိုင်မှန်းသိရလို့ System Design ကို ပြောင်းလိုက်ရတာမျိုးလည်း ရှိတက်ပါတယ်။

အခြေခံလိုအပ်ချက်တွေ ပြည့်စုံလို့ လက်တွေ့ **Coding** အဆင့်ကိုသွားတဲ့အခါမှာတော့ Coding Standards, Coding Convention, Coding Design Principle, Best Practices စတဲ့ Code Quality နဲ့ပက်သက်တဲ့အပိုင်းတွေ ကို အလေးထားရပါမယ်။ အဓိကကျတဲ့ Coding Structure Pattern တစ်ခုဖြစ်တဲ့ MVC အကြောင်းကို **Professional Web Developer** ရဲ့ **အခန်း (၁၃)** မှာ ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ နောက်ထပ်အရေးပါတဲ့ Practice တစ်ခုဖြစ်တဲ့ SOLID Principle အကြောင်းကိုတော့ **အခန်း (၃) – SOLID Design Principles** မှာ ဆက်လက် ဖော်ပြပါတယ်။

Software ဟာ မျှော်မှန်းထားတဲ့အနေအထားအတိုင်း မှန်မှန်ကန်ကန် အလုပ်လုပ်ကြောင်း သေချာစေဖို့ အဆင့် လိုက် **Test** Method အမျိုးမျိုးနဲ့ စမ်းသပ်စီစစ်ဖို့လိုပါတယ်။ Unit Testing လို့ခေါ်တဲ့ Unit တစ်ခုချင်း၊ Function တစ်ခုချင်း စီကို စမ်းသပ်တဲ့နည်းစနစ်ရှိပါတယ်။ Integration Testing လို့ခေါ်တဲ့ Unit တွေ ပေါင်းစပ်တွဲဖက် ထားတဲ့ ရလဒ်ကို စစ်ဆေးရတဲ့အဆင့်ရှိပါတယ်။ Specification နဲ့ ကိုက်ညီမှုရှိမရှိ စီစစ်ရတဲ့ System Testing နဲ့ Acceptance Testing လုပ်ငန်းတွေလည်းဟာ Software Development Process ရဲ့ အစိတ်အပိုင်းတစ်ရပ်ပဲဖြစ်

ပါတယ်။ Unit Testing အကြောင်းကို **အခန်း (၂) – Test Driven Development** မှာ ဆက်လက်ဖော်ပြပါမယ်။

Software Product တစ်ခုဆိုတာ ရေးပြီး ပြီးသွားပြီ၊ ပြန်လှည့်ကြည့်စရာ မလိုတော့ဘူးဆိုတာမျိုး မရှိပါဘူး။ နည်းပညာ အဆင့်မြင့်လာတာနဲ့အမျှ Software ကိုလည်း အဆင့်မြှင့်တင်မှုတွေ လိုက်လံဆောင်ရွက်ပေးဖို့ လိုအပ်တက်ပါတယ်။ Business Model နဲ့ Requirements ပြောင်းလဲသွားတဲ့အခါမှာလည်း လိုအပ်သလိုပြင်ဆင်မှုတွေကို ဆောင်ရွက်ပေးရမှာ ဖြစ်ပါတယ်။ ဒါကြောင့် စဉ်ဆက်မပြတ်အဆင့်မြှင့်တင်သွားနိုင်ဖို့အတွက် အစီအစဉ်တွေ ချမှတ်ထားရမှာဖြစ်ပါတယ်။ ဒီနေရာမှာ Source Code Management System တွေ၊ Build Automation နည်းစနစ်တွေနဲ့ Virtualization နည်းပညာတွေရဲ့ အကူအညီကိုယူရမှာဖြစ်ပါတယ်။ Source Code Management System အကြောင်းကို **အခန်း (၅)**၊ Build Automation အကြောင်းကို **အခန်း (၇)** နဲ့ Virtualization နည်းပညာတစ်ခုဖြစ်တဲ့ Docker အကြောင်းကို **အခန်း (၁၇)** တို့မှာ အသီးသီး ဆက်လက်ဖော်ပြပေးပါမယ်။

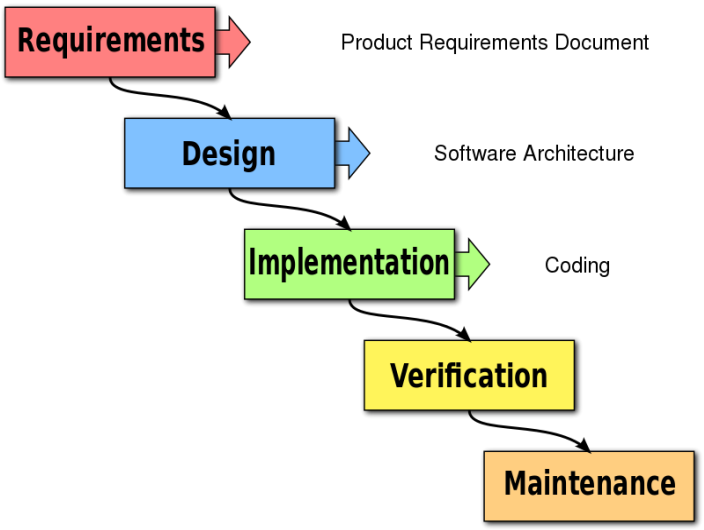
Software Solution ဟာ အသုံးပြုလို့ရတဲ့အဆင့်ကိုရောက်ပြီဆိုရင်တော့ စတင်အသုံးပြုနိုင်အောင် တပ်ဆင်ခြင်း၊ ဖြန့်ဝေခြင်း လုပ်ငန်းတွေ ဆောင်ရွက်ရမှာပါ။ Server အတွက် လိုအပ်တဲ့ **Production Environment** Setup ပြုလုပ် ခြင်းလုပ်ငန်းတွေ ဆောင်ရွက်ရပါတယ်။ Load Balancer တွေ လိုအပ်ရင် Setup လုပ်ရပါမယ်။ Database Replication တွေ လိုအပ်ရင် စီစဉ်ရပါတယ်။ အသုံးများတဲ့ Server Architecture တွေအကြောင်းကို **အခန်း (၂၀) – Common Server Architectures** မှာ ဆက်လက်ဖော်ပြပါမယ်။

Technical Support ဟာလည်း အရေးပါတဲ့အဆင့်ဖြစ်ပါတယ်။ Solution ကို အသုံးပြုမယ့်သူတွေအတွက် လိုအပ်ရင် Training တွေစီမံပေးရပါမယ်။ သို့မဟုတ်ရင်လည်း Manual တွေ Documentation တွေ စီစဉ်ပေးထားရပါတယ်။ ပြဿနာ တစ်စုံတစ်ရာကြုံလာတဲ့အခါ အသုံးပြုသူက Report လုပ်လို့ ရနိုင်မယ့် နည်းစနစ်တွေ၊ လမ်းညွှန်တွေသတ်မှတ် ပေးရပါမယ်။ တစ်ချို့ မေးခွန်းတွေကို အရေးကြီးမှုအခြေအနေ အဆင့်အလိုက် ဖြေရှင်းပေးမယ့် Policy တွေ သတ်မှတ်ရပါမယ်။ ဒီနေရာမှာ တစ်ဘက်တစ်လမ်းက အထောက်အကူပေးနိုင်တဲ့ Issue Tracking System အကြောင်းကိုတော့ **အခန်း (၆)** မှာ ဆက်လက်ဖော်ပြပေးမှာ ဖြစ်ပါတယ်။

Software Team တွေမှာ System Analysis, Software Engineer, Programmer, Designer, Front-end Developer, Back-end Developer, System Administrator, Tester စသဖြင့် Role အမျိုးမျိုးနဲ့ တာဝန်ကိုယ်စီပေးထားကြသူတွေရှိနိုင်ပါတယ်။ ဖော်ပြခဲ့တဲ့ လုပ်ငန်းအဆင့်တွေထဲက ဘယ်အဆင့်ကို ဘယ်သူတာဝန်ယူမလဲဆိုတာကတော့ Team တစ်ခုနဲ့တစ်ခု လက်တွေ့မှာ တူမှာမဟုတ်ပါဘူး။ ဒီအဆင့်တွေကို ဘယ်လိုအစီအစဉ်၊ ဘယ်လို Process Management Model တွေနဲ့ ဆောင်ရွက်မလဲဆိုတဲ့ကိစ္စမှာတော့ အများစုထားပြီးသုံးကြတဲ့ နည်းစနစ်တစ်ချို့ ရှိပါတယ်။ အဲ့ဒီထဲက အဓိကအကျဆုံးလို့ဆိုရမယ့် နည်းစနစ်နှစ်ခုအကြောင်းကို ဖော်ပြပေးလိုက်ပါတယ်။

1.2 – Waterfall Model

Waterfall Model လို့ခေါ်တဲ့ Development Process Management စနစ်တစ်မျိုး ရှိပါတယ်။ Software Development လုပ်ငန်းစဉ်မှာ ပါဝင်တဲ့ လုပ်ငန်းတွေကို အဆင့်လိုက် တစ်ခုပြီးတစ်ခု လုပ်သွားတဲ့နည်းစနစ်ဖြစ်ပါတယ်။ ဥပမာ - ပထမဆုံးအနေနဲ့ လိုအပ်တဲ့ Requirements Specification တစ်ခုရရှိအောင်ကို အသေးစိတ် ရေးဆွဲရပါတယ်။ လိုချင်တဲ့ Requirements Specification အတိအကျရပြီဆိုတော့မှ System Design ကို စတင် ဆောင်ရွက်ရပါတယ်။ အသေးစိတ် Design ရပြီဆိုတော့မှ Coding လုပ်တဲ့ အဆင့်ကို သွားပါတယ်။ Coding လုပ်ပြီးသွားလို့ Product ထွက် လာတဲ့အချိန်မှ စမ်းသပ်တဲ့ Testing လုပ်ငန်းကို ဆက်လက်ဆောင်ရွက်သွားတဲ့ နည်းစနစ် ဖြစ်ပါတယ်။



ပုံ (၁.၃) - Waterfall Model

Source – Wikipedia

ဒီလိုအဆင့်လိုက် ဆောင်ရွက်သွားတဲ့အတွက် ရလာတဲ့အားသာချက်ကတော့၊ အဆင့်တိုင်းမှာ ရှင်းလင်းတိကျတဲ့ ဦးတည်ချက်ကို ရရှိမှာဖြစ်ပါတယ်။ Junior Developer တွေပါဝင်မှုများတဲ့ Team တွေနဲ့ ကိုက်ညီနိုင်ဖြစ်ပါတယ်။ ဘာကြောင့်လည်းဆိုတော့ ဦးဆောင်သူ Lead Developer နဲ့ Project Manager တွေက လိုအပ်တဲ့ Requirements Specification နဲ့ System Design ကြိုတင်ဆွဲထားပေးမှာမို့ Junior Developer တွေက သိပ်ပြီး ခေါင်းစားနေစရာမလိုပဲ ပေးထားတဲ့ Spec နဲ့ Design ကိုကြည့်ပြီး Coding လုပ်သွားယုံပဲမို့ အခက်အခဲမရှိလှပဲ ဆောင်ရွက်နိုင်မှာဖြစ်ပါတယ်။

ဒါပေမယ့်လက်တွေ့မှာ တိကျတဲ့ Specification ဆိုတာ ရရှိဖို့ခက်ပါတယ်။ ရှေ့ပိုင်းမှာလည်း ပြောခဲ့ပြီးဖြစ်ပါတယ်။ လက်တွေ့အသုံးပြုမယ့် User က Specification ကိုကြည့်ပြီး သူလိုချင်တဲ့ရလဒ်ဟုတ်မဟုတ် ဆုံးဖြတ်နိုင်မှာမဟုတ်ပါဘူး။ Software Product ကို မြင်ရတဲ့အချိန်ရောက်မှ တစ်ကယ်သူလိုချင်တာ မဟုတ်တဲ့အခါ အဆင့်လိုက်ဆောင်ရွက်ခြင်းဟာ ပြဿနာအကြီးအကျယ်တက်ရတော့တာပါ။ Requirements Specification မှာက လည်းက လွဲတဲ့အခါ၊ System Design လည်းလိုက်ပြီး လွဲတော့တာပါပဲ။ ဒါကိုကြည့်ပြီး Coding လုပ်လိုက်လို့

Product ထွက်လာတော့မှ လွဲနေပြီဆိုတော့ Specification နဲ့ System Design အတွက် ရင်းနှီးခဲ့ရတဲ့အချိန်တွေ အလကားဖြစ်ကုန်တော့မှာ ဖြစ်ပါတယ်။

ဒီလိုအားနည်းချက်ရှိနေပေမယ့်လည်း Waterfall Model ဟာ သူ့အနေရာနဲ့သူ အသုံးဝင်ဆဲဖြစ်ပါတယ်။ အထူးသဖြင့် Requirement Specification ကို အတိအကျသိနိုင်တဲ့အခြေအနေတွေမှာဆိုရင် အသုံးဝင်မှာဖြစ်ပါတယ်။ တိကျတဲ့ Specification ဆိုတာရဖို့ခက်တယ်လို့ ပြောခဲ့ပါတယ်။ ဒါက Product အသစ်တွေ၊ Innovative ဖြစ်တဲ့ ထူးခြားတဲ့ Product မှာပြောတာပါ။ ဥပမာ - Product တစ်ခုကို တစ်ကြိမ်ဆောင်ရွက်ခဲ့ဘူးတယ် ဆိုပါစို့။ လိုအပ်လို့ အခဲ့ဒီ Product မျိုး နောက်တစ်ကြိမ် ထပ်လုပ်ရတယ်ဆိုရင် တိကျတဲ့ Specification ရရှိဖို့ အလားအလာ အများကြီးကောင်းသွားပါတယ်။

အမှန်တစ်ကယ် အသုံးပြုမယ့် User က Product ကိုမြင်ရမှ သူလိုချင်တာ ဟုတ်မဟုတ် သိနိုင်တယ်လို့ ပြောခဲ့ပါတယ်။ ဒါလည်း ခြွင်းချက်ရှိပါတယ်။ တစ်ခါတစ်ရံ၊ တစ်ချို့လုပ်ငန်းတွေမှာ လုပ်ငန်းရဲ့ Domain Knowledge အပြည့်အစုံရှိယုံမက နည်းပညာကိုလည်း ကျွမ်းကျင်နားလည်တဲ့သူတွေ ရှိနေတက်ပါတယ်။ သူတို့မှာ သူတို့ကိုယ်ပိုင် Product Vision တိတိကျကျရှိနေပြီး ဖြစ်တက်ပါတယ်။ အဲ့ဒီလိုအခြေအနေမျိုးအတွက် Software ရေးသားပေးရမှာဆိုရင်လည်း အတော်လေးတိကျတဲ့ Specification ကို အသင့်ရရှိမှာဖြစ်လို့ Waterfall Model က အသုံးဝင်လာမှာပဲ ဖြစ်ပါတယ်။

Waterfall Model ရဲ့ နောက်အားသာချက်တစ်ခုကို အထက်မှာလည်း ပြောခဲ့ပြီးဖြစ်ပါတယ်။ Software တစ်ခုကို အဖွဲ့လိုက်ပူး ပေါင်းရေးသားကြတဲ့အခါမှာ၊ အတွေ့အကြုံနည်းသေးတဲ့ Junior Developer တွေ ပါဝင်မှုများနေတယ်ဆိုရင် Waterfall Model နဲ့ပဲအလုပ်လုပ်ကြရမှာဖြစ်ပါတယ်။ Junior တွေမှာ ပြင်လွယ်ပြောင်းလွယ်ပြီး Flexible ဖြစ်တဲ့ Code တွေ ရေးသားနိုင်မှု အရည်အသွေး အားနည်းတက်ပါတယ်။ Waterfall Model နဲ့မှသာ ကြိုတင်ဖန်တီးပေးထားတဲ့ Specification နဲ့ Design ကို အားပြုပြီး Coding ပိုင်းကို Junior Developer တွေက Focus လုပ်နိုင်မှာပဲဖြစ်ပါတယ်။

1.3 – Iterative Development

Waterfall Model မှာရှိနေတဲ့ အားနည်းချက်တွေကို လေ့လာသုံးသပ်ပြီး၊ ထပ်မံတီထွင်လာကြတဲ့ Processes Management Method အမျိုးမျိုးရှိပါတယ်။ Iterative Development, Incremental Development, Extreme Programming, Rapid Application Development, Feature Driven Development စသဖြင့် အမျိုးမျိုးရှိပါတယ်။ ဒီနည်းစနစ် တွေကိုစုစည်းပြီး Agile Development လို့လည်း ခေါ်ပါတယ်။ နည်းစနစ်တစ်ခုချင်းစီမှာ ပါဝင်တဲ့ အသေးစိတ်လုပ်ငန်းစဉ်တွေ ကွဲပြားသွားကြပေမယ့် ရည်ရွယ်ချက်ကတော့ တူကြပါတယ်။ Agile Development ရဲ့ ရည်ရွယ်ချက်တွေကတော့ –

1. **Adaptive Planning** – ပုံသေသတ်မှတ်ထားခြင်းမဟုတ်ပဲ လိုအပ်သလို ပြင်လွယ်ပြောင်းလွယ်တဲ့ Plan ပေါ်မှာ အခြေခံနိုင်ဖို့
2. **Evolutionary Development** – Product တစ်ခုလုံးကို တစ်ခါတည်း တည်ဆောက်မယ့်အစား အဆင့်လိုက် တည်ဆောက်သွားနိုင်ဖို့

3. **Early Delivery** – အသုံးပြုသူက Product တစ်ခုလုံးပြီးစီးအချောသတ်အောင် စောင့်ဖို့မလိုပဲ စောနိုင်သမျှ အစောဆုံး Product ကို စတင်အသုံးပြုနိုင်ဖို့
4. **Continuous Improvement** – Software ရဲ့ လုပ်ဆောင်ချက်နဲ့ အရည်အသွေး အဆင့်မြှင့်တင်မှုတွေကို အဆက်မပြတ် ဆောင်ရွက်သွားနိုင်ဖို့
5. **Rapid and Flexible Response to Change** – Requirement နဲ့ နည်းပညာ အပြောင်းအလဲရှိလာတိုင်း လိုအပ်သလို လက်ခံပြောင်းလဲနိုင်တဲ့ Product တစ်ခုဖြစ်အောင် စီမံရေးသားဖို့

– တို့ပဲဖြစ်ပါတယ်။

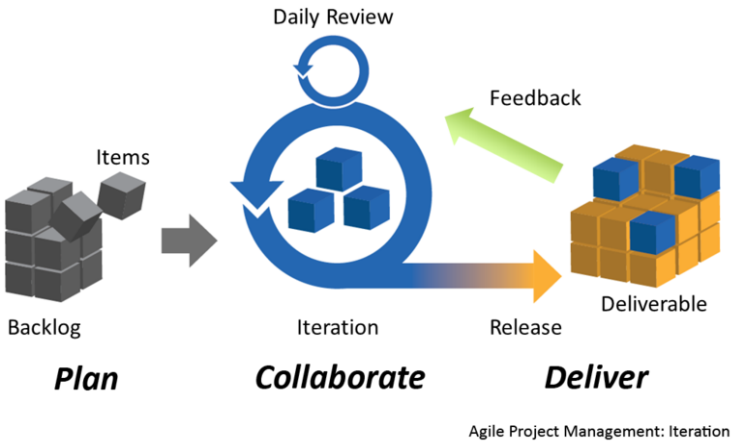
Agile Development ဆိုတာ နည်းစနစ်တစ်မျိုးကို ခေါ်တာမျိုးမဟုတ်ပါဘူး။ ရည်ရွယ်ချက် (Principle) ကို ခေါ်တာပါ။ အထက်မှာဖော်ပြခဲ့တဲ့ ရည်ရွယ်ချက်တွေနဲ့ ဖန်တီးထားတဲ့ မည်သည့် Processes Management နည်းစနစ်ကိုမဆို Agile Development (သို့မဟုတ်) Agile Method (သို့မဟုတ်) Agile Process Management လို့ ခေါ်ဆို နိုင်ခြင်းပဲ ဖြစ်ပါတယ်။

Iterative Development ဆိုတာ Agile နည်းစနစ်များစွာထဲမှာ အဓိကကျတဲ့ နည်းစနစ်တစ်ခုပဲဖြစ်ပါတယ်။ ဒီနည်းစနစ်မှာလည်း ပထမဦးဆုံးအနေနဲ့ Requirement Specification ကို စတင်တည်ဆောက်ရတာပါပဲ။ ဒါပေမယ့် အားလုံးကို အသေးစိတ်မသတ်မှတ်ပဲ၊ အခြေခံလိုအပ်ချက်တွေပါဝင်တဲ့ Project Scope ရရှိပြီး လိုအပ်နိုင်တဲ့အချိန်၊ လူအင်အားစတဲ့ Resource တွေကို Estimate လုပ်နိုင်စေဖို့ လိုအပ်သလောက်ကိုသာ စတင်ရေးဆွဲရပါတယ်။ ဒီအဆင့်ကို Iterative Development မှာ **Inception** လို့ခေါ်ပါတယ်။

နောက်တစ်ဆင့်အနေနဲ့ Non-Functional Requirement ကို ဦးဆုံးဖြေရှင်းပါတယ်။ Software မှာ မျက်မြင်ပါဝင်မယ့် လုပ်ဆောင်ချက်တွေမဟုတ်ပဲ၊ နောက်ကွယ်ကအခြေခံတွေဖြစ်တဲ့ Maintainability, Security, Performance, Scalability စတဲ့လုပ်ငန်း တွေအတွက် အခြေခံ Architecture Foundation ကို တည်ဆောက်ခြင်းဖြစ်ပါတယ်။ Specification ဟာ အခြေအနေပေါ်မူတည်ပြီး ပြောင်းလဲနိုင်တယ်ဆိုပေမယ့်၊ Function Requirement ပိုင်းမှာ အပြောင်းအလဲက ပိုများမှာ ဖြစ်ပါတယ်။ Architecture Requirement မှာ အပြောင်းအလဲ ရှိတယ်ဆိုရင်တောင် Functional ပိုင်းအပြောင်းအလဲနဲ့ ယှဉ်ရင် မပြောပလောက်ပါဘူး။ ဒီအဆင့်ကို **Elaboration** လို့ခေါ်ပါတယ်။

လိုအပ်တဲ့အခြေခံ Architecture ရပြီဆိုတော့မှ Product မှာပါဝင်တဲ့ လုပ်ဆောင်ချက်များထဲက တစ်ခုကို ဦးဆုံးတည်ဆောက်ပါတယ်။ အဲ့ဒီလုပ်ဆောင်ချက်ကို လက်တွေ့အသုံးပြုလို့ရတဲ့အဆင့်ထိအပြီးဆောင်ရွက်ရပါတယ်။ Prototype တွေ လုပ်ရပါတယ်။ Coding တွေလုပ်ရပါတယ်။ ရရှိလာတဲ့ရလဒ်ကိုမှ လေ့လာသုံးသပ်မှုတွေ၊ User Feedback တွေရယူပြီး Specification ကိုရော၊ Code ကိုပါ လိုအပ်သလိုပြင်ဆင်ရပါတယ်။ Software Development Process မှာ ပါဝင်တဲ့ လုပ်ငန်းအားလုံးကို Software Product တစ်ခုလုံးအတွက် မဟုတ်ပဲ Feature တစ်ခုအတွက် အစအဆုံးအကုန် ဆောင်ရွက်သွားတဲ့ သဘောဖြစ်ပါတယ်။ ဒီအဆင့်ကိုတော့ **Construction** လို့ခေါ်ပါတယ်။ ဒီလုပ်ဆောင်ချက်တစ်ခုက လိုချင်တဲ့အနေအထားနဲ့ အမှန်တစ်ကယ်ကိုက်ညီပြီဆိုတော့မှ နောက်လုပ်ဆောင်ချက်တစ်ခုကို ဆက်လက်ဆောင်ရွက်ခြင်း ဖြစ်ပါတယ်။

ဒီနည်းနဲ့ Feature တစ်ခုကို လက်စသတ်နိုင်ပြီဆိုရင် အရင်ရရှိထားပြီး Feature တွေနဲ့ ပေါင်းစပ်ရပါတယ်။ ပေါင်းစပ်ပြီး ရရှိလာတဲ့ ရလဒ်အတွက်လည်း User Feedback တွေ ထပ်မံရယူပါတယ်။ ဒီအဆင့်ကို **Transition** လို့ ခေါ်ပါတယ်။ ဒီနည်းအတိုင်း လုပ်ဆောင်ချက်တစ်ခုပြီးတစ်ခုကို အလှည့်ကျဆောင်ရွက်သွားတဲ့ နည်းစနစ်ဖြစ်လို့ Iterative Development လို့ ခေါ်ခြင်းဖြစ်ပါတယ်။ တစ်ဆင့်ပြီးတစ်ဆင့် Product မှာပါဝင်တည်း Feature တွေလည်းတိုးလာလို့ Incremental Development လို့လည်း ခေါ်ပါတယ်။



ပုံ (၁.၄) - Iterative Development

Source – Wikipedia

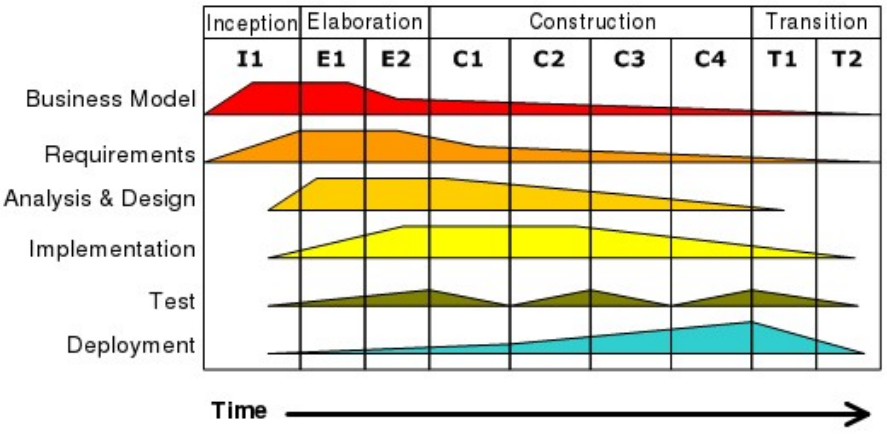
Iterative Development နည်းနဲ့ ဒီလိုတည်ဆောက်သွားတဲ့အတွက် လိုအပ်ချက်နဲ့ အမှန်တစ်ကယ်ကိုက်ညီခြင်း ရှိမရှိကို Product တစ်ခုလုံးပြီးအောင် စောင့်စရာမလိုပဲ သိရှိနိုင်လို့ Waterfall Model မှာရှိခဲ့တဲ့ Risk တွေလည်း နည်းသွားမှာ ဖြစ်ပါတယ်။ Iterative Development ကို လက်တွေ့အကောင်အထည်ဖော်ရာမှာ အသုံးပြုရတဲ့ SCRUM လို့ခေါ်တဲ့ Project Management နည်းစနစ်တစ်ခုကို **အခန်း (၇) – Team Collaboration** မှာ ထည့်သွင်း ဖော်ပြပေးပါဦးမယ်။

Iterative Development မှာရှိနိုင်တဲ့ အဓိကအားနည်းချက်ကတော့၊ Team မှာ ပါဝင်တဲ့ Developer တိုင်းဟာ အတွေ့အကြုံကြွယ်ဝပြီး ကျွမ်းကျင် Developer တွေဖြစ်ဖို့ လိုအပ်နိုင်ခြင်းဖြစ်ပါတယ်။ စနစ်က Flexible ဖြစ်တဲ့ Plan, Requirement နဲ့ Design ကို အခြေခံထားတာမို့ Developer တွေကလည်း Flexible ဖြစ်ဖို့လိုပါတယ်။ အပိုင်းလိုက်ခွဲခြား တည်ဆောက်ထားပေမယ့် လိုအပ်သလို ပေါင်းစပ်နိုင်တဲ့ Modular စနစ်မျိုးကို ရေးသားနိုင်သူတွေဖြစ်ရပါတယ်။ လိုအပ်သလို ပြုပြင်ပြောင်းလဲရမှာမို့ Maintainable ဖြစ်တဲ့ Code တွေ၊ Extensible ဖြစ်တဲ့ Code တွေရေးနိုင်ဖို့ဟာ မဖြစ်မနေလိုအပ်ချက် ဖြစ်လာပါတယ်။ အဆင့်လိုက် ခွဲခြားစမ်းသပ်ရမှာဖြစ်လို့ Unit Testing နဲ့ Continues Integration နည်းစနစ်တွေကို နားလည်အသုံးချနိုင်ဖို့ လိုအပ်ပါတယ်။ Developer တွေမှာ ဒီလိုအရည်အချည်းတွေမရှိရင် Iterative Development နည်းစနစ်ကို အမှန်တစ်ကယ် လက်တွေ့ အသုံးချဖို့မှာ အခက်အခဲတွေနဲ့ ကြုံတွေ့ရမှာပဲဖြစ်ပါတယ်။

Conclusion

Development Process Management တွေ အမျိုးမျိုးရှိကြပေမယ့်၊ လိုရင်းကို ခြုံကြည့်လိုက်ရင် အဓိကကွာခြားချက် တစ်ခုပဲရှိတာကို တွေ့ရမှာဖြစ်ပါတယ်။ Waterfall Model လို နည်းစနစ်မျိုးမှာ Planning ပိုင်းဖြစ်တဲ့ Requirement Analysis နဲ့ System Design ရေးဆွဲခြင်းကဲ့သို့ လုပ်ငန်းများကို အချိန်ပေးပြီး ဦးစွာဆောင်ရွက်ရပါတယ်။ Iterative Development လို နည်းစနစ်မျိုးမှာတော့ Planning ပိုင်းကို အလွန်အမင်းအချိန်မပေးပဲ Development ပိုင်းကို အတက်နိုင်ဆုံး အရင်သွားပြီးမှ လိုအပ်သလို Plan ပိုင်းကို တွဲဖက်ဆောင်ရွက်သွားတဲ့ နည်းစနစ်ဖြစ်ပါတယ်။

Waterfall Model က Development Process မှာ ပါဝင်တဲ့ အဆင့်တွေကို အစီအစဉ်အတိုင်း တစ်ဆင့်ပြီးမှ တစ်ဆင့် ဆောင်ရွက်ရပေမယ့် Iterative Development ကတော့ ပါဝင်တဲ့အဆင့်တွေကို အခုလိုပုံသဏ္ဍာန်မျိုးတဲ့ တွဲဖက်ဆောင် ရွက်သွားတဲ့သဘော ဖြစ်ပါတယ်။



ပုံ (၁.၅) - Timeline of Iterative Development

Source: Wikipedia

ပထမဆုံး Inception အဆင့်မှာ Business Model နဲ့ Requirement Specification လိုကိစ္စမျိုးကို အဓိက အချိန်ပေးပေမယ့် Coding နဲ့ Testing လုပ်ငန်းတွေလည်း လုပ်လို့ရသလောက် စလုပ်နေပါပြီ။ အလားတူပဲ Elaboration အဆင့်မှာ လည်း Architecture Design လုပ်ငန်းက အဓိကဆိုပေမယ့် Coding နဲ့ Testing တွေကိုပါ ဆောင်ရွက်နေပါပြီ။ Construction အဆင့်မှာတော့ Coding က အဓိကဆိုပေမယ့် Business Model, Requirement နဲ့ Design တို့ကိုလည်း ရရှိလာတဲ့ Feedback များပေါ်အခြေခံပြီး လိုအပ်သလိုဆက်လက် ဆောင်ရွက်ရခြင်းဖြစ်ပါတယ်။ မသေချာတဲ့အတွက် Risk မြင့်တဲ့ Planning ကို အချိန်အရမ်းပေးမယ့်အစား၊ Coding ကို စောနိုင်သမျှစောအောင်၊ Product ကို လက်တွေ့အသုံးပြု မယ့်သူလက်ထဲ စောနိုင်သမျှစောစော ရောက်အောင် ဆောင်ရွက်ခြင်းဖြစ်ပါတယ်။ ဒါကို တစ်ချို့ကလည်း အမှတ်မှားပြီး Agile Method တွေမှာ Plan မလိုဘူးလို့ ဆိုတက်ကြပါတယ်။ အမှန်တော့ Plan မရှိတာမဟုတ်ပါဘူး။ Plan ကို Flexible ဖြစ်အောင် စီမံထားခြင်းသာဖြစ်ပါတယ်။

လိုရင်းကိုပြန်ကောက်ရရင်တော့၊ မည်သည့် Software Development Project မဆို Plan, Design, Implementation, Test စသဖြင့် အဆင့်ဆင့်ပါဝင်ကြတာချည်းပဲ ဖြစ်ပါတယ်။ အဲ့ဒီအဆင့်တွေကို စီမံပုံစီမံနည်းအမျိုးက ရွေးထုတ်ဖော်ပြခဲ့တဲ့ Waterfall Model နဲ့ Iterative Development တို့မှာ အားသာချက်အားနည်းချက်ကိုယ်စီ ရှိကြပါတယ်။ Waterfall Model ဟာ သဘောသဘာဝအရစနစ်ကျပေမယ့်၊ ပုံသေသတ်မှတ်ထားတဲ့ Specification နဲ့ Design ပေါ်မှာ အခြေခံရတဲ့အတွက် အားနည်းချက်ရှိနေပါတယ်။ ဒီအားနည်းချက်ကို ဖယ်ထုတ်နိုင်ရင်တော့ Waterfall Model ကို အသုံးပြုသင့်ပါတယ်။ တစ်နည်းအားဖြင့် တိကျသေချာတယ်လို့ အာမခံနိုင်လောက်တဲ့ Requirement ရှိထားရင် Waterfall Model ကို အသုံးပြုသင့်ပါတယ်။

ကြိုတင်မှန်းဆရခက်ပြီး အပြောင်းအလဲများနိုင်တဲ့ အခြေအနေတွေမှာ Iterative Development ကို အသုံးပြုသင့်ပါတယ်။ လက်တွေ့မှာ Iterative Development နဲ့ အခြား Agile Method တွေဟာ Waterfall ရဲ့ အားနည်းချက်တွေကို လေ့လာဖန်တီးထားကြတာမို့ ပိုကောင်းပါတယ်။ အပြောင်းအလဲဒါဏ်ကို ပိုခံနိုင်ပါတယ်။

နောက်အခန်းတွေမှာ Software Project နဲ့ Development Process များကို စီမံရာမှာ အသုံးဝင်မယ့် နည်းစနစ်တွေနဲ့ Tool တွေကို ဆက်လက်ဖော်ပြပေးသွားမှာဖြစ်ပါတယ်။

ပြင်တာကတစ်နေရာ၊ မဆီမဆိုင်လို့ထင်ရတဲ့ တစ်ခြားနေရာမှာ
သွားမှားနေတယ်ဆိုတာ ဖြစ်တက်တဲ့ပြဿနာပါ။ ဒါကြောင့်
ပြင်ဆင်မှုတစ်ခုပြုလုပ်လိုက်တိုင်း အခြားလုပ်ဆောင်ချက်တွေကိုပါ
သေချာအောင် ပြန်စမ်းဖို့လိုနိုင်ပါတယ်။ ဒီလုပ်ငန်းကိုအမြဲ
Manual လုပ်နေရမယ်ဆိုရင် အဆင်ပြေမှာမဟုတ်ပါဘူး။

Professional Web Developer Course

HTML5, PHP/MySQL, jQuery/Ajax, Mobile Web စသည့်
Professional Web Developer တစ်ဦး သိရှိထားသင့်သည့်
နည်းပညာများကို စုစည်းသင်ကြားခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၂) – Test Driven Development

မနက်ဖြန် Project Presentation လုပ်ရတော့မယ်။ (၆) လလောက် အချိန်ပေးပြီး ကြိုးစားထား သမျှအားလုံးဟာ ဒီနေ့အတွက်ပဲ။ အောင်မြင်တဲ့လုပ်ငန်းရှင် Customer တွေကို ဆွဲဆောင်ရတာ လွယ်တဲ့အလုပ်မဟုတ်ဘူး။ အတော်လက်ပေါက်ကပ်တဲ့သူတွေ။ ပေါက်ပေါက်ရှာရှာလည်းမေးတက်တယ်။ ဒါကြောင့်လည်း အောင်မြင်ကြတာဖြစ်မှာပေါ့။ ပြဿနာတော့မရှိပါဘူး။ ကိုယ့်ဘက်ကလည်း အားလုံးထောင့်စိအောင် လုပ်ထားပြီးသား။

အခြေခံလုပ်ဆောင်ချက်တွေကို အရင်ရှင်းပြရမယ်။ ပြီးမှ တစ်ကယ့် Key Point ကို ထုတ်ပြရမယ်။ ခလုပ်တစ်ချက်နှိပ်ယုံနဲ့ အထိရောက်ဆုံး Delivery Route ကို အလိုအလျောက် ဆွဲပေးနိုင်မှာလေ။ ဒီလောက်ကောင်းတဲ့စိတ်ကူး၊ မကြုံက်စရာ ဘာအကြောင်းမှမရှိဘူး။ ဒါတင် ဘယ်ကမလဲ။ Mobile App ကနေလည်း Delivery Route ကို Maps နဲ့ ပြပေးနိုင်သေးတယ်။

အင်း... အခုမှကြည့်မိတယ်။ Map ပေါ်မှာပြတဲ့လိပ်စာမှာ စာလုံးနည်းနည်းသေးနေတယ်။ သေချာ ကြည့်ရင်တော့ မြင်ရပါတယ်။ သိပ်ပြဿနာမရှိလှပါဘူး။ ဒါပေမယ့် ဒီအပိုင်းက အဓိကဆိုတော့ ဖြစ်နိုင်ရင် Perfect ဖြစ်ချင်တယ်။ စာလုံး Size ပြင်တာ ဘာမှခက်တဲ့ကိစ္စလည်းမဟုတ်ဘူး။ Font-Size: 9pt ဆိုတဲ့ အပိုင်းကို Font-Size: 11pt လို့ ပြင်လိုက်ယုံပဲ။ ဒါလေးတစ်ချက်တော့ ပြင်လိုက်ဦးမယ်။ ဘာမဟုတ်တာလေးနဲ့ Impression Out နေပါဦးမယ်။

အိုကေ... ပြင်ပြီးပြီ။ စမ်းကြည့်တာလည်း အဆင်ပြေတယ်။

– နောက်တစ်နေ့။ အခြေခံလုပ်ဆောင်ချက်များ ရှင်းလင်းတင်ပြပြီးနောက် –

"အခုတွေ့ရမှာကတော့ ဒီစနစ်ရဲ့ အဓိကအကျဆုံးလုပ်ဆောင်ချက်ဖြစ်ပါတယ်။ ခလုပ်တစ်ချက်နှိပ်ယုံနဲ့ လက်ခံရရှိထားတဲ့ Order တွေရဲ့ လိပ်စာကိုအခြေခံပြီး အထိရောက်ဆုံး Delivery Rout ကို အလိုအလျောက် ဆွဲပေးမှာဖြစ်ပါတယ်"

"တယ်ဟုတ်ပါလား။ အဲ့ဒီ Route ကို ပစ္စည်းပို့သမားကို ဘယ်လိုပေးမလဲ"

"အဲ့ဒီအတွက် Mobile App တစ်ခုစီစဉ်ထားပါတယ်။ App ကိုဖွင့်ပြီး Sync လို့ပြောလိုက်တာနဲ့ Delivery Route ကို Mobile Device ပေါ်မှာ Map နဲ့ ပြပေးမှာပါ"

"ဟုတ်လား။ စိတ်ကူးကောင်းသားပဲ။ ပြကြည့်ပါဦး"

"ဟုတ်ကဲ့... အခုလို Sync Button ကိုနှိပ်လိုက်တာနဲ့ ... ... "

– မရတော့ပါလား။ မနေ့ကပဲစမ်းထားသေးတယ်။ ဘာဖြစ်တာပါလိမ့်။ ပြဿနာပဲ –

"ခဏလေးပါခင်ဗျ။ Internet Connection ကြောင့်လားမသိဘူး။ Data Sync မဖြစ်ဘူး ဖြစ်နေ တယ်"

– အင်တာနက်က ရနေရဲ့သားနဲ့ အလုပ်မလုပ်ဘူး၊ ပြဿနာပဲ –

" ... ... "

"မင်းစနစ်ကဆိုးတော့မဆိုးပါဘူး။ ဒါပေမယ့် ပစ္စည်းပို့သမားကိုမပေးနိုင်ရင် ဆွဲထားတဲ့ လမ်းကြောင်းက အသုံးမဝင်ဘူးလေ"

"ဟုတ်ပါတယ်ခင်ဗျ။ ဒါကို ထည့်စဉ်းစားဖြီးသာပါ။ မနေ့ကတောင် အလုပ်လုပ်ပါသေး တယ်"

"အေး... အလုပ်လုပ်တယ်ဆိုရင် ပြလေ"

" ... ... "

"တစ်ကယ်ပါဗျာ... ကျိမ်ဆိုကျိမ်ပြပါ့မယ်။ မနေ့ကတောင် စမ်းထားပါသေးတယ်။ အကောင်းကြီးပါ။ ခုမှ ဘာဖြစ်မှန်းမသိဘူး။ အလုပ်မလုပ်တော့ဘူး"

ဒီလိုအဖြစ်အပျက်မျိုးဟာ Software Developer တွေ Project ပြဿနာတိုင်း ဒူးတုန်ရခြင်းရဲ့ အကြောင်းရင်းဖြစ်ပါ တယ်။ ဘာမှ မှားစရာမရှိဘူးလို့ ယူဆထားတဲ့ စနစ်တစ်ခုက အရေးအကြောင်းမှ အလုပ်မလုပ်တော့တာမျိုး ဖြစ်တက်ပါတယ်။ မဖြစ်စလောက် လို့ယူဆရတဲ့ ပြင်ဆင်မှုလေးတစ်ခုကြောင့် စနစ်ကအလုပ်မလုပ်တော့မျိုး လည်း ဖြစ်နိုင်ပါတယ်။ မဆီမဆိုင်လို့ ယူဆရတဲ့နေရာမှာ ပြဿနာသွားတက်နေတက်ပါတယ်။

ဘယ်လောက်ပဲ စနစ်ကျအောင် ရေးသားထားတဲ့ Code ဖြစ်ပါစေ၊ Code Base ကြီးလာတဲ့အခါမှာ ဒီလိုပြဿနာ တွေဟာ အနည်းနဲ့အများ ရှိလာတက်ကြတာပါပဲ။ ဒါကြောင့်ပဲ အလုပ်လုပ်နေတဲ့ Code ကို ထိရမှာ၊ ပြင်ရမှာကို Developer တွေ စိုးရွံ့တက်ကြပါတယ်။ လုပ်ဆောင်ချက် (၁၀၀) လောက်ပါတဲ့ စနစ်ကြီးတစ်ခုမှာ ပြင်ဆင်မှု လေးတစ်ခု လုပ်လိုက်တိုင်း သေချာအောင်ဆိုပြီး လုပ်ဆောင်ချက် (၁၀၀) လုံးကို ပြန်လိုက်စမ်းရမလိုဖြစ်နေပါ တယ်။

Software တိုင်းမှာ ရုတ်တရက်သတိမမူမိနိုင်တဲ့ အမှားလေးတွေ ရှိတက်ပါတယ်။ Technical Debt တွေရဲ့ အကျိုးဆက်ပါ။ နောက်မှပြင်မယ်ဆိုပြီး အလွယ်ရေးလိုက်မိတဲ့ Code တစ်ခုက မထင်မှတ်တဲ့အချိန်မှာ ထပြီး ပြဿနာရှာခြင်းပဲဖြစ် ပါတယ်။ Technical Debt တွေကို အပြတ်ရှင်းဖို့ဆိုရင် Code ကို Refactor လုပ်ရပါတယ်။

Refactor ဆိုတာ အလုပ်လုပ်နေပြီးသား Code ကို (အလုပ်လုပ်ပုံမပြောင်းလဲစေပဲ) Code အရည်အသွေးတိုးတက်လာအောင် ပြင်ဆင်ခြင်း ဖြစ်ပါတယ်။ အရည်အသွေးကောင်းတဲ့ Quality Code ဆိုတာ ရေးသားပုံရှင်းလင်းပြီး၊ နားလည်ရလွယ်တဲ့ Code ပြင်ဆင်ရလွယ်တဲ့ Code ကိုဆိုလိုတာပါ။

ဒီနေရာမှာ အရေးပါလာတဲ့ Principle နှစ်ခုကတော့ D.R.Y နဲ့ Decoupling တို့ပဲဖြစ်ပါတယ်။ ဒီနှစ်ခုအကြောင်းကို **Professional Web Developer ရဲ့ အခန်း (၁၃) – Model-View-Cotroller** မှာလည်း ပြောခဲ့ဖူးပါတယ်။

D.R.Y (Don't Repeat Yourself) ဆိုတာ၊ အရည်အသွေးကောင်းတဲ့ Code ဖြစ်ဖို့ Code Duplication ကို လျှော့ချရမယ်ဆိုတဲ့ Principle ဖြစ်ပါတယ်။ Code Duplication လျှော့ချခြင်းဆိုတာ ထပ်ခါထပ်ခါလုပ်ဖို့လိုအပ်တဲ့ တူညီတဲ့လုပ် ဆောင်ချက်တွေကို အကြိမ်ကြိမ်ပြန်လည်ရေးသားမနေပဲ၊ တစ်ကြိမ်သာ သီးခြားခွဲပြီးရေးသားထားခြင်း ဖြစ်ပါတယ်။ အလားတူ လုပ်ဆောင်ချက် ထပ်မံဆောင်ရွက်ဖို့လိုလာရင် အဲ့ဒီခွဲခြားရေးသားတဲ့ Function (သို့မဟုတ်) Component ကို ခေါ်ယူ အသုံးပြုရမှာပါ။ ဒီတော့ အခြေအနေအရ ပြင်ဆင်ပြောင်း လဲဖို့လိုလာရင် ခွဲခြားရေးသားထားတဲ့ တစ်နေရာမှာပြင်လိုက်ယုံပါပဲ။ အကယ်၍များ လုပ်ဆောင်ပုံတူညီတဲ့ကိစ္စတွေကို ဟိုနား ဒီနား ထပ်ခါထပ်ခါရေသားထားမိရင် နောင်လိုအပ်လို့ အဲ့ဒီလုပ်ဆောင်ချက်ကို ပြင်ဆင်ပြောင်းလဲရတဲ့အခါ ရေးသားထားမိသမျှနေရာတိုင်းမှာ လိုက်ပြင်ရတော့မှာပါ။

Decoupling ဆိုတာကတော့ Function တစ်ခု (သို့မဟုတ်) Component တစ်ခုဟာ အလုပ်တစ်ခုကိုပဲ လုပ်သင့်တယ် ဆိုတဲ့ အဓိပ္ပါယ်ဖြစ်ပါတယ်။ ပြီးတော့ တစ်ခြား Function တစ်ခုပေါ်မှီခိုနေခြင်း မရှိလေကောင်းလေဆိုတဲ့ အဓိပ္ပါယ်လည်း ဖြစ်ပါတယ်။ အကယ်၍များ Function တစ်ခုရဲ့လုပ်ဆောင်ချက်က အခြား Function ကိုမှီခိုနေရင်၊ မှီခိုနေရတဲ့ Function ကို ပြင်ဆင်လိုက်တဲ့အခါ သူ့မှာလာပြီး သက်ရောက်နိုင်ပါတယ်။ ဒီတော့ ပြင်ချင်လိုတဲ့ Function ကတစ်ခု၊ ဒါပေမယ့် အဲ့ဒီ Function ကိုမှီခိုနေတဲ့ Function ကိုပါ လိုက်ပြင်ရတော့မယ့်သဘောဖြစ်နေတက်ပါတယ်။

Refactor လုပ်တယ်ဆိုရင် Code ထဲမှာရှိနေတဲ့ Duplicate တွေကို ရှာဖွေပြင်ဆင်ခြင်းနဲ့၊ Tightly Couple ဖြစ်နေတဲ့ Function တွေကို ပြင်ဆင်ရေးသားခြင်းအားဖြင့် Code Structure ကို ပြုပြင်ခြင်းဖြစ်ပါတယ်။ ဒီနေရာမှာ ဇာတ်လမ်းစ တော့တာပါပဲ။ Refactor လုပ်မှ Technical Debt တွေနည်းမှာပါ။ ဒါပေမယ့် Refactor လုပ်လိုက်မှ မူလအလုပ်လုပ် နေတာတွေ၊ အလုပ်မလုပ်တော့တာမျိုးဖြစ်သွားတက်လို့ အရမ်းအရမ်းလဲ Refactor မလုပ်ရဲပြန်ပါဘူး။

Project ကိုစတင်စဉ်က ဘယ်လောက်ပဲသတိတွေထားပြီး Code အရည်အသွေးကို ထိမ်းသိမ်းလာခဲ့ပေမယ့် အချိန်ကြာလာတာနဲ့အမျှ Technical Debt တွေကတော့ ရှိလာတက်တာပါပဲ။ ကျွန်တော်တို့လိုအပ်နေတာကရေးထားတဲ့ Code ကို ပြင်ဆင်မှုတစ်ခုလုပ်လိုက်တိုင်း အလိုအလျောက် စမ်းသပ်ပေးနိုင်မယ့် Test Method တွေဖြစ်ပါတယ်။ တစ်နေရာမှာပြင်လိုက်တဲ့အတွက် အခြားနေရာမှာသွားပြီး ပြဿနာဖြစ်မနေကြောင်းကို အာမခံပေးနိုင်မယ့် စနစ်တွေ လိုအပ်နေတာပါ။

2.1 – Software Testing

Software တစ်ခုရဲ့အရည်အသွေးကို ထိမ်းသိမ်းစီစစ်နိုင်ဖို့အတွက် Test လုပ်ပုံလုပ်နည်း အမျိုးမျိုးရှိပါတယ်။ သတ်မှတ်ထားတဲ့ Specification နဲ့ ကိုက်ညီခြင်းရှိမရှိ စမ်းသပ်တဲ့ System Test နဲ့ Acceptance Test တို့လို Test Method တွေရှိသလို၊ ရေးထားတဲ့ Code တွေ မှန်ကန်စွာအလုပ်လုပ်မလုပ်ကို Test လုပ်တဲ့ Unit Test, Integration Test, Regression Test စတဲ့ Method တွေလည်းရှိပါတယ်။

Unit Test – Function တစ်ခု (သို့မဟုတ်) Component တစ်ခုကို Test လုပ်ခြင်းဖြစ်ပါတယ်။ ဥပမာ – စက်ဘီး တစ်စီး တည်ဆောက်မယ်ဆိုရင်၊ တာယာကောင်းရဲ့လား၊ ကျွတ်လုံရဲ့လား၊ ဂွေဖြောင့်ရဲ့လား စသဖြင့် အစိတ်အပိုင်း တစ်ခုခြင်းကို စမ်းသပ်ခြင်းဖြစ်ပါတယ်။ အသေးဆုံးနဲ့ အခြေခံအကျဆုံး Unit တွေကို စမ်းသပ်ခြင်းဖြစ်ပါတယ်။

Integration Test – Function တစ်ခုနဲ့တစ်ခု ပေါင်းစပ်ပြီးတဲ့နောက် အားလုံးပုံမှန်အလုပ်လုပ်ရဲ့လား စမ်းသပ်တဲ့အဆင့် ဖြစ်ပါတယ်။ ဥပမာ - ဂွေမှာ၊ ကျွတ်နဲ့တာယာတပ်ဆင်လိုက်တဲ့အခါ မျှော်မှန်းထားသလိုအလုပ်လုပ်ရဲ့လား၊ ကျွတ်သေးနေသလား၊ တာယာအံမကျဖြစ်နေသလား စပ်သပ်တဲ့အဆင့်ဖြစ် ပါတယ်။

Regression Test – အမှားတစ်ခု ပြင်ဆင်လိုက်တဲ့အခါ၊ အဲ့ဒီအမှားတစ်ကယ်ပြေလည်သွားရဲ့လား၊ ပြင်လိုက်တဲ့အတွက် အခြားသက်ရောက်မှုတွေ ရှိနေသလား စမ်းသပ်ခြင်းဖြစ်ပါတယ်။ ဥပမာ - ဂွေလိမ်နေလို့ ဖြောင့်လိုက်တဲ့အခါ ဂွေဖြောင့် သွားယုံသာမက တပ်ဆင်ထားတဲ့ ကျွတ်နဲ့တာယာပါ မူလအတိုင်း အဆင်ပြေပြေ ဆက်လက်အလုပ်လုပ်ရဲ့လား စစ်ဆေးခြင်း ဖြစ်ပါတယ်။

System Test – Software မှာ Runtime Error တွေမရှိတာသေချာအောင် စမ်းသပ်ခြင်းဖြစ်ပါတယ်။ Specification က သတ်မှတ်ထားတဲ့ သတ်မှတ်ချက်တွေအတိုင်း မှန်မမှန်ကိုလည်း ဒီအဆင့်မှာစစ်ပါတယ်။

Acceptance Test – လိုအပ်တဲ့ လုပ်ဆောင်ချက်တွက် အမှန်တစ်ကယ် ပါဝင်ပြည့်စုံခြင်း ရှိမရှိ၊ လက်တွေ့အသုံးဝင်ခြင်း ရှိမရှိ စစ်ဆေးခြင်းဖြစ်ပါတယ်။ အသုံးပြုသူကိုယ်တိုင် စမ်းတဲ့အဆင့်ပါ။

ဒီထဲကမှ Code Quality အတွက်အခြေခံအကျဆုံးနဲ့ ထိရောက်မှုအရှိဆုံးကတော့ Unit Test ပဲဖြစ်ပါတယ်။ Unit Test က နောက်ဆုံးရရှိလာတဲ့ Software Product ဟာ Specification နဲ့ကိုက်ညီမှုရှိမရှိ၊ လက်တွေ့အသုံးဝင်မဝင်လို ကိစ္စမျိုးကိုတော့ အာမခံမပေးပါဘူး။ ဒါပေမယ့် ရေးထားတဲ့ Code အစိတ်အပိုင်းတိုင်းဟာ မျှော်မှန်းထားတဲ့အတိုင်း မှန်ကန်စွာ အလုပ်လုပ်နေကြောင်းကိုတော့ အာမခံပေးနိုင်မှာဖြစ်ပါတယ်။ Unit Test က Code Quality ကို အာမခံပေး ပါတယ်။

2.2 – Test Codes

Unit Test အကြောင်းမလေ့လာခင်၊ ပိုပြီးအခြေခံကျတဲ့ Test Code တွေအကြောင်းကို အရင်ဖော်ပြလိုပါတယ်။ Test Code ဆိုတာ Software ရဲ့ လုပ်ဆောင်ချက်နဲ့ မသက်ဆိုင်ပဲ စမ်းသပ်စီစစ်မှုလုပ်ဖို့ ရည်ရွယ်ချက်နဲ့ ထည့်သွင်းရေးသား ထားတဲ့ Code တွေကိုဆိုလိုတာပါ။ Test Code တွေထဲမှာ အခြေခံ အကျဆုံးကတော့ **Debug Log** ဖြစ်ပါတယ်။

> Code နမူနာတွေဖော်ပြတဲ့အခါ၊ အချို့ Code တွေဟာ လက်တွေ့ကူးယူစမ်းသပ်လို့ရတဲ့ Code တွေ ဖြစ်ပြီး၊ အချို့ Code တွေကတော့ ဆိုလိုရင်း သဘောသဘာဝကို ပေါ်လွင်အောင် နမူနာ အနေနဲ့ ရေးပြတဲ့ Pseudo-Code တွေဖြစ်ပါတယ်။ လက်တွေ့စမ်းသပ်တဲ့အခါ၊ ပေးထားတဲ့ အတိုင်း တိုက်ရိုက် ကူးယူမယ့်အစား နမူနာ Code ကို နားလည်အောင်ဖတ်ပြီး၊ ဆိုလိုရင်းကို နားလည်ပြီဆိုမှ ကိုယ်တိုင် ရေးသား စမ်းသပ်သင့်ပါတယ်။

ဥပမာ - လက်ရှိအပူချိန်ကို ရယူပြီး သတ်မှတ်အပူချိန်ကို ကျော်လာတဲ့အခါ Warning Message ပေးတဲ့ လုပ်ဆောင်ချက် တစ်ခုဆိုပါစို့။ ဒီလုပ်ဆောင်ချက်ရရှိ ဖို့အတွက် Function နှစ်ခုလိုနိုင်ပါတယ်။ လက်ရှိအပူချိန် ရယူပေးတဲ့ Function နဲ့ ရရှိလာတဲ့အပူချိန်ကို စီစစ်ပေးတဲ့ Function တို့ ဖြစ်ပါတယ်။ JavaScript ကိုအသုံးပြုပြီး Code နမူနာဖော်ပြရရင် အခုလိုဖြစ်မှာပါ။

Pseudo-code

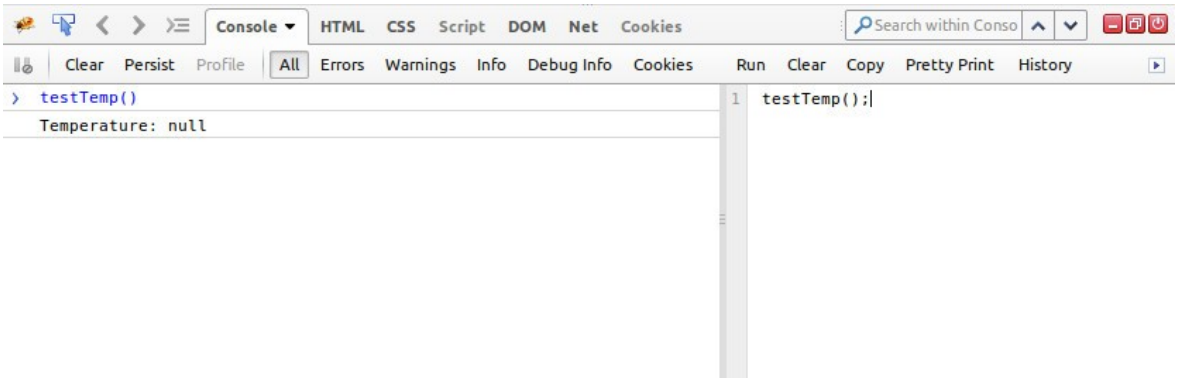
```
1. function getTemp() {
2.   return getTempFromServer("/path/to/api");
3. }
4.
5. function testTemp() {
6.   var temp = getTemp();
7.   if(temp.degc > 32) print("Warning: High Temperature");
8. }
```

နမူနာအရ `getTemp()` Function က လက်ရှိအပူချိန်ရယူပေးတဲ့ လုပ်ဆောင်ချက်ကိုလုပ်ပြီး၊ `testTemp()` Function ကတော့ ရလာတဲ့ Temperature ကိုစီစစ်ပေးပါတယ်။ အပူချိန်က 32 Degree ကို ကျော်လာရင် Warning Message ဖော်ပြပေးမှာဖြစ်ပါတယ်။ `testTemp()` Function ကို Evaluate လုပ်လိုက်တဲ့အခါ မျှော်မှန်းထားသလို အလုပ်မလုပ်ရင် ဖြစ်နိုင်ခြေ အမှားနှစ်ခုရှိပါတယ်။ `getTemp()` Function က လက်ရှိ အပူချိန်ကို ယူမပေးနိုင်တာဖြစ် နိုင်သလို၊ `testTemp()` Function ကိုယ်တိုင်မှာ အမှားရှိနေတာလည်း ဖြစ်နိုင်ပါတယ်။ ဒါကိုသေချာစေဖို့အတွက် အခုလို Debug Log တစ်ခု ထည့်လိုက်နိုင်ပါတယ်။

Pseudo-code

```
5. function testTemp() {
6.   var temp = getTemp();
7.   console.log("Temprature: " + temp.degc);
8.   if(temp.degc > 32) print("Warning: High Temperature");
9. }
```

လိုင်းနံပါတ် (၇) မှာ `console.log` ကိုသုံးပြီး `temp` ရဲ့ လက်ရှိတန်ဖိုးကို Console မှာ ဖော်ပြစေခြင်း ဖြစ်ပါတယ်။



ပုံ(၂.၁) - Console Log in Firebug Console

`testTemp()` Function ကို Evaluate လုပ်ကြည့်လိုက်လို့ **ပုံ(၂.၁)** မှာတွေ့ရတဲ့ ရလဒ်မျိုး ရရှိတယ်ဆိုရင် temp Variable ရဲ့ တန်ဖိုးဟာ null ဖြစ်နေကြောင်း သိရနိုင်ပါတယ်။ ဒါကြောင့် ပြဿနာက `getTemp()` Function က ပြန်ပေးတဲ့တန်ဖိုး မမှန်လို့ဖြစ်ကြောင်း သိရှိရမှာပဲဖြစ်ပါတယ်။ ဒီလို Code ထဲမှာ ရောစပ်ရေးသားတဲ့ Debug Log တွေ ကို အခြေခံအကျဆုံး Test Code တွေလို့ မှတ်ယူနိုင်ပါတယ်။

နောက်ထပ်အရေးပါတဲ့ Test Code တွေကတော့ **Stub** နဲ့ **Mock** တို့ပဲဖြစ်ပါတယ်။ အမှန်တစ်ကယ် အလုပ်မလုပ်ပဲ၊ မူရင်း Function ကို တုပထားတဲ့ Fake Function တွေဖြစ်ပါတယ်။ Stub နဲ့ Mock က အနည်းငယ်ကွာပါတယ်။ Stub Function တစ်ခုဟာ အခြေခံအားဖြင့် ဘာအလုပ်မှမလုပ်ပါဘူး။ မူရင်း Function က ပေးနိုင်တဲ့ Return Value ကိုပဲ တိုက်ရိုက် ပြန်ပေးတဲ့ Function အတုတစ်မျိုးဖြစ်ပါတယ်။ Mock ကတော့ Stub လို တန်ဖိုးတစ်ခု ပြန်ပေးယုံမဟုတ်ပဲ၊ မူရင်း Function ရဲ့ အလုပ်လုပ်ပုံကိုပါ တုပထားတက်ပါသေးတယ်။ Code နမူနာတစ်ချို့ ဖော်ပြပါမယ်။

Pseudo-code

```
function getTempStub() {
  console.log("Working with stub: getTemp()");
  return {
    degc: 24,
    degf: 45
  }
}

function getTempMock(degc) {
  console.log("Working with mock: getTemp()");
  return {
    degc: degc,
    degf: Math.round((degc * (5/9) + 32))
  }
}

function getTemp() {
  return getTempFromServer("/path/to/api");
}
```

ပေးထားတဲ့နမူနာမှာ `getTemp()` Function က မူရင်း Function ဖြစ်ပါတယ်။ မူရင်း Function က Server ကို Request ပြုလုပ်ပြီး လက်ရှိအပူချိန်ကို ရယူတဲ့အလုပ်ကို ဆောင်ရွက်ပါတယ်။ ပြဿနာက၊ ဒီ Function ကို စမ်းဖို့အတွက် အင်တာနက်အဆက်အသွယ် လိုနိုင်ပါတယ်။ ဒီတော့ `getTemp()` Function ကို စမ်းချင်တဲ့အခါတိုင်းမှာ အင်တာနက် အဆက်အသွယ်ရှိမှပဲ စမ်းလို့ရတော့မလို ဖြစ်နေပါတယ်။ ဒါကြောင့် `getTempStub()` ဆိုတဲ့ Stub Function တစ်ခုကို သီးခြားသတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။ `getTempStub()` Function က `getTemp()` ကို တုပထားခြင်း ဖြစ်ပြီး တစ်ကယ်အလုပ်မလုပ်ပါဘူး။ `getTemp()` Function က ပြန်ပေးမယ့် တန်ဖိုးနဲ့ ပုံစံတူတန်ဖိုးတစ်ခုကို ပြန်ပေးတဲ့အလုပ်ကို လုပ်ပေးခြင်းဖြစ်ပါတယ်။ ဒါကြောင့် `getTemp()` Function ကို ခေါ်ယူအသုံးပြုဖို့လိုတဲ့နေရာ တိုင်းမှာ `getTempStub()` Function ကို အစားထိုးအသုံးပြုခြင်းအားဖြင့် ကျန်လုပ်ဆောင်ချက်တွေကို အင်တာနက် အဆက်အသွယ် မလိုတော့ပဲစမ်းနိုင်သွားမှာဖြစ်ပါတယ်။

`getTempMock()` ဆိုတဲ့အမည်နဲ့လည်း Function တစ်ခုသတ်မှတ်ထားပါသေးတယ်။ သူလည်းပဲ စမ်းသပ်ဖို့ရည်ရွယ်ချက်နဲ့ `getTemp()` ကို တုပထားတဲ့ Function တစ်ခုပဲဖြစ်ပါတယ်။ ဒါပေမယ့် သူကတော့ `getTempStub()` လို တန်ဖိုးကို ဒီအတိုင်း ပြန်ပေးတာမဟုတ်တော့ပါဘူး။ သူကိုခေါ်ယူစဉ်မှာ Degree Celsius တန်ဖိုးကိုလက်ခံပြီး၊ `getTemp()` Function က ပြန်ပေးမယ့် Object နဲ့အလားတူ Object တစ်ခုရအောင် အဲ့ဒီ Degree Celsius ကို အသုံးပြုတည်ဆောက်ယူပါတယ်။ Return Value တစ်ခုပြန်ရယုံနဲ့ လုံလောက်တဲ့အခြေအနေတွေမှာ `getTempStub()` ကို သုံးနိုင်ပြီး Return Value အမျိုးမျိုးနဲ့ စမ်းသပ်ဖို့လိုတဲ့အခါ `getTempMock()` ကို အသုံးပြုနိုင်မှာပဲဖြစ်ပါတယ်။

အမှန်တော့ Stub တွေ Mock တွေဟာ မူရင်း Function ကို တုပယုံသက်သက်နဲ့ စမ်းသက်ယုံသက်သက်အတွက်တင် အသုံးဝင်တာမဟုတ်ပါဘူး။ သူတို့ကို Function Model အနေနဲ့လည်း ယူဆနိုင်ပါတယ်။ မူရင်း Function ဖြစ်တဲ့ `getTemp()` မှာ လိုအပ်တဲ့ လုပ်ဆောင်ချက်တွေကို ရေးသားထားပေမယ့် နောက်ဆုံးရမယ့် Return Value ဟာ ဘယ်လိုဘယ်ပုံ ဖွဲ့စည်းပြီး ဘယ်လိုတန်ဖိုး ဖြစ်မယ်ဆိုတဲ့ သက်မှတ်ချက် မပါဝင်ပါဘူး။ `getTempStub()` Function က `getTemp()` ကို တုပထားယုံသက်သက်မဟုတ်ပါဘူး။ Return Value ရဲ့

ဖွဲ့စည်းပုံကိုပါ သတ်မှတ်ထားတဲ့သဘောဖြစ် ပါတယ်။ `getTempStub()` ကို ကြည့်ပြီး `getTemp()` Function က ပြန်ပေးမယ့်တန်ဖိုးဟာ ဘယ်လိုပုံစံ ဖြစ်မယ်ဆိုတာကို သိရှိနိုင်မှာဖြစ်ပါတယ်။ အလားတူပဲ `getTempMock()` Function ကို ကြည့်ပြီး `getTemp()` Function ရဲ့ တစ်ကယ့် အလုပ်လုပ်ပုံကို သိရှိနိုင်ပါတယ်။ ဒါကြောင့် Stub တွေ Mock တွေကို စမ်းသပ်ယုံ၊ တုပယုံ သက်သက် သဘောမထားပဲ Function Model လို့လည်း သဘောထားနိုင်ပါတယ်။

Stub တွေ Mock တွေကို နေရာတိုင်းမှာသုံးဖို့တော့မလိုပါဘူး။ သုံးလေ့လည်းမရှိကြပါဘူး။ အထူးသဖြင့် Remote Data Source ကို ချိတ်ဆက်အလုပ်လုပ်ရမယ့် နေရာတွေမှာသာ အဲ့ဒီ Remote Data Source ကို အမှန်တစ်ကယ်ချိတ်ဆက် နေစရာမလိုပဲ စမ်းသပ်နိုင်အောင် အသုံးပြုလေ့ရှိကြပါတယ်။ Server Request လုပ်တဲ့ Function တွေ၊ Database နဲ့ ချိတ်ဆက်အလုပ်လုပ်တဲ့ Function တွေနဲ့၊ GPS တို့ Fingerprint တို့လို External Input Device တွေနဲ့ ချိတ်ဆက် အလုပ်လုပ်ရတဲ့ နေရာတွေမှာသုံးကြလေ့ရှိပါတယ်။

2.3 Unit Testing

Unit Testing ဆိုတာကတော့ Function တစ်ခု (Unit တစ်ခု) ဟာ မျှော်မှန်းထားသလို အလုပ်လုပ်ရဲ့လားလို့ စမ်းသပ်တဲ့ နည်းစနစ်တစ်ခုဖြစ်ပါတယ်။ တစ်နည်းအားဖြင့် Function တစ်ခုကို စမ်းသပ်ပေးတဲ့ Test Function တစ်မျိုးဖြစ်ပါတယ်။ ဥပမာ - ပေးလိုက်တဲ့ရက်စွဲနှစ်ခုကို နှိုင်းယှဉ်ပြီး၊ "just now", "3 hours ago", "1 week ago" စသဖြင့် ကြည့်ကောင်း အောင် Format လုပ်ပြီး ပြန်ပေးတဲ့ Function တစ်ခုရှိတယ်ဆိုပါစို့။

JavaScript

```
function prettyDate(now, time) {
  var date = new Date(time || ""),
    diff = (((new Date(now)).getTime() - date.getTime()) / 1000),
    day_diff = Math.floor(diff / 86400);

  return day_diff == 0 && (
    diff < 60 && "just now" ||
    diff < 120 && "1 minute ago" ||
    diff < 3600 && Math.floor( diff / 60 ) + " minutes ago" ||
    diff < 7200 && "1 hour ago" ||
    diff < 86400 && Math.floor( diff / 3600 ) + " hours ago"
  ) ||
    day_diff == 1 && "Yesterday" ||
    day_diff < 7 && day_diff + " days ago" ||
    day_diff < 31 && Math.ceil( day_diff / 7 ) + " weeks ago";
}
```

ဒီ Function က ရက်စွဲနှစ်ခုလက်ခံပါတယ်။ လက်ရှိအချိန် now နဲ့ နှိုင်းယှဉ်လိုတဲ့အချိန် time တို့ပဲဖြစ်ပါတယ်။ ရက်စွဲ နှစ်ခုကို ပေးလိုက်ချိန်မှာ လိုချင်တဲ့ရလဒ်ကို မှန်မှန်ကန်ကန် ပြန်ပေးနိုင်ရဲ့လား စမ်းသပ်ပေးတဲ့ Test Function တစ်ခုကို အခုလို ရေးသားနိုင်ပါတယ်။

JavaScript

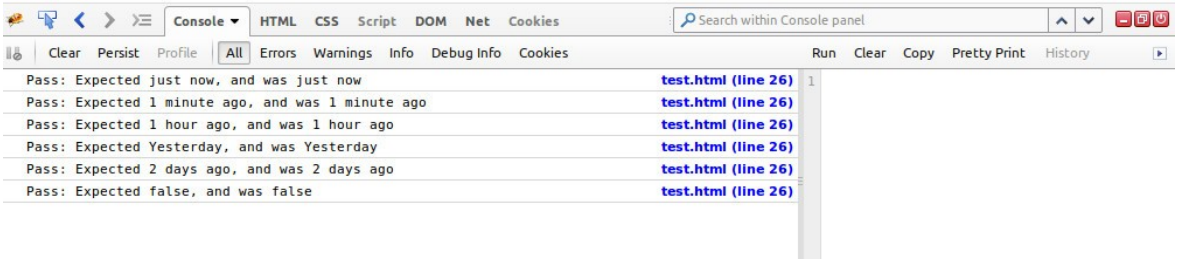
```
function prettyDateTest(then, expected) {
  var result = prettyDate("2014/09/10 22:25:00", then);
    if (result !== expected) {
      console.log("Fail: Expected " + expected + ", but was " + result);
  } else {
    console.log("Pass: Expected " + expected + ", and was " + result);
  }
}
```

`prettyDate()` Function ကပြန်ပေးတဲ့တန်ဖိုးနဲ့ ကိုက်ညီရမယ်လို့ သတ်မှတ်ထားတဲ့ တန်ဖိုးကို တိုက်ဆိုင်စစ်ဆေး ပေးခြင်းဖြစ်ပါတယ်။ ဒါကြောင့် အောက်ပါအတိုင်း ရေးသားစမ်းသပ်နိုင်ပြီဖြစ်ပါတယ်။

JavaScript

```
prettyDateTest("2014/09/10 22:24:30", "just now");
prettyDateTest("2014/09/10 22:23:30", "1 minute ago");
prettyDateTest("2014/09/10 21:23:30", "1 hour ago");
prettyDateTest("2014/09/09 22:23:30", "Yesterday");
prettyDateTest("2014/09/08 22:23:30", "2 days ago");
prettyDateTest("2013/09/08 22:23:30", false);
```

`prettyDateTest()` Function ရဲ့ ပထမ Parameter အနေနဲ့ လက်ရှိအချိန်ကိုပေးပြီး၊ ရလဒ်အနေနဲ့ "just now" ရရှိရမယ်လို့ စစ်ထားပါတယ်။ လွန်ခဲ့တဲ့ တစ်မိနစ်ကအချိန်ကိုပေးလိုက်ရင်တော့ "1 minute ago" ကို ရရှိရမယ်လို့ စစ်ထားပါတယ်။ ဒီနည်းနဲ့ ထောင့်စိအောင်၊ မဖြစ်နိုင်တဲ့ ရက်စွဲတွေအပါအဝင် ရက်စွဲအမျိုးမျိုးနဲ့ အချိန်အမျိုးမျိုးကို ထည့်သွင်းစမ်းသပ်နိုင်ပါတယ်။ ရရှိလာမယ့်ရလဒ်နမူနာကို **ပုံ (၂.၂)** မှာ ဖော်ပြထားပါတယ်။



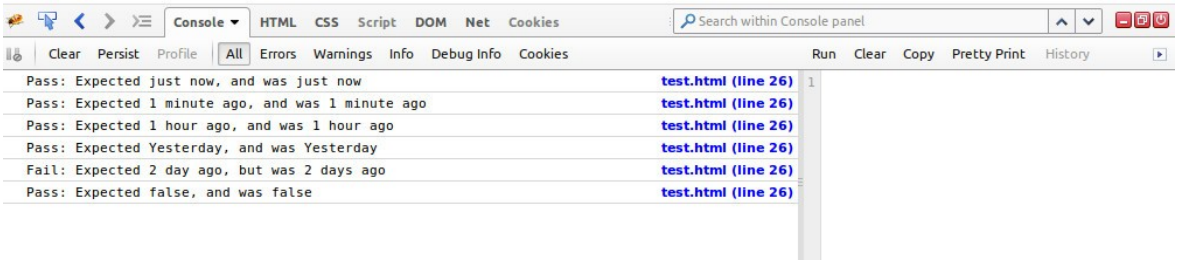
ပုံ (၂.၂) – Unit Test Function: All Pass

ပေးလိုက်တဲ့ ရက်စွဲအားလုံးအတွက် မျှော်မှန်းထားတဲ့ရလဒ်အတိုင်း အားလုံးမှန်မှန်ကန်ကန် ရရှိနေကြောင်းကို ဒီနည်းနဲ့ သိနိုင်ပါတယ်။ အမှားရှိတဲ့အခါ ဘယ်လိုဖော်ပြမလဲ သိလိုရင်အခုလို မျှော်မှန်းရလဒ်အမှားတစ်ခုနဲ့ စမ်းသပ်ကြည့် နိုင်ပါတယ်။

JavaScript

```
5. prettyDateTest("2014/09/08 22:23:30", "2 day ago");
```

မျှော်မှန်းရလဒ်က **2 day ago** ဖြစ်ပေမယ့် `prettyDate()` Function ကပြန်ပေးတဲ့ တစ်ကယ့်ရလဒ်က **2 days ago** ဖြစ်နေတဲ့အတွက် ဒီတစ်ချက်မှာ မှားနေကြောင်းကို အခုလိုတွေ့ရမှာဖြစ်ပါတယ်။



ပုံ (၂.၃) - Unit Test Function: One Fail

တစ်ကယ့်လက်တွေ့မှာ Function တစ်ခုကို စမ်းဖို့အတွက် Unit Test Function တစ်ခု လိုက်ရေးနေရမယ်ဆိုရင် သိပ်အလုပ်မတွင်ပါဘူး။ နမူနာမှာ ရိုးရှင်းတဲ့လုပ်ဆောင်ချက်ကို ဖော်ပြထားလို့ပါ။ လက်တွေ့မှာ ထည့်သွင်းစဉ်းစားရမှာတွေ အများကြီးရှိပါတယ်။ ပြီးတော့ အမှန်တစ်ကယ် အလုပ်လုပ်တဲ့ Code နဲ့ Test Code တွေကို ရောမရေပဲ သီးခြားစီခွဲဖို့လဲ လိုအပ်နိုင်ပါသေးတယ်။ ဒီနေရာမှာ အသုံးဝင်လာတာကတော့ Test Framework တွေပဲဖြစ်ပါတယ်။

2.4 – Test Frameworks

Test Framework တွေက Test Function တွေ တစ်ခုချင်း ကိုယ်တိုင်လိုက်ရေးနေစရာမလိုပဲ Unit Test လုပ်နိုင်စေမယ့် လုပ်ဆောင်ချက်တွေ ပါဝင်လေ့ရှိပါတယ်။ အခြေအနေအမျိုးမျိုး၊ Condition အမျိုးမျိုးအတွက် Test လုပ်လို့ရစေမယ့် လုပ်ဆောင်ချက်တွေ ပါဝင်လေ့ရှိပါတယ်။ Test Result ကို စနစ်တစ်ကျ စုစည်းဖော်ပြပေးတဲ့ Result Formatter တွေ ပါဝင်လေ့ရှိကြပါတယ်။

QUnit လို့ခေါ်တဲ့ JavaScript Unit Testing Framework တစ်ခုကို နမူနာအနေနဲ့ဖော်ပြပေးပါမယ်။ Framework သုံးနည်းကို ဖော်ပြလိုတာမဟုတ်ပါဘူး။ Test Framework တွေရဲ့သဘာဝကို သိရှိသွားစေဖို့က အဓိကရည်ရွယ်ချက် ဖြစ်တဲ့အတွက် သဘောသဘာဝပိုင်းကို ဦးစားပေးလေ့လာစေလိုပါတယ်။ လက်တွေ့မှာ အသုံးပြုတဲ့ Language နဲ့ Project အမျိုးအစား ပေါ်မူတည်ပြီး အသုံးပြုရမယ့် Test Framework လည်းကွဲပြားသွားမှာပဲဖြစ်ပါတယ်။

QUnit ကို jQuery ဖန်တီးရှင်ဖြစ်သူ John Resig ကဖန်တီးထားခြင်းဖြစ်ပါတယ်။ jQuery, jQueryUI နဲ့ အခြား JavaScript Project အများအပြားအတွက် Test Framework အနေနဲ့အသုံးပြု ထားပါတယ်။ QUnit ကို jQuery CDN ကနေ Download ရယူနိုင်ပါတယ်။

လက်ရှိဒီစာရေးသားနေစဉ်မှာ နောက်ဆုံးထွက်ရှိထားတဲ့ Version ကတော့ QUnit 1.15.0 ဖြစ်ပါတယ်။ စတင်အသုံးပြုနိုင် ဖို့ အတွက် JavaScript Source Code သာမက Test Result ကို Format လုပ် ဖော်ပြရာမှာအသုံးပြုတဲ့ CSS Style ကိုပါ ရယူဖို့လိုအပ်ပါတယ်။ တိုက်ရိုက်ရယူနိုင်မယ့် Link တွေကို ဖော်ပြပေးလိုက်ပါတယ်

- http://code.jquery.com/qunit/qunit-1.15.0.js
- http://code.jquery.com/qunit/qunit-1.15.0.css

လိုအပ်တဲ့ဖိုင်တွေရယူပြီးရင် စတင်စမ်းသပ်နိုင်ဖို့အတွက် HTML Document တစ်ခုတည်ဆောက်ပြီး QUnit JavaScript File နဲ့ CSS Style File တို့ကို အခုလိုချိတ်ဆက်ပေးရပါတယ်။

HTML

```
 1. <!DOCTYPE html>
 2. <html>
 3. <head>
 4.     <meta charset="utf-8">
 5.     <title>QUnit Test</title>
 6.     <link rel="stylesheet" href="css/qunit.css">
 7. </head>
 8. <body>
 9.     <div id="qunit"></div>
10.
11.     <script src="js/qunit.js"></script>
12.     <script src="js/app.js"></script>
13.     <script src="js/tests.js"></script>
14. </body>
15. </html>
```

နမူနာရဲ့ လိုင်းနံပါတ် (၉) မှာ `<div>` Element တစ်ခုထည့်သွင်းထားပြီး `id` ကို `qunit` လို့ပေးထားပါတယ်။ QUnit က Test Result တွေကို အဲ့ဒီ `<div>` ထဲမှာ ဖော်ပြပေးမှာဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၆) မှာ CSS Style ဖိုင်ကို ချိတ်ဆက်ထားပြီး လိုင်းနံပါတ် (၁၁) မှာ QUnit JavaScript ဖိုင်ကိုချိတ်ဆက်ထားပါတယ်။ QUnit ရဲ့ အကူအညီနဲ့ Test Code တွေကို မူရင်း Code နဲ့ ရောထားစရာမလိုပဲ သီးခြားခွဲပြီး ရေးသားနိုင်မှာဖြစ်ပါတယ်။ ဒါကြောင့် လိုင်းနံပါတ် (၁၂) မှာ ချိတ်ဆက်ထားတဲ့ `app.js` ထဲမှာ လက်တွေ့အလုပ်လုပ်မယ့် Code တွေကို ရေးသားမှာဖြစ်ပြီး၊ လိုင်းနံပါတ် (၁၃) မှာ ချိတ်ဆက်ထားတဲ့ `tests.js` မှာတော့ Test Code တွေကို ခွဲခြားရေးသားမှာဖြစ်တယ်လို့ မှတ်ယူပေးပါ။

စောစောကနမူနာအဖြစ် ဖော်ပြခဲ့တဲ့ `prettyDate()` Function ကိုပဲ ဆက်လက်စမ်းသပ်ကြည့်ပါမယ်။ Unit Test လုပ်ဖို့အတွက် `prettyDateTest()` Function မလိုတော့ပါဘူး။ QUnit အကူအညီနဲ့ Test Code တွေကို ရေးသွားတော့မှာပါ။

QUnit ရဲ့ `test()` Function ကိုသုံးပြီး Test Code တွေကိုစတင်ရေးသားနိုင်ပါတယ်။ `test()` Function အတွက် Parameter နှစ်ခုပေးရပါတယ်။ ပထမတစ်ခုက Test Name ဖြစ်ပြီး ဒုတိယတစ်ခုကတော့ Test Case တွေပါဝင်တဲ့ Function ဖြစ်ပါတယ်။ QUnit မှာ `ok()`, `equal()`, `deepEqual()`, `notEqual()`,

`notDeepEqual()` စသဖြင့် မျှော်မှန်းရလဒ်နဲ့ လက်တွေ့ရလဒ်ကို တိုက်စစ်လို့ရတဲ့ Function အမျိုးမျိုးရှိပါတယ်။ `ok()` ကတော့ ရလာတဲ့ရလဒ်က `true` ဟုတ်မဟုတ် စစ်ပေးပါတယ်။ `true` ဆိုရင် Test Pass ဖြစ်ပြီး ကျန်ရလဒ် တွေဆိုရင်တော့ Test Fail ဖြစ်မှာပဲဖြစ်ပါတယ်။ `equal()` နဲ့ `notEqual()` Function တွေကတော့ ရလာတဲ့ တန်ဖိုးဟာ မျှော်မှန်းထားတဲ့တန်ဖိုးနဲ့ ညီမညီကိုစစ်ပေးပါတယ်။ `deepEqual()` နဲ့ `notDeepEqual()` တို့ကတော့ အခြေခံအားဖြင့် `equal()`, `notEqual()` တို့နဲ့အတူတူပါပဲ။ ဒါပေမယ့် နှိုင်းယှဉ်လိုတဲ့ရလဒ်က Object တစ်ခု ဖြစ်မယ်ဆိုရင်တော့ `deepEqual()` သို့မဟုတ် `notDeepEqual()` ကို အသုံးပြုသင့်ပါတယ်။ အခြား Function တွေရှိသေးပေမယ့် အခြေခံအားဖြင့် ဒီလောက်ဆိုရင် Test Code တွေကို QUnit အကူအညီနဲ့ စတင်ရေးသားနိုင်ပြီဖြစ်ပါ တယ်။ `tests.js` ထဲမှာ အခုလိုရေးသားစမ်းသပ်ကြည့်နိုင်ပါတယ်။

JavaScript

```
1. QUnit.test("Prettydate Basics", function( assert ) {
2.   var now = "2014/09/10 22:25:00";
3.   assert.equal(prettyDate(now, "2014/09/10 22:24:30"), "just now");
4.   assert.equal(prettyDate(now, "2014/09/10 22:23:30"), "1 minute ago");
5.   assert.equal(prettyDate(now, "2014/09/10 21:23:30"), "1 hour ago");
6.   assert.equal(prettyDate(now, "2014/09/09 22:23:30"), "Yesterday");
7.   assert.equal(prettyDate(now, "2014/09/08 22:23:30"), "2 day ago");
8.   assert.equal(prettyDate(now, "2013/09/08 22:23:30"), false);
9. });
```

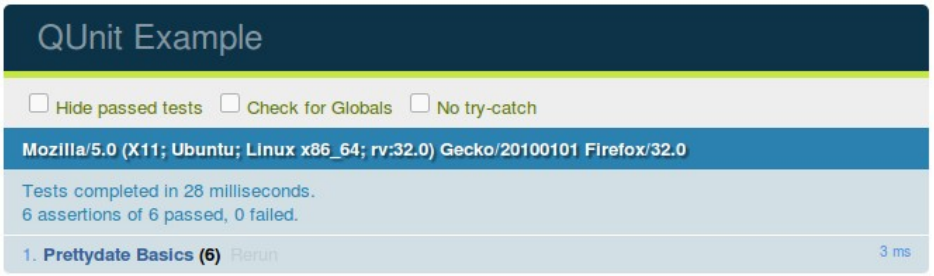
နမူနာမှာ `equal()` ကိုသုံးပြီး `prettyDate()` Function ကို ခေါ်စဉ်မှာ ရရှိလာမယ့် လက်တွေ့ရလဒ်ဟာ မျှော်မှန်း ရလဒ်နဲ့ တူညီခြင်းရှိမရှိစီစစ်ထားပါတယ်။ လက်တွေ့ရလဒ်က မျှော်မှန်းထားတာနဲ့မကိုက်ညီပဲ အမှားရှိနေခဲ့ရင် QUnit က အခုလိုဖော်ပြပေးမှာဖြစ်ပါတယ်။



ပုံ (၂.၄) - QUnit - Test Result - One Failed

ခြောက်ကြိမ်စမ်းသပ်မှုမှာ တစ်ကြိမ် Fail ဖြစ်နေကြောင်းဖော်ပြတဲ့အပြင် Fail ဖြစ်ပုံဖြစ်နည်းအသေးစိတ်ကိုပါ တွဲဖက် ဖော်ပြထားပါတယ်။ **2 day ago** ဆိုတဲ့ရလဒ်ကို မျှော်လင့်ထားပေမယ့် တစ်ကယ့်ရလဒ်က **2 days ago** ဖြစ်နေကြောင်း ကို တွဲဖက်ဖော်ပြနေခြင်းဖြစ်ပါတယ်။

အကယ်၍ Test တွေအားလုံး Pass ဖြစ်တယ်ဆိုရင်တော့ **ပုံ (၂.၅)** မှာဖော်ပြထားသလိုရလဒ်ကို ရရှိမှာဖြစ်ပါ တယ်။ ခြောက်ကြိမ်စီစစ်ရာမှာ ခြောက်ကြိမ်လုံး Pass ဖြစ်ပြီး Fail တစ်ကြိမ်မှမဖြစ်တဲ့အတွက် စမ်းသပ်မှု အောင်မြင်ကြောင်း ကိုဖော်ပြခြင်းဖြစ်ပါတယ်။



ပုံ (၂.၅) - QUnit - Test Result - All Pass

တစ်ခါတစ်ရံမှာ မမျှော်မှန်းနိုင်တဲ့အမှားတစ်ချို့ကြောင့် Test (၆) ကြိမ်လုပ်ခိုင်းထားပေမယ့် (၆) ကြိမ်လုံးမစမ်း ဖြစ်ပဲ ကျန်သွားတာမျိုးလည်း ဖြစ်တက်ပါတယ်။ အဲ့ဒီလိုကျန်ခဲ့ရင်လည်း Test Fail ဖြစ်ကြောင်းဖော်ပြဖို့အတွက် `expect()` Function ကို သုံးနိုင်ပါတယ်။

JavaScript

```
1. QUnit.test("Prettydate Basics", function( assert ) {
2.   expect(6);
3.   var now = "2014/09/10 22:25:00";
4.   assert.equal(prettyDate(now, "2014/09/10 22:24:30"), "just now");
5.   assert.equal(prettyDate(now, "2014/09/10 22:23:30"), "1 minute ago");
6.   assert.equal(prettyDate(now, "2014/09/10 21:23:30"), "1 hour ago");
7.   assert.equal(prettyDate(now, "2014/09/09 22:23:30"), "Yesterday");
8.   assert.equal(prettyDate(now, "2014/09/08 22:23:30"), "2 days ago");
9. });
```



ပုံ (၂.၆) - QUnit - Test Result - Expect Failed

(၆) ကြိမ်စမ်းဖို့မျှော်မှန်းထားပေမယ့် (၅) ကြိမ်ပဲစမ်းဖြစ်တဲ့အတွက် Test Fail ဖြစ်ကြောင်းဖော်ပြလာခြင်းဖြစ်ပါတယ်။

Test လုပ်ဖို့အတွက် Function တစ်ခုကို QUnit ကနေခေါ်လိုက်တဲ့အခါ အဲ့ဒီ Function က တန်ဖိုးတစ်ခု ပြန်ပေးယုံ မဟုတ်ပဲ တစ်ခြားလုပ်ဆောင်ချက်တစ်ခုကို ဆောင်ရွက်သွားတဲ့အခါ Side-Effect တွေရှိတက်ပါတယ်။ ဥပမာ - Database ထဲကို Record တစ်ခုထည့်သွင်းပေးတဲ့ Function အလုပ်လုပ်မလုပ် စမ်းဖို့အတွက် အဲ့ဒီ Function ကို ခေါ် လိုက်တဲ့ အခါ Record ကိုလည်း Database ထဲမှာ ထည့်သွင်းသွားမှာမို့ Side-Effect ရှိနေပါတယ်။ ကျွန်တော်တို့က Record ထည့်ချင်တာမဟုတ်ပါဘူး။ Record ထည့်တဲ့အလုပ် တစ်ကယ်လုပ်မလုပ်ကိုပဲ Test လုပ်ချင်တာပါ။

Side-Effect တွေကို စီမံဖို့အတွက် QUnit မှာ Module, Setup နဲ့ Teardown ဆိုတဲ့လုပ်ဆောင်ချက်တွေ ပါဝင်ပါတယ်။

QUnit မှာအမျိုးတူရာ Test Code တွေကို စုစည်းဖို့အတွက် Module တွေသတ်မှတ်လို့ရပါတယ်။ `QUnit.module()` ကြေငြာချက်တိုင်းရဲ့ အောက်မှာဆက်လက်ရေးသားထားတဲ့ Test Code တွေကို သက်ဆိုင်ရာ Module Name အလိုက် စုစည်း အလုပ်လုပ်ပေးသွားမှာဖြစ် ပါတယ်။ ဥပမာ –

JavaScript

```
QUnit.module( "Module A" );
QUnit.test("Test One", function( assert ) {
  //
});
QUnit.test("Test Two", function( assert ) {
  //
});

QUnit.module( "Module B" );
QUnit.test("Test Three", function( assert ) {
  //
});
QUnit.test("Test Four", function( assert ) {
  //
});
```

ဒီနည်းနဲ့ Test One နဲ့ Test Two တို့ကို Module A အတွင်းမှာစုစည်းပေးသွားမှာဖြစ်ပြီး Test Three နဲ့ Test Four တို့ကိုတော့ Module B အတွင်းမှာ စုစည်းပေးသွားမှာဖြစ်ပါတယ်။ Module ကြေငြာသတ်မှတ်ရာမှာ Setup နဲ့ Teardown တို့ကိုလည်း တစ်ခါတည်း သတ်မှတ်နိုင်ပါတယ်။ `setup` မှာ သတ်မှတ်ထားတဲ့ လုပ်ဆောင်ချက်တွေကို Test မစတင်မီ QUnit က ဆောင်ရွက်ပေးသွားမှာဖြစ်ပါတယ်။ `teardown` မှာ သတ်မှတ်ထားတဲ့ လုပ်ဆောင်ချက်တွေကို တော့ Test တွေအားလုံးပြီး သွားချိန်မှာ ဆောင်ရွက်ပေးသွားမှာဖြစ်ပါတယ်။ ဥပမာ - Database နဲ့ချိတ်ဆက်ပြီး Record တွေပယ်ဖျက်ပေးတဲ့ Function ကို စမ်းသပ်လိုတယ်ဆိုပါစို့။

Pseudo-code

```
QUnit.module( "Module A", {
  setup: function() {
    records.add(98, dummy);
    records.add(99, dummy);
  },
  teardown: function() {
    records.reset();
  }
});

QUnit.test("Database Test", function( assert ) {
  var total = records.count();
  assert.equal(record.remove([98, 99]).count(), total - 2);
});
```

ပေးထားတဲ့နမူနာအရ `setup` မှာ Database ထဲကို Dummy Data တွေနဲ့ Record နှစ်ခုထည့်သွင်းလိုက်ပါတယ်။ ဒါကြောင့် Test လုပ်ချိန်မှာ တစ်ကယ့် Record ကိုဖျက်ပြီး စမ်းဖို့မလိုတော့ပဲ အဲ့ဒီ Dummy Record တွေကို Remove လုပ်ပြီး စမ်းကြည့်နိုင်သွားပါတယ်။ Database ထဲက အခြား Record တွေကို ဒီ Test ကြောင့် သက်ရောက်မှု မရှိစေ တော့ပါဘူး။ `teardown` မှာတော့ Auto-ID တွေကို Populate ပြန်လုပ်စေပါတယ်။ ဒါကြောင့် Test လုပ်လိုက်လို့ Auto-ID တွေ လွဲသွားခဲ့ရင်လည်း ပြန်မှန်သွားစေမှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Test Code ကြောင့် ဖြစ်ပေါ်လာနိုင်တဲ့ မလို လားအပ်တဲ့ Side-Effect တွေကို ရှောင်ရှားနိုင်မှာ ဖြစ်ပါတယ်။

QUnit ရဲ့ ကျန် လုပ်ဆောင်ချက်တွေကို qunitjs.com မှာ ဆက်လက်လေ့လာနိုင်ပါတယ်။

2.5 – Test Driven Development

TDD လို့ခေါ်တဲ့ Test Driven Development ကတော့ Code ရေးသားနည် Concept တစ်ခုဖြစ်ပါတယ်။ လက်တွေ့ အလုပ်လုပ်မယ့် Code မရေးခင် Test Code ကို ဦးဆုံးရေးရတဲ့ Practice တစ်မျိုးပါ။ တစ်နည်းအားဖြင့် Test Code ကို Design အဖြစ်သော်လည်ကောင်း၊ Function Model အဖြစ်သော်လည်ကောင်း အသုံးချခြင်းလည်းဖြစ်ပါတယ်။ ဥပမာ - တည်နေရာ စာရင်းကို ပေးလိုက်ရင်၊ အကွာအဝေးအလိုက်စီပြီး ပြန်ပေးတဲ့ ရလဒ်ကို လိုချင်တယ်ဆိုပါစို့။ အဲ့ဒီ ရလဒ် ရဖို့အတွက် Code ကို အရင်မရေးပါဘူး။ Test Code ကိုအရင် အခုလိုရေးရမှာဖြစ်ပါတယ်။

JavaScript

```
QUnit.test("Sort by distance test", function( assert ) {
  var list = {"A": 12, "B": 32, "C": 8};
  assert.deepEqual(sortByDistance(list), [
    ["C", 8], ["A", 12], ["B", 32]
  ]);
});
```

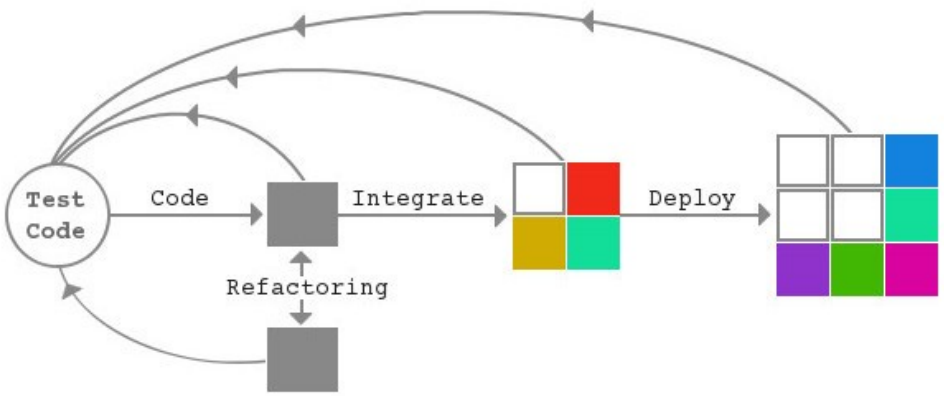
ဒီ Test ကို စမ်းကြည့်လိုက်ရင် Fail ဖြစ်မှာဖြစ်ပါတယ်။ `sortByDistance()` Function က ရေးတောင် မရေးရသေးတဲ့အတွက် ဖြစ်ပါတယ်။ တစ်ကယ့်လုပ်ဆောင်ချက် Code မရေးရသေးခင်ကတည်းက လိုချင်တဲ့ ရလဒ်ကို အရင်သတ် မှတ်လိုက်ခြင်းဖြစ်ပါတယ်။ Test Code ရပြီဆိုတော့မှ တစ်ကယ့်လုပ်ဆောင်ချက်အတွက် Code ကို ဆက်လက်ရေးသားရမှာ ဖြစ်ပါတယ်။

JavaScript

```
function sortByDistance(list) {
  var result = [];
  for(x in list){
    result.push([x, list[x]])
  }
  result.sort(function(a,b){ return a[1] - b[1] });
  return result.reverse();
}
```

Function ရေးပြီးတဲ့အခါ Test Code ကို ပြန်စမ်းကြည့်ရမှာဖြစ်ပါတယ်။ Fail ဖြစ်နေသေးရင် Function ကို လိုအပ် သလိုပြင်ပြီး Test Code ကိုနောက်တစ်ကြိမ် ထပ်စမ်းရပါမယ်။ ဒီနည်းနဲ့ Test Pass မဖြစ်မချင်း Function ကို ပြုပြင်သွားရမှာဖြစ်ပါတယ်။ Test Pass ဖြစ်သွားပေမယ့် ပြုပြင်မှုတစ်ခုကြောင့် နောင်မှာ ဒီ Function က မျှော်မှန်းသလို အလုပ်မလုပ်တော့တာမျိုး ဖြစ်တက်ပါတယ်။ ဥပမာ - တစ်နေရာမှာ `sort()` Function ကိုပြင်လိုက်တဲ့အတွက် `sortByDistance()` ရဲ့ အလုပ်လုပ်ပုံ ပြောင်းသွားနိုင်ပါတယ်။ ဒါပေမယ့် ကြိုတင်ရေးသားထားတဲ့ Test Code ရှိတဲ့ အတွက်ကြောင့် ဒီအမှားမျိုးရှိလာတဲ့အခါ ချက်ခြင်းသိရမှာဖြစ်ပါတယ်။ Function အားလုံးအတွက်သာ TDD Approach အတိုင်း Test Code တွေ ဦးစွာရေးသားထားခဲ့ရင် တစ်

နေရာမှာပြင်လိုက်လို့ နောက်တစ်နေရာမှာ သွားမှားနေတာကို မသိလိုက်ပဲကျန်သွားတယ်ဆိုတာမျိုး မဖြစ်နိုင်တော့ပါဘူး။ မျှော်မှန်းရလဒ်လွဲသွားတာနဲ့ Test Fail ဖြစ်တော့မှာပါ။ ဒီနည်းနဲ့ Code Quality လည်း တိုးတက်လာမှာပဲဖြစ်ပါတယ်။



ပုံ (၂.၇) - TDD Workflow

ပုံ (၂.၇) ကိုလေ့လာကြည့်ပါ။ အရင်ဆုံး မျှော်မှန်းရလဒ်ပါဝင်တဲ့ Test Code ကိုရေးရပါတယ်။ Function ရေးပြီးတဲ့ အခါ Test Code ကို Run ပြီး မျှော်မှန်းရလဒ်နဲ့ လက်တွေ့ရလဒ် ကိုက်ညီမှုရှိမရှိ စစ်ရပါတယ်။ Refactoring လုပ်ခဲ့ရင်လည်း Test Code ကို ပြန် Run ခြင်းအားဖြင့် မူလမျှော်မှန်းရလဒ်အတိုင်း ဆက်လက်ရရှိနေကြောင်း သေချာအောင်စစ် ရပါတယ်။ အဲ့ဒီ Function ကို တစ်ခြား Function တွေနဲ့ ပေါင်းစပ်ပြီးတဲ့အခါ Test Code ကို ထပ် Run ရပါတယ်။ အားလုံးအဆင်ပြေလို့ ဒီ Function Unit ပါဝင်တဲ့ Module ကို Deploy လုပ်လိုက်တဲ့အခါမှာလည်း Test Code ကို ထပ် Run ပေးရပါတယ်။ ဒီနည်းနဲ့ ပြင်ဆင်မှုတွေ ဘယ်လိုပဲလုပ်လုပ် Function ရဲ့ အလုပ်လုပ်ပုံဟာ မူလမျှော်မှန်းထားတဲ့ အတိုင်း အလုပ်လုပ်နေတယ်ဆိုတာ သေချာစေမှာဖြစ်ပါတယ်။ ဒီလို Test Code ကို အခြေခံရေးသားရတဲ့ နည်းစနစ် ဖြစ်တဲ့အတွက် Test Driven Development လို့ ခေါ်ခြင်းဖြစ်ပါတယ်။ Test First Approach လို့လည်းခေါ်ပါတယ်။

ဒီလို Test Code ကိုအခြေခံရေးသားခြင်းအားဖြင့် ရရှိလာတဲ့ အားသာချက်က အမှားနည်းယုံတင်မကပါဘူး။ Code ရဲ့ ဖွဲ့စည်းပုံနဲ့ Maintainability ပါ တိုးတက်လာပါတယ်။ ဘာဖြစ်လို့လဲဆိုတော့ Function တစ်ခုရေးတော့မယ်ဆိုရင် အလုပ် လုပ်ပြီးရော ရေးလို့မရတော့ပါဘူး။ Test Code Run လို့ရအောင် သေချာစဉ်းစားပြီးမှ ရေးရပါတော့မယ်။ ပြီးတော့ အခြား Function တွေကို အလွန်းအမင်းမှီခိုလို့လည်း မရတော့ပါဘူး။ လုပ်စရာရှိတဲ့ အလုပ်ကို တစ်ခုတည်း သီးခြားအလုပ် လုပ်နိုင်မှာသာ Test လုပ်ရတာ ထိရောက်မှာဖြစ်ပါတယ်။ Function တစ်ခုက လုပ်ဆောင်ချက်တွေ အများအပြားကို လုပ်နေရင်လည်း အဆင်မပြေတော့ပါဘူး။ Function တစ်ခု လုပ်ဆောင်ချက်တစ်ခု ဖြစ်နေမှ Test လုပ်ရတာထိရောက်မှာပါ။ ဒါ့အပြင် Refactoring လုပ်ဖို့လည်း ကြောက်စရာမလိုတော့ပါဘူး။ တစ်ခုပြင်လိုက်လို့ သက်ရောက်မှုတွေ အခြားနေရာ တွေမှာ ရှိတယ်ဆိုရင်လည်း၊ ချက်ခြင်းသိရမှာမို့ ဒီအကျိုး ဆက်အမှားတွေ Code ထဲမှာ ကျန်နေတာမျိုး မဖြစ်တော့ပါဘူး။ များများ Refactoring လုပ်နိုင်လေ Code တွေကလည်းပိုပြီးရှင်းလင်း နားလည်ရလွယ်ကူလေဖြစ်မှာပါ။

ပြောရမယ်ဆိုရင် TDD Approach ဟာ ကျွန်တော်တို့ Developer တွေ၊ ကိုယ်ရေးထားတဲ့ Code အပေါ် ယုံကြည်မှု ပိုမို တိုးတက်လာအောင် အကူအညီပေးမှာဖြစ်ပါတယ်။ တစ်နည်းအားဖြင့် Project ပြဿနာတိုင်း ဒူးတုန်စရာ မလိုတော့အောင် အကူအညီပေးမှာပဲဖြစ်ပါတယ်။

Conclusion

Ruby on Rails လို့ခေါ်တဲ့ အလွန်ထင်ရှားတဲ့ Web Application Framework ကို တီထွင်ခဲ့သူ David Heinemeier Hansson က (၂၀၁၄) ခုနှစ် ဧပြီလထဲမှာ "TDD is dead. Long live testing." ဆိုတဲ့ ခေါင်းစဉ်နဲ့ ဆောင်းပါးတစ်ပုဒ် ရေးခဲ့တဲ့အတွက် Developer တွေအကြား အတော်လေးလှုပ်ခတ်သွားခဲ့ပါသေးတယ်။ အမှန်တော့ Ruby on Rails ဟာ TDD ကို အားပေးတဲ့ Framework တစ်ခုဖြစ်ပါတယ်။ ဒါပေမယ့် တစ်ချို့လူတွေက TDD ကို ကိုးကွယ်ရာတစ်ခုလောက် နီးနီး သဘောထားပြီး ဒီနည်းမှမသုံးရင် Developer ကောင်းမဟုတ်တော့သလို ပြောကြတာကို အတော်မြင်ပြင်းကတ်နေပုံ ရပါတယ်။ DHH တို့လို အလွန်တော်တဲ့ Programmer တွေကတော့ ပြဿနာမရှိပါဘူး။ Unit Test တွေ မပါပဲနဲ့လည်း အမှားနည်းပြီး Maintainable ဖြစ်တဲ့ Quality Code တွေကို ရေးနိုင်ကြပါတယ်။ လူတိုင်းတော့ ဒီလိုရေးနိုင်မှာမဟုတ် ပါဘူး။ ဒါကြောင့် ဒီလိုဝေဖန်ထောက်ပြ ချက်တစ်ချို့ရှိနေပေမယ့် Test Driven Approach ဟာ Quality Code တွေ ရေးဖို့ များစွာအထောက်အကူပြုတယ်ဆိုတဲ့ အခြေခံအချက်ကို အသိအမှတ်ပြုရမှာပဲဖြစ်ပါတယ်။

တစ်ဘက်ကနားလည်ရမှာကတော့၊ TDD ကြောင့် Code အရည်အသွေးဟာ နှေ့ခြင်းညခြင်း အလိုအလျောက် တိုးတက် သွားမှာ မဟုတ်ပါဘူး။ အသုံးချသူ Developer က ဘယ်လောက်ထိ ထိရောက်အောင်အသုံးချနိုင်သလဲ ဆိုတဲ့ပေါ်မှာ အများကြီးမူတည်ပါသေးတယ်။ TDD စာအုပ်တွေ၊ Online Article တွေမှာ TDD Best Practice ဆိုပြီး သတိပြုလိုက်နာ သင့်တဲ့ အချက်တွေကိုဖော်ပြထားတာတွေ အများကြီးရှိပါတယ်။ ဆက်လက်လေ့လာပြီး သင့်တော်သလိုအသုံးချသွားသင့် ပါတယ်။

Unit Test အပြင် အခြား Test Method တွေကိုလည်း TDD ပုံစံ ရေးသားမှုမှာ ထည့်သွင်းအသုံးပြုနိုင်ပါတယ်။ ဒါပေမယ့် ဒီ Test Method တွေကို မသုံးနိုင်ရင်တောင်မှ Unit Test သက်သက်နဲ့တင် အတော်ထိရောက်နိုင်ပါတယ်။ Unit Test Code တွေ ရေးရတာ လက်တွေ့မှာ မခက်ခဲပါဘူး။ ဒီလိုလွယ်ကူတဲ့ နည်းစနစ်တစ်ခုကြောင့် Code Quality ကို များစွာတိုးတက်စေမှာဖြစ်လို့ TDD ဟာ Rockstar Developer တွေ မဖြစ်မနေသိရှိအသုံးချရမယ့် နည်းစနစ်တစ်ခုပဲ ဖြစ်ပါတယ်။

ပြုပြင်ထိမ်းသိမ်းရလွယ်တဲ့ Code တွေ ရေးသားနိုင်ဖို့အတွက်
အများစံထားလိုက်နာလေ့ရှိတဲ့ Code Design Principle တွေကို
သိရှိလိုက်နာဖို့ လိုအပ်ပါတယ်။

Rockstar Developer Course

Project Management, Web Service, Server Architecture
NodeJS အစရှိသည် ဤစာအုပ်ပါ အကြောင်းအရာများကို
စာရေးသူကိုယ်တိုင် သင်ကြားပေးခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၃) – SOLID Design Principles

ပြီးခဲ့တဲ့အခန်းမှာ Test Driven Development ရဲ့ အကူအညီနဲ့ အမှားနည်းတဲ့ Quality Code တွေရေးသားနိုင်ပုံကို လေ့လာခဲ့ပါတယ်။ ဒီအခန်းမှာတော့ ပြုပြင်ထိမ်းသိမ်းရလွယ်တဲ့ Maintainable Code တွေရေးသားရာမှာ အထောက်အကူ ဖြစ်စေမယ့် နည်းစနစ်များဖြစ်တဲ့ SOLID Design Principle အကြောင်းကို ဖော်ပြသွားမှာပဲဖြစ်ပါတယ်။ SOLID ဆိုတာ ဟာ –

1. Single Responsibility Principle
2. Open/Close Principle
3. Liskov Substitution Principle
4. Interface Segregation Principle
5. Dependency Inversion Principle

– တို့ရဲ့ အတိုကောက်ဖြစ်ပါတယ်။

Object-Oriented Programming နဲ့အတူ တွဲဖက်အသုံးပြုလေ့ရှိတဲ့ Object-Oriented Design Principle တွေဖြစ်ပါတယ်။

3.1 – Object-Oriented Programming

Object-Oriented Programming ဟာ Syntax အရခက်ခဲလှတဲ့ နည်းစနစ်မဟုတ်ပေမယ့် Concept အရ နက်နဲတဲ့ ဘာသာရပ်ဖြစ်ပါတယ်။ ပါဝင်တဲ့လုပ်ဆောင်ချက်တွေ များပြားပြီး ရှုပ်ထွေးကျယ်ပြန့်တဲ့ Software Product တွေတည် ဆောက်ရာမှာ Extensible ဖြစ်အောင်နဲ့ ပိုပြီးတော့ Maintainable ဖြစ်အောင် ဖွဲ့စည်းရေးသားနိုင်ဖို့ဆိုတဲ့ ရည်ရွယ်ချက်နဲ့ ပေါ်ပေါက်လာခြင်း ဖြစ်ပါတယ်။ ပိုပြီးအခြေခံကျတဲ့ Structured Programming (သို့မဟုတ်) Imperative Programming တို့မှာ ပရိုဂရမ်တစ်ခုဆိုတာဟာ ညွှန်ကြားချက်တွေနဲ့အတူ တွဲဖက်အသုံးပြုရမယ့် အချက်အလက် Data တွေကို စုစည်း ရေးသားထားတဲ့ Instruction Set ကြီးတစ်ခုသာဖြစ်ပါတယ်။ Object-oriented Programming မှာတော့ ညွှန်ကြားချက် နဲ့ Data တွေကို ပြန်လည်အသုံးပြုနိုင်တဲ့ Instruction Set ငယ်များ (သို့မဟုတ်) Object များအဖြစ် ခွဲခြားစုစည်းပြီး၊ အဲ့ဒီ Object များ အပြန်အလှန် ဆက်သွယ်စေခြင်း တွဲဖက်လုပ်ငန်းဆောင်ရွက်စေခြင်းအားဖြင့် ပရိုဂရမ်ကို တည်ဆောက် ရပါတယ်။

Object-oriented Programming ရဲ့ အခြေခံသဘောတရား (၅) ခု ရှိပါတယ်။ အဲ့ဒါတွေကတော့ –

1. Object
2. Information Hiding
3. Inheritance
4. Interface
5. Polymorphism – တို့ပဲဖြစ်ပါတယ်။

Object

Object ဆိုတာဟာ အချက်အလက် Data တွေနဲ့ လုပ်ဆောင်ချက် Function တွေပေါင်းစပ်ပါဝင်တဲ့ အစိတ်အပိုင်းတစ်ခု ဖြစ်ပါတယ်။ ဒီ Object တွေကို လက်တွေ့ကမ္ဘာထဲက သက်ရှိသက်မဲ့ အရာဝတ္ထုတွေနဲ့ ခိုင်းနှိုင်းဖော်ပြလေ့ ရှိကြပါတယ်။ ဥပမာ – စက်ဝိုင်းတစ်ခုဟာ Object တစ်ခုဖြစ်ပါတယ်။ သူ့မှာ အချင်း၊ အချင်းဝက်ဆိုတဲ့ အချက်အလက်တွေ ပါဝင်ပါတယ်။ ငှက်တစ်ကောင်ဟာလည်း Object တစ်ခုဖြစ်ပါတယ်။ သူ့မှာလဲ တောင်ပံနှစ်ခု၊ ခြေထောက်နှစ်ချောင်း စတဲ့အချက် အလက်တွေ ပါဝင်ပါတယ်။ ဒါ့အပြင် ပျံသန်းနိုင်ခြင်း၊ အသံပြုနိုင်ခြင်းတို့လို Function တွေလည်း ပါဝင်နိုင်ပါတယ်။ စက်ဝိုင်းမှာ ရိုးရိုးစက်ဝိုင်းနဲ့ ဘဲဥပုံစက်ဝိုင်းဆိုပြီး အခြေခံတူညီပေမယ့် အသေးစိတ်ကွဲပြားတဲ့ မျိုးကွဲတွေ ရှိနိုင်ပါတယ်။ ငှက်မှာလည်း ပျံနိုင်တဲ့ငှက်၊ မပျံနိုင်တဲ့ငှက်ဆိုပြီး အခြေခံ Data နဲ့ Function တူပေမယ့် အသေးစိတ်ကွဲပြားတဲ့ မျိုးကွဲတွေ ရှိနိုင်ပါတယ်။ ဒါကို Object Type နဲ့ Subtype လို့ခေါ်ပြီး Object Type နဲ့ Subtype တွေ ဆက်စပ်ပုံကိုတော့ Object Hierarchy လို့ ခေါ်ပါတယ်။

Object နဲ့ Object Hierarchy တွေ တည်ဆောက် ရရှိနိုင်ဖို့အတွက် Programming Language အတော်များများက Class တွေကို အသုံးပြုကြပါတယ်။ Class ဆိုတာဟာ Object တစ်ခုတည်ဆောက်လိုက်တဲ့အခါ အဲ့ဒီ Object မှာ ပါဝင်ရမယ့် Data နဲ့ Function တို့ကို သတ်မှတ်ထားတဲ့ Specification တစ်ခုပဲဖြစ်ပါတယ်။

> ဒီအခန်းမှာဖော်ပြတဲ့ Code နမူနာအများစုဟာ ဆိုလိုရင်းသဘောသဘာဝကို ပေါ်လွင်အောင် ဥပမာပေး တဲ့ Pseudo-Code တွေဖြစ်ပါတယ်။ လက်တွေ့ကူးယူစမ်းသပ်ဖို့မဟုတ်ပဲ ဆိုလိုရင်းသဘော သဘာဝကို နားလည်အောင် လေ့လာဖို့ဖြစ်ပါတယ်။

ဥပမာ – Class တစ်ခုက အခုလို သတ်မှတ်ထားတယ် ဆိုကြပါစို့ –

Pseudo-code

```
class Square
{
    var width = 5;
    var height = 5;

    function setSize(size) {
        width = size;
        height = size;
    }

    function getArea() {
        return width * height;
    }
}
```

ဒီ Class ကိုအသုံးပြုပြီး Object တစ်ခုတည်ဆောက်လိုက်တိုင်း၊ အဲ့ဒီ Object မှာ width နဲ့ height ဆိုတဲ့ Data တန်ဖိုးနှစ်ခု ပါဝင်သွားမှာဖြစ်ပြီး `setSize()` နဲ့ `getArea()` ဆိုတဲ့ Function နှစ်ခုလည်း ပါဝင်သွားမှာဖြစ်ပါ တယ်။ Object တစ်ခုမှာ ပါဝင်တဲ့ Data ကို Object Property လို့ ခေါ်ပြီး Function ကိုတော့ Object Method လို့လည်းခေါ်ကြပါတယ်။

Object တွေ တည်ဆောက်ဖို့အတွက် Language အားလုံးနီးပါးက `new` Keyword ကို အသုံးပြုလေ့ ရှိကြပါ တယ်။ ပြီးခဲ့တဲ့ နမူနာမှာပေးထားတဲ့ Class ကိုအသုံးပြုပြီး Object တွေကို အခုလိုတည်ဆောက်နိုင်မှာဖြစ်ပါ တယ်။

Pseudo-code

```
var block = new Square();
var box = new Square();
```

ဒီတော့ `block` နဲ့ `box` ဆိုတဲ့ Square Object နှစ်ခုရရှိသွားပါတယ်။ Object နှစ်ခုလုံးမှာ ကိုယ်ပိုင် `width`, `height` တန်ဖိုးတွေ ရှိကြပါတယ်။ `block` Object ရဲ့ `width`, `height` တန်ဖိုးကို ပြောင်းလိုက်ခြင်းဟာ `box` Object ရဲ့ `width`, `height` တန်ဖိုးတွေကို သက်ရောက်စေမှာမဟုတ်ပါဘူး။ Class တစ်ခုတည်းကနေ ဖြစ်တည် လာတဲ့ Object နှစ်ခုဆိုပေမယ့် သီးခြားစီ အလုပ်လုပ်တဲ့သဘော ဖြစ်ပါတယ်။

Class တွေ Object တွေ အကြောင်းပြောပြီဆိုရင် အရေးပါလာတာကတော့ Constructor ပဲဖြစ်ပါတယ်။ Constructor က အခြေခံအားဖြင့် Object တည်ဆောက်စဉ်မှာ အလုပ်လုပ်ပေးတဲ့ Function တစ်မျိုးဖြစ်ပါ တယ်။ ပြီးခဲ့တဲ့ နမူနာ Class ကို အခုလို ပြင် ဆင်နိုင်ပါတယ်။

Pseudo-code

```
class Square
{
    var width;
    var height;

    function Square(size) {
        width = size;
        height = size;
    }

    function setSize(size) {
        width = size;
        height = size;
    }

    function getArea() {
        return width * height;
    }
}
```

Language အများစုမှာ Class Name နဲ့ အမည်တူတဲ့ Function တွေကို Constructor အဖြစ် အသုံးပြုလေ့ရှိကြပါတယ်။ နမူနာလိုင်းနံပါတ် (၆) မှာ ရေးသားထားတဲ့ `Square()` Function ဟာ ဒီ Class ကနေ Object တည်ဆောက်တိုင်းမှာ အလိုအလျောက် အလုပ်လုပ်သွားမယ့် Constructor Function ဖြစ်ပါတယ်။

Pseudo-code

```
var block = new Square(5);
var box = new Square(10);
```

နမူနာအရ `block` Object ရဲ့ `width`, `height` တန်ဖိုးဟာ 5 ဖြစ်မှာဖြစ်ပြီး `box` Object ရဲ့ `width`, `height` တန်ဖိုးကတော့ 10 ဖြစ်မှာပဲဖြစ်ပါတယ်။ Class တစ်ခုတည်းကနေ ဖန်တီးယူခြင်းဖြစ်ပေမယ့် Constructor အတွက် ပေးလိုက်တဲ့ Parameter တန်ဖိုးပေါ်မူတည်ပြီး Object Property တွေရဲ့ Initial Value က လည်း ကွဲပြားသွားမှာပဲ ဖြစ်ပါတယ်။ ဒီနည်းနဲ့ ကိုယ်ပိုင် Data (Property) နဲ့ ကိုယ်ပိုင် Function (Method) တွေ ကိုယ်စီရှိတဲ့ Object တွေ တည်ဆောက်ပြီး အဲ့ဒီ Object တွေ ဆက်သွယ်ပူးပေါင်းခြင်းအားဖြင့် ပရိုဂရမ်ကို အလုပ်လုပ်စေခြင်းဟာ Object-oriented Programming ရဲ့ အခြေခံအကျဆုံး သဘောသဘာဝပဲ ဖြစ်ပါတယ်။

Information Hiding

Object တစ်ခုမှာ ကိုယ်ပိုင် Data နဲ့ Function တွေရှိတယ်ဆိုပေမယ့် အဲ့ဒီ Data တွေ Function တွေအားလုံးကို အခြား Object တွေက သိရှိဖို့၊ ရယူအသုံးပြုဖို့ မလိုအပ်ပါဘူး။ ဥပမာ - ကားဆိုတဲ့ ပရိုဂရမ်တစ်ခု ဖြစ်ပေါ်လာဖို့၊ အင်ဂျင်၊ ဂီယာ၊ ဘရိတ်၊ စတီယာရင် စသဖြင့် သူနေရာနဲ့သူ အရေးပါတဲ့ Object တွေ ပေါင်းစပ်ခြင်းအားဖြင့် တည်ဆောက်ထားတယ် ဆိုပါစို့။ အင်ဂျင်၊ ဂီယာ စတဲ့ Object တွေဟာ အပြန်အလှန်ဆက်သွယ်ပြီး ပူးပေါင်းဆောင်ရွက်ဖို့လိုပေမယ့်၊ အင်ဂျင်ရဲ့ အတွင်းပိုင်းအလုပ်လုပ်ပုံကို ဂီယာကသိစရာမလိုပါဘူး။ အင်ဂျင်လည်ပါတ်နှုံး အနှေးအမြန်ကိုဂီယာက သိဖို့လိုနိုင်ပေမယ့်၊ အဲ့ဒီ အင်ဂျင်လည်ဖို့အတွက် လောင်စာ Inlet Valve နဲ့ Outlet

Valve ဖွင့်/ပိတ် ပုံတိုင်မင်တွေ ဘယ်လိုအလုပ်လုပ်သလဲ၊ Piston တွေက လောင်စာပေါက်ကွဲအား‌ကို ဘယ်လို တုံ့ပြန်ကသလဲစတဲ့ ရှုပ်ထွေးတဲ့ အသေးစိတ်အချက်အလက်တွေကို တော့ သိဖို့မလိုပါဘူး။ ဒီသဘောအတိုင်းပါပဲ၊ Object တွေမှာလည်း အခြား Object တွေကနေ ရယူအသုံးပြုခွင့် မပေးသင့်တဲ့ Data နဲ့ Function တွေ ရှိတက်ပါတယ်။ အဲ့ဒီလို Data နဲ့ Function တွေကို အခြား Object တွေက ရယူခြင်း၊ ပြင်ဆင်ခြင်း မပြုနိုင်အောင် ဆောင်ရွက်ခြင်းကို Information Hiding လို့ ခေါ်ခြင်းပဲ ဖြစ်ပါတယ်။ Information hiding ကို Encapsulation လို့လည်း ခေါ်ကြပါတယ်။

Information Hiding ကို စနစ်တစ်ကျ အသုံးချမယ်ဆိုရင် လွပ်လပ်ပြီး သီးခြားစီအလုပ်လုပ်တဲ့ Object တွေ ရရှိနိုင်မှာ ဖြစ်လို့ ပိုပြီးပြုပြင်ထိမ်းသိမ်းရလွယ်ကူတဲ့ ပရိဂရမ်တွေကို ရရှိမှာပဲဖြစ်ပါတယ်။ ဥပမာ – အင်ဂျင်ပျက်ရင် အင်ဂျင်ကိုပဲ ပြင်စရာလိုမှာ ဖြစ်ပြီး ဂီယာကို ထိစရာမလိုတော့ပါဘူး။ ဒါအပြင် ပိုပြီး Flexible ဖြစ်တဲ့ ပရိဂရမ်တွေကိုလည်း တည်ဆောက်နိုင်မှာဖြစ်ပါတယ်။ စောစောကဥပမာကို ပြန်ကြည့်မယ်ဆိုရင် ကားထုတ်လုပ်သူတွေက လိုအပ်လာရင် အင်ဂျင်အမျိုးအစားတစ်မျိုးတည်းမှာပဲ ကားတစ်စီးအတွက် Auto ဂီယာနဲ့တပ်ဆင်ပြီး နောက်ကားတစ်စီးအတွက် Manual ဂီယာနဲ့ တပ်ဆင်နိုင်မှာဖြစ်ပါတယ်။ ဂီယာက ဂီယာအလုပ်ပဲလုပ်ပြီး၊ အင်ဂျင်က အင်ဂျင်အလုပ်ကို သီးခြားလုပ်တဲ့အတွက် အင်ဂျင် တစ်မျိုးတည်းမှာ ဂီယာစနစ်အမျိုးမျိုးနဲ့ လိုအပ်ရင် တွဲဖက်တပ်ဆင်နိုင်ခြင်းပဲ ဖြစ်ပါတယ်။

Information Hiding လုပ်ဆောင်ချက် ရရှိဖို့အတွက် Language အများစုက Public, Private, Protected, Static စတဲ့ Access Control Modifier တွေကို အသုံးပြု ကြလေ့ရှိပါတယ်။ Language ပေါ်မူတည်ပြီး ဒီ Access Control တွေရဲ့ အသေးစိတ် အလုပ်လုပ်ပုံကွဲပြားနိုင်ပါတယ်။

Pseudo-code

```
class Engine
{
    public var kilo;
    private var pistons = 4;
    private var valves = 12;

    public function dirve() {
        kilo++;
    }

    public function check() {
        if( dirtyOil() )
            print "Please change engine oil!";
    }

    private function dirtyOil() {
        if(kilo > 3000) return true;
        return false;
    }
}
```

နမူနာ Engine Class မှာ `kilo`, `pistons` နဲ့ `valves` ဆိုပြီး Data Property သုံးခုပါဝင်ပါတယ်။ အဲ့ဒီထဲက `pistons` နဲ့ `valves` တို့ကို `private` လို့ သတ်မှတ်ထားတဲ့အတွက်၊ ဒီတန်ဖိုးနှစ်ခုဟာ ပြင်ပကနေ ရယူ

အသုံးပြု လို့မရတဲ့ တန်ဖိုးတွေဖြစ်သွားမှာပါ။ အလားတူပဲ Function သုံးခုပါဝင်ရာမှာ `dirtyOil()` Function ကို `private` လို့ ကြေငြာသတ်မှတ်ထားတဲ့အတွက် ဒီ Function ကိုလည်း ပြင်ပကနေ ရယူအသုံးပြုနိုင်မှာ မဟုတ်ပါဘူး။ `private` Data နဲ့ Function တွေကို Class အတွင်းမှာတော့ရယူအသုံးပြုနိုင်ပါတယ်။ နမူနာမှာ ဆိုရင် `check()` Function က `dirtyOil()` Function ကို ခေါ်ယူအသုံးပြုထားပါတယ်။

Engine Class ကနေ Object တစ်ခုကို အခုလို တည်ဆောက်ရယူထားတယ် ဆိုပါစို့ –

Pseudo-code

```
1. var v6 = new Engine();
2. v6.drive();
3. v6.check();
```

တည်ဆောက်ရရှိထားတဲ့ `v6` Object ကနေ `public` Function များဖြစ်ကြတဲ့ `drive()` သို့မဟုတ် `check()` Function တို့ကို ခေါ်ယူအလုပ်လုပ်စေနိုင်ပေမယ့် `dirtyOil()` Function ကိုတော့ `private` ဖြစ်နေတဲ့ အတွက် ခေါ်ယူအလုပ် လုပ်စေနိုင်မှာ မဟုတ်ပါဘူး။ ဒီနည်းနဲ့ Data တွေ Function တွေမှာ Access Control Modifier တွေ ကြေငြာသတ်မှတ်ခြင်း အားဖြင့် Information Hiding လုပ်ဆောင်ချက် ကိုရရှိနိုင်ပါတယ်။ Access Control သတ်မှတ် ထားခြင်းမရှိတဲ့ Data နဲ့ Function တွေအတွက် Default Modifier ကို အသုံးပြုကြလေ့ရှိပါတယ်။ ဥပမာ - PHP Programming Language မှာဆိုရင် Default Modifier က `public` ဖြစ်တဲ့ အတွက် Access Control သတ်မှတ် ထားခြင်းမရှိတဲ့ Data နဲ့ Function အားလုံးဟာ `public` ဖြစ်နေမှာပဲဖြစ်ပါတယ်။

အခြား `protected`, `package public` စသဖြင့် အသုံးပြုနိုင်တဲ့ Access Control Modifier တွေ Language အလိုက် ရှိတက်ကြပါသေးတယ်။ ဒီထဲက ထူးခြားတာကတော့ `static` Access Control Modifier ပဲဖြစ်ပါတယ်။ Function သို့မဟုတ် Data တန်ဖိုးတစ်ခုခုကို `static` လို့ ကြေငြာသတ်မှတ်ထားမယ်ဆိုရင် အဲ့ဒီ Function သို့မဟုတ် Data ကို Object တည်ဆောက်စရာမလိုပဲ တိုက်ရိုက်ရယူအသုံးပြုနိုင်ပါတယ်။ အထူးသဖြင့် Object တစ်ခုချင်းစီရဲ့ သီးခြားကိုယ်ပိုင် တန်ဖိုးဖြစ်ဖို့မလိုတဲ့ ဘုံတန်ဖိုးတွေကို `static` အဖြစ် ကြေငြာသတ်မှတ်ကြလေ့ရှိပါတယ်။ ဥပမာ –

Pseudo-code

```
1. class Math
2. {
3.     static var PI = 3.1416;
4.     static function round(value) {
5.         return parseInt(value);
6.     }
7. }
```

ဒီ `Math` Class မှာပါဝင်တဲ့ `PI` နဲ့ `round()` တို့ကို `static` လို့ ကြေငြာသတ်မှတ်ထားတဲ့အတွက် Object တည် ဆောက်စရာမလိုပဲ အခုလို တိုက်ရိုက်ရယူအသုံးပြုနိုင်မှာဖြစ်ပါတယ်။

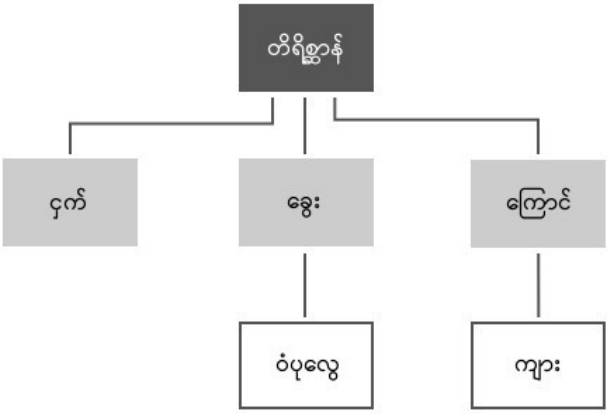
Pseudo-code

```
1. print Math.PI;
2. print Math.round(4.2);
```

`PI`, `round()` စတဲ့ Data နဲ့ Function တွေဟာ Object ပြောင်းတိုင်း လိုက်ပြောင်းစရာမလိုတဲ့ ပုံသေတန်ဖိုးနဲ့၊ လုပ်ဆောင်ချက်တွေ ဖြစ်တဲ့အတွက် ဒီလိုအခြေအနေမှာ `static` Access Control Modifier ကို အသုံးပြုကြလေ့ရှိ ခြင်း ဖြစ်ပါတယ်။

Inheritance

Inheritance ဆိုတာကတော့ Object သို့မဟုတ် Class တစ်ခုက အခြား Object သို့မဟုတ် Class တစ်ခုရဲ့ Data နဲ့ Function တွေကို ဆက်ခံရယူနိုင်စေတဲ့ လုပ်ဆောင်ချက်ပဲ ဖြစ်ပါတယ်။ Inheritance ရဲ့ အကူအညီနဲ့ အချို့လုပ်ဆောင်ချက်တွေကို ထပ်ခါထပ်ခါ ပြန်လုပ်စရာမလိုပဲ ပြန်လည်အသုံးချနိုင်စေသလို၊ သဘောသဘာဝ ဆင်တူနဲ့ Object တွေကို အပြန်အလှန်ဆက်စပ် စေခြင်းအားဖြင့် စနစ်ကျပြီး ပြင်ဆင်ထိမ်းသိမ်းရ လွယ်ကူတဲ့ Object Hierarchy ကို တည်ဆောက်ထားနိုင်မှာပဲဖြစ်ပါတယ်။



ပုံ (၃.၁) - Object Hierarchy

နမူနာပုံမှာလေ့လာကြည့်မယ်ဆိုရင် ငှက်၊ ခွေး၊ ကြောင် စတဲ့ Object များက တိရိစ္ဆာန် Object ကနေဆက်ခံပြီး Inherit လုပ်ထားတဲ့အတွက် ငှက်၊ ခွေး၊ ကြောင် အားလုံးတို့မှာ တိရိစ္ဆာန် Object မှာရှိတဲ့ Data နဲ့ Function တွေ အားလုံး ရရှိ ပါဝင်သွားမှာပဲ ဖြစ်ပါတယ်။ အလားတူပဲ၊ ဝံပုလွေ Object ဟာ ခွေး Object ကို ဆက်လက်ဆက်ခံထားတဲ့အတွက် တိရိစ္ဆာန် Object နဲ့ ခွေး Object တို့မှာ ပါဝင်တဲ့ Data နဲ့ Function တွေအားလုံးကို ရရှိပါဝင်သွားမှာဖြစ်ပါတယ်။

ဆက်ခံတဲ့ Child Object တွေမှာ မူလ Parent Object မှာ မပါဝင်တဲ့ Data နဲ့ Function တွေ ဖြည့်စွက်ပါဝင်နိုင်သလို မူလ Parent Object ရဲ့ Data နဲ့ Function တွေကို လိုအပ်သလို ပြောင်းလဲသတ်မှတ်ပြီးတော့လည်း ထားနိုင်ပါတယ်။

Pseudo-code

```
class Animal
{
    var legs = 4;

    function eat() {
       ...
    }
}

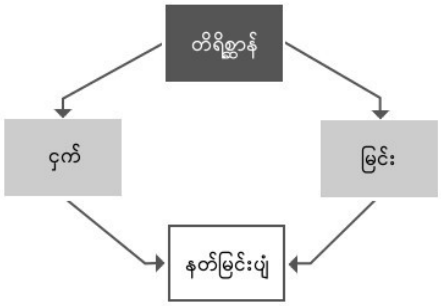
class Bird extends Animal
{
    var legs = 2;
    var wings = true;
}

class Dog extends Animal
{
    function bark() {
        ...
    }
}
```

နမူနာမှာလေ့လာကြည့်ရင် `Animal` Class မှာ `legs` ဆိုတဲ့ Data နဲ့ `eat()` ဆိုတဲ့ Function တို့ပါဝင်ပါတယ်။ `Animal` Class ကို `Bird` Class နဲ့ `Dog` Class တို့က `extends` Keyword ကိုသုံးပြီး ဆက်ခံ Inherit လုပ်ထားကြပါတယ်။ ဒါကြောင့် `Bird` နဲ့ `Dog` Class တို့ဟာ `Animal` Class ရဲ့ `legs` နဲ့ `eat()` ကို အလိုအလျောက် ဆက်ခံရရှိသွားပါတယ်။ ဒါပေမယ့် `Bird` Class မှာ `legs` ရဲ့ တန်ဖိုးကို ပြောင်းထားပြီး `wings` ဆိုတဲ့ တန်ဖိုး အသစ် တစ်ခုလည်း ဖြည့်စွက် ပါဝင်လာပါတယ်။ ပြီးတော့ `Dog` Class မှာဆိုရင်လည်း `bark()` ဆိုတဲ့ Function တစ်ခု ဖြည့်စွက် ပါဝင်လာပါတယ်။

ဒါကြောင့် အခုနေ `Dog` Object တစ်ခု တည်ဆောက်မယ်ဆိုရင် `legs` ဆိုတဲ့ Data နဲ့ `eat()`, `bark()` ဆိုတဲ့ Function တို့ ပါဝင်သွားမှာပဲ ဖြစ်ပါတယ်။

Inheritance မှာ Single Inheritance နဲ့ Multiple Inheritance ဆိုပြီးနှစ်မျိုးရှိပါတယ်။ Single Inheritance ဆိုတာကတော့ Object သို့မဟုတ် Class တစ်ခုဟာ အခြား Object သို့မဟုတ် Class တစ်ခုကိုသာ ဆက်ခံရတဲ့ နည်းစနစ်ဖြစ်ပြီး Multiple Inheritance ဆိုတာကတော့ Class သို့မဟုတ် Object တစ်ခုဟာ အခြား တစ်ခုထက်ပိုသော Class များ၊ Object များကို ဆက်ခံရယူနိုင်တဲ့ နည်းစနစ်ဖြစ်ပါတယ်။ Java, Ruby စတဲ့ Language တွေမှာ Multiple Inheritance ကို ခွင့်မပြုကြပဲ Single Inheritance ကိုသာ ခွင့်ပြုကြပါတယ်။ C++ နဲ့ Python တို့လို Language တွေကတော့ Multiple Inheritance ကို ခွင့်ပြုထားပါတယ်။ Language က ခွင့်ပြုသည်ဖြစ်စေ၊ ခွင့်မပြုသည်ဖြစ်စေ လက်တွေ့မှာ Multiple Inheritance ကို ရှောင်ကြလေ့ရှိပါတယ်။ "Deadly diamond of death" လို့ခေါ်တဲ့ ပြဿနာ ကြောင့်ပဲဖြစ်ပါတယ်။



ပုံ (၃.၂) - Deadly Diamond of Death

ပုံ (၃.၂) မှာလေ့လာကြည့်ရင် ငှက်နဲ့ မြင်းတို့ဟာ တိရိစ္ဆာန်ကို ဆက်ခံထားကြပါတယ်။ နတ်မြင်းပျံဆိုတဲ့ Object တစ်ခု ရရှိလိုတဲ့အတွက် ငှက်နဲ့ မြင်း နှစ်ခုတို့ထံကနေ Multiple Inheritance အနေနဲ့ ဆက်ခံလိုက်တဲ့ အခါ အခုလို ပြဿနာ ပေါ်ပေါက်လာတက်ပါတယ်။

Pseudo-code

```
class Animal
{
    function giveBirth() {
        ...
    }
}

class Bird extends Animal
{
    var legs = 2;
    var wings = true;
    function giveBirth() {
        layEggs();
    }
}

class Horse extends Animal
{
    var legs = 4;
    function giveBirth() {
        birthLive();
    }
}

class FlyingHorse extends Bird, Horse
{
    ...
}

var pegasus = new FlyingHorse();
pegasus.giveBirth();
```

နမူနာမှာလေ့လာကြည့်ရင် `Animal` Class မှာ `giveBirth()` Function ပါဝင်ပြီး၊ `Bird` နဲ့ `Horse` Class

နှစ်ခု လုံးမှာလည်း `giveBirth()` Function ကို ပြုပြင်ထားပါတယ်။ ငှက်တွေက ဥ ဥခြင်းအားဖြင့်သားဖွားပြီး၊ မြင်းတွေ ကတော့ အကောင်လိုက် မွေးကြလေ့ရှိတဲ့အတွက် သဘောတရားမတူလို့ ပြုပြင်ထားရခြင်းဖြစ်ပါတယ်။ `FlyingHorse` Class က `Bird` နဲ့ `Horse` နှစ်ခုလုံးထံကနေ Inherit လုပ်ထားပါတယ်။ လိုင်းနံပါတ် (၃၁) မှာ လေ့လာကြည့်ပါ။ `FlyingHorse` Class ရဲ့ Object ဖြစ်တဲ့ `pegasus` ကနေ `giveBirth()` ကို ခေါ်ယူ အလုပ် လုပ်စေတဲ့အခါ ပြဿနာကြုံရမှာဖြစ်ပါတယ်။ `Bird` နဲ့ `Horse` Class နှစ်ခုလုံးမှာ `giveBirth()` Function ရှိနေတဲ့အတွက် `Bird` ရဲ့ `giveBirth()` ကို ဆက်ခံ အလုပ်လုပ်ပေးရမှာလား၊ `Horse` ရဲ့ `giveBirth()` ကို ဆက်ခံအလုပ်လုပ် ပေးရမှာလားဆိုတဲ့ ပြဿနာကို ကြုံတွေ့ရခြင်းပဲဖြစ်ပါတယ်။ ဒီပြဿနာကို Deadly diamond of death လို့ ခေါ်တာပါ။

Multiple Inheritance ကို လက်ခံတဲ့ Language တွေမှာတော့ ဒီပြဿနာကို နည်းလမ်းအမျိုးမျိုးနဲ့ ဖြေရှင်းထားကြ ပါတယ်။ Perl, Python စတဲ့ Language တွေက၊ `extends` လုပ်စဉ်မှာ ပေးခဲ့တဲ့အစီအစဉ်ပေါ်မူတည်ပြီး "ဦးစားပေး" Class စာရင်းထား ရှိခြင်းအားဖြင့်၊ Deadly diamond ပြဿနာကြုံလာတဲ့အခါ စာရင်းထဲမှာ Priority ပိုမြင့်တဲ့ Class ရဲ့ Function ကို ဦးစားပေးအဖြစ် သတ်မှတ်အသုံးပြု ပေးသွားမှာပဲဖြစ်ပါတယ်။

တစ်ချို့ Language တွေမှာဆိုရင် Inherit လုပ်ခွင့်မပြုတဲ့ Syntax ရေးထုံးတွေရှိတက်ပါသေးတယ်။ ဥပမာ - Java မှာ Class တစ်ခုကို `final` လို့ ကြေငြာသတ်မှတ်ထားရင် အခြား Class များက Inherit လုပ်ယူခြင်းကို ခွင့်မပြုတဲ့ Class ဖြစ်သွားမှာပဲ ဖြစ်ပါတယ်။ ဒါအပြင် Inherit လုပ်ခွင့်ပြုပေမယ့် မူလ Parent Class/Object ရဲ့ Data နဲ့ Function ကို Child Class တွေထဲမှာ ပြုပြင်ခြင်းကို ကန့်သတ်ထားနိုင်ပါသေးတယ်။

Interface

ပရိဂရမ်တစ်ခုမှာ Text Box, Password Box နဲ့ Email Box ဆိုပြီး Input Box အမျိုးမျိုးပါဝင်တယ်ဆိုကြပါစို့။ ဒီ Input Box တွေ တစ်ခုနဲ့တစ်ခု ဖော်ပြပုံတူကြသလို၊ ပါဝင်တဲ့ လုပ်ဆောင်ချက်တွေ တူကြပါတယ်။ Size Property ကို ပြောင်းခြင်းအားဖြင့် Input Box ရဲ့ အရွယ်အစားကို ပြင်နိုင်ပါတယ်။ တန်ဖိုးတစ်ခုထည့်သွင်းလိုရင် Input Box ကို Click လုပ်ပြီး Keyboard ကနေ ထည့်သွင်းနိုင်ပါတယ်။ Input Box မှာ ထည့်သွင်းထားတဲ့ တန်ဖိုးကို လက်ခံရယူပြီး လိုအပ်တဲ့လုပ်ငန်းတွေ ဆက်လက်ဆောင် ရွက်နိုင်ပါတယ်။ ဒါပေမယ့် အသေးစိတ် ကွာခြားချက်တွေတော့ ရှိပါတယ်။ Text Box ဆိုရင် ထည့်သွင်းလိုက်တဲ့စာတွေကို ဖော်ပြပြီး၊ Password Box ဆိုရင်တော့ စာတွေအစား Star လေးတွေနဲ့ အစား ထိုးဖော်ပြနိုင်ပါတယ်။ ဒါပေမယ့် Text Box သုံးရမယ့် နေရာမှာ Password Box ကို အစားထိုး အသုံးပြုနိုင်ပါတယ်။ Password Box သုံးရမယ့်နေရာမှာလည်း Text Box ကို အစားထိုး အသုံးပြုနိုင်ပါတယ်။ ဘာဖြစ်လို့လဲဆိုတော့ ကြိုတင် သတ်မှတ်ထားတဲ့ Interface တစ်ခုတည်းကနေ ရရှိလာတဲ့ Object တွေမို့ပဲဖြစ်ပါတယ်။ Text Box က ပြန်ပေးတဲ့ တန်ဖိုးနဲ့ Password Box က ပြန်ပေးတဲ့ တန်ဖိုး၊ သဘောသဘာဝ တူညီမှာဖြစ်တဲ့ အတွက် အခုလိုအသုံးပြု နိုင်ခြင်းပဲဖြစ် ပါတယ်။ ဒီတော့ Text Box နဲ့ တွဲဖက်အလုပ်လုပ်ရမယ့် Object ကတစ်မျိုး၊ Password Box နဲ့ တွဲဖက်အလုပ် လုပ်ရမယ့် Object က တစ်မျိုးဆိုပြီး ခွဲခြားရေးသားနေစရာ မလိုတော့ပဲ Object တစ်မျိုးတည်းနဲ့ပဲ Text Box နဲ့ရော Password Box နဲ့ပါ တွဲဖက်အလုပ်လုပ်စေနိုင်မှာဖြစ်လို့ Code Design လည်း အများကြီးတိုးတက်ကောင်းမွန်သွားမှာပဲ ဖြစ်ပါတယ်။

Interface ဆိုတာဟာ အဲ့ဒီလိုမျိုး အသေးစိတ် လုပ်ဆောင်ချက်ကွာခြားပေမယ့် အပြန်အလှန် ဖလှယ်အသုံးပြုနိုင်တဲ့ Object တွေ တည်ဆောက်နိုင်ဖို့အတွက် ဘုံစံကိုသတ်မှတ်ပေးတဲ့ နည်းစနစ်တစ်မျိုးပဲ ဖြစ်ပါတယ်။ ဒါ

ကြောင့် Interface ကို Protocol (သို့မဟုတ်) Contract လို့လည်းခေါ်ကြပါသေးတယ်။ အောက်ပါ Code နမူနာကို လေ့လာကြည့်ပါ။

Pseudo-code

```
interface InputBox
{
    function setValue(val);
    function showValue();
    function resize(width, height);
}

class TextBox implement InputBox
{
    var value;
    function setValue(val) {
        value = val;
    }
    function showValue() {
        print value;
    }
    function resize(width, height) {
        ...
    }
}

class PasswordBox implement InputBox
{
    var value;
    function setValue(val) {
        value = val;
    }
    function showValue() {
        for(var i=0; i < value.length; i++) {
            print "*";
        }
    }
    function resize(width, height) {
        ...
    }
}
```

လိုင်းနံပါတ် (၁) မှာ `InputBox` Interface ကို ကြေငြာသတ်မှတ်ထားပါတယ်။ `setValue()`, `showValue()` နဲ့ `resize()` ဆိုတဲ့ Function သုံးခုကို ကြေငြာသတ်မှတ်ထားပေမယ့် အဲဒီ Function တစ်ခုချင်းရဲ့ လုပ်ဆောင်ချက် အသေးစိတ်ကိုတော့ သတ်မှတ်မထားပါဘူး။ Interface တစ်ခုရဲ့ထုံးစံအတိုင်း ပါဝင်ရမယ့် လုပ်ဆောင်ချက် Function စာရင်းနဲ့ Function Signature လို့ခေါ်တဲ့၊ အဲဒီ Function တွေကိုတည်ဆောက်တဲ့အခါ ပါဝင်သင့်တဲ့ Parameter စာရင်းတို့ကိုသာ ကြေငြာသတ်မှတ်ထားခြင်း ဖြစ်ပါတယ်။

ဒီ Interface ကို ဆက်ခံတည်ဆောက်ထားတဲ့ Object တိုင်းမှာ Interface က သတ်မှတ်ထားတဲ့ Function အားလုံး မဖြစ်မနေပါဝင်ရမှာဖြစ်ပြီး Function Signature ကလည်း Interface မှာ သတ်မှတ်ထားတဲ့အတိုင်း အတိအကျဖြစ်ရ မှာဖြစ်ပါတယ်။ ဒါကြောင့်လည်း Interface တစ်ခုတည်းက ဆင်းသက်လာတဲ့ Object များဟာ

တစ်ခုနဲ့တစ်ခု အပြန် အလှန် ဖလှယ်အသုံးပြုနိုင်ခြင်းဖြစ်ပါတယ်။ နမူနာမှာ `TextBox` နဲ့ `PasswordBox` ဆိုတဲ့ Class နှစ်ခုက Input Box Interface ကို ဆက်ခံတည်ဆောက်ထားကြပါတယ်။ ဒါကြောင့် Class နှစ်ခုလုံးမှာ အသေးစိတ် အလုပ်လုပ်ပုံမတူ ပေမယ့် `setValue()`, `showValue()` နဲ့ `resize()` ဆိုတဲ့ ပါဝင်ရမယ့် Function အားလုံး ပါဝင်ပါတယ်။

ပြီးခဲ့တဲ့အခန်းမှာ Test Driven Development အကြောင်းဖော်ပြစဉ်က၊ Stub တွေ Mock တွေရဲ့ အခန်းကဏ္ဍကို ဖော်ပြခဲ့ပါတယ်။ Interface ရဲ့ အကူအညီနဲ့ တစ်ကယ့် Object အစား Mock Object တွေနဲ့ လိုအပ်သလို အစားထိုး စမ်းသပ်နိုင်တဲ့ အားသာချက်ကိုလည်း ရနိုင်ပါသေးတယ်။ တစ်ကယ့် Object ရော Mock Object ကပါ Interface တစ်ခုတည်းက ဆင်းသက်မယ်ဆိုရင် အပြန်အလှန် ဖလှယ်အသုံးပြုနိုင်တဲ့ Object တွေဖြစ်မှာမို့၊ တစ်ကယ့် Object အစား Mock Object နဲ့ အလွယ်တစ်ကူ အစားထိုး စမ်းသပ်နိုင်မှာပဲ ဖြစ်ပါတယ်။

Polymorphism

Polymorphism ဆိုတာကတော့ Object တစ်ခုရဲ့ Function တွေကို တစ်မျိုးတည်းပုံသေအလုပ်လုပ်တာမျိုး မဟုတ်ပဲ ခေါ်ယူအသုံးပြုတဲ့အခြေအနေပေါ် မူတည်ပြီး ပြောင်းလဲအလုပ်လုပ်နိုင်အောင် ဖန်တီးတဲ့ နည်းစနစ်တစ်ခုဖြစ်ပါတယ်။ Static Polymorphism နဲ့ Dynamic Polymorphism ဆိုပြီး နှစ်မျိုးရှိပါတယ်။ Static Polymorphism လုပ်ဆောင် ချက်ရရှိဖို့အတွက် Function Overload နည်းစနစ် ကိုအသုံးပြုပြီး Dynamic Polymorphism လုပ်ဆောင်ချက် ရရှိဖို့ အတွက်တော့ ပြောခဲ့ပြီးဖြစ်တဲ့ Inheritance နဲ့ Interface တို့ကို သုံးနိုင်ပါတယ်။

Pseudo-code

```
class Shape
{
    function area(r) {
        return 3.14 * r * r;
    }

    function area(x, y) {
        return x * y;
    }

    function area(x, y, z) {
        return x * y * z;
    }
}
```

နမူနာမှာလေ့လာကြည့်ရင် Shape Class မှာ Parameter စာရင်းမတူတဲ့ `area()` Function သုံးခုပါဝင်တာကိုတွေ့ရ မှာဖြစ်ပါတယ်။ ဒါကို Function Overload လို့ခေါ်ပြီး၊ ခေါ်ယူတဲ့အခါ ပေးတဲ့ Parameter စာရင်း (Function Signature) ပေါ်မူတည်ပြီး အလုပ်လုပ်တဲ့ Function လည်းပြောင်းသွားမှာပဲဖြစ်ပါတယ်။ ဥပမာ –

Pseudo-code

```
var item = new Shape();
item.area(2);           // => 12.56
item.area(2, 2);        // => 4
item.area(2, 2, 2);     // => 8
```

ဒီနည်းနဲ့ ခေါ်ယူတဲ့ Function Signature ပေါ်မူတည်ပြီး လုပ်ဆောင်ပုံပြောင်းလဲနိုင်စေခြင်းကို Static Polymorphism လို့ ခေါ်ခြင်းဖြစ်ပါတယ်။ Dynamic Polymorphism ကိုတော့ Interface အကြောင်းဖော်ပြစဉ်က တွေ့မြင်ခဲ့ကြရပြီးဖြစ် ပါတယ်။

Pseudo-code

```
interface Animal
{
    function move();
}

class Dog implement Animal
{
    function move() {
        print "Dog walk and run";
    }
}

class Bird implement Animal
{
    function move() {
        print "Bird fly";
    }
}
```

နမူနာမှာ `Animal` Interface ကနေ `Dog` နဲ့ `Bird` ဆိုတဲ့ Class နှစ်ခု ဆက်ခံတည်ဆောက်ထားပါတယ်။ ပြီးတော့ Interface က သတ်မှတ်ထားတဲ့အတိုင်း `move()` Function ကိုလည်း Implement လုပ်ထားပါသေးတယ်။ `Dog` ရဲ့ `move()` Function နဲ့ `Bird` ရဲ့ `move()` Function တို့ အသေးစိတ် အလုပ်လုပ်ပုံ မတူကြပါဘူး။ ဒါကြောင့် Animal Interface က လာတဲ့ Object ချင်း တူပေမယ့် `move()` Function ကို ခေါ်ယူတဲ့အခါမှာ Object ပေါ် မူတည်ပြီး `move()` Function ရဲ့ အလုပ်လုပ်ပုံလည်း အခုလို ပြောင်းလဲသွားမှာပဲဖြစ်ပါတယ်။

Pseudo-code

```
var milo = new Dog();
var jay = new Bird();

milo.move();        // => Dog walk and run
jay.move();         // => Bird fly
```

ဒီလို Interface ကနေ Implement လုပ်ယူခြင်းအားဖြင့်၊ သို့မဟုတ်၊ အခြား Class တစ်ခုကနေ Inheritance လုပ်ယူခြင်းအားဖြင့် အမျိုးအစားတူပေမယ့် လုပ်ဆောင်ပုံမတူတဲ့ Function တွေ ရရှိနိုင်ခြင်းကို Dynamic

Polymorphism လို့ခေါ်ပါ တယ်။

ဒါကြောင့် Object-oriented Programming ဟာ ကျယ်ပြန့်ရှုပ်ထွေးတဲ့ ပရိုဂရမ်တွေကို Extensible ဖြစ်အောင်၊ Maintainable ဖြစ်အောင် ကူညီပေးတယ်လို့ဆိုပေမယ့်၊ Class တွေ Object ကို အခြေခံလိုက်ယုံသက်သက်နဲ့တော့ ပရိုဂရမ်က အလိုအလျောက် စနစ်ကျကောင်းမွန်သွားမှာ မဟုတ်ပါဘူး။ ဖော်ပြခဲ့တဲ့ Information Hiding, Inheritance, Interface နဲ့ Polymorphism တို့ ကို အသေအချာနားလည်ပြီး၊ သူ့နေရာနဲ့သူ ထိရောက်အောင် အသုံးချနိုင်မှသာ Extensible ဖြစ်ပြီး Maintainable ဖြစ်တဲ့ Object-oriented ပရိုဂရမ်တွေကို ရရှိမှာပဲဖြစ်ပါ တယ်။

3.2 – OOP in JavaScript

JavaScript Programming Language ဟာ Object-oriented နည်းစနစ်များကို အခြေခံထားတဲ့ OOP Language တစ်ခုဖြစ်ပါတယ်။ ဒါပေမယ့် အခြား OOP Language တွေနဲ့မတူပါဘူး။ အဓိကအကျဆုံးထူးခြားချက်ကတော့၊ အခြား OOP Language တွေမှာ Object တွေ ရရှိဖို့အတွက် Class တွေကို အသုံးပြုကြပြီး၊ JavaScript မှာတော့ Class တွေ ကိုအသုံးမပြုပဲ အခြား Object တစ်ခုကို မူပွားယူခြင်း၊ ဖြည့်စွက်ပြင်ဆင်ခြင်းအားဖြင့် နောက် Object တွေကို တည် ဆောက်ရပါတယ်။ Class တွေကို အခြေခံတဲ့ OOP Language တွေကို Classical OOP လို့ ခေါ်ပြီး JavaScript ကို တော့ Prototype-based OOP လို့ ခေါ်ပါတယ်။

ဖြည့်စွက်ချက် ။ ။ အနာဂါတ်မှာ JavaScript ဖြစ်လာမယ့် ECMAScript 6 (ES6) မှာတော့ Class ရေးထုံး စတင်ပါဝင်လာပြီ ဖြစ်ပါတယ်။ NodeJS လို JavaScript Run-time မှာ စတင်အသုံးပြုလို့ ရနေပါပြီ။ မကြာခင်မှာ Web Browser တွေထဲမှာ လည်း Class ရေးထုံးကို အသုံးပြုထားတဲ့ JavaScript Code တွေကို ရေးသား အသုံးပြုနိုင်တော့မှာပါ။

Classical OOP မှာ Class တွေကို ဦးဆုံးတည်ဆောက်ပြီးမှ အဲ့ဒီ Class ကနေတစ်ဆင့် Object တွေ တည် ဆောက်ရပါတယ်။ Object Design ကို အရင်သတ်မှတ်ပြီးမှ တစ်ကယ့် Object ကို တည်ဆောက်တဲ့ သဘောပါ။ Prototype-based OOP မှာတော့ Object အလွတ်တစ်ခုကို အရင်တည်ဆောက်လိုက်ပြီး နောက်မှလိုအပ်တဲ့ Data နဲ့ Function တွေကို လိုအပ်သလို တွဲဖက်သတ်မှတ်ရခြင်း ဖြစ်ပါတယ်။ Object Design တစ်ခုပုံသေမထားပဲ လိုအပ်သလို ပြုပြင်ဖြည့် စွက်သွားတဲ့သဘော ဖြစ်ပါတယ်။ ဒါဟာ Classical OOP နဲ့ JavaScript ရဲ့ Prototype-based OOP ရဲ့ အဓိကကျတဲ့ ကွာခြားချက်ဖြစ်ပါတယ်။

Creating Objects

JavaScript မှာ Object တွေရရှိဖို့အတွက် JSON လို့ခေါ်တဲ့ နည်းစနစ်ကို အသုံးပြုနိုင်ပါတယ်။ JSON အကြောင်းကို တော့ နောက်တစ်ခန်းမှ ဖော်ပြပါမယ်။ ဒီနေရာမှာတော့ Classical OOP ရဲ့ Class နဲ့ ပိုပြီးနီးစပ်တဲ့ Object Constructor အသုံးပြုခြင်းအားဖြင့် Object တွေ တည်ဆောက်နိုင်ပုံကို ဖော်ပြပေးပါမယ်။

JavaScript မှာ Function တွေကို Object Constructor အနေနဲ့လည်း အသုံးပြု နိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
1. function Person() {}
```

အခြေခံအားဖြင့် `Person` အမည်နဲ့ Function အလွတ်တစ်ခုကို တည်ဆောက်လိုက်ခြင်းဖြစ်ပါတယ်။ ထူးခြားချက်အနေ နဲ့ ဒီ Function ကနေတစ်ဆင့် Object များကို အခုလို တည်ဆောက်ယူနိုင်မှာပဲဖြစ်ပါတယ်။

JavaScript

```
1. var alice = new Person();
2. var bob = new Person();
```

ဒီနည်းနဲ့ `alice` နဲ့ `bob` လို့ခေါ်တဲ့ ကိုယ်ပိုင် Data နဲ့ Function တွေ မရှိသေးတဲ့ Object အလွတ်နှစ်ခု ရရှိသွားမှာပဲ ဖြစ်ပါတယ်။ `Person` Object Constructor အတွက် Data နဲ့ Function တွေကို အခုလို သတ်မှတ်ပေးနိုင်ပါတယ်။

JavaScript

```
1. Person.prototype.fullName = "Jame Doe";
2. Person.prototype.sayName = function() {
3.     console.log("Hello, I am " + this.fullName);
4. }
```

`prototype` Attribute ကနေတစ်ဆင့် Object Constructor အတွက် Data နဲ့ Function တွေကို တွဲဖက်ပေးရခြင်းပဲဖြစ်ပါတယ်။ ဒီလိုနောက်မှတွဲဖက်ခြင်းမဟုတ်ပဲ Object Constructor ကြေငြာကတည်းက တစ်ခါတည်း သတ်မှတ် လိုရင်လည်း အခုလိုသတ်မှတ်နိုင်ပါတယ်။

JavaScript

```
1. function Person(name) {
2.     this.fullName = name;
3.     this.sayName = function() {
4.         console.log("Hello, I am " + this.fullName);
5.     }
6. }
```

Function အတွင်းထဲမှာ `this` Keyword ကို သုံးပြီးသတ်မှတ်လိုတဲ့ Data နဲ့ Function တွေကို ကြိုတင်သတ်မှတ် နိုင်ခြင်းဖြစ်ပါတယ်။ ခုနေ Person Object တစ်ခုတည်ဆောက်ပြီး `sayName()` Function ကို အလုပ်လုပ်စေရင် အခုလိုရလဒ်ကို ရရှိမှာဖြစ်ပါတယ်။

JavaScript

```
var jame = new Person('Jame Doe');
jame.sayName();                    // => Hello, I am Jame Doe
```

Information Hiding

JavaScript မှာ Public, Private စတဲ့ Access Control Modifier တွေ မရှိပါဘူး။ ဒါကြောင့် Information Hiding လုပ်ဆောင်ချက် ရရှိလိုရင် အခြားနည်းလမ်းတစ်ခုကို အသုံးပြုရပါတယ်။ Object တည်ဆောက်စဉ်က Data နဲ့ Function တွေကို Constructor အတွင်းမှာ `this` နဲ့ တွဲဖက်သတ်မှတ်ခဲ့တာကို သတိပြုမိမှာပါ။ Data နဲ့ Function တွေကို `this` နဲ့ မတွဲဖက်ပဲ Constructor အတွင်းမှာ ဒီအတိုင်းရေးသားခြင်းအားဖြင့် ပြင်ပကနေ ရယူအသုံးမပြုနိုင်တဲ့ Private Data နဲ့ Function များကို ရရှိနိုင်မှာဖြစ်ပါတယ်။

JavaScript

```
function Person(name) {
    this.fullName = name;
    this.sayName = function() {
        sayHello(this.fullName);
    }

    function sayHello(name) {
        console.log("Hello, I am " + name);
    }
}

var foo = new Person("James");
foo.sayName();    // => Hello, I am James
foo.sayHello();   // => TypeError: foo.sayHello is not a function
```

`Person` Constructor အတွင်းမှာ ရေးသားထားတဲ့ `sayHello()` Function ဟာ Constructor အတွင်းမှာသာ ရယူအသုံးပြုနိုင်တဲ့ Private Function တစ်ခုဖြစ်ပါတယ်။ ပြင်ပကနေရယူဖို့ကြိုးစားတဲ့အခါ လိုင်းနံပါတ် (၁၄) မှာပြထား သလို Error ဖြစ်သွားမှာပါ။ ဒီနည်းနဲ့ Information Hiding လုပ်ဆောင်ချက်ကို JavaScript မှာ ရရှိနိုင်ပါတယ်။

Inheritance

JavaScript မှာ Object Constructor ရဲ့ Prototype ကို မူပွားယူခြင်းအားဖြင့် Inheritance လုပ်ဆောင်ချက် ရရှိနိုင် ပါတယ်။

JavaScript

```
function Person() {};
Person.prototype.fullName = "Jame Doe";
function Student() {};
Student.prototype = Object.create(Person.prototype);
```

လိုင်းနံပါတ် (၄) မှာ `Person` ရဲ့ Prototype ကို Build-in Method တစ်ခုဖြစ်တဲ့ `Object.create()` ရဲ့ အကူအညီနဲ့ `Student` ရဲ့ Prototype အဖြစ် ကူးယူသတ်မှတ်လိုက်ခြင်းဖြစ်ပါတယ်။ ဒီနည်းနဲ့ `Person` မှာ ရှိနေတဲ့ Data နဲ့ Function တို့ကို `Student` က ဆက်ခံရရှိသွားစေပါတယ်။ Prototype အနေနဲ့တွဲဖက်ထားခြင်းမဟုတ်ပဲ Object Constructor မှာ တစ်ခါတည်း ကြေငြာထားတဲ့ Data နဲ့ Function တို့ကိုတော့ အခုလို Inherit လုပ်ယူနိုင် ပါတယ်။

JavaScript

```
function Person(name) {
    this.fullName = name;
    this.sayName = function() {
        console.log("Hello, I am " + this.fullName);
    };
}

function Student(name, major) {
    Person.call(this, name);

    this.major = major;
    this.sayMajor = function() {
        console.log("I learn " + this.major);
    }
}

foo = new Student("Jame", "Computer Sciences");
foo.sayName();        // => Hello, I am Jame
foo.sayMajor();       // => I learn Computer Sciences
```

နမူနာမှာ အဓိကကျတာကတော့ လိုင်းနံပါတ် (၉) က `Person.call()` ဖြစ်ပါတယ်။ JavaScript မှာ Function တွေ ကိုယ်တိုင်က Object တွေဖြစ်ကြပါတယ်။ ဒါကြောင့် `Person` ဟာ Object Constructor တစ်ခုဖြစ်သလို Object တစ်ခုလည်း ဖြစ်နေပါတယ်။ `call()` Function ဟာ Person ဆိုတဲ့ Function Object ရဲ့ Build-in Method တစ်ခုပါ။

`Student` Constructor ထဲမှာ `Person.call()` ဆိုပြီး ရေးသားထားတဲ့အတွက် `Student` Constructor ကို အသုံးပြုပြီး Object တည်ဆောက်တိုင်းမှာ `Person` Constructor ကိုလည်း ခေါ်ယူအလုပ်လုပ်ပေးသွားမှာဖြစ်

ပါတယ်။ `call()` Function ထဲမှာ `this` ကို လက်ဆင့်ကမ်းပေးထားတာကို သတိပြုပါ။ ဒီလိုလက်ဆင့်ကမ်းပေးထားတဲ့ အတွက် `Person` Constructor အလုပ်လုပ်တဲ့အခါ အသုံးပြုတဲ့ `this` က `Person` ရဲ့ `this` မဟုတ်တော့ပဲ `Student` ရဲ့ `this` ကို အသုံးပြု အလုပ်လုပ်သွားစေမှာဖြစ်ပါတယ်။ ဒါကြောင့် `Person` Constructor ထဲမှာ `this` အတွက်သတ်မှတ်ထားတဲ့ သတ်မှတ်ချက်တိုင်းဟာ `Student` ရဲ့ `this` ကို သက်ရောက်သွားပြီး၊ `Person` ရဲ့ လုပ်ဆောင်ချက်အားလုံးကို `Student` က ဆက်ခံရရှိ သွားမှာပဲဖြစ်ပါတယ်။

Inheritance ရရှိသွားတဲ့အတွက် Polymorphism လုပ်ဆောင်ချက်လည်း တွဲဖက်ရရှိသွားနိုင်မှာပဲဖြစ်ပါတယ်။

Interface

JavaScript မှာ Interface လုပ်ဆောင်ချက် မပါဝင်ပါဘူး။ Interface ဆိုတာဟာ အသေးစိတ် လုပ်ဆောင်ချက်ကွာခြားပေမယ့် အပြန်အလှန် ဖလှယ်အသုံးပြုနိုင်တဲ့ Object တွေ တည်ဆောက်နိုင်ဖို့အတွက် ဘုံစံကိုသတ်မှတ်ပေးတဲ့ နည်းစနစ်တစ်မျိုးဖြစ်တယ်လို့ ပြောခဲ့ပြီးဖြစ်ပါတယ်။ Static Type Language တွေမှာ Interface ရှိမှသာ အပြန်အလှန် ဖလှယ်အသုံးပြုနိုင်တဲ့ Object တွေကို ရရှိနိုင်မှာ ဖြစ်ပေမယ့် JavaScript ကတော့ Dynamic Type Language ဖြစ်တဲ့အတွက် ဘုံစံတစ်ခု သီးခြားမလိုအပ်ပဲ၊ Object တွေကို အလိုအလျောက် အပြန်အလှန် ဖလှယ်အသုံးပြုနိုင်တဲ့အတွက်ပဲ ဖြစ်ပါတယ်။

JavaScript

```
function Student() {
    this.sayHello = function() {
        console.log("Hello, I'm a student");
     }
}

function Worker() {
    this.sayHello = function() {
        console.log("Hello, I'm a worker");
    }
}

function greet(person) {
    person.sayHello();
}

var alice = new Student();
var bob = new Worker();

greet(alice);       // => Hello, I'm a student
greet(bob);         // => Hello, I'm a worker
```

နမူနာမှာ `greet()` Function ကို ခေါ်ယူစဉ်ပေးတဲ့ Object က `Student` Object လည်းဖြစ်နိုင်သလို၊ `Worker` Object လည်း ဖြစ်လို့ရပါတယ်။ ဒီ Object နှစ်ခုဟာ Interface တစ်ခုတည်းကနေ ဆက်ခံထားခြင်းမဟုတ်ပေမယ့် အလို အလျောက် အပြန်အလှန်ဖလှယ်အသုံးပြုလို့ရနေတဲ့ သဘောပဲဖြစ်ပါတယ်။ ပေးလိုက်တဲ့ Object မှာ `sayHello()` Function ရှိဖို့သာ လိုပါတယ်။ ဒီလိုလိုအပ်တဲ့ Data နဲ့ Function တွေ ရှိနေကြောင်း သေချာဖို့ကိုတော့ ရေးသားသူ ကိုယ်တိုင်က ဂရုစိုက်ရေးသားပေးရမှာပဲဖြစ်ပါတယ်။

3.3 SOLID

SOLID Principles ရဲ့အဓိပ္ပါယ်အကျဉ်းကို ဒီအခန်းအစမှာ ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ ဒီ Principles တွေဟာ ပြုပြင်ထိမ်းသိမ်းရလွယ်တဲ့ ပရိဂရမ်တွေ ရေးသားဖန်တီးရာမှာ အဓိကကျတဲ့ သဘောတရားတွေဖြစ်ပါတယ်။ OOP ကို Code ရေးသားမှုလမ်းညွှန် လို့ဆိုနိုင်ပြီး SOLID ကိုတော့ Code Design လမ်းညွှန်လို့ ဆိုနိုင်ပါတယ်။ SOLID ကို Principle of OOD (Object Oriented Design) လို့လည်း ခေါ်ပါတယ်။ Uncle Bob လို့လူသိများတဲ့ Robert C. Martin ဆိုသူ Software ပညာရှင်တစ်ဦးက အစပြုဖော်ထုတ်ခဲ့တဲ့ သဘောတရားများဖြစ်ပါတယ်။

Single Responsibility Principle

SOLID ရဲ့ S ကို ကိုယ်စားပြုတဲ့ Single Responsibility Principle က အခုလိုဆိုပါတယ်။

" Object တစ်ခုဟာ လုပ်ငန်းတာဝန် တစ်ခုကိုသာ ဆောင်ရွက်သင့်တယ် "

Object တစ်ခုမှာ Data နဲ့ Function ပေါင်းများစွာပါဝင်နိုင်ပါတယ်။ ဒါပေမယ့် ရည်ရွယ်ချက်ကတော့ တစ်ခုတည်းပဲ ဖြစ်သင့်ပါတယ်။ Data နဲ့ Function တွေဟာ တူညီတဲ့ Object ရဲ့ အဓိက ဦးတည်ချက်ရရှိရေးအတွက် ပံ့ပိုးပေးရန် အတွက်သာ ဖြစ်သင့်ပါတယ်။ Object ရဲ့ အဓိကဦးတည်ချက်နဲ့ မသက်ဆိုင်တဲ့ လုပ်ဆောင်ချက်ကို ဆောင်ရွက်ပေးတဲ့ Data တွေ Function တွေ မပါဝင်သင့်ပါဘူး။

Pseudo-code

```
function Student(name, major) {
    this.name = name;
    this.major = major;
    this.signRollCall = function() { this.rollCall++ } ;

    this.displayInfo = function() {
        return "<b>" + this.name + "</b>" +
                ": <i>" + this.major + "</i>";
    }
}
```

နမူနာ `Student` Constructor ရဲ့တာဝန်ကတော့ ကျောင်းသားတစ်ယောက်နဲ့ ပက်သက်တဲ့ အချက်အလက်များကို စုစည်းထားဖို့ဖြစ်ပါတယ်။ Object တည်ဆောက်စဉ်မှာ `name` နဲ့ `major` ကို Assign လုပ်ပြီး `signRollCall()` ကို ခေါ်ယူတိုင်းမှာ `rollCall` တန်ဖိုးကို တစ်တိုးပေးသွားမှာဖြစ်ပါတယ်။ ဒီလုပ်ငန်းတွေက ကျောင်းသားအချက် အလက်များစုစည်းထားဖို့ဆိုတဲ့ ဦးတည်ချက်ကိုထောက်ပံ့ပေးတဲ့ လုပ်ဆောင်ချက်များဖြစ်ပါတယ်။ `displayInfo()` Function ကတော့ ကျောင်းသားရဲ့ `name` နဲ့ `major` ကို HTML Format လုပ်ပြီးဖော်ပြစေဖို့ဖြစ်ပါတယ်။ ဒီလုပ်ဆောင် ချက်ကတော့ မူလရည်ရွယ်ချက်ကနေ သွေဖီသွားတဲ့အတွက် Single Responsibility ကို ချိုးဖောက်မိသွားပါပြီ။ `Student` Object ရဲ့ တာဝန်က ကျောင်းသားနဲ့သက်ဆိုင်တဲ့ အချက်အလက်များ စုစည်းထားဖို့အပြင် ကျောင်းသားရဲ့ `name` နဲ့ `major` ကို Format လုပ်ပြီး ဖော်ပြဖို့ဆိုတဲ့ နောက်တာဝန်တစ်ခုကိုပါ ယူထားရသလို ဖြစ်နေပါတယ်။ ကြည့်လိုက်ရင် ကျောင်းသားနဲ့ပက်သက်တဲ့ အလုပ်ပဲလို့ အလွယ်ယူဆနိုင်စရာရှိပေမယ့်၊ လက်တွေ့မှာ ဒီတာဝန်ကိုနောက် Constructor တစ်ခုနဲ့ ခွဲထုတ်လိုက်တာက ပိုကောင်းပါတယ်။

Pseudo-code

```
function Student(name, major) {
    this.name = name;
    this.major = major;
    this.signRollCall = function() {  this.rollCall++  }
}

function ViewFormat() {
    this.displayStudentInfo = function(student) {
        return "<b>" + student.name + "</b>" +
                ": <i>" + student.major + "</i>";
    }
}
```

နမူနာမှာ ကျောင်သားအချက်အလက်တွေကို HTML Format လုပ်ပြီး ဖော်ပြနိုင်တဲ့ `displayStudentInfo()` ကို `ViewFormat` လို့ခေါ်တဲ့ နောက် Constructor နဲ့ခွဲခြားထားလိုက်ပါတယ်။ ဒါကြောင့် Student က ကျောင်းသား အချက်အလက်ဆိုင်ရာလုပ်ငန်းတစ်ခုကိုသာ တာဝန်ယူပြီး လိုအပ်သလို Format လုပ်ဖော်ပြခြင်းကိုတော့ `ViewFormat` က တာဝန်ယူသွားမှာပဲဖြစ်ပါတယ်။

ဒီလိုခွဲခြားထားလိုက်တဲ့အတွက် Format အပြောင်းအလဲရှိလာတဲ့အခါ သက်ဆိုင်ရာ `ViewFormat` ကိုသာ ပြင်ဆင်ဖို့ လိုပြီး `Student` ကို ထိစရာမလိုတော့တဲ့ အားသာချက်ကိုရရှိပါတယ်။ ဒါ့အပြင် နောင်လိုအပ်လို့ XMLFormat လို့ခေါ်တဲ့ Format လို့ခေါ်တဲ့ လုပ်ဆောင်ချက်လိုအပ်လာရင်လည်း Student ကို သွားပြင်စရာမလိုပဲ၊ သီးခြား Implement လုပ်နိုင် တဲ့အားသာချက်ကိုရရှိမှာပဲဖြစ်ပါတယ်။ Single Responsibility Principle ဆိုတာ ဒီနည်းနဲ့ ပြုပြင်ထိမ်းသိမ်း ရလွယ်ကူတဲ့ Code Design ရရှိစေဖို့ လမ်းညွှန်ထားတဲ့ သဘောသဘာဝတစ်ခုပဲ ဖြစ်ပါတယ်။

Open/Close Principle

SOLID ရဲ့ O ကို ကိုယ်စားပြုတဲ့ Open/Closed Principle ရဲ့အဓိပ္ပါယ်က ဒီလိုပါ။

" Object တစ်ခုအား ဖြည့်စွက်ခြင်းကို ကြိုဆိုသင့်ပေမယ့် ပြင်ဆင်ခြင်းကို လက်မခံသင့်ပါ "

မူရင်းအဓိပ္ပါယ်က Open to extension, closed to modification ဖြစ်ပါတယ်။ လက်ရှိအလုပ်လုပ်နေတဲ့ အသေအချာ စမ်းသပ်ထားပြီးသား Object တစ်ခုကို Bug Fix လုပ်တာကလွဲရင်၊ Requirement ပြောင်းလဲသွားတဲ့ အတွက်ကြောင့် သော်လည်းကောင်း၊ အခြားအကြောင်းတစ်ခုကြောင့်သော်လည်းကောင်း နောင်မှာပြင်ဆင်ခြင်း မပြုလုပ်သင့်ဘူးဆိုတဲ့ အဓိပ္ပါယ်ပါ။ ပြင်ဆင်စရာအကြောင်းရှိလာတဲ့အခါ၊ အလုပ်လုပ်နေတဲ့ Data နဲ့ Function တွေကို ပြင်မယ့်အစား Data အသစ် Function အသစ်တွေဖြည့်စွက်ခြင်းနည်းလမ်းဖြင့်သာ လိုချင်တဲ့ လုပ်ဆောင်ချက်ရအောင် ဆောင်ရွက်သင့် တယ်လို့ဆိုတဲ့သဘော ဖြစ်ပါတယ်။

Pseudo-code

```
function Order(items) {
    this.items = items;
    this.amount = function() {
        var total = 0;
        for(item in this.items) {
            total += item.value;
        }
        return total;
    }
}
```

နမူနာကိုလေ့လာကြည့်ပါ။ ပေးလာတဲ့ Item စာရင်းကနေ ကျသင့်ငွေတွက်ပေးတဲ့ `amount()` Function ပါဝင်ပါတယ်။ နောင်မှာ လိုအပ်ချက်အရ တန်ဖိုးတွက်တဲ့အခါ မြန်မာကျပ်နဲ့ပါ တွက်ချက်ပေးစေလိုတယ်ဆိုရင် အခုလိုပြင်ဆင်လိုက်နိုင် ပါတယ်။

Pseudo-code

```
function Order(items) {
    this.items = items;
    this.amount = function(currency) {
        var total = 0;
        for(item in this.items) {
            total += item.value;
        }

        if(currency == "MMK")
            return total * getExchangeRate();
        else
            return total;
    }
}
```

`amount()` Function မှာ `currency` Parameter ထည့်ပြီး `currency` က MMK ဆိုရင် တစ်မျိုးတွက်ပြီး၊ မပါရင် တစ်မျိုးတွက်အောင် ပြင်ဆင်လိုက်ခြင်း ဖြစ်ပါတယ်။ လိုအပ်တဲ့လုပ်ဆောင်ချက် ရသွားမှာဖြစ်ပေမယ့် မူလအလုပ်လုပ် နေပြီးသား Function ကို ပြင်ဆင်လိုက်ခြင်းအားဖြင့် Open/Closed Principle ကို ချိုးဖောက်လိုက်ခြင်းပဲဖြစ်ပါတယ်။ ပိုမိုမှန်ကန်တဲ့ နည်းလမ်းကတော့ အခုလိုဖြစ်မှာပါ။

Pseudo-code

```
function Order(items) {
    this.items = items;
    this.amount = function() {
        var total = 0;
        for(item in this.items) {
            total += item.value;
        }
        return total;
    }

    this.kyatAmount = function() {
        var total = this.amount();
        return total * getExchangeRate();
    }
}
```

လက်ရှိအလုပ်လုပ်နေတဲ့ `amount()` Function ကို မပြင်ပဲ `kyatAmount()` Function ဖြည့်စွက်လိုက်ခြင်းဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Kyat Amount လိုတဲ့နေရာမှာ `amount()` Function အစား `kyatAmount()` Function ကို အစားထိုးအသုံးပြုနိုင်မှာပဲ ဖြစ်ပါတယ်။

Open/Closed Principle ဟာ SOLID မှာ လိုက်နာဖို့အခက်ဆုံး တစ်ခုဖြစ်ပါတယ်။ လက်တွေ့မှာ ရှိနေတဲ့ Code ကို လုံးဝမပြင်ပဲ ပြောင်းလဲသွားတဲ့ Requirement ရဲ့လိုအပ်ချက်ကို ပြည့်စုံစေဖို့ မလွယ်ပါဘူး။ အနည်းနဲ့အများတော့ ပြင်ဆင်ဖို့ လိုအပ်တက်ပါတယ်။ တက်နိုင်သလောက် သတိပြုလိုက်နာရမှာပဲဖြစ်ပါတယ်။ ပြီးခဲ့တဲ့အခန်းမှာ လုပ်ဆောင်ချက်တစ်ခုကို ပြင်လိုက်လို့ အခြားမသက်ဆိုင်ဘူးလို့ ထင်ရတဲ့ လုပ်ဆောင်ချက်မှာ ပြဿနာသွားတက်နေတက်ကြောင်း ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ Open/Closed Principle က ဒီပြဿနာမျိုးကိုလျော့ချစေနိုင်ဖို့အတွက် အလုပ်လုပ်နေတဲ့ လုပ်ဆောင်ချက်ကို မပြင်ဖို့ တိုက်တွန်းခြင်းဖြစ်ပါတယ်။

Liskov Substitution Principle

SOLID ရဲ့ L ကို ကိုယ်စားပြုတဲ့ Liskov Substitution Principle ရဲ့အဓိပ္ပါယ်က ဒီလိုပါ။

" Inherit လုပ်ယူထားတဲ့ Sub-Class ကို မူလ Class အစား သုံးလို့ရနိုင်ရမယ် "

ဒီ Principle ကို စတင်ဖော်ထုတ်ခဲ့သူ Barbara Liskov ခေါ် ကွန်ပျူတာသိပ္ပံပညာရှင်ကို အစွဲပြုပြီး Liskov Substitution Principle လို့ခေါ်ခြင်းဖြစ်ပါတယ်။ Class တစ်ခုကို Inherit လုပ်တယ်ဆိုတာ အခြေခံအားဖြင့် မူလ Class ရဲ့ Data နဲ့ Function တွေကို ဆက်ခံရရှိလိုတဲ့အတွက်ဖြစ်ပါတယ်။ ဆက်ခံရရှိထားတဲ့ မူလ Class ရဲ့ Data နဲ့ Function တွေကို လိုအပ်သလို ပြင်ဆင်ရနိုင်ပါတယ်။ အဲ့ဒီလိုပြင်ဆင်တဲ့အခါ မူလ Class မှာ သတ်မှတ်ခဲ့တဲ့ သဘောသဘာဝကနေ လုံးဝပြောင်းလဲသွားအောင် ပြင်ဆင်ခြင်းမပြုဖို့ Liskov Substitution Principle က တားမြစ်ထားပါတယ်။ ဒါ့အပြင် Sub-Class အတွင်းမှာ မူလ Class မှာမပါတဲ့ Data နဲ့ Function တွေကို ဖြည့်စွက်ဖို့လည်း လိုအပ်နိုင်ပါတယ်။ အဲ့ဒီလိုဖြည့် စွက်ရာမှာလည်း၊ ဖြည့်စွက်လိုက်တဲ့ လုပ်ဆောင်ချက်တွေက မူလ Class ရဲ့ သဘောသဘာဝကနေ လုံးဝ ကွဲပြားသွားစေတဲ့ ဖြည့်စွက်မှုမျိုး မလုပ်ဖို့ကိုလည်း တားမြစ်ထားပါတယ်။

ဒီတော့မှ၊ Class က ရရှိလာတဲ့ Object ကို အသုံးပြုရမယ့်နေရာမှာ၊ လိုအပ်ချက်အရ Sub-Class ကနေရရှိလာတဲ့ Object နဲ့ အစားထိုးအသုံးပြုမယ်ဆိုရင် ပြဿနာတစ်စုံတစ်ရာမရှိပဲ အသုံးပြုလို့ရစေမှာဖြစ်ပါတယ်။ Sub-Class ဟာ မူလ Class ရဲ့ လုပ်ဆောင်ချက်တွေကို ဆက်ခံထားပြီး၊ ပြင်ဆင်ဖြည့်စွက်မှုတွေကြောင့်လည်း အလုပ်လုပ်ပုံသဘာဝ ကွဲပြား သွားခြင်းမရှိတဲ့အတွက် ဖြစ်ပါတယ်။

Liskov Substitution Principle အကြောင်းပြောကြရာမှာ နမူနာပေးလေ့ရှိတာကတော့ Square Class ဖြစ်ပါတယ်။

Pseudo-code

```
function Rectangle () {
    this.setWidth = function(width) {
        this.width = width;
    }
    this.setHeight = function(height) {
        this.height = height;
    }
    this.area = function() {
        return this.width * this.height;
    }
}

function Square() {
    Rectangle.call(this);
    this.setWidth = function(width) {
        this.width = width;
        this.height = width;
    }
    this.setHeight = this.setWidth;
}
```

နမူနာမှာ `Square` Constructor ဟာ `Rectangle` ကို ဆက်ခံထားပါတယ်။ ဒါပေမယ့် `Square` ရဲ့ ထုံးစံအတိုင်း `width` နဲ့ `height` ဟာ တူညီရမှာဖြစ်လို့ `setWidth()` Function မှာ `width` နဲ့ `height` ကို တူညီအောင်သတ်မှတ် ထားသလို `setHeight()` Function ကလည်း အတူတူပဲအလုပ်လုပ်ရမှာဖြစ်လို့ `setWidth()` ကိုပဲ ပြန်လည်ရယူ အသုံးပြုထားပါတယ်။ ဒါဟာ မူလလုပ်ဆောင်ချက်ကနေ သွေဖီသွားတဲ့အတွက် Liskov Substitution Principle ကို ချိုး ဖောက်လိုက်ခြင်းပဲဖြစ်ပါတယ်။ `Rectangle` Object အသုံးပြုရမယ့်နေရာမှာ `Square` Object ကို အစားထိုးအသုံးပြုခဲ့မယ့် ဆိုရင်လည်း ပြဿနာရှိနိုင်ပါတယ်။ Rectangle လို့သဘောထားပြီး Width နဲ့ Height ကို သီးခြားစီ သတ်မှတ်ဖို့ ကြိုးစားပေမယ့် လက်တွေ့မှာ သတ်မှတ်နိုင်မှာ မဟုတ်လို့ မလိုလားအပ်တဲ့ Bug တွေ ထွက်ပေါ်လာမှာပဲဖြစ်ပါတယ်။

Interface Segregation Principle

SOLID ရဲ့ I ကို ကိုယ်စားပြုတဲ့ Interface Segregation Principle ရဲ့အဓိပ္ပါယ်က ဒီလိုပါ။

" လုပ်ဆောင်ချက်ပေါင်းများစွာပါဝင်တဲ့ Interface ကြီးတစ်ခုအစား၊ ကဏ္ဍအလိုက် ခွဲခြားထားတဲ့ သီးသန့် Interface ငယ်များကိုသာ အသုံးပြုသင့်တယ် "

3 Pin Head, 2 Pin Head, USB 2.0 Port, USB 3.0 Port စသဖြင့် အပေါက်များစွာပါဝင်တဲ့ Power Outlet Interface ကြီး တစ်ခုကို မျက်စိထဲမြင်ကြည့်လို့ရပါတယ်။ Single Responsibility Principle အမြင်အရကြည့်မယ်ဆိုရင် Power Outlet ဆိုတဲ့ လုပ်ဆောင်ချက် တစ်ခုတည်းကို ဆောင်ရွက်နေခြင်းဖြစ်ပေမယ့်၊ Port အမျိုးအစားပေါင်းများစွာကို Support လုပ်ပေးတဲ့ Interface ကြီးတစ်ခုဖြစ်နေပါတယ်။ USB Power Outlet လုပ်ဆောင်ချက်ရဖို့အတွက် အဲ့ဒီ Interface ကြီးကိုသုံးမယ်ဆိုရင် ကျန် Outlet တွေက အသုံးမလိုပဲနဲ့ ပါဝင်နေတဲ့သဘော ဖြစ်နေပါတယ်။ တစ်ခြား Outlet တစ်ခုမှာ Short Circuit ဖြစ်နေလို့ ကျန် Outlet တွေပါ သုံးမရတော့ဘူးဆိုတာမျိုး ဖြစ်နိုင်ပါတယ်။ Interface Segregation Principle က Client (Object ကို ရယူအသုံးပြုသူ) ဟာ လိုချင်တဲ့လုပ်ဆောင်ချက်ရဖို့အတွက် မလိုတဲ့ လုပ်ဆောင်ချက်တွေကို မှီခိုနေရာတာမျိုး မဖြစ်သင့်ဘူးလို့ ဆိုထားပါတယ်။ ဒါကြောင့် Interface ကြီးတစ်ခုအစား ကဏ္ဍအလိုက် ခွဲခြားထားတဲ့ သီးသန့် Interface များကိုသာ အသုံးပြုသင့်တယ်ဆိုတဲ့ အဓိပ္ပါယ်လည်း ဖြစ်ပါတယ်။

Interface ရေးထုံးပါဝင်တဲ့ OOP Language တွေမှာ Interface တွေ Design လုပ်ကတည်းက ဒီအချက်ကို ထည့်သွင်း စဉ်းစားသင့်ပြီး JavaScript လို Interface ရေးထုံး မပါဝင်တဲ့ Language တွေမှာတော့ Object တစ်ခုချင်းစီမှာပါဝင်တဲ့ လုပ်ဆောင်ချက်တွေကို သတိပြု ကန့်သတ်ခြင်းအားဖြင့် Interface Segregation လုပ်ဆောင်ချက်ကို ရရှိနိုင်ပါတယ်။

Pseudo-code

```
function Car() {
  this.engine = function() { ... }
  this.break = function() { ... }
  this.playRadio = function() { ... }
  this.ejectCD = function() { ... }
}
```

နမူနာ `Car` Constructor မှာ `engine()`, `break()` လုပ်ဆောင်ချက်များအပြင် Audio ပိုင်းဆိုင်ရာ လုပ်ဆောင် ချက်များကိုပါ ပေါင်းစပ်ထည့်သွင်းထားပါတယ်။ Interface Segregation Principle အရ အခုလို ပြုပြင်လိုက်နိုင်ပါတယ်။

Pseudo-code

```
function SpeedControl() {
  this.engine = function() { ... }
  this.break = function() { ... }
}
function AudioControl() {
  this.playRadio = function() { ... }
  this.ejectCD = function() { ... }
}

function Car() {
  this.speed = new SpeedControl();
  this.audio = new AudioControl();
}
```

`SpeedControl` နဲ့ `AudioControl` ဆိုတဲ့ Interface နှစ်ခု ခွဲထုတ်လိုက်ပြီးမှ အဲ့ဒီ Interface နှစ်ခုကို `Car` Constructor မှာ ရယူအသုံးပြုထားခြင်းဖြစ်ပါတယ်။ ဒီနည်းနဲ့ `SpeedControl` နဲ့ `AudioControl` တို့ကို သီးခြား စီ Maintain လုပ်နိုင်မှာ ဖြစ်သလို Audio လုပ်ဆောင်ချက်မလိုတဲ့အခါမှာလဲ ချန်လှပ်ထားနိုင်မှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Client Interface တွေ ခွဲထုတ်၊ အမှန်တစ်ကယ်လိုအပ်တဲ့ Interface များကိုသာ ချိတ်ဆက်အလုပ် လုပ်စေခြင်းအားဖြင့် မလိုအပ်တဲ့ လုပ်ဆောင်ချက်တွေ ပါဝင်နေခြင်းနဲ့မလိုအပ်တဲ့ လုပ်ဆောင်ချက်တွေကို မှီခိုနေရခြင်း မရှိတော့ပဲ Interface Segregation Principle လုပ်ဆောင်ချက်ကို ရရှိသွားမှာဖြစ်ပါတယ်။

Dependency Inversion Principle

SOLID ရဲ့ D ကို ကိုယ်စားပြုတဲ့ Dependency Inversion Principle ရဲ့အဓိပ္ပါယ်က ဒီလိုပါ။

" အဆင့်မြင့် Object ဟာ အဆင့်နိမ့် Object ရဲ့ လုပ်ဆောင်ချက်ကို မမှီခိုသင့်ဘူး "

အဆင့်မြင့် (Higher Level) နဲ့ အဆင့်နိမ့် (Lower Level) ဆိုတဲ့ ဆိုလိုရင်းကိုမြင်သာစေဖို့အတွက် Code နမူနာကို ကြည့်ပါ။

Pseudo-code

```
function PDF() {
    this.read = function() {
     // parse and read PDF file
    }
}

function Reader() {
    this.book = new PDF();
    this.read = this.book.read();
}
```

နမူနာမှာ `PDF` ဟာပိုပြီးအခြေခံကျတဲ့ အဆင့်နိမ့် Object ဖြစ်ပါတယ်။ `Reader` ကတော့ `PDF` ကို အသုံးချထားတဲ့ အဆင့်မြင့် Object ဖြစ်ပါတယ်။ အဆင့်မြင့် Object ဖြစ်တဲ့ `Reader` ရဲ့ `read()` Function ဟာ လက်တွေ့

မှာ အဆင့်နိမ့် Object ဖြစ်တဲ့ PDF ရဲ့ read() ကို ပြန်လည်အသုံးပြုထားခြင်းဖြစ်ပါတယ်။ ဒါကြောင့် အဆင့်မြင့် Object ဖြစ်တဲ့ Reader ဟာ အဆင့်နိမ့် Object ဖြစ်တဲ့ PDF ရဲ့ အသေးစိတ်လုပ်ဆောင်ချက်ကို မှီခိုနေတဲ့သဘော ဖြစ်လာပါတယ်။ ဒီလိုမှီခိုနေတဲ့အတွက် ဖြစ်ပေါ်လာတဲ့ အားနည်းချက်ကတော့ ပရိဂရမ်ကို ပြင်ဆင်ရခက်ခဲသွားခြင်းပဲ ဖြစ်ပါတယ်။ Reader ဟာ PDF က လွဲရင် တစ်ခြား File အမျိုးအစားနဲ့ အလုပ်လုပ်နိုင်မှာ မဟုတ်တော့ပါဘူး။ PDF ကို လုံးလုံးမှီခိုထားတဲ့အတွက်ကြောင့်ဖြစ်ပါတယ်။ ဒါကြောင့် Dependency Inversion Principle က သတ်မှတ်ထားတဲ့ အတိုင်း အခုလို ပြင်ဆင်လိုက်နိုင်ပါတယ်။

Pseudo-code

```
function PDF() {
    this.read = function() {
        // parse and read PDF file
    }
}

function Epub() {
    this.read = function() {
        // parse and read epub file
    }
}

function Reader(book) {
    this.book = book;
    this.read = this.book.read();
}
```

နမူနာမှာ PDF နဲ့ Epub ဆိုတဲ့ Low Level Object နှစ်ခုပါဝင်ပြီး Reader ဆိုတဲ့ Hight Level Object တစ်ခုပါဝင် ပါတယ်။ Reader ဟာ လက်တွေ့မှာ PDF နဲ့ Epub တို့ရဲ့ read() Function ကို အခြေခံပြီးအလုပ်လုပ်ခြင်း ဖြစ်ပေမယ့် PDF သို့မဟုတ် Epub တစ်ခုခုကို အသေးစိတ်မှီခိုခြင်းမရှိပါဘူး။ အဲ့ဒီလိုမှီခိုမယ့်အစား Reader Object တည် ဆောက်စဉ်မှာ PDF သို့မဟုတ် Epub အသုံးပြုစေလိုတဲ့ Object ကိုပေးဖို့ သတ်မှတ်ထားပြီး ပေးလာတဲ့ Object ရဲ့ read() Function ကို အသုံးပြုလိုက်ခြင်းအားဖြင့် Reader ဟာ PDF သို့မဟုတ် Epub သို့မဟုတ် အခြား File Format တွေကိုဖတ်ရှုပေးနိုင် Object တစ်ခုဖြစ်သွားပါတယ်။ Dependency Inversion အကူအညီနဲ့ ပရိဂရမ်ရဲ့အလုပ် လုပ်ပုံ အများကြီးပိုပြီး Flexible ဖြစ်သွားတဲ့ သဘောပဲ ဖြစ်ပါတယ်။

Pseudo-code

```
var pdf = new PDF();
var PDFReader = new Reader(pdf);

var epub = new Epub();
var EpubReader = new Reader(epub);
```

Interface ရေးထုံးပါဝင်တဲ့ Language တွေမှာတော့ Dependency Inversion လုပ်ဆောင်ချက် ရရှိဖို့ Interface တွေကို အားကိုးကြလေ့ရှိပါတယ်။ အဆင့်မြင့် Object က အဆင့်နိမ့် Object ကို မမှီခိုပဲ ကြားခံ Abstraction ဖြစ်တဲ့ Interface ကိုမှီခိုလိုက်ခြင်းအားဖြင့် Dependency Inversion လုပ်ဆောင်ချက်ကို ရရှိနိုင်ခြင်းပဲဖြစ်ပါ

တယ်။ လက်တွေ့မှာ ဘယ်လို နည်းစနစ်ကိုပဲသုံးသုံး၊ အဆင့်မြင့် Object က အဆင့်နိမ့် Object ရဲ့ Detail အသေးစိတ်လုပ်ဆောင်ချက်ကို မမှီခိုဘူး ဆိုရင် Dependency Inversion Principle ကို လိုက်နာနေခြင်းပဲဖြစ်ပါတယ်။

Conclusion

ဒီအခန်းမှာဖော်ပြခဲ့တဲ့ Code နမူနာတွေဟာ လက်တွေ့အသုံးချနမူနာတွေမဟုတ်ပဲ၊ OOP, SOLID စတဲ့ သဘောသဘာဝတွေ ပေါ်လွင်အောင် အသားပေးဖော်ပြထားတဲ့ ရိုးစင်းသောနမူနာ Code များသာဖြစ်ပါတယ်။ လက်တွေ့မှာ Code တွေ က ဒီလောက် ရိုးစင်းမှာမဟုတ်တဲ့အတွက် ဘယ်လောက်ထိ လိုက်နာအသုံးချနိုင်သလဲ ဆိုတာ ရေးသားသူရဲ့ အတွေ့အကြုံ ပေါ်မူတည်မှာပဲ ဖြစ်ပါတယ်။ ပြောခဲ့တဲ့ နည်းစနစ်နဲ့ သဘောသဘာဝ အားလုံးကို တစ်သဝေမတိမ်းလိုက်နာပြီး အကုန်လုံး အတိအကျ လိုက်လုပ်နေဖို့ မဟုတ်ပါဘူး။ သူ့နေရာနဲ့သူ သင့်တော်သလို အသုံးချသွားနိုင်မယ်ဆိုရင် စနစ်ကျပြီး ပြုပြင်ထိမ်း သိမ်းရလွယ်ကူတဲ့ ပရိဂရမ်တွေ ရေးသားဖန်တီးရာမှာ အထောက်အကူဖြစ်စေမှာပဲဖြစ်ပါတယ်။

ပြန်လည်အသုံးပြုလို့ရတဲ့ Reusable Code ဟာ
အသုံးဝင်ပါတယ်။ ပြန်လည်အသုံးပြုလို့ရတဲ့
Reusable Service ကတော့ ပိုအသုံးဝင်ပါတယ်။

Professional Web Developer (စာအုပ်)
Web Standard, jQuery, PHP, MySQL, Ajax, CMS, MVC,
HTML5, Mobile Web, Web Application Security စသည့်
အကြောင်းအရာများကို ရေးသားဖော်ပြထားသည့်စာအုပ်
အောက်ပါလိပ်စာတွင် အခမဲ့ Download ရယူနိုင်သည်။

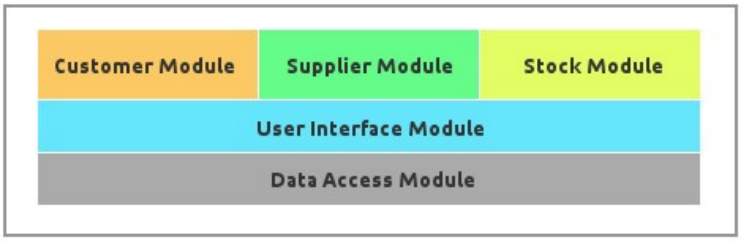
http://pwdbook.com

အခန်း(၄) – Service Oriented Architecture

SOA လို့ အတိုကောက်ခေါ်တဲ့ Service Oriented Architecture ဆိုတာဟာ Software Solution တွေ တည်ဆောက်ရာမှာ တစ်ပေါင်းတစ်စည်းတည်း စုစည်းအလုပ်လုပ်တဲ့ Software Package အဖြစ် တည်ဆောက်မယ့်အစား သီးခြား အလုပ်လုပ်နိုင်တဲ့ Software ငယ်များအဖြစ် ခွဲခြားတည်ဆောက်တဲ့ နည်းစနစ်ဖြစ်ပါတယ်။

Software Solution တစ်ခုကို ကဏ္ဍအလိုက် ခွဲခြားတည်ဆောက်တယ်ဆိုတာ အမှန်တော့ ထူးဆန်းတဲ့ကိစ္စတစ်ခုမဟုတ်ပါဘူး။ ဥပမာ - စီးပွားရေးလုပ်ငန်းသုံး Business Solution တစ်ခုတည်ဆောက်တယ်ဆိုပါစို့။ Customer စာရင်းစီမံတဲ့ အစိတ်အပိုင်းကသပ်သပ်၊ Supplier စာရင်းစီမံတဲ့ အစိတ်အပိုင်းကသပ်သပ်၊ Order စာရင်းစီမံတဲ့အစိတ်အပိုင်းက သပ်သပ်၊ Payment ပိုင်းစီမံတဲ့အစိတ်အပိုင်းကသပ်သပ်၊ Stock ပိုင်းစီမံတဲ့ အစိတ်အပိုင်းကသပ်သပ် စသဖြင့် အပိုင်းလိုက်၊ အပိုင်းလိုက် ခွဲခြားရေးသားပြီး အဲ့ဒီအစိတ်အပိုင်းတွေကို ပေါင်းစပ်ခြင်းအားဖြင့် လိုချင်တဲ့ Software Solution ရအောင် ဖန်တီးယူနိုင်ပါတယ်။

ဒီနည်းစနစ်ကို Modular Design လို့ခေါ်ပါတယ်။ Modular Design ကိုအသုံးပြုခြင်းအားဖြင့် ခွဲခြားထားတဲ့ အပိုင်းလိုက် သီးခြားစီ စမ်းသပ်နိုင်ခြင်း၊ သီးခြားစီ ပြင်ဆင်ထိမ်းသိမ်းနိုင်ခြင်းစတဲ့ အားသာချက်တွေကိုရရှိနိုင်ပါတယ်။ ဒါအပြင် Module တွေကို အမျိုးမျိုးတွဲစပ်ခြင်းအားဖြင့် မတူကွဲပြားတဲ့ Solution တွေကိုလည်း ရရှိနိုင်ပါသေးတယ်။ ဥပမာ – တစ်ချို့ Solution တွေ မှာ Customer, Supplier နဲ့ Stock တို့သာ ပါဝင်ပြီး တစ်ချို့ Solution တွေမှာ Stock, Order, Payment တို့ပဲ ပါဝင်တဲ့ သဘောမျိုး လိုအပ်ရင်ရရှိနိုင်ပါတယ်။

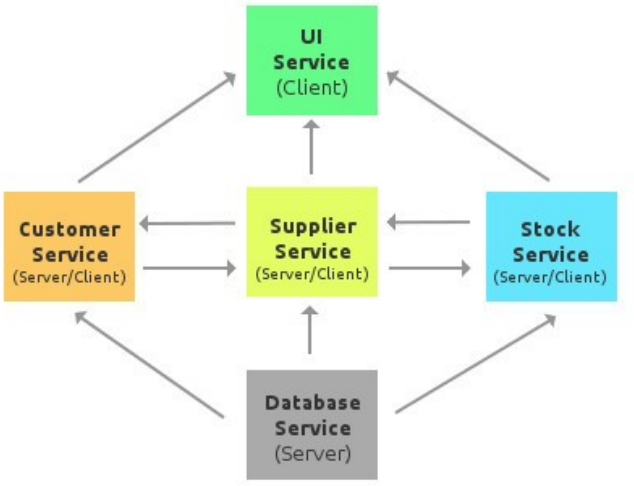


ပုံ (၄.၁) - Modular Design

Modular Design နဲ့ SOA မတူပါဘူး။ Modular Design မှာပါဝင်တဲ့ အစိတ်အပိုင်းတွေကို Module လို့ခေါ်ပြီး SOA မှာ ပါဝင်တဲ့ အစိတ်အပိုင်းတွေကို Service လို့ခေါ်ပါတယ်။ အခေါ်အဝေါ်အားဖြင့် Software ဆိုတာဟာ အသုံးပြုသူ User သုံးဖို့ ရည်ရွယ်ဖန်တီးထားတဲ့ ကွန်ပျူတာပရိုဂရမ်တစ်ခုဖြစ်ပါတယ်။ Service ဆိုတာကတော့ အသုံးပြုသူ User တင်မကပဲ အခြားကွန်ပျူတာပရိုဂရမ်တွေကပါ ဆက်သွယ်အသုံးပြုနိုင်တဲ့ Software တစ်မျိုးပဲ ဖြစ်ပါတယ်။ လူမလိုပဲ Machine to Machine ဆက်သွယ်အလုပ်လုပ်နိုင်တဲ့ Software တစ်မျိုးလို့လည်း ဆိုနိုင်ပါတယ်။

Modular Design နဲ့တည်ဆောက်ထားတဲ့ Solution တစ်ခု အလုပ်လုပ်ဖို့အတွက် သက်ဆိုင်ရာ Module တွေကို တွဲဆက် ပေးရပါတယ်။ SOA မှာပါဝင်တဲ့ Service တွေကတော့ တွဲဆက်ဖို့မလိုပဲ တစ်ခုချင်း သီးခြားအလုပ်လုပ် ကြပါတယ်။ Service တစ်ခုနဲ့တစ်ခု အပြန်အလှန် ဆက်သွယ်ခြင်းအားဖြင့် လိုအပ်တဲ့ Solution ကိုရရှိတဲ့ သဘောဖြစ်ပါတယ်။ SOA မှာ အစိတ်အပိုင်း (၃)ရပ်၊ ပါဝင်လေ့ရှိပါတယ်။ Service Provider (Server), Service Consumer (Client) နဲ့ Interface (Protocol) တို့ပဲဖြစ်ပါတယ်။

Service Provide သို့မဟုတ် Server ဆိုတာဟာ သူ့ရဲ့လုပ်ဆောင်ချက်တွေကို အခြားပရိုဂရမ်တစ်ခုကနေ ရယူ အသုံးပြု ခွင့်ပေးထားတဲ့ ပရိုဂရမ်တစ်မျိုးဖြစ်ပါတယ်။ Service Consumer သို့မဟုတ် Client ဆိုတာကတော့ Server ပရိုဂရမ်က ပေးထားတဲ့ လုပ်ဆောင်ချက်တွေကို ရယူအသုံးပြုထားတဲ့ ပရိုဂရမ်တစ်မျိုးဖြစ်ပါတယ်။ SOA နည်းစနစ်မှာပါဝင်တဲ့ Service တွေမှာ တစ်ချို့ Service တွေက Server ဖြစ်ပြီး တစ်ချို့ Service တွေ ကတော့ Client တွေဖြစ်ကြပါတယ်။ Service တစ်ခုဟာ Server အနေနဲ့ ရော Client အနေနဲ့ပါ အလုပ်လုပ်မယ် ဆိုရင်လည်း လုပ်နိုင်ပါတယ်။ Interface သို့မဟုတ် Protocol ဆိုတာကတော့ Service တွေ အချင်းချင်း အပြန်အလှန်ဆက်သွယ်ရမှာ စံထားပြီးလိုက်ရမယ့် Communication Standard တစ်မျိုးပဲဖြစ်ပါတယ်။ Server တွေ Client တွေ ဟာ သတ်မှတ်ထားတဲ့ Interface ကို အသုံးပြုခြင်းအားဖြင့် အပြန်အလှန် ဆက်သွယ်နိုင်ကြ ခြင်းပဲ ဖြစ်ပါတယ်။



ပုံ (၄.၂) - Service Oriented Architecture

4.1 – Service Layers

Software Solution ကို SOA နည်းစနစ်အတိုင်း Service များအဖြစ် ခွဲပြီးတည်ဆောက်တော့မယ်ဆိုရင် ဘယ်လို အပိုင်း၊ ဘယ်လို လုပ်ဆောင်ချက်တွေကို သီးခြား Service အဖြစ် တည်ဆောက်သင့်သလဲဆိုတဲ့အချက်က စဉ်းစားစရာဖြစ်လာ ပါတယ်။ အခြေခံအားဖြင့် Service Layer သုံးဆင့်ရှိတယ်လို့ ဆိုနိုင်ပါတယ်။ **ပုံ (၄.၂)** မှာ လည်း လေ့လာကြည့်နိုင် ပါတယ်။

Utility Services

အခြေခံအကျဆုံး Service Layer ကတော့ **Utility Service** များဖြစ်ပါတယ်။ ဒီအဆင့်မှာ Database နဲ့ ဆက်သွယ် အလုပ်လုပ်ပေးနိုင်တဲ့ Service တွေ၊ File Read/Write အပါအဝင် File System ပိုင်းဆိုင်ရာအလုပ်တွေ လုပ်ပေးနိုင်တဲ့ Service တွေ၊ User Action တွေ Event တွေကို Log လုပ်ပေးနိုင်တဲ့ Logging Service တွေ၊ ကွန်ပျူတာ Hardware နဲ့ Resource တွေကို စီမံပေးနိုင်တဲ့ Service တွေ၊ Data Format တွေ အပြန်အလှန် ပြောင်းပေးနိုင်တဲ့ Encoding Service တွေနဲ့ အခြားအခြေခံလုပ်ဆောင်ချက်ပိုင်းဆိုင်ရာ Service တွေ ပါဝင်လေ့ ရှိပါတယ်။

Utility Service တွေဟာ အများအားဖြင့် Server အနေနဲ့အလုပ်လုပ်ကြပြီး အခြေခံအားဖြင့် Application Agnostic ဖြစ်ကြပါတယ်။ Application Agnostic ဆိုတာကတော့ Project တစ်ခုအတွက်တင်မကပဲ အခြား Project များ၊ အခြား Application များမှာပါ ပြန်လည်အသုံးပြုနိုင်တဲ့ အခြေခံ Foundation Service များပဲဖြစ်ကြပါတယ်။

Core Services

တည်ဆောက်လိုတဲ့ Software Solution ရဲ့ Common Logic သို့မဟုတ် Key Logic အဖြစ်အလုပ်လုပ်မယ့် Service တွေကတော့ Core Service များပဲဖြစ်ပါတယ်။ ဥပမာ – Customer Management Service, Supplier Management Service, Stock Management Service, Order Management Service, Payment Service စသဖြင့် Solution လိုအပ်ချက်အလိုက် တည်ဆောက်ထားရတဲ့ Service များပဲဖြစ်ပါတယ်။

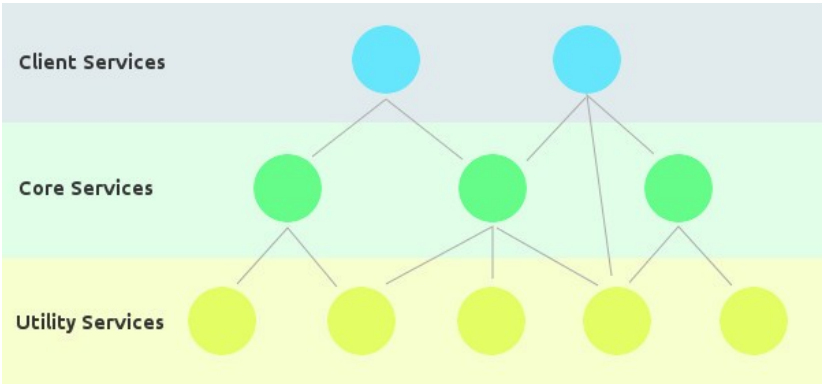
Core Service တွေဟာ Utility Service တွေလိုတော့ Application Agnostic ဖြစ်မှာမဟုတ်တော့ပါဘူး။ သက်ဆိုင်ရာ Solution အတွက်ရည်ရွယ်တည်ဆောက်ထားတဲ့ Service များဖြစ်တဲ့အတွက် သက်ဆိုင်ရာ Solution အတွက်သာ အသုံးဝင်တဲ့ Service များပဲဖြစ်ပါတယ်။ Core Service တွေဟာ Client အနေနဲ့ရော၊ Server အနေနဲ့ပါ အလုပ်လုပ်လေ့ ရှိကြပါတယ်။

ဥပမာ – Core Service တစ်ခုဖြစ်တဲ့ Customer Management Service ဟာ Customer တွေရဲ့ အချက်အလက်တွေကို သိမ်းဆည်းခြင်းလုပ်ငန်းအတွက် Utility Service ဖြစ်တဲ့ Database Service ရဲ့လုပ်ဆောင်ချက်တွေကို ရယူ အသုံးပြုဖို့လိုနိုင်ပါတယ်။ တစ်ချိန်တည်းမှာပဲ၊ အခြား Service တွေဖြစ်ကြတဲ့ Report Service, Account Service စတဲ့ Client Service တွေအတွက် လိုအပ်တဲ့လုပ်ဆောင်ချက်တွေကိုလည်း ပံ့ပိုးပေးနိုင်ပါတယ်။ ဒီတော့ Core Service တွေ ဟာ Utility Service များနဲ့ Client Service များရဲ့ ကြားခံနေရာကနေ အလုပ်လုပ်ပေးတဲ့ Service များလို့လည်း ဆိုနိုင် ပါတယ်။

Client Services

Services တွေထဲမှာ အသုံးပြုသူ User နဲ့ တိုက်ရိုက်ထိတွေ့ဖို့လိုတဲ့ Service တွေကတော့ Client Service များပဲဖြစ်ပါတယ်။ လုပ်ဆောင်ချက်အားဖြင့် လိုအပ်သလို ပြန်လည်ရယူအသုံးပြုနိုင်ဖို့ထက် လုပ်ငန်းတစ်ခုတည်းကို တိတိကျကျ ဆောင်ရွက်နိုင်ဖို့ ရည်ရွယ်ဖန်တီးထားတဲ့ Service များဖြစ်ကြပါတယ်။ ဥပမာ – Invoice Service တစ်ခုဟာ Invoice Document တွေ တည်ဆောက်ပေးတဲ့ လုပ်ငန်းတစ်ခုတည်းကိုသာလုပ်ပေးတဲ့ Client Service ဖြစ်နိုင်ပါတယ်။ အဲ့ဒီ Invoice Service အလုပ်လုပ်နိုင်ဖို့အတွက် Core Service တစ်ခုဖြစ်တဲ့ Payment

Service ကပေးထားတဲ့ Tax Calculation, Currency Conversion စတဲ့ လုပ်ဆောင်ချက်တွေကို ရယူအသုံးပြုဖို့ လိုနိုင်ပါတယ်။ ဒါ့အပြင် Invoice File တွေကို သိမ်းဆည်းဖို့အတွက် Utility Service တစ်ခုဖြစ်တဲ့ File System Service ရဲ့ လုပ်ဆောင်ချက်တွေကိုလည်း ရယူ အသုံးပြုဖို့လိုနိုင်ပါသေးတယ်။



ပုံ (၄.၃) - Service Layers

SOA Solution တွေမှာ ရှိတက်လေ့ရှိတဲ့ Service Layers တွေကို ဥပမာ အနေနဲ့ဖော်ပြခြင်းသာဖြစ်ပါတယ်။ SOA ဟာ Architecture Design Concept တစ်ခုသာဖြစ်လို့ လက်တွေ့အသုံးချရာမှာတော့ မိမိလိုအပ်ချက်နဲ့သင့်တော်သလို အသုံးချနိုင်ပါတယ်။ တည်ဆောက်လိုတဲ့ Solution အမျိုးအစားနဲ့ Requirement Specification တွေကွဲပြားသွားတာနဲ့အမျှ SOA Solution တစ်ခုမှာ ပါဝင်ဖွဲ့စည်းတက်တဲ့ Service Layer တွေ Service အမျိုးအစားတွေ၊ ကွဲပြားသွားမှာဖြစ် ပါတယ်။

4.2 – Service Types

SOA Solution တစ်ခုကို Design လုပ်ရာမှာ အထောက်အကူဖြစ်စေဖို့နဲ့ အိုင်ဒီယာရစေဖို့အတွက် ရှိတက်တဲ့ Service အမျိုးအစားများတွေကိုလည်း ဖော်ပြပေးလိုက်ပါတယ်။

User Interface Services – User Interface Service တွေဟာအသုံးပြုသူ User နဲ့ တိုက်ရိုက်ထိတွေ့ရတဲ့ Service အမျိုးအစားများဖြစ်ပြီး အသုံးပြုသူပေးတဲ့ Input Data တွေကို လက်ခံရယူခြင်း၊ အသုံးပြုသူရဲ့ Request သို့မဟုတ် Event တွေကို တုန့်ပြန်ခြင်းစတဲ့ လုပ်ငန်းများကို ဆောင်ရွက်ပေးတဲ့ Service များပဲ ဖြစ်ပါတယ်။ Service Layer အရ ဆိုရင် Client Layer မှာပါဝင်တဲ့ Service များဖြစ်ကြပါတယ်။ အများအားဖြင့် အခြား Service များကပေးတဲ့ လုပ်ဆောင်ချက်များကိုရယူပြီး၊ အသုံးပြုသူ User နားလည်ထိတွေ့နိုင်တဲ့ အသွင်အပြင်ဖြစ်အောင် User Interface တစ်ခုနဲ့ဖော်ပြပေးတဲ့ Service များဖြစ်ကြပါတယ်။

Policy Services – User Authentication, Authorization နဲ့ Role Management တို့လို လုပ်ဆောင်ချက်တွေဟာ Policy Service တွေဖြစ်ပါတယ်။ Access Right ကန့်သတ်တားတဲ့ Service တွေကို ဆက်သွယ်ဖို့အတွက် User ID, Password စတဲ့ အချက်အလက်တွေကို ပေးပို့ရမယ့်နည်းလမ်းတွေ၊ User Role အလိုက် ခွင့်ပြုထားတဲ့ လုပ်ဆောင်ချက် တွေကိုလည်း Policy Service တွေမှာ ထည့်သွင်းသတ်မှတ်ထားနိုင်ပါတယ်။

Process Services – Process Service တွေကိုတော့ Controller Service လို့လည်း ခေါ်နိုင်ပါတယ်။ ဥပမာ – UI Service တစ်ခုက Data Service တစ်ခုထံက အချက်အလက်ကိုတောင်းခံတဲ့အခါ Process Service ကနေ တစ်ဆင့် တောင်းခံဖို့ လိုနိုင်ပါတယ်။ ဒီတော့မှ Process Service က Policy Service မှာ သတ်မှတ်ထားတဲ့ သတ်မှတ်ချက်များနဲ့ တိုက်ဆိုင်စစ်ဆေးပြီး Policy နဲ့ကိုက်ညီရင် လက်ဆင့်ကမ်းဆက်သွယ်ပေးပြီး၊ Policy နဲ့ မကိုက်ညီရင်တော့ လက်ဆင့် ကမ်းဆက်သွယ်ပေးဖို့ ငြင်းပယ်သွားနိုင်မျာဖြစ်ပါတယ်။ ဒီလို Process Service ကြားခံမထားပဲ UI Service က Policy ကို တိုက်ရိုက်ဆက်သွယ်ပြီး ဆုံးဖြတ်လို့လည်း ရနိုင်ပါတယ်။ ဒါပေမယ့် နောင်မှာ Policy ပြောင်းသွားတဲ့အခါ UI Service ကိုလည်း လိုက်ပြင်ရတက်လို့ ပြုပြင်ထိမ်းသိမ်းရ ပိုမိုလွယ်ကူ အောင် ကြားခံ Process Service များ အသုံးပြုကြခြင်းဖြစ်ပါတယ်။

Connector Services – SOA Solution တစ်ခုမှာပါဝင်တဲ့ Service တွေက ကွန်ပျူတာတစ်လုံးတည်းမှာ အမြဲ ရှိနေမှာ မဟုတ်ပါဘူး။ မတူကွဲပြားတဲ့ Location နဲ့ Network တွေမှာ တည်ရှိနေနိုင်တဲ့ Service တွေဖြစ်တဲ့ အတွက် Connector Service တွေက အသုံးပြုလိုတဲ့ Service တည်ရှိရာ Location ကို Resolve လုပ်ပေးခြင်း၊ ချိတ်ဆက်ဆက်သွယ်ပေးခြင်း လုပ်ငန်းများကို ဆောင်ရွက်ပေးပါတယ်။ Connector Service များရဲ့အကူအညီနဲ့ Service Location ကို လွပ်လပ်စွာ ပြောင်းနိုင်တဲ့အတွက် ပိုပြီး Flexible ဖြစ်တဲ့ Service Infrastructure ကို ရရှိ နိုင်မှာပဲဖြစ်ပါတယ်။

Infrastructure Service – Infrastructure Service တွေကတော့ Application Specific မဟုတ်ပဲ ပြန်လည် အသုံး ပြုနိုင်တဲ့ Utility Service အမျိုးအစားများဖြစ်ပါတယ်။ ဘယ် Service တွေကို CPU Process ဘယ်လောက်ထိ ခွင့်ပြုမလဲ၊ RAM ပမာဏ ဘယ်လောက်ခွင့်ပြုမလဲ စတဲ့ Service တွေရဲ့ Resource အသုံးပြုမှု ကို စီမံပေးတဲ့ Service တွေ၊ Performance ပိုကောင်းလာစေဖို့အတွက် Cache လုပ်ဆောင်ချက်ကို ဆောင်ရွက် ပေးနိုင်တဲ့ Service တွေနဲ့ များပြား လာတဲ့အချက်အလက်တွေကို လိုအပ်သလို Archive လုပ်ပေးနိုင်တဲ့ Service မျိုးတွေဟာလည်း Infrastructure Service များထဲမှာပါဝင်ပါတယ်။

Data Access Services – File System I/O Service များနဲ့ Database Service များကို Data Access Service လို့ ခေါ်နိုင်ပါတယ်။ File Read/Write လုပ်ငန်းများ၊ File Format ဆိုင်ရာလုပ်ငန်းများ၊ Data Encode/Decode လုပ်ငန်းများ၊ Data Model တည်ဆောက်ခြင်းလုပ်ငန်းများ၊ Data Relationship နဲ့ Index များစီမံခြင်းလုပ်ငန်း များ စတဲ့ File နဲ့ Data တွေကို စီမံပေးနိုင်တဲ့ Utility Service များဟာ Data Access Service အမျိုးအစားထဲမှာ ပါဝင်ပါတယ်။

Information Services – Information Service တွေကတော့ သတင်းအချက်အလက်များကို ရယူပေးခြင်း၊ ဖြန့် ဝေပေးခြင်းလုပ်ငန်းများကို ဆောင်ရွက်ပေးတဲ့ Service များဖြစ်ပါတယ်။ အဲ့ဒီလိုရယူဖြန့်ဝေရာမှာ သက်ဆိုင်ရာ ကဏ္ဍအလိုက် ပါဝင်သင့်တဲ့ သတင်းအချက်အလက်၊ ဖြစ်သင့်တဲ့ သတင်းအချက်အလက်ဖွဲ့စည်းပုံစတဲ့ Logic တွေကို ဆုံးဖြတ်ပေးတဲ့ Service ပဲဖြစ်ပါတယ်။ Reporting Service တွေဟာလည်း Information Service ထဲမှာ ပါဝင်ပါတယ်။ တနည်း အားဖြင့် အချက်အလက်ဖွဲ့စည်းပုံ Information Structure ကို သတ်မှတ်ပေးထားတဲ့ Service အမျိုးအစားဖြစ်ပါတယ်။

Monitor Services – Monitor Service တွေကတော့ အခြား Service များ တစ်ခုနဲ့တစ်ခု ဆက်သွယ်အလုပ်လုပ်မှု မှတ်တမ်းကို စီစစ်ပေးတဲ့ Service များဖြစ်ပါတယ်။ Monitor Service များရဲ့ အကူအညီနဲ့ ဘယ်လို Service တွေက အဓိကအလုပ်လုပ်ရသလဲဆိုတဲ့ အချက်အလက်တွေ၊ ဘယ်လို Service တွေက လုပ်ငန်းဆောင်ရွက်ရာမှာ Bottleneck ဖြစ်နေစေသလဲဆိုတဲ့ အချက်အလက်တွေနဲ့ ဘယ်လိုအချိန်တွေမှာ ဆက်သွယ်ဆောင်ရွက်ရမှုများတက်သလဲဆိုတဲ့အချက်အလက်တွေကို လေ့လာသိရှိရနိုင်စေမှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့စနစ်လုပ်ဆောင်ချက်ပိုကောင်းလာဖို့ ဘယ်လို Service တွေ ဖြည့်စွက်သင့်သလဲ၊ ဘယ် Service တွေကို Improve လုပ်သင့်သလဲဆိုတဲ့ အချက်အလက်တွေကို ရရှိနိုင်မှာဖြစ်ပါတယ်။ တစ်နည်းအားဖြင့် Monitor Service ဆိုတာ Solution ကလိုအပ်တဲ့ လုပ်ဆောင်ချက်ကို ဆောင်ရွက်ပေးခြင်းမဟုတ်ပဲ Service တွေကို ပြင်ဆင်ထိမ်းသိမ်းရာမှာ အထောက်အကူဖြစ်စေဖို့ ရည်ရွယ်ထားတဲ့ Service များပဲဖြစ်ပါတယ်။ လုပ်ဆောင်ချက် အဆင့်တိုင်းကို မှတ်တမ်းတင်ပေးတဲ့ Logging Service တွေဟာလည်း Monitor Service ထဲမှာ ပါဝင်ပါတယ်။

Development Services – Service တွေ တစ်ခုနဲ့တစ်ခု ဆက်သွယ်ဆောင်ရွက်ပုံကို Simulate လုပ်ပေးနိုင်တဲ့ Service တွေ၊ Service တစ်ခုရဲ့ လုပ်ဆောင်ချက်ရလဒ်နဲ့ အခြား အထွေထွေအချက်အလက် အသေးစိတ်များကို ဖော်ပြပေးနိုင် တဲ့ Debug Service တွေ၊ Service တွေ မျှော်မှန်းထားတဲ့အတိုင်း အလုပ်လုပ်မလုပ် စမ်းသပ်ပေးနိုင်တဲ့ Test Service တွေ၊ Service စာရင်းနဲ့ အသုံးပြုပုံ Implementation Detail ကို ဖော်ပြပေးနိုင်တဲ့ Directory Service တွေဟာ Development Service တွေပဲဖြစ်ပါတယ်။ Service အသစ်တစ်ခုဖန်တီးပြီးတဲ့အခါမှာ အလိုအလျောက် Directory ထဲမှာ ထည့်သွင်းပေးခြင်း၊ စမ်းသပ် ပေးသွားခြင်းနဲ့ Policy Service ထဲမှာ မှတ်တမ်းတင်ပေးခြင်းတို့လို လုပ်ငန်းတွေကို အလိုအလျောက် ဆောင်ရွက်ပေးနိုင်တဲ့ Publishing Service တွေဟာလည်း Development Service တွေပဲဖြစ်ပါတယ်။ တနည်းအားဖြင့် Development Service တွေဟာ Solution က လိုအပ်တဲ့ လုပ်ဆောင်ချက်ကို ဆောက်ရွက်ပေးခြင်း မဟုတ်ပဲ၊ အခြား Service များ တည်ဆောက်ရာမှာ အထောက်အကူဖြစ်စေဖို့ ရည်ရွယ်ထားတဲ့ Development Tool များ ဖြစ်ပါတယ်။

ပြောခဲ့တဲ့ Service အမျိုးအစားတွေထဲမှာ မပါဝင်ပဲ တိကျတဲ့လုပ်ဆောင်ချက်တစ်ခု ရရှိဖို့အတွက် ရည်ရွယ်ဖန်တီးထားတဲ့ Service တွေကိုတော့ **Application Service** လို့ ခေါ်နိုင်ပါတယ်။ Application Service တွေရဲ့ သဘောသဘာဝနဲ့ အလုပ် လုပ်ပုံကတော့ တည်ဆောက်လိုတဲ့ Solution ပေါ်မူတည်ပြီး ကွဲပြားသွားမှာပဲဖြစ်ပါတယ်။ Business Solution တစ်ခုမှာဆိုရင် Customer, Supplier, Client, Payment, Order, Stock တို့လို Service တွေ ပါဝင်နိုင်ပြီး Social Collaboration Solution တစ်ခုမှာဆိုရင်တော့ Real-time Messaging, Group Messaging, Contact List စတဲ့ Service တွေပါဝင်မှာဖြစ်ပါတယ်။ ပုံသေပြောလို့မရတော့ပဲ Requirement ပေါ်မူတည် တည်ဆောက်ရတဲ့ Service များပဲ ဖြစ်ပါတယ်။

4.3 – Why SOA

Software Solution တစ်ခုကို SOA နည်းစနစ်နဲ့ တည်ဆောက်လိုက်ခြင်းအားဖြင့် ရရှိလာမယ့်အကျိုးကျေးဇူးတွေ အများကြီးရှိပါတယ်။ အချို့ Service တွေဟာ လက်ရှိ Project အတွင်းမှာ Reusable ဖြစ်ယုံသာမက၊ အခြား Project တွေမှာလည်း ပြန်လည်အသုံးပြုနိုင်မှာပါ။ အထူးသဖြင့် Data Access Service တွေ၊ Infrastructure Service တွေနဲ့ Development Service တွေအပါအဝင် Utility Layer မှာရှိတဲ့ Service အများစုကို မတူကွဲပြားတဲ့ Software Project တွေမှာ ပြန်လည်အသုံးပြုနိုင်မှာပဲဖြစ်ပါတယ်။

One Cores, Multiple Clients

တစ်ချိန်က Windows, Linux, Mac အစရှိတဲ့ Desktop Platform အမျိုးမျိုးမှာ အလုပ်လုပ်နိုင်တဲ့ Cross-platform App တွေ တည်ဆောက်ရခြင်းဟာ စိန်ခေါ်မှုတစ်ရပ် ဖြစ်ခဲ့ပါတယ်။ ကနေ့ခေတ်မှာ Software ဆိုတာကို Desktop ကွန်ပျူတာနဲ့သာမက၊ Tablet, Smart Phone အစရှိတဲ့ Mobile Device တွေကနေလည်း တွင်ကျယ်စွာ အသုံးပြုလာ ကြပြီဖြစ်ပါတယ်။ တစ်ချိန်မှာ Smart Watch, Smart Glass စတဲ့ Wearable Device တွေနဲ့ အခြားအခြားသော Device တွေလည်း အသုံးပြုလာကြနိုင်ပါတယ်။ ဒီလို Device တွေ အမျိုးမျိုးဖြစ်လာတဲ့အတွက် တစ်ချိန်က Cross-platform ဆိုတဲ့ ပြဿနာအတွက်သာ ခေါင်းစားခဲ့ကြရာက၊ ယနေ့အချိန်မှာတော့ Cross-device ဆိုတဲ့ ပြဿနာက လည်း ခေါင်းစားစရာဖြစ်လာပါတယ်။

ဒီပြဿနာကိုဖြေရှင်းပေးနိုင်တဲ့ နည်းပညာတွေနဲ့ နည်းစနစ်တွေ အမျိုးမျိုးရှိရာမှာ SOA နည်းစနစ်ဟာ ထိရောက်မှုအရှိ ဆုံးနည်း စနစ်တစ်ခုပဲဖြစ်ပါတယ်။ Software ရဲ့ အစိတ်အပိုင်းတွေကို သီးခြားစီအလုပ်လုပ်နိုင်တဲ့ ပရိဂရမ်ငယ်များအဖြစ် တည်ဆောက်ထားခြင်းအားဖြင့် Utility Layer နဲ့ Core Layer မှာရှိနေတဲ့ အခြေခံ လုပ်ဆောင်ချက်နဲ့ Key Service တွေကို တစ်ကြိမ်သာ ဖန်တီးဖို့လိုပြီး၊ UI Service တွေကိုသာ Device အမျိုးမျိုးနဲ့ Platform အမျိုးမျိုးအတွက် ခွဲခြား တည်ဆောက်ပေးရတော့မှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ မတူကွဲပြားတဲ့ Device တိုင်းအတွက် App တစ်ခုစီ အစအဆုံးပြန်လည် ဖန်တီးနေစရာ မလိုတော့တဲ့ အားသာချက်ကို ရရှိမှာပဲဖြစ်ပါတယ်။

Device အမျိုးမျိုးအတွက် UI Service တွေနဲ့ Core Service တွေ၊ အပြန်အလှန်ဆက်သွယ်ဖို့အတွက် HTTP နဲ့ SOAP တို့လို Communication Protocol တွေကို Interface အဖြစ်အသုံးပြုကြပါတယ်။ ဒီအကြောင်းကို မကြာခင် ဆက်လက်ဖော်ပြပါမယ်။

Development Productivity

SOA နည်းစနစ်ကပေးတဲ့ နောက်ထပ်အားသာချက်ကတော့ Development Productivity ပဲဖြစ်ပါတယ်။ Software Solution ကို သီးခြား ပရိဂရမ်ငယ်များအဖြစ် ခွဲခြားတည်ဆောက်ခြင်းဖြစ်တဲ့အတွက် သီးခြားစီ ခွဲခြားစမ်းသပ်နိုင်မှာ ဖြစ်ပါတယ်။ ဒါ့အပြင် သီးခြားစီ ပြုပြင်ထိမ်းသိမ်းနိုင်တဲ့အတွက် Solution တစ်ခုလုံးကို ထိမ်းသိမ်းရခြင်းထက် ပိုမိုရိုရှင်း ထိရောက်နိုင်မှာပဲဖြစ်ပါတယ်။

အဖွဲ့လိုက် ဆောင်ရွက်ရာမှာလည်း Developer တွေကို တာဝန်ခွဲအပ်ရတာ ပိုမိုထိရောက်နိုင်ပါသေးတယ်။ သီးခြား Service တွေတည်ဆောက်ဖို့ သီးခြားစီ တာဝန်ပေးထားတဲ့ Developer တွေဟာ မိမိတို့တာဝန်ကျရာ Service ကို နှစ်သက်ရာ နည်းပညာ၊ ကျွမ်းကျင်ရာနည်းစနစ်တွေ အသုံးပြုဖန်တီးနိုင်မှာဖြစ်လို့ ရေးသားဖန်တီးရာမှာ ပိုမိုထိရောက် လျင်မြန်စေမှာ ဖြစ်ပါတယ်။ Service အချင်းချင်း ဆက်သွယ်ဖို့ သတ်မှတ်ထားတဲ့ Interface စံ ညီနေသ၍ အခြား Service မှာ တာဝန်ကျသူ Developer တွေ အသုံးပြုထားတဲ့ နည်းပညာတွေ၊ နည်းစနစ်တွေနဲ့ လိုက်ညီနေဖို့ မလိုတော့ပါဘူး။

မိမိတို့ နှစ်သက်ကျွမ်းကျင်ရာနည်းပညာ ဆိုတဲ့နေရာမှာ လုံးဝမတူကွဲပြားတဲ့ Language တွေ Platform တွေပါဖြစ်နိုင် ပါသေးတယ်။ ဥပမာ – Infrastructure Service တွေကို အကောင်းဆုံး Performance ရရှိစေဖို့အတွက် C/C++, Go တို့လို လျင်မြန်တဲ့ Language တွေကို အသုံးပြုနိုင်ပြီး၊ Monitor Service တွေကို Real-time

concurrency လုပ်ဆောင်ချက် ပိုမိုကောင်းမွန်တဲ့ NodeJS လို နည်းပညာနဲ့ တည်ဆောက်ထားနိုင်ပါတယ်။ Application Service တွေကိုတော့ အသင့်သုံး Business Component တွေ စုံလင်တဲ့ Java, .Net တို့လို နည်းပညာမျိုးတဲ့ တည်ဆောက်နိုင်ပါတယ်။ ဒီနည်းနဲ့ Developer တွေဟာ မိမိတို့ အပိုင်နိုင်ဆုံးနည်းပညာကို ခွဲခြားအသုံးပြုနိုင်လို့ Development Productivity ပိုကောင်းလာတဲ့အပြင် သက်ဆိုင်ရာကဏ္ဍအတွက် အထိရောက်ဆုံးနည်းပညာကို ရွေးချယ်အသုံးပြုနိုင်လို့ ပိုမိုကောင်းမွန်တဲ့ စွမ်းဆောင်ရည် Efficiency ကိုလည်း ရရှိနိုင်မှာပဲ ဖြစ်ပါတယ်။

Scalability

SOA နည်းစနစ်နဲ့ ခွဲခြားတည်ဆောက်ထားတဲ့ Service တွေကို တစ်လုံးထက်ပိုတဲ့ကွန်ပျူတာတွေမှာ ခွဲခြားပြီး Deploy လုပ်ထားနိုင်ယုံသာမက၊ လိုအပ်ရင် Network အမျိုးမျိုးနဲ့ တည်နေရာအမျိုးမျိုးမှာလည်း ခွဲခြားထားနိုင်ပါသေးတယ်။ ဥပမာ – Data Access Service တွေက တစ်နေရာ၊ Application Service တွေက တစ်နေရာ၊ Monitor Service တွေက တစ်နေရာ စသဖြင့် ခွဲခြားထားနိုင်ပါတယ်။ အလုပ်များများလုပ်ရမယ့် Monitor Service အတွက် Processing Power ပိုကောင်းတဲ့ ကွန်ပျူတာတွေသုံးပြီး Data Store အဓိကဖြစ်တဲ့ Data Access Service တွေအတွက် Storage Capacity ပိုများတဲ့ ကွန်ပျူတာတွေကို အသုံးပြုနိုင်ပါတယ်။ ဒီနည်းနဲ့ Service တစ်ခုခြင်းအတွက် အထိရောက်ဆုံးဖြစ် အောင် သီးသန့် Setting လုပ်ထားတဲ့ Hardware Infrastructure ကို ရရှိနိုင်မှာဖြစ်ပြီး၊ လိုအပ်လို့ နောင်တစ်ချိန်မှာ အဆင့်မြှင့်တင်ရမယ်ဆိုရင်လည်း လိုအပ်တဲ့ Service တစ်ခု ခြင်းစီအတွက် ရွေးချယ်တိုးမြှင့်သွားနိုင်မှာဖြစ်တဲ့အတွက် ပိုမို ထိရောက်စေမှာဖြစ်ပါတယ်။

Service Oriented Business Model

Software Product တစ်ခုပေါ်မှာ အခြေခံထားတဲ့ Business Model အမျိုးမျိုး ရှိနိုင်ပါတယ်။ SOA နည်းစနစ်နဲ့ တည် ဆောက်ထားတဲ့ Solution တစ်ခုမှာတော့ Business Model ကို နောက်ဆုံးရလဒ်ဖြစ်တဲ့ Product ပေါ်မှာ တင်မကပဲ Service တွေ ပေါ်မှာအခြေပြုထားတဲ့ Business Model တွေလည်း တည်ဆောက်ရရှိနိုင်ပါတယ်။ Service တွေကို Interface ကနေတစ်ဆင့် ဆက်သွယ်နိုင်အောင် တည်ဆောက်ရတဲ့အတွက် အဲ့ဒီ Service တွေကို ကိုယ့်ရဲ့ Solution မှာ ပါဝင်တဲ့ အခြား Service တွေကသာမက၊ ပြင်ပကနေလည်း ဆက်သွယ်အသုံးပြုခွင့် ပေးနိုင်ပါတယ်။ ဒီနည်းနဲ့ အလားတူ လုပ်ဆောင်ချက် ရရှိလိုသူတွေအနေနဲ့ ကိုယ်တိုင်ဖန်တီးနေဖို့မလိုတော့ပဲ ကျွန်တော်တို့တည်ဆောက်ထားတဲ့ Service ကနေတစ်ဆင့် အသင့်အသုံးပြုနိုင်မှာဖြစ်ပါတယ်။

ကနေ့အချိန်မှာ Service ကို အခြေခံထားတဲ့ Business Model တွေကို အတော်လေး တွင်တွင်ကျယ်ကျယ် အသုံးပြုနေ ကြပါပြီ။ တစ်ချို့လည်း မိမိတို့ရဲ့ Service တွေကို အခမဲ့ ဆက်သွယ်အသုံးပြုခွင့် ပေးကြပြီး၊ တစ်ချို့လည်း အသုံးပြုတဲ့ အကြိမ်အရေအတွက်အလိုက် အခကြေးငွေကောက်ခံကြပါတယ်။ တစ်ချို့လည်းအသုံးပြုခွင့်ကို လစဉ်ကြေး၊ နှစ်စဉ်ကြေး အနေနဲ့ ကောက်ခံခြင်းအားဖြင့် သူတို့ရဲ့ Business Model ကို Service များပေါ်မှာ တည်ဆောက်ထားတက်ကြပါတယ်။

ဒီလို Service Oriented Business Model တွေပေါ်ပေါက်လာဖို့အတွက် အဓိကကျတဲ့အခန်းကဏ္ဍကနေ ပါဝင်လာတာ ကတော့ Web Technology ပဲဖြစ်ပါတယ်။

4.4 – Web Services

Service တွေဟာ တစ်ခုနဲ့တစ်ခု ဆက်သွယ်ဖို့အတွက် ကြားခံဆက်သွယ်ရေး Interface လိုပါတယ်။ SOAP, XML-RPC, REST စသဖြင့် အသုံးများတဲ့ Interface အမျိုးမျိုးရှိပါတယ်။ ဒီ Interface တွေမှာ တူညီတဲ့အချက်တစ်ချက် ရှိပါတယ်။ Interface အားလုံးက HTTP ကို အခြေခံဆက်သွယ်ရေး Protocol အဖြစ် အသုံးပြုကြခြင်းပဲဖြစ်ပါတယ်။ HTTP ဟာ Web Technology တစ်ခုဖြစ်တဲ့အတွက် HTTP ကို အခြေခံ ဆက်သွယ်ရေး Interface အဖြစ် အသုံးပြုထား တဲ့ Service တွေကို Web Service လို့ ခေါ်ခြင်းပဲဖြစ်ပါတယ်။ တစ်နည်းအားဖြင့်၊ ကနေ့ခေတ်မှာ Service အများစုဟာ HTTP ကို ဆက်သွယ်ရေး Interface အဖြစ် အသုံးပြုထားတဲ့အတွက် "Service အများစုဟာ Web Service များ ဖြစ်တယ်" လို့လည်း ဆိုနိုင်ပါတယ်။

Web Service တွေ တည်ဆောက်ရာမှာ အဓိကသတိပြုရမယ့် နည်းပညာနှစ်ရပ်ရှိပါတယ်။ အဲ့ဒါတွေကတော့ REST နဲ့ JSON တို့ပဲဖြစ်ပါတယ်။

Representational State Transfer (REST)

REST လို့ အတိုကောက်ခေါ်တဲ့ Representational State Transfer ဆိုတာဟာ ဆက်သွယ်ရေး Interface တွေ ဖန်တီးရာမှာ လိုက်နာဖို့ သတ်မှတ်ထားတဲ့ နည်းပညာသတ်မှတ်ချက်များ ဖြစ်ပါတယ်။ REST နည်းစနစ်ကို World Wide Web Consortium က HTTP ရဲ့ အစိတ်အပိုင်းတစ်ရပ်အဖြစ် တွဲဖက်တီထွင်ထားခြင်းဖြစ်ပါတယ်။ တစ်နည်းအားဖြင့် HTTP ဟာ REST နည်းစနစ်နဲ့အညီ ဖန်တီးထားတဲ့ နည်းပညာဖြစ်တဲ့အတွက် HTTP ကို ဆက်သွယ်ရေး Interface အဖြစ် အသုံးပြုထားတယ်ဆိုရင် REST နည်းစနစ်ကို အသုံးပြုနေခြင်းဖြစ်တယ်လို့လည်း ဆိုနိုင်ပါတယ်။ HTTP အကြောင်းကိုတော့ ဒီနေရာမှာ အသေးစိတ်မပြောတော့ပါဘူး။ **Professional Web Developer** မှာ ဖော်ပြခဲ့ပြီးဖြစ်လို့ လေ့လာလို တယ်ဆိုရင် **Professional Web Developer** ရဲ့ **အခန်း (၁)** နဲ့ **အခန်း (၁၇)** တို့မှာ လေ့လာနိုင်ပါတယ်။

REST နည်းစနစ်က သတ်မှတ်ထားတဲ့ သတ်မှတ်ချက်များထဲမှ ထူးခြားချက်တစ်ချို့ကို ရွေးထုတ်ဖော်ပြလိုက်ပါတယ်။

Stateless – Service တစ်ခုနဲ့တစ်ခု ဆက်သွယ်ရာမှာ ပါဝင်ရမယ့် အချက်အလက်များကို သိမ်းဆည်းမထားပဲ ဆက်သွယ်မှုလုပ်တိုင်းမှာ ပါဝင်ရမယ့်အချက်အလက်အားလုံးကို ထည့်သွင်းပေးပို့ခြင်းအားဖြင့် ဆက်သွယ်ရမယ်လို့ဆိုပါတယ်။ ဒီသဘောသဘာဝကို Stateless လို့ခေါ်ပြီး HTTP ဟာ Stateless Protocol တစ်ခု ဖြစ်ပါတယ်။ ဒီ Stateless ဆိုတဲ့ အချက်ကို ထည့်သွင်းစဉ်းစားခြင်းအားဖြင့် Service တွေတည်ဆောက်ရာမှာ များစွာရိုးရှင်းလွယ်ကူသွားစေနိုင်ပါ တယ်။ ဥပမာ – ဆက်သွယ်မှုအကြိမ်တိုင်းမှာ ပါဝင်ရမယ့် အချက်အလက်တွေကို ပေးပို့မှာဖြစ်လို့၊ ဆက်သွယ်မှုမပြီးခင် အဆက်အသွယ် ပြတ်တောက်သွားခဲ့ရင် ဘယ်လိုလုပ်မလဲဆိုတဲ့ကိစ္စမျိုးအတွက် ခေါင်းစားနေဖို့ မလိုတော့ပါဘူး။ ပြတ်တောက်သွားခဲ့ရင် နောက်တစ်ကြိမ် ထပ်မံဆက်သွယ်လိုက်ယုံသာရှိလို့ Service Design လည်း များစွာရိုးရှင်းသွားမှာပဲ ဖြစ်ပါတယ်။

Cachable – ဆက်သွယ်ရယူသူ Service ဟာ ရရှိထားတဲ့ရလဒ်ကို Cache အဖြစ် ယာယီသိမ်းဆည်းထားပြီး လိုအပ်သလို ပြန်လည်အသုံးပြုနိုင်ရမယ်လို့လည်း ဆိုပါတယ်။ ဒီနည်းနဲ့ တစ်ကြိမ်ရယူထားဖူးတဲ့ရလဒ်ကို နောင်လိုအပ်တဲ့အခါ အမြဲတမ်း Server Service ထံကနေ ပြန်လည်တောင်းခံနေစရာမလိုပဲ ယာယီသိမ်းဆည်း

ထားတဲ့ ရလဒ်ကို ပြန်လည် အသုံးပြုနိုင်မှာဖြစ်လို့ စနစ်ရဲ့စွမ်းဆောင်ရည်နဲ့ အမြန်နှုန်း တိုးတက်သွားစေမှာဖြစ်ပါတယ်။ အချက်အလက်ပေးပို့သူ Server Service အနေနဲ့လည်း ဘယ်လိုရလဒ်တွေကို Cache အဖြစ်ယာယီသိမ်းဆည်းသင့်တယ်၊ ဘယ်လိုအချက်အလက် တွေကို မသိမ်းဆည်းသင့်ဘူး စသဖြင့် Client Service ကို အသိပေးနိုင်ရမှာ ဖြစ်ပါတယ်။ ဒီတော့မှ နောက်ဆုံးရလဒ်ကို ရယူရမယ့်အစား မှားယွင်းပြီး Cache ရလဒ်ကို ဖော်ပြမိတယ်ဆိုတဲ့ အမှားမျိုးကို ရှောင်ရှားနိုင်မှာဖြစ်ပါတယ်။

Uniform Interface – Service တစ်ခုကပေးနိုင်တဲ့ လုပ်ဆောင်ချက်တွေကို ရယူအသုံးပြုနိုင်ဖို့အတွက်၊ လုပ်ဆောင်ချက် တိုင်းအတွက် သီးခြား Identification (ID) ရှိရမယ်လို့ဆိုပါတယ်။ အဲ့ဒီ ID ဟာ လုပ်ဆောင်ချက် ရလဒ်ရဲ့အစိတ်အပိုင်း တစ်ခု မဖြစ်သင့်ပဲသီးခြားတည်ရှိနိုင်ရမယ်လို့လည်း ဆိုပါတယ်။ Web Technology မှာ တော့ ဒီလုပ်ဆောင်ချက် ရရှိဖို့ URL တွေကို အသုံးပြုပါတယ်။ Web Resource တိုင်းအတွက် ညွှန်းဆိုထားတဲ့ ကိုယ်ပိုင် URL ကိုယ်စီရှိတဲ့အတွက် သက်ဆိုင်ရာ Resource ကို ရရှိလိုသူ Client Service က အဲ့ဒီ URL တွေကို အသုံးပြုခြင်းအားဖြင့် ဆက်သွယ်ရယူနိုင်ခြင်း ဖြစ်ပါတယ်။ ပြီးတော့ အဲ့ဒီ ID တွေဟာ အဓိပ္ပါယ်ရှိရမယ်လို့လည်း သတ်မှတ်ထားပါသေးတယ်။ ဆိုလိုတာက ID ကို ကြည့်လိုက်ယုံနဲ့ ဘယ်လိုလုပ်ဆောင်ချက်ကို ဆောင်ရွက်သွားစေမှာလဲဆိုတာကို သိရှိနိုင်ရမယ်ဆိုတဲ့သဘောပါ။ Web Technology မှာ အသုံးပြုတဲ့ URL တွေ ဟာ အခြေခံအားဖြင့် ဖတ်ရှုနားလည်နိုင်လောက်တဲ့ အရာတွေဖြစ်ပေမယ့်၊ တစ်ခါတစ်ရံမှာလည်း ရှုပ်ထွေးပြီး နားလည်ရခက်တဲ့ URL တွေ ဖြစ်သွားတက်လို့၊ Service ဖန်တီးသူက ကြည့်လိုက်ယုံနဲ့ နားလည်ရလွယ်တဲ့ URL တွေဖြစ်နေအောင် သတိပြုဖန်တီးပေးဖို့ လိုတက်ပါတယ်။ ဒီအကြောင်းကို **Professional Web Developer** ရဲ့ **အခန်း (၁၇)** မှာ ဖော်ပြထားပါတယ်။ လိုရင်းကတော့ REST နည်းစနစ်နဲ့အညီ Service တွေ တည်ဆောက်တဲ့အခါ၊ လုပ်ဆောင်ချက်တိုင်းအတွက် ကိုယ်ပိုင် ID ရှိအောင် Design လုပ် ရမှာဖြစ်ပြီး အဲ့ဒီ ID တွေကို ကြည့်လိုက်ယုံနဲ့ ဆောင်ရွက်သွားမယ့်လုပ်ဆောင်ချက်ကို သိရှိရနိုင်စေတဲ့ Uniform Interface ရှိစေရမယ်ဆိုတဲ့ သဘောပဲ ဖြစ်ပါတယ်။

Code on Demand – Service တစ်ခုနဲ့တစ်ခု ဆက်သွယ်ဆောင်ရွက်ရာမှာ Data အချက်အလက်များကို အပြန်အလှန် ဖလှယ်ခြင်းအားဖြင့်သာ ဆက်သွယ်ဆောင်ရွက်ဖို့ REST က သတ်မှတ်ထားပါတယ်။ တစ်နည်းအားဖြင့် Service တစ်ခုရဲ့ Function ကို ရယူပြီး၊ အဲ့ဒီ Function ကို နောက် Service တစ်ခုက ချိတ်ဆက်အလုပ်လုပ်ခြင်းမျိုးကို မလုပ်ရပါဘူး။ ဒါပေမယ့် Service တစ်ခုက ရလဒ်အနေနဲ့ ပြန်ပေးတဲ့အခါမှာ Data ကိုသာမကပဲ ပရိုဂရမ် Code ကို ပြန်ပေးနိုင်ပါတယ်။ ဒီနည်းနဲ့ Service တစ်ခုကပေးတဲ့ Code ကို နောက် Service က လိုသလို မွန်းမံဖြည့်စွက်ခြင်းအားဖြင့် အသုံးပြုနိုင်အောင် Design လုပ်ထားနိုင်ပါတယ်။ တစ်နည်းအားဖြင့် Service တစ်ခုနဲ့တစ်ခု တွဲချိတ်ထားတဲ့သဘောမျိုးနဲ့ Service တစ်ခုက နောက် Service တစ်ခုရဲ့ Function ကို ခေါ်မသုံးသင့်ပေမယ့်၊ လိုအပ်ရင် Code ကိုရလဒ်အနေနဲ့ ပေးနိုင်တဲ့ဆိုတဲ့ သဘောပဲဖြစ်ပါတယ်။ သတိပြုရမှာက၊ Server Service ပေးလိုက်တဲ့ Code ဟာ Client Service ထံ၊ Code အနေနဲ့ရောက်သွားပြီး Client Service ပေါ်မှာသာ အလုပ်လုပ်တယ်ဆိုတဲ့ အချက်ပဲဖြစ်ပါတယ်။ Server Service က Code ကို ပေးယုံသာပေးပြီး၊ အဲ့ဒီ Code ကို အလုပ်လုပ် စေတဲ့ကိစ္စကိုတော့ သူကိုယ်တိုင်မလုပ်တဲ့ သဘောပါ။ Web Technology မှာ ဒီနည်းစနစ်ကို Client-side Scripting လို့ခေါ်ပြီး JavaScript ကို Code Exchange လုပ်ဖို့အတွက် သုံးပါတယ်။

Data Format – Server Service တစ်ခုဟာ Data အချက်အလက်များကို ရလဒ်အနေနဲ့ ပြန်လည်ပေးပို့တဲ့အခါ အဲ့ဒီ Data ရဲ့ ဖွဲ့စည်းပုံ Format ကို တစ်ပါတည်း တွဲဖက်ထည့်သွင်းပေးရပါတယ်။ ဒီနည်းနဲ့ Data Format တစ်

မျိုးတည်းကို မှီခိုဖို့မလိုတော့ပဲ ဘယ်လို Data Format နဲ့ ရလဒ်ပြန်ပေးသလဲဆိုတာကို Server Service က Client Service ကို အသိပေးခြင်းအားဖြင့် Data Format အမျိုးမျိုးကို လိုအပ်သလို အသုံးပြုသွားနိုင်မှာပဲဖြစ်ပါတယ်။ အသုံးများတဲ့ Data Format တွေကတော့ JSON, XML နဲ့ YAML တို့ပဲဖြစ်ကြပါတယ်။ ဒီ Data Format တွေကို Programming Language အားလုံးနီးပါးက နားလည်အလုပ်လုပ်ပေးနိုင်ကြပါတယ်။ တစ်နည်းအားဖြင့် Client Service အားလုံးက ဘယ်လို Data Format နဲ့ပဲ ရလဒ်ကို ရရှိသည်ဖြစ်စေ လက်ခံအလုပ်လုပ်ပေးနိုင်တယ်ဆိုတဲ့ သဘောလည်း ဖြစ်ပါတယ်။ ဒါပေမယ့် လက်တွေ့မှာ စနစ် Consistency အတွက် Format တစ်ခုကိုသာ ရွေးချယ်အသုံးပြုကြလေ့ရှိပါတယ်။ XML-RPC, SOAP စတဲ့ နည်းပညာတွေကို ဆက်သွယ်ရေး Interface အဖြစ်အသုံးများခဲ့တဲ့ အချိန်တွေတုန်းက XML ကို Data Format အနေနဲ့ အသုံးများခဲ့ကြပါတယ်။ ကနေ့အချိန်မှာတော့ Code on Demand လုပ်ဆောင်ချက်ကို အသုံးချ ထားတဲ့ Web Service တွေ ပိုမိုတွင်ကျယ်လာတာနဲ့ အမျှ၊ Code Exchange အတွက် Standard အဖြစ်အသုံးပြုတဲ့ JavaScript နဲ့ တွဲဖက်အလုပ်လုပ်ရာမှာ ပိုပြီး ထိရောက်တဲ့ JSON ကို အသုံးများလာကြပါတယ်။

JavaScript Object Notation (JSON)

JSON (ဂျေဆွန်) လို့အတိုကောက်ခေါ်တဲ့ JavaScript Object Notation ဆိုတာဟာ JavaScript Object တွေ တည်ဆောက်ဖို့အတွက် အသုံးပြုရတဲ့ နည်းစနစ်တစ်မျိုးဖြစ်ပါတယ်။ ပြီးခဲ့တဲ့အခန်းမှာ JavaScript Object တွေကို Object Constructor ကနေ တစ်ဆင့် တည်ဆောက်နိုင်ပုံကို ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။

JavaScript

```
function Person() {
    this.fullName = "James Doe";
    this.age = 32;
    this.walk = function() {
        console.log(this.fullName + " is walking.");
    }
}

var james = new Person();
james.walk();    // => James Doe is walking.
```

နမူနာမှာ `Person()` Constructor ကို သုံးပြီး `james` ဆိုတဲ့ Object တစ်ခုကို တည်ဆောက်ထားပါတယ်။ အလားတူ Object မျိုး ရရှိဖို့အတွက် JSON ကို အသုံးပြုပြီး အခုလိုတည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
var james = {
    fullName: "James Doe",
    age: 32,
    walk: function() {
        console.log(this.fullName + " is walking.");
    }
}

james.walk();    // =>James Doe is walking
```

Constructor ကနေတစ်ဆင့် Object ကို တည်ဆောက်ခြင်းမဟုတ်ပဲ JSON ရေးထုံးနဲ့ Object တစ်ခုကို တိုက်ရိုက် တည်ဆောက်ယူခြင်းဖြစ်ပါတယ်။ JSON ဟာ အမှန်တော့ Hash Array တစ်ခုသာဖြစ်ပါတယ်။ Array တစ်ခုဖြစ်တဲ့ အတွက် Index တစ်ခုနဲ့တစ်ခုကြားမှာ Comma နဲ့ ပိုင်းခြားပေးရပါတယ်။ Index တစ်ခုဟာ Property တစ်ခုဖြစ်ပါတယ်။ Index ရဲ့ Key နဲ့ Value ကို တော့ Colon နဲ့ ပိုင်းခြားပေးရပါတယ်။ Key အတွက် နှစ်သက်ရာအမည်ကို ပေးနိုင်ပါတယ်။ Value အတွက် တန်ဖိုးသတ်မှတ်ရာမှာတော့ JavaScript Data Type များဖြစ်တဲ့ Number, String, Boolean တို့ သာမက Array, Function, Object နဲ့ null တို့ကိုလည်း အသုံးပြုနိုင်ပါတယ်။

JavaScript

```
var james = {
    fullName: "James Doe",
    age: 32,
    walk: function() {
        console.log(this.fullName + " is walking.");
    },
    address: {
        street: "52, Main Road",
        city: "Yangon",
    },
    phones: [
        "123 456 789",
        "987 654 321"
    ],
    children: null
}

james.address.city;      // => Yangon
james.phones[0];         // => 987 654 321
```

JSON ဟာ မူလက JavaScript Programming Language နဲ့ တွဲဖက်အသုံးပြုဖို့ ရည်ရွယ်ဖန်တီးခဲ့ပေမယ့် အခုအခါ မှာတော့ မည်သည့် Programming Language နဲ့မဆို တွဲဖက်အသုံးပြုနိုင်တဲ့ Language Independent Data Format တစ်ခုအဖြစ် အသုံးတွင်ကျယ်လာခဲ့ပါတယ်။ အထက်မှာပြောခဲ့သလို Web Service တွေအတွက် အဓိက Client-side Language ဖြစ်တဲ့ JavaScript နဲ့ တွဲဖက်အလုပ်လုပ်ရာမှာ ပိုမိုအဆင်ပြေခြင်းနဲ့ သူ့အရင်က အသုံးတွင်ကျယ်ခဲ့တဲ့ XML ထက် ရေးထုံးပိုင်း ပိုမိုရိုးရှင်းခြင်းတို့ကြောင့် အခုနောက်ပိုင်းမှာ Service တွေ တစ်ခုနဲ့တစ်ခုကြား ဆက်သွယ်ရာမှာ အဓိက Data Exchange Format အဖြစ် အသုံးပြုလာခဲ့ကြပါတယ်။

လက်တွေ့မှာ Data Exchange အတွက်သုံးတဲ့ Data JSON နဲ့ JavaScript Object JSON တို့ အနည်းငယ် ကွာပါတယ်။ အသိသာဆုံး ကွာခြားချက်ကတော့ Data JSON မှာ Function တွေ Regular Expression တွေပါဝင်လို့မရခြင်း ပဲဖြစ်ပါတယ်။ JavaScript Object JSON မှာတော့ Property တစ်ခုအတွက်တန်ဖိုးကို Function သို့မဟုတ် Regular Expression အဖြစ် သတ်မှတ်ထားနိုင်ပါတယ်။

ကနေ့အချိန်မှာ REST Web Service လို့ပြောလိုက်ရင် JSON ကို Data Exchange Format အဖြစ်သုံးပြီး HTTP ကို ဆက်သွယ်ရေး Interface အဖြစ်အသုံးပြုထားတဲ့ Service များကို ဆိုလိုခြင်းပဲဖြစ်ပါတယ်။ REST နည်းစနစ်က သတ်မှတ်ထားတဲ့ Stateless, Cachable, Uniform Interface စတဲ့ လုပ်ဆောင်ချက်အများစုကို HTTP က ပေး

ထားပြီး ဖြစ်ပါတယ်။ ကျွန်တော်တို့ကသာ ကျွန်တော်တို့ရဲ့ Service တွေကို Design လုပ်တဲ့နေရာမှာ ဒီသတ်မှတ်ချက်တွေ ကနေ သွေဖီသွားခြင်းမရှိအောင် သတိပြု Design လုပ်ရမှာပဲဖြစ်ပါတယ်။ Language အနေနဲ့လည်း Client Service တွေ အတွက် JavaScript ကို အဓိကအဖြစ် အသုံးပြုတက်ကြပေမယ့် Core Service တွေနဲ့ Utility Service တွေကို တော့ နှစ်သက်ရာ နည်းပညာနဲ့ အသုံးပြုတည်ဆောက် နိုင်ပါတယ်။ ကြားခံဆက်သွယ်ရေး Interface အဖြစ်သာ HTTP ကို အသုံးပြုဖို့ လိုပါတယ်။

Conclusion

Software Development မှာ 3-Tires Architecture, Modular Design, Layered System, Model-View-Controller စသဖြင့် Architecture ပိုင်းဆိုင်ရာ နည်းစနစ်အမျိုးမျိုးရှိကြပါတယ်။ တစ်ခုနဲ့တစ်ခု အသေးစိတ်တွေ မတူကြပေမယ့် လိုရင်းကတော့ Software Solution တစ်ခုကို ကဏ္ဍအလိုက် ခွဲခြားပြီးတည်ဆောက်ခြင်းပဲဖြစ်ပါတယ်။ ဒီလိုခွဲခြား တည်ဆောက်တော့မှ ရေရှည်ပြုပြင်ထိမ်းသိမ်းရေးမှာ ထိရောက်မှာဖြစ်တဲ့အတွက် အခုလို နည်းစနစ်အမျိုးမျိုးကို တီထွင် လာခဲ့ကြခြင်းပဲဖြစ်ပါတယ်။

Service Oriented Architecture ဆိုတာဟာ၊ အခုမှ ပေါ်လာတဲ့ နည်းစနစ်သစ်တစ်ခုမဟုတ်ပေမယ့် Internet နဲ့ Web Technology တို့ရဲ့ ဖွံ့ဖြိုးတိုးတက်လာမှုနဲ့အတူ ကနေ့အချိန်မှာ ပိုမိုအသုံးဝင်ပြီး တွင်ကျယ်လာတဲ့ Software Design Architecture တစ်ခု ဖြစ်လာခဲ့ပြီ ဖြစ်ပါတယ်။

အပိုင်း (၂)

Project Management

အမှားတစ်ခုပြုလုပ်မိလို့၊ မူလအခြေအနေပြန်ရအောင် Code ကို Manual ပြန်ပြင်ရတယ်ဆိုတာ မဖြစ်သင့်ပါဘူး။ အဖွဲ့လိုက်ပူးပေါင်းဆောင်ရွက်ရာမှာ၊ တစ်ဦးရေးသားထားတဲ့ Code နဲ့ နောက်တစ်ဦးရေးသားထားတဲ့ Code ပေါင်းစပ်တဲ့လုပ်ငန်းကို Manual လုပ်ရတယ်ဆိုတာလည်း မဖြစ်သင့်ပါဘူး။

Ubuntu – သင့်အတွက် Linux (စာအုပ်)
Ubuntu Linux အသုံးပြုနည်းအား Installation မှ စတင်၍ System Configuration နှင့် Development Environment တည်ဆောက်ခြင်းအထိ အဆင့်လိုက်ရေးသား ဖော်ပြထားသည့်စာအုပ် အောက်ပါလိပ်စာတွင် အခမဲ့ ရယူနိုင်သည်။

http://u4ubook.com

အခန်း(၅) – Version Control System

"ကဲ... Demo လုပ်ဖို့ အားလုံးအသင့်ပဲလားလေ့။"

"ဟုတ်ကဲ့.. နေ့လည်လောက်ဆိုရင်တော့ အသင့်ဖြစ်ပါပြီ။"

"ဘယ်လိုဖြစ်တာလဲ။ မနေ့ကတည်းက အားလုံးပြီးနေပြီးသား မဟုတ်လား။"

"ဟုတ်ပါတယ်။ Code တစ်ချို့ပြင်လိုက်မိတာ အလုပ်မလုပ်တော့လို့ ပြန်စစ်နေတာခင်ဗျ။"

"ဟာ... ပြဿနာပဲကွာ။ ဘာလို့ခုချိန်မှပြင်တာလဲ။ ဒီနေ့ Demo ပြမယ်လို့ကြေငြာပြီးနေပြီ။ နဂို အလုပ်လုပ်နေတဲ့အတိုင်း ပြန်ထားလိုက်လေ။"

"ဟုတ်ကဲ့... နဂိုအတိုင်းပြန်ထားပြီးပါပြီ။ ဒါလည်း အလုပ်မလုပ်သေးဘူး။ ဘာဖြစ်သွားလဲ မသိဘူး။"

"ခက်တာပဲကွာ။ တော်သေးတာပေါ့၊ အရင် Version တွေ ငါသိမ်းထားမိလို့။ Share Folder ထဲ ကိုဝင်ပြီး အရင် Version ကို ကူးယူလိုက်။"

"Share Folder ထဲမှာတွေ့ပါတယ်။ ဒါပေမယ့် project-latest, project-update, project-final, project-stable, project-update-final, project-real-latest အများကြီးဖြစ်နေတယ်။ အဲ့ဒီထဲက ဘယ်ဟာကို ကူးယူရမလဲ"

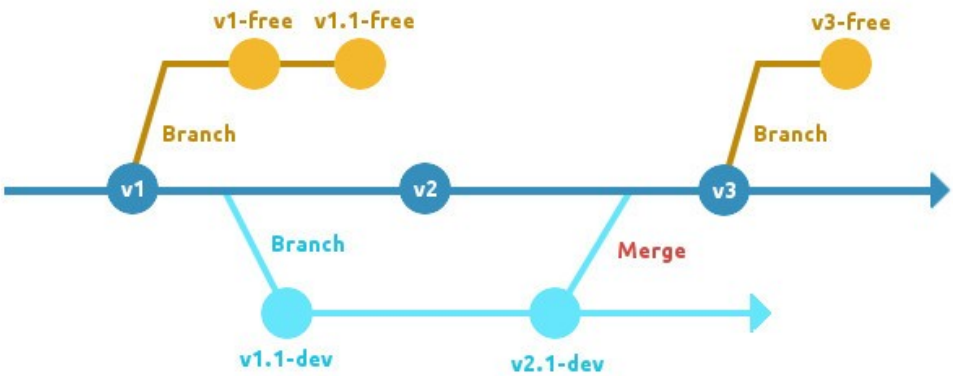
"project-latest ကိုယူလေကွာ။ အဲ... နေဦး။ project-real-latest ဖြစ်မယ်ထင်တယ်။ ရှုပ်တယ်ကွာ ဘယ်ဟာနောက်ဆုံး Version လဲသေချာအောင်၊ တစ်ခုချင်းသာ လိုက်စမ်းကြည့်လိုက်တော့။"

ဒါဟာ Version Control System အသုံးမပြုတဲ့ Software Team တွေမှာ ကြုံတွေ့ရလေ့ရှိတဲ့ ပြဿနာတစ်ရပ်ပဲ ဖြစ်ပါတယ်။ Software Project တစ်ခုဆိုတာဟာ အချိန်နဲ့အမျှ ပြောင်းလဲနေတက်တဲ့သဘာဝ ရှိပါတယ်။ ရှေ့ပိုင်းမှာလည်း၊ တိကျတဲ့ Requirement ရရှိဖို့ ခက်ခဲတက်ကြောင်းနဲ့၊ အလုပ်လုပ်နေတဲ့ Code ကို တစ်နေရာမှာ ပြုပြင်ခြင်းအားဖြင့် အခြား တစ်နေရာမှာ မမျှော်လင့်ပဲ သက်ရောက်သွားပြီး Bug တွေ ဖြစ်ပေါ်တက်ကြောင်းတို့ကို ဆွေးနွေးခဲ့ပြီးဖြစ်ပါတယ်။ ဒီလို အခက်အခဲတွေ ရှိတက်လို့ဆိုပြီး ပြုပြင်မှုတွေကို မလုပ်ချင်လို့ မရပါဘူး။

အပြောင်းအလဲတွေရှိလာလို့ ပြုပြင်ဖြည့်စွက်ရတဲ့အခါမှာ၊ မမျှော်မှန်းထားတဲ့ အမှားတစ်ခုကြောင့် မူလအလုပ်လုပ်နေတဲ့ စနစ်ကို ထိခိုက်ပြီး အလုပ်မလုပ်တော့တာမျိုး ဖြစ်တက်ပါတယ်။ ဒါကြောင့် လက်ရှိအလုပ်လုပ်နေတဲ့ အဆင့်တစ်ခုကို Version တစ်ခုအဖြစ် သတ်မှတ်ပြီး၊ သီးခြားခွဲခြားသိမ်းဆည်းထားပြီးမှ ဖြည့်စွက်ပြင်ဆင်မှုတွေကို ဆက်လက်ဆောင်ရွက်ရတဲ့ သဘောရှိပါတယ်။ အဲ့ဒီလိုခွဲခြားထားမှလည်း၊ ဖြည့်စွက် Code တွေကို ရဲရဲဝံ့ဝံ့ ဖြည့်စွက်စမ်းသပ်နိုင်မှာဖြစ်ပါတယ်။ အသစ်ဖြည့်စွက်မှုကြောင့် အမှားအယွင်း တစ်စုံတစ်ရာ ရှိခဲ့ရင်လည်း မူလအလုပ်လုပ်နေတဲ့အခြေအနေကို အချိန်မရွေး ပြန်လည်ရရှိနိုင်မှာဖြစ်ပါတယ်။ Version ခွဲခြားမထားရင် မှားယွင်းမှုဖြစ်သွားမှာကို စိုးရိမ်စိတ်နဲ့ ပြင်ဆင်ဖြည့်စွက်မှုတွေကို ရဲရဲဝံ့ဝံ့ မလုပ်ရဲတော့တဲ့အတွက် အလုပ်မတွင်ဖြစ်တက်ပါတယ်။ ဒီထက်ပိုဆိုးတာက၊ ပြင်ဆင်ဖြည့်စွက်မှုကြောင့် အမှားအယွင်းရှိလာတဲ့အခါ မူလအခြေအနေ ပြန်ရောက်ဖို့ကို အတော်လေး အချိန်ပေးပြီး ပြန်လည်ရှင်းလင်းရတက်လို့ အချိန်တွေ ကုန်တက်ပါသေးတယ်။

ဒီလို Version တွေခွဲခြား မှတ်တမ်းတင်တဲ့ လုပ်ငန်းဆောင်ရွက်ရာမှာ၊ Source Code ဖိုင်တွေကို ကိုယ်တိုင် Copy တွေ ကူး၊ နာမည်အမျိုးမျိုးပေး စသဖြင့် Manual လုပ်နေစရာမလိုပဲ၊ စနစ်ကျထိရောက်အောင် ကူညီပေးနိုင်တာကတော့ Version Control System (VCS) တွေပဲဖြစ်ပါတယ်။ Version Control System ကို Revision Control System လို့လည်းခေါ်သလို၊ Source Code Management System (SCM) လို့လည်း ခေါ်တက်ကြပါသေးတယ်။

Version Control System တွေက Version တွေသာမက Branch လုပ်ဆောင်ချက်တွေလည်း ပါဝင်တက်ပါတယ်။ ဥပမာ - Code Base တစ်ခုတည်းကို အခြေခံပြီး၊ Free Version, Pro Version စသဖြင့် Branch များ ခွဲခြားဆောင် ရွက်နိုင်ခြင်းပဲ ဖြစ်ပါတယ်။



ပုံ (၅.၁) - Branching in VCS

ပုံ (၅.၁) မှာလေ့လာကြည့်ပါ။ v1, v2, v3 စတဲ့ Version တွေဟာ အဆင့်လိုက်မှတ်တမ်းတင်ထားတဲ့ မူလ Version တွေဖြစ်ပါတယ်။ ဒီမူလ Code Base ကနေ လုပ်ဆောင်ချက်အနည်းငယ်ကွဲပြားတဲ့ သီးခြား Version တွေဖန်တီးဖို့ အတွက် နောက်ထပ် Project တစ်ခု ဖွင့်ဖို့မလိုပါဘူး။ လက်ရှိ Code Base ကနေပဲ Branch အဖြစ် ခွဲထုတ်လိုက်ပြီး အတူတစ်ကွ ဆက်လက်ဆောင်ရွက်နိုင်စေမှာဖြစ်ပါတယ်။ မူလ Code Base ကို တစ်ချို့ VCS တွေက Trunk လို့ခေါ်ပြီး၊ တစ်ချို့ကတော့ master လို့ ခေါ်တက်ကြပါတယ်။

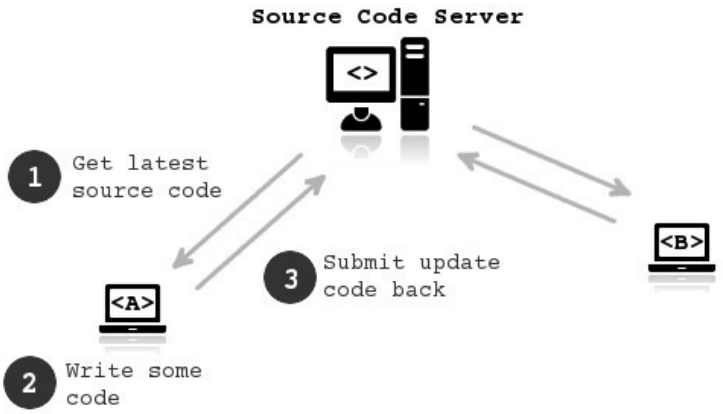
VCS တွေက master နဲ့ Branch တွေအကြား အပြန်အလှန် ကူးပြောင်းအလုပ်လုပ်နိုင်အောင် စီမံပေးထားကြပါတယ်။ master ကို ရွေးထားပြီး ပြင်ဆင်မှုတွေပြုလုပ်ရင် ပြုလုပ်လိုက်တဲ့ပြင်ဆင်မှုတွေက master Code Base ကိုပဲ သက်ရောက်စေမှာဖြစ်သလို၊ Branch တစ်ခုကိုရွေးထားပြီး ပြင်ဆင်မှုတွေပြုလုပ်ရင်တော့ ရွေးချယ်ထားတဲ့ Branch ကိုပဲ ပြင်ဆင်မှုက သက်ရောက်စေမှာ ဖြစ်ပါတယ်။

Branch တွေကို စမ်းသပ်လုပ်ဆောင်ချက်တွေ ထည့်သွင်းစမ်းသပ်ဖို့လည်း အသုံးပြုကြပါတယ်။ ဒီနည်းနဲ့ စမ်းသပ်လိုတဲ့ Code တွေက master Code Base ကို မထိခိုက်စေပဲ၊ သီးခြားခွဲထားတဲ့ Development Branch ပေါ်မှာ လွပ်လပ်လပ်လပ် စမ်းသပ်ရေးသားနိုင်စေမှာဖြစ်ပါတယ်။ Development Branch ပေါ်မှာ ရေးသားထားတဲ့ စမ်းသပ် လုပ်ဆောင်ချက်ကို မူလ Version မှာ ပါဝင်သင့်တယ်လို့ အတည်ပြုနိုင်တဲ့အခါမှာ အဲ့ဒီ Development Branch ကို master Code Base နဲ့ ပေါင်းစပ်လိုက်နိုင်ပါတယ်။ ဒီလုပ်ဆောင်ချက်ကို Merge လုပ်တယ်လို့ ခေါ်ပါတယ်။

ပုံ (၅.၁) မှာ ဖော်ပြထားချက်အရ၊ အလယ်က v1, v2, v3 စတဲ့ Version တွေဖြစ်ပြီး၊ အပေါ်က v1-free, v1.1-free စတဲ့ Version တွေကတော့ မူလ master Code Base ကနေ လုပ်ဆောင်ချက်တစ်ချို့ ပြောင်းလဲထားတဲ့ သီးခြား Version များအဖြစ် Branch ခွဲယူထားခြင်းဖြစ်ပါတယ်။ အောက်ဘက်က v1.1-dev, v2.1-dev စတဲ့ Version တွေက တော့ စမ်းသပ်လုပ်ဆောင်ချက်တွေ ထည့်သွင်းစမ်းသပ်နိုင်ဖို့ ခွဲခြားထားတဲ့ Branch ဖြစ်ပြီး အဲ့ဒီ Branch ကို မူလ master Code Base နဲ့ တစ်နေရာမှာ ပြန်လည် Merge လုပ်ထားတာကိုလည်း တွေ့ရမှာဖြစ်ပါတယ်။

VCS တွေက master နဲ့ Branch တွေပေါ်က မှတ်သားထားတဲ့ Version အမှတ်တိုင်းကို ပြန်သွားနိုင်အောင်လည်း စီမံ ထားပေးလို့ အမှားအယွင်းတစ်စုံတစ်ရာကြောင့်ပဲဖြစ်ဖြစ် အခြားအကြောင်း တစ်ခုခုကြောင့်ပဲဖြစ်ဖြစ် လိုအပ်လာတဲ့အခါ အရင် Version တွေကို အချိန်မရွေး ပြန်လည်ရယူနိုင်မှာပဲဖြစ်ပါတယ်။

ဒါတင်မက၊ အဖွဲ့လိုက်ပူးပေါင်းဆောင်ရွက်ရတဲ့အခါမှာလည်း Code Base တစ်ခုတည်းကနေ VCS အကူအညီနဲ့ ရယူရေး သားခြင်းအားဖြင့် Developer တစ်ဦးရေးသားထားတဲ့ Code နဲ့ နောက် Developer တစ်ဦးရေးသားထားတဲ့ Code ကို ပေါင်းစပ်တဲ့လုပ်ငန်းကို Manual လုပ်နေစရာမလိုတော့ပဲ VCS က စနစ်တစ်ကျပေါင်းစပ်ပေးသွားမှာပဲဖြစ်ပါတယ်။ ဒါကြောင့် VCS တွေကို Version တွေစီမံဖို့အတွက်သာမက အဖွဲ့လိုက်ပူးပေါင်းဆောင်ရွက်ရာမှာ အခြေခံ Tool တစ်ခု အဖြစ် အသုံးပြုတက်ကြပါသေးတယ်။ ဒါကြောင့်လည်း VCS တွေကို Source Code Management System (SCM) လို့ခေါ်ကြခြင်းဖြစ်ပါတယ်။ Version ကိုသာမက၊ Source Code နဲ့ ပက်သက်တဲ့ အခြားစီမံမှုလုပ်ငန်းများကိုပါ ဆောင်ရွက်ပေးတဲ့အတွက် ဖြစ်ပါတယ်။



ပုံ (၅.၂) - Centralized VCS Workflow

ဒီလိုအဖွဲ့လိုက်ပူးပေါင်းဆောင်ရွက်နိုင်ဖို့ Centralize Source Code Server လိုအပ်တက်ပါတယ်။ ဒီတော့မှ **ပုံ (၅.၂)** မှာ ဖော်ပြထားသလို၊ Team မှာပါဝင်တဲ့ Developer တွေက၊ အခြား Developer များရေးသားထားတဲ့ နောက်ဆုံးရလဒ်ကို Central Server ထံမှရယူခြင်း၊ မိမိတို့ရေးသားဖြည့်စွက်ချက်တွေကို Central Server ထံ ပြန်လည်ပေးပို့ခြင်းအားဖြင့် Code Base တစ်ခုတည်းကို Developer အများ ပူးပေါင်းဆောင်ရွက်ခြင်း လုပ်ဆောင်ချက်ကို ရရှိမှာပဲဖြစ်ပါတယ်။

5.1 VCS Terminologies

Version Control System တွေအများကြီးရှိပါတယ်။ စနစ်တစ်ခုနဲ့တစ်ခု အသေးစိတ်လုပ်ဆောင်ချက်တွေ မတူကြပေမယ့် အခြေခံရည်ရွယ်ချက်တူပြီး၊ ပါဝင်တဲ့ Function တွေနဲ့ လုပ်ငန်းစဉ် Operation တွေကို ခေါ်တဲ့အခေါ်အဝေါ်တွေလည်း တူတက်ပါတယ်။ VCS တစ်ခုမှာပါဝင်တက်တဲ့ အသုံးအနှုန်းတွေကို ဖော်ပြပေးလိုက်ပါတယ်။

revision/version – Source Code ကို ပြင်ဆင်မှုတစ်ချို့လုပ်ပြီး မှတ်တမ်းတင်သိမ်းဆည်းလိုက်တဲ့ အမှတ်တစ်ခုကို version (သို့မဟုတ်) revision လို့ ခေါ်ပါတယ်။

branch – အတိုင်းအတာတစ်ခုထိ ပြီးမြောက်ပြီးသား Software Project တစ်ခုရဲ့ Source Code ကို ပြင်ဆင်ဖြည့်စွက်မှုတွေ ပြုလုပ်လိုတဲ့အခါ၊ မူလ Code Base ပေါ်မှာ တိုက်ရိုက်လုပ်မယ့်အစား သီးခြား branch တစ်ခုအဖြစ် မူပွားယူပြီးမှ ဆောက်ရွက်ကြလေ့ရှိပါတယ်။ ဒီနည်းနဲ့ စမ်းသပ်မှုကြောင့် ဖြစ်ပေါ်လာတဲ့ မလိုလားအပ်တဲ့ ပြဿနာတွေ မူလ master Code Base ကို သက်ရောက်ခြင်း မရှိတော့မှာပါ။ သီးခြား branch ပေါ်မှာ ဆောင်ရွက်နေတဲ့ ဖြည့်စွက်ပြင်ဆင်ချက်ဟာ လက်ခံနိုင်တဲ့ အတိုင်းအတာတစ်ခု ရောက်ပြီဆိုတော့မှ အဲ့ဒီ branch ကို မူလ master Code Base နဲ့ ပေါင်းစပ်စေခြင်း အားဖြင့် master Code Base ထဲမှာလည်း ဖြည့်စွက်ပြင်ဆင်ချက်အသစ်တွေ ရောက်ရှိပါဝင်သွားစေမှာ ဖြစ်ပါတယ်။

fork – fork ဆိုတာ branch လိုပဲ မူလ Code Base ကို မူပွားယူခြင်းဖြစ်ပါတယ်။ ဒါပေမယ့်၊ မူလ Code Base နဲ့ ပြန်လည်ပေါင်းစပ်ဖို့မရည်ရွယ်ပဲ သီးခြား Project တစ်ခုအနေနဲ့ ဆောင်ရွက်ဖို့ ရည်ရွယ်ချက်နဲ့ မူပွားယူတဲ့ သဘောမျိုးဖြစ်ပါတယ်။

master/trunk – နောက်မှ ခွဲယူထားတဲ့ branch မူကွဲမဟုတ်ပဲ၊ မူလစတင်ကတည်း အခြေပြုထားတဲ့ မူလ Code Base ကို master branch (သို့မဟုတ်) trunk လို့ခေါ်ပါတယ်။

merge – branch တစ်ခုနဲ့ ခွဲခြားစီမံနေတဲ့ Code Base ကို master (သို့မဟုတ်) အခြား branch တစ်ခုခုနဲ့ ပေါင်းစပ်စေတဲ့ လုပ်ဆောင်ချက်ဖြစ်ပါတယ်။ branch တစ်ခုတည်းမှာပဲ Developer တစ်ဦးရဲ့ပြင်ဆင်ချက်ကို အခြားတစ်ဦးရဲ့ ပြင်ဆင်ချက်နဲ့ ပေါင်းစပ်ခြင်းလုပ်ငန်းကိုလဲ merge လုပ်တယ်လို့ပဲ ခေါ်ပါတယ်။

commit/checkin – ပြင်ဆင်ဖြည့်စွက်မှုတွေပါဝင်တဲ့ Code ကို Version သစ်တစ်ခုအဖြစ် မှတ်တမ်းတင်လိုက်တဲ့ လုပ်ဆောင်ချက်ကို commit လုပ်တယ် (သို့မဟုတ်) checkin လုပ်တယ်လို့ခေါ်ပါတယ်။

change/diff – ပြင်ဆင်ထားတဲ့ Code နဲ့ နောက်ဆုံး commit လုပ်ခဲ့တဲ့ Version တို့အကြား ကွာခြားသွားတဲ့ ကွာခြားချက်ကို change (သို့မဟုတ်) diff လို့ခေါ်ပါတယ်။ ကွာခြားသွားတယ်ဆိုတာ အသစ်တိုးလာတာ လည်း ဖြစ်နိုင်ပါတယ်၊ မူလ Code ကို ပြင်လိုက်တာလည်း ဖြစ်နိုင်ပါတယ်၊ မူလ Code အချို့ကို ဖယ်ထုတ် လိုက်တာ လည်း ဖြစ်နိုင်ပါတယ်။

delta compression – Version အသစ်တစ်ခုအဖြစ် မှတ်တမ်းတင်သိမ်းဆည်းတဲ့အခါ Code Base တစ်ခုလုံးကို နောက်တစ်ကြိမ် ထပ်မသိမ်းတော့ပဲ၊ ကွားခြားသွားတဲ့ diff ကိုသာ သိမ်းဆည်းတဲ့လုပ်ဆောင်ချက်ကို delta compression လို့ခေါ်ပါတယ်။

change list/patch – ဥပမာ – Open Source Project တစ်ခုကို fork လုပ်ယူပြီး ကိုယ်ပိုင်ဖြည့်စွက်မှုတွေ လုပ်တယ် ဆိုပါစို့။ ရရှိလာတဲ့ ပြင်ဆင်ဖြည့်စွက်မှုမှတ်တမ်း Version ကို မူလ Project မှာပါဝင်သွားစေလိုတဲ့အခါ – မိမိ ပြုလုပ်ထားတဲ့ ပြင်ဆင်ဖြည့်စွက်မှုမှတ်တမ်းကို change list (သို့မဟုတ်) patch အဖြစ်ထုတ်ယူရပါတယ်။ ရရှိလာတဲ့ patch ဖိုင်ကိုမူလ Project ရဲ့ တာဝန်ခံထံ ပေးပို့နိုင်ပါတယ်။ တာဝန်ခံက စီစစ်လေ့လာပြီး ပေါင်းစပ်ပေးမယ်ဆိုရင် မိမိပြုလုပ်ထားတဲ့ ပြင်ဆင်ဖြည့်စွက်မှုတွေလည်း မူလ Project မှာ ပါဝင်သွားမှာဖြစ်ပါတယ်။

repository – Version အားလုံးနဲ့ Branch အားလုံးတို့ကို စုစည်းသိမ်းဆည်းထားတဲ့ Code Base ကြီးကို repository လို့ ခေါ်ပါတယ်။

checkout – Central Source Server ကနေ၊ နောက်ဆုံး Update Version (သို့မဟုတ်) Version တွေ ထဲကတစ်ခုကို ကိုယ့် Local Machine ထဲ ကူးယူတဲ့ လုပ်ဆောင်ချက်ကို checkout လုပ်တယ်လို့ခေါ်ပါတယ်။ တစ်ချို့ VCS တွေမှာ တော့ branch တစ်ခုကနေ နောက်တစ်ခုကိုကူးပြောင်းတဲ့ လုပ်ဆောင်ချက်အတွက်လည်း checkout ဆိုတဲ့ အသုံးအနှုံးကို ပဲသုံးပါတယ်။

working copy – Central Source Server ကနေ၊ checkout လုပ်ယူလိုက်လို့ ကိုယ့် Local Machine ထဲ ရောက်ရှိလာတဲ့ Version ကို working copy လို့ခေါ်ပါတယ်။

clone – clone ဆိုတာကတော့ Central Server ပေါ်က repository ကြီးတစ်ခုလုံးကို ကိုယ့် Local Machine မှာ မူပွား ယူလိုက်တဲ့ လုပ်ဆောင်ချက်ဖြစ်ပါတယ်။ checkout နဲ့မတူတာက၊ checkout က Version တစ်ခုကိုသာ ရယူခြင်းဖြစ်ပြီး၊ clone က Version တွေ branch တွေအားလုံးပါဝင်တဲ့ repository တစ်ခုလုံးကို ရယူခြင်းဖြစ်ပါတယ်။

conflict – Local Copy ပေါ်မှာ ပြင်ဆင်ဖြည့်စွက်မှုတစ်ချို့ လုပ်လိုက်တယ်ဆိုပါစို့။ အဲ့ဒီ ပြင်ဆင်မှုကို Central Server ထံမပေးပို့မီ Server က နောက်ဆုံး Version ကို အရင်ရယူရတဲ့သဘော ရှိပါတယ်။ ဒီလိုရယူလိုက်တဲ့ အခါ ကိုယ်ပြင်ဆင် ထားတဲ့ Code ဖိုင် နဲ့ Server ကနေရရှိလာတဲ့ နောက်ဆုံး Version ရဲ့ Code ဖိုင် တူညီနေတဲ့ အခါ conflict ဖြစ်ပါတယ်။ VCS က ကိုယ်ပြင်ထားတာကို နောက်ဆုံးအနေနဲ့ မှတ်ယူပေးရမလား၊ Server ကနေ ရရှိလာတဲ့ ပြင်ဆင်ချက်ကို နောက် ဆုံးအနေနဲ့ မှတ်ယူပေးရမလား မဆုံးဖြတ်နိုင်တဲ့အတွက် conflict ဖြစ်ရခြင်း ဖြစ်ပါတယ်။ ဒီလို conflict တွေရှိလာတဲ့ အခါ ကိုယ်တိုင်စီစစ်ပြီး ပေါင်းစပ်၍သော်လည်းကောင်း၊ နှစ်ခုထဲက တစ်ခုကို နောက်ဆုံးအဖြစ် ဆုံးဖြတ်ပေး၍သော် လည်းကောင်း **resolve** လုပ်ပေးရတက်ပါတယ်။

head – HEAD လို့ သုံးကြလေ့ရှိပြီး နောက်ဆုံး commit, နောက်ဆုံး Version ကို ညွှန်းဆိုတဲ့ Keyword ဖြစ်ပါတယ်။

tag/label – commit လုပ်ပြီး မှတ်တမ်းတင်ခဲ့တဲ့ Version ပေါင်းများစွာရှိနိုင်ပါတယ်။ အဲ့ဒီထဲကမှ အရေးပါတဲ့ Version တွေကို အမည်ပေးတဲ့လုပ်ငန်းကို tag (သို့မဟုတ်) label လို့ခေါ်ပါတယ်။ ဥပမာ – အဆင့်ဆင့် သိမ်းဆည်းလာတဲ့ Version တွေထဲက Version အမှတ်တစ်ခုကို 0.1.0 လို့ tag လုပ်ထားနိုင်ပါတယ်။

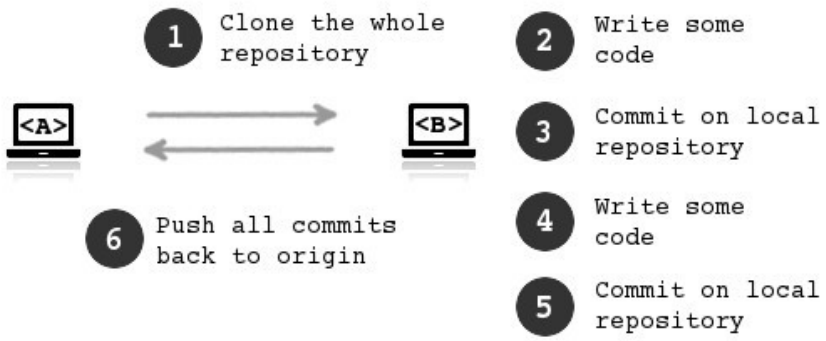
Version နံပါတ် ပေးပုံပေးနည်းကို မကြာခင်မှာ ဆက်လက်ဖော်ပြသွားပါမယ်။

5.2 – Centralized VCS vs. Distributed VCS

Version Control System တွေမှာ အမျိုးအစားအားဖြင့် နှစ်မျိုးရှိပါတယ်။ တစ်မျိုးကို Centralized Version Control System လို့ခေါ်ပြီး နောက်တစ်မျိုးကိုတော့ Decentralized Version Control System (သို့မဟုတ်) Distributed Version Control System လို့ ခေါ်ပါတယ်။

Centralized VCS တွေမှာ Source Code Repository ကို Central Server တစ်လုံးမှာထားပြီး Developer တွေက ကိုယ့် Local Machine ပေါ်မှာ Working Copy ကို ရယူအလုပ်လုပ်ကြရပါတယ်။ ဆိုလိုတာက၊ Branch တွေနဲ့ Version မှတ်တမ်းဟာ Central Server မှာသာရှိနေပြီး Developer တွေက နောက်ဆုံး Version ကိုသာ ရယူ အလုပ်လုပ်ကြခြင်း ဖြစ်ပါတယ်။ Commit လုပ်လိုက်တိုင်းမှာ Update Source Code ကို Central Server ထံ ပေးပို့ခြင်းအားဖြင့် Version မှတ်တမ်းသစ်ကို Central Server ပေါ်မှာ တိုက်ရိုက်သိမ်းဆည်းသွားမှာ ဖြစ်ပါတယ်။

Distributed VCS တွေမှာတော့ Central Server ရှိဖို့ မလိုပါဘူး။ မည်သူမဆို မိမိတို့ Local Machine ပေါ်မှာ Repository တစ်ခုတည်ဆောက်နိုင်ပါတယ်။ အခြား Developer တွေက အဲ့ဒီ Repository ကို ရယူဖို့လိုတဲ့အခါ Repository တစ်ခုလုံးကို Clone လုပ်ယူနိုင်ပါတယ်။ ဒါကြောင့် Team မှာ ပါဝင်သူအားလုံးမှာ Repository တစ်ခုလုံးရဲ့ မူပွားတွေ ကိုယ်စီရှိနေတဲ့သဘောဖြစ်ပြီး ပြုလုပ်လိုတဲ့ ပြင်ဆင်ဖြည့်စွက်မှုတွေကို မူလ Original Repository နဲ့ ဆက်သွယ်ဖို့မလိုပဲ ကိုယ့်မူပွားပေါ်မှာ ဆက်လက် ဆောင်ရွက်ရပါတယ်။ **ပုံ (၅.၃)** မှာ လေ့လာကြည့်ပါ။



ပုံ (၅.၃) - Distributed VCS Workflow

Version တစ်ခုအဖြစ် မှတ်တမ်းတင်ဖို့ Commit လုပ်လိုက်တဲ့အခါမှာ မူလ Original Repository ကို သွားပြီး သက်ရောက် တာမျိုးမဟုတ်ပဲ ကိုယ့် Local Repository ပေါ်မှာပဲ သက်ရောက်မှာဖြစ်ပါတယ်။ Code တွေ ပေါင်းစပ်ဖို့လို အပ်လာတဲ့အခါမှာ တစ်ဦးမှာရှိနေတဲ့ Version တွေနဲ့ အခြားတစ်ဦးထံမှာရှိနေတဲ့ Version တွေကို အတွဲလိုက်ပေါင်းစပ်ယူ နိုင်ပါတယ်။

Distributed VCS တွေမှာ Version မှတ်တမ်းများ သိမ်းဆည်းခြင်း၊ Branch များခွဲယူခြင်း၊ ပြန်လည် ပေါင်းစပ်ခြင်း၊ စတဲ့ အခြေခံလုပ်ဆောင်ချက်တွေကို Network Communication မလိုပဲ Local Machine ပေါ်မှာပဲ ဆောင်ရွက်နိုင်တဲ့ အတွက် အများကြီးပိုမြန်ပါတယ်။ အားလုံးမှာ Repository မူပွားကိုယ်စီရှိနေတဲ့အတွက် Server ကွန်ပျူတာ ပျက်သွားလို့ Source Code တွေ ဆုံးရှုံးရတယ်ဆိုတာမျိုးလည်း မဖြစ်နိုင်တော့ပါဘူး။ ဒီလို အားသာချက်တွေကြောင့် အခုနောက်ပိုင်းမှာ Distributed VCS တွေကို ပိုပြီး အသုံးများလာကြပါတယ်။

5.3 – Git

Git ဟာ Linux Kernel ရဲ့ မူလဖန်တီးရှင်ဖြစ်တဲ့ Linus Torvalds ကိုယ်တိုင် (၂၀၀၅) ခုနှစ်မှာ စတင်တီထွင်ခဲ့တဲ့ Distributed VCS ဖြစ်ပါတယ်။ Open Source Software တစ်ခုဖြစ်ပြီး လက်ရှိ လူသုံးအများဆုံး VCS လည်းဖြစ်ပါတယ်။ Linus Torvalds နဲ့ Linux Kernel Developer တွေဟာ မူလက BitKeeper လို့ခေါ်တဲ့ DVCS တစ်ခုကို အသုံးပြု ပြီး Linux Kernel တစ်ခုလုံးကို ထိမ်းသိမ်းလာခဲ့ကြတာ ဖြစ်ပါတယ်။



BitKeeper ဟာ Open Source Software တစ်ခု မဟုတ်ပါဘူး။ Propitiatory Software တစ်ခုပါ။ ဒါပေမယ့် Linux Kernel Developer တွေကို ချွင်းချက် အနေနဲ့အခမဲ့အသုံးပြုခွင့် ပေးထားခဲ့ပါတယ်။ ၂၀၀၅ ရောက်တဲ့အခါမှာ ပြဿနာတစ်ခုကြောင့် Bitkeeper ရဲ့ မူလပိုင်ရှင်က Linux Kernel Developer တွေ အတွက် ပေးထားတဲ့ အခမဲ့အသုံးပြုခွင့်ကို ပြန်လည် ရုတ်သိမ်းလိုက်ပါတယ်။ ဒါကြောင့် Linux Kernel Source Code ကို ဆက်လက်ထိမ်းသိမ်းနိုင်ဖို့အတွက် အခြား VCS တစ်ခုနဲ့ အစားထိုးအသုံး ပြုဖို့ လိုအပ်လာပါတယ်။

Linux Kernel Development ရဲ့ဦးဆောင်သူဖြစ်တဲ့ Linus Torvalds အနေနဲ့ BitKeeper ရဲ့ Distributed သဘောသဘာဝကို နှစ်သက်သဘောကျပါတယ်။ အခြား VCS တွေကို လေ့လာကြည့်တဲ့အခါမှာတော့ အများစုက Centralize VCS တွေ ဖြစ်နေကြပြီး၊ သူလိုချင်တဲ့ အမြန်နှုန်းမရရှိပဲ အလုပ်လုပ်ပုံ နှေးကွေးကြတာကို တွေ့ရပါတယ်။ ဒါကြောင့် မြန်ဆန်တိကျစွာ အလုပ်လုပ်နိုင်တဲ့ VCS တစ်ခု သူကိုယ်တိုင်ရေးဖို့ ဆုံးဖြတ်ပြီး ၂၀၀၅ ခုနှစ် ဧပြီလထဲမှာ Git ကို စတင် ဖန်တီးခဲ့ခြင်းပဲဖြစ်ပါတယ်။ ဒီလိုဖန်တီးရာမှာ အဓိကဦးတည်ချက်သုံးချက် ရှိခဲ့ပါတယ်။

1. Distributed VCS ဖြစ်ရမယ်၊
2. အမြန်နှုန်းကောင်းရမယ်၊
3. ဘယ်လိုအခြေအနေမှာပဲဖြစ်ဖြစ် Data ပျက်စီးဆုံးရှုံးခြင်း လုံးဝမရှိအောင် အာမခံနိုင်တဲ့ စနစ်တစ်ခု ဖြစ်ရမယ်

– ဆိုတဲ့ အချက်တွေပဲဖြစ်ပါတယ်။

Git Installer ကို http://git-scm.com/downloads မှာ ရယူနိုင်ပါတယ်။ Linux, Mac, Windows စတဲ့ အဓိက OS အားလုံးမှာအသုံးပြုနိုင်ပါတယ်။ Ubuntu Linux မှာဆိုရင်တော့ အောက်ပါအတိုင်း အလွယ်တစ်ကူ Install လုပ်နိုင်ပါတယ် –

```
$ sudo apt-get install git-core
```

Windows အတွက် Official Installer အစား **msysgit** လို့ခေါ်တဲ့ Package တစ်ခုကိုလည်း အစားထိုး အသုံးပြုနိုင်ပါ တယ်။ msysgit က Git သာမက Git Bash, Windows Explorer Integration စတဲ့ ဆက်စပ်ပရိုဂရမ်တွေကိုပါ ထည့်သွင်းပေးတဲ့အတွက် ပိုမိုပြည့်စုံတဲ့ Installer ဖြစ်တယ်လို့ ဆိုနိုင်ပါတယ်။ msysgit ကို အောက်ပါလိပ်စာမှာ Download ရယူနိုင်ပါတယ်။

http://msysgit.github.io/

ဒီနေရာမှာ Install လုပ်ပုံအဆင့်ဆင့်ကို တစ်ဆင့်ချင်း ဖော်ပြမနေတော့ပါဘူး။ အသုံးပြုပုံကိုသာ ဆက်လက် ဖော်ပြပေး သွားမှာဖြစ်ပါတယ်။ ပြီးတော့ Git ဟာ Command Line ပရိုဂရမ်တစ်ခုဖြစ်ပါတယ်။ Graphical User

Interface နဲ့ အသုံးပြုလိုတယ်ဆိုရင်လည်း UI Client တွေ ထပ်မံထည့်သွင်းအသုံးပြု နိုင်ပေမယ့် Command Line ကနေအသုံးပြုခြင်းက ပိုမိုထိရောက်ပြီးအလုပ်တွင်စေနိုင်တဲ့အတွက် Command Line ကနေ အသုံးပြုပုံကို သာ ဖော်ပြသွားမှာပါ။

> ဒီအခန်းမှာဖော်ပြသွားမယ့် Command တွေဟာ လက်တွေ့ကူးယူစမ်းသပ်လို့ရတဲ့ Command တွေ ဖြစ်ပါတယ်။ ဒါပေမယ့် Operating System အမျိုးအစား၊ Install လုပ်ထားတဲ့ Git Version, File Permission စတဲ့အချက်တွေပေါ်မူတည်ပြီး အချို့ကွဲလွဲမှုတွေ ရှိနိုင်တဲ့အတွက်၊ တိုက်ရိုက်ကူးယူ စမ်းသပ်ခြင်းမပြုပဲ အလုပ်လုပ်ပုံသဘာဝကို အဓိကထားလေ့လာသင့်ပါတယ်။

Configuration

Git ကို Install လုပ်ပြီးနောက်၊ စတင်အသုံးပြုနိုင်ဖို့အတွက် အသုံးပြုမယ့်သူရဲ့ အမည်နဲ့ Email လိပ်စာကို Configuration ဖိုင်ထဲမှာ ဖြည့်သွင်းပြီး သတ်မှတ်ပေးရပါတယ်။ Global Config နဲ့ Local Config ဆိုပြီး Configuration ဖိုင် အမျိုး အစား နှစ်မျိုးရှိပါတယ်။ စက်ထဲမှာရှိတဲ့ Git Repository အားလုံးကို သက်ရောက်စေလိုတဲ့ Setting တွေကို Global Config ထဲမှာသတ်မှတ် ပေးရပါတယ်။ သက်ဆိုင်ရာ Repository တစ်ခုကိုသာ သက်ရောက်စေလိုတဲ့ Setting တွေကို တော့ Local Config ထဲမှာ သတ်မှတ်ထားနိုင်ပါတယ်။

Configuration ဖိုင်ထဲမှာ Setting တွေ သတ်မှတ်တယ်ရတယ်ဆိုပေမယ့် ဖိုင်တွေကို လိုက်ဖွင့်ပြီး သတ်မှတ်နေစရာမလိုပါ ဘူး။ Configuration ဖိုင်တွေရဲ့ တည်နေရာဟာ အသုံးပြုတဲ့ OS နဲ့ Install လုပ်စဉ်က ရွေးချယ်ခဲ့တဲ့ အနေအထားပေါ် မူတည်ပြီး တစ်ယောက်နဲ့တစ်ယောက်မတူပဲ ကွဲပြားနိုင်ပါတယ်။ Setting တွေသတ်မှတ်ဖို့ အတွက် git config ဆိုတဲ့ Command ကိုသုံးနိုင်ပါတယ်။ အသုံးပြုမယ့်သူရဲ့ အမည်နဲ့ Email လိပ်စာကို Global Config ဖိုင်ထဲမှာ ထည့်သွင်း သွားစေဖို့ အခုလို Command ကို အသုံးပြုနိုင်ပါတယ်။

```
$ git config --global user.name "John Doe"
$ git config --global user.email "me@johndoe.com"
```

Global Config အဖြစ်သတ်မှတ်လိုတဲ့ Setting တွေကို `--global` Option နဲ့ တွဲဖက်သတ်မှတ်ရခြင်းဖြစ်ပါတယ်။ `--global` Option မပါရင်တော့ Local Config အဖြစ် သိမ်းဆည်းမှတ်တမ်းတင်သွားမှာ ဖြစ်ပါတယ်။ အမည် သတ်မှတ်ဖို့အတွက် `user.name` ကိုသုံးရပြီး Email သတ်မှတ်ဖို့အတွက် `user.email` ကို သုံးရခြင်း ဖြစ်ပါတယ်။ သတ်မှတ်ထားတဲ့ တန်ဖိုးမှန်မမှန် ပြည်လည်စီစစ်လိုရင် အခုလိုပြန်လည် စီစစ်နိုင်ပါတယ်။

```
$ git config --global user.name    # => John Doe
```

Git မှာ `user.name` နဲ့ `user.email` တို့လို Setting တွေ အများကြီးရှိပါတယ်။ အောက်ပါ Website လိပ်စာမှာ အသေးစိတ် လေ့လာနိုင်ပါတယ်။

http://git-scm.com/docs/git-config

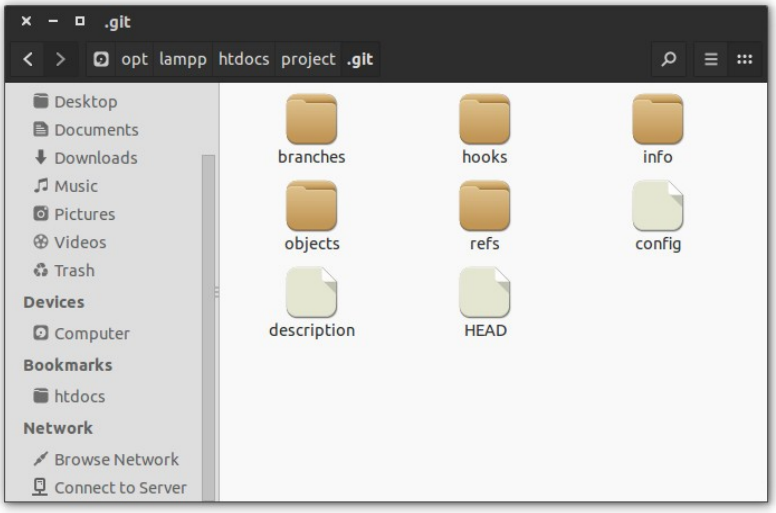
Initializing a Repository

Git ကိုအသုံးပြုပြီး Version မှတ်တမ်းစီမံနိုင်ဖို့အတွက်၊ ပထမဦးဆုံးအနေနဲ့ Source Code Directory ကို Git Repository အဖြစ် ကြေငြာသတ် မှတ်ပေးရပါတယ်။ ဒီလိုကြေငြာသတ်မှတ်ဖို့အတွက် `git init` Command ကို သုံးရပါတယ်။

```
$ git init

Initialized empty Git repository in /path/to/dir/.git/
```

နမူနာမှာပြထားသလို `git init` ကို Run လိုက်တာနဲ့ `.git` အမည်နဲ့ Empty Repository တစ်ခုကို Source Code Directory ထဲမှာ တည်ဆောက်လိုက်ကြောင်း Message ပေးလာမှာဖြစ်ပါတယ်။



ပုံ (၅.၄) - Inside .git Directory

`.git` Directory ကိုဖွင့်ကြည့်လိုက်ရင် **ပုံ (၅.၄)** မှာဖော်ပြထားသလို တွေ့ရမှာဖြစ်ပါတယ်။ Version မှတ်တမ်းတွေ Branch တွေနဲ့ အခြားလိုအပ်တဲ့ အချက်အလက်တွေကို `.git` Directory ထဲမှာ သိမ်းဆည်းသွားမှာဖြစ်ပါတယ်။

Linux အပါအဝင် Unix-like OS တွေမှာ Dot နဲ့စတဲ့ File နဲ့ Directory တွေဟာ Hidden File တွေဖြစ်တယ်ဆိုတာကို သတိပြုပါ။ File Manager နဲ့ ပြန်ကြည့်တဲ့အခါမှာပဲဖြစ်ဖြစ်၊ Terminal ထဲမှာ ကြည့်တဲ့အခါမှာပဲဖြစ်ဖြစ် Hidden File တွေကို ဖော်ပြစေမှ `.git` Directory ကို တွေ့ရမှာဖြစ်ပါတယ်။

`.git` Directory ထဲက အရေးကြီးတဲ့ နှစ်ခုကတော့ `objects` နဲ့ `hooks` တို့ဖြစ်ပါတယ်။ Git က Version မှတ်တမ်းတွေကို `objects` Directory ထဲမှာသိမ်းပါတယ်။ Encrypt လုပ်ပြီးသိမ်းမှာဖြစ်တဲ့အတွက် `objects` Directory ကိုဖွင့်ကြည့်ရင် Hash Value တွေကိုသာ တွေ့ရမှာဖြစ်ပါတယ်။ `hooks` ကိုတော့ Shell Script တွေ သိမ်း ဖို့သုံးပါတယ်။ ဥပမာ - Version မှတ်တမ်းတစ်ခု သိမ်းဆည်းဖို့အတွက် Commit လုပ်လိုက်တဲ့အခါ တွဲဖက်လုပ်ဆောင် သွားစေလိုတဲ့လုပ်ငန်းများ၊ Branch တစ်ခုနဲ့တစ်ခု Merge လုပ်လိုက်တဲ့အခါ ဆောင်ရွက်သွားစေလိုတဲ့ လုပ်ဆောင်ချက်များ စသဖြင့် Shell Script တွေရေးသားပြီး `hooks` ထဲမှာ သိမ်းဆည်းထားနိုင်ခြင်း ဖြစ်ပါတယ်။

နောက်ထပ်သတိပြုသင့်တဲ့ ဖိုင်တစ်ခုဖြစ်တဲ့ `config` ကတော့ Local Configuration File ဖြစ်ပါတယ်။ Text Editor နဲ့ ဖွင့် ကြည့်လိုက်ရင် အခုလိုတွေ့ရနိုင်ပါတယ်။

Config

```
[core]
	repositoryformatversion = 0
	filemode = true
	bare = false
	logallrefupdates = true
```

Git နဲ့အလုပ်တစ်ခုလုပ်တိုင်းမှာ ဒီ Configuration ဖိုင်ထဲမှာ ရေးသားထားတဲ့ Setting တွေကို ဦးဆုံးစီစစ်အသုံးပြုပြီး အလုပ်လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ `HEAD` ဆိုတဲ့ ဖိုင်ထဲမှာတော့ လက်ရှိအလုပ်လုပ်နေတဲ့ Branch ကိုညွှန်းထားတဲ့ အညွှန်းလိပ်စာ သိမ်းဆည်းထားပါတယ်။ Version မှတ်တမ်းတစ်ခု သိမ်းဆည်းဖို့ Commit လုပ်လိုက်တိုင်းမှာ `HEAD` ဖိုင်ထဲက Branch အညွှန်းကိုကြည့်ပြီး Version မှတ်တမ်းကို သိမ်းဆည်းပေးသွားမှာဖြစ်ပါတယ်။

လက်တွေ့မှာ Version မှတ်တမ်းသိမ်းခြင်း၊ Branch တွေခွဲခြင်းစတဲ့ အခြေခံလုပ်ငန်းတွေလုပ်ဖို့အတွက် `.git` Directory ကို ပြင်ဖို့မလိုပါဘူး။ အထူးလိုအပ်ချက်ရှိတော့မှသာ ကိုယ်တိုင်ဖွင့်ပြီးစီမံဖို့လိုမှာပါ။ ဆက်လက်ဖော်ပြမယ့် Git Command တွေကို Run တဲ့အခါမှာ Source Code Directory ထဲမှာပဲ Run ရပါတယ်။ `.git` Directory ထဲမှာ Run စရာမလို ပါဘူး။

Recording and Managing Versions

`git init` နဲ့ Initialize လုပ်ထားပြီးဖြစ်တဲ့ Git Repository ထဲမှာ လိုအပ်တဲ့ Source Code ဖိုင်နဲ့ Directory တွေ တည်ဆောက်ခြင်း Code ရေးသားခြင်း၊ Compile ပြုလုပ်ခြင်းစတဲ့ လုပ်ငန်းတွေကို လုပ်ရိုးလုပ်စဉ်အတိုင်း ဆောင်ရွက်နိုင် ပါတယ်။ ရေးသားထားတဲ့ Code တွေကို Version တစ်ခုအဖြစ် မှတ်တမ်းတင်တော့မယ်ဆိုရင်တော့၊ Version မှတ်တမ်းထဲမှာ ပါဝင်စေလိုတဲ့ဖိုင်နဲ့ Directory တွေကို Index ထဲထည့်ပေးရပါတယ်။ နမူနာရစေ

ဖို့အတွက် Git Repository ထဲမှာ ဖိုင်အလွတ် တစ်ချို့ကို အခုလိုတည်ဆောက် ကြည့်နိုင်ပါတယ်။

```
$ touch index.html style.css script.js
```

`touch` Command က File တွေမရှိသေးရင် ဆောက်ပေးတယ်လို့ပဲ အလွယ်မှတ်ယူပေးပါ။ `touch` Command မသုံးချင်ရင်လည်း နှစ်သက်ရာနည်းလမ်းနဲ့ `index.html`, `style.css` နဲ့ `script.js` ဆိုတဲ့ ဖိုင်အလွတ် (၃) ခုကိုတည်ဆောက် ပေးနိုင်ပါတယ်။ ပြီးတဲ့အခါ `git status` Command ကို Run ကြည့်လိုက်ရင် အခုလို ရလဒ်ကို တွေ့ရမှာဖြစ်ပါတယ်။

```
$ git status

On branch master
Initial commit
Untracked files:
(use "git add <file>..." to include in what will be committed)

      index.html
      script.js
      style.css

nothing added to commit but untracked files present (use "git add" to track)
```

`git status` က Repository ရဲ့ လက်ရှိအခြေအနေကို လေ့လာဖို့ အသုံးပြုရခြင်းဖြစ်ပါတယ်။ ပြန်ပေးလာတဲ့ Message ရဲ့ နောက်ဆုံးလိုင်းကို လေ့လာကြည့်ရင် Version တစ်ခုအဖြစ် မှတ်တမ်းတင်စရာဘာမှ မရှိသေးတဲ့ အကြောင်း၊ ဒါပေမယ့် Index ထဲ မထည့်ရသေးတဲ့ Untracked files တွေရှိနေတဲ့အတွက် `git add` Command ကိုသုံးပြီး Track လုပ်ပေးသင့်ကြောင်း ဖော်ပြထားတာကို တွေ့ရမှာဖြစ်ပါတယ်။

`git add index.html`, `git add script.js` စသဖြင့် ဖိုင်တစ်ခုချင်းကို Index ထဲထည့်ပေးသွားလို့ ရပါတယ်။ ဖိုင်အားလုံးကို Index ထဲ ထည့်လိုရင်တော့ `git add .` လို့ ပြောလိုက်ရင်ရပါတယ်။ Dot သင်္ကေတလေးက လက်ရှိ Directory ထဲက ဖိုင်အားလုံးဆိုတဲ့ အဓိပ္ပါယ်ဖြစ်ပါတယ်။

```
$ git add .
$ git status

On branch master
Initial commit
Changes to be committed:
(use "git rm --cached <file>..." to unstage)

      new file:   index.html
      new file:   script.js
      new file:   style.css
```

`git add .` နဲ့ ဖိုင်အားလုံး Index ထဲထည့်ပြီးနောက် `git status` ကို ပြန် Run ကြည့်တဲ့အခါ၊ Version မှတ်တမ်းတင်ဖို့အတွက် ဖိုင်အသစ် (၃) ခု ရှိနေကြောင်း ပြောလာမှာဖြစ်ပါတယ်။

Index ထဲမှာ ဖိုင်အသစ် (၃) ခုရှိနေပြီး Version မှတ်တမ်းတင်ဖို့ အသင့်ဖြစ်နေပြီမို့ `git commit` Command နဲ့ မှတ်တမ်းတင်နိုင်ပါပြီ။ အဲ့ဒီလိုမှတ်တမ်းတင်ရာမှာ မှတ်ချက် (Comment) ပေးရပါတယ်။ မှတ်ချက်ကို နှစ်သက်သလိုပေး လို့ရပေမယ့် လက်ရှိမှတ်တမ်းတင်တော့မယ့် Version မှာဘာတွေတိုးလာသလဲ၊ ဘာတွေပြောင်းသွားသလဲ၊ သတိပြုသင့်တာ တွေက ဘာတွေလဲ စသဖြင့် ရှင်းပြထားတဲ့ မှတ်ချက်ဖြစ်သင့်ပါတယ်။

```
$ git commit -m "First commit"

[master (root-commit) 11bf458] First commit
 3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html
 create mode 100644 script.js
 create mode 100644 style.css
```

နမူနာမှာ `git commit` Command ကို Run တဲ့အခါ `-m` Option ကိုသုံးပြီး မှတ်ချက် Comment ကို တစ်ခါတည်း တွဲဖက် ထည့်သွင်းပေးလိုက်ခြင်းဖြစ်ပါတယ်။ အဲ့ဒီလို တစ်ခါတည်းမပေးခဲ့ရင် Git က မှတ်ချက် Comment ပေးဖို့ သီးခြားထပ်မံ တောင်းဆိုလာမှာဖြစ်ပါတယ်။

ဆက်လက်ပြီး၊ နမူနာအနေနဲ့ `index.html` ကို ပြင်ပြီး `git status` ပြန် Run ကြည့်နိုင်ပါတယ်။

```
$ echo 'Hello, world!' > index.html
$ git status

On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

      modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
```

`index.html` ဖိုင်ကို ဖွင့်ပြင်နေမယ့်အစား `echo` Command ကိုသုံးပြီး `'Hello, world!'` ဆိုတဲ့စာကို `index.html` ထဲမှာ ထည့်သွင်းလိုက်စေခြင်းဖြစ်ပါတယ်။ `echo` မသုံးချင်ရင်လည်း နှစ်သက်ရာနည်းလမ်းနဲ့ ဖိုင်ကိုဖွင့် ပြင်နိုင်ပါတယ်။

နမူနာမှာလေ့လာကြည့်ရင် `git status` က `index.html` ပြင်ထားတာကို သိရှိကြောင်း ဖော်ပြတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။ ပြန်ပေးလာတဲ့ Message ရဲ့ နောက်ဆုံးလိုင်းကို ကြည့်ရင် `git add` နဲ့ ပြင်ထားတဲ့ဖိုင်ကို Index ထဲ ပြန်ထည့်ပေးဖို့ ပြောထားတာကိုတွေ့ရနိုင်ပါတယ်။ ဒါမှမဟုတ် `git commit -a` ကို သုံးပြီး တစ်ခါတည်း Commit လုပ်လိုက လုပ်နိုင်ကြောင်းလည်း ပြောထားပါသေးတယ်။ Index ထဲကို တစ်ကြိမ်ထည့်

ထားဖူးတဲ့ဖိုင်ကို နောက်တစ်ခါ `git add` နဲ့ ထပ်ထည့် မနေတော့ပဲ Commit လုပ်တဲ့အခါကျမှ `add` လုပ်ပေးဖို့ `-a` Option နဲ့ တွဲပြီး အသုံးပြုနိုင် ခြင်းဖြစ်ပါတယ်။

```
$ git commit -am "Updated index.html"

[master f76103d] Updated index.html
 1 file changed, 1 insertion(+)
```

`git commit` ကို Run တဲ့အခါ `-a` Option ရော `-m` Option ပါနှစ်ခုလုံးပါအောင် `-am` လို့ တွဲပြီးအသုံးပြုလိုက် ခြင်းဖြစ်ပါတယ်။ Git က ဖိုင် (၁) ခု ပြောင်းသွားကြောင်းနဲ့ Code Line တစ်လိုင်း တိုးသွားကြောင်း ပြန်လည် ဖော်ပြလာတာကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။ အခု ဆိုရင် Commit နှစ်ကြိမ်ပြုလုပ်ခဲ့ပြီဖြစ်လို့ ကျွန်တော်တို့မှာ Version နှစ်ခုရရှိသွားပြီဖြစ်ပါတယ်။ `git log` Command နဲ့ Version မှတ်တမ်းကို ပြန်လည်လေ့လာနိုင်ပါ တယ်။

```
$ git log

commit f76103d35ad7b2a2641bf16d1baf6de27a5981cf
Author: Ei Maung <eimg@fairwayweb.com>
Date:   Tue Mar 17 15:42:02 2015 +0630

     Updated index.html

commit 11bf458a3a3cc45e65e5c20e16c59d92d66038e7
Author: Ei Maung <eimg@fairwayweb.com>
Date:   Tue Mar 17 12:48:56 2015 +0630

      First commit
```

`git log` Command က မှတ်တမ်းတင်ခဲ့တဲ့ Version တွေကို စာရင်းနဲ့ဖော်ပြလာခြင်းဖြစ်ပါတယ်။ Version တစ်ခု စီမှာ စာလုံး (၄၀) ပါဝင်တဲ့ Commit Hash ကိုယ်စီရှိကြပါတယ်။ Commit Hash ဆိုတာ အဲ့ဒီ Version ရဲ့ ID ပဲဖြစ်ပါ တယ်။ နောင်အဲ့ဒီ Version ကို ပြန်လိုချင်ရင် Commit Hash ကို အသုံးပြုပြီး ပြန်လည်ရယူနိုင်ပါ တယ်။ Log ထဲမှာ Commit Hash နဲ့အတူ Commit ပြုလုပ်ခဲ့သူ၊ Commit ပြုလုပ်ခဲ့တဲ့အချိန်နဲ့ Commit Comment တို့ကို စာရင်းထဲမှာ ဖော်ပြနေတာကို တွေ့ရမှာဖြစ်ပါတယ်။ `git log` ကို သပ်ရပ်သွားအောင် `--pretty` Option နဲ့ တွဲပြီး သုံးနိုင်ပါ တယ်။

```
$ git log --pretty=oneline

f76103d35ad7b2a2641bf16d1baf6de27a5981cf Updated index.html
11bf458a3a3cc45e65e5c20e16c59d92d66038e7 First commit
```

`--pretty=oneline` လို့ပြောလိုက်တဲ့အတွက် Log ကိုဖော်ပြရာမှာ စောစောကလို ရှုပ်ရှက်ခတ်နေအောင် မ ပြတော့ ပဲ Commit တစ်ခု တစ်ကြောင်းနှုန်းနဲ့ ဖော်ပြမှာဖြစ်ပါတယ်။ `online` အစား စိတ်ကြိုက် Format နဲ့

အစားထိုးသုံးနိုင်ပါတယ်။

```
$ git log --pretty=format:"%h - %an, %ar : %s"

f76103d - Ei Maung, 17 minutes ago : Updated index.html
11bf458 - Ei Maung, 3 hours ago : First commit
```

`%h` Placeholder က Short Commit Hash ဆိုတဲ့အဓိပ္ပါယ်ပါ။ Commit Hash ရဲ့ စာလုံး (၄၀) လုံးကို မဖော်ပြတော့ပဲ အတိုကောက်အနေနဲ့ ရှေ့ဆုံး (၇) လုံးကိုသာဖော်ပြတော့မှာ ဖြစ်ပါတယ်။ `%an` က Author Name, `%ar` က Author Date Relative နဲ့ `%s` ကတော့ Commit Comment Subject ဆိုတဲ့ အဓိပ္ပါယ်ပဲဖြစ်ပါတယ်။ Version တစ်ခုနဲ့တစ်ခု ကူးပြောင်းလိုတဲ့အခါ Short Commit Hash ကို သုံးပြီးတော့လည်း ကူးပြောင်းနိုင်ပါတယ်။ ဥပမာ – ပထမဆုံး Version ကိုပြန်သွားချင်ရင်၊ သူရဲ့ Short Commit Hash က `11bg458` ဖြစ်တဲ့အတွက် အခုလို ပြန်သွားနိုင်ပါတယ်။

```
$ git checkout 11bf458

Note: checking out '11bf458'.
You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by performing another checkout.
If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -b with the checkout command again. Example:

    git checkout -b new_branch_name

HEAD is now at 11bf458... First commit
```

နမူနာကိုလေ့လာကြည့်ရင် Git က လက်ရှိမှာ Commit Hash `11bf485` ကို ပြောင်းလဲအသုံးပြုနေကြောင်း၊ ဒီ Commit ကို Branch အဖြစ် ခွဲထုတ်လိုတယ်ဆိုရင်လည်း `git checkout -b` နဲ့ ခွဲထုတ်နိုင်ကြောင်း အသိပေးလာတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။

တစ်ချို့အရေးပါတဲ့ Commit တွေကို နောင်ကူးပြောင်းအသုံးပြုရ လွယ်ကူစေဖို့အတွက် Tag လုပ်ထားနိုင်ပါတယ်။

```
$ git tag v1
```

လက်ရှိရောက်ရှိနေတဲ့ Commit ဖြစ်တဲ့ `11bf485` ကို `v1` လို့ Tag လုပ်ပြီး အမည်ပေးလိုက်ခြင်းဖြစ်ပါတယ်။

နောက်ဆုံး Version ကို ပြန်သွားလိုရင် `git checkout master` ကို Run ပြီး ပြန်သွားနိုင်ပါတယ်။ ဒါကြောင့် `git checkout master` နဲ့ နောက်ဆုံး Version ကိုပြန်သွားပြီး `git log` Run ကြည့်ရင် အခုလို တွေ့ရမှာဖြစ်ပါတယ်။

```
$ git checkout master
$ git log --pretty=format:'%h %s %d'

f76103d Updated index.html  (HEAD, master)
11bf458 First commit  (tag: v1)
```

`%d` Placeholder က Reference Name ဆိုတဲ့အဓိပ္ပါယ်ပါ။ နမူနာမှာကြည့်ရင် Commit `11bf458` ကို `v1` လို့ Tag လုပ်ထားကြောင်းတွေ့နိုင်ပါတယ်။ ဒါကြောင့်အဲ့ဒီ Commit ကိုသွားချင်ရင် အခုလိုအလွယ်တစ်ကူသွားလို့ ရသွားပါတယ်။

```
$ git checkout v1

Note: checking out 'v1'.
You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by performing another checkout.
If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -b with the checkout command again. Example:

     git checkout -b new_branch_name

HEAD is now at 11bf458... First commit
```

ဒီနည်းနဲ့ အရေးပါတဲ့ Commit တွေကို Tag လုပ်ပြီး မှတ်သားထားနိုင်ပါတယ်။ ပေးထားတဲ့ Tag ကို ပြန်ဖျက်ချင်ရင် တော့ `git tag` Command ကို `-d` Option နဲ့ တွဲသုံးနိုင်ပါတယ်။

```
$ git checkout master
$ git tag -d v1
$ git log --pretty=format:'%h %s %d'

f76103d Updated index.html  (HEAD, master)
11bf458 First commit
```

`-d` Option က မှတ်တမ်းတင်ထားတဲ့ Commit ကိုဖျက်တာမဟုတ်ပါဘူး။ Commit ကို Reference လုပ်ထားတဲ့ Tag ကို ဖျက်ပေးခြင်းဖြစ်ပါတယ်။

Working with Branch

VCS တွေမှာ ပါဝင်လေ့ရှိတဲ့ Branch လုပ်ဆောင်ချက်ရဲ့အသုံးဝင်ပုံကို အထက်မှာဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ Git မှာ Branch တွေစီမံဖို့အတွက် `git branch` Command ကို သုံးပါတယ်။

```
$ git branch

* master
```

လောလောဆယ် Branch တွေမရှိသေးတဲ့အတွက် `git branch` ကို Run ကြည့်တဲ့အခါ `master` တစ်ခုပဲရှိနေကြောင်း ဖော်ပြခြင်း ဖြစ်ပါတယ်။ လက်ရှိရောက်ရှိနေတဲ့ Version ကို Branch အသစ်တစ်ခုအဖြစ် အခုလို ခွဲထုတ်နိုင်ပါတယ်။

```
$ git branch dev
$ git branch

  dev
* master
```

`git branch dev` လို့ ပြောလိုက်တဲ့အချိန်မှာ `dev` ဆိုတဲ့ Branch မရှိသေးတဲ့အတွက် Git က `dev` Branch ကို အလိုအလျောက် တည်ဆောက်ပေးသွားပါတယ်။ ဒါကြောင့် နောက်တစ်ကြိမ် `git branch` Run ကြည့်တဲ့အခါမှာ `dev` နဲ့ `master` ဆိုပြီး Branch နှစ်ခု ဖြစ်သွားတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။ `master` ရဲ့ ရှေ့က Asterisk (*) က လက်ရှိ `master` Branch နဲ့ အလုပ်လုပ်နေကြောင်း ဖော်ပြခြင်းဖြစ်ပါတယ်။

အသစ်တည်ဆောက်ထားတဲ့ `dev` Branch ကို `git checkout` နဲ့ ကူးပြောင်းနိုင်ပါတယ်။

```
$ git checkout dev

Switched to branch 'dev'

$ git branch

* dev
  master
```

ဒီနည်းနဲ့ Branch တွေ လိုသလိုတည်ဆောက်ပြီး အပြန်အလှန် ကူးပြောင်းအလုပ်လုပ်နိုင်ပါတယ်။ Branch အသစ်တစ်ခု ဆောက်ပြီး ချက်ချင်းကူးပြောင်းသွားစေတဲ့ Short Cut အနေနဲ့ အခုလိုလည်း အသုံးပြုနိုင်ပါတယ်။

```
$ git checkout -b beta

Switched to a new branch 'beta'

$ git branch

* beta
  dev
  master
```

ဒီနည်းနဲ့ `beta` ဆိုတဲ့ Branch ကို တည်ဆောက်လိုက်ယုံမက တစ်ခါတည်း ကူးပြောင်းပေးသွားပါတယ်။ တည်ဆောက်ထားတဲ့ Branch တစ်ခုကို ပြန်လည်ပယ်ဖျက်စေလိုရင် `git branch` ကို `-d` Option နဲ့ တွဲသုံးနိုင်ပါတယ်။

```
$ git branch -d dev

Deleted branch dev (was f76103d).
```

Branch တစ်ခုကို ကူးပြောင်းပြီးနောက် ပြုလုပ်တဲ့ Commit တွေဟာ လက်ရှိရောက်ရှိနေတဲ့ Branch ကိုသာ သက် ရောက်တော့မှာ ဖြစ်ပါတယ်။

```
$ git checkout beta
$ echo 'body { margin: 0 }' > style.css
$ git commit -am "Updated style.css"

[beta d731a7f] Updated style.css
 1 file changed, 1 insertion(+)
```

```
$ git log --pretty=oneline

d731a7fcdd7dd5a478e2532e1d254a0c69184176 Updated style.css
f76103d35ad7b2a2641bf16d1baf6de27a5981cf Updated index.html
11bf458a3a3cc45e65e5c20e16c59d92d66038e7 First commit
```

`beta` Branch မှာ `style.css` ဖိုင်ကိုပြင်ပြီး Commit လုပ်လိုက်ပါတယ်။ ပြီးတော့ Log ပြန်ကြည့်တဲ့အခါ Version သုံးခု ဖြစ်သွားတာကို တွေ့ရမှာဖြစ်ပါတယ်။ `master` Branch ကို ပြန်သွားပြီး Log ပြန်ကြည့်တဲ့ အခါမှာတော့ Version နှစ်ခုသာ ရှိတာကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။

```
$ git checkout master
$ git log -pretty=oneline

f76103d35ad7b2a2641bf16d1baf6de27a5981cf Updated index.html
11bf458a3a3cc45e65e5c20e16c59d92d66038e7 First commit
```

ဒီနည်းနဲ့ Source Code Repository တစ်ခုတည်းကို Branch တွေအမျိုးမျိုးခွဲပြီး စီမံဆောင်ရွက်နိုင်မှာပဲဖြစ်ပါ တယ်။ Branch တစ်ခုပေါ်မှာ သိမ်းဆည်းထားတဲ့ Version တွေကိုနောက် Branch တစ်ခုနဲ့ ပေါင်းစပ်လိုရင်တော့ `git merge` ကိုသုံးပြီး ပေါင်းစပ်နိုင်ပါတယ်။

```
$ git checkout master
Switched to branch 'master'

$ git merge beta

Updating f76103d..d731a7f / Fast-forward
 style.css | 1 +
 1 file changed, 1 insertion(+)
```

`master` Branch ပေါ်မှာ `git merge beta` ကို Run လိုက်တဲ့အတွက် `beta` Branch ပေါ်က Version တွေဟာ `master` Branch နဲ့ ပေါင်းစပ်သွားမှာပဲဖြစ်ပါတယ်။ ဒါကြောင့် `master` Branch မှာ Log ကြည့်မယ်ဆိုရင် Version သုံးခုဖြစ်သွားတာကို တွေ့ရမှာပဲဖြစ်ပါတယ်။

```
$ git log -pretty=oneline

d731a7fcdd7dd5a478e2532e1d254a0c69184176 Updated style.css
f76103d35ad7b2a2641bf16d1baf6de27a5981cf Updated index.html
11bf458a3a3cc45e65e5c20e16c59d92d66038e7 First commit
```

Merge Conflicts

Branch တွေ Merge လုပ်ရာမှာ တစ်ခါတစ်ရံ Conflict တွေရှိတဲ့ပါတယ်။ Merge လုပ်လိုတဲ့ Branch နှစ်ခုလုံးက တူညီ တဲ့ဖိုင်တစ်ခုကို ပြင်ဆင်ထားမိတဲ့အခါမှာ ဖြစ်လေ့ရှိပါတယ်။ စမ်းသပ်ကြည့်နိုင်ဖို့အတွက် `master` Branch ပေါ်က `script.js` ကို အခုလို ပြင်ဆင်ပြီး Commit လုပ်လိုက်ပါမယ်။

```
$ git checkout master

Switched to branch 'master'

$ echo 'var app = {}' > script.js
$ git commit -am "Updated script.js in master"

[master 281e6f3] Updated script.js in master
 1 file changed, 1 insertion(+)
```

ပြီးတဲ့အခါ `beta` Branch ကိုကူးပြီး `script.js` ကိုပဲ ပြင်ပြီး Commit လုပ်လိုက်ပါဦးမယ်။

```
$ git checkout beta

Switched to branch 'beta'

$ echo 'var app = []' > script.js
$ git commit -am "Updated script.js in beta"

[beta 9147870] Updated script.js in beta
 1 file changed, 1 insertion(+)
```

`master` Branch ကို ပြန်သွားပြီး `beta` နဲ့ Merge လုပ်ခိုင်းတဲ့အခါ Conflict ပြဿနာကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။

```
$ git checkout master

Switched to branch 'master'

$ git merge beta

Auto-merging script.js
CONFLICT (content): Merge conflict in script.js
Automatic merge failed; fix conflicts and then commit the result.
```

`script.js` ကို `master` ရော `beta` ကပါ ပြင်ထားတဲ့အတွက် ဘာကိုအတည်ယူရမလဲ Git က မဆုံးဖြတ်နိုင်လို့ ဒီ ပြဿနာတက်ရ ခြင်းဖြစ်ပါတယ်။ `script.js` ကို ဖွင့်ကြည့်ရင် အခုလို တွေ့ရမှာပါ။

```
<<<<<<< HEAD
var app = {}
=======
var app = []
>>>>>>> beta
```

Conflict ညိနေတဲ့နေရာကို နှစ်ပိုင်းခွဲပြီး `master script.js` ထဲက Code နဲ့ `beta script.js` ထဲက Code တို့ကို ရောပေးထားပါတယ်။ ကိုယ်အတည်ပြုလိုတဲ့ အနေအထားဖြစ်အောင် Code ဖိုင်ကို စိတ်တိုင်းကျ ပြင်ပါ။ ပြီးရင် Commit လုပ်ပေးခြင်းအားဖြင့် Conflict ကို Resolve လုပ်ပြီး ဖြစ်သွားပါလိမ့်မယ်။

```
$ git commit -am "Merge with conflict resolve"

[master 08582cf] Merge with conflict resolve
```

ခုနေ Log ခေါ်ကြည့်ရင် အခုလိုတွေ့ရမှာပဲ ဖြစ်ပါတယ်။

```
$ git log --pretty=oneline

08582cf9e218e12b74ff24711692ba2efea936dd Merge with conflict resolve
91478702aebe0fe49d667316b11c6fe4bc50062f Updated script.js in beta
281e6f362f37beb9384bf0ebf15605f81f21582e Updated script.js in master
d731a7fcdd7dd5a478e2532e1d254a0c69184176 Updated style.css
f76103d35ad7b2a2641bf16d1baf6de27a5981cf Updated index.html
11bf458a3a3cc45e65e5c20e16c59d92d66038e7 First commit
```

`master` မှာ `script.js` ကိုပြင်ပြီး Commit လုပ်ခဲ့တဲ့ Version, `beta` မှာ `script.js` ကိုပြင်ပြီး Commit လုပ်ခဲ့တဲ့ Version နဲ့ Conflict ကို Resolve လုပ်ပြီး Commit လုပ်ထားတဲ့ Version အားလုံးကို မှတ်တမ်းတင်ပြီး ဖြစ် နေတာကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။

Clone Repository

Repository တစ်ခုကို မူပွားယူဖို့အတွက် `git clone` ကို သုံးနိုင်ပါတယ်။ `git clone` နဲ့ ကိုယ့်စက်ထဲက Repository တစ်ခုကို မူပွားယူနိုင်သလို၊ Network သို့မဟုတ် အင်တာနက်ပေါ်က Repository ကိုလည်း မူပွားယူနိုင်ပါတယ်။ `git clone` နောက်ကနေ မူပွားယူလိုတဲ့ Repository ရဲ့ တည်နေရာလိပ်စာကို ညွှန်းပေးဖို့ပဲ လိုပါတယ်။ ဥပမာ –

```
$ git clone path/to/repository new_repo
$ git clone smb://192.168.1.100/repository new_repo
$ git clone ssh://server/path/to/repository new_repo
$ git clone https://github.com/user/repository new_repo
```

`git clone` နဲ့အတူ မူရင်း Repository ရဲ့လိပ်စာပဲပေးမယ်ဆိုရင် မူပွားရဲ့အမည်ကိုလည်း မူရင်း Repository အမည် အတိုင်းထားပေးသွားမှာဖြစ်ပါတယ်။ ဒါမှမဟုတ် နမူနာမှာပြထားသလို `git clone` နောက်ကနေ မူရင်း Repository လိပ်စာနဲ့အတူ မူပွားအတွက်ပေးလိုတဲ့ အမည်ကိုလည်း တွဲဖက်ပေးလိုရင် ပေးနိုင်ပါတယ်။

```
$ cd ~
$ git clone /opt/lampp/htdocs/project

Cloning into 'project'...
done.

$ ls project

index.html  script.js  style.css
```

နမူနာအနေနဲ့ `git clone` နဲ့ ကျွန်တော်တို့ စောစောကစမ်းသပ်ထားတဲ့ Repository ကို Home Directory ထဲမှာ မူပွားယူထားပါတယ်။ မူပွားရရှိလာတဲ့ project Directory ထဲက ဖိုင်စာရင်းကိုကြည့်လိုက်တဲ့အခါ ကျွန်တော်တို့ တည် ဆောက်ထားခဲ့တဲ့ `index.html`, `script.js` နဲ့ `style.css` တို့ ပါဝင်လာတာကို တွေ့ရမှာဖြစ်ပါတယ်။ ဒါ့အပြင်၊ မူပွားရရှိလာတဲ့ Repository ထဲမှာ `git log` Run ကြည့်လိုက်ရင် ကျွန်တော်တို့ မှတ်တမ်းတင်ထားခဲ့တဲ့ Version အားလုံးလည်း ပါဝင်လာတာကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။

```
$ cd project
$ git log -pretty=oneline

08582cf9e218e12b74ff24711692ba2efea936dd Merge with conflict resolve
91478702aebe0fe49d667316b11c6fe4bc50062f Updated script.js in beta
281e6f362f37beb9384bf0ebf15605f81f21582e Updated script.js in master
d731a7fcdd7dd5a478e2532e1d254a0c69184176 Updated style.css
f76103d35ad7b2a2641bf16d1baf6de27a5981cf Updated index.html
11bf458a3a3cc45e65e5c20e16c59d92d66038e7 First commit
```

`git branch` Run ကြည့်ရင်တော့ `master` Branch တစ်ခုသာပါဝင်လာတာကို တွေ့ရပါလိမ့်မယ်။ ကျန် Branch တွေက မပါလာတာတော့မဟုတ်ပါဘူး၊ Reference လုပ်ယုံသာလုပ်ထားတာမို့ အခြား Branch တွေကို လိုချင်ရင် `git checkout` နဲ့ ထပ်မံရယူဖို့ လိုပါတယ်။

```
$ git branch

* master

$ git branch -a

* master
  remotes/origin/HEAD -> origin/master
  remotes/origin/beta
  remotes/origin/master

$ git checkout beta

Branch beta set up to track remote branch beta from origin.
Switched to a new branch 'beta'

$ git branch

* beta
  master
```

နမူနာမှာ လေ့လာကြည့်ပါ။ ပထမဆုံး `git branch` Run ကြည့်တော့ `master` တစ်ခုကိုသာ တွေ့ရပါတယ်။ `git branch` ကို `-a` Option နဲ့ ထပ် Run ကြည့်တော့မှ မူလ Repository ရဲ့ Branch တွေကို ညွှန်းထားတဲ့ အညွှန်းတွေ ရှိနေတာ တွေ့ရမှာဖြစ်ပါတယ်။ ဒါကြောင့် `git checkout` နဲ့ `beta` Branch ကို ရယူပြီး `git branch` ပြန် Run ကြည့်တဲ့အခါ မှာတော့ `beta` Branch လည်း Branch စာရင်းထဲမှာ ပါဝင်လာတာကို တွေ့ရမှာပဲဖြစ်ပါတယ်။ ဒီလို မူရင်း Repository ရဲ့ Branch တွေကို ညွှန်းထားတဲ့အတွက်ကြောင့် မူရင်း Repository မှာ အပြောင်းအလဲရှိရင် ရှိတဲ့ အပြောင်းအလဲတွေကို `git pull` Command နဲ့ အလွယ်တစ်ကူ ရယူနိုင်မှာဖြစ်ပါတယ်။

```
$ cd /opt/lampp/htdocs/project
$ touch app.js
$ git add app.js
$ git commit -m "Added app.js"

[master 81f48dc] Added app.js
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 app.js

$ git log --pretty=oneline -3

81f48dc2faddcd42b0c479f53db522b64dd146ec Added app.js
08582cf9e218e12b74ff24711692ba2efea936dd Merge with conflict resolve
91478702aebe0fe49d667316b11c6fe4bc50062f Updated script.js in beta
```

နမူနာအနေနဲ့ မူရင်း Repository ထဲမှာ ဖိုင်တစ်ခုထပ်တိုးပြီး Commit လုပ်ထားပါတယ်။ Log မှာပြန်ကြည့်တဲ့အခါ ထပ်တိုးလာတဲ့ Commit ကို တွေ့ရမှာဖြစ်ပါတယ်။ `git log` မှာတွဲသုံးထားတဲ့ `-3` Option က၊ နောက်ဆုံး Commit (၃) ခုပဲကြည့်မယ်လို့ ပြောလိုက်တဲ့သဘောပါ။ `-3` အစားအခြားတန်ဖိုးတွေနဲ့ အစားထိုးအသုံးပြုနိုင်ပါတယ်။

မူပွားယူထားတဲ့ Clone Repository ထဲမှာ Log ခေါ်ကြည့်ရင်၊ မူရင်း Repository မှာပြုလုပ်ထားတဲ့ နောက်ဆုံး Commit မပါပဲ မူလ Clone လုပ်ယူစဉ်ကပါဝင်လာတဲ့ Commit တွေသာ ရှိနေတာကို တွေ့ရမှာဖြစ်ပါတယ်။

```
$ cd ~/project
$ git log --pretty=oneline -3

08582cf9e218e12b74ff24711692ba2efea936dd Merge with conflict resolve
91478702aebe0fe49d667316b11c6fe4bc50062f Updated script.js in beta
281e6f362f37beb9384bf0ebf15605f81f21582e Updated script.js in master
```

ဒါကြောင့် `git pull` နဲ့ မူရင်း Repository ထံက၊ နောက်ဆုံး Update ကို ရယူကြည့်ပါမယ်။

```
$ git pull origin master

remote: Counting objects: 1, done.
remote: Total 1 (delta 0), reused 0 (delta 0)
Unpacking objects: 100% (1/1), done.
From /opt/lampp/htdocs/project
 * branch            master     -> FETCH_HEAD
   08582cf..81f48dc  master     -> origin/master
Updating 08582cf..81f48dc
Fast-forward
 app.js | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 app.js
```

`git pull` နဲ့ အတူ `origin master` ကို တွဲသုံးထားပါတယ်။ `origin` အကြောင့် ခဏနေမှပြောပါမယ်။ `master` ကတော့ Branch အမည်ဖြစ်တဲ့အတွက် `master` Branch ပေါ်က Update တွေကို ရယူမယ်လို့ ဆိုလိုက် ခြင်းဖြစ်ပါတယ်။ ဒါကြောင့် ရယူလိုတာက အခြား Branch ဆိုရင်လည်း `master` အစား၊ ရယူလိုတဲ့ Branch အမည်နဲ့ အစားထိုးအသုံးပြုနိုင်ပါတယ်။

အမှန်တော့၊ မူရင်း Repository က နောက်ဆုံး Update တွေ ရယူဖို့အတွက် `git fetch` ကို သုံးရတာပါ။ ဒါပေမယ့် `git fetch` က ရယူယုံသက်သက်ပဲ အလုပ်လုပ်ပါတယ်၊ ရရှိလာတဲ့ Update ကို လက်ရှိ Clone Repository နဲ့ပေါင်း စပ်ဖို့အတွက် `git merge` ကို ထပ်သုံးပေးရပါတယ်။ `git pull` ကတော့ Update တွေကို ရယူပေးယုံမက တစ်ခါတည်းလည်း ပေါင်းစပ်ပေးပါတယ်။ ဒါကြောင့် `git pull` ဆိုတာ `git fetch` နဲ့ `git merge` ကို ပေါင်း စပ်ထားတဲ့သဘောလို့လဲ ဆိုနိုင်ပါတယ်။

နောက်ဆုံး Update ရယူပြီး Log ပြန်ကြည့်တဲ့အခါမှာတော့ Clone Repository ထဲမှာလည်း မူရင်း Repository မှာပြုလုပ်ခဲ့တဲ့ နောက်ဆုံး Commit ပါဝင်လာတာကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။

```
$ git log --pretty=oneline -3

81f48dc2faddcd42b0c479f53db522b64dd146ec Added app.js
08582cf9e218e12b74ff24711692ba2efea936dd Merge with conflict resolve
91478702aebe0fe49d667316b11c6fe4bc50062f Updated script.js in beta
```

`git pull` နဲ့အတူ တွဲသုံးခဲ့တဲ့ `origin` အကြောင်းအနည်းငယ်ပြောဖို့ လိုပါမယ်။ `origin` ဆိုတာ `git` Command မဟုတ်ပါဘူး။ အညွှန်းအမည်တစ်ခုသာဖြစ်ပါတယ်။ မူရင်း Repository တည်ရှိရာ တည်နေရာကို ညွှန်းထား တဲ့ အညွှန်းတစ်ခု ဖြစ်ပါတယ်။ `git clone` နဲ့ Repository ကို မူးပွားယူလိုက်စဉ်မှာ၊ Git က Clone Repository အတွက် `origin` ဆိုတဲ့ အမည်နဲ့ မူရင်း Repository တည်ရှိရာအညွှန်းတစ်ခုကို အလိုအလျောက် ထည့်သွင်းပေးသွား ခြင်းဖြစ်ပါတယ်။ `git remote` Command ကို Run ကြည့်ရင် မူရင်း Repository အညွှန်း စာရင်းကို တွေ့ရမှာဖြစ် ပါတယ်။

```
$ git remote

origin
```

လောလောဆယ် `origin` အမည်နဲ့ အညွှန်းတစ်ခုသာ ရှိနေပါတယ်။ `origin` က ဘယ်ကိုညွှန်းထားတာလဲ သိချင်ရင် တော့ `git remote show origin` လို့ Run ကြည့်နိုင်ပါတယ်။

```
$ git remote show origin

* remote origin
  Fetch URL: /opt/lampp/htdocs/project
  Push  URL: /opt/lampp/htdocs/project
```

နမူနာအရ `origin` က `/opt/lampp/htdocs/project` Repository ကို ညွှန်းထားတာတွေ့ရမှာပါ။

Git ရဲ့ ထူးခြားချက်က Repository တစ်ခုမှာ တစ်ခုထက်ပိုတဲ့ Remote အညွှန်းတွေ ရှိနိုင်ခြင်းဖြစ်ပါတယ်။ ဒီ အချက်က Open Source Project တွေမှာ အတော်လေးအသုံးဝင်ပါတယ်။ ဥပမာ – Open Source Project တစ် ခုကို Team A နဲ့ Team B လို့ ခေါ်တဲ့ အဖွဲ့နှစ်ဖွဲ့က သီးခြားစီ Maintain လုပ်နေတယ်ဆိုပါစို့။ ကျွန်တော်တို့က Team A ရဲ့ Repository ကို ရော Team B ရဲ့ Repository ကိုပါ Git Remote အညွှန်းများအဖြစ် ညွှန်းထားမယ်ဆို ရင်၊ Team A ရဲ့ Repository မှာရှိနေတဲ့ Update တွေရော Team B ရဲ့ Repository မှာရှိနေတဲ့ Update တွေကိုပါ တစ်နေရာထဲ ကနေ ရယူနေလို့ရနိုင်မှာပဲဖြစ်ပါတယ်။

Git Remote အညွှန်းတစ်ခု ထပ်ထည့်ချင်ရင် git remote add နဲ့ ထည့်သွင်းနိုင်ပါတယ်။

```
$ git remote add teamb /var/www/project
$ git remote

teamb
origin
```

`git remote add` ကိုသုံးပြီး `teamb` ဆိုတဲ့ Remote အည္ညွန်းတစ်ခု ထပ်ထည့်လိုက်ပါတယ်။ ဒါကြောင့် `git remote` ပြန် Run ကြည့်တဲ့အခါ Remote အည္ညွန်း နှစ်ခုဖြစ်သွားတာကို တွေ့ရမှာပဲဖြစ်ပါတယ်။ `teamb` ထံက Update ကို လိုချင်ရင် –

```
$ git pull teamb master
```

– လို့ ရယူနိုင်ပြီး `origin` ထံက Update ကိုလိုချင်ရင်တော့၊ စောစောကလိုပဲ –

```
$ git pull origin master
```

– လို့ ရယူနိုင်မှာပဲဖြစ်ပါတယ်။ Remote အည္ညွန်းတစ်ခုကို ပယ်ဖျက်လိုရင် `git remote remove` ကို သုံးနိုင်ပါတယ်။

```
$ git remote remove teamb
$ git remote

origin
```

`git remote remove` နဲ့ `teamb` အည္ညွန်းကို ပယ်ဖျက်ပြီး `git remote` ပြန် Run ကြည့်တဲ့အခါ `origin` အည္ညွန်းတစ်ခုပဲကျန်တော့တာကို တွေ့ရမှာပဲဖြစ်ပါတယ်။

Centralize Repository

Git Repository တစ်ခုကို Centralize Repository တစ်ခုအနေနဲ့ တစ်ဦးထက်ပိုတဲ့ Developer တွေက ဝိုင်းဝန်းအသုံး ပြုနိုင်ပါတယ်။ အဲ့ဒီလိုအသုံးပြုနိုင်ဖို့အတွက် Working Directory မပါပဲ Version တွေ၊ Branch တွေနဲ့ အခြားလိုအပ်တဲ့ အည္ညွန်းတွေ သာပါဝင်တဲ့ Bare Repository ကို အသုံးပြုရပါတယ်။ နည်းလမ်း (၂) မျိုးနဲ့ Bare Repository တစ်ခုကို ဖန်တီးယူနိုင်ပါတယ်။ ပထမနည်းလမ်းကတော့ Directory အလွတ်တစ်ခု ထဲမှာ `git init` ကို `--bare` Option နဲ့တွဲသုံးပြီး Bare Repository အလွတ်တစ်ခုအဖြစ် ကြေငြာနိုင်ပါတယ်။

```
$ mkdir rockstar && cd rockstar
$ git init --bare

Initialized empty Git repository in /opt/lampp/htdocs/rockstar
```

`rockstar` အမည်နဲ့ Directory အလွတ်တစ်ခုတည်ဆောက်ပြီး အဲ့ဒီ Directory ထဲမှာ `git init --bare` ကို Run ခြင်းအားဖြင့် `rockstar` Directory ဟာ Git Bare Repository တစ်ခုဖြစ်သွားပါတယ်။ အဲ့ဒီ Repository ကို ဖွင့်ကြည့်လိုက်ရင် ရိုးရိုး Git Repository မှာ ပါဝင်လေ့ရှိတဲ့ `.git` Directory အတွင်းက `objects`, `hooks`, `branches`, `config`, `HEAD` စတဲ့ ဖိုင်နဲ့ Directory တွေကို တွေ့ရမှာပဲဖြစ်ပါတယ်။ တနည်းအားဖြင့် `rockstar` ဟာ Source Code ဖိုင်တွေသိမ်းဆည်းဖို့ မဟုတ်ပဲ၊ Version မှတ်တမ်းတွေကိုသာ သိမ်းဆည်းဖို့ ရည်ရွယ် တည်ဆောက်လိုက်တဲ့ Repository တစ်ခု ဖြစ်သွားခြင်း ဖြစ်ပါတယ်။ ဒါကြောင့် Source Code ဖိုင်တွေကို သူ့ထဲမှာ တိုက်ရိုက် ရေးသားတည်ဆောက်ခြင်း မလုပ်ရတော့ပါဘူး။ သူ့ကို Version မှတ်တမ်း သိမ်းဆည်းပေးတဲ့ Centralize Repository အဖြစ်သာ မှတ်ယူရတော့မှာပါ။

Bare Repository တစ်ခုတည်ဆောက်နည်း နောက်တစ်နည်းကတော့ Clone လုပ်ယူစဉ်မှာ `--bare` Option ကို အသုံးပြုခြင်းပဲ ဖြစ်ပါတယ်။

```
$ cd ~
$ git clone --bare /opt/lampp/htdocs/project

Cloning into bare repository 'project.git'...
done.
```

ဒီနည်းနဲ့ မူလ Source Code တွေနဲ့ Version မှတ်တမ်းတွေ ရှိနေပြီးသား Repository တစ်ခုကို Bare Repository တစ်ခုအဖြစ် မူပွားယူနိုင်ခြင်းဖြစ်ပါတယ်။ နမူနာမှာလေ့လာကြည့်ရင် Git က မူလ Repository ဖြစ်တဲ့ project ကို `project.git` အမည် ရှိတဲ့ Bare Repository အဖြစ် Clone လုပ်ပေးသွားခြင်းဖြစ်တာကို တွေ့ရနိုင်ပါတယ်။ Git Repository Directory တွေကို အမည်ပေးတဲ့အခါ နောက်ဆုံးက `.get` ထည့်ပြီး ပေးကြလေ့ရှိပါတယ်။ မဖြစ်မနေပေးဖို့ လိုတာမျိုးမဟုတ်ပေမယ့် အမည်ကို ကြည့်လိုက်ယုံနဲ့ Git Repository ဖြစ်ကြောင်း သိသာစေနိုင်လို့ Git Repository အဖြစ်သုံးမယ့် Directory တွေကို အမည်ပေးတဲ့အခါ အသုံးပြုသင့်တဲ့ နည်းလမ်းတစ်ခုပဲဖြစ်ပါတယ်။

တည်ဆောက်ထားတဲ့ Bare Repository ကို Centralize Repository အဖြစ် Project မှာပါဝင်သူအားလုံး Access လုပ်နိုင်မယ့် တစ်နေရာမှာ ထားဖို့လိုပါတယ်။ Developer တစ်ဦးရဲ့စက်ထဲမှာထားပြီး Network Share အဖြစ် Share ပေးလိုက်ရင်လည်း ရပါတယ်။ ဒါမှမဟုတ် ပိုပြီးစနစ်ကျစေချင်ရင် သီးခြား Server ကွန်ပျူတာတစ်လုံးမှာထားပြီး၊ SSH သို့မဟုတ် HTTP တို့ကနေ တစ်ဆင့် ရယူနိုင်အောင်လည်း ထားနိုင်ပါတယ်။ ဒီနေရာမှာတော့ SSH Server နဲ့ HTTP Server များ Setup လုပ်ပုံတို့ကို ထည့်သွင်းမဖော်ပြနိုင်တော့ပါဘူး။ သဘောသဘာဝပိုင်းကိုသာ ဆက်လက်ဖော်ပြပေးသွား ပါမယ်။

Project မှာပါဝင်သူတစ်ဦးအနေနဲ့ Code တွေ စတင်ရေးသားနိုင်ဖို့အတွက် ပထမဦးဆုံးအနေနဲ့ Centralize Repository အဖြစ်သတ်မှတ်ထားတဲ့ Bare Repository ကို Clone လုပ်ယူဖို့လိုပါတယ်။ Clone လုပ်ပုံလုပ်နည်းက ရိုးရိုး Repository ကို Clone လုပ်ပုံနဲ့အတူတူပဲ ဖြစ်ပါတယ်။

```
$ git clone /opt/lampp/htdocs/rockstar

Cloning into 'rockstar'...
warning: You appear to have cloned an empty repository.
done.
```

ဘာမှမရှိသေးတဲ့ Repository အလွတ်ဖြစ်နေတယ်ဆိုရင် Git က နမူနာမှာပြထားသလို Waring တစ်ခုပေးနိုင်ပါတယ်။ Clone ရယူထားတဲ့ Local Repository ထဲမှာ Source Code ဖိုင်နဲ့ Directory တွေတည်ဆောက်ခြင်း၊ Code များရေးသားခြင်း နဲ့ Version မှတ်တမ်းများ Commit လုပ်ခြင်းတို့ကို လုပ်ရိုးလုပ်စဉ်အတိုင်း ဆက်လက်ဆောင်ရွက် နိုင်ပါတယ်။

Clone မလုပ်ပဲ Git Repository အလွတ်တစ်ခုတည်ဆောက်ပြီး `git remote add` နဲ့ Centralize Repository ကို ညွှန်းပေးလို့လည်းရပါတယ်။ ဥပမာ –

```
$ git init

Initialized empty Git repository in /home/eimg/rockstar/.git/

$ git remote add origin /opt/lampp/htdocs/rockstar
```

ကိုယ်ရဲ့ Local Repository ထဲမှာ မှတ်တမ်းတင်ထားတဲ့ Version မှတ်တမ်းကို Centralize Repository ထံ ပေးပို့သိမ်းဆည်းလိုတဲ့အခါ `git push` ကို သုံးရပါတယ်။

```
$ git push origin master

Counting objects: 3, done.
Writing objects: 100% (3/3), 228 bytes | 0 bytes/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To /opt/lampp/htdocs/rockstar
* [new branch]      master -> master
```

စောစောကကြည့်ခဲ့တဲ့ `git pull` မှာလိုပဲ `origin` ဆိုတဲ့ Remote အညွှန်းနဲ့ `master` ဆိုတဲ့ ပေးပို့လိုတဲ့ Branch တို့ကို `git push` နဲ့အတူ တွဲဖက်ပေးရခြင်း ဖြစ်ပါတယ်။ ကိုယ်က Version မှတ်တမ်းတွေကို Centralize Repository ထံပေးပို့နေသလို အခြား Developer များကလည်း ပေးပို့နေမှာမို့ Centralize Repository ထံကနေ နောက်ဆုံး Update ကိုလည်း ပြန်လည်ရယူဖို့ လိုပါသေးတယ်။ Centralize Repository မှာ Update ရှိနေတာကို မရယူပဲနဲ့လည်း ကိုယ့်ရဲ့ Version တွေကို ပေးပို့လို့ရမှာ မဟုတ်ပါဘူး။

```
$ git push origin master

To /opt/lampp/htdocs/rockstar
 ! [rejected]        master -> master (fetch first)
error: failed to push some refs to '/opt/lampp/htdocs/rockstar'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

နမူနာကိုလေ့လာကြည့်ရင် Centralize Repository က ပေးပို့လာတဲ့ Version တွေကို လက်ခံဖို့ ငြင်းပယ်နေတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။ Centralize Repository ထဲမှာ Update တွေရှိနေတဲ့အတွက် အဲဒီ Update တွေကို `git pull` နဲ့ အရင်ရယူပြီးမှ ပြန်လည်ပေးပို့ဖို့လည်း ပြောထားပါသေးတယ်။

```
$ git pull origin master

remote: Counting objects: 4, done.
remote: Compressing objects: 100% (2/2), done.
Unpacking objects: 100% (3/3), done.
From /opt/lampp/htdocs/rockstar
 * branch            master     -> FETCH_HEAD
   6a05a77..b7ab351  master     -> origin/master
Merge made by the 'recursive' strategy.
 LICENSE | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 LICENSE
```

`git pull` နဲ့ Centralize Repository က နောက်ဆုံး Update ကိုရယူတဲ့အခါ ကိုယ့် Local Version တွေနဲ့ Centralize Repository ကပြန်ပေးလာတဲ့ Version တွေကို Git က Merge လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ ဒီနေရာမှာ တစ်ခါတစ်ရံ Conflict တွေရှိတက်ပါတယ်။ Branch တွေ Merge လုပ်တုန်းကလိုပဲ Local မှာ ကိုယ်ပြင်ထားတဲ့ ဖိုင်နဲ့ Centralize Repository က ပြန်ပေးလာတဲ့ နောက်ဆုံး Version ထဲက ပြင်ထားတဲ့ ဖိုင်ချင်းတိုက်နေရင် Merge Conflict ဖြစ်တက်ခြင်း ဖြစ်ပါတယ်။ အဲဒီလို Conflict တွေရှိလာရင်တော့ Conflict ရှိနေတဲ့ဖိုင်ကို လိုသလိုဖွင့်ပြင်ပြီး Commit ပြန်လုပ်ပေးဖို့လိုမှာဖြစ်ပါတယ်။

နောက်ဆုံး Update ရယူပြီးနောက်ကိုယ့်ရဲ့ Local Version Update ကို Centralize Repository ထံ ပြန်လည်ပေးပို့ လိုက ပေးပို့နိုင်ပြီးဖြစ်ပါတယ်။

```
$ git push origin master
Counting objects: 7, done.
Delta compression using up to 4 threads.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 604 bytes | 0 bytes/s, done.
Total 5 (delta 0), reused 0 (delta 0)
To /opt/lampp/htdocs/rockstar
   b7ab351..72044c5  master -> master
```

ဒီနေရာမှာ တစ်ခုသတိပြုရမှာက၊ Bare Repository တစ်ခုထံကိုသာ Push လုပ်လို့ရပါတယ်။ ရိုးရိုး Repository တစ်ခု ကနေ Clone လုပ်ထားရင်တော့ မူရင်း Repository ထံက Update ကို Pull လုပ်ယူလို့ရပေမယ့်၊ ကိုယ့် Update ကိုတော့ ပြန်လည်ပေးပို့လို့ ရမှာမဟုတ်ပါဘူး။ ဒါကြောင့် Centralize Repository အဖြစ်ထားရှိလိုတဲ့ အခါတွေမှာ Bare Repository တွေကိုသာ Centralize Repository အဖြစ် ထားရှိအသုံးပြုရခြင်းဖြစ်ပါတယ်။

Other

Git မှာ အခြားအသုံးဝင်တဲ့ အခြေခံလုပ်ဆောင်ချက် တစ်ချို့ ရှိပါသေးတယ်။ Commit ပြုလုပ်လိုက်ပြီးမှာ အဲ့ဒီ Commit ကို ပြန်လည်ပြင်ဆင်လိုတယ်ဆိုရင် `--amend` Option ကို သုံးနိုင်ပါတယ်။

```
$ git commit --amend -am "Commit comment"
```

နမူနာမှာပေးထားသလို Commit ပြုလုပ်စဉ်မှာ `--amend` Option ကိုထည့်သွင်းအသုံးပြုခဲ့ရင် Commit အသစ်အနေနဲ့ မှတ်တမ်းမတင်တော့ပဲ နောက်ဆုံးပြုလုပ်ခဲ့တဲ့ Commit အစား အစားထိုးမှတ်တမ်းတင်သွားမှာဖြစ်ပါတယ်။

အရင်ပြုလုပ်ခဲ့တဲ့ Commit တွေကို အပြီးဖျက်ပြစ်လိုရင်တော့ `git reset` ကို သုံးနိုင်ပါတယ်။ ဖျက်လိုတဲ့ Commit ရဲ့ Hash ID ကိုသုံးရမှာဖြစ်လို့ Log ထဲမှာ Commit ID ကိုအရင်ကြည့်ဖို့တော့လိုပါတယ်။

```
$ git log --pretty=format:"%h %s"

72044c5 Merge branch 'master' of /opt/lampp/htdocs/rockstar
9e4b188 Added index.html
b7ab351 Added LICENSE
6a05a77 Added README
```

```
$ git reset --hard b7ab351

HEAD is now at b7ab351 Added LICENSE

$ git log --pretty=format:"%h %s"

b7ab351 Added LICENSE
6a05a77 Added README
```

နမူနာကိုလေ့လာကြည့်ရင် မူလက Commit (၅) ခုရှိခဲ့ပေမယ့် `git reset --hard b7ab351` လို့ပြောလိုက်တဲ့ အချိန်မှာ နောက်ဆုံး Version အဖြစ် `b7ab351` ကို သတ်မှတ်လိုက်တဲ့အတွက် `b7ab351` နောက်ပိုင်း Commit တွေ အားလုံးပျက်သွားမှာဖြစ်ပါတယ်။ ဒီနည်းလမ်းဟာ Source Code တွေ Loss ဖြစ်နိုင်လို့ ထူးခြားတဲ့လိုအပ်ချက်မရှိရင် မသုံးသင့်တဲ့နည်းလမ်းတစ်ခုဖြစ်ပါတယ်။

နောက်ထပ်သတိပြုသင့်တဲ့အချက်တစ်ချက် ရှိပါသေးတယ်။ Windows, Mac, Linux စတဲ့ Operating System တွေမှာ သုံးကြတဲ့ Line Ending နည်းစနစ်တွေ ကွဲပြားတက်ပါတယ်။ တစ်ချို့က Carriage Return (CR) သင်္ကေတကို Line Ending အဖြစ် သုံးကြပါတယ်။ တစ်ချို့က Line Feed (LF) သင်္ကေတကိုသုံးပါတယ်။ တစ်ချို့ကတော့ နှစ်ခုလုံး ပေါင်းပြီး (CRLF) ဆိုပြီး သုံးပါတယ်။ Developer တစ်ယောက်က Linux နဲ့ရေးထားတဲ့ Source Code ကို နောက် Developer တစ်ယောက်က Windows မှာ ဖွင့်လိုက်တဲ့အခါ Line Ending ပြောင်းသွားနိုင်ပါတယ်။ အဲ့ဒီလို ပြောင်းသွားတဲ့အခါ Git က Source Code ကို ပြင်လိုက်တယ်လို့ ယူဆလိုက်မှာပါ။ တစ်ကယ်တမ်းက ပြင်လိုက်တာမဟုတ်ပဲ OS မတူလို့ Line Ending ပြောင်းသွားတာပါ။ ဒီပြဿနာမျိုး မတက်စေဖို့အတွက် Windows အသုံးပြုသူတွေဟာ ဒီ Setting ကို သတ်မှတ်ထားသင့်ပါတယ်။

```
$ git config --global core.autocrlf true
$ git config --global core.safecrlf true
```

Mac သို့မဟုတ် Linux အသုံးပြုသူတွေကတော့ ဒီလို သတ်မှတ်ထားသင့်ပါတယ်။

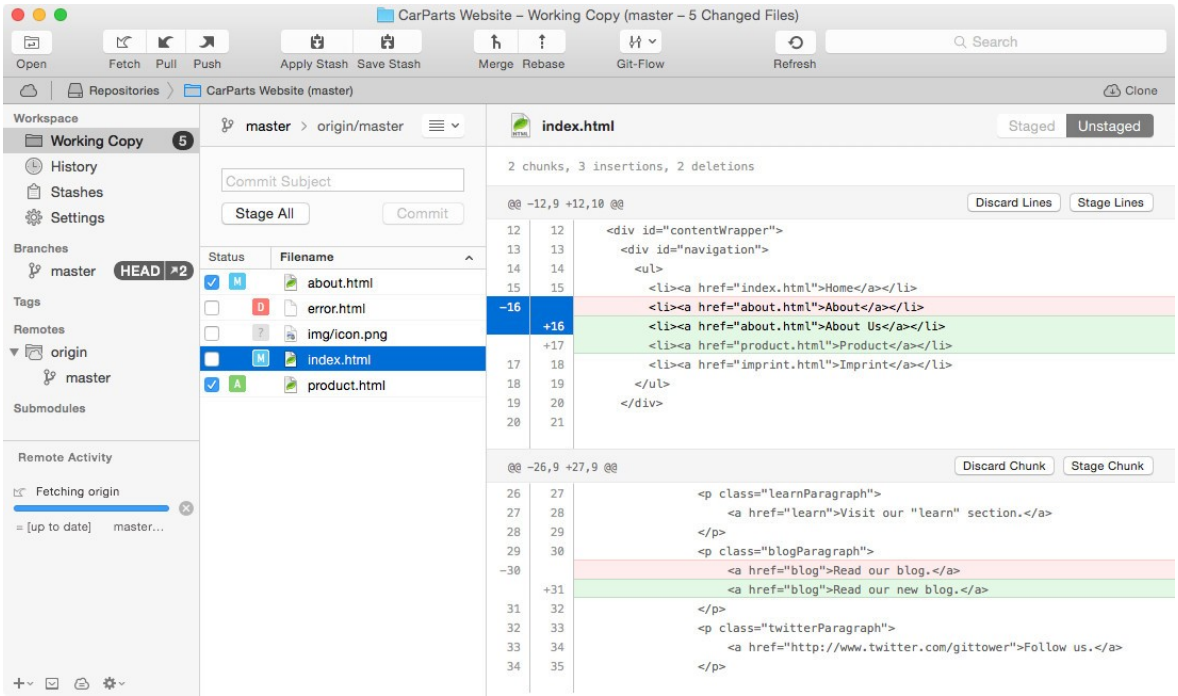
```
$ git config --global core.autocrlf input
$ git config --global core.safecrlf true
```

ဒီတော့မှ Git က Line Ending ကို သင့်တော်အောင် အလိုအလျောက် Convert လုပ်ပြီး အလုပ်လုပ်ပေးသွားမှာဖြစ်လို့ OS မတူလို့ Line Ending ပြောင်းပြီးဖြစ်ရတဲ့ ပြဿနာမဖြစ်တော့မှာပဲဖြစ်ပါတယ်။ Windows Setting မှာ `autocrlf true` လို့ ပြောထားတဲ့အတွက် Commit လုပ်ချိန်မှာ Line Ending ကို LF ပြောင်းပြီးမှ Commit လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ Checkout လုပ်ချိန်မှာတော့ Windows Line Ending ဖြစ်တဲ့ CRLF ပြောင်းပြီး Checkout လုပ်ပေးသွား မှာပဲဖြစ်ပါတယ်။ Mac နဲ့ Linux အတွက်တော့ `autocrlf input` လို့ ပြောထားတဲ့အတွက် Checkout မှာ Line Ending Conversion ကို လုပ်မနေတော့ပဲ Commit မှာပဲ Line Ending Conversion ကို လုပ် ပေးသွားမှာဖြစ်ပါတယ်။ `safecrlf true` Setting ကတော့ Line Ending Conversion လုပ်လိုက်တဲ့အတွက် Source Code ရဲ့ Integrity ကို ထိခိုက်မသွားအောင်စစ်ဆေးဖို့ သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။ `safecrlf` လို့ ပြောထားတဲ့အတွက် Source Code ကို ထိခိုက်နိုင်စရာ ရှိတယ်ဆိုရင် Convert မလုပ်တော့ပဲ Commit တွေကို ငြင်းပယ် သွားမှာ ဖြစ်ပါတယ်။

နောက်ထပ်သတိပြုစရာတစ်ချက်ကတော့ Indention ဖြစ်ပါတယ်။ Code တွေရေးသားတဲ့အခါ Indent တွေ မဖြစ်မနေ ပါဝင်ကြပါတယ်။ တစ်ချို့က Tab ကို Indent လုပ်ဖို့သုံးပြီး တစ်ချို့ကတော့ Space ကို Indent လုပ်ဖို့ သုံးပါတယ်။ ဘာသုံးသုံး တစ်မျိုးတည်း သုံးမယ်ဆိုရင်ရပါတယ်။ Tab နဲ့ Space ကို ရောသုံးတဲ့အခါမှာတော့ ပြဿနာရှိတက်ပါတယ်။ Developer တစ်ယောက် က Indent အတွက် Space သုံးထားတယ်။ နောက် Developer တစ်ယောက်က Code ကို ပြင်တာမဟုတ်ပေမယ့် တစ်နေရာမှာ Indent Space အစား Tab ထည့်လိုက်မိတယ်ဆိုရင် Space နဲ့ Indent တွေရော နေပြီး မလိုလားအပ်တဲ့ ပြဿနာတစ်ချို့ ရှိတက်ပါတယ်။ Git ကတော့ Default အနေနဲ့ Tab နဲ့ Space တွေရောပြီးသုံး ထားရင် Commit တွေကို လက်မခံပဲ ငြင်းပယ်သွားမှာဖြစ်ပါတယ်။ ဒီပြဿနာကိုရှောင်ရှားဖို့အတွင် Git Setting ကို ပြင်မနေပဲ ရေးသားသူတွေကြားထဲမှာ Coding Standard တစ်ခု ထားပြီးတော့ ညှိနှိုင်းကြဖို့ လိုအပ်ပါတယ်။

Git ကို Command Line ကနေမသုံးပဲ User Interface နဲ့ သုံးချင်သူများအနေနဲ့ TortoiseGit, Giggle, Tower စတဲ့ UI Software တွေကို အသုံးပြုနိုင်ပါတယ်။ တစ်ချို့ အခမဲ့ရပြီး တစ်ချို့ လိုင်စင်ဝယ်သုံးရပါတယ်။ အောက်ပါ လိပ်စာမှာ အသုံးများတဲ့ Git GUI Software စာရင်းကို လေ့လာနိုင်ပါတယ်။

http://git-scm.com/downloads/guis



ပုံ (၅.၅) - Git Tower – Git GUI Software on Mac

နောက်ဆုံးတစ်ချက်အနေနဲ့၊ Git Repository တွေကို အဖွဲလိုက်ပူးပေါင်းဆောင်ရွက်နိုင်ဖို့ Centralize Server တစ်ခုနဲ့ Setup လုပ်ရာမှာ Github (github.com) နဲ့ BitBucket (bitbucket.org) တို့လို Source Code Hosting Service တွေကိုလည်း လေ့လာအသုံးပြုသင့်ကြောင်း အကြံပြုလိုပါတယ်။ ဒီ Service တွေရဲ့အကူအညီနဲ့ Centralize Server တစ်ခုကို ကိုယ်တိုင် Setup လုပ်နေစရာမလိုပဲ အလွယ်တစ်ကူ ရရှိနိုင်ပါတယ်။

Software Versioning

ဆက်လက်ဖော်ပြချင်တာကတော့ Software တစ်ခုကို Version နံပါတ်တွေ ပေးပုံပေးနည်းဖြစ်ပါတယ်။ အခြေခံအားဖြင့် Commit တစ်ခုဟာ Version တစ်ခုဖြစ်တယ်လို့ အကြမ်းဖျင်းအားဖြင့် ပြောနိုင်ပါတယ်။ ဒါပေမယ့် မှတ်သားရခက်တဲ့ Commit Hash တွေကို Version နံပါတ်အဖြစ် သုံးလို့တော့ အဆင်မပြေပါဘူး။ အရေးပါတဲ့ Commit တွေကို နားလည် မှတ်သားရ လွယ်တဲ့ Version နံပါတ်တွေ သတ်မှတ်ထားပေးဖို့လိုပါတယ်။ Git ရဲ့ Tag လုပ်ဆောင်ချက်ကို ဒီနေရာမှာ အသုံးချနိုင်ပါတယ်။

Github ကိုပူးတွဲတည် ထောင်ခဲ့သူတစ်ဦးဖြစ်တဲ့ Tom Preston-Werner ရေးသားတင်ပြထားတဲ့ Semantic

Versioning ဆိုတဲ့ Version နံပါတ်သတ် မှတ်ရာမှာ လိုက်နာသင့်တဲ့ နည်းလမ်းများဆိုတာ ရှိပါတယ်။ သတ်မှတ်ချက်အပြည့်အစုံပါဝင် တဲ့ Version နံပါတ်တစ်ခုဟာ **ပုံ (၅.၆)** မှာဖော်ပြထားသလို ပုံစံဖြစ်မှာပါ။



ပုံ (၅.၆) - Semantic Version Number

Software တစ်ခု ပထမဆုံးစတင်စမ်းသပ်အသုံးပြုလို့ရတဲ့ အဆင့်ကို Version 0.1.0 လို့သတ်မှတ်ရမှာ ဖြစ်ပါတယ်။ အဲဒီအဆင့်မှာ Software က စပြီးစမ်းသုံးလို့ရနေပါပြီ။ ဒါပေမယ့် သတ်မှတ်လုပ်ဆောင်ချက်တွေတော့ စုံလင်အောင် မပါဝင် သေးပါဘူး။ အဲ့ဒီ အဆင့်မှာပဲ အများတစ်ချို့ စတွေ့လို့ ပြင်ဆင်လိုက်မယ်ဆိုရင် Version 0.1.1, 0.1.2, 0.1.3 စသဖြင့် ပြင်ဆင်ချက်အလိုက် နောက်ဆုံးက PATCH နံပါတ်လည်း လိုက်တိုးလာမှာဖြစ်ပါတယ်။

နောက်ထပ်ဖြည့်စွက်လုပ်ဆောင်ချက်တစ်ချို့ ဖြည့်စွက်လိုက်လို့ ပိုပြည့်စုံလာပြီဆိုရင်တော့ Version 0.2.0 အဖြစ်သတ် မှတ်ရမှာဖြစ်ပါတယ်။ 0.2.0 အဆင့်မှာ ပြုလုပ်ခဲ့တဲ့ အများပြင်ဆင်ချက်တွေကိုတော့ 0.2.1, 0.2.2, 0.2.3 စသဖြင့်အဆင့်လိုက်ပေးသွားရမှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ 0.3.0, 0.3.2, 0.3.2, 0.4.0, 0.4.1 စသဖြင့် လုပ်ဆောင်ချက်တိုးလာတိုင်း MINOR Version နံပါတ်တိုးလာသလို့ MINOR Version တစ်ခုချင်းစီမှာ ပြင်ဆင်မှုတွေ လုပ်လိုက်တိုင်း PATCH Version နံပါတ်တွေလည်း တိုးလာမှာပဲဖြစ်ပါတယ်။

ဒီနည်းနဲ့ နောက်ဆုံး အများအသုံးပြုနိုင်ဖို့ ကြေငြာနိုင်လောက်အောင် ပြည့်စုံလာပြီဆိုရင်တော့ MAJOR Version နံပါတ် တိုးသွားပြီး 1.0.0 ဖြစ်သွားမှာပဲဖြစ်ပါတယ်။ ဒါပေမယ့် အများကိုကြေငြာတော့မှာမို့ ပြည့်စုံပြီးအများနည်းဖို့လိုပါတယ်။ အသေအချာမစမ်းသပ်ရသေးတဲ့ အခြေအနေမှာ 1.0.0-pre-alpha လို့ သတ်မှတ်ထားသင့်ပါတယ်။ စမ်းသပ်မှုတွေ စတင်ပြီဆိုရင်တော့ 1.0.0-alpha ကိုယ်တိုင်စမ်းသပ်ယုံမက ပြင်ပ User တွေကိုလည်း စတင်စမ်းသပ်ခွင့်ပြုပြီဆိုရင် 1.0.0-beta စသဖြင့် သတ်မှတ်သွားရမှာဖြစ်ပါတယ်။ အဲ့ဒီ pre-alpha, alpha နဲ့ beta အဆင့်တွေမှာ အများပြင်ဆင်ချက် တွေ ရှိခဲ့တယ်ဆိုရင် PRE-RELEASE PATCH နံပါတ်ကို တိုးသွားနိုင်ပါတယ်။ 1.0.0-beta.1, 1.0.0-beta.2 စသဖြင့် ပေးသွားနိုင်ပါတယ်။

စိတ်ကျေနပ်လောက်အောင် စမ်းသပ်ပြုပြင်မှုတွေဆောင်ရွက်ပြီးလို့ အသင့်ဖြစ်နေပြီဆိုရင်တော့ 1.0.0-rc လို့ သတ်မှတ်နိုင်ပါတယ်။ rc က Release Candidate ဆိုတဲ့အဓိပ္ပါယ်ပါ။ ဒီအတိုင်း Release လုပ်တော့မယ်လို့ ဆုံးဖြတ်လိုက်တဲ့အဆင့်ဖြစ် ပါတယ်။ 1.0.0-rc အဆင့်မှာ ပြဿနာတစ်စုံတစ်ရာမရှိတော့ဘူးဆိုရင်တော့ PRE-RELEASE ကိုဖြုတ်လိုက်ပြီး Version 1.0.0 အနေနဲ့ အများ အသုံးပြုနိုင်အောင် ကြေငြာနိုင်ပြီပဲဖြစ်ပါတယ်။

1.0.0 အဖြစ် ကြေငြာပြီးနောက် အသေးစားပြင်ဆင်ချက်တွေကို 1.0.1, 1.0.2 စသဖြင့် ဆက်လက် Release လုပ်သွားလို့ ရပါတယ်။ အသေးစားဖြည့်စွက်ချက်တွေပြုလုပ်ခဲ့ရင်တော့ 1.1.0, 1.2.0 စသဖြင့် ဆက်လက် Release လုပ်သွားနိုင်ပါတယ်။ အဲ့ဒီအဆင့်တွေမှာလည်း 1.1.0-beta, 1.2.0-rc စသဖြင့် PRE RELEASE သတ်မှတ်ချက်တွေ လိုအပ်ရင်တွဲသုံးနိုင် ပါသေးတယ်။

ပြင်ဆင်ဖြည့်စွက်ချက်တွေ တစ်ဖြည်းဖြည်းများလာပြီး၊ သိသာမြင်သာတဲ့ နောက်အဆင့်တစ်ခုကို ရောက်လာပြီဆိုရင်၊ (သို့မဟုတ်) Software ကို နည်းစနစ်သစ်၊ ပုံစံသစ်တွေနဲ့ ပြန်လည်ဆန်းသစ်တော့မယ်ဆိုရင်တော့ 2.0.0 ဆိုပြီး MAJOR Version နံပါတ်သစ်တစ်ခုနဲ့ ဆက်လက်ဆောင်ရွက်ရမှာဖြစ်ပါတယ်။

Commit Hash (သို့မဟုတ်) Software ကို Build လုပ်လိုက်ရာမှာ ရရှိလာတဲ့ Unique ID (သို့မဟုတ်) Release လုပ်ခဲ့တဲ့ရက်စွဲ စတဲ့အချက်အလက်တွေကို Version နံပါတ်နဲ့တွဲဖက်မယ်ဆိုရင် နောက်ဆုံးကနေ + သင်္ကေတနဲ့ တွဲဖက်နိုင် ပါတယ်။ ဒါပေမယ့် အချက်အလက်အနေနဲ့သာ ထည့်သွင်းသင့်ပြီး အဲ့ဒီအချက်အလက်ကို မှီခိုအလုပ်လုပ်တာမျိုးတော့ မလုပ်ရပါဘူး။ ဥပမာ – Version 2.5.1-rc.2+72044c5, 1.4.3+20150318 စသဖြင့် တွဲဖက်ပေးနိုင်ပါတယ်။

Software တိုင်းက ဒီနည်းတစ်မျိုးတည်းကို သုံးနေတာတော့ မဟုတ်ပါဘူး။ တစ်ချို့လည်း Version ကို နံပါတ်တင်မက နာမည်နဲ့လည်း ပေးတက်ကြပါတယ်။ ဥပမာ - Flash MX, Photoshop CS, Windows XP။ တစ်ချို့လည်း ခုနှစ်နဲ့ ပေးတက်ကြပါတယ်။ ဥပမာ – Windows 2000, Ubuntu 14.04။ တစ်ချို့လည်း Internal Version နံပါတ်နဲ့ အများ ကို ကြေငြာတဲ့နံပါတ်ဆိုပြီး နှစ်မျိုးထားတက်ကြပါတယ်။ ဥပမာ – Java7 ရဲ့ တစ်ကယ့် Internal Version နံပါတ်က Java 1.7 ဖြစ်ပါတယ်။ လက်တွေ့မှာ Version နံပါတ်တွေကို ကိုယ့်နည်းကိုယ့်ဟန်နဲ့ အမျိုးမျိုးပေးတက်ကြပါတယ်။ ဒါကြောင့် အခုဖော်ပြခဲ့တဲ့နည်းအတိုင်းအမြဲပေးရမယ်လို့တော့ မဆိုလိုပါဘူး။ ဒါပေမယ့် ဒီနည်းစနစ်ဟာ အများစံထားသုံး နေတဲ့ နည်းစနစ်တစ်ခုဖြစ်လို့ လိုက်နာအသုံးပြုသင့်တဲ့နည်းစနစ်တစ်ခုပဲဖြစ်ပါတယ်။

Conclusion

Software Project တစ်ခုကို တစ်ဦးတည်းရေးသားသည်ဖြစ်စေ၊ Team နဲ့ ပူးပေါင်းရေးသားသည်ဖြစ်စေ Version Control System တစ်ခုကိုတော့ မဖြစ်မနေ အသုံးပြုသင့်ပါတယ်။ တစ်ချို့လည်း Centralize Code Base လိုအပ်ပြီး အဖွဲ့လိုက်ရေးသားရတဲ့ Project တွေမှသာ VCS တွေလိုတယ်လို့ အမှတ်မှားကြပါတယ်။ လက်တွေ့မှာ Version Control System တွေက Centralize Code Base တစ်ခုရရှိအောင် အကူအညီပေးယုံမှ မက၊ အဆင့်လိုက် မှတ်တမ်းတင်ပေးတဲ့ အတွက် Code Loss မဖြစ်အောင် ကာကွယ်ပေးနိုင်ခြင်းနဲ့၊ Branch လုပ်ဆောင်ချက်နဲ့အတူ Code Base တစ်ခုတည်းကို သုံးပြီး မတူကွဲပြားတဲ့ Version တွေကို ရေးသားစီမံနိုင်အောင်လည်း ကူညီပေးနိုင်လို့ တစ်ဦးတည်းရေးသားတဲ့ Software Project တွေမှာလည်း အသုံးပြုသင့်ပါတယ်။

နမူနာအနေနဲ့ လက်ရှိလူသုံးအများဆုံး VCS ဖြစ်တဲ့ Git အကြောင်းကိုဖော်ပြခဲ့ပေမယ့် အခြား VCS တွေကို စမ်းသပ် လေ့လာသင့်ပါတယ်။ Subversion (Centralize), Mercurial (Distributed), Bazaar (Distributed) စတဲ့ VCS တွေ ရှိပါသေးတယ်။

Joel Spolsky လို့ခေါ်တဲ့ လူသိများထင်ရှားတဲ့ ပညာရှင်တစ်ဦးဖော်ထုတ်ခဲ့တဲ့ "The Joel Test: 12 Steps to Better Code" ဆိုတဲ့ အချက် (၁၂) ချက်ပါဝင်တဲ့ Software Team တစ်ခုရဲ့ အရည်အသွေးကို တိုင်းတာတဲ့ နည်းလမ်းတစ်ခု ရှိပါတယ်။ အဲ့ဒီ အချက်တွေထဲမှာ နံပါတ် (၁) အချက်က "Do you use source control?" ဆိုတဲ့ အချက်ဖြစ်ပါတယ်။ Source Code တွေစီမံနိုင်တဲ့ စနစ်တစ်ခုကို အသုံးပြုခြင်း၊ မပြုခြင်းဟာ Software Team တစ်ခုရဲ့ Performance ပေါ်မှာ အများကြီး သက်ရောက်မှုရှိစေမှာပဲဖြစ်ပါတယ်။

The Joel Test ထဲက ကျန်အချက်တွေနဲ့အတူ နောက်ထပ်အရေးကြီးတဲ့ အကြောင်းအရာတစ်ခုဖြစ်တဲ့ Issue Tracking System အကြောင်းကို နောက်အခန်းမှာ ဆက်လက်ဖော်ပြပေးသွားပါမယ်။

ပြင်ရတာမခက်ပေမယ့်၊ သိပ်အရေးမကြီးလို့ နောင်မှ
ပြင်မယ်ဆိုပြီးထားလိုက်တဲ့ Bug ကို၊ မေ့ပြီးမပြင်မိလို့
အသုံးပြုသူလက်ထဲရောက်မှ အဲ့ဒီ Bug ပြန်ပေါ်တဲ့အတွက်
ကိုယ့် Software ရဲ့အရည်အသွေးကိုမေးခွန်းထုတ်စရာ
ဖြစ်သွားတာလောက် နောင်တရဖို့ကောင်းတာ မရှိပါဘူး။

Professional Web Developer Course

HTML5, PHP/MySQL, jQuery/Ajax, Mobile Web စသည့်
Professional Web Developer တစ်ဦး သိရှိထားသင့်သည့်
နည်းပညာများကို စုစည်းသင်ကြားခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၆) – Issue Tracking System

StackOverflow လို့ခေါ်တဲ့ Software Developer တိုင်းအားထားရတဲ့ Q&A Platform ကို ပူးတွဲတည်ထောင်သူ တစ်ဦးဖြစ်ပြီး၊ သူ့ရဲ့ နည်းပညာ ဆောင်းပါးတွေကြောင့် နာမည်ကျော်ကြားတဲ့ Joel Spolsky ဆိုတဲ့ပညာရှင်တစ် ယောက်က Software Development Team တစ်ခုရဲ့ အရည်အသွေးကို အချက် (၁၂) ချက်နဲ့တိုင်းတာနိုင်တယ်လို့ ဆိုထားပါတယ်။

အဲ့ဒီအချက်တွေကတော့ –

1. Do you use source control?
၁။ Source Control (VCS) အသုံးပြုသလား

2. Can you make a build in one step?
၂။ Build လုပ်ငန်းကို တစ်ကြိမ်တည်းတစ်ဆင့်တည်းနဲ့ ဆောင်ရွက်နိုင်ရဲ့လား

3. Do you make daily builds?
၃။ နေ့စဉ် Build လုပ်ရဲ့လား

4. Do you have a bug database?
၄။ Bug Database တစ်ခုထားရှိသလား

5. Do you fix bugs before writing new code?
၅။ Code အသစ်မရေးခင် ရှိနေတဲ့ Bug တွေကို ဦးဆုံးရှင်းလင်းတဲ့ အလေ့အထရှိရဲ့လား

6. Do you have an up-to-date schedule?
၆။ Up-to-date Schedule ထားရှိသလား

7. Do you have a spec?
၇။ Specification တစ်ခု ပြင်ဆင်ရေးသားထားရှိသလား

8. Do programmers have quiet working conditions?
၈။ Programmer တွေ အနှောင့်အယှက်ကင်းကင်း အလုပ်လုပ်လို့ရတဲ့ အခြေအနေရှိရဲ့လား

9. Do you use the best tools money can buy?

၉။ အကောင်းဆုံး Development Tool တွေကို အသုံးပြုရဲ့လား

10. Do you have testers?

၁၀။ Tester တွေ သီးခြားခန့်ထားရဲ့လား

11. Do new candidates write code during their interview?

၁၁။ အလုပ်မခန့်ခင် လျှောက်ထားသူကို လက်တွေ့ Code ရေးခိုင်းကြည့်ပြီးမှ ခန့်တာလား

12. Do you do hallway usability testing?

၁၂။ Hallway Usability Testing လို့ခေါ်တဲ့ နည်းစနစ်ကို ကျင့်သုံးရဲ့လား

– စတဲ့ အချက်တွေပဲဖြစ်ပါတယ်။ ဒီအချက်တွေထဲက တစ်ချက်ကိုလိုက်နာတိုင်း အဲ့ဒီ Team ရဲ့ စွမ်းဆောင်ရည်ဟာ တစ်ဆင့်တိုး တက်တယ်လို့ ဆိုထားပါတယ်။ အချက်အားလုံးလိုက်နာနိုင်မယ်ဆိုရင်တော့ အရည်အသွေးမြင့် Software Development Team တစ်ခု ဖြစ်မှာပဲဖြစ်ပါတယ်။

နံပါတ် (၁) အချက်ဖြစ်တဲ့ **Source Control** ရဲ့အရေးပါမှုအကြောင်းကိုတော့ ပြီးခဲ့တဲ့အခန်းမှာ ဖော်ပြခဲ့ပြီး ဖြစ်ပါတယ်။

နံပါတ် (၂) အချက်ဖြစ်တဲ့ **One-step Build** အကြောင်းကိုတော့ **အခန်း (၈) – Workflow Automation** ရောက်တဲ့ အခါ ဆက်လက်ဖော်ပြပါမယ်။

နံပါတ် (၃) အချက်ဖြစ်တဲ့ **Daily Build** ဆိုတာကဒီလိုပါ။ Version Control System တစ်ခုအကူအညီနဲ့ အဖွဲ့လိုက် ပူးပေါင်းရေးသားကြတဲ့အခါ၊ Developer တစ်ဦးဖြည့်စွက်လိုက်တဲ့ Code က သူ့အတွက်အဆင်ပြေပေမယ့် အခြား Developer တွေတာဝန်ယူ ဆောင်ရွက်နေတဲ့အပိုင်းကို သွားပြီးထိခိုက်သက်ရောက်နေတက်ပါတယ်။ အဲ့ဒီလိုအခြေအနေမှာ သူပြင်မပေးမချင်း ကျန်တဲ့သူတွေက ဆက်အလုပ်လုပ်လို့ မရတော့ဘူးဆိုတာမျိုးဖြစ်နေတက်ပါတယ်။ ဒါကြောင့် စနစ်တစ်ခုလုံး အမြဲအလုပ်လုပ်နေတယ်ဆိုတာ သေချာအောင် နေ့စဉ်စီစစ်ခြင်းလုပ်ငန်းကို ဆောင်ရွက်ဖို့လိုပါတယ်။ တစ်နည်းအားဖြင့် Daily Build အနေနဲ့ နေ့တိုင်း Project ကို Build လုပ်ဖို့လိုအပ်ပါတယ်။ ဒီလိုအပ်ချက်ကို ဖြည့်စည်းဖို့အတွက် VCS ကနေ Commit လုပ်လိုက်တာနဲ့ အလိုအလျောက် Build လုပ်ပေးနိုင်တဲ့ Continues Integration နည်းပညာတွေ ရှိပါတယ်။

နံပါတ် (၄) အချက်ဖြစ်တဲ့ **Bug Database** အကြောင်းကိုတော့ မကြာခင်ဆက်လက်ဖော်ပြပေးသွားပါမယ်။

နံပါတ် (၅) အချက်ဖြစ်တဲ့ **Fix Bugs Before Writing New Code** ဆိုတာက ဒီလိုပါ။ အမှားတစ်ခု (သို့မဟုတ်) Bug တစ်ခုဟာတွေ့တွေ့ချင်း ပြင်နိုင်လေ၊ ပြင်ရသက်သာလေပါပဲ။ အချိန်ကြာလာတာနဲ့အမျှ လုပ်ဆောင်ချက်တွေများ သထက်များလာပြီး ရှာဖွေပြင်ဆင်ရ ခက်ခဲလာတက်ပါတယ်။ ပြင်ဖို့မပြောနဲ့ အဲ့ဒီအမှားကို ပြန်ရှာရတာတောင် မလွယ် တော့ပဲ ခက်လာတော့မှာပါ။ ရှာတွေ့ပြန်တော့လည်း ပူပူနွေးနွေး လုပ်လက်စတုန်းက

လောက် အဲ့ဒီ ပြဿနာရဲ့ ရင်းမြစ်နဲ့ သဘာသဘာဝကို သိရှိနိုင်တော့မှာမဟုတ်ပါဘူး။ ဒါကြောင့် လက်တွေ့မှာ အလုပ်မြန်မြန်ပြီးချင်လို့ ရှိနေတဲ့ အမှားတွေ ကို နောက်မှပြင်မယ်သဘောထားရင် မြန်မြန်မပြီးတဲ့အပြင် ပိုကြာသွားတက်ပါတယ်။ ဒါကြောင့် လုပ်ဆောင်အချက်အသစ် တွေ ထည့်သွင်းခြင်းမပြုမီ ရှိနေတဲ့အမှားတွေကို အရင်ပြင်တဲ့အလေ့လအထက ရေရှည်မှာပိုပြီးထိရောက် မြန်ဆန်တဲ့ Productivity ကို ရရှိစေမှာပဲဖြစ်ပါတယ်။

နံပါတ် (၆) အချက်ဖြစ်တဲ့ **Up-to-date Schedule** က နံပါတ် (၅) နဲ့ ဆက်စပ်နေပါတယ်။ Software Project တစ်ခုကို စတင်စဉ်မှာ ပြီးစီးမယ့်အချိန် Release Schedule ကြိုတင်ခန့်မှန်းထားရပါတယ်။ ပြဿနာက၊ Requirement ဆိုတာ အခြေအနေအမျိုးမျိုးကြောင့် ပြောင်းသွားတက်လို့ မူလ Release Schedule က နောင်မှာ လက်တွေ့မကျတော့ တာမျိုး ဖြစ်လာတက်ပါတယ်။ အဲ့ဒီလိုဖြစ်လာတဲ့အခါ မူလသတ်မှတ်ထားတဲ့အတိုင်း ပြီးလိုဇောနဲ့ Bug ကို အရင်မပြင်ပဲ သတ်မှတ်ထားတဲ့ လုပ်ဆောင်ချက်တွေကို တစ်ခုပြီးတစ်ခုဆက် ထည့်သွားဖို့ အားထုတ်မိတက်ကြပါတယ်။ အဲ့ဒီအခါမှာ Bug တွေကို နောက်မှ ပြင်ရလို့ ကြာသင့်တာထက် ပိုကြာသွားတက်သလို၊ Bug တွေကုန်အောင် အရှင်းမိပဲ Software ကို Release လုပ်မိတက်တာမျိုးလည်း ဖြစ်တက်ပါတယ်။ ဒါကြောင့် လက်တွေ့ကျတဲ့ Up-to-date Schedule ရှိနေမှ Software တစ်ခုရဲ့ အရည်အသွေးကလည်း ကောင်းမွန်မှာ ဖြစ်ပါတယ်။

နံပါတ် (၇) အချက်ဖြစ်တဲ့ **Spec** ကလည်း အရေးပါပါတယ်။ လိုရင်းကတော့ Software Project တစ်ခုကို လက်တန်း မလုပ်ပဲ ကြိုတင်ပြင်ဆင်ထားတဲ့ Specification တစ်ခုပေါ်မှာ အခြေခံပြီး ဆောင်ရွက်ဖို့ပါပဲ။ ဒီတော့မှ လက်ရှိ Software ရဲ့ အခြေအနေ နဲ့ Specification ကို တိုက်ဆိုင်စစ်ဆေးပြီး၊ ဖြစ်နိုင်ခြေရှိတဲ့ Release Schedule မှန်းဆတဲ့ လုပ်ငန်းတွေ၊ Software ကို ဖြန့်ချိရာမှာ ကြိုတင်ပြင်ဆင်သင့်တဲ့လုပ်ငန်းတွေနဲ့ အခြားအထွေထွေကိစ္စရပ်များကို စနစ်တကျ ဆောင်ရွက်နိုင်မှာဖြစ်ပါတယ်။ တိကျတဲ့ Spec တစ်ခုရရှိဖို့ခက်တဲ့အကြောင်းကို **အခန်း (၁)** မှာ ဖော်ပြခဲ့ပြီး ဖြစ်ပါတယ်။ ဒီလိုခက် ခဲလို့ဆိုပြီး Spec လုံးဝမရှိလို့တော့ မရပါဘူး။

နံပါတ် (၈) အချက်ဖြစ်တဲ့ **Working Condition** နဲ့ပက်သက်လို့တော့ နည်းနည်း အငြင်းပွားစရာရှိပါတယ်။ တစ်ချို့ကလည်း Developer တွေအတွက် ဆိတ်ငြိမ်ပြီး အနှောက်အယှက်ကင်းကင်း အလုပ်လုပ်နိုင်တဲ့ Private Office တွေ ဖန်တီးပေးထားမှ ပိုပြီးအလုပ်တွင်တယ်လို့ ဆိုကြပါတယ်။ တစ်ချို့ကလည်း အတူတစ်ကွ ပူးပေါင်းဆောင်ရွက်နိုင်အောင် Developer တွေကို တစ်နေရာတည်းမှာ စုစည်းထားမှ ပိုပြီးအလုပ်တွင်တယ်လို့ ဆိုကြပါတယ်။ လိုရင်းကတော့၊ အရည် အသွေးမြင့် Software တွေဖန်တီးနိုင်ဖို့ဆိုရင် Developer တွေအတွက် မလိုလားအပ်တဲ့ အနှောက်အယှက်တွေကင်းပြီး၊ ပူးပေါင်းဆောင်ရွက်မှုကိုအားပေးတဲ့ Working Condition တစ်ခု ဖန်တီးထားပေးနိုင်ဖို့ လိုပါတယ်။

နံပါတ် (၉) အချက်ဖြစ်တဲ့ **Best Tools** ဆိုတာလည်း အလုပ်တွင်စေဖို့အတွက်ပါပဲ။ လိုင်စင်ဝယ်သုံးရတဲ့ IDE ကောင်း ကောင်းတစ်ခုကို သုံးလိုက်လို့ အလုပ်ပိုတွင်မယ်ဆိုရင် ကုန်ကျစရိတ်ကို နှမျောမနေပဲ၊ ဝယ်သုံးရပါမယ်။ မြန်နှုန်းမြင့် အင်တာနက်ရှိမှ အလုပ်တွင်မယ်ဆိုရင် မြန်နှုန်းမြင့် အင်တာနက် တပ်ဆင်ရပါမယ်။ Commercial Database System ဝယ်သုံးလိုက်မှ စွမ်းဆောင်ရည်ပိုကောင်းသွားမယ်ဆိုရင် ဝယ်သုံးရပါမယ်။ ကုန်ကျစရိတ်ကို နှမျောနေတာနဲ့ ထိရောက်သင့် သလောက်မထိရောက်လို့ အချိန်တွေကြာပြီး စရိတ်တွေအတက်ခံမယ့်အစား၊ လိုအပ်တဲ့ Tool ကို ဝယ်ယူအသုံးပြုလိုက်တာ က ရေရှည်အတွက် ပိုအကျိုးရှိတယ်ဆိုတဲ့ အဓိပ္ပါယ်ပဲဖြစ်ပါတယ်။

နံပါတ် (၁၀) အချက်ဖြစ်တဲ့ **Tester** ဆိုတာလည်း အရေးပါတဲ့အချက်တစ်ချက်ပါတယ်။ **အခန်း (၂)** မှာ Test Method အမျိုးမျိုးအကြောင်းကိုဖော်ပြခဲ့ပါတယ်။ Tester တွေရဲ့တာဝန်ကတော့ System Test လုပ်ငန်းကို ဆောင်ရွက်ဖို့ပဲဖြစ် ပါတယ်။ Software ဟာ Specification က သတ်မှတ်ထားတဲ့ သတ်မှတ်ချက်များအတိုင်း မှန်မှန် ကန်ကန်အလုပ်လုပ်ရဲ့ လား၊ သတ်မှတ်ထားတဲ့ လုပ်ဆောင်ချက်တွေ အပြည့်အစုံ ပါရဲ့လား၊ မရှိသင့်တဲ့ လုပ်ဆောင်ချက်တွေနဲ့ အမှားတွေရှိနေသလား၊ စသဖြင့် ထောင့်စိအောင်စိစစ်ပေးမယ့်သူများဖြစ်ပါတယ်။ Tester ဆိုတာ Software ရဲ့ Quality ကို စီစစ်အာမခံ ပေးရမယ့်သူများ ဖြစ်ပါတယ်။ ဒါကြောင့် သူတို့ကို Quality Assurance Engineer များလို့လည်း ခေါ်နိုင်ပါတယ်။ Joel Spolsky ရဲ့အဆိုအရဆိုရင် Team တစ်ခုမှာ Developer နှစ်ယောက်ကို Tester တစ်ယောက်နှုန်းနဲ့ တွဲဖက်ထားရှိသင့် တယ်လို့ ဆိုထားပါတယ်။

နံပါတ် (၁၁) အချက်ဖြစ်တဲ့ **Interview** ကတော့ Team မှာ ပါဝင်တဲ့ Developer ဟာ ရှိသင့်တဲ့ အရည်အချင်းတွေရှိနေတာ သေချာစေဖို့ ဖြစ်ပါတယ်။ Developer တစ်ယောက်ဟာ အမှန်တစ်ကယ် အရည်အချင်းရှိမရှိဆိုတာကို သူမှာရှိထားတဲ့ ဘွဲ့လက်မှတ်တွေ၊ အောင်လက်မှတ်တွေကို ကြည့်ပြီးမဆုံးဖြတ်သင့်သလို၊ အမေးအဖြေသက်သက် အင်တာဗျူးနဲ့လည်း မဆုံးဖြတ်သင့်ပါဘူး။ လက်တွေ့မှာ၊ လေးစားလောက်ဖွယ် Degree တွေကိုင်ထားပေမယ့် အခြေခံ Programming Logic တောင် မပိုင်နိုင်တဲ့သူတွေအများကြီးရှိပါတယ်။ ဒါကြောင့် Team မှာပါဝင်တဲ့ Developer များကို ခန့်ထားဖို့ စီစစ်ရာမှာ လက်တွေ့ Code ရေးခိုင်းပြီး အမှန်တစ်ကယ် အရည်အချင်း ရှိသူဟုတ်မဟုတ် စီစစ်ဖိုလိုတယ်ဆိုတဲ့အချက်ကို ထည့်သွင်း ထားခြင်းဖြစ်ပါတယ်။

နံပါတ် (၁၂) အချက်ဖြစ်တဲ့ **Hallway Usability Testing** ဆိုတာကတော့၊ လမ်းသွားလမ်းလာ Hallway မှာတွေ့တဲ့သူကို ဆွဲခေါ်ပြီးကိုယ်ဖန်တီးထားတဲ့ Software ကို စမ်းသုံးခိုင်းကြည့် သင့်တယ်ဆိုတဲ့သဘောပါ။ ကိုယ့် Software အကြောင်း လုံးဝမသိတဲ့ Hallway ထဲမှာတွေ့တဲ့သူတောင် ချက်ချင်းသုံးနိုင်တယ်ဆိုရင် ကိုယ့် Software က တော်တော်ကို Usable ဖြစ်တယ်လို့ မှတ်ယူနိုင်ပါတယ်။ မသုံးနိုင်ဘူး ဆိုရင်လည်း သူ့အတွက်ဘာတွေက အသုံးပြုရခက်နေတာလဲဆိုတဲ့အချက် အပါအဝင် သူ့ဆီကပြန်ရလာတဲ့ Feedback က Software ရဲ့ Usability ကို မြှင့်တင်ဖို့ အထောက်အကူ ဖြစ်စေမှာဖြစ်ပါတယ်။

ဒီအချက်တွေပါဝင်တဲ့ Article ဟာ (၂၀၀၀) ခုနှစ်ကတည်းက ရေးသားထားတဲ့ Article တစ်ခုဖြစ်ပေမယ့် ကနေ့အချိန်ထိ ခေတ်မီနေဆဲဖြစ်ပါတယ်။ အောက်ပါလိပ်စာမှာ မူရင်းအတိုင်းလေ့လာနိုင်ပါတယ်။

http://www.joelonsoftware.com/articles/fog0000000043.html

6.1 – Bug Database

Joel Spolsky ရဲ့အချက် (၁၂) ချက်ထဲမှာ နံပါတ် (၄) အချက်က Bug Database ဖြစ်ပါတယ်။ Software တိုင်းမှာ Bug တွေရှိတက်ကြောင်းကို ရှေ့ပိုင်းတွေမှာလည်း ဆွေးနွေးခဲ့ပြီးဖြစ်ပါတယ်။ တစ်ချို့ Bug တွေကို ရေးသားနေစဉ် ကတည်းက Developer ကိုယ်တိုင် သတိပြုတွေ့ရှိနိုင်ပေမယ့်၊ တစ်ချို့ကိုတော့ စမ်းသပ်တဲ့အဆင့် ရောက်တော့မှ Tester တွေက တွေ့ရှိရတက်ပါတယ်။ ဒါတင်မက တစ်ချို့ Bug တွေဆိုရင် အမှားကင်းပြီလို့ Developer တွေရော၊ Tester တွေကပါ လက်ခံယူဆပြီး အများအသုံးပြုနိုင်အောင် Release လုပ်လိုက်ပြီးနောက်မှ တွေ့ရတက်ပါတယ်။ Bug တွေ မရှိချင်လို့တော့မရပါဘူး။ Software မှန်ရင်တော့ Bug တွေ ရှိမှာပဲဖြစ်ပါတယ်။ အဲ့ဒီ Bug တွေကို စနစ်တစ်ကျစီမံနိုင်ဖို့ သာ လိုပါတယ်။

ပထမဆုံးအနေနဲ့ အရေးကြီးတာကတော့ ရှိနေတဲ့ Bug တွေကို သိရှိမှတ်မိဖို့ပဲဖြစ်ပါတယ်။ Developer အများစုက ကိုယ်တွေ့ရှိထားတဲ့ Bug တွေကို အမြဲမှတ်မိမယ်လို့ ယူဆထားတတ်ကြပါတယ်။ လက်တွေ့မှာ မမှတ်မိနိုင်ပါဘူး။ အထူးသဖြင့် စနစ်တစ်ခုလုံး အလုပ်မလုပ်တော့ပဲ ရပ်တန့်သွားလောက်အောင် မဆိုးတဲ့ Minor Bug တွေကို မပြင်ပဲထားတာ အချိန်ကြာ လာတာနဲ့အမျှ မေ့မေ့လျော့လျော့ ဖြစ်သွားတတ်ပါတယ်။ နောင်တစ်ချိန်ကျတော့မှ၊ ပြင်ဖို့ဘာမှမခက်ခဲတဲ့ Bug တစ်ခုကို မေ့ပြီးမပြင်လိုက်မိပဲ Release လုပ်လိုက်မိလို့ နောင်တရတတ်ပါတယ်။ ဒါကြောင့် တွေ့ရှိသမျှ Bug တိုင်းကို မှတ်တမ်း တင်ထားဖို့လိုပါတယ်။

ပြီးတော့ Software တစ်ခုကို တစ်ယောက်ထက်ပိုတဲ့ Developer တွေက ပူးပေါင်းရေးသားကြတဲ့အခါ၊ တစ်ဦးတွေ့ရှိထားတဲ့ Bug ကို ကျန် Developer များကလည်း သိရှိနေဖို့ လိုအပ်ပါတယ်။ ဒါကြောင့် But မှတ်တမ်းကို Developer တိုင်း အချိန်မရွေး ကြည့်ရှုနိုင်ဖို့အတွက် Centralize Database တစ်ခုနဲ့ထားရှိသင့်ပါတယ်။ ဒါ့အပြင် Developer မဟုတ်တဲ့၊ Manager, Tester နဲ့ User တွေ တွေ့ရှိလာတဲ့ Bug တွေလည်း ရှိနိုင်ပါတယ်။ အဲ့ဒီလို တွေ့ရှိလာတဲ့ Bug တွေကို Developer တွေသိရှိနိုင်ဖို့ Bug Database ထဲကို ထည့်သွင်းပေးနိုင်တဲ့ Bug Report လုပ်ဆောင်ချက်မျိုးတွေ Bug Database မှာ ရှိထားသင့်ပါတယ်။

Bug Database တစ်ခုမှာ Bug တွေကို မှတ်တမ်းတင်ဖို့အတွက် အခြေခံ Field (၅) ခု အနည်းဆုံး ပါဝင်သင့်ပါတယ်။

1. **Steps to reproduce** – မှတ်တမ်းတင်ထားတဲ့ Bug ကို ပြန်လည်လေ့လာလိုရင် ဆောင်ရွက်ရမယ့် အဆင့်များ
2. **Expected behavior** – ရရှိမယ်လို့ မျှော်မှန်းထားတဲ့ ရလဒ် (ရလဒ်အမှန်)
3. **Buggy behavior** – အမှန်တစ်ကယ်ရရှိလာတဲ့ ရလဒ် (ရလဒ်အမှား)
4. **Assigned person** – တာဝန်ယူဖြေရှင်းရမယ့်သူ
5. **Status** – တာဝန်ယူဖြေရှင်းပြီးစီးခြင်း ရှိ၊ မရှိ အခြေအနေ

– တို့ပဲဖြစ်ပါတယ်။

Bug တစ်ခုကို မှတ်တမ်းတင်လိုက်တာနဲ့ အဲ့ဒီ Bug ကို ပြည်လည်လေ့လာလိုတဲ့အခါ ဆောင်ရွက်ရမယ့် **Reproducing Steps** ကို တစ်ပါတည်းတွဲဖက် မှတ်တမ်းတင်ဖို့လိုပါတယ်။ ဘယ်လိုအချိန်ဘယ်လိုအခြေအနေမှာ Bug ကို တွေ့ရတာလဲ ဆိုတဲ့အဆင့်တွေကို တွဲဖက်မှတ်တမ်းတင်ထားမှ တာဝန်ယူဖြေရှင်းရမယ့်သူက Bug ကို ပြည်လည်လေ့လာပြီး ဘာကြောင့် ဖြစ်နိုင်သလဲ ဆိုတဲ့ပြဿနာရင်းမြစ်နဲ့ အဖြေကို ရှာဖွေနိုင်မှာဖြစ်ပါတယ်။

တစ်ခါတစ်ရံကျတော့ တစ်ကယ်မှားနေတာမဟုတ်ပဲ မှားတယ်ထင်ရတာမျိုးလည်း ရှိတက်ပါတယ်။ ဒါကြောင့် Bug ကို မှတ်တမ်းတင်တဲ့သူက၊ သူမျှော်လင့်ထားတဲ့ရလဒ် **Expected behavior** နဲ့ သူရရှိတဲ့ **Buggy behavior** တို့ကို တွဲဖက်မှတ်တမ်းတင် ပေးဖို့လိုပါတယ်။ ဥပမာ – Expected အနေနဲ့ "Mobile Phone Category တွင် Mobile Phone စာရင်းကိုသာ ဖော်ပြသင့်သည်" လို့ မှတ်တမ်းတင်ပြီး Bug အနေနဲ့ "Mobile Phone Category တွင် Laptop ကွန်ပျူတာများကိုပါ ဖော်ပြနေသည်" လို့ မှတ်တမ်းတင်နိုင်ပါတယ်။ ဒီတော့မှ တာဝန်ယူဖြေရှင်းရမယ့်သူက တစ်ကယ်ပဲမှားပြီး Laptop ကွန်ပျူတာတွေ Mobile Phone စာရင်းထဲမှာဝင်နေတာလား၊ တမင်

ရည်ရွယ်ချက်ရှိရှိ Related Item အနေနဲ့ Mobile Phone စာရင်းနဲ့အတူ Laptop ကွန်ပျူတာတွေကို တွဲပြထားတာလား ဆုံးဖြတ်ပြီး၊ လိုအပ်သလို ဆက်လက်ဆောင်ရွက်နိုင်မှာဖြစ်ပါတယ်။ ပေးထားတဲ့နမူနာအရဆိုရင် Bug ကို မှတ်တမ်းတင်သူက မှားနေတယ်ထင်လို့ မှတ်တမ်းတင်ပေမယ့် လက်တွေ့မှာ တစ်ကယ့်အမှားမဟုတ်ပဲ တမင်ရည်ရွယ်ထည့်သွင်းထားတဲ့ Feature တစ်ခု ဖြစ်နိုင် ပါတယ်။

Bug တစ်ခုမှတ်တမ်းတင်တာနဲ့ အဲ့ဒီ Bug ကို တာဝန်ယူဖြေရှင်းရမယ့်သူ **Assigned Person** ကို တစ်ခါတည်း တွဲဖက် ထည့်သွင်းသင့်ပါတယ်။ အကယ်၍ Bug ကို မှတ်တမ်းတင်တဲ့သူက ဘယ်သူတာဝန်ယူရမလဲ ဆုံးဖြတ်နိုင်တဲ့သူ မဟုတ်ရင် လည်း၊ ဆုံးဖြတ်နိုင်သူဖြစ်တဲ့ Team Leader ကို Assigned Person အဖြစ် တွဲဖက် ထည့်သွင်းသင့်ပါတယ်။ နောက်တော့ မှ Team Leader က အမှန်တစ်ကယ် တာဝန်ယူရမယ့် Developer ကို လွှဲပြောင်းပြီး တာဝန်ပေးသွားပါလိမ့်မယ်။ ဘယ်သူကိုမှ Assigned လုပ်ထားခြင်းမရှိတဲ့ Bug တွေတော့ စာရင်းထဲမှာ မရှိသင့်ပါဘူး။ အနဲဆုံး Team Leader ကိုတော့ Assigned လုပ်ထားပေးသင့်ပါတယ်။

Bug တစ်ခုကို မှတ်တမ်းတင်လိုက်တာနဲ့၊ အဲ့ဒီ Bug ကို စပြင်နေပြီလား၊ မပြင်သေးဘူးလား၊ ပြင်ပြီးသွားပြီလား စတဲ့ အခြေအနေ **Status** အမျိုးမျိုးရှိနိုင်ပါတယ်။ အဲ့ဒီ အခြေအနေကို Bug နဲ့အတူ အမြဲတွဲဖတ် မှတ်သားထားဖို့ လိုပါတယ်။ ဒီတော့မှ ပြင်ပြီးကြောင်း မှတ်တမ်းတင်ထားတဲ့ Bug တွေဟာ တစ်ကယ်လိုချင်တဲ့ Behavior ရရှိအောင် ပြင်ဆင်ပြီးပြီ လား ပြန်လည်စမ်းစစ်နိုင်မှာဖြစ်ပါတယ်။ တစ်ချို့ တစ်ကယ့်အမှားမဟုတ်လို့ မပြင်ဖို့ ဆုံးဖြတ်တယ်ဆိုရင်လည်း မူလထည့်သွင်းသူက Status ကိုကြည့်ပြီး ဒါဟာ ရည်ရွယ်ချက်ရှိရှိနဲ့ တမင် ထည့်သွင်းထားတဲ့ Feature မို့ မပြင်ခြင်းဖြစ် ကြောင်းကို သိရှိနိုင်မှာဖြစ်ပါတယ်။

ဒီအခြေခံအချက် (၅) ချက်ကတော့ အခြေခံအနေနဲ့ Bug Database တစ်ခုမှာ မဖြစ်မနေပါဝင်သင့်တဲ့ အချက်တွေဖြစ် ပါတယ်။ ပိုပြီးပြည့်စုံလိုတယ်ဆိုရင် Bug မှတ်တမ်းနဲ့ အတူ Screenshot တွေ တွဲဖက်မှတ်တမ်းတင်နိုင်ပါတယ်။ အကယ်၍ အလားတူ Bug ကို ယခင်ကရှိခဲ့ဖူးတယ်ဆိုရင်၊ ယခင် Bug ကိုညွှန်းထားတဲ့အညွှန်းကို တွဲဖက်မှတ်တမ်းတင်နိုင်ပါ တယ်။ Bug တွေရစဉ်မှာ အသုံးပြုနေတဲ့ OS Version, Browser Version နဲ့ အခြားဆက်စပ် Software တွေနဲ့ Version တွေကို တွဲဖက်မှတ်တမ်းတင်မယ်ဆိုရင် ပြင်ဆင်ရမယ့် Developer အတွက် အထောက်အကူ ဖြစ်နိုင်ပါတယ်။ Bug တွေကို အမျိုးအစားအလိုက် Category ခွဲပြီးတော့လည်း မှတ်တမ်းတင်ထားနိုင်ပါတယ်။ Bug ရဲ့အရေးပါမှုအဆင့် Priority ကိုလည်း တွဲဖက်မှတ်တမ်းတင်နိုင်ပါတယ်။

Bug Database အနေနဲ့ နှစ်သက်ရာ Tool ကို သုံးနိုင်ပါတယ်။ Project ကသေးပြီး တစ်ဦးတည်းရေးသားခြင်းဖြစ်လို့ ရိုးရိုး Text File တစ်ခုနဲ့ မှတ်တမ်းတင်ထားမယ်ဆိုလည်း ထားနိုင်ပါတယ်။ ပိုပြီးစနစ်ကျအောင် Spreadsheet တစ်ခုနဲ့ Column တွေခွဲပြီး အသေအချာ Color-Code လုပ်ထားမယ်ဆို လည်းထားနိုင်ပါတယ်။ Bugzilla, Trac, RedMine, MantisBT စတဲ့ အသင့်သုံးလို့ရတဲ့ Bug Tracking System တွေ သုံးမယ်ဆိုလည်း သုံးနိုင်ပါတယ်။ Bug Database ထားရှိခြင်းရဲ့ အရေးပါမှုကို သိရှိဖို့ကသာ လိုရင်းဖြစ်ပြီး လက်တွေ့သုံးတဲ့အခါ နှစ်သက်ရာ Tool ကိုသုံးနိုင်ပါတယ်။

Bug Reporting Best Practices

Bug Database တစ်ခုနဲ့ Bug တွေမှတ်တမ်းတင်ရာမှာ အခြေခံစည်းမျဉ်းလေးတစ်ချို့ သတ်မှတ်ထားဖို့လိုပါတယ်။ ဥပမာ - "Reproducing Steps တွေမဖြစ်မနေထည့်သွင်းရမယ်" ပြီးတော့ "Bug မှတ်တမ်း အသစ်တစ်ခုမထည့်သွင်း မီ အလားတူ Bug ကို ယခင်ကထည့်သွင်းခဲ့ဖူးခြင်းရှိမရှိ သေချာအောင်အရင်ကြည့်ရမယ်" စသဖြင့် အခြေခံ စည်းမျဉ်း လေးတစ်ချို့ သတ်မှတ်ထားရပါတယ်။ ဒီလိုသတ်မှတ်မထားရင် Bug Database ထဲမှာ အမှန်တစ်ကယ်အသုံးမဝင်တဲ့ Bug Report တွေနဲ့ ပြည့်နှက်ပြီး စီမံရ ခက်ခဲသွားတက်ပါတယ်။ အသေးစိတ်စည်းမျဉ်းတွေ ဘယ်လိုသတ်မှတ်မလဲဆိုတာ အသုံးပြုမယ့် Team ရဲ့ ဖွဲ့စည်းပုံနဲ့ Project အမျိုးအစားပေါ်မှာ မူတည်ပါတယ်။ ဒီနေရာမှာတော့ လိုက်နာသင့်တဲ့ အခြေခံ စည်းမျဉ်းတစ်ချို့ကို ရွေးထုတ် ဖော်ပြလိုက်ပါတယ်။

၁။ Bug တစ်ခုကို ဖြေရှင်းပြီးတဲ့အခါ ဖြေရှင်းပြီးကြောင်း မှတ်တမ်းတင်ရပါတယ်။ အဲ့ဒီလိုမှတ်တမ်းတင်ရာမှာ အမှန်တစ်ကယ် ဖြေရှင်းလိုက်လို့ ပြေလည်သွားတာဆိုရင် Status ကို **Fixed** လို့ တွဲဖက်မှတ်တမ်းတင်သင့်ပါတယ်။

၂။ အမှန်တစ်ကယ် Software မှာ ရှိနေတဲ့ပြဿနာကြောင့်မဟုတ်ပဲ အခြားပြဿနာကြောင့်ဖြစ်နေတဲ့အတွက် ဖြေရှင်းလို့ မရနိုင်ရင် (ဥပမာ – Hardware ကြောင့်ဖြစ်တဲ့ ပြဿနာ၊ အခြား Software တစ်ခုကြောင့် ဖြစ်တဲ့ ပြဿနာ) Status ကို **Won't Fix** လို့ တွဲဖက်မှတ်တမ်းတင်ထားသင့်ပါတယ်။

၃။ Bug တစ်ခုပါလို့ မှတ်တမ်းတင်ထားပေမယ့် Reproduction Steps တွေမပါရင် (သို့မဟုတ်) စမ်းကြည့်တော့ အဲ့ဒီ ပြဿနာတစ်ကယ်ရှိမနေရင် Status ကို **Not Repro** လို့ တွဲဖက်မှတ်တမ်းတင်သင့်ပါတယ်။ သတိပြုရမှာက လုံးဝ Reproduce လုပ်လို့မရမှ ဒီလိုသတ်မှတ်ရမှာပါ။ တစ်ခါတစ်ရံ Reproduce လုပ်လို့ရပြီး၊ တစ်ခါတစ်ရံ မရဘူးဆိုတဲ့ Bug မျိုးကလည်း ရှိတက်ပါသေးတယ်။ ရှာဖွေပြင်ဆင်ရ အခက်ခဲဆုံး Bug အမျိုးအစားဖြစ်ပါတယ်။

၄။ အလားတူ Bug ကိုပဲ တစ်ကြိမ်ထက်ပိုပြီး ထည့်သွင်းထားမိခြင်း ဖြစ်နေရင်တော့ ထပ်နေတဲ့ Bug ရဲ့ Status ကို **Duplicate** လို့ တွဲဖက်မှတ်တမ်းတင်သင့်ပါတယ်။

၅။ Bug မဟုတ်ပဲ၊ Feature ကို Bug လို့ယူဆပြီး မှားယွင်းထည့်သွင်းထားမိတဲ့ Bug တွေရဲ့ Status ကိုတော့ **By Design** လို့ တွဲဖက်မှတ်တမ်းတင်သင့်ပါတယ်။

၆။ Bug တစ်ခုအတွက် ဖြေရှင်းပြီးကြောင်း Status ကိုလက်ခံရရှိပြီဆိုရင် အမှန်တစ်ကယ်ဖြေရှင်းပြီးခြင်း ဟုတ်မဟုတ် စီစစ်ပြီး၊ အမှန်တစ်ကယ် ပြေလည်သွားတာဆိုရင် အပြီးပိတ်သိမ်းလိုက်ရပါတယ်။ သတိပြုရမှာ၊ အဲ့ဒီလို ပိတ်သိမ်းခွင့်ကို မူလ Bug မှတ်တမ်း ထည့်သွင်းခဲ့သူကိုပဲ ပိတ်သိမ်းခွင့်ပြုသင့်ပါတယ်။ မူလမှတ်တမ်းတင်ခဲ့သူမှသာ သူမှတ်တမ်းတင်ခဲ့တဲ့ Bug အမှန်တစ်ကယ် ပြေလည်သွားခြင်းရှိမရှိကို ဆုံးဖြတ်နိုင်မှာဖြစ်ပါတယ်။

၇။ Software ရဲ့ Version ကို သတိပြုရပါမယ်။ ဥပမာ – Version 0.1.9 မှာ Bug တွေ့လို့ မှတ်တမ်းတင်လိုက်တယ်။ ဖြေရှင်းပြီးသွားလို့ ပိတ်သိမ်းလိုက်တယ်ဆိုပါစို့။ Bug Database ထဲမှာ ဖြေရှင်းပြီးကြောင်း မှတ်တမ်းတင်ပိတ်သိမ်းထား ပြီးဖြစ်လို့ဆိုပြီး Version 0.1.10 မှာ ထပ်မစမ်းမိရင် ပြဿနာဖြစ်နိုင်ပါတယ်။ ဒါကြောင့်

Version တွေကို စနစ်တစ်ကျ စီမံပြီး Bug Report တွေ မှတ်တမ်းတင်ရာမှာလည်း ဘယ် Version အတွက်လည်း ဆိုတာကို တွဲဖက်မှတ်တမ်းတင်ထား သင့်ပါတယ်။

၈။ Bug Database ရှိနေရက်နဲ့ ပြဿနာတစ်ခုတွေ့တဲ့အခါ Developer ထံ Email သို့မဟုတ် အခြားနည်းလမ်းနဲ့ အဲ့ဒီ Bug အကြောင်းပေးပို့လာရင် လက်မခံသင့်ပါဘူး။ Developer တွေအနေနဲ့ Bug Database ထဲက Bug က လွဲရင် အခြား နည်းလမ်းနဲ့ပေးပို့လာတဲ့ Bug တွေကို တာဝန်ယူဆောင်ရွက်မှာ မဟုတ်ဘူးလို့ ပြတ်ပြတ်သားသား သတ်မှတ်ထားသင့်ပါတယ်။ ဒီတော့မှာ Team မှာပါဝင်သူအားလုံး Bug Database ကို အသုံးပြုဖို့ တွန်းအားတစ်ခု ဖြစ်စေသလို၊ Database ထဲမရောက်လို့ မှတ်တမ်းပျောက်သွားပြီး မပြင်မိပဲ ကျန်သွားတယ်ဆိုတဲ့ Bug တွေလည်း မရှိတော့မှာဖြစ်ပါတယ်။

၉။ ကိုယ်တိုင် Bug တစ်ခုတွေ့လာတဲ့အခါမှာလည်း အဲ့ဒီ Bug အကြောင်း သက်ဆိုင်ရာ Developer ကို တိုက်ရိုက် မပြောပဲ၊ Bug Database ထဲကိုသာ မှတ်တမ်းအဖြစ် ထည့်သွင်းလိုက်သင့်ပါတယ်။ ဒီတော့မှ အခြား Developer တွေအနေနဲ့ရှိနေတဲ့ Bug တွေအကြောင်းသိချင်ရင် Bug Database မှာ အမြဲကြည့်ရမယ်ဆိုတဲ့ တွန်းအားကို ဖြစ်စေမှာဖြစ်ပါတယ်။

၁၀။ Bug တစ်ခုကို တာဝန်ယူဖြေရှင်းဖို့ Developer တစ်ယောက်ကို Assign လုပ်လိုက်တိုင်းမှာ၊ Assign လုပ်ခံရသူထံ Email သို့မဟုတ် နည်းလမ်းတစ်မျိုးမျိုးနဲ့ အလိုအလျောက် Notify လုပ်ဖို့ စီစဉ်ထားသင့်ပါတယ်။ ဒီနည်းနဲ့ မူလက Bug Database ကို ထိရောက်အောင် မသုံးဖြစ်တဲ့ Developer တွေရှိရင် အသုံးပြုဖို့ တွန်းအားဖြစ်စေမှာဖြစ်ပါတယ်။ အသင့် သုံး Bug Tracking System တွေမှာတော့ ဒီလိုလုပ်ဆောင်ချက်မျိုး အသင့်ပါပြီးဖြစ်ပါတယ်။

၁၁။ Bug Database ကို Bug Report အတွက်သာမက Feature Request အတွက်ပါအသုံးပြုနိုင်ပါတယ်။ Project Manager တစ်ယောက်အနေနဲ့ Software မှာ လုပ်ဆောင်ချက်အသစ် ထည့်သွင်းစေလိုတဲ့အခါ (သို့မဟုတ်) ရှိနေတဲ့ လုပ်ဆောင်ချက်ကို ပြင်ဆင်စေလိုတဲ့အခါ၊ Bug Database ထဲကိုထည့်သွင်းပြီး တာဝန်ပေးလိုတဲ့ Developer ကို Assign လုပ်နိုင်ပါတယ်။ ဒီနည်းနဲ့ Bug Database ဆိုတာ Bug မှတ်တမ်းသက်သက်မဟုတ်တော့ပဲ Developer တွေ နေ့စဉ် လက်စွဲထားအလုပ်လုပ်ရတဲ့ Job Assignment List တစ်ခု ဖြစ်သွားမှာဖြစ်ပါတယ်။

6.2 – Bug Database vs. Issue Tracking System

Bug Tracking System (Bug Database) နဲ့ Issue Tracking System ဆိုတာ သဘောတရားအရ အတူတူပါပဲ။ အခြေခံအားဖြင့်ပြောရရင် Bug Database က Bug တွေ မှတ်တမ်းတင်ဖို့အတွက်သုံးပြီး Issue Tracking System ကိုတော့ Bug အပါအဝင် Software မှာရှိနေတဲ့ အထွေထွေပြဿနာ Issue တွေကို မှတ်တမ်းတင်ဖို့ သုံးခြင်း ဖြစ်ပါတယ်။ Issue Tracking System တွေက Bug Database ထက် လုပ်ဆောင်ချက်တွေ ပိုမိုစုံလင်ပြီး ပိုမိုကျယ်ပြန့်တက် ပါတယ်။

Bug Database ကို Software Team မှာပါဝင်တဲ့ Project Manager, Developer နဲ့ Tester တွေက အဓိကအသုံးပြု ပေမယ့် Issue Tracking System ကိုတော့ အသုံးပြုသူ User တွေ Management Level အရာရှိတွေ၊

Marketing Staff တွေနဲ့ လုပ်ငန်းတစ်ခုလုံးမှာ ပါဝင်သူတွေက အသုံးပြုနိုင်ဖို့ ရည်ရွယ်ဖန်တီးထားတက်ကြပါတယ်။ ဒီလိုရည်ရွယ်ချက် ကျယ်ပြန့်သွားပေမယ့် အလုပ်လုပ်ပုံသဘောတရားကတော့ Bug Database နဲ့အတူတူပါပဲ။ ရှိနေတဲ့ ပြဿနာ Issue တစ်ခုကို စနစ်မှာမှတ်တမ်းတင်ပြီး၊ အဲ့ဒီပြဿနာကို သက်ဆိုင်ရာ တာဝန်ရှိသူတစ်ဦးကို Assign လုပ်ခြင်း၊ တာဝန်ရှိသူက လိုသလိုဆောင် ရွက်ပြီးတဲ့အခါ ဖြေရှင်းပြီးကြောင်း မှတ်တမ်းတင်ခြင်း၊ အမှန်တစ်ကယ်ဖြေရှင်းပြီးကြောင်း သေချာတဲ့အခါ ပိတ်သိမ်းခြင်း လုပ်ငန်းတွေကို ဆောင်ရွက်နိုင်တဲ့ စနစ်ပဲဖြစ်လို့ Issue Tracking System နဲ့ Bug Database ဆိုတာ အခြေခံအားဖြင့် အတူတူပဲဖြစ်တယ်လို့ ဆိုနိုင်ခြင်း ဖြစ်ပါတယ်။

Issue Tracking System ကို Ticket Management System (သို့မဟုတ်) Customer Relationship Management (CRM) System လို့လည်း ခေါ်တက်ကြပါသေးတယ်။ Software အပါအဝင် ဝန်ဆောင်မှုလုပ်ငန်းတွေမှာ၊ အသုံးပြုသူ User နဲ့ Customer ဘက်က အကြောင်းကြားလာတဲ့ သူတို့ရဲ့အခက်အခဲပြဿနာ Issue တွေကို လက်ခံထည့်သွင်းခြင်း၊ သက်ဆိုင်ရာ Technician ကို ဖြေရှင်းဖို့တာဝန်ပေးခြင်း၊ ဖြေရှင်းဆောင်ရွက်ပြီးစီးတဲ့အခါ ပိတ်သိမ်းခြင်း စသဖြင့် အခေါ် အဝေါ် ကွဲသွားပေမယ့် အခြေခံအလုပ်လုပ်ပုံ Workflow ဆင်တူတဲ့ စနစ်တွေဖြစ်ပါတယ်။

Software Project တစ်ခုကို စီမံတဲ့နေရာမှာ Bug တွေသာမက Feature တွေနဲ့ Customer Support Request တွေ ကိုပါ စီမံဖို့လိုအပ်တက်လို့ ရိုးရိုး Bug Database တစ်ခုထက် ပိုမိုပြည့်စုံတဲ့ Issue Tracking System တွေကို အသုံးပြု ခြင်းအားဖြင့် ပိုမိုထိရောက်အကျိုးရှိစေနိုင်ပါတယ်။ တွေ့ရလေ့ရှိတဲ့ ပြဿနာလေးတစ်ခုကို နမူနာအနေနဲ့ ဖော်ပြပေး လိုက်ပါတယ်။

> **Project Manager** – "Search Result ကို Bookmark လုပ်ထားလို့ရတဲ့ လုပ်ဆောင်ချက်က ဘာလို့ မပါသေးတာလဲ။ ပြီးခဲ့တဲ့ Meeting မှာ ဒီလုပ်ဆောင်ချက်ကို ဦးစားပေးထည့်ဖို့ ပြောထားပြီးသား မဟုတ်လား။"
>
> **Team Leader** – "လုပ်ဆောင်ချက်အသစ်တစ်ခု ရှိလာတိုင်း သက်ဆိုင်ရာ Developer တွေကို ချက်ခြင်းတာဝန် သတ်မှတ်ပေးထားပြီးသားပါ။ Search Result Bookmark က ဘယ်သူ့တာဝန်လဲ။ "
>
> **Developer** – "ကျွန်တော်ပါခင်ဗျ။ လက်ရှိလုပ်လက်စ Newsletter လုပ်ဆောင်ချက်လေး မပြတ် သေးလို့ Search Result Bookmark ဘက်ကို မလှည့်နိုင်သေးတာပါ။"
>
> **Project Manager** – "Newsletter က Minor Feature ပဲ။ သိပ်အရေးမကြီးဘူးလေ။ ပိုပြီးအဓိက ကျတဲ့ Search Result Bookmark ကို အရင်လုပ်ရမှာပေါ့။ Team Leader က ဘယ်ဟာ Minor, ဘယ်ဟာ Major သေချာရှင်းပြမထားဘူးလား။"
>
> **Team Leader** – "Major, Minor ခွဲထားပြီးသား Feature List ကို Email နဲ့ Developer အကုန်လုံးကိုပို့ထားပြီးသားပဲ။"

> **Developer** – "ကျွန်တော် အဲ့ဒီ Email မရပါလား။"
>
> **Team Leader** – "ဘာလို့မရရမှာလဲ၊ အသေအချာပို့လိုက်တာပဲ။"
>
> **Developer** – "မရပါဘူး။ သေချာအောင် နောက်တစ်ခေါက်ထပ်ကြည့်လိုက်ဦးမယ်။ ဩော်... ဖြစ်ရ မယ်၊ Email က Spam Folder ထဲ ရောက်နေတာကိုး။"
>
> **Project Manager** – "မင်းတို့လွဲကြတာ အရေးမကြီးဘူး။ ငါက Management ပိုင်းက လူကြီး တွေ ကို၊ ဒီလုပ်ဆောင်ချက် Demo လုပ်မလို့ Meeting ချိန်းထားပြီးသား။ အခုတော့ ပြစရာမရှိ ဘူး၊ ပြဿနာတက်ပြီ။ နောင်ကို သေချာအောင်လုပ်ကြကွာ။"

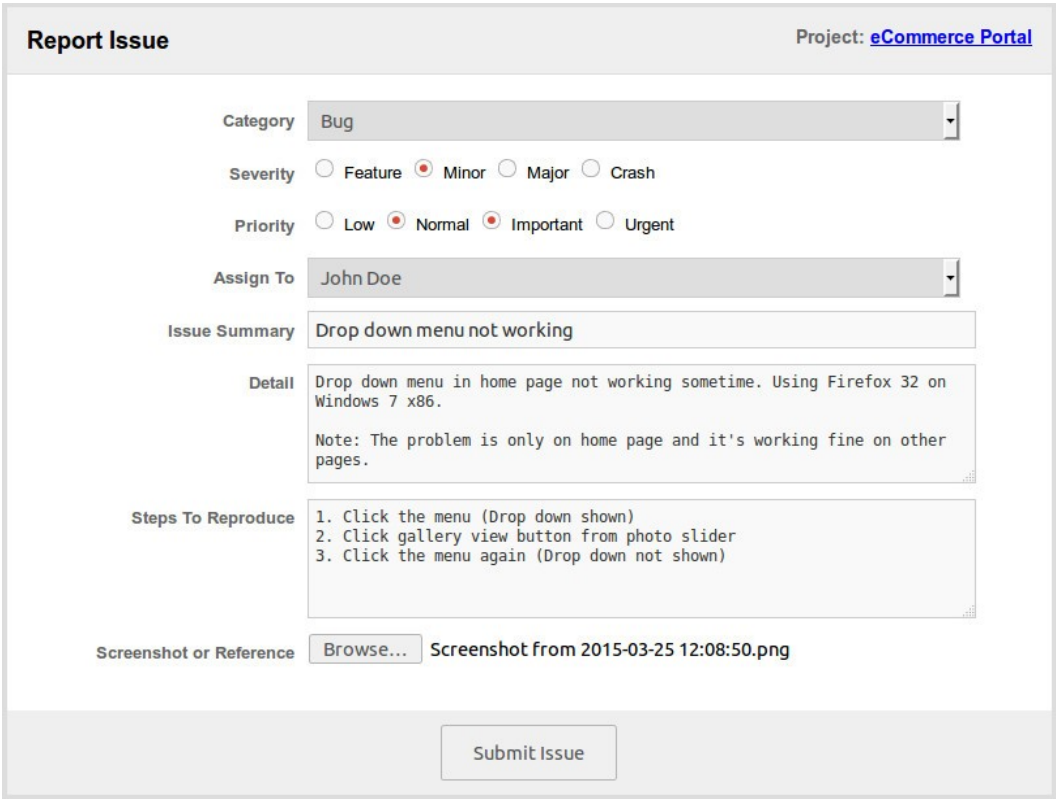
ဒါဟာ Software Project တွေမှာ တွေ့ရလေ့ရှိတဲ့ ပြဿနာတစ်ခုပဲဖြစ်ပါတယ်။ ဒီစာအုပ်မှာ အကြိမ်ကြိမ်ပြော နေတဲ့ အချက်တစ်ချက်ရှိပါတယ်။ Specification ဆိုတာ တိကျဖို့ခက်ခဲပြီး အမြဲပြောင်းလဲနေတက်တဲ့ သဘောရှိ ပါတယ်။ Specification ပြောင်းလဲတယ်ဆိုတာ Software ရဲ့ အလုပ်လုပ်ပုံနဲ့ ပါဝင်ရမယ့် Feature တွေ ပြောင်းလဲသွားခြင်းပဲ ဖြစ်ပါတယ်။ ဒီလို အမြဲပြောင်းလဲနေတက်တဲ့ Feature တွေကို စီမံဖို့အတွက် Issue Tracking System တစ်ခုကို အသုံးပြုမှသာ ထိရောက်မှုရှိမှာဖြစ်ပါတယ်။

6.3 – Standard Issue Tracking System Features

Issuer Tracking System တွေမှာ အခြေခံအနေနဲ့ အောက်ပါလုပ်ဆောင်ချက်တွေ ပါဝင်လေ့ရှိပါတယ်။

User and Role Management – Issuer Tracking System တစ်ခုကို Developer တွေ Tester တွေသာမက၊ Manager တွေ Marketing Staff တွေနဲ့ Customer တွေအပါအဝင် အဖွဲ့အစည်းတစ်ခုလုံးမှာ ရှိတဲ့သူအားလုံး အသုံးပြု နိုင်ပါတယ်။ အဲ့ဒီလို အသုံးပြုရာမှာ အရေးကြီးတာကတော့ Role Management ဖြစ်ပါတယ်။ ဥပမာ – Lead Developer မှသာ Issue တွေကို Assign လုပ်လို့ရမယ်၊ Customer တွေက Read Only Access ပဲရမယ်၊ QA Engineer မှသာ Issues တွေကို ပိတ်သိမ်းလို့ရမယ် စသဖြင့် Role တွေ အမျိုးမျိုး ခွဲခြားစီမံနိုင်ဖို့လိုပါတယ်။ ဘယ် User ကို ဘယ်လို Role ပေးမလဲဆိုတာက သက်ဆိုင်ရာ အဖွဲ့အစည်းရဲ့ Policy ပေါ်မူတည်ပါတယ်။ ဒါ ကြောင့် Issuer Tracking System တစ်ခုဟာ အဲ့ဒီလို User တွေနဲ့ Role တွေကို လွယ်လွယ်လပ်လပ် စီမံလို့ရနိုင်တဲ့ လုပ်ဆောင်ချက်တွေ ပါဝင်ရပါတယ်။

Project Management – Issue Tracking System ကို Project တစ်ခုတည်းအတွက်သုံးတာဖြစ်နိုင်သလို Project နှစ်ခုသုံးခုအတွက် Issue Tracking System တစ်ခုတည်းသုံးတာလည်း ဖြစ်နိုင်ပါတယ်။ ဒါကြောင့် Project တွေ ခွဲခြား စီမံလို့ရတဲ့ လုပ်ဆောင်ချက်ဟာလည်း Issue Tracking System တစ်ခုမှာ မဖြစ်မနေ ပါဝင်ရ မယ့် လုပ်ဆောင်ချက်ပဲ ဖြစ်ပါတယ်။



ပုံ (၆.၁) - Example Issue Report Form

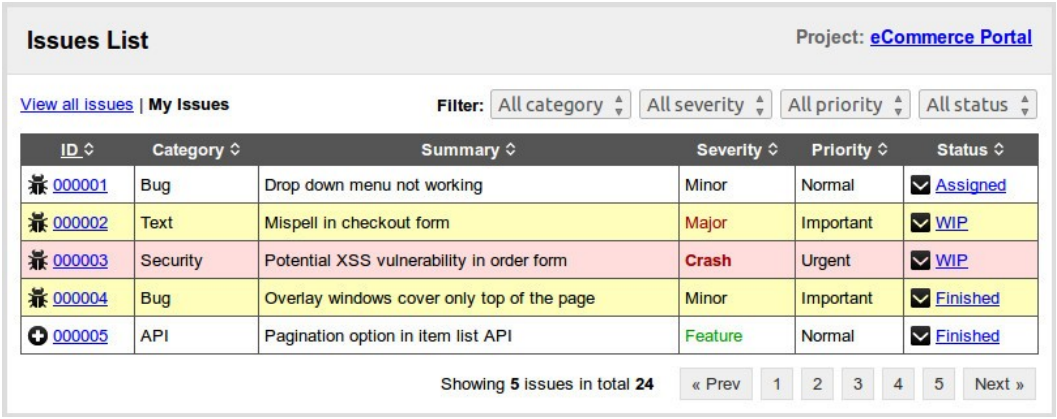
Issue Management – ဒါကတော့ အဓိကလုပ်ဆောင်ချက်ပါ။ Issue Tracking System တစ်ခုဟာ Bug Report နဲ့ Feature Request တွေအပါအဝင် Issue တွေကို Priority အလိုက် မှတ်တမ်းတင်လို့ရတဲ့ လုပ်ဆောင်ချက် ပါဝင်ရပါမယ်။ သက်ဆိုင်ရာတာဝန်ရှိသူကို Assign လုပ်လို့ရတဲ့ လုပ်ဆောင်ချက်တွေ ပါဝင်ရမယ်။ **ပုံ (၆.၁)** မှာ ပေးထားတဲ့ Issue Report Form နမူနာကို လေ့လာကြည့်ပါ။ အပေါ်နားမှာ လက်ရှိ အလုပ်လုပ်နေတဲ့ Project ကိုဖော်ပြထားပါတယ်။ အဓိပ္ပါယ်က Issue တွေကို သက်ဆိုင်ရာ Project အလိုက် ခွဲခြားစီမံနိုင်တဲ့သဘောပဲဖြစ်ပါတယ်။

Form မှာပါဝင်တဲ့ Field တွေကိုလေ့လာကြည့်မယ်ဆိုရင် ပထမဦးဆုံး အနေနဲ့ Issue Category ကို ရွေးခိုင်းထားပါတယ်။ UI Bug, Text Typo, API Bug, Feature Request စသဖြင့် ထည့်သွင်းလိုတဲ့ Issue အမျိုးအစားကို ရွေးချယ်ရခြင်းဖြစ်ပါတယ်။ ဒါကြောင့် အဲ့ဒီ Issue Category တွေ စီမံလို့ရတဲ့ **Category Management** လုပ်ဆောင်ချက်ဟာလည်း Issue Tracking စနစ်တစ်ခုမှာပါဝင်ရမယ့် အခြေခံလုပ်ဆောင်ချက် တစ်ခုပဲဖြစ်ပါတယ်။

ဆက်လက်ပြီး Issue နဲ့ Severity Level နဲ့ Priority Level တို့ကို ရွေးချယ်ရပါတယ်။ ဒီတော့မှာ နောင် Issue တွေကို ဖြေရှင်းတဲ့အခါ အရေးပါမှုအလိုက် ဦးစားပေးရွေးချယ်ဖြေရှင်းနိုင်မှာဖြစ်ပါတယ်။ တစ်လက်စတည်း ဒီ Issue ကို တာဝန် ယူဖြေရှင်းရမယ့်သူကိုလည်း တွဲဖက် Assign လုပ်ရပါတယ်။ ပြီးတော့မှ Issue ခေါင်းစဉ်၊ အကြောင်းအရာနဲ့ Reproduction Steps တို့ကို ဖြည့် သွင်းရပါတယ်။ နောက်ဆုံးအနေနဲ့ Screenshot သို့မဟုတ်

ရည်ညွှန်းကိုးကားသင့်တဲ့ ဖိုင်တစ်ခုခုရှိရင် တွဲဖက်ပေးရပါတယ်။

ပုံ (၆.၂) ကိုလေ့လာကြည့်ရင်တော့ မှတ်တမ်းတင်ထားတဲ့ Issue စာရင်းကို တွေ့ရမှာဖြစ်ပါတယ်။ Issue ရဲ့ Severity နဲ့ Priority ပေါ်မူတည်ပြီး ဦးစားပေး Issue ကို ဦးစားပေးဖြစ်ကြောင်း မြင်သာအောင် ဖော်ပြထားပါတယ်။ Category အလိုက်သော်လည်းကောင်း၊ Severity Level အလိုက်သော်လည်းကောင်း၊ Status အလိုက်သော်လည်းကောင်း Sorting စီကြည့်မယ်ဆိုရင် စီနိုင်ပါတယ်။ ပြီးတော့ သက်ဆိုင်ရာ Category တစ်ခုပဲကြည့်မယ်၊ သို့မဟုတ် Status တစ်မျိုးကိုပဲ ကြည့်မယ်စသဖြင့် Filter လုပ်နိုင်ပါသေးတယ်။ ဒါတွေဟာ Issue Management အနေနဲ့ ပါဝင်ရမယ့် အခြေခံလုပ် ဆောင်ချက်တွေပဲဖြစ်ပါတယ်။



Issues List — Project: eCommerce Portal

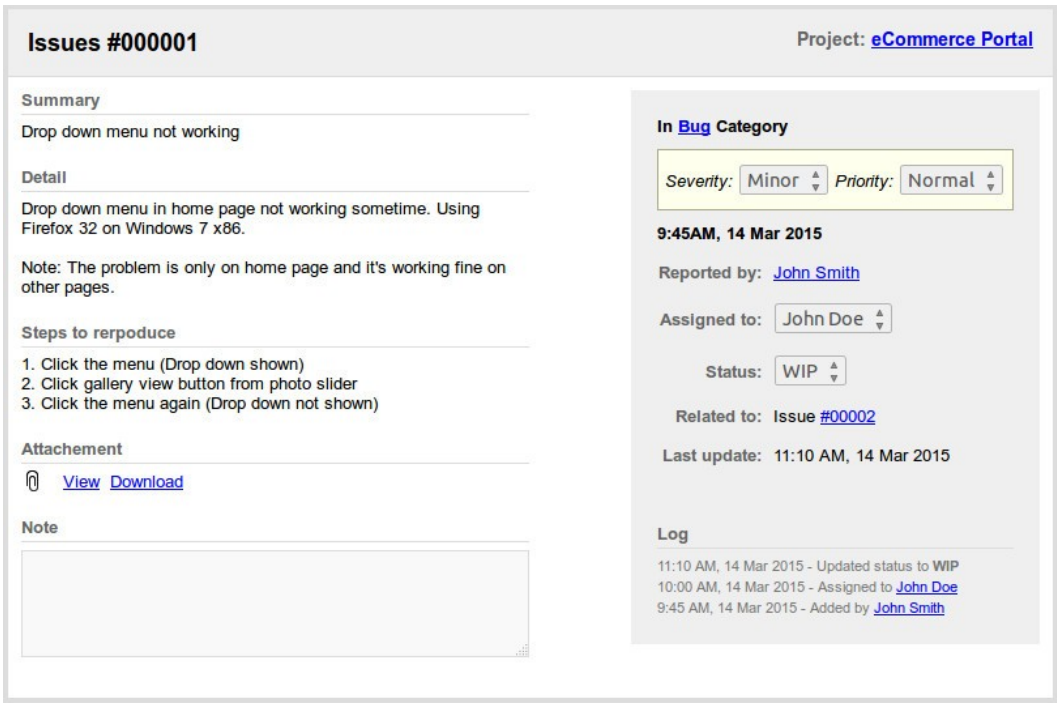
View all issues | **My Issues** — **Filter:** All category | All severity | All priority | All status

ID	Category	Summary	Severity	Priority	Status
000001	Bug	Drop down menu not working	Minor	Normal	Assigned
000002	Text	Mispell in checkout form	Major	Important	WIP
000003	Security	Potential XSS vulnerability in order form	Crash	Urgent	WIP
000004	Bug	Overlay windows cover only top of the page	Minor	Important	Finished
000005	API	Pagination option in item list API	Feature	Normal	Finished

Showing **5** issues in total **24** — « Prev 1 2 3 4 5 Next »

ပုံ (၆.၂) - Example Issues List

Log & Notification – Issue တစ်ခုကို ထည့်သွင်းလိုက်တာနဲ့ ဘယ်သူထည့်သွင်းလိုက်တာလဲ၊ ဘယ်သူ Assign လုပ်လိုက်တာလဲ၊ ဆောင်ရွက်ပြီးစီးကြောင်း မှတ်တမ်းတင်တာဘယ်သူလဲ၊ ဘယ်သူပိတ်သိမ်းလိုက်တာလဲ၊ စတဲ့ Log ကို တွဲဖက်မှတ်တမ်း တင်နိုင်ဖို့လိုပါတယ်။ ဒါအပြင် Assign လုပ်လိုက်တဲ့အခါ Assign လုပ်ခြင်းခံရသူထံ **Notify** လုပ်ပေးဖို့ လိုသလို၊ အဲ့ဒီ Issue နဲ့ ပက်သက်သူ အခြားသူများကထံ အပြောင်းအလဲတစ်ခုဖြစ်တိုင်း Notify လုပ်ပေးနိုင်ဖို့လည်းလို အပ်ပါတယ်။



ပုံ (၆.၃) - Example Issue Detail

ပုံ (၆.၃) မှာလေ့လာကြည့်ရင် Issue တစ်ခုရဲ့ Detail View နမူနာကို တွေ့နိုင်ပါတယ်။ Issue ရဲ့ အသေးစိတ် အချက် အလက်များနဲ့အတူ Log ကိုလည်း တွဲဖက်ဖော်ပြထားတာကို တွေ့မြင်နိုင်ပါတယ်။

ဒါတွေဟာ Issue Tracking System တစ်ခုမှာ မဖြစ်မနေပါဝင်သင့်တဲ့ အခြေခံလုပ်ဆောင်ချက်တွေဖြစ်ပါတယ်။ ဒီထက် ပိုပြီး ပြည့်စုံတဲ့ System တွေမှာဆိုရင်တော့ အောက်ပါလုပ်ဆောင်ချက်များ ဖြည့်စွက်ပါဝင်တက်ပါသေးတယ်။

VCS Integration – Issue Tracking System ကို Version Control System နဲ့တွဲဖက်ထားနိုင်တဲ့ လုပ်ဆောင်ချက် ပါဝင်တက်ပါတယ်။ ဒီနည်းနဲ့ Issue နဲ့ သက်ဆိုင်ရာ Source Code ကို တိုက်ရိုက်တွဲဖက်ပေးထားနိုင်ခြင်း၊ Issue ပိတ် သိမ်းချိန်မှာ ဖြေရှင်းချက် Commit ID ကို တွဲဖက်မှတ်တမ်းတင်နိုင်ခြင်းစတဲ့ ဖြည့်စွက်လုပ်ဆောင်ချက်တွေကို ရရှိနိုင် ပါတယ်။

Duplicate Bug Detection – Bug Report တွေမှာဖြစ်တက်တဲ့ပြဿနာက၊ Bug တစ်ခုကို တစ်ဦးကတွေ့လို့ မှတ်တမ်း တင်ထားပြီးခြင်းကိုမသိပဲ၊ နောက်တစ်ဦးက ထပ်မံမှတ်တမ်းတင်မိတဲ့အတွက် Duplicate တွေ ဖြစ်တက်ပါတယ်။ တစ်ချို့ Issue Tracking System တွေက Duplicate Bug တွေကို အလိုအလျောက် Detect လုပ်ပေးနိုင်တဲ့ လုပ်ဆောင် ချက်တွေ ပါဝင်တက်ပါတယ်။

Custom Fields Management – Bug Database တစ်ခုမှာ အခြေခံအဖြစ် ပါဝင်သင့်တဲ့အချက် (၅) ချက် ရှိကြောင်း ပြောခဲ့ပြီး ဖြစ်ပါတယ်။ Project တစ်ခုနဲ့တစ်ခုပါဝင်သင့်တဲ့ Field တွေ ကွဲပြားနိုင်ပါတယ်။ ဒါကြောင့်

Issue မှတ်တမ်း တင်ရာမှာ ဘယ်လိုအချက်တွေပါဝင်သင့်သလဲဆိုတဲ့ကိစ္စကို ပုံသေမထားပဲ လွပ်လပ်စွာစီမံနိုင်တဲ့ လုပ်ဆောင်ချက်ပါဝင်မယ် ဆိုရင် Project လိုအပ်ချက်နဲ့အညီ Issue မှတ်တမ်းတင်ရာမှာ ဖြည့်သွင်းရမယ့် Field တွေကို လိုသလို စီမံနိုင်မှာဖြစ်ပါ တယ်။

Time Tracking and Progress – Issue တွေမှတ်တမ်းတင်ရာမှာ အပြီးသတ်ဆောင်ရွက်ရမယ့် Due Date တွေ သတ်မှတ်ပြီး Due Date ကျော်နေတဲ့ Issue တွေကို Calendar တစ်ခုပေါ်မှာ စုစည်းလေ့လာနိုင်တဲ့ လုပ်ဆောင်ချက် ကလည်း အသုံးဝင်ပါတယ်။ ဒါ့အပြင် ဆောင်ရွက်ပြီးစီးမှုအခြေအနေ Progress Percentage နဲ့ အတူ စုစုပေါင်း Issue ဘယ်နှစ်ခုရှိနေသလဲ၊ ပြီးခဲ့တဲ့အပါတ်ထဲ မှာ Issue ဘယ်နှစ်ခု ပိတ်သိမ်းလိုက်နိုင်သလဲ စတဲ့ မှတ်သားဖွယ်အချက်အလက် တွေကို Chart များ Graph များနဲ့ ဖော်ပြနိုင်တဲ့ လုပ်ဆောင်ချက်တွေ ပါဝင်မယ် ဆိုရင်လည်း အသုံးဝင်ပါတယ်။

File Management – Issue မှတ်တမ်းနဲ့အတူ ရည်ညွှန်းကိုးကားလိုတဲ့ Document တွေနဲ့ Screenshot တွေ အပါအဝင် ဖိုင်တွေကိုတွဲဖက်မှတ်တမ်းတင်ရလေ့ ရှိပါတယ်။ အဲ့ဒီလို တွဲဖက်ထားတဲ့ ဖိုင်တွေကို သီးခြားစီမံလို့ရတဲ့ File Manager လုပ်ဆောင်ချက်လည်း ဖြည့်စွက်ပါဝင်နိုင်ပါတယ်။

Knowledge Base – Bug Report ပြုလုပ်ရာတွင် လိုက်နာရမည့် စည်းမျဉ်းများ၊ Issue တစ်ခုပိတ်သိမ်းရာတွင် ဆောင် ရွက်ရမည့် Procedure များ၊ ဖြစ်လေ့ရှိသည့် ပြဿနာများနှင့် ၄င်းတို့အတွက် Solution များ၊ စတဲ့ အချက်အလက်တွေ ကို Knowledge Base တစ်ခုအနေနဲ့ တွဲဖက်မှတ်တမ်းတင်ထားနိုင်မယ်ဆိုရင် ပိုမိုပြည့်စုံသွားမှာဖြစ်ပါတယ်။

Discussion – ထည့်သွင်းထားတဲ့ Issue အတွက် Assign လုပ်ခံရသူက မူလ Issue ထည့်သွင်းသူနဲ့ မရှင်းလင်းတဲ့ အချက် အလက်တွေ ပြန်လည်မေးမြန်းတိုင်ပင်ခြင်း၊ Team Leader က Issue နဲ့ ပက်သက်တဲ့ မှတ်ချက်နဲ့ ညွှန်ကြားချက်တွေကို တွဲဖက် ပေးခြင်းစတဲ့ လုပ်ငန်းများ ဆောင်ရွက်နိုင်တဲ့ Comment, Forum လုပ်ဆောင်ချက်မျိုးတွေ ပါဝင်မယ်ဆိုရင်လည်း အကျိုးရှိစေမှာ ပဲဖြစ်ပါတယ်။

ဒီလိုလုပ်ဆောင်ချက်တွေ စုံလင်စွာပါဝင်တဲ့ Issue Tracking System တစ်ခုကို တစ်ချို့က Project Management System လို့လည်း ခေါ်ကြပါတယ်။ အဓိကရည်ရွယ်ချက်က Issue တွေမှတ်တမ်းတင်ဖို့ဆိုပေမယ့် ဖြည့်စွက် လုပ်ဆောင် ချက်တွေရဲ့ အကူအညီနဲ့ Project တစ်ခုလုံးနဲ့ပက်သက်တဲ့ ကိစ္စတွေကို ဒီစနစ်ကနေတစ်ဆင့် ပူးပေါင်းဆောင်ရွက်နိုင်မှာ ဖြစ်တဲ့အတွက် Project Management System လို့ ခေါ်ကြခြင်းလည်းဖြစ်ပါတယ်။

Software Team တစ်ခုအနေနဲ့ ထိရောက်အောင်မြင်တဲ့ ပူးပေါင်းဆောင်ရွက်မှုကို ရရှိလိုတယ်ဆိုရင်၊ Bug တွေ နည်းပြီး အရည်အသွေကောင်းတဲ့ Software တွေ ဖန်တီးလိုတယ်ဆိုရင်၊ Bug Database သို့မဟုတ် Issue Tracking System သို့မဟုတ် Project Management System တစ်ခုခုကို ရွေးချယ်အသုံးပြုဖို့ လိုအပ်ပါတယ်။ တစ်ခုသတိပြုရမှာက၊ လုပ်ဆောင်ချက်တွေ များများပါတိုင်းလည်း ကောင်းတာတော့မဟုတ်ပါဘူး။ မလိုအပ်ပဲ ရှုပ်ထွေးပြီ အသုံးပြုရခက်ခဲ နေမယ်ဆိုရင်၊ Team အတွက် အထောက်အကူမဖြစ်တဲ့အပြင် အချိန်ပိုကုန်သွားနိုင်ပါတယ်။ ရိုးရိုးရှင်းရှင်းနဲ့ ကိုယ့်လိုအပ် ချက်ကိုက်ညီမယ့် စနစ်မျိုးကို ရွေးချယ် အသုံးပြု နိုင်ဖို့လိုပါတယ်။

Conclusion

Software Team တစ်ခုအတွက် Issue Tracking System ကပေးနိုင်တဲ့ အကျိုးကျေးဇူးတွေ အများကြီးရှိပါတယ်။

Issue တွေကို မှတ်တမ်းတင်ရာမှာ သက်ဆိုင်ရာ Issue တစ်ခုချင်းကို တာဝန်ယူရမယ့်သူတွေပါ တစ်ခါတည်း တွဲဖက် မှတ်တမ်း တင်နိုင်တဲ့အတွက် ဘယ် Issue က ဘယ်သူ့တာဝန်လဲဆိုတဲ့ကိစ္စ လုံးဝရှင်းလင်း သွားမှာဖြစ်လို့ တာဝန်ခွဲဝေမှုနဲ့ တာဝန်ယူမှုပိုင်းမှာ ပိုမိုထိရောက်ရှင်းလင်းသွားမှာဖြစ်ပါတယ်။ ဒါ့အပြင် Issue တွေကို အရေးပါမှုအဆင့်အလိုက်လည်း မှတ်တမ်းတင်ထား နိုင်တဲ့အတွက် ပိုအရေးကြီးတဲ့ Issue တွေကို ဦးစားပေး ဆောင်ရွက်ခြင်းအားဖြင့် ပိုမိုထိရောက်တဲ့ Time Management နဲ့ Release Schedule Management တို့ကို ရရှိနိုင်မှာဖြစ်မှာပါ။

Developer နဲ့ Tester တွေအပြင် အဖွဲ့အစည်းအတွင်းက အခြားတာဝန်ရှိသူတွေကိုပါ Project Development မှာ ပါဝင် လာအောင်အားပေးတဲ့အတွက် ဘက်စုံပူးပေါင်းဆောင်ရွက်မှုကို ရရှိနိုင်မှာဖြစ်ပါတယ်။ Issue Tracking System အကူ အညီနဲ့ Software ရဲ့ လက်ရှိ Progress ကို Developer မဟုတ်သူတွေကပါ သိရှိနိုင်လို့ လုပ်ငန်း ဆောင်ရွက်ရာမှာ အထောက်အကူဖြစ်စေမှာ ပဲဖြစ်ပါတယ်။

Customer တွေအနေနဲ့လည်း Issue တွေကို လေ့လာခြင်းအားဖြင့် သူတို့သွားချင်တဲ့ Direction ဟုတ်မဟုတ် မှန်းဆနိုင် မှာဖြစ်တဲ့အတွက် Feedback တွေကို စောစောစီးစီးပေးနိုင်မှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Software Project တွေမှာရှိတက်တဲ့ မတိကျတဲ့ Requirement ပြဿနာကို တစ်ဘက်တစ်လမ်းက ပြေလည်စေနိုင်ပါတယ်။

ဒါကြောင့် အရည်အသွေးကောင်း Software Product တွေဖန်တီးမယ့် စွမ်းဆောင်ရည်မြင့် Software Team တစ်ခု အနေနဲ့ Issue Tracking System တွေကို ထိရောက်အောင် အသုံးချနိုင်ဖို့လိုပါတယ်။

ထိရောက်တဲ့ပူးပေါင်းဆောင်ရွက်မှု ဖြစ်စေဖို့အတွက်
စနစ်တစ်ကျသတ်မှတ်ထားတဲ့ လုပ်ငန်းစဉ်နဲ့ ရှင်းလင်းမြင်သာတဲ့
ဦးတည်ချက်တို့ လိုအပ်ပါတယ်။

Rockstar Developer Course

Project Management, Web Service, Server Architecture
NodeJS အစရှိသည် ဤစာအုပ်ပါ အကြောင်းအရာများကို
စာရေးသူကိုယ်တိုင် သင်ကြားပေးခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၇) – Team Collaboration

အခန်း (၁) မှာ Agile Principle အကြောင်း ပြောခဲ့တာမှတ်မိဦးမှာပါ။ Agile Principle အရ Software Project တစ်ခုကို စီမံရာမှာ အခြေခံထားလိုက်နာရတဲ့ မူ (၅) ချက်ကို ပြန်လည်ဖော်ပြလိုက်ပါတယ်။

1. **Adaptive Planning** – ပုံသေသတ်မှတ်ထားခြင်းမဟုတ်ပဲ လိုအပ်သလို ပြင်လွယ်ပြောင်းလွယ်တဲ့ Plan ပေါ်မှာ အခြေခံရမယ်။
2. **Evolutionary Development** – Product တစ်ခုလုံးကို တစ်ခါတည်း တည်ဆောက်မယ့်အစား အဆင့်လိုက် တည်ဆောက်သွားရမယ်။
3. **Early Delivery** – အသုံးပြုသူက Product တစ်ခုလုံးပြီးစီးအချောသတ်အောင် စောင့်ဖို့မလိုပဲ စောနိုင်သမျှ အစောဆုံး Product ကို စတင်အသုံးပြုနိုင်ရမယ်။
4. **Continuous Improvement** – အဆင့်မြှင့်တင်မှုတွေကို အဆက်မပြတ် ဆောင်ရွက်သွားရမယ်။
5. **Rapid and Flexible Response to Change** – Requirement နဲ့ နည်းပညာ အပြောင်းအလဲရှိလာတိုင်း မှာ လိုအပ်သလိုလက်ခံပြောင်းလဲနိုင်တဲ့ Product တစ်ခုဖြစ်ရမယ်။

ဒီအချက်တွေကိုလေ့လာကြည့်ရင် စိတ်ဝင်စားဖို့ကောင်းသလောက် လက်တွေ့အကောင်အထည်ဖော်ဖို့ ခက်ခဲတဲ့ အချက်တွေ ဖြစ်တယ်ဆိုတာကို တွေ့ရမှာဖြစ်ပါတယ်။ တိကျတဲ့ Requirement Specification တစ်ခုကိုသာ အခြေခံလို့ရမယ့်ဆိုရင် Software Project တစ်ခုကို စီမံခန့်ခွဲရတာ လွယ်နိုင်ပါတယ်။ Team မှာ ပါဝင်သူ အားလုံးက တစ်ယောက်နဲ့တစ်ယောက် စကားတွေအများကြီး ပြောနေစရာမလိုပဲ Specification က သတ်မှတ်ချက်တွေအတိုင်း ကိုယ်တာဝန်ကျရာကို လုပ်လိုက် ယုံပါပဲ။

ဒါပေမယ့် လက်တွေ့မှာ တိကျတဲ့ Specification တစ်ခုရဖို့ဆိုတာ မလွယ်ပါဘူး။ Agile က သတ်မှတ်ထားတဲ့ Adaptive Planning နဲ့ Flexible Response to Change ဆိုတဲ့ သတ်မှတ်ချက်များနဲ့အညီ အခြေအနေအရ လိုအပ်သလို ပြင်လွယ် ပြောင်းလွယ်တဲ့ Up-to-date Specification ပေါ်မှာ အခြေခံနိုင်ဖို့ လိုပါတယ်။ Spec ပြောင်းသွားတိုင်း ပြောင်းသွား တိုင်း၊ ပြောင်းရခြင်း ရည်ရွယ်ချက်ကိုနားလည်အောင်၊ ပြောင်းသွားတဲ့အချက်တွေကို သိရှိအောင်၊ သိရှိတဲ့အတိုင်း ညီညီ ညွတ်ညွတ်နဲ့ Team တစ်ခုလုံးက အမှန်တစ်ကယ်လိုက်နာဆောင်ရွက်အောင် ဆက်သွယ်ညှိနှိုင်းရပါတော့မယ်။

Evolutionary Development, Early Delivery နဲ့ Continues Improvement ဆိုတဲ့ သဘောတရားတွေကလည်း ဒီအတိုင်းပါပဲ။ Software ကို တစ်ခါတည်း အစအဆုံးတည်ဆောက်အခြင်းမဟုတ်ပဲ အပိုင်းလိုက်အပိုင်းလိုက် အဆင့်မြှင့် တင်တည်ဆောက်သွားခြင်းဖြစ်လို့၊ မျက်စိစုံမှတ်ပြီး ငါတာဝန်ယူထားတာ Feature (၁၀) ခု၊ အဲ့ဒီ (၁၀) ခုကိုပဲ တစ်ခုပြီးမှ တစ်ခုလုပ်တယ်ဆိုပြီး ထိုင်လုပ်နေလို့ မရတော့ပါဘူး။ Software ပထမအဆင့်မှာ ဘယ် Feature တွေ ပါဝင်မှာလဲ။ Software တစ်ခုလုံးပြီးအောင် မစောင့်ပဲ အဲ့ဒီ ပထမအဆင့်ကို Release လုပ်နိုင်ဖို့ဆို

ရင် ငါဘက်က ဘာတွေ လုပ်ပေး မလဲ။ စသဖြင့် တစ်ခြား Developer တွေနဲ့ အဆင့်တိုင်းအဆင့်တိုင်းမှာ ညှိနှိုင်းပြီး ပူးပေါင်းဆောင်ရွက်ရပါတော့မယ်။

ဒါကြောင့် ပူးပေါင်းဆက်သွယ်နိုင်မှု Collaboration နဲ့ ပူးပေါင်းဆောင်ရွက်နိုင်မှု Teamwork ကောင်းမှသာ Agile Principle တွေနဲ့ Software Project တစ်ခုကို စီမံနိုင်မှာဖြစ်ပါတယ်။ Collaboration မကောင်းပဲ Agile Principle တွေ ကိုသုံးဖို့ ကြိုးစားရင်၊ အောင်မြင်ဖို့ထက် တစ်ယောက်တစ်ပေါက်နဲ့ ဟိုမရောက်ဒီမရောက်ဖြစ်သွားဖို့က ပိုများပါတယ်။

ပြီးခဲ့တဲ့အခန်းမှာ ဖော်ပြခဲ့တဲ့ Issue Tracking System ဟာ Team Collaboration အတွက် များစွာအရေးပါတဲ့ Tool တစ်ခုဖြစ်ပါတယ်။ ဒီအခန်းမှာ နောက်ထပ်အရေးပါတဲ့ Team Collaboration နည်းစနစ်တွေအကြောင်းကို ဆက်လက် ဖော်ပြသွားပါမယ်။

ထိရောက်စွာပူးပေါင်းဆောင်ရွက်နိုင်တဲ့ Agile Software Team တစ်ခုရဲ့ အလုပ်လုပ်ပုံလုပ်ငန်းစဉ်က ဒီလိုဖြစ်ပါလိမ့်မယ်။

1. Software Project တစ်ခု စတင်တော့မယ်ဆိုတာနဲ့ Project Manager (သို့မဟုတ်) Product Owner က **Functional Specification** တစ်ခုကို စတင်ရေးသားပါတယ်။
2. Specification ပေါ်အခြေခံပြီး **Project Schedule** ကိုရေးဆွဲထားရပါတယ်။
3. Specification မှာပါဝင်တဲ့ **User Story** တွေကိုထုတ်ယူပြီး **Product Backlog** ဖန်တီးပါတယ်။
4. ပြီးတဲ့အခါ Iteration တစ်ခုစတင်နိုင်ဖို့အတွက် **Sprint Planning Meeting** ခေါ်ပြီး **Sprint Backlog** ကို ဖန်တီးပါတယ်။
5. Development Team က Sprint Backlog မှာပါဝင်တဲ့ User Story တွေပေါ်မှာအခြေခံပြီး Implementation Detail အတွက် **Task List** ကို တည်ဆောက်ပါတယ်။
6. Sprint Backlog နဲ့ Task List ကို အခြေခံပြီး လိုအပ်တဲ့ Code တွေကို စတင်ရေးသားပါတယ်။ ဒီလိုရေးသားရာမှာ Code Quality ကောင်းစေဖို့အတွက် Code Review နည်းစနစ်တစ်ခုဖြစ်တဲ့ Pair Programming ကို အသုံးပြုပါတယ်။
7. တည်ဆောက်ရရှိလာတဲ့ ရလဒ်မှာတွေ့ရှိတဲ့ Bug နဲ့ Feature Request တွေကို **Issue Tracking System** ထဲမှာ မှတ်တမ်းတင်ပါတယ်။
8. Project Manager (သို့မဟုတ်) Product Owner က Issue Tracking System ထဲမှာ ရှိနေတဲ့ Bug နဲ့ Feature Request တွေရဲ့ အခြေအနေနဲ့ချိန်ဆပြီး Specification နဲ့ Schedule ကို Update လုပ်ပါတယ်။
9. **Up-to-date Schedule** နဲ့ချိန်ညှိပြီး Product Backlog ကို ပြန်လည်စီစဉ်ပါတယ်။
10. ပြီးတဲ့အခါ အမှတ်စဉ် (၄) ကို ပြန်သွားပြီး နောက် Iteration တစ်ခုကို ပြန်လည်စတင်ပါတယ်။

လုပ်ငန်းစဉ်ထဲမှာပါဝင်တဲ့ Functional Specification, Project Schedule, User Story, Product Backlog, Sprint Planning Meeting, Sprint Backlog, Task List, Code Review, Pair Programming စတဲ့ အခေါ်အဝေါ်တွေနဲ့ နည်း စနစ်တွေကို ဆက်လက်ဖော်ပြသွားပါမယ်။

7.1 – Specification

Team Collaboration အတွက် ပထမဦးဆုံးအနေနဲ့ အရေးပါတဲ့ အချက်ကတော့ Specification ဖြစ်ပါတယ်။ တိကျတဲ့ Spec တစ်ခုရဖို့ခက်ခဲတက်လို့ Specification ရေးဆွဲတဲ့လုပ်ငန်းအတွက် အလွန်အမင်းအချိန်ကုန်မခံသင့်ကြောင်းကို **အခန်း (၁)** မှာ ပြောခဲ့ပါတယ်။ ဒါပေမယ့် Spec လုံးဝမရှိလို့တော့ မရပါဘူး။ လက်တွေ့မှာ Software မှာ ဘာတွေပါပြီး ဘယ်ပုံဘယ်နည်းအလုပ်လုပ်သင့်တယ်ဆိုတဲ့ Function Specification ကိုတော့ အတိအကျချ ရေးထားသင့်ပါတယ်။ အပြောင်းအလဲတွေကြောင့် မလိုအပ်ပဲ အချိန်တွေ မကုန်စေဖို့အတွက် ဘယ်လိုနည်းပညာတွေနဲ့ ဘယ်လိုတည်ဆောက်မယ် ဆိုတဲ့ Implementation Detail ကိုတော့ Spec ထဲမှာ ထည့်သွင်းမရေးသားပဲ ရှောင်သင့်ပါတယ်။

လက်တွေ့မှာ၊ Spec ကို လမ်းညွှန်အနေနဲ့ကိုင်ပြီး Developer တွေက Software ကို ဖန်တီးကြရမှာပါ။ အဲ့ဒီ Spec ကို ကိုင်ပြီးတော့ပဲ Tester တွေက ဒီ Software က အမှန်တစ်ကယ် မျှော်မှန်းထားတဲ့အတိုင်း အလုပ်လုပ်ရဲ့လားစမ်းသပ်ရမှာပါ။ Spec ရှိထားမှ Software မှာ ပါဝင်ရမယ့် အချက်တွေနဲ့ပက်သက်ပြီး Customer တွေ၊ Manager တွေ၊ Tester တွေနဲ့ ပူပေါင်းဆောင်ရွက်ရာမှာ ဘုံအဖြစ် ကိုးကားစရာတစ်ခုကို ရရှိမှာ ဖြစ်ပါတယ်။

Spec ဆိုတာ Software ကို Code တွေရေးသားပြီး တစ်ကယ်မဖန်တီးမီ ရိုးရိုးစာနဲ့ ရေးပြီး ကြိုတင်ဖန်တီးလိုက်တဲ့ သဘောဖြစ်လို့၊ Software ကို အစမ်းတည်ဆောက်လိုက်တာနဲ့တူပါတယ်။ ဒါကြောင့် Spec ရေးပြီးချိန်မှာ တွေ့ကြုံရမယ့် အခက်အခဲတွေကို ကြိုတင်မှန်းဆမိပြီးဖြစ်လို့ အလုပ်ပိုတွင်သွားနိုင်ပါတယ်။

Function Specification တစ်ခုရှိထားခြင်းအားဖြင့် ရရှိလာတဲ့ ဖြည့်စွက်အကျိုးရလဒ်ကတော့၊ နောင်တစ်ချိန် Software ပြီးသွားလို့ Documentation တွေရေးတဲ့အခါမှာ ပြန်လည်အသုံးဝင်လာပါလိမ့်မယ်။ Documentation ကို အစအဆုံး အကုန်ပြန်ရေးနေစရာမလိုတော့ပဲ Spec ကိုပဲ End-user နားလည်တဲ့ အသုံးအနှုန်းနဲ့ ပြောင်းပေးလိုက်ရင် Documentation ဖြစ်လာနိုင်ပါတယ်။

Function Spec တစ်ခုမှာ ပါဝင်သင့်တဲ့အချက်အချို့ကို ဖော်ပြပေးလိုက်ပါတယ်။

Overview – Spec ရေးသားရာမှာ Software ရဲ့ ရည်ရွယ်ချက်နဲ့၊ ပါဝင်တဲ့ လုပ်ဆောင်ချက် Feature စာရင်းကို လိုရင်းတိုရှင်း ဖော်ပြထားတဲ့ Overview နဲ့ အစပြုသင့်ပါတယ်။ ဥပမာ –

"ဤ Software သည် Bug နှင့် Feature များကို မှတ်တမ်းတင်နိုင်သည့် Bug Database အဖြစ် Software Team များတွင် အသုံးပြုရန်ဖြစ်သည်။ ဤ Software တွေ Developer, Tester နှင့် Manager တို့အတွက် Account များတည်ဆောက်နိုင်သည့် လုပ်ဆောင်ချက်၊ Bug category များ စီမံနိုင်သည့်လုပ် ဆောင်ချက်နှင့် Bug များကို Priority နှင့် Status အလိုက် ခွဲခြားစီမံနိုင်သည့် လုပ်ဆောင်ချက်တို့ ပါဝင်မည်ဖြစ်သည်။"

Scenarios – Spec တစ်ခုမှာ ဆက်လက် ပါဝင်သင့်တာကတော့ User Story တွေပဲဖြစ်ပါတယ်။ Software ရဲ့ အလုပ် လုပ်ပုံကို အသုံးပြုသူ User တစ်ဦးချင်းစီရဲ့ ရှုထောင့်ကနေ ရေးသားသတ်မှတ်ထားခြင်း ဖြစ်ပါတယ်။ ဥပမာ -

"Project Manager က သတ်မှတ်ထားသည့် Username/Password ကို သုံး၍ Login ဝင်သည်။ ထို့နောက် Dashboard တွေ့ပေါ်လာသည့် Project များအတွင်းမှ စီမံလိုသည့် Project ကို ရွေးချယ်သည်။ ရွေးချယ်လိုက်သည့် Project အတွက်မှတ်တမ်းတင်ထားသည် Issue စာရင်းပေါ်လာသည့် အခါ၊ Status Filter Option မှ Assigned ကို ရွေးချယ်ခြင်းအားဖြင့် Assign လုပ်ယုံသာလုပ်ရသေးပြီး စတင်ဆောင်ရွက်ခြင်းမရှိသေးသည် Issue များကို ရွေးချယ် ကြည့်ရှုသည်။"

စသဖြင့် User Role အမျိုးမျိုးနဲ့ ပါဝင်တဲ့ Feature အမျိုးမျိုးတို့အတွက် User Story တွေကို တက်နိုင်သလောက် ပြည့်စုံအောင် ရေးသားဖော်ပြထားသင့်ပါတယ်။ ပေးထားတဲ့ User Story အရ Login Authentication လုပ်ဆောင်ချက်ပါဝင်ရပါတော့မယ်။ အချက်အလက်တွေကို စုစည်းဖော်ပြပေးတဲ့ Dashboard လုပ်ဆောင်ချက် ပါဝင်ရပါတော့မယ်။ Multiple Project လုပ်ဆောင်ချက် ပါဝင်ရပါတော့မယ်။ Issue List, Issue Filter နဲ့ Issue Status တို့ပါဝင်ရပါတော့ မယ်။

User Story ဆိုတာ ဒါပါပဲ။ ဘာလုပ်ရမယ်လို့ သတ်မှတ်ထားခြင်းမဟုတ်ပဲ၊ ဘယ်လိုလိုချင်တယ်လို့ သတ်မှတ် ထားခြင်းဖြစ် ပါတယ်။ လုပ်ရမယ့်အလုပ်ကို Spec ထဲမှာ အတိအကျသတ်မှတ်ထားခြင်းမဟုတ်ပဲ၊ လိုချင်တဲ့ အနေအထားကိုသာ သတ် မှတ်ထားခြင်းဖြစ်ပါတယ်။

Non Goals – Spec မှာ ဖော်ပြသင့်တဲ့ နောက်တစ်ချက်ကတော့ "မလုပ်ဖို့ဆုံးဖြတ်ထားတဲ့အချက်များ" ဖြစ်ပါ တယ်။ Spec စတင်မရေးသားမီ တိုင်ပင်ဆွေးနွေးပြီးရလာတဲ့ ရလဒ်အရတစ်ချို့ လုပ်ဆောင်ချက်တွေကို ရည်ရွယ်ချက်ရှိရှိ မလုပ်ပဲနေဖို့ ဆုံးဖြတ်ထားတက်ကြပါတယ်။ ဥပမာ –

"Internet Explorer Version 8 နှင့် ရှေ့ပိုင်း Version အဟောင်းတွေကို Support မလုပ်ပါ။"

– စသဖြင့် တစ်ခါတည်းထည့်သွင်းရေးသားထားရပါတယ်။ မလုပ်ရခြင်း အကြောင်း ရင်းကိုလည်း တွဲဖက် ဖော်ပြထားနိုင် ရင် ပိုကောင်းပါတယ်။ နောင်လက်တွေ့ Software ကို ရေးသားတော့မှ ဒီလုပ်ဆောင်ချက်လေး ထည့်လုပ်လိုက်ရင် ကောင်းမယ်လို့ စိတ်ကူးပေါ်လာပြီး လုပ်မိတဲ့အခါ မလိုအပ်ပဲ အချိန်တွေကုန်တက်ပါတယ်။ မူလကတည်းက အကြောင်းရှိ လို့ မလုပ်ဖို့ ဆုံးဖြတ်ထားတဲ့အချက်တွေကို တစ်ခါတည်းရှင်းအောင် ထည့် ရေးထားရပါတယ်။

Screen Flow – Screen Flow Diagram တစ်ခုလဲ Spec မှာပါသင့်ပါသေးတယ်။ Home Screen ကနေ Register ကို နှိပ်လိုက်ရင် ရောက်သွားမယ့် Screen, အဲ့ဒီ Screen ကမှ Register Button ကို နှိပ်လိုက်ရင် ရောက်သွားမယ့် Screen, Login ကို နှိပ်ပြီး Login ဝင်ပြီးနောက်ရောက်သွားမယ့် Screen စသဖြင့် User Interface Screen တွေရဲ့ ဆက်စပ်မှုကို Diagram တစ်ခုနဲ့ ဖော်ပြထားသင့်ပါတယ်။

Screen by Screen Detail – ဆက်လက်ပြီး Screen တစ်ခုချင်းစီမှာ ပါဝင်မယ့် အချက်အလက်အသေးစိတ်ကို ဆက်လက်ဖော်ပြရပါတယ်။ ဥပမာ –

"Login Screen မှာ Login ဝင်ဖို့အတွက် Username/Password ထည့်သွင်းနိုင်တဲ့ Form ပါဝင်မယ်။ Password

မေ့နေရင် Account Recover လုပ်နိုင်တဲ့လုပ်ဆောင်ချက်ပါဝင်မယ်။ Remember Me Option ပါဝင်မယ်။ Account အသစ်ဆောက်ချင်ရင် Register Page ကို သွားနိုင်တဲ့ လုပ်ဆောင်ချက် ပါဝင်မယ်။"

"Dashboard မှာ Project စာရင်းဖော်ပြမယ်၊ စုစုပေါင်း Open Issue အရေအတွက်ကို ဖော်ပြမယ်။ Logout လုပ်ဆောင်ချက် ပါဝင်မယ်။ Account Password ပြောင်းနိုင်တဲ့ Setting လုပ်ဆောင်ချက်ပါဝင်မယ်။"

စသဖြင့် Screen တစ်ခုချင်းမှာ ပါဝင်မယ့် အချက်အလက်တွေကို အသေးစိတ် ဖော်ပြရပါတယ်။ အကြမ်းဆွဲထားတဲ့ User Interface Prototype တွေရှိရင်လည်း တွဲဖက်ထားသင့်ပါတယ်။

Todo – တစ်ချို့ ချက်ခြင်းမဆုံးဖြတ်နိုင်သေးတဲ့ အချက်တွေရှိရင် Todo အနေနဲ့ မြင်သာအောင် မှတ်ချက်ထည့်သွင်းထား သင့်ပါတယ်။ တစ်ကယ်တာဝန်ယူရမယ့် Developer (သို့မဟုတ်) သက်ဆိုင်ရာကဏ္ဍမှာ ကျွမ်းကျင်တဲ့ Expert နဲ့ တိုင်ပင် ပြီးမှ ရေးသားဖော်ပြမယ့်အချက်တွေကို "အသေးစိတ်ဆွေးနွေးပြီးမှ ဖော်ပြရန်" လို့ မြင်သာအောင် မှတ်ချက်ပေးထားရပါမယ်။

Up-to-date – နောက်ဆုံးအနေနဲ့ သတိပြုသင့်တာကတော့ Spec ကို ရေးပြီး၊ အဲ့ဒီအတိုင်း မဖြစ်ဖြစ်အောင် လုပ်ရမယ် လို့ ပုံသေမယူဆပဲ ပြောင်းလဲလာတဲ့ Requirement နဲ့ အခြေအနေနဲ့ ဆီလျော်အောင် အခါအားလျော်စွာ ပြင်ဆင်ပြီး အမြဲ Update ဖြစ်နေအောင် စီစဉ်ထားသင့်ပါတယ်။

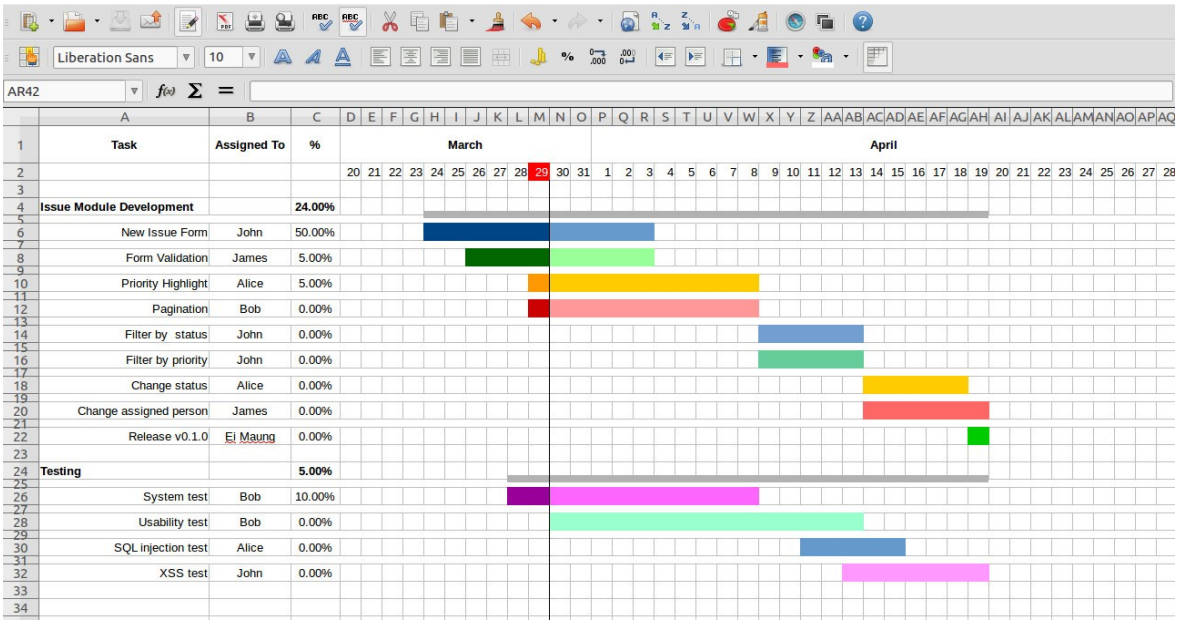
Team နဲ့ အလုပ်လုပ်ရတဲ့အခါ အားလုံးက Developer တွေချည်းမဟုတ်ပါဘူး။ Manager တွေ၊ Tester တွေ၊ Admin Staff တွေ၊ Marketing သမားတွေနဲ့ User တွေလည်း ပါဝင်နိုင်ပါတယ်။ Software မှာပါဝင်တဲ့ Feature တွေနဲ့ ပက်သက်ပြီး ပြောစရာရှိလာတဲ့အခါ Meeting ခန်းတွေထဲမှာ ဟိုတုန်းကဟိုလိုပြောခဲ့တယ်၊ ဒီတုန်းကဒီလိုပြောခဲ့တယ်လို့ အငြင်းအခုံတွေလုပ်ပြီး အချိန်ကုန်မယ့်အစား အသေအချာမှတ်တမ်းတင်ထားတဲ့ Spec တစ်ခုကို ကိုင်ပြီး ဆက်သွယ် ဆောင်ရွက်တာက ပိုပြီးထိရောက်မှာပဲ ဖြစ်ပါတယ်။

7.2 – Project Scheduling

ပြီးခဲ့တဲ့အခန်းမှာ ဖော်ပြခဲ့တဲ့ Software Team တစ်ခုရဲ့အရည်အသွေးကို တိုင်းတာတဲ့ အချက် (၁၂) ချက်ထဲမှာ Up-to-date Schedule တစ်ခု ထားရှိရဲ့လားဆိုတဲ့အချက်လည်း တစ်ချက်အပါအဝင်ဖြစ်ပါတယ်။ Requirement တစ်ခုသာ တိတိကျကျရှိ မယ်ဆိုရင် Software Project တစ်ခုရဲ့ Release Schedule ရေးဆွဲရတာ ဘာမှမခက်ပါဘူး။ ဒါပေမယ့် Requirement ဆိုတာ ပြောင်းလဲတက်တဲ့အတွက် သူပြောင်းတိုင်း Schedule ကလည်း သင့်တော်သလို လိုက်ပြောင်းနိုင်ဖို့ လိုပါတယ်။ Project Schedule တစ်ခုကိုရေးဆွဲတဲ့အခါ Specification မှာ ပါဝင်တဲ့ User Story တွေပေါ်မှာ အခြေခံ ရေးဆွဲရမှာ ဖြစ်ပါတယ်။ တနည်းအားဖြင့် Specification နဲ့ User Story ပြောင်းသွားရင် Schedule လည်း လိုက် ပြောင်းနိုင်ရမှာပဲဖြစ်ပါတယ်။

The Gantt Chart

Project Schedule ရေးဆွဲရာမှာ အသုံးဝင်တဲ့ နည်းစနစ်ကတော့ The Gantt Chart ဖြစ်ပါတယ်။ Bar Chart တစ်မျိုး ကိုသုံးပြီး Task တစ်ခုချင်းစီရဲ့ စမယ့်အချိန်နဲ့ ဆုံးမယ်အချိန်တို့ကို ဖော်ပြခြင်းအားဖြင့် Project ရဲ့ Progress ကို မြင်သာအောင် သတ်မှတ်တဲ့နည်းလမ်းဖြစ်ပါတယ်။

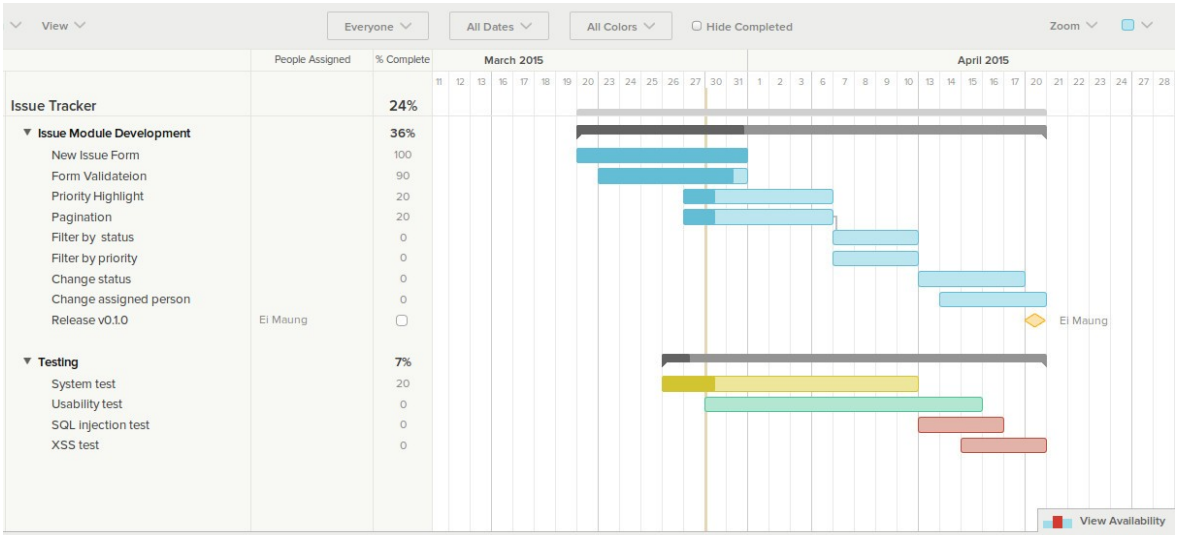


ပုံ (၇.၁) - Example Gantt Chart in Spreadsheet

ပုံ (၇.၁) မှာ လေ့လာကြည့်ရင် Spreadsheet တစ်ခုကိုအသုံးပြုပြီး Project Plan တစ်ခုကို The Gantt Chart အနေနဲ့ ရေးဆွဲထားတာကိုတွေ့ရမှာ ဖြစ်ပါတယ်။ Chart ရဲ့ အတိုင်အဖြစ် Task, Assigned To နဲ့ Progress Percentage တို့ကိုထားပါတယ်။ အတန်းအနေနဲ့ ရက်စွဲကိုထားပါတယ်။ ယနေ့ရက်စွဲကို မြင်သာအောင် ကန့်လတ်ဖြတ်လိုင်းတစ်ခုနဲ့ဖော် ပြထားပါသေးတယ်။ Task တစ်ခုချင်းစီအတွက် စမယ့်ရက်နဲ့ ဆုံးမယ့်ရက်ကို Bar နဲ့ဖော်ပြထားသလို၊ ပြီးစီမှု Progress ကိုလဲ Bar ထဲမှာ တစ်ခါတည်း အရောင်ခွဲပြီး ဖော်ပြထားပါသေးတယ်။ ပြီးတော့ Task တွေကို အမျိုးတူရာစုပြီး အုပ်စု ခွဲ ထားပါသေးတယ်။ နမူနာမှာ Issue Module Development နဲ့ Testing ဆိုပြီး Task အုပ်စုနှစ်ခု ရှိပါတယ်။

ဒီနည်းနဲ့ Project Deadline အဖြစ် လက်စွဲထားအသုံးပြုနိုင်တဲ့ Up-to-date Schedule တစ်ခုကို ရရှိနိုင်မှာပဲဖြစ်ပါတယ်။ Development Team မှာမပါဝင်တဲ့ Management ပိုင်းကလူတွေ၊ Customer တွေ၊ Marketing သမားတွေနဲ့ ဆက်သွယ်ဆောင်ရွက်ရာမှာလည်း ဒီ Chart ကိုပဲ ကိုးကားအသုံးပြုနိုင်မှာဖြစ်ပါတယ်။

Microsoft Excel တို့ LibreOffice Calc တို့လို Spreadsheet Software တစ်ခုကိုသုံးပြီး Gantt Chart Project Schedule ကို အလွယ်တစ်ကူ တည်ဆောက်နိုင်ပါတယ်။ ပိုပြီးထိရောက်ချင်ရင်တော့ Gantt Chart လုပ်ဆောင်ချက် ပါဝင်တဲ့ Project Management Software တွေကို အသုံးပြုနိုင်ပါတယ်။ အဲ့ဒီ Software တွေက User Story (သို့) Task List ကို ပြုပြင်လိုက်ယုံနဲ့ Project Schedule ကို အလိုအလျောက် Update လုပ်ပေးနိုင်ကြပါတယ်။



ပုံ (၇.၂) - Example Project Scheudle on Team Gantt

ပုံ (၇.၂) မှာ နမူနာအနေနဲ့ ဖော်ပြထားတာကတော့ Team Gantt လို့ခေါ်တဲ့ Project Management Software တစ်ခု ဖြစ်ပါတယ်။ ဘယ်ဘက်ခြမ်းမှာရှိတဲ့ List ကို Update လုပ်ပေးလိုက်ရင် ညာဘက်ခြမ်းက Project Schedule ကလည်း အလိုအလျောက် Update ဖြစ်သွားမှာပဲဖြစ်ပါတယ်။ ဒီနည်းနဲ့ List ပြောင်းသွားတိုင်း အလို အလျောက် Up-to-date ဖြစ်သွားတဲ့ Project Schedule ကို ရရှိနိုင်မှာဖြစ်ပါတယ်။ အောက်ပါလိပ်စာမှာ စမ်းသပ်အသုံးပြုနိုင်ပါတယ်။

http://teamgantt.com/

တစ်ခုသတိပြုသင့်တာက၊ Software Project တစ်ခုမှာပါဝင်တဲ့ Task တွေ၊ User Story တွေ၊ Feature တွေက အများကြီးရှိမှာပါ။ အဲ့ဒီ Task တွေအားလုံးကို Schedule ထဲမှာ အသေးစိတ်မထည့်သင့်ပါဘူး။ အထူးသဖြင့် Specification ထဲမှာ ဖော်ပြထားတဲ့ User Story တွေကို အခြေခံပြီးတော့သာ အကျဉ်းချုပ်သဘောမျိုး ရေးဆွဲ ထားသင့်ပါတယ်။

7.3 – SCRUM and Backlog

SCRUM ဆိုတာ **အခန်း (၁)** မှာ ဖော်ပြခဲ့တဲ့ Iterative Development Model နဲ့အတူ တွဲဖက်သုံးတဲ့ Team Colloboration နည်းစနစ်တစ်မျိုး ဖြစ်ပါတယ်။ SCRUM အကြောင်းပြောတော့မယ်ဆိုရင် SCRUM Team တစ်ခု မှာ ပါဝင်သူတွေရဲ့ တာဝန်ခွဲဝေပုံကနေ စပြောရပါမယ်။ SCRUM Team တစ်ခုမှာ Product Owner, SCRUM Master နဲ့ Development Team ဆိုပြီး သုံးပိုင်းပါဝင်ပါတယ်။

Product Owner ဆိုတာ Stakeholder (Customer, Higher Management) တွေကို ကိုယ်စားပြုပြီး Product မှာ ပါဝင်ရမယ့် လုပ်ဆောင်ချက်တွေကို ဆုံးဖြတ်သူဖြစ်ပါတယ်။ Stakeholder တွေနဲ့ညှိနှိုင်းပြီး စကားပြောရ မယ့် အပိုင်းကို တာဝန်ယူသူလည်းဖြစ်ပါတယ်။ တနည်းအားဖြင့် Product Owner ဆိုတာ Stakeholder နဲ့ Development Team ကြားက ကြားခံဆက်သွယ်ဆောင်ရွက်ပေးသူပဲ ဖြစ်ပါတယ်။ ရိုးရိုး Team တစ်ခုရဲ့ Project

Manager နဲ့ သဘောတရား ဆင်ပေမယ့် Project Manager လို Developer တွေကို ဘယ်သူဘာလုပ်ရမယ်လို့ တာဝန်ခွဲဝေ ပေးတာတွေဘာတွေ မလုပ်ပါဘူး။ ပါဝင်ရမယ့် Feature (User Story) တွေနဲ့ အဲဒီ Feature တွေရဲ့ ဦးစားပေးအဆင့်ကိုသာ စီမံဆုံးဖြတ်သူ ဖြစ်ပါတယ်။

Development Team ကတော့ Software ကို အမှန်တစ်ကယ် ရေးသားမယ့်သူတွေဖြစ်ပါတယ်။ ရိုးရိုး Team တွေနဲ့ မတူပဲ ထူးခြားသွားတာက၊ Development Team မှာပါဝင်သူတွေရဲ့ တာဝန်ကို Product Owner တို့ SCRUM Master တို့က မဆုံးဖြတ်ပဲ သူတို့ကိုယ်တိုင် ဆုံးဖြတ်ပြီး သူတို့ကိုယ်တိုင် တာဝန်ယူသွားမယ့် Self-organized Team ပဲဖြစ်ပါ တယ်။

SCRUM Master ရဲ့တာဝန်ကတော့ Development Team ဟာ SCRUM Practice တွေကို လိုက်နာရဲ့လား။ မလိုက် နာနိုင်တဲ့သူ (သို့မဟုတ်) မလိုက်နာနိုင်လောက်စရာ အကြောင်းတွေရှိနေသလား စတဲ့အချက်တွေကို လေ့လာသုံးသပ်ပြီး SCRUM Practice တွေကို အပြည့်အဝလိုက်နာနိုင်ရေးအတွက် လိုအပ်တဲ့ ကိစ္စတွေကို စီမံဆောင်ရွက်ပေးရသူဖြစ်ပါတယ်။ SCRUM Meeting တွေကို ဦးဆောင်ရပါတယ်၊ လိုအပ်တဲ့ SCRUM Training တွေပေးရပါတယ်၊ Developer တွေ ကိုယ်တိုင်တာဝန်ယူနိုင်မှုကို လမ်းညွှန်အားပေးရပါတယ်။

Sprint

SCRUM နည်းစနစ်မှာအဓိကအကျဆုံးတစ်ခုကတော့ Sprint ဖြစ်ပါတယ်။ Iterative Development Model ဆိုတာ Software တစ်ခုလုံးကို အစအဆုံးတည်ဆောက်မယ့်အစား၊ လုပ်ဆောင်ချက်တစ်ခုချင်းကို သုံးလို့ရတဲ့အထိ အပိုင်းလိုက် အပိုင်းလိုက် ခွဲခြားတည်ဆောက်သွားတဲ့ နည်းစနစ်တစ်ခုဖြစ်တယ်လို့ ပြောခဲ့တာကို မှတ်မိဦးမှာပါ။ Sprint ဆိုတာကတော့ အချိန်ကာလတစ်ခု သတ်မှတ်ပြီး အဲဒီကာလအတွင်းမှာ လုပ်ဆောင်ချက်တစ်ခု (သို့မဟုတ်) လုပ်ဆောင်ချက်တစ်ချို့ကို အသုံးပြုလို့ရတဲ့အဆင့်ထိ ပြီးမြောက်အောင် ဆောင်ရွက်ဖို့ ကန့်သတ်လိုက်တဲ့ အချိန်ကာလသတ်မှတ်ချက်တစ်ခုကို ဆိုလို တာပါ။ အချိန်သတ်မှတ်ချက်ဟာ Team တစ်ခုနဲ့တစ်ခုမတူပါဘူး။ တစ်ချို့က (၁) ပါတ် သတ်မှတ်တက်ကြပါတယ်။ တစ်ချို့က (၂) ပါတ် သတ်မှတ်တက်ကြပါတယ်။ တစ်ချို့လည်း (၁) လထိ သတ်မှတ်တက်ကြပါတယ်။ (၂) ပါတ်က လက်တွေ့မှာ အသင့်တော်ဆုံးဖြစ်ပါတယ်။

Sprint တစ်ခုကို စတင်တော့မယ်ဆိုတာနဲ့ Team တစ်ခုလုံးပါဝင်တဲ့ Sprint Meeting ခေါ်ရပါတယ်။ Sprint Meeting မှာ Sprint Backlog တစ်ခုကို တည်ဆောက်နိုင်ဖို့ ညှိနှိုင်းကြရပါတယ်။ Sprint Backlog ဆိုတာ သတ်မှတ်ထားတဲ့ အချိန်ကာလအတွင်း အပြီးဆောင်ရွက်မယ့် Feature စာရင်းပဲ ဖြစ်ပါတယ်။ သတ်မှတ်ကာလအတွင်းမှာ သတ်မှတ်ထားတဲ့ Backlog ထဲက Feature တွေကို အပြီးဆောင်ရွက်ကြရမှာ ဖြစ်ပါတယ်။ Sprint တစ်ခုပြီးသွားရင် အသုံးချလို့ရတဲ့ Feature တစ်ခု အနည်းဆုံး ထွက်ပေါ်လာသင့်ပါတယ်။ Feature တစ်ခု ထွက်ပေါ်လာတယ်ဆိုတာ အဖြစ်ထွက်ပေါ်လာခြင်း မျိုးမဟုတ်ပဲ၊ အမှန်တစ်ကယ် Release လုပ်လို့ရတဲ့ အခြေအနေထိ ထွက်ပေါ်လာရမှာဖြစ်ပါတယ်။ ဒါပေမယ့် Feature ထွက်ပေါ်မလာလို့ဆိုပြီး Sprint ရဲ့ အချိန်ကို သက်တမ်းမတိုးရပါဘူး။ သတ်မှတ်ရက်ပြည့်တာနဲ့ ရပ်ပြီး Sprint Review Meeting ခေါ်ရပါတယ်။ ထွက်ပေါ်လာတဲ့ရလဒ်ကို လေ့လာပြီး၊ မျှော်မှန်းထားသလို မအောင်မြင်ခဲ့ရင်လည်း မအောင်မြင်ရ ခြင်းအကြောင်းရင်းကို စမ်းစစ်ပြီး၊ နောက် Sprint တစ်ခု စတင်ဖို့ ပြင်ရပါတယ်။

ဒီနည်းနဲ့ Sprint တွေကိုသုံးပြီး Iterative Development Model ကို အသက်ဝင်စေခြင်းဖြစ်ပါတယ်။

Sprint နဲ့ သဘောသဘာဝတူတဲ့ (၂) ခု ရှိပါသေးတယ်။ Spike နဲ့ Tracer Bullet လို့ခေါ်ကြပါတယ်။ Sprint လိုပဲ အချိန်ကာလသတ်မှတ်ပြီး ဆောင်ရွက်ပေမယ့် အမှန်တစ်ကယ် Feature တစ်ခုကို Develop လုပ်ခြင်းမဟုတ်ပဲ အကြောင်းအရာတစ်ခုကို စမ်းသပ်လေ့လာဖို့ ရည်ရွယ်ခြင်းကို **Spike** လို့ခေါ်ပါတယ်။ သုတေသနလုပ်ဖို့လိုတဲ့ ကိစ္စတွေနဲ့ လေ့လာသူယူဖို့ လိုတဲ့ကိစ္စတွေကို Sprint လို အချိန်သတ်မှတ်ပြီး သတ်မှတ်ချိန်အတွင်း အပြီးလေ့လာခြင်းဖြစ်ပါတယ်။

နောက်တစ်ခုကတော့ Sprint လိုပဲ အချိန်ကာလသတ်မှတ်ပြီး ဆောက်ရွက်ပေမယ့် မြင်သာတဲ့ Feature မဟုတ်ပဲ၊ Code Quality ပိုကောင်းအောင် ဆောင်ရွက်တဲ့ကိစ္စတွေကိုတော့ **Tracer Bullet** လို့ ခေါ်ပါတယ်။ Code Refactoring ကို Sprint လို အချိန်သတ်မှတ်ပြီး သတ်မှတ်ချိန်အတွင်း အပြီးဆောင်ရွက်ခြင်းဖြစ်တယ်လို့ ဆိုနိုင်ပါတယ်။ ပုံမှန်အားဖြင့် Sprint တစ်ခုနဲ့တစ်ခုကြားထဲမှာ လိုအပ်ရင် Spike နဲ့ Tracer Bullet တွေကို ကြားဖြတ် ဆောင်ရွက် တက်ကြပါတယ်။

Sprint Planning Meeting

Sprint တစ်ခုစတင်တော့မယ်ဆိုတာနဲ့ Sprint Planning Meeting ခေါ်ရပါတယ်။ အဲ့ဒီ Meeting မှာ သတ်မှတ်ထားတဲ့ Sprint ကာလအတွင်း ဘာတွေလုပ်မှာလည်း တစ်ခါတည်းဆုံးဖြတ်ရပါတယ်။ ပုံမှန်အားဖြင့် Sprint Planning Meeting ရဲ့ ပထမပိုင်းမှာ Sprint ပြီးသွားတဲ့အခါ ထွက်ပေါ်လာဖို့ မျှော်မှန်းတဲ့ရလဒ်ကို ဆုံးဖြတ်ရပါတယ်။ အဲ့ဒီရလဒ် ရရှိဖို့အတွက် ဆောင်ရွက်ရမယ့် လုပ်ငန်းတွေကဘာတွေလည်း ဆုံးဖြတ်ရပါတယ်။

Meeting ဒုတိယပိုင်းမှာတော့၊ သတ်မှတ်ထားတဲ့ ရလဒ်ထွက်ပေါ်အောင် ဆောင်ရွက်ရမယ့် Feature တွေကို ရွေးထုတ်ပြီး Sprint Backlog တစ်ခုတည်ဆောက်ခြင်း၊ Implementation Plan အသေးစိတ်တိုင်ပင်ခြင်း တို့ကို Development Team က ဆက်လက်ဆောင်ရွက်ရပါတယ်။

Daily SCRUM Meeting

Sprint တစ်ခု စပြီဆိုတာနဲ့ Daily SCRUM Meeting ကို နေ့စဉ်ခေါ်ရပါတယ်။ အကြာကြီး လုပ်စရာမလိုပါဘူး။ (၁၅) မိနစ် Meeting လောက်ဆိုရင်လုံလောက်ပါတယ်။ ထိုင်ပြောမှလည်းမဟုတ်ပါဘူး။ ပေါ့ပေါ့ပါးပါး မတ်တက်ရပ်ဆွေးနွေး လည်းရပါတယ်။ ဒါပေမယ့် အချိန်နဲ့နေရာကိုတော့ ပုံသေသတ်မှတ်ထားဖို့လိုပါတယ်။ အကြောင်းအမျိုးမျိုးနဲ့ တစ်ဦး တစ်ယောက် မပါဝင်နိုင်ရင်လည်း မစောင့်ရပါဘူး။ သတ်မှတ်ထားတဲ့ အချိန်မှာ Meeting ကို ပုံမှန်စတင်ရပါတယ်။ SCRUM Master က ဒီ Meeting ကို တာဝန်ယူ Coordinate လုပ်ပေးရပါတယ်။

Daily SCRUM Meeting မှာ အချက် (၃)ချက်ကို အဓိကထားဆွေးနွေးရပါတယ်။ Sprint Planning Meeting မှာ သတ်မှတ်ခဲ့တဲ့ မျှော်မှန်းချက် အောင်မြင်ဖို့အတွက် မနေ့က ဘာတွေလုပ်ခဲ့သလဲ။ ဒီနေ့ဘာတွေဆက်လုပ်မလဲ။ အခက်အခဲ ဘာရှိနေသလဲ စတဲ့အချက် (၃) ချက်ကို အဓိကဆွေးနွေးရမှာဖြစ်ပါတယ်။

Sprint Review Meeting

Sprint တစ်ခုအတွက် သတ်မှတ်ချိန်ပြည့်တာနဲ့ လုပ်ငန်းတွေအားလုံး ပြီးသည်ဖြစ်စေ၊ မပြီးသည်ဖြစ်စေ Sprint Review Meeting ခေါ်ရပါတယ်။ Sprint Review Meeting မှာလည်း (၂) ပိုင်း ပါဝင်ပါတယ်။ ပထမတစ်ပိုင်းမှာ

အောင်မြင်ပြီးစီး ခဲ့တဲ့ Feature တွေကဘာတွေလဲ၊ မအောင်မြင်ပဲ ကျန်သွားတာတွေကဘာတွေလည်း Review လုပ်ရပါတယ်။ ပြီးတဲ့အခါ အောင်မြင်ပြီးစီးခဲ့တဲ့ Feature တွေကို Demo ပြရပါတယ်။

နောက်တစ်ပိုင်းမှာတော့၊ ပြီးခဲ့တဲ့ Sprint နဲ့ပက်သက်ပြီး Developer အားလုံးရဲ့မှတ်ချက်ကို ပေးရပါတယ်။ အောင်မြင်မှု၊ မအောင်မြင်မှုတွေကို နောက် Sprint တွေမှာ ကိုးကားနိုင်အောင် သင်ခန်းစာထားလေ့လာရပါတယ်။ နောင် Sprint တွေမှာ သတိပြုဆောင်ရွက်သင့်တဲ့ အချက်တွေကို ဆွေးနွေးတိုင်ပင်ရပါတယ်။

Product Backlog and Sprint Backlog

Product Backlog ဆိုတာဟာ Software တစ်ခုလုံးပြီးမြောက် အောင်မြင်ဖို့အတွက် ဆောင်ရွက်ရမယ့် User Story တွေကို စုစည်းထားခြင်းဖြစ်ပါတယ်။ Sprint Backlog ဆိုတာကတော့ Sprint တစ်ခုစတင်တော့မယ်ဆိုတာနဲ့ Sprint မှာ ဆောင်ရွက်မယ့် Feature (User Story) တွေကို Product Backlog ထဲကနေ ရွေးထုတ်ထားတဲ့ စာရင်းဖြစ်ပါတယ်။

Product Backlog ကို တာဝန်ယူစီစဉ်သူကတော့ Product Owner ဖြစ်ပါတယ်။ Product Owner ကပဲ Product Backlog မှာပါဝင်တဲ့ User Story တွေရဲ့ အရေးပါမှုအဆင့် Priority ကိုဆုံးဖြတ်စီစဉ်ပါတယ်။ Product Backlog ထဲက User Story တွေဟာ အမှန်တော့၊ Specification ထဲက User Story တွေပဲဖြစ်ပါတယ်။ ထူးခြားတာကတော့ Spec ထဲ မှာ User Story တွေကို ရေးတဲ့အခါ ပြည့်စုံအောင်ရေးရပေမယ့်၊ Backlog ထဲရောက်တဲ့အခါမှာတော့ အတက်နိုင်ဆုံး လိုရင်းတိုရှင်း အကျဉ်းချုပ် ရေးသားထားတာမျိုး ဖြစ်နိုင်ပါတယ်။

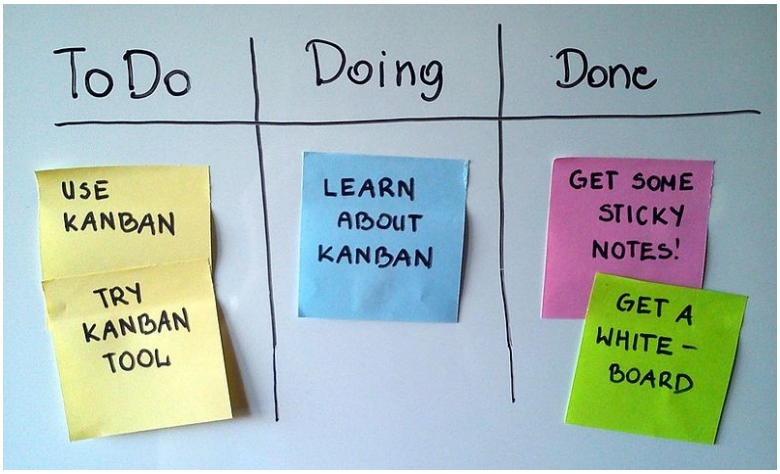
SCRUM Team တစ်ခုရဲ့ ဆောင်ရွက်ရန် ကိစ္စရပ်များအားလုံးကို အဲ့ဒီ User Story တွေ ပေါ်မှာ အခြေပြုဆောင်ရွက် ကြရပါတယ်။ Team မှာပါတဲ့ မည်သူမဆို User Story တွေကို Backlog ထဲ ထည့်လို့ရပါတယ်။ ဒါပေမယ့် ဘယ် User Story ကို ဦးစားပေးမလဲ စီစဉ်ဆုံးဖြတ်ခြင်းကိုတော့ Product Owner ကသာ ဆုံးဖြတ်ရပါတယ်။

Backlog ထဲမှာ User Story တွေချည်းပဲရှိရမယ်လို့တော့ မဆိုလိုပါဘူး။ အခြားအချက်တွေဖြစ်တဲ့ Feature Request တွေ၊ Bug Report တွေ၊ Technical Work တွေ Research Task တွေလည်း လိုအပ်ရင် ပါဝင်နိုင်ပါတယ်။ Technical Work ဆိုတာက ဥပမာ – "Server ကို Update လုပ်ရမယ်" ဆိုတဲ့ သတ်မှတ်ချက်မျိုးဖြစ်ပါတယ်။ Research Task ဆိုတာကတော့ ဥပမာ – "မြန်မာနိုင်ငံရှိမြို့များကို စာရင်းပြုစုရမယ်" ဆိုတဲ့ သတ်မှတ်ချက်မျိုးဖြစ်ပါတယ်။ ဒါတွေဟာ Product အတွက် User Story မဟုတ်ပေမယ့် Backlog ထဲမှာ ပါဝင်နိုင်တဲ့ အကြောင်းအရာတွေဖြစ်ပါတယ်။

Sprint Planning Meeting မှာ လိုအပ်တဲ့အချက်တွေဆုံးဖြတ်ပြီးပြီဆိုရင် Development Team က မျှော်မှန်းအချက် အတိုင်း အကောင်အထည်ဖော်နိုင်ဖို့အတွက် ဆောင်ရွက်ရမယ့် User Story တွေကို Product Backlog ထဲကနေ Sprint Backlog ထဲကို ရွေးထုတ်ပြောင်းရွှေ့ရပါတယ်။ Sprint ကာလမှာ Sprint Backlog ထဲက User Story တွေကိုသာ အဓိကထား ဆောင်ရွက်သွားကြတော့မှာဖြစ်ပါတယ်။ Sprint စပြီဆိုတာနဲ့ အခြေခံအားဖြင့် Sprint Backlog ထဲကို User Story အသစ်တွေ ထပ်မထည့်ရတော့ပါဘူး။ မဖြစ်မနေလိုအပ်လို့ ထည့်ရင်လည်း Product Owner နဲ့ တစ်ခြားသူတွေ မထည့်ပဲ Developer တွေကိုယ်တိုင်ကိုပဲ ထည့်ခွင့်ပြုရပါတယ်။

Task Board

Backlog နဲ့ Task တွေကိုစီမံဖို့အတွက် Task Board တွေကိုအသုံးပြုတက်ကြပါတယ်။ Task Board ဆိုတာ ခေါင်းစဉ် တပ်ပြီး Column တွေခွဲခြားရေးဆွဲထားတဲ့ White Borad တစ်ခုသာ ဖြစ်ပါတယ်။ Sprint Board လို့ လည်းခေါ်ပါတယ်။



ပုံ (၇.၃) - Simplest Task Board

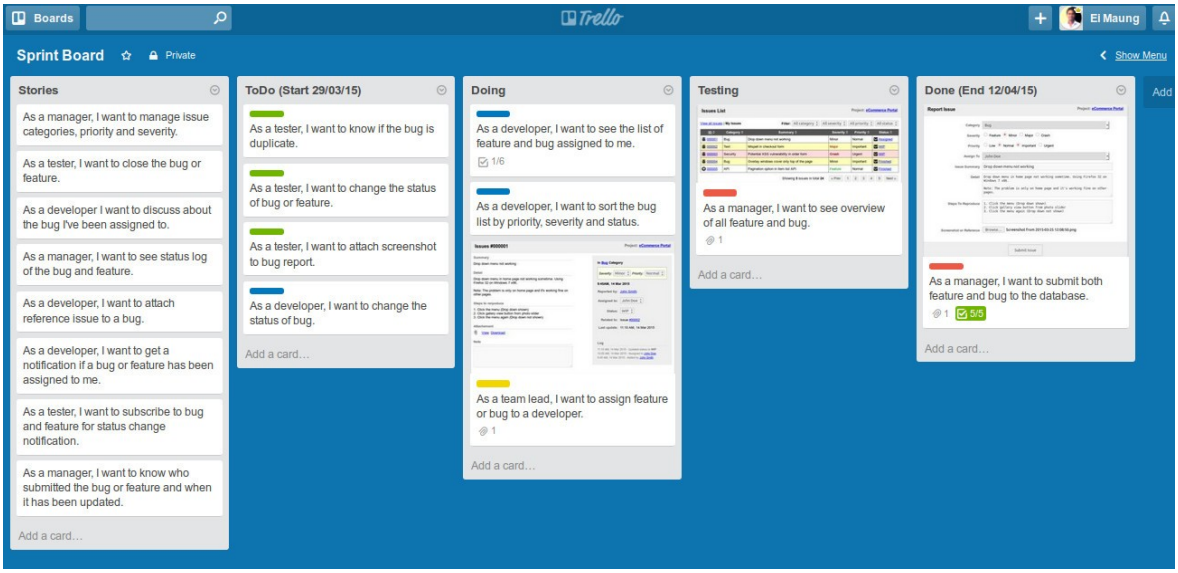
Source – Wikipedia

ပုံ (၇.၃) မှာ နမူနာကြည့်ပါ။ White Board တစ်ခုမှာ ToDo, Doing နဲ့ Done ဆိုပြီး Column (၃) ခုခွဲထားပါ တယ်။ ဆောင်ရွက်ရန် Task တွေကို ToDo Column ထဲမှာ Sticky Note လေးတွေသုံးပြီး ကပ်ထားပါတယ်။ Sprint တစ်ခုစပြီ ဆိုတာနဲ့ Sprint မှာ ပါဝင်တဲ့ User Story တွေအားလုံးကို ToDo Column မှာ စုစည်းထားရပါ တယ်။ Category ခွဲတဲ့ အနေနဲ့ အမျိုးအစားမတူတဲ့ User Story တွေအတွက် အရောင်မတူတဲ့ Sticky Note လေး တွေခွဲပြီး အသုံးပြုကြလေ့ရှိ ပါတယ်။ User Story တစ်ခုကို စတင်ဆောင်ရွက်နေပြီဆိုရင် ဆောင်ရွက်နေတဲ့ User Story အတွက် Stick Note ကို ToDo ကနေခွာပြီး Doing မှာ ပြောင်းကပ်ပေးရပါတယ်။ ဆောင်ရွက်ပြီးစီး သွားတဲ့ User Story အတွက် Stick Note ကိုတော့ Done Column မှာ ပြောင်းကပ်ထားရပါတယ်။ ဒီနည်းနဲ့ Sprint Board ကို တစ်ချက်ကြည့်လိုက်ယုံနဲ့ ဘယ် User Story တွေပြီးနေပြီး၊ ဘယ် User Story တွေစလုပ်ပြီ၊ User Story ဘယ်လောက် ကျန်နေသေးတယ် စတဲ့ Overview ကို သိရနိုင်ပါတယ်။

နမူနာအနေနဲ့ အရိုးရှင်းဆုံး Board တစ်ခုကိုပေးထားတာပါ။ ပုံမှန်အားဖြင့် Sprint Board တစ်ခုမှာ Column (၅) ခု ရှိနိုင်ပါတယ်။ Stories, ToDo, Doing, Testing, Done တို့ပဲဖြစ်ပါတယ်။

ပုံ (၇.၄) ကိုလေ့လာကြည့်ပါ။ Stories Column က Product Backlog အတွက်ပါ။ Product မှာ ပါဝင်ရမယ့် User Story တွေကို Stories Column မှာစုစည်းထားပါတယ်။ ToDo Column ကတော့ Sprint Backlog အတွက် ပါ။ လက်ရှိ Sprint မှာ ဆောင်ရွက်မယ့် User Story တွေကို Stories ကနေ ToDo ထဲကို ပြောင်းရွှေ့ထားရပါ တယ်။ Doing ကတော့ လက်ရှိဆောင်ရွက်နေတဲ့ User Story စာရင်းဖြစ်ပြီး၊ Testing ကတော့ ဆောင်ရွက် ပြီးစီးသွားလို့ သေချာအောင် စမ်းသပ်နေတဲ့ User Story များအတွက်ဖြစ်ပါတယ်။ Testing အဆင့်မှာ စမ်းပြီးလို့

ပြီးမြောက်တာ သေချာတဲ့အခါ Done Column ထဲကို ရွှေ့ထားရမှာပဲဖြစ်ပါတယ်။



ပုံ (၇.၄) - A Sprint Board Example on Trello

နမူနာမှာသုံးထားတာက Trello လို့ခေါ်တဲ့ Online Tool တစ်ခုဖြစ်ပါတယ်။ Software တစ်ခုဖြစ်သွားတဲ့အတွက် White Board ထက်တော့ လုပ်ဆောင်ချက်နည်းနည်း ပိုစုံသွားပါတယ်။ User Story တွေနဲ့အတူ Screenshot တွေ၊ ဖိုင်တွေကို Attach လုပ်ထားလို့ရပါတယ်။ User Story တစ်ခုချင်းအတွက် Checklist (Task List) တွေတွဲထားလို့ရပါတယ်။ Sticky Note တွေကို အရောင်ခွဲသုံးသလိုပဲ သူ့မှာလည်း User Story တွေကို အရောင် Label လေးတွေနဲ့ တွဲထားလို့ ရပါတယ်။ User Story တစ်ခုချင်းအလိုက် Comment မှတ်ချက်တွေ ထည့်သွင်းထားလို့ ရပါတယ်။ ဒါပေမယ့် White Board ကတော့ ပိုပြီးမြင်သာပါတယ်။ ဥပမာ - ရုံးရောက်တာနဲ့ Board ကို တစ်ချက်လှမ်းကြည့်လိုက်ယုံနဲ့ အခြေအနေကို ချက်ခြင်းမှန်းဆနိုင်ပါတယ်။ သူ့အားသာချက်နဲ့သူပဲမို့ နှစ်သက်ရာကို သုံးနိုင်ပါတယ်။ Trello ကို စမ်းသပ်လိုရင်တော့ အောက်ပါလိပ်စာမှာ စမ်းသပ်အသုံးပြုနိုင်ပါတယ်။

http://trello.com/

SCRUM နဲ့ သဘောတရားဆင်တဲ့ Kanban ဆိုတဲ့ နည်းစနစ်တစ်မျိုးလည်း ရှိပါသေးတယ်။ အခြေခံအားဖြင့် SCRUM က Development ပိုင်းကို Focus ထားတဲ့ Process Management နည်းစနစ်တစ်ခုဖြစ်ပြီး Kanban ကတော့ Business Operation တစ်ခုလုံးကို စီမံဖို့ ရည်ရွယ်ထားတဲ့ Process Management နည်းစနစ်ဖြစ်ပါတယ်။ ဒါပေမယ့် Task Board ကို သုံးကြတာချင်း တူပါတယ်။ ကွာသွားတာက Kanban မှာ Development ပိုင်းကိစ္စတွေသာမက၊ Specification ရေးသားမှုအပိုင်းတွေ၊ Data Analyst ကိစ္စတွေ၊ Deployment ကိစ္စတွေ အပါအဝင် Software Development Work-flow တစ်ခုလုံးကို Task Board သုံးပြီးစီမံတက်ပါတယ်။ ပြီးတော့ Kanban မှာ Sprint လို နည်းစနစ်မျိုးကို မသုံးပါဘူး။ အဲ့ဒီအစား Column တစ်ခုမှာ ပါဝင်ရမယ့် Task အရေအတွက်ကို ကန့်သတ်ပြီး တစ်ခုပြီးတာနဲ့ နောက်တစ်ခုထပ်ထည့်ပြီး အဆက်မပြတ်ဆောင်ရွက်သွားတဲ့ နည်းစနစ်ကို သုံးပါတယ်။ ဥပမာ – Doing Column မှာ User Story (၄) ခုထက် ပိုထည့်ခွင့်မပြုပဲ လက်ရှိလုပ်နေ

တဲ့အလုပ်ပြီးသွားလို့ Doing Column မှာ User Story တွေလျော့သွားပြီဆိုမှ ထပ်ထည့်ခွင့် ပြုတဲ့စနစ်မျိုးကို သုံးပါတယ်။ Iterative Development ဆိုတာထက် Continues Development ဆိုတဲ့ သဘာဝဘက်ကို အသားပေးတဲ့သဘောဖြစ်ပါတယ်။

အခြေခံအားဖြင့် SCRUM ကို Agile Development Model လို့ခေါ်ပြီး Kanban ကိုတော့ Lean Development Model လို့ ခေါ်တက်ကြပါတယ်။ SCRUM နဲ့ Kanban နှစ်ခုပေါင်းပြီး Scrumban ဆိုပြီး နည်းစနစ်သစ်တစ် မျိုးကိုလည်း တီထွင်ထားကြပါသေးတယ်။ အခြား Agile Method တစ်ခုဖြစ်တဲ့ Extreme Programming (XP) ဆိုတာလည်း ရှိပါသေးတယ်။ ဒါတွေအကြောင်းကိုတော့ ဒီနေရာမှာ မဖော်ပြနိုင်တော့ပါဘူး။ စိတ်ဝင်စားရင် ဆက်လက်လေ့လာကြည့် သင့်ပါတယ်။

7.4 – Task List

ပြီးခဲ့တဲ့အခန်းမှာ Issue Tracking System တွေကို Bug Database အနေနဲ့သာမက Feature List အဖြစ်နဲ့ပါ အသုံးပြုနိုင် ကြောင်းဖော်ပြခဲ့ပါတယ်။ Feature List ဆိုတဲ့သဘောက Software မှာ ဘယ်လို Features တွေ ပါဝင်ရမလဲ၊ ဘယ်သူတွေ တာဝန်ယူရမလဲ စတဲ့အချက်တွေကို ဖော်ပြထားပါတယ်။ Feature List ထဲက Feature တစ်ခု ကို လက်တွေ့ရေးသား အကောင်အထည်ဖော်ရာမှာ အသုံးပြုရမယ့် နည်းပညာတွေနဲ့ ဆောင်ရွက်သွားရမယ့် လုပ်ငန်းစဉ် အသေးစိတ်တော့ ပါဝင်မှာမဟုတ်ပါဘူး။ အလားတူပဲ Bug တွေနဲ့ပက်သက်ရင်လည်း ဘယ်လိုအခြေအနေ မှာတွေ့ရတဲ့ Bug လည်း၊ ဘယ်သူတာဝန်ယူဖြေရှင်းရမှာလဲဆိုတဲ့ အချက်တွေပဲ ပါဝင်မှာပါ။ အဲ့ဒီ Bug ကို ဖြေရှင်းရာမှာ ဆောင်ရွက် သွားရမယ့် အသေးစိတ်အချက်အလက်တွေတော့ ပါဝင်မှာ မဟုတ်ပါဘူး။

ဒီနေရာမှာ အသုံးဝင်လာတာက Task List ဖြစ်ပါတယ်။ Task List ဆိုတာ သက်ဆိုင်ရာ Feature (သို့မဟုတ်) Bug ကို ဆောင်ရွက်ရာမှာ အသုံးပြုရမယ့် အသေးစိတ် Implementation Detail လို့လည်း ဆိုနိုင်ပါတယ်။

Task List တစ်ခုရရှိဖို့အတွက် Google Keep, Evernote, Remember The Milk စတဲ့ Task List Manager ပရိဂရမ် တွေသုံးနိုင်သလို ရိုးရိုး Text ဖိုင်တစ်ခုနဲ့လည်း ဖန်တီးနိုင်ပါတယ်။ ကျွန်တော်ကတော့ ရိုးရိုး Text ဖိုင်တစ်ခုကိုသာ အသုံးပြုဖို့ အကြံပြုလိုပါတယ်။ Text ဖိုင်တစ်ခုနဲ့ မှတ်တမ်းတင်ထားရင် Task List ကို သက်ဆိုင်ရာ Source Code နဲ့ အတူ တွဲထားနိုင်မှာဖြစ်ပါတယ်။ ဒါကြောင့် လက်ရှိ Code Base ကို Pull (သို့မဟုတ်) Clone လုပ်ယူတဲ့အခါ Source Code နဲ့အတူ Task List လည်း တစ်ခါတည်း ပါဝင်သွားစေမှာပါ။ တနည်းအားဖြင့် Task List ကိုယ်တိုင်က Source Code ရဲ့ အစိတ်အပိုင်းတစ်ရပ်ဖြစ်သွားပြီး၊ အဲ့ဒီ Task List ကိုလည်း Version Control လုပ်နိုင်စေမှာဖြစ်ပါတယ်။ Text ဖိုင်တစ်ခုနဲ့ မှတ်တမ်းတင်ထားတဲ့ နမူနာ Task List တစ်ခုကို ဖော်ပြပေးလိုက်ပါတယ် –

Plain Text

```
Design:
 # TODO
 [-] Two column layout for issue detail page
   Main contents such as category, subject and etc. should be in left column.
 [-] Issue list table design
   Issue list table should highlight priority and severity

Coding:
 # TODO
 [-] Issue form validation @today
   category, subject, reproduction steps, severity and priority are mandatory
 [-] SQL Injection filter in issue submission @due (15-03-28)
 [-] Issue log in issue details

 # DONE
 [+] Issue list Sorting (15-03-27 23:35)

 # CANCELLED
 [x] Issue delete on issue detail (15-03-27 23:46)

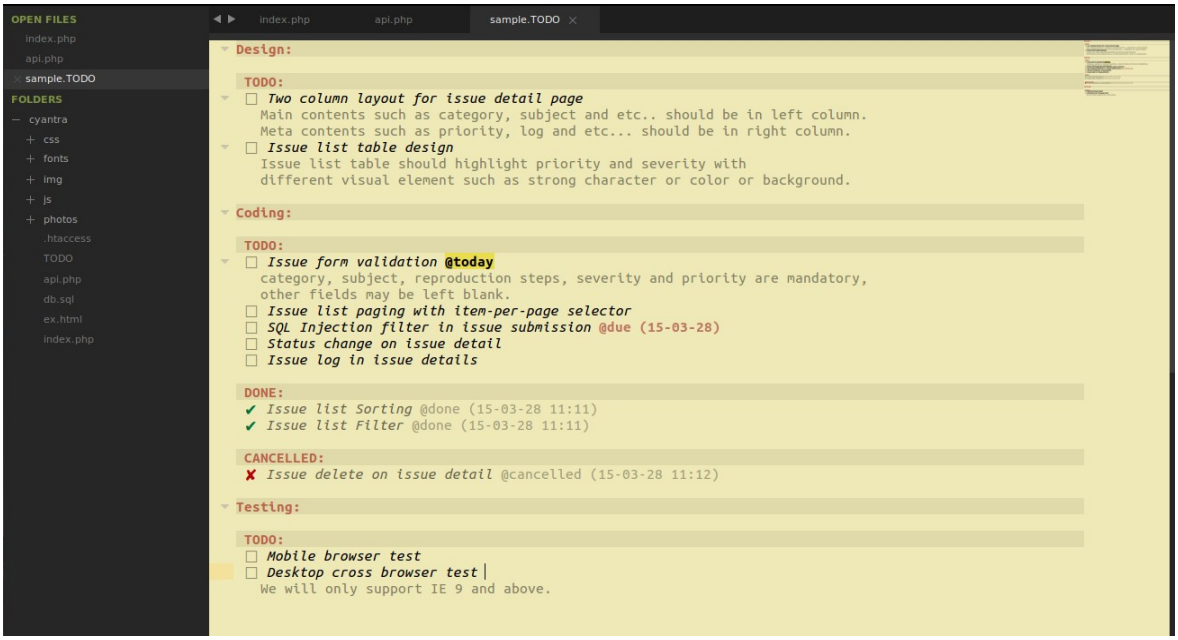
Testing:
 # TODO
 [-] Desktop cross browser test
   We will only support IE 9 and above.
```

Task List တစ်ခုမှာ Task တွေကိုအမျိုးအစားခွဲထားသင့်ပါတယ်။ နမူနာကိုလေ့လာကြည့်ရင် Design, Coding, Testing ဆိုပြီး အမျိုးအစား Category ခွဲထားတာကို တွေ့ရနိုင်ပါတယ်။ နောက်ထပ် ရှိသေးရင်ထပ်ခွဲထားနိုင်ပါတယ်။

သက်ဆိုင်ရာ Category တစ်ခုစီမှာ TODO, DONE နဲ့ CANCELLED ဆိုပြီး အုပ်စု (၃) စု ပါဝင်သင့်ပါတယ်။ TODO ကတော့ ဆောင်ရွက်ရန် Task များဖြစ်ပါတယ်။ DONE ကတော့ ဆောင်ရွက်ပြီး Task များဖြစ်ပါတယ်။ CANCELLED ကတော့၊ မူလကဆောင်ရွက်ရန် သတ်မှတ်ခဲ့ပေမယ့် ဆက်လက်မဆောင်ရွက်တော့ဖို့ ဆုံးဖြတ်လိုက်တဲ့ Task များပဲဖြစ် ပါတယ်။ TODO ထဲက Task တစ်ခုကို ဆောင်ရွက်ပြီးစီးသွားတဲ့အခါ DONE ထဲကို ရွှေ့ထားပေးဖို့ လိုပါတယ်။ အဲ့ဒီလို ရွှေ့လိုက်တဲ့အခါ (နမူနာမှာပြထားသလို) ရွှေ့လိုက်တဲ့နေ့ရက်ကို တွဲဖက်မှတ်တမ်းတင်ထားသင့်ပါတယ်။ ဒီတော့မှာ ဘယ့်နေ့ က ဘယ် Task ပြီးသွားတယ်ဆိုတဲ့ မှတ်တမ်းကို ရရှိမှာဖြစ်ပါတယ်။ အလားတူပဲ Task တစ်ခုကို ဆက်လက်မဆောင် ရွက်ဖို့ ဆုံးဖြတ်တယ်ဆိုရင်လည်း CANCELLED အောက်ကို ရွှေ့ပြီး ရက်စွဲနဲ့အတူ မှတ်တမ်းတင်ထားသင့်ပါတယ်။

နမူနာမှာလေ့လာကြည့်ရင် ကနေ့အပြီးလုပ်ရမယ့် Task တစ်ခုကို @today ဆိုတဲ့ Tag တစ်ခုနဲ့ တွဲဖက်မှတ်တမ်းတင် ထားတာကို တွေ့ရနိုင်ပါတယ်။ သတ်မှတ်ရက်တစ်ခုမှာ အပြီးဆောင်ရွက်ရမယ့် Task တွေကိုလည်း @due ဆိုတဲ့ Tag နဲ့ တွဲဖက်မှတ် တမ်းတင်ထားတာကို တွေ့ရမှာပဲဖြစ်ပါတယ်။ ဒါအပြင် အသေးစိတ်မှတ်ချက်ထည့်သွင်းဖို့လိုတဲ့ Task တွေအတွက် အသေးစိတ် မှတ်ချက်တွေကိုလည်း တွဲဖက်ထားပါသေးတယ်။

နမူနာမှာဖော်ပြထားသလို စာလုံးအရောင်တွေ ခွဲခြားဖော်ပြစေဖို့အတွက် အသုံးပြုနေတဲ့ Text Editor (သို့မဟုတ်) IDE ရဲ့ Syntax Highlight လုပ်ဆောင်ချက်ကို အသုံးချနိုင်ပါတယ်။ ဒါမှမဟုတ် IDE (သို့မဟုတ်) Text Editor နဲ့အတူတွဲဖက်ပါ ဝင်လာတဲ့ Task Management လုပ်ဆောင်ချက်ကိုလည်း အသုံးပြုနိုင်ပါတယ်။

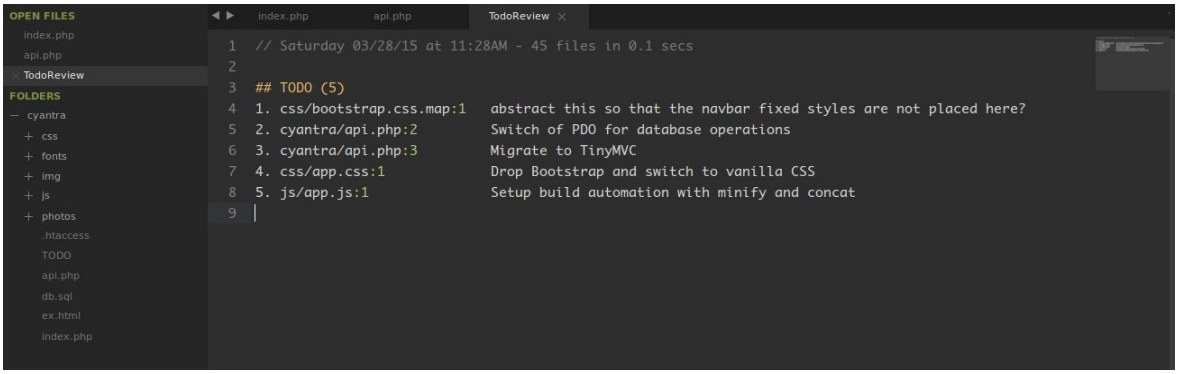


ပုံ (၇.၅) - PlainTasks - Task List Manager in Sublime Text 3

ပုံ (၇.၅) မှာ PlainTasks လို့ခေါ်တဲ့ Sublime Text 3 Package တစ်ခုကို နမူနာအနေနဲ့ ဖော်ပြထားပါတယ်။ PlainTasks က Text ဖိုင်တစ်ခုအနေနဲ့ သိမ်းဆည်းထားတဲ့ Task List ကို Icon တွေ၊ အရောင်တွေနဲ့၊ Format လုပ်ပြီး သပ်ရပ်သွားအောင်ဖော်ပြပေးနိုင်ပါတယ်။ ဒါ့အပြင် Task List ကို Manage လုပ်နိုင်တဲ့ Feature တွေလဲ ပါဝင်ပါသေး တယ်။ ဥပမာ – Task တစ်ခုကိုရွေးပြီး Keyboard Shortcut Ctrl + D ကို နှိပ်ခြင်းအား ဖြင့် ဆောင်ရွက်ပြီးစီးကြောင်း သတ်မှတ်နိုင်ပါတယ်။ စမ်းသပ် အသုံးပြုလိုရင် အောက်ပါလိပ်စာမှာ ရယူနိုင်ပါ တယ်။

https://github.com/aziz/PlainTasks

Code Comment ထဲမှာ ထည့်သွင်းမှတ်သားခဲ့တဲ့ TODO တွေကို ဆွဲထုတ်ပြီး Task List အဖြစ် ဖော်ပြပေးနိုင်တဲ့ လုပ်ဆောင်ချက်တွေလည်း Text Editor နဲ့ IDE တွေမှာ ပါဝင်တက်ပါတယ်။ ပုံ (၇.၆) မှာတော့ TodoReview လို့ ခေါ်တဲ့ Source Code ထဲမှာ ထည့်သွင်းရေးသားထားတဲ့ Task တွေကို ဆွဲထုတ်ပြီး Task List တစ်ခုအနေနဲ့ Generate လုပ်ပေးနိုင်တဲ့ Sublime Text 3 Package တစ်ခုကို နမူနာအနေနဲ့ ဖော်ပြပေးထားပါတယ်။



ပုံ (၇.၆) - TodoReview - Task List Extractor in Sublime Text 3

Source Code ထဲမှာ TODO: NOTE: FIXME: CHANGE: စတဲ့ Keyword တွေခံပြီး Comment တွေထည့်သွင်းမှတ်သားထားခဲ့မယ်ဆိုရင် TodoReview က Code Base တစ်ခုလုံးကို Scan လုပ်ပြီး အဲ့ဒီ Task တွေကို Task List တစ်ခုအနေနဲ့ စုစည်းဖော်ပြပေးမှာပဲဖြစ်ပါတယ်။ စမ်းသပ်အသုံးပြုလိုရင် အောက်ပါလိပ်စာမှာ ရယူနိုင်ပါတယ်။

https://github.com/jonathandelgado/SublimeTodoReview

အကယ်၍ စာဖတ်သူက Sublime Text 3 မသုံးပဲ အခြား Text Editor (သို့မဟုတ်) IDE တစ်ခုခုကို သုံးတယ်ဆိုရင် လည်း အလားတူလုပ်ဆောင်ချက်မျိုးရရှိနိုင်မယ့် နည်းလမ်းတွေကိုလေ့လာအသုံးပြုသင့်ပါတယ်။ တစ်ချို့ IDE တွေမှာ Build-in တစ်ခါတည်း ပါဝင်လာတက်ပေမယ့် တစ်ချို့ IDE တွေမှာတော့ Extension (သို့မဟုတ်) Plugin (သို့မဟုတ်) Package အနေနဲ့ ထပ်မံထည့်သွင်းရနိုင်ပါတယ်။

7.5 – Code Review

Team Collaboration နည်းစနစ်တွေထဲမှာ နောက်ဆုံးအနေနဲ့ ဖော်ပြချင်တဲ့ အချက်ကတော့ Code Review ပဲ ဖြစ်ပါ တယ်။ Developer တွေအနေနဲ့ Software Project တွေမှာမြင်ရတဲ့ Functional Feature တွေကို အရင်လုပ်ပြီး နောက် တော့မှ Security, Performance စတဲ့ Non-function တွေကို လိုက်လုပ်တက်ကြလေ့ရှိပါတယ်။ ပြဿနာတစ်ခုရဲ့ အဖြေရရှိရေးကိုသာ အလေးထားတက်ကြတဲ့သဘာဝရှိလို့ အဖြေတစ်ခုထွက်ရေးကိုသာ ဦးစားပေးတက်ကြပြီး ရေရှည်လုံခြုံ ရေးနဲ့ စွမ်းဆောင်ရည်ကိစ္စတွေကို နောက်မှ လိုက်လုပ်တက်ကြတာပါ။

အဲ့ဒီလိုအလေ့အထကြောင့် Software Project ကတော့ ပြီးသွားပေမယ့် Code ရဲ့ Performance က မကောင်းတာတို့၊ လုံခြုံရေးအားနည်းချက်တွေ ရှိနေတာတို့ ဖြစ်တက်ပါတယ်။ Security နဲ့ Performance ဆိုတာ တစ်ကယ့်တော့ စ ကတည်းက စနစ်တစ်ကျ ဆောင်ရွက်ရတဲ့လုပ်ငန်းတွေဖြစ်ပါတယ်။ အဲ့ဒီလုပ်ငန်းတွေကို နောင်မှ လိုက်လုပ်တဲ့အခါ စနစ် မကျတာ၊ မပြည့်စုံတော့ပဲကျန်ခဲ့တာတွေ ဖြစ်လာတက်ပါတယ်။ ဒီပြဿနာကို လျှော့ချဖို့အတွက် Code Review ဆိုတဲ့ နည်းစနစ်ကို သုံးကြပါတယ်။

Code Review ဆိုတာ လိုရင်းကတော့ Developer တစ်ဦးရေးထားတဲ့ Code ကို အခြား Developer တွေက ဝိုင်းဝန်း စီစစ်ကြခြင်းဖြစ်ပါတယ်။ တွေ့ရလေ့ရှိတဲ့ အမှားတွေရှိနေသလား၊ မသုံးသင့်တဲ့ Deprecated Syntax တွေသုံးထားသလား၊ Database Connection လို Resource တွေကို သုံးပြီးတဲ့အခါ ပြန်ပိတ်ရဲ့လား၊ Code Structure ကောင်းမွန်ရဲ့လား စသဖြင့် ဝိုင်းဝန်းစီစစ်ကြခြင်းဖြစ်ပါတယ်။ တစ်ချို့ပြဿနာတွေကို ဘေးလူကပို မြင်တက်တဲ့သဘာဝရှိလို့ မူလ Developer သတိမထားမိခဲ့တဲ့ အမှားတွေကို Code Review မှာ ရှာဖွေတွေ့ရှိပြီး ပြင်လိုက်နိုင်မယ်ဆိုရင် Code ရဲ့ အရည်အသွေးဟာ လည်း တိုးတက်လာမှာပဲဖြစ်ပါတယ်။

Code Quality အပြင် ဖြည့်စွက်ရရှိမယ့် အကျိုးကျေးဇူးကတော့ Knowledge Transfer ဖြစ်ပါတယ်။ Developer တစ်ဦးရဲ့ အလေ့အကျင့် ကောင်းတွေကို အခြား Developer တွေက Code Review Session တွေမှာ အပြန်အလှန် နမူနာယူလေ့လာခွင့် ရကြမှာဖြစ်ပါတယ်။ လူညီဖို့တော့လိုပါတယ်။ လူမညီရင်တော့ Code Quality ကောင်းဖို့ဆိုတဲ့ ရည်ရွယ်ချက်နဲ့လုပ်တဲ့ Code Review ကနေ၊ ကြည့်မရတဲ့ Developer ရဲ့ Code ကို တရားဝင်ပြစ်တင်ဝေဖန်ခွင့်ပေးလိုက်တဲ့ ရန်တိုက်ပေးတဲ့ပွဲဖြစ်သွားနိုင်ပါတယ်။ လိုရင်းဖြစ်တဲ့ Code Quality ကောင်းမွန်တိုးတက်ရေး ဆိုတဲ့ Objective ကို မူတည်ဆောင်ရွက်နိုင်ဖို့လိုပါတယ်။

Code Review နည်းလမ်း (၃) မျိုး ရှိပါတယ်။ Formal Review, Light Review နဲ့ Pair Programming တို့ပဲဖြစ် ပါတယ်။ Formal Review ဆိုတာကတော့ အစည်းအဝေးတွေခေါ်၊ Code တွေကို Print ထုတ်၊ စာရွက်ပေါ်မှာ တစ်ကြောင်းချင်း တစ်လိုင်းချင်းဖတ်ပြီး ဝိုင်းပြီးစီစစ်ကြခြင်း ဖြစ်ပါတယ်။ Fagan Inspection လို့ခေါ်တဲ့ Formal Code Review လုပ်ရာမှာ ကြိုတင်ပြင်ဆင်ပုံတွေ၊ Meeting ခေါ်ပုံတွေနဲ့ Review လုပ်ပုံတွေကို သတ်မှတ်ထားတဲ့ Code Review Procedure တစ်ခုရှိပါတယ်။ Light Review ဆိုတာကတော့ အဲ့ဒီလောက် တင်းတင်းကြပ်ကြပ် မဟုတ်ပဲ အလျှင်း သင့်သလို တစ်ဦးရေးထားတဲ့ Code ကို နောက်တစ်ဦးက စီစစ်ပေးခြင်းဖြစ်ပါတယ်။ Light Review အနေနဲ့ Code Review Tool တွေကိုလည်း သုံးတက်ကြပါတယ်။ Code Review Tool ဆိုတာ လူတစ်ယောက်က Code ဖိုင်ကို ဖွင့်ဖတ်စီစစ်ခြင်းမဟုတ်ပဲ ကွန်ပျူတာပရိုဂရမ်တစ်ခုနဲ့ တွေ့ရလေ့ရှိတဲ့ အမှားတွေကို အလိုအလျောက်စီစစ်ခြင်းဖြစ်ပါတယ်။ Static Code Analysis လို့လည်းခေါ်ပါတယ်။ JavaScript အတွက်ဆို ရင် JSLint, JSHint စတဲ့ Code Quality Tool တွေရှိသလို Closure Compiler လို Code Analysis Tool တွေ လည်း ရှိပါတယ်။

Pair Programming

Pair Programming ဆိုတာ အခြား Agile Development နည်းစနစ်တစ်ခုဖြစ်တဲ့ Extreme Programming ကနေ လာတဲ့ နည်းစနစ်ဖြစ်ပါတယ်။ ကွန်ပျူတာတစ်လုံးကို Programmer နှစ်ဦးက တစ်ပြိုင်တည်းသုံးခြင်းအားဖြင့် Code ရေးတဲ့နည်းစနစ် ဖြစ်ပါတယ်။ တစ်ယောက်က ထိုင်ရေးပြီး၊ နောက်တစ်ယောက်က Review လုပ်တဲ့ သဘောမျိုးနဲ့ ထိုင် ကြည့်ရပါတယ်။ Code ရေးသူကို Driver လို့ခေါ်ပြီး၊ ဘေးက Review လုပ်သူကို Navigator လို့ခေါ်ပါတယ်။

ရေးတဲ့သူက သတ်မှတ်လုပ်ဆောင်ချက် ရရှိအောင် ရေးသားနေချိန်မှာ၊ Navigator က နောက်ဆက်လုပ်ရမယ့် အဆင့် အတွက် ကြိုတင်စဉ်းစားထားနိုင်ပါတယ်၊ လက်ရှိရေးနေတဲ့အပိုင်းကို ပိုကောင်းသွားအောင်လုပ်လို့ရ မယ့် နည်းလမ်းတွေကို စဉ်းစားထားနိုင်ပါတယ်။ ဒီနည်းနဲ့ Code ရေးနေတဲ့ Driver က ရေးနေတာကို ရပ်ပြီး နောက်အဆင့်အတွက် စဉ်းစားရ တာတို့၊ သေချာရဲ့လား တစ်ခေါက်ပြန်စစ်ရတာတို့ လုပ်စရာမလိုတော့ပဲ

လက်ရှိ Code ကို အပြည့်အဝ အာရုံစိုက်နိုင်သွား ပါတယ်။ မှားစရာရှိမယ်ဆိုရင်လည်း မမှားခင်ကတည်းက Navigator က တွေ့ရှိထောက်ပြနိုင်တဲ့အတွက် စမ်းကြည့်ပြီးမှား နေလို့ နောက်တစ်ခေါက် ပြန်ပြင်လိုက်ရတယ် ဆိုတဲ့ အလုပ်အတွက် အချိန်ကုန်စရာမလိုတော့ပါဘူး။ Code Quality တိုးတက်သွားတဲ့အပြင် ပိုပြီးတော့လည်း အလုပ်တွင်သွားပါတယ်။ ကွန်ပျူတာနှစ်လုံးနဲ့ နှစ်ယောက်ခွဲ ပြီးရေးတာထက် Pair Programming နည်းလမ်း က ရေရှည်မှာ အလုပ်ပိုတွင်တက်ပါတယ်။

Pair Programming ဟာ အလေ့အကျင့်ကောင်းတစ်ခုဖြစ်ပေမယ့် လက်တွေ့အကောင်အထည်ဖော်ဖို့တော့ နည်း နည်း ခက် ပါတယ်။ တွဲဖက်ရေးသားမယ့် Developer နှစ်ဦးရဲ့ အရည်အချင်း Level က ညီနေဖို့လိုနိုင်ပါတယ်။ မ ညီရင်မရဘူးတော့ မဟုတ်ပါဘူး။ မညီရင် ပိုတော်တဲ့သူက Driver လုပ်ရမလဲ၊ နောက်တစ်ယောက်က Driver လုပ်ရမလား၊ ဘယ်ဟာ ပို အကျိုးရှိမလဲဆိုတာမျိုးတွေ စဉ်းစားရပါတော့မယ်။ နောက်ပြီးတော့ Developer ဆို တာမျိုးက တစ်ကိုယ်တော်သမားတွေ ရှိတက်ပါတယ်။ တစ်ယောက်ထဲ ခေါင်းအေးအေးနဲ့လုပ်ရမှ အာရုံပိုရ တယ်ဆိုတာမျိုးလည်း ရှိနိုင်ပါတယ်။ အဲ့လိုလူမျိုးကို Pair လုပ်ပေးလိုက်ရင် သူ့အတွက်သက်တောင့်သက်သာ မ ဖြစ်တက်အတွက် ပိုပြီးအလုပ်မတွင် ဖြစ်သွားတက်ပါသေး တယ်။ အခြေအနေနဲ့ ချိန်ညှိပြီး သင်တော်ရင် အသုံးပြုကြဖို့ဖြစ်ပါတယ်။

Conclusion

ဖော်ပြခဲ့တဲ့ Collaboration နည်းစနစ်တွေကို တစ်ခုမကျန် အကုန်သုံးဖို့တော့ မဟုတ်ပါဘူး။ ကိုယ့် Team ရဲ့ အနေ အထားနဲ့ သင့်တော်သလို ချိန်ညှိအသုံးပြုကြရမှာဖြစ်ပါတယ်။ ဥပမာ – နေရာအတည်တစ်ကျမရှိပဲ အင် တာနက်အကူအညီ နဲ့ အလုပ်လုပ်ကြတဲ့ Remote Team တွေမှာဆိုရင် SCRUM တို့ Pair Programming တို့ကို အသုံးချနိုင်ဖို့ မလွယ် ပါဘူး။ Issue Tracking System ကိုသာ အားကိုးပြီးလုပ်ကြရမှာပါ။ သိပ်ကြီးတဲ့ အဖွဲ့ အစည်းကြီးတွေမှာဆိုရင်တော့ ဖော်ပြခဲ့တဲ့ နည်းစနစ်အားလုံးကို တစ်နေရာထဲမှာ စုပေါင်းပေးထားတဲ့ Project Management Software တွေကို အသုံး ပြုရနိုင်ပါတယ်။ တစ်ဦးနှစ်ဦးပဲပါတဲ့ Team လေးတွေမှာဆိုရင်တော့ အခြားနည်းစနစ်တွေမလိုပဲ Issue Tracking နဲ့ Task List လောက်နဲ့တင် လုံလောက်နိုင်ပါတယ်။

အဆုံးသတ်အနေနဲ့ ဒီအခန်းရဲ့အစမှာ ဖော်ပြခဲ့တဲ့ Agile Team တစ်ခုရဲ့ လုပ်ငန်းစဉ်ကို နောက်တစ်ခေါက် ပြန်လည်ဖော်ပြ လိုက်ပါတယ်။

1. Software Project တစ်ခု စတင်တော့မယ်ဆိုတာနဲ့ Project Manager (သို့မဟုတ်) Product Owner က **Functional Specification** တစ်ခုကို စတင်ရေးသားပါတယ်။
2. Specification ပေါ်အခြေခံပြီး **Project Schedule** ကိုရေးဆွဲထားရပါတယ်။
3. Specification မှာပါဝင်တဲ့ **User Story** တွေကိုထုတ်ယူပြီး **Product Backlog** ဖန်တီးပါတယ်။
4. ပြီးတဲ့အခါ Iteration တစ်ခုစတင်နိုင်ဖို့အတွက် **Sprint Planning Meeting** ခေါ်ပြီး **Sprint Backlog** ကို ဖန်တီးပါတယ်။
5. Development Team က Sprint Backlog မှာပါဝင်တဲ့ User Story တွေပေါ်မှာအခြေခံပြီး Implementation Detail အတွက် **Task List** ကို တည်ဆောက်ပါတယ်။
6. Sprint Backlog နဲ့ Task List ကို အခြေခံပြီး လိုအပ်တဲ့ Code တွေကို စတင်ရေးသားပါတယ်။ ဒီလိုရေးသားရာမှာ Code Quality ကောင်းစေဖို့အတွက် Code Review နည်းစနစ်တစ်ခုဖြစ်တဲ့ Pair Programming ကို အသုံးပြုပါတယ်။
7. တည်ဆောက်ရရှိလာတဲ့ ရလဒ်မှာတွေ့ရှိတဲ့ Bug နဲ့ Feature Request တွေကို **Issue Tracking System** ထဲမှာ မှတ်တမ်းတင်ပါတယ်။
8. Project Manager (သို့မဟုတ်) Product Owner က Issue Tracking System ထဲမှာ ရှိနေတဲ့ Bug နဲ့ Feature Request တွေရဲ့အခြေအနေနဲ့ချိန်ဆပြီး Specification နဲ့ Schedule ကို Update လုပ်ပါတယ်။
9. **Up-to-date Schedule** နဲ့ချိန်ညှိပြီး Product Backlog ကို ပြန်လည်စီစဉ်ပါတယ်။
10. ပြီးတဲ့အခါ အမှတ်စဉ် (၄) ကို ပြန်သွားပြီး နောက် Iteration တစ်ခုကို ပြန်လည်စတင်ပါတယ်။

Software Project စီမံရာမှာ အရေးပါတဲ့ နောက်ထပ်နည်းစနစ်တွေကိုတော့ Development Workflow Automation ခေါင်းစဉ်နဲ့ နောက်တစ်ခန်းမှာ ဆက်လက်ဖော်ပြသွားပါမယ်။

Automate လုပ်ထားလို့ရတဲ့နိုင်တဲ့ကိစ္စတစ်ခုကို ထပ်ခါထပ်ခါ
Manual လုပ်နေရင် အချိန်တွေ၊ လုပ်အားတွေ ဖြုန်းတီးတာပါပဲ။
ထိရောက်အလုပ်တွင်မှာလည်းမဟုတ်ပါဘူး။

Professional Web Developer (စာအုပ်)
Web Standard, jQuery, PHP, MySQL, Ajax, CMS, MVC,
HTML5, Mobile Web, Web Application Security စသည့်
အကြောင်းအရာများကို ရေးသားဖော်ပြထားသည့်စာအုပ်
အောက်ပါလိပ်စာတွင် အခမဲ့ Download ရယူနိုင်သည်။

http://pwdbook.com

အခန်း(၈) – Development Workflow Automation

Software Project တစ်ခုမှာ Code ရေးသားခြင်းနဲ့အတူ Developer တွေ ပုံမှန်တွဲဖက်လုပ်ကြရလေ့ရှိတဲ့ လုပ်ငန်း (၄) ခုရှိ ပါတယ်။ အဲ့ဒါတွေကတော့ –

1. Scaffolding
2. Package Management
3. Build နဲ့
4. Deployment – တို့ပဲဖြစ်ပါတယ်။

8.1 – Scaffolding

Software Project တစ်ခုကို စတင်ရေးသားတော့မယ်ဆိုတာနဲ့ ပထမဆုံးလုပ်ရတဲ့အလုပ်က၊ လိုအပ်တဲ့ Directory Structure နဲ့ Source Code ဖိုင်တွေ တည်ဆောက်ရခြင်းပဲ ဖြစ်ပါတယ်။ လုပ်ဆောင်ချက် ထွေထွေထူးထူးမပါတဲ့ ရိုးရိုး Web Page လေးတစ်ခု ဖန်တီးတော့မယ်ဆိုရင်တောင် အခုလို Structure တော့ အနည်းဆုံး လိုအပ်မှာပဲဖြစ်ပါတယ်။

```
WebPage
├── css
│   ├── responsive.css
│   └── style.css
├── img
│   └── logo.png
├── js
│   ├── app.js
│   └── jquery.js
└── index.html
```

Directory Structure တည်ဆောက်ပြီးတဲ့အခါ အခြေခံ Code Structure ကို ဆက်လက်တည်ဆောက်ရပါတယ်။ HTML Document တိုင်းမှာ အနည်းဆုံး အခုလို Structure တော့ ပါဝင်ရလေ့ရှိပါတယ်။

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document Title</title>
  <link rel="stylesheet" href="css/style.css">
</head>
```

```
 8. <body>
 9.
10.   <script src="js/jquery.js"></script>
11.   <script src="js/app.js"></script>
12. </body>
13. </html>
```

အလားတူပဲ CSS ဖိုင်တစ်ခုမှာလည်း အခုလို Normalize Style က အမြဲပါရတက်ပါတယ်။

CSS

```
 1. html, body {
 2.   margin: 0;
 3.   padding: 0;
 4. }
 5.
 6. article, aside, details, figcaption, figure, footer,
 7. header, hgroup, main, menu, nav, section, summary {
 8.     display: block;
 9. }
10.
11. a:active, a:hover {
12.     outline: 0;
13. }
14.
15. img {
16.     border: 0;
17. }
18.
19. table {
20.     border-collapse: collapse;
21.     border-spacing: 0;
22. }
```

ဒီလို အမြဲပါရတဲ့ အခြေခံ Directory Structure နဲ့ Code Structure တည်ဆောက်ခြင်းလုပ်ငန်းကို Scaffolding လို့ ခေါ်ပါတယ်။ Project အသစ်တစ်ခုစတိုင်း Scaffolding လုပ်ငန်းကို တစ်ခုချင်း ကိုယ်တိုင်လိုက်လုပ်နေရရင် အချိန်တွေ ကုန်ပါတယ်။ ဒီလိုအချိန်မကုန်စေဖို့ဆိုရင် Boilerplate Code တွေကို သုံးနိုင်ပါတယ်။

Boilerplate Code ဆိုတာ အခြေခံအားဖြင့်ပါဝင်လေ့ရှိတဲ့ Code တွေကို ကြိုတင်ရေးသားထားတဲ့ Source Code တစ်မျိုးဖြစ်ပါတယ်။ Project တစ်ခုစတော့မယ်ဆိုရင် Directory Structure နဲ့ အခြေခံ Code Structure ကို ကိုယ်တိုင် ရေးနေမယ့်အစား Boilerplate Code ကို အသင့်ယူအသုံးပြုလိုက်နိုင်တဲ့သဘော ဖြစ်ပါတယ်။ Web Development အတွက် ထင်ရှားတဲ့ Boilerplate နှစ်ခုရှိပါတယ်။ HTML5 Boilerplate နဲ့ Google ကပေးထားတဲ့ Web Starter Kit တို့ပဲဖြစ်ပါတယ်။

HTML5 Boilerplate ကို html5boilerplate.com မှာ Download ရယူနိုင်ပါတယ်။ Download ရယူထားတဲ့ Archive ဖိုင်ကို Extract လုပ်ကြည့်လိုက်ရင် အခုလို Directory Structure ကို တွေ့ရမှာဖြစ်ပါတယ်။

```
HTML5-boilerplate
├── css
│   ├── main.css
│   └── normalize.css
├── img
├── js
│   ├── vendor
│   │   ├── jquery-1.11.2.min.js
│   │   └── modernizr-2.8.3.min.js
│   ├── main.js
│   └── plugins.js
├── .htaccess
├── .gitignore
├── 404.html
├── favicon.ico
└── index.html
```

Web App Project တစ်ခုအတွက် အခြေခံအားဖြင့် လိုအပ်တဲ့ Directory Structure နဲ့ Code ဖိုင်တွေကို အသင့်တည်ဆောက်ထားပေးခြင်းဖြစ်ပါတယ်။ Web App Project တိုင်းမှာ လိုအပ်လေ့ရှိတဲ့ normalize.css, jQuery, Modernizr တို့လို Library တွေသာမက .htaccess, .gitignore စတဲ့ Setting ဖိုင်တွေကိုပါ တစ်ခါတည်း ထည့်သွင်းပေးထားပါတယ်။ index.html ကို ဖွင့်ပြီးလေ့လာကြည့်ရင် အခုလိုတွေ့ရမှာ ဖြစ်ပါတယ်။

HTML

```
<!doctype html>
<html class="no-js" lang="">
  <head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <title></title>
    <meta name="viewport" content="width=device-width, initial-scale=1">

    <link rel="stylesheet" href="css/normalize.css">
    <link rel="stylesheet" href="css/main.css">
    <script src="js/vendor/modernizr-2.8.3.min.js"></script>
  </head>
  <body>
    <!--[if lt IE 8]>
      <p class="browserupgrade">You are using an <strong>outdated</strong>
      browser. Please <a href="http://browsehappy.com/">upgrade your
      browser</a> to improve your experience.</p>
    <![endif]-->

    <p>Hello world! This is HTML5 Boilerplate.</p>

    <script src="//ajax.googleapis.com/ajax/libs/jquery.min.js"></script>
    <script>window.jQuery || document.write('<script
            src="js/vendor/jquery-1.11.2.min.js"><\/script>')</script>
    <script src="js/plugins.js"></script>
    <script src="js/main.js"></script>

    <script>
      (function(b,o,i,l,e,r){b.GoogleAnalyticsObject=l;b[l]||(b[l]=
      function(){(b[l].q=b[l].q||[]).push(arguments)});b[l].l=+new Date;
```

```
27.       e=o.createElement(i);r=o.getElementsByTagName(i)[0];
28.       e.src='//www.google-analytics.com/analytics.js';
29.       r.parentNode.insertBefore(e,r)}(window,document,'script','ga'));
30.       ga('create','UA-XXXXX-X','auto');ga('send','pageview');
31      </script>
32.   </body>
33. </html>
```

Web App တစ်ခုအတွက် အခြေခံ Code Structure ကို အသင့်ရေးသားထားပေးခြင်း ဖြစ်ပါတယ်။ ရေးထားတဲ့ Code ကိုလေ့လာကြည့်ရင်၊ လိုင်းနံပါတ် (၇) မှာ Mobile Device တွေမှာ အဆင်ပြေပြေဖော်ပြစေဖို့အတွက် လိုအပ်တဲ့ Viewport Meta Element ကို အသင့်ထည့်ထားပေးတာကို တွေ့ရမှာဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၁၄) မှာ Internet Explorer 8 နဲ့ သူ့ရှေ့ပိုင်း Browser အဟောင်းတွေကနေ အသုံးပြုရင် Browser ကို Upgrade လုပ်ဖို့ သတိပေးတဲ့ Message ကိုလည်း တစ်ခါတည်း ထည့်သွင်းပေးထားပါတယ်။ လိုင်းနံပါတ် (၂၀) နဲ့ (၂၁) ကို လေ့လာကြည့်ရင် jQuery Library ကိုချိတ်ဆက်ရာမှာ Google CDN က jQuery ကို အရင်ချိတ်ဆက်ပြီး မရတော့မှ Local jQuery ကို ချိတ်ဆက်တဲ့ လုပ်ဆောင်ချက်ကို ရေးသားထားပေးပါတယ်။ လိုင်းနံပါတ် (၂၅) ကနေ (၃၁) ထိမှာ Google Analytic Tracking Code ကိုလည်း တစ်ခါတည်း ထည့်သွင်းထားပေးပါသေးတယ်။ Web App Project တစ်ခုမှာ လိုအပ်တဲ့ အခြေခံ Code ကို တစ်ခါတည်း အသင့်ရေးထားပေးခြင်း ဖြစ်ပါတယ်။

ဒါ index.html တစ်ခုပဲကြည့်ရပါသေးတယ်။ .htaccess ဖိုင်ထဲမှာဆိုရင်လည်း Application Performance နဲ့ လုံခြုံရေး ကောင်းစေဖို့အသုံးပြုရတဲ့ Apache Setting တွေ အသင့်ရေးသားထားပေးပါတယ်။ plugin.js ထဲမှာလည်း Console လုပ်ဆောင်ချက်မပါဝင်တဲ့ Browser တွေမှာ console.log သုံးပြီး Test Code တွေထည့်ရေးမိတဲ့ အခါ Error မတက်အောင် ဖြေရှင်းပေးတဲ့ Code ကို အသင့်ရေးသားထားပေးပါတယ်။ Boilerplate ဆိုတာ ဒါပါပဲ။ အခြေခံ လိုအပ်ချက်ဖြစ်တဲ့ Directory Structure နဲ့ Source Code တွေကို အသင့်ကြိုတင်ရေးသားထားခြင်းပဲဖြစ်ပါတယ်။ ဒါကြောင့် နောင် Web App Project တစ်ခု စတော့မယ်ဆိုတိုင်း Directory Structure တစ်ခုကို ကိုယ်တိုင်ဆောက် မနေပဲ HTML5 Boilerplate ကို Download ရယူခြင်းအားဖြင့် အသင့်ရရှိနိုင်မှာပဲဖြစ်ပါတယ်။

Google Web Starter Kit ကိုတော့ developers.google.com/web/starter-kit/ မှာ Download ရယူနိုင်ပါတယ်။ ရရှိလာတဲ့ Archive ဖိုင်ကို Extract လုပ်ကြည့်လိုက်ရင် အခုလိုတွေ့ရမှာဖြစ်ပါတယ်။

```
web-starter-kit
├── app
│   ├── fonts
│   ├── images
│   ├── scripts
│   │   └── main.js
│   ├── styles
│   │   ├── components
│   │   ├── _h5bp.scss
│   │   └── main.scss
│   ├── basic.html
│   ├── favicon.ico
│   ├── index.html
│   ├── manifest.json
│   ├── manifest.webapp
```

```
├── gulpfile.js
└── package.json
```

အမှန်တော့ Web Starter Kit က HTML5 Boilerplate ကိုပဲ Mobile Device တွေနဲ့အဆင်ပြေစေဖို့အတွက် Responsive Design လုပ်ဆောင်ချက် ထပ်မံဖြည့်စွက်ပေးထားခြင်းဖြစ်ပါတယ်။ ဒါ့အပြင် Performance ကောင်းစေဖို့ အတွက် Google Page Speed လုပ်ဆောင်ချက်၊ ရေးသားထားတဲ့ App ကို စမ်းသပ်ရာမှာ ပိုမို အဆင်ပြေစေဖို့အတွက် Build-in Server နဲ့ အသင့်သုံးနိုင်တဲ့ CSS Component တွေဖြည့်စွက်ပေးထားပါသေးတယ်။ ပိုပြီးအခြေခံကျတဲ့ Web App တစ်ခုတည်ဆောက်လိုရင် HTML5 Boilerplate မူရင်းကို သုံးသင့်ပြီး၊ Mobile Device တွေမှာ အလုပ်လုပ်ဖို့ အဓိက ရည်ရွယ်တယ်ဆိုရင်တော့ Web Starter Kit ကို အသုံးပြုသင့်ပါတယ်။

Boilerplate တွေဟာ Software Project တစ်ခုကို အလျှင်အမြန်အစပြုနိုင်စေဖို့ ကူညီပေးတဲ့ Tool တွေဖြစ်ပါတယ်။ ဒါပေမယ့် လက်တွေ့မှာ Boilerplate ရရှိဖို့အတွက် သက်ဆိုင်ရာ Website ကိုသွားပြီး Download လုပ်နေရပါသေးတယ်။ ဒီထက်ပိုပြီး အလုပ်တွင်ဖို့ဆိုရင် Command တစ်ချို့ Run လိုက်တာနဲ့ Boilerplate ကို အလိုအလျောက်တည် ဆောက်ပေးနိုင်တဲ့ Scaffold Generator (သို့မဟုတ်) Code Generator တွေကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။

Code Generator တွေကပေးတဲ့ အားသာချက်ကတော့ Project ကို အစပြုနိုင်တဲ့ Boilerplate ကို တည်ဆောက်ပေးနိုင် ယုံသာမက၊ Code ရေးသားနေရင်း လိုအပ်တဲ့ Directory Structure ကိုလည်း ဖြည့်စွက်တည်ဆောက်ပေးနိုင်ကြသလို၊ အခြားလိုအပ်တဲ့ Code တွေကိုလည်း အလိုအလျောက် ရေးပေးနိုင်ပါသေးတယ်။ ဥပမာ – MVC Pattern ကို အသုံး ပြုထားတဲ့ Project တွေမှာ ဆိုရင် လုပ်ဆောင်ချက်တစ်ခု ထည့်သွင်းတိုင်းမှာ Model ဖိုင်၊ View ဖိုင် နဲ့ Controller ဖိုင် တို့ကို တည်ဆောက်ပေးဖို့လိုတက်ပါတယ်။ လုပ်ဆောင်ချက်အသစ်တစ်ခု ထည့်သွင်းလိုတိုင်း ဒီဖိုင်တွေကို ကိုယ်တိုင် တည် ဆောက်နေမယ့်အစား Code Generator ကို တည်ဆောက်ခိုင်းလိုက်နိုင်ခြင်းဖြစ်ပါတယ်။ ဒါအပြင် Database Table အမည်ပေးလိုက်ယုံနဲ့ အခြေခံ CRUD လုပ်ငန်းကို အလိုအလျောက်ရေးပေးတဲ့ လုပ်ဆောင်ချက်မျိုးတွေလည်း ပါဝင်တက် ပါသေးတယ်။

MVC နဲ့ CRUD အကြောင်းကိုဒီနေရာမှာ အကျယ်မချဲ့တော့ပါဘူး။ **Professional Web Developer** ရဲ့ **အခန်း (၉)** နဲ့ **အခန်း (၁၃)** တို့မှာ လေ့လာနိုင်ပါတယ်။

Ruby on Rails, CakePHP, Django, Symfony, ASP.NET MVC, ExpressJS စတဲ့ Framework တွေမှာ Scaffold Generator တွေ တစ်ခါတည်း အသင့်ပါဝင်ပါတယ်။ သူတို့မှာပါတဲ့ Generator တွေကတော့ Boilerplate တည်ဆောက် ဖို့ထက်စာရင် MVC Structure နဲ့ CRUD လုပ်ဆောင်ချက်တို့ကို အဓိကဆောင်ရွက်ပေးကြခြင်းဖြစ်ပါတယ်။ အခုနောက် ပိုင်း Modern Web App တွေမှာ Yeomen လို့ခေါ်တဲ့ Scaffold Generator ကို အသုံးများကြပါတယ်။ Yeomen ကို တော့ Boilerplate တည်ဆောက်ဖို့ရော၊ ဖြည့်စွက်လိုအပ်ချက်တွေအတွက်ပါ Code Generator အဖြစ်သုံးနိုင်ပါတယ်။

8.2 – Yeomen

Yeomen ဟာ NodeJS ကို အသုံးပြုရေးသားထားတဲ့ Scaffold Generator ဖြစ်ပါတယ်။ JavaScript Web App တွေ အတွက် အသုံးများပေမယ့် Code Generator လိုအပ်တဲ့ မည်သည့် Project အမျိုးအစားအတွက်မဆို အသုံးပြုနိုင်ပါတယ်။ Yeomen ကို အသုံးပြုနိုင်ဖို့အတွက် NodeJS လိုပါတယ်။ NodeJS Installer ကို nodejs.org မှာ Download ရယူနိုင်ပါတယ်။ Windows, Mac, Linux မိမိအသုံးပြုလိုရာ OS အတွက် Installer ကို ရွေးချယ်ရယူနိုင်ပါတယ်။ Ubuntu Linux မှာဆိုရင်တော့ ကိုယ်တိုင် Download သွားလုပ်မနေပဲ အခုလို Install လုပ်နိုင်ပါတယ် –

```
$ sudo apt-get install nodejs nodejs-legacy
```

NodeJS နဲ့အတူ NPM လို့ခေါ်တဲ့ NodeJS Package Manager တစ်ခု တစ်ခါတည်း တွဲဖက်ပါဝင်ပါတယ်။ NPM ဆိုတာ NodeJS Software Package တွေကို ရှာဖွေခြင်း၊ Install ပြုလုပ်ခြင်း၊ Update ပြုလုပ်ခြင်း စတဲ့လုပ်ငန်းတွေကို ဆောင်ရွက်စီမံပေးတဲ့ Software တစ်မျိုးပဲဖြစ်ပါတယ်။ Yeoman ဟာ NodeJS နဲ့ရေးသားထားတဲ့ Software Package တစ်ခုဖြစ်တဲ့အတွက် NPM ကိုသုံးပြီး Yeoman ကို အခုလို Install လုပ်ယူနိုင်ပါတယ် –

```
$ sudo npm install -g yo
```

NPM ဟာ Command Line Tool တစ်ခုဖြစ်တဲ့အတွက် Terminal (သို့မဟုတ်) Windows Command Line ကနေတစ်ဆင့် အသုံးပြုရခြင်းဖြစ်ပါတယ်။ Ubuntu Linux မှာသာ ရှေ့ဆုံးက `sudo` Command ထည့်ရပြီး Windows မှာ ဆိုရင်တော့ `sudo` ထည့်စရာမလိုပါဘူး။ `sudo` အကြောင်းအသေးစိတ်သိရှိလိုရင်တော့ **Ubuntu – သင့်အတွက် Linux ရဲ့ အခန်း (၁၀)** မှာ လေ့လာနိုင်ပါတယ်။

`npm install yo` လို့ပြောလိုက်တာနဲ့ NPM က Yeoman Scaffold Generator ကို Download ယူပြီး Install လုပ်ပေးသွားမှာပါ။ အဲ့ဒီလို Install လုပ်တဲ့အခါ လက်ရှိ Directory ထဲမှာ Install လုပ်သွားမှာဖြစ်လို့၊ လက်ရှိ Directory ထဲမှာပဲ အသုံးပြုလို့ရမယ်ဆိုတဲ့ သဘောလည်းဖြစ်ပါတယ်။ နောင်ဘယ်နေရာကနေမဆို အသုံးပြုလိုတယ်ဆိုရင် NodeJS Global Directory ထဲမှာ Install လုပ်သွားဖို့လိုပါတယ်။ `-g` Option က ဒီရည်ရွယ်ချက်နဲ့ ထည့်သွင်းထားတာပါ။ `-g` Option သုံးထားတဲ့အတွက် NPM က Yeoman ကို NodeJS Global Directory ထဲမှာ Install လုပ်သွားပြီး၊ နောင် အသုံးပြုလိုရင် ထပ်မံ Install လုပ်စရာမလိုတော့ပဲ အသင့်သုံးနိုင်သွားမှာပါ။

> ဒီအခန်းမှာ ဆက်လက်ဖော်ပြမယ့် Command တွေဟာ လက်တွေ့ကူးယူစမ်းသပ်နိုင်တဲ့ Command တွေ ဖြစ်ပါတယ်။ ဒါပေမယ့် အသုံးပြုထားတဲ့ Operating System, Install လုပ်ထားတဲ့ Tool တွေရဲ့ Version နဲ့ File Permission သတ်မှတ်ထားပုံ အခြေအနေပေါ်မူတည်ပြီး အပြောင်းအလဲ ရှိနိုင်တာကို သတိပြုစေလိုပါတယ်။ လက်တွေ့စမ်းသပ်ရင်း ပြဿနာတစ်စုံတစ်ရာရှိလာတဲ့အခါ ဖော်ပြလာတဲ့ Error Message များနဲ့ Instruction များကို ကိုယ်တိုင်လေ့လာပြီး ဖြေရှင်းဖို့ လိုအပ်နိုင်ပါတယ်။

လက်တွေ့မှာ Scaffold Generator တစ်ခုတည်းဆိုရင် မပြည့်စုံပါဘူး။ Development Workflow တစ်ခုလုံးကို Auto-mate လုပ်နိုင်ဖို့အတွက် Package Manager တွေ Build Tool တွေ လိုပါသေးတယ်။ လက်စနဲ့ လိုအပ်တဲ့ Tool တွေ အကုန်လုံးကို တစ်ခါတည်း Install လုပ်ထားသင့်ပါတယ်။

```
$ sudo npm install -g yo bower grunt-cli
```

`yo`, `bower` နဲ့ `grunt-cli` ဆိုတဲ့ Tool သုံးခုကို တစ်ခါတည်း Install လုပ်လိုက်ခြင်းဖြစ်ပါတယ်။ Bower နဲ့ Grunt အကြောင်းကို မကြာခင်ဆက်လက်ဖော်ပြပါမယ်။ လောလောဆယ် Yeoman အကြောင်းကိုပဲ ဆက်ပါဦးမယ်။ Yeoman ကိုသုံးပြီး App Boilerplate တွေ Generate လုပ်နိုင်ဖို့အတွက် Generator တွေကို ဆက်လက် Install လုပ်ပေးရပါဦးမယ်။ Generator စာရင်းကို yeoman.io/generators မှာ လေ့လာနိုင်ပါတယ်။ စာရင်းထဲက အသုံးပြုလိုရာ Generator တစ်ချို့ကို ကြိုတင် Install လုပ်ထားသင့်ပါတယ်။ ဥပမာ –

```
$ sudo npm install -g generator-webapp generator-backbone
```

Web App Generator နဲ့ BackboneJS App Generator တို့ကို Install လုပ်လိုက်ခြင်းပဲဖြစ်ပါတယ်။ Generator တွေ Install လုပ်ပြီးပြီဆိုရင် Yeoman ကိုသုံးပြီး Boilerplate တွေ စတင် Generate လုပ်နိုင်ပြီပဲ ဖြစ်ပါတယ်။ အခုနေ `yo` Command ကို Run ကြည့်ရင် ဒီလိုရလဒ်ကို ရရှိမှာဖြစ်ပါတယ်။

```
$ yo

? 'Allo Ei! What would you like to do? (Use arrow keys)
  Run a generator
  Backbone
❯ Webapp
  Mocha
  ──────────────
  Update your generators
  Install a generator
(Move up and down to reveal more choices)
```

`yo` Command က Install လုပ်ထားတဲ့ Generator စာရင်းကို ဖော်ပြပေးခြင်းဖြစ်ပါတယ်။ အသုံးပြုလိုတဲ့ Generator ကိုရွေးပြီး Generate လုပ်နိုင်ပါတယ်။ နမူနာအနေနဲ့ `Webapp` ကိုရွေးပြီး Enter နှိပ်လိုက်ရင် အခုလို ရလဒ်ကိုရရှိမှာပါ။

```
Make sure you are in the directory you want to scaffold into.
This generator can also be run with: yo webapp

     _-----_
    |       |    .--------------------------.
    |--(o)--|    |    Welcome to Yeoman,    |
   `---------´   |   ladies and gentlemen!  |
    ( _´U`_ )    '--------------------------'
    /___A___\
     |  ~  |
   __'.___.'__
 ´   `  |° ´ Y `

Out of the box I include HTML5 Boilerplate, jQuery, and a Gruntfile.js to
build your app.
? What more would you like? (Press <space> to select)
❯◉ Bootstrap
 ○ Sass
 ◉ Modernizr
```

Message ရဲ့ ပထမဆုံးလိုင်းမှာ၊ Source Code ဖိုင်တွေ စတင်တည်ဆောက်တော့မှာမို့ Project Directory ကို မှန်အောင်ရွေးထားရဲ့လားလို့ သတိပေးတာကို တွေ့ရမှာဖြစ်ပါတယ်။ အကယ်၍ Project Directory ကို မရွေးရသေးရင် ဒီနေရာတင်ရပ်ပြီး Project Directory ကို `cd` Command နဲ့ အရင်ရွေးပေးဖို့လိုပါတယ်။

ဆက်လက်ပြီး၊ နောင်တစ်ချိန် Web App Boilerplate တစ်ခု Generate လုပ်လိုရင် `yo webapp` Command နဲ့ တိုက်ရိုက် Generate လုပ်နိုင်ကြောင်းကိုလည်း Message ထဲမှာ ဖော်ပြထားပါသေးတယ်။ Web App နဲ့အတူ HTML5 Boilerplate, jQuery နဲ့ Setting ဖိုင်တစ်ခုဖြစ်တဲ့ Gruntfile.js တို့ကို တစ်ခါတည်းထည့်ပေးထားကြောင်းလည်း ဖော်ပြထားပါသေးတယ်။ ဆက်လက်ဖြည့်စွက်လိုတဲ့ Library ရှိရင်လည်း ဖော်ပြထားတဲ့ စာရင်းထဲကနေ ရွေးပြီး ထည့်နိုင်ပါ တယ်။ နမူနာမှာ Bootstrap, Sass နဲ့ Modernizr ဆိုပြီး ရွေးစရာ (၃) ခုပေးထားပါတယ်။

နမူနာအနေနဲ့ Bootstrap CSS Framework နဲ့ Modernizr JavaScript Library တို့ကို ရွေးထားပါတယ်။ Enter နှိပ်ပြီး နောက်တစ်ဆင့်ကို သွားတဲ့အခါ Yeoman က လိုအပ်တဲ့ Directory Structure နဲ့ Source Code ဖိုင်တွေကို စတင် တည်ဆောက်ပါတော့တယ်။

```
? What more would you like? Bootstrap, Modernizr
   create Gruntfile.js
   create package.json
   create .gitignore
   create .gitattributes
   create .bowerrc
   create bower.json
   create .jshintrc
   create app/styles/main.css
   create app/favicon.ico
   create app/robots.txt
   create app/index.html
   create app/scripts/main.js
   invoke   mocha
```

ဆက်လက်ပြီး `bower install` နဲ့ `npm install` ဆိုတဲ့ Command နှစ်ခုကို အလိုအလျောက် Run ပေးသွားပါ တယ်။

```
I'm all done. Running bower install & npm install for you to install the
required
dependencies. If this fails, try running the command yourself.

   create      test/bower.json
   create      test/.bowerrc
   create      test/spec/test.js
   create      test/index.html
bower chai#~1.8.0      cached git://github.com/chaijs/chai.git#1.8.1
bower chai#~1.8.0      validate 1.8.1 against git://github.com/chaijs/chai.git
...
```

အမှန်တော့ လိုအပ်တဲ့ Boilerplate ကို တည်ဆောက်ပြီးနေပါပြီ။ အင်တာနက်ကနေ လိုအပ်တဲ့ ဖြည့်စွက် Library ဖိုင်တွေကို ရယူနေတာပါ။ အဲ့ဒီလို Download လုပ်နေတာကို မစောင့်လိုရင် Keyboard ကနေ Ctrl+C နှိပ်ပြီးရပ်လိုက်ပါ။ နောက်တော့မှာ `bower install` နဲ့ `npm install` Command နှစ်ခုကို ကိုယ်တိုင် Run ပြီး ဆက်လုပ်နိုင်ပါတယ်။ တည်ဆောက်ရရှိထားတဲ့ Code Base ကို လေ့လာကြည့်ရင် အခုလို Structure ကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။

```
webapp
├── app
│   ├── images
│   ├── scripts
│   │   └── main.js
│   ├── styles
│   │   └── main.css
│   ├── favicon.ico
│   ├── index.html
│   └── robots.txt
├── test
├── bower.json
├── .gitignore
├── Gruntfile.js
└── … …
```

HTML5 Boilerplate ကိုပဲ လိုအပ်သလိုဖြည့်စွက်ထားတဲ့ Boilerplate တစ်ခုဖြစ်ပါတယ်။ ဒါပေမယ့် Website မှာ ကိုယ် တိုင်သွားပြီး Download လုပ်နေစရာမလိုတော့ပါဘူး။ `yo webapp` လို့ ပြောလိုက်တာနဲ့ Web App Boilerplate တစ်ခုကို ချက်ခြင်းရရှိနိုင်မှာပဲဖြစ်ပါတယ်။ အခြား App အမျိုးအစားတွေ Generate လုပ်လိုရင် လည်း –

```
$ yo backbone
$ yo angular
$ yo php
$ yo wordpress
$ yo laravel
$ yo django
$ yo firefox-os
```

– စသဖြင့် အသင့် Generate လုပ်ယူနိုင်ပါတယ်။ အသုံးပြုလိုတဲ့ Generator တွေကို ကြိုတင် Install လုပ်ထားဖို့ သာလို ပါတယ်။

ဒီနည်းနဲ့ Yeoman အကူအညီနဲ့ Scaffolding လုပ်ငန်းကို Automate လုပ်ထားနိုင်ခြင်းပဲ ဖြစ်ပါတယ်။ Generator တွေ အနေနဲ့ အသင့်ရှိနေတဲ့ Generator တွေကိုသုံးနိုင်သလို ကိုယ်တိုင်လည်း Generator တွေ တည်ဆောက်ထားနိုင်ပါတယ်။

Generator တစ်ခု ကိုယ်တိုင်တည်ဆောက် နိုင်ဖို့အတွက် `yo generator` ကို သုံးနိုင်ပါတယ်။

```
$ sudo npm install -g generator-generator
$ yo generator

     _-----_
    |       |    .--------------------------.
    |--(o)--|    |  Create your own Yeoman  |
   `---------´   |      generator with      |
    ( _´U`_ )    |       superpowers!       |
    /___A___\    '--------------------------'
     |  ~  |
   __'.___.'__
 ´   `  |° ´ Y `

? Would you mind telling me your username on GitHub? eimg
? What's the base name of your generator? tinymvc
   create package.json
   create app/index.js
  … …
```

`yo generator` က GitHub Account နဲ့ Generator အမည်တို့ကို မေးလာပါလိမ့်မယ်။ နမူနာမှာ GitHub Account အဖြစ် `eimg` ကိုပေးထားပြီး Generator အမည်အနေနဲ့ `tinymvc` လို့ပေးထားပါတယ်။ ရရှိလာတဲ့ Directory Structure ကို လေ့လာကြည့်ရင် အခုလိုတွေ့ရမှာ ဖြစ်ပါတယ်။

```
generator-tinymvc
├── app
│   ├── templates
│   └── index.js
├── node_modules
├── test
│   └── test-app.js
```

```
├── .gitignore
├── .jshintrc
├── package.json
└── …
```

`generator-tinymvc` ဆိုတဲ့အမည်နဲ့ Generator Boilerplate တစ်ခုကို တည်ဆောက်ပေးသွားခြင်းဖြစ်ပါတယ်။ ရရှိလာတဲ့ Structure ထဲမှာ အရေးအကြီးဆုံးက `app` Directory နဲ့ သူ့ထဲက `index.js` တို့ပဲ ဖြစ်ပါတယ်။ `app` Directory ထဲမှာ Source Code Template တွေကို သိမ်းဆည်းရပြီး `index.js` ကတော့ Generator Script ဖြစ်ပါ တယ်။ နမူနာအနေနဲ့ `app/template` Directory ထဲမှာ `index.html` နဲ့ `css/style.css` ဖိုင်တို့ကို ထည့် သွင်းလိုက်ပါမယ်။ Directory Structure အခုလိုဖြစ်သွားမှာပါ။

```
generator-tinymvc
├── app
│   ├── templates
│   │   ├── css
│   │   │   └── style.css
│   │   ├── index.html
│   │   └── … …
│   └── index.js
├── node_modules
├── test
│   └── test-app.js
├── .gitignore
├── .jshintrc
├── package.json
└── … …
```

ပြီးတဲ့အခါ `index.js` ကိုဖွင့်ပြီး ဒီ Code နဲ့အစားထိုးလိုက်ပါ။

JavaScript

```
var yeoman = require('yeoman-generator');

module.exports = yeoman.generators.Base.extend({
  writing: {
    app: function () {
      this.fs.copy(
        this.templatePath('css/style.css'),
        this.destinationPath('css/style.css')
      );
      this.fs.copy(
        this.templatePath('index.html'),
        this.destinationPath('index.html')
      );
    }
  }
});
```

ဒါဟာ NodeJS နဲ့ ရေးသားထားတဲ့ ပရိုဂရမ်တစ်ခုဖြစ်ပြီး `require()`, `module.exports` စတဲ့ Code တွေရဲ့ အဓိပ္ပါယ်ကို **အခန်း (၁၀)** မှာ NodeJS အကြောင်းဖော်ပြတဲ့အခါ ထည့်သွင်းဖော်ပြပေးပါမယ်။ လိုရင်း ကတော့ Generator ကို Run တဲ့အခါ `app/template` ထဲက `css/style.css` နဲ့ `index.html` တို့ကို ကူးယူပေးပါလို့ သတ်မှတ်ထားလိုက်ခြင်းဖြစ်ပါတယ်။ ကျွန်တော်တို့ကိုယ်ပိုင် Generator တစ်ခုတည်ဆောက် ရရှိသွားပြီဖြစ်ပါတယ်။ ရရှိထားတဲ့ Generator ကို NPM Package အနေနဲ့ အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ cd generator-tinymvc
$ sudo npm link

/usr/lib/node_modules/generator-tinymvc -> /home/eimg/generator-tinymvc
```

`npm link` Command က လက်ရှိ Generator Directory ရဲ့ Symbolic Link (Shortcut) ကို NodeJS Package Directory ထဲမှာ ထည့်သွင်းပေးသွားမှာဖြစ်ပါတယ်။ ဒါကြောင့်မို့ လက်ရှိ `generator-tinymvc` ဟာ NodeJS Package တစ်ခုဖြစ်သွားပြီ ဖြစ်ပါတယ်။ Yeoman ကနေတစ်ဆင့် ဒီ Generator ကို စတင်အသုံးပြုနိုင်ပြီဖြစ်ပါတယ်။

```
$ yo tinymvc

  create css/style.css
  create index.html
```

ကျွန်တော်တို့ဖန်တီးထားတဲ့ `tinymvc` Generator ကိုသုံးပြီး App Boilerplate တစ်ခုတည်ဆောက်လိုက်ခြင်းပဲ ဖြစ် ပါတယ်။ ရရှိလာတဲ့ Structure ကို လေ့လာကြည့်ရင် အခုလိုတွေ့ရမှာ ဖြစ်ပါတယ်။

```
tinymvc
├── css
│   └── style.css
└── index.html
```

`css/style.css` နဲ့ `index.html` ဆိုတဲ့ Source Code ဖိုင်တွေပါဝင်တဲ့ Boilerplate တစ်ခုကို ရရှိသွားပြီပဲ ဖြစ် ပါတယ်။ Yeoman Generator အကြောင်းအသေးစိတ်ကို အောက်ပါလိပ်စာမှာ ဆက်လက်လေ့လာနိုင်ပါ တယ်။

http://yeoman.io/authoring/

8.3 – Package Management

Package ဆိုတာ အသင့်သုံးနိုင်တဲ့ Software Component တစ်ခု (သို့မဟုတ်) Library တစ်ခုကို ဆိုလိုတာပါ။ အသင့် သုံးလို့ရတဲ့ Package တွေရှိနေချိန်မှာ App တစ်ခုအတွက် လိုအပ်တဲ့လုပ်ဆောင်ချက် အားလုံးကို ကိုယ်တိုင်ရေးနေစရာ မလိုပါဘူး။ သင့်တော်တဲ့ Package တွေကို သူ့နေရာနဲ့သူ ရယူအသုံးချရမှာပဲဖြစ်ပါတယ်။

တစ်ချို့ Package တွေဟာ သီးခြားစီ ရပ်တည်အလုပ်လုပ်ကြပေမယ့် တစ်ချို့ Package တွေကတော့ တစ်ခုနဲ့ တစ်ခု အပြန်အလှန် မှီခိုနေကြပါတယ်။ ဥပမာ – jQuery UI ကို အသုံးပြုလိုရင် jQuery ရှိမှအသုံးပြုလို့ရပါမယ်။ BackboneJS က jQuery နဲ့ UnderscoreJS တို့ပါမှ ပြည့်စုံပါတယ်။ ဒီနည်းနဲ့ Package တွေ အပြန်အလှန် မှီခိုနေခြင်းကို Dependency လို့ခေါ်ပါတယ်။

ပုံမှန်အားဖြင့် Package တစ်ခုကို အသုံးပြုလိုရင် သက်ဆိုင်ရာ Package Author ရဲ့ Website ကနေ Download ရယူရ ပါတယ်။ ဒီတော့ Package တစ်ခု အသုံးပြုလိုတိုင်း အင်တာနက်မှာရှာ၊ တွေ့တဲ့အခါ Download လုပ်၊ ရလာတဲ့ Archive ကို Extract လုပ်၊ ရလာတဲ့ Source Code ကို Project Code Base ထဲ ကူးထည့်၊ စသဖြင့် အဆင့်ဆင့်လုပ်ရပါတယ်။ Package Manager တွေက အဲ့ဒီလို Package တွေ ရှာဖွေခြင်းနဲ့ ရယူအသုံးပြုခြင်း လုပ်ငန်းကို လွယ်ကူအောင် စီမံ ပေးထားကြပါတယ်။

Package Manager တွေက Install လုပ်ထားတဲ့ Package တစ်ခုအတွက် Update Version တွေရှိလာတဲ့အခါမှာ Update Version ကို နောက်တစ်ကြိမ် Download လုပ်တဲ့အလုပ်ကို ကိုယ်တိုင်လုပ်စရာမလိုအောင် စီမံပေးနိုင်ပါတယ်။ Package Dependency တွေကိုလည်း စီမံပေးနိုင်ပါတယ်။ ဥပမာ – jQuery UI ကို Install လုပ်တဲ့အခါ သူ့ရဲ့ Dependency ဖြစ်တဲ့ jQuery ကို အလိုအလျောက် ရယူပေးနိုင်ပါတယ်။ ဆက်လက်ပြီး BackboneJS အတွက် jQuery ထပ်လိုတဲ့အခါမှာတော့ ပထမတစ်ကြိမ် ရယူထားပြီးဖြစ်တဲ့အတွက် အချိန်ကုန်ခံပြီးထပ်မံရယူမနေတော့ပဲ ရှိနေတဲ့ jQuery ကိုပဲ အသုံးပြုပေးသွားမှာဖြစ်ပါတယ်။ ဒါ့အပြင် Dependency Version ကိုလည်း ညှိပေးနိုင်ပါသေးတယ်။ ဥပမာ – jQuery UI 1.11 ကိုအသုံးပြုနိုင်ဖို့ jQuery 2.0 လိုတယ်ဆိုကြပါစို့။ လက်ရှိ ရှိနေတာက jQuery 1.7 ဖြစ်နေရင် Version မကိုက်တဲ့သဘောပဲ ဖြစ်ပါတယ်။ အဲ့ဒီလို အခြေအနေမျိုးတွေကို ဖြေရှင်းနိုင်တဲ့နည်းစနစ်တွေ Package Manager တွေမှာ ပါဝင်တက်ပါတယ်။

Node Package Manager (NPM) ဟာ Package Manager တစ်မျိုးဖြစ်ပါတယ်။ သူ့အကူအညီနဲ့ NodeJS Package တွေကို အလွယ်တစ်ကူ Install ပြုလုပ်ခြင်း၊ Update ပြုလုပ်ခြင်းတို့ကို ဆောင်ရွက်နိုင်ပါတယ်။ Package တစ်ခုကို Install လုပ်လိုရင် (သို့မဟုတ်) Update ပြုလုပ်လိုရင် `npm install` Command ကို အခုလို အသုံးပြုနိုင်ပါတယ်။

```
$ sudo npm install -g ionic
```

npm က Install လုပ်လိုတဲ့ Package အတွက် Dependency တွေ ရှိနေရင် တစ်ခါတည်း အလိုအလျောက် ရယူထည့် သွင်းပေးသွားမှာ ဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Package တစ်ခုရယူလိုတိုင်း Package Author ရဲ့ Website ကို သွားပြီး ကိုယ်တိုင် Download လုပ်နေစရာမလိုတော့ပဲ အသင့် ရယူအသုံးပြုနိုင်ခြင်းဖြစ်ပါတယ်။ Install လုပ်

ထားပြီးတဲ့ Package စာရင်းကို လေ့လာလိုရင် တော့ npm list Command ကို အသုံးပြု နိုင်ပါတယ်။

```
$ npm list -g

/usr/lib
├─┬ generator-generator@0.6.1
│ │ ├─┬ chalk@1.0.0
│ │ │ ├── ansi-styles@2.0.1
│ │ │ ├── escape-string-regexp@1.0.3
│ │ │ ├─┬ has-ansi@1.0.3
│ │ │ │ ├── ansi-regex@1.1.1
│ │ │ │ └── get-stdin@4.0.1
│ │ │ ├─┬ strip-ansi@2.0.1
│ │ │ │ └── ansi-regex@1.1.1
│ │ │ └── supports-color@1.3.1
│ │ ├── github@0.2.3
│ │ ├── lodash@2.4.1
│ │ ├─┬ npm-name@1.0.4

… …
```

Package တစ်ခုကို Uninstall ပြုလုပ်လိုရင်တော့ `npm rm` (သို့မဟုတ်) `npm uninstall` ကို အသုံးပြုနိုင်ပါတယ်။

```
$ sudo npm uninstall -g generator-tinymvc

unbuild generator-tinymvc@0.0.0
```

ဒီနည်းနဲ့ Package တွေ Install ပြုလုပ်ခြင်း၊ Update ပြုလုပ်ခြင်း၊ Dependency များစီမံခြင်းတို့ကို ကိုယ်တိုင် Package Manager အကူအညီနဲ့ စီမံနိုင်ခြင်းပဲဖြစ်ပါတယ်။

8.4 – Bower

NPM ဟာ NodeJS နဲ့အတူ တွဲဖက်ပါဝင်လာတဲ့ Package Manager တစ်ခုဖြစ်တဲ့အားလျော်စွာ၊ NodeJS Package တွေကို စီမံဖို့အတွက်သာအသုံးများပါတယ်။ Web App တွေအတွက် Package တွေကိုစီမံဖို့ Bower လို့ခေါ်တဲ့ Package Manager တစ်ခုကို အသုံးပြုနိုင်ပါတယ်။ Yeoman ကို Install လုပ်စဉ်က Bower ကို တစ်ခါတည်း ထည့်သွင်း Install လုပ်ခဲ့ပြီးဖြစ်ပါတယ်။ အကယ်၍ မလုပ်ရသေးရင်လည်း အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ sudo npm install -g bower
```

Package တွေ Install လုပ်ဖို့အတွက် `bower install` ကို အသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

```
$ bower install jquery

bower jquery#*              not-cached git://github.com/jquery/jquery.git#*
bower jquery#*                 resolve git://github.com/jquery/jquery.git#*
bower jquery#*               download https://github.com/jquery/2.1.3.tar.gz
bower jquery#*               progress received 0.1MB
bower jquery#*               progress received 0.2MB
… …
bower jquery#~2.1.3           install jquery#2.1.3

jquery#2.1.3 bower_components/jquery
```

Bower က Install လုပ်လိုက်တဲ့ Package တွေကို `bower_components` ဆိုတဲ့ Directory ထဲမှာ သိမ်းဆည်းပေး သွားမှာဖြစ်ပါတယ်။ နမူနာမှာ jQuery ကို Install လုပ်ထားပါတယ်။ Install လုပ်ပြီးနောက် ရရှိလာတဲ့ Directory Structure ကို လေ့လာကြည့်ရင် အခုလိုတွေ့ရမှာပါ။

```
myapp
├── bower_components
│   └── jquery
│       ├── bower.json
│       ├── dist
│       │   ├── jquery.js
│       │   ├── jquery.min.js
│       │   └── jquery.min.map
│       ├── MIT-LICENSE.txt
│       └── src
├── css
│   └── style.css
└── index.html
```

`bower_components` Directory အတွင်းမှာ jQuery ရဲ့ မူလ Source Code တွေရော အသင့်သုံးလို့ရအောင် Minified လုပ်ထားပေးတဲ့ Version တို့ကိုပါ Download ရယူထည့်သွင်းပေးထားတာကို တွေ့ရမှာဖြစ်ပါတယ်။ Bower Package စာရင်းကို ဒီနေရာမှာ လေ့လာ နိုင်ပါတယ်။

http://bower.io/search/

Bower ရဲ့ထူးခြားချက်က၊ Bower Package တွေကိုသာမက၊ Git Repository တွေနဲ့ Website URL တွေကိုပါ Install လုပ်ပေးနိုင်ခြင်းဖြစ်ပါတယ်။ ဥပမာ – GitHub မှာ Host လုပ်ထားတဲ့ Git Repo တစ်ခုကို အခုလို Install လုပ်နိုင်ပါ တယ်။

```
$ bower install eimg/jquery-voter
… …
bower jquery-voter#*          resolved git://github.com/eimg/jquery-voter.git
bower jquery-voter#*           install jquery-voter

jquery-voter#143a070c8a bower_components/jquery-voter
```

GitHub Handle လို့ခေါ်တဲ့ GitHub Username/Git Repo Name ကို တွဲဖက်ပေးရခြင်းဖြစ်ပါတယ်။ GitHub မဟုတ် တဲ့ တစ်ခြား Website တွေက Package တွေကို Install လုပ်လိုရင်လည်း Package ရှိရာ URL ကိုညွှန်းပေးခြင်းဖြင့် လည်း Install လုပ်နိုင်ပါသေးတယ်။

```
$ bower install http://eimaung.com/u4urc/.vimrc

… …
bower .vimrc#*                  install .vimrc#e-tag:1206bc

.vimrc#e-tag:1206bc-5b bower_components/.vimrc
```

Bower Package တစ်ခုကို ကိုယ်တိုင် ဖန်တီးလိုတယ်ဆိုရင် `bower init` Command ကို သုံးပြီး ဖန်တီးနိုင်ပါတယ်။ နမူနာအနေနဲ့ Directory တစ်ခုဆောက်ပြီး bower install နဲ့ Package တစ်ချို့ Install လုပ်လိုက်ပါမယ်။

```
$ mkdir mypackage && cd mypackage
$ bower install jquery bootstrap

… …
bower jquery#>= 1.9.1           install jquery#2.1.3
bower bootstrap#~3.3.4          install bootstrap#3.3.4

jquery#2.1.3 bower_components/jquery

bootstrap#3.3.4 bower_components/bootstrap
└── jquery#2.1.3
```

အခုချိန် Directory Structure ကိုလေ့လာကြည့်ရင် အခုလိုတွေ့ရမှာပါ။

```
mypackage
└── bower_components
    ├── bootstrap
    │   ├── bower.json
    │   ├── dist
    │   ├── fonts
    │   └── js
    └── jquery
        ├── bower.json
        ├── dist
        └── src
```

`bower_components` ထဲမှာ `bootstrap` နဲ့ `jquery` ဆိုတဲ့ Component နှစ်ခုရှိနေပါတယ်။ ဆက်လက်ပြီး App ရေးသားခြင်းလုပ်ငန်းကို လုပ်ရိုးလုပ်စဉ်အတိုင်း ဆက်လက်ဆောင်ရွက်နိုင်ပါတယ်။ နမူနာအနေနဲ့ `app.js` ဆိုတဲ့ ဖိုင်နဲ့ `index.html` ဆိုတဲ့ ဖိုင်နှစ်ခု ထည့်သွင်းလိုက်ပါမယ်။ Directory Structure အခုလို ပြောင်းသွားမှာပါ။

```
mypackage
├── bower_components
│   ├── bootstrap
│   │   ├── bower.json
│   │   ├── dist
│   │   ├── fonts
│   │   └── js
│   └── jquery
│       ├── bower.json
│       ├── dist
│       └── src
├── app.js
└── index.html
```

ဒီ App ဟာ jQuery နဲ့ Bootstrap ဆိုတဲ့ Package နှစ်ခုကို အသုံးပြုထားတဲ့ App တစ်ခုဖြစ်ပါတယ်။ App ကို တည်ဆောက် ပြီးစီတဲ့အခါ Bower Package အဖြစ် ပြောင်းဖို့အတွက် `bower.json` ဆိုတဲ့ Package Information ဖိုင်တစ်ခု ထည့်သွင်းပေးရပါတယ်။ ကိုယ်တိုင် တည်ဆောက်နိုင်သလို `bower init` ကို Run ပြီး တော့လည်း တည် ဆောက်နိုင်ပါတယ်။

```
$ bower init
? name: mypackage
? version: 0.1.0
? description: Bower package test
? main file: app.js
? authors: Ei Maung <eimg@fairwayweb.com>
? set currently installed components as dependencies?: Yes

{
  name: 'mypackage',
  version: '0.1.0',
  authors: [
    'Ei Maung <eimg@fairwayweb.com>'
  ],
  description: 'Bower package test',
  main: 'app.js',
  keywords: [
    'test'
  ],
  dependencies: {
    bootstrap: '~3.3.4',
    "jquery": '~2.1.3'
  }
}
```

`bower init` က Package အမည်၊ Version, Author စတဲ့အချက်အလက်တွေကို မေးပါလိမ့်မယ်။ Bower Package တွေဟာ ကျွန်တော်တို့ **အခန်း (၅)** မှာဖော်ပြခဲ့တဲ့ Semantic Versioning ကို လိုက်နာရပါတယ်။ ဒါ ကြောင့် Version နံပါတ်ကို နှစ်သက်သလိုပေးနိုင်ပေမယ့် Semantic Version သတ်မှတ်ချက်နဲ့ကိုက်ညီတဲ့ Version နံပါတ် ဖြစ်ဖို့တော့လိုပါတယ်။

`name` အတွက်အမည်ပေးရာမှာ အောက်ပါသတ်မှတ်ချက်တွေနဲ့ ကိုက်ညီဖို့လိုပါတယ်။

1. အခြား Package တစ်ခုခုက မပေးရသေးတဲ့ Unique Name ဖြစ်ရပါမယ်။
2. a to z စာလုံးသေးတွေနဲ့ 0 to 9 ဂဏန်းတွေ အသုံးပြုနိုင်ပါတယ်။ Dash နဲ့ Dot ကိုလည်း သုံးနိုင်ပါတယ်။
3. Dash နဲ့ Dot တွေကို နှစ်ခုတွဲ၊ သုံးခုတွဲ မသုံးသင့်ပါဘူး (ဥပမာ my--package လို့ မပေးသင့်ပါဘူး)။
4. စာလုံးရေ (၅၀) အောက်မှာပဲ ပေးရပါတယ်။

`main file` နေရာမှာ ဒီ Package ရဲ့ အဓိက Source Code ဖိုင် (သို့မဟုတ်) ဖိုင် Array ကို ပေးရပါတယ်။ လက် ရှိ Install လုပ်ထားတဲ့ Package တွေကို Dependency စာရင်းထဲ ထည့်ပေးရမလားဆိုတဲ့ မေးခွန်းကို Yes လို့ ဖြေထား တဲ့အတွက် ကျွန်တော်တို့ Install လုပ်ထားတဲ့ jQeury နဲ့ Bootstrap ကို Dependency စာရင်းထဲမှာ အလိုအလျောက် ထည့်သွင်းပေးသွားမှာ ဖြစ်ပါတယ်။ မေးခွန်းအားလုံး ဖြေပြီးတဲ့အခါ `bower.json` မှာ ပါဝင်မယ့် JSON Code ကို ဖော်ပြပေးပါလိမ့်မယ်။ JSON Code ကိုလေ့လာကြည့်ရင် ကျွန်တော်တို့ ဖြေခဲ့တဲ့ အဖြေတွေကိုပဲ သူ့နေရာနဲ့သူ ထည့် သွင်းပေးထားခြင်း ဖြစ်တယ်ဆိုတာ တွေ့ရမှာပါ။

`bower init` နဲ့ Package Information တွေ သတ်မှတ်ပြီးနောက်မှ ထပ်မံ Install လုပ်တဲ့ Package တွေကို Dependency စာရင်းမှာ အလိုအလျောက် ပါဝင်သွားစေလိုရင် `--save` Option သုံးပြီး Install လုပ်နိုင်ပါတယ်။

```
$ bower install underscore --save
```

`--save` Option နဲ့ ထပ်မံ Install လုပ်လိုက်တဲ့ Package တွေကို Dependency စာရင်းထဲမှာ တစ်ခါတည်း ထည့် သွင်းပေးသွားမှာမို့ `bower.json` ရဲ့ `dependencies` Section မှာလည်း အခုလို ပြောင်းလဲသွားမှာ ဖြစ်ပါတယ်။

JSON

```
"dependencies": {
  "bootstrap": "~3.3.4",
  "jquery": "~2.1.3" ,
  "underscore": "~1.8.2"
}
```

နမူနာအရ ဒီ Package အလုပ်လုပ်ဖို့အတွက် Bootstrap 3.3.4, jQuery 2.1.3 နဲ့ UnderscoreJS 1.8.2 တို့ကို Dependency အနေနဲ့ လိုအပ်တယ်လို့ သတ်မှတ်ထားခြင်း ဖြစ်ပါတယ်။ Dependency Package Version နံပါတ်ကို Range နဲ့လည်း သတ်မှတ်နိုင်ပါတယ်။ ဥပမာ >=1.2.9 ဆိုရင် 1.2.9 နှင့်အထက် ဆိုတဲ့အဓိပ္ပါယ်ဖြစ်ပါတယ်။ >1.2.9 <2.0.0 ဆိုရင်တော့ 1.2.9 နဲ့ 2.0.0 ကြားဆိုတဲ့ အဓိပ္ပါယ်ဖြစ်ပါတယ်။ `bower.json` ဖိုင်မှာ ထည့်သွင်းသတ်မှတ်နိုင်တဲ့ Option အပြည့်အစုံကို အောက်ပါလိပ်စာမှာ လေ့လာနိုင်ပါတယ်။
https://github.com/bower/bower.json-spec

ကျွန်တော်တို့ တည်ဆောက်လိုက်တဲ့ Bower Package ကို bower install Command ကနေတစ်ဆင့် Install လုပ်နိုင်ဖို့အတွက် Register လုပ်ပေးဖို့လိုပါတယ်။ Register လုပ်လိုတဲ့ Bower Package ဟာ Git Repo တစ်ခု ဖြစ်သင့်ပြီး Semantic Version သတ်မှတ်ချက်နဲ့အညီ ပေးထားတဲ့ Git Commit Tag လည်း ပါဝင်ရပါတယ်။ အခုလို သတ်မှတ်နိုင်ပါတယ်။

```
$ git init && git add .
$ git commit -m "Initial commit"
$ git tag 0.1.0
```

Git Repo လည်ဖြစ်ပြီ၊ Version Tag လည်းသတ်မှတ်ပြီးပြီဆိုရင် Package ကို အများရယူနိုင်တဲ့ တစ်နေရာမှာ Publish လုပ်ထားပေးရပါတယ်။ GitHub ကို အသုံးပြုကြလေ့ရှိပါတယ်။ GitHub မှာ Git Repo တစ်ခု Publish လုပ်ပုံကို ဒီနေရာမှာ မပြောတော့ပါဘူး။ github.com မှာ သွားပြီး ကိုယ်တိုင်ပဲလေ့လာကြည့်လိုက်ပါ။ GitHub က လိုအပ်တဲ့ Instruction တွေကို အပြည့်အစုံပေးထားပါတယ်။

Register လုပ်ဖို့အတွက် bower register Command ကို အသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

```
$ bower register mypackage https://github.com/eimg/mypackage.git
```

နမူနာမှာပေးထားတဲ့ `mypackage` ဆိုတာက Package အမည်ဖြစ်ပါတယ်။ သူနောက်က လိပ်စာကတော့ Git Repo တည်ရှိရာကို ညွှန်းထားတဲ့လိပ်စာ ဖြစ်ရမှာဖြစ်ပါတယ်။ Bower က Package Registration အတွက် ဘာမှ ကန့်သတ် မထားပါဘူး။ မည်သူမဆို မိမိတို့ Package ကို Register လုပ်နိုင်တယ်ဆိုတဲ့ သဘောပဲဖြစ်ပါတယ်။ Package အမည် ကသာ အရင်ကသူများ Register မလုပ်ဖူးတဲ့ Unique Name ဖြစ်ဖို့လိုပါတယ်။ ဒီလို Register လုပ်ပြီးပြီဆိုရင် `bower install` နဲ့ မည်သူမဆို ရယူအသုံးပြုနိုင်ပြီဖြစ်ပါတယ်။

`bower install` က Package တစ်ခုကို Install လုပ်တိုင်းမှာ `bower.json` ဖိုင်ထဲမှာ ပါဝင်တဲ့ Dependency တွေကို အလိုအလျောက် တစ်ခါတည်းထည့်သွင်း Install လုပ်ပေးသွားမှာဖြစ်တဲ့အတွက် ကျွန်တော်တို့ Package ကို Install လုပ်ရင် Bootstrap, jQuery နဲ့ UnderscoreJS တို့ကိုလည်း Bower က တစ်ခါတည်း Install လုပ်ပေးသွား တော့မှာ ဖြစ်ပါတယ်။

ဒီနည်းနဲ့ ကိုယ့် Project အတွက်လိုအပ်တဲ့ Package တွေကို Bower အကူအညီနဲ့ ရယူနိုင်သလို၊ ကိုယ့် App ကို လည်း အများ ရယူအသုံးပြုနိုင်တဲ့ Bower Package အဖြစ် တည်ဆောက်လိုရင် တည်ဆောက်ထားနိုင်မှာဖြစ်ပါတယ်။

8.5 – Build

Software Project တစ်ခုမှာ Development နဲ့ Production ဆိုပြီး အခြေအနေနှစ်ခုအမြဲရှိပါတယ်။ ဥပမာ အားဖြင့်၊ Software ကို တည်ဆောက်ရေးသားနေဆဲ Development Stage မှာ Error Message တွေကို ပြည့်စုံနိုင်သမျှ ပြည့်စုံ အောင် ဖော်ပြနိုင်လေ ကောင်းလေပဲဖြစ်ပါတယ်။ ဒီတော့မှ ပြဿနာတစ်စုံတစ်ရာရှိလာတဲ့အခါ Error Message က ပေး တဲ့ အချက်အလက်ကို လေ့လာပြီး ဖြေရှင်းနိုင်မှာဖြစ်ပါတယ်။ ဒါပေမယ့် အများအသုံးပြုနိုင်ဖို့ ဖြန့်ချီလိုက်တဲ့ Production Stage မှာတော့ Error Message တွေ အကုန်ပြနေလို့ မဖြစ်တော့ပါဘူး။ မလွဲသာမရှောင်သာ ဖော်ပြရမယ့် Fatal Error တွေလောက်ကိုသာ ဖော်ပြပြီး၊ Warning တွေ၊ Notice တွေကိုတော့ မဖော်ပြသင့်တော့ပါဘူး။

ပြီးတော့ User ထည့်သွင်းတဲ့အချက်အလက် အစစ်အမှန်မရှိသေးခင်မှာ၊ Dummy Data လို့ခေါ်တဲ့ နမူနာ အချက်အလက် တွေကို Development Stage မှာ ခတ္တယာယီသဘောနဲ့ ထည့်သွင်းစမ်းသပ်ရတက်ပါတယ်။ Production Stage မှာ အဲ့ဒီယာယီ အချက်အလက်တွေကို ပြန်လည်ရှင်းလင်းထားပေးဖို့ လိုပါတယ်။ ဒါ့အပြင် Development Stage မှာ အမှား ရှာရလွယ်အောင် Function တစ်ခုချင်းစီရဲ့ လုပ် ဆောင်ချက်တွေကို မှတ်တမ်းတင်တဲ့ Debug Log တွေ အသုံးပြုရတက် ပါတယ်။ Production Stage မှာ Debug Log ယူနေဖို့ မလိုတော့ပါဘူး။ Unit Test လို Test Code တွေ ထည့်သွင်း ရေးသားထားတယ်ဆိုရင်လည်း Development Stage မှာသာ အဲ့ဒီ Test Code တွေကို Run ဖို့လိုပါတယ်။ Production Stage မှာ Run ဖို့ မလိုတောပါဘူး။ ဒါတင် မကသေးပါဘူး။ Development Stage မှာ စမ်းခါနီးတော့မှ ချက်ခြင်း Compile လုပ်ပြီးစမ်းနေတဲ့ ကိစ္စတွေရှိရင်လည်း Production Stage အတွက်ဆိုရင် တစ်ခါတည်း အပြီး Compile လုပ် ထားပေးဖို့ လိုနိုင်ပါတယ်။

လိုရင်းကတော့ Development Stage မှာ Code ရေးရတာ ထိရောက်မြန်ဆန်ရေးကို အသားပေးတဲ့ Setting တွေကို အသုံးပြုရပြီး Production Stage မှာတော့ Software ရဲ့ စွမ်းဆောင်ရည် ကောင်းမွန်ရေးကိုအသားပေးတဲ့ Setting တွေကို အသုံးပြုရမှာဖြစ်ပါတယ်။

Build လုပ်တယ်ဆိုတာ Development Code ကို Production ဖြစ်အောင် ပြောင်းယူတဲ့လုပ်ငန်းစဉ်ပဲဖြစ်ပါတယ်။ Software ကို ရေးသားတည်ဆောက်စဉ်မှာ Build လုပ်တဲ့လုပ်ငန်းကို မကြာမကြာလုပ်ရမှာပါ။ နေ့စဉ် လုပ်ရမှာပါ။ ဒါကြောင့် Build လုပ်တဲ့လုပ်ငန်းစဉ်ကို တစ်ခုချင်း ကိုယ်တိုင်လုပ်နေရမယ်ဆိုရင် အလုပ်တွင်မှာ မဟုတ်ပါဘူး။ Build Process မှာပါဝင်တဲ့ လုပ်ငန်းစဉ်အသေးစိတ်ကတော့ Project တစ်ခုနဲ့တစ်ခု တူမှာမဟုတ်ပါဘူး။ Web App တစ်ခုကို Build လုပ်တဲ့အခါမှာတော့ အောက်ပါ လုပ်ငန်းတွေ ပါဝင်ရလေ့ရှိပါတယ်။

1. Run tests
2. Compile preprocessor
3. Generate static template
4. Lint JavaScript
5. Minify JavaScript, CSS and HTML
6. Concatenate JavaScript and CSS
7. Optimize Images
8. Create distribution package

Project ကို Production ပြောင်းတော့မယ်ဆိုရင် **Test** တွေအကုန် Pass ဖြစ်ထားဖို့လိုပါတယ်။ ဒါကြောင့် Build လုပ်ငန်းစဉ်ရဲ့ ပထမဆုံးအလုပ်ကတော့ Unit Test အပါအဝင် Test Code တွေကို Run ဖို့ပဲဖြစ်ပါတယ်။ Project မှာ CoffeeScript, LESS စတဲ့ **Preprocessor** နည်းပညာတွေသုံးထားတယ်ဆိုရင် သက်ဆိုင်ရာ Compiler ကို သုံးပြီး JavaScript နဲ့ CSS Code တွေအဖြစ် ပြောင်းယူရပါတယ်။ အကယ်၍ **Template** Language တွေ သုံးထားတယ် ဆိုရင်လည်း Static HTML Template တွေရအောင် Generate လုပ်ထားဖို့လိုပါတယ်။ ဆက်လက်ပြီး JavaScript Code Quality ကောင်းဖို့အတွက် Code Analysis Tool တွေကိုသုံးပြီး **Lint** လုပ်ဖို့လိုပါတယ်။ ပြီးတဲ့အခါ JavaScript, CSS နဲ့ HTML တွေကို ဖိုင် Size သေးသွားအောင် **Minify** လုပ်ရပါတယ်။ HTTP Request တွေကို လျှော့ချနိုင်ဖို့ Minify လုပ်ထားပြီးသား JavaScript နဲ့ CSS တွေကို **Concatenate** လုပ်ပြီး ပေါင်းစပ်ပေးရပါတယ်။ HTTP Request တွေလျှော့ချခြင်းဟာ Performance Optimization အတွက် အဓိကကျတဲ့ ဆောင်ရွက်ချက်တစ်ခုဖြစ်ကြောင်းကို **Professional Web Developer** ရဲ့ **အခန်း (၂၀)** မှာ ဖော်ပြထားပြီး ဖြစ်ပါတယ်။ **Image** တွေကို အရည်အသွေးမကျ စေပဲ ဖိုင် Size သေးသွားအောင် Optimize လုပ်ပေးရပါတယ်။ အားလုံးပြီးပြီဆိုတော့မှ ရလာတဲ့ရလဒ်ကို အသင့်သုံးနိုင် တဲ့ **Package** ဖြစ်အောင် ပြင်ဆင်ထားပေးရမှာဖြစ်ပါတယ်။

အခန်း (၆) မှာဖော်ပြခဲ့တဲ့ Software Team တစ်ခုရဲ့ စွမ်းဆောင်ရည်တိုင်းတာတဲ့ အချက် (၁၂) ချက်ထဲမှာလည်း Build နဲ့ပက်သက်ပြီး (၂) ချက်ပါဝင်ပါတယ်။ Build လုပ်ငန်းစဉ်တစ်ခုလုံးကို တစ်နေရာတည်းကနေ တစ်ထိုင်တည်း ဆောင်ရွက်နိုင်ဖို့လိုတယ်ဆိုတဲ့အချက်နဲ့ နေ့စဉ် Build လုပ်ရမယ်ဆိုတဲ့အချက်တို့ပဲဖြစ်ပါတယ်။ ဒီလို Build လုပ်ငန်းစဉ်တစ်ခုလုံးကို တစ်နေရာတည်းကနေ တစ်ထိုင်တည်းဆောင်ရွက်နိုင်ပြီး၊ နေ့စဉ်လည်း Build လုပ်နိုင်ဖို့ဆိုရင် Manual လုပ်နေလို့ အဆင် မပြေတော့ပါဘူး။ Automate လုပ်ထားရပါတော့မယ်။

8.6 – Grunt

Grunt ဆိုတာအမြဲတမ်းထပ်ခါထပ်ခါလုပ်ရတဲ့ လုပ်ငန်းတွေကို အလိုအလျောက်လုပ်ပေးနိုင်အောင် Automate လုပ်ထား နိုင်တဲ့ Task Runner Tool တစ်ခုပဲဖြစ်ပါတယ်။ NodeJS ကို အသုံးပြုရေးသားထားတဲ့ Tool တစ်ခုဖြစ်ပြီး Yeoman ကို Install လုပ်စဉ်က တစ်ခါတည်းထည့်သွင်း Install လုပ်ခဲ့ပြီးဖြစ်ပါတယ်။ မလုပ်ရသေးရင်လည်း အခုလို Install လုပ်နိုင် ပါတယ်။

```
$ sudo npm install -g grunt-cli
```

Grunt ကို စတင်အသုံးပြုနိုင်ဖို့ Project Directory ထဲမှာ package.json ဆိုတဲ့ဖိုင်နဲ့ Gruntfile.js ဆိုတဲ့ ဖိုင်တို့ကို တည်ဆောက်ခြင်းအားဖြင့် စတင်အသုံးပြုနိုင်ပါတယ်။ package.json ဖိုင်ဆိုတာ `bower.json` လိုပဲ Package Information တွေကိုစုစည်းထားတဲ့ ဖိုင်တစ်ခုဖြစ်ပါတယ်။ Bower Package Manager က Package Information အတွက် `bower.json` ကို သုံးပြီး NPM Package Manager ကတော့ `package.json` ကို Package Information အတွက် အသုံးပြုခြင်းဖြစ်ပါတယ်။ ဒါကြောင့် `package.json` ဖိုင်ရရှိဖို့အတွက် `npm init` Command နဲ့ တည်ဆောက်ယူနိုင် ပါတယ်။ `package.json` ဖိုင်ထဲမှာ Dependency များအဖြစ် Grunt နဲ့ Grunt Plugin များကို ကြေငြာပေးရလေ့ရှိပါတယ်။ နမူနာ `package.json` ဖိုင်တစ်ခု ဖော်ပြပေးလိုက်ပါတယ်။

JSON

```
{
  "name": "my-project-name",
  "version": "0.1.0",
  "devDependencies": {
    "grunt": "~0.4.5"
  }
}
```

`my-project-name` နေရာမှာ နှစ်သက်ရာအမည်တစ်ခုကို ပေးနိုင်ပါတယ်။ `devDependencies` နေရာမှာ `grunt 0.4.5` ကို Dependency အနေနဲ့ ကြေငြာပေးထားပါတယ်။ Build လုပ်လိုက်ချိန်မှာ အလိုအလျောက် လုပ် သွားစေလိုတဲ့ Task တွေကို ကိုယ်တိုင်ရေးသားလို့ရသလို အသင့်ရေးပေးထားတဲ့ Plugin တွေကိုလည်း အသုံးပြုနိုင်ပါတယ်။ အသုံးပြုလိုတဲ့ Plugin တွေကို ထပ်မံ Install လုပ်ပေးဖို့တော့လိုပါတယ်။ ဥပမာ – JavaScript Code တွေကို Minify လုပ်ပေးနိုင်တဲ့ Grunt Plugin တစ်ခုကို အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ sudo npm install grunt-contrib-uglify --save-dev
```

`--save-dev` Option က `package.json` ရဲ့ `devDependencies` စာရင်းထဲမှာ `grunt-contrib-uglify` ကို အလိုအလျောက်ထည့် သွင်းသွားစေလိုတဲ့အတွက် အသုံးပြုရခြင်းဖြစ်ပါတယ်။ `package.json` ကို ပြန် ဖွင့်ကြည့်ရင် အခုလိုတွေ့ရမှာပါ။

JSON

```
"devDependencies": {
  "grunt": "^0.4.5",
  "grunt-contrib-uglify": "^0.5.0"
}
```

`Gruntfile.js` ထဲမှာတော့ Grunt Plugin တွေအတွက် Configuration တွေနဲ့ Build လုပ်ချိန်မှာ အလို အလျောက် အလုပ်လုပ်သွားစေလိုတဲ့ Task တွေ ပါဝင်မှာဖြစ်ပါတယ်။ နမူနာ `Gruntfile.js` ကို ဖော်ပြ လိုက်ပါတယ်။

JavaScript

```
module.exports = function(grunt) {

grunt.initConfig({
   uglify: {
        build: {
          files: {
            'js/app.min.js': ['js/mobile.js', 'js/app.js']
          }
        }
    }
```

```
11.    });
12.
13.    grunt.loadNpmTasks('grunt-contrib-uglify');
14.    grunt.registerTask('default', ['uglify']);
15.
16. };
```

နမူနာရဲ့ လိုင်းနံပါတ် (၇) မှာ လေ့လာကြည့်ရင် `js/mobile.js` နဲ့ `js/app.js` တို့ကို Minify လုပ်ပြီး `js/app.min.js` အမည်နဲ့သိမ်းပေးဖို့ သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။ အခုဆိုရင် `package.json` ရော `Gruntfile.js` ပါရရှိသွားပြီမို့ Grunt ကိုအသုံးပြုပြီး စတင် Build လုပ်နိုင်ပြီ ဖြစ်ပါတယ်။ ကျွန်တော်တို့ရဲ့ လက်ရှိ Source Code Directory Structure က အခုလိုဖြစ်မှာပါ။

```
my-project
├── js
│   ├── app.js
│   └── mobile.js
├── node_modules
│   ├── grunt
│   └── grunt-contrib-uglify
├── Gruntfile.js
└── package.json
```

`js` Directory ထဲမှာ `app.js` နဲ့ `mobile.js` ဆိုတဲ့ Script ဖိုင်နှစ်ခုရှိနေပါတယ်။ Build လုပ်ဖို့အတွက် `grunt` Command ကို Run လိုက်ယုံပါပဲ။

```
$ grunt

Running "uglify:build" (uglify) task
>> 1 file created.

Done, without errors.
```

grunt Command က `Gruntfile.js` ထဲမှာ သတ်မှတ်ထားတဲ့ Configuration အတိုင်း `app.js` နဲ့ `mobile.js` တို့ကို Minify လုပ်ပြီး `app.min.js` ဆိုတဲ့အမည်နဲ့ Generate လုပ်ပေးသွားမှာဖြစ်ပါတယ်။

```
my-project
├── js
│   ├── app.js
│   ├── app.min.js
│   └── mobile.js
├── node_modules
│   ├── grunt
│   └── grunt-contrib-uglify
├── Gruntfile.js
└── package.json
```

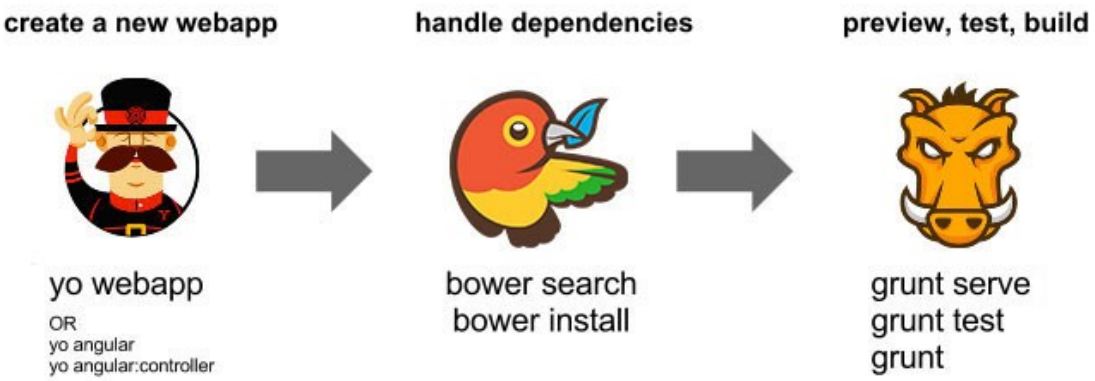
Source Code Directory Structure ကို ပြန်လေ့လာကြည့်ရင် `js` Directory ထဲမှာ `app.min.js` ဆိုတဲ့ Output ဖိုင်တစ်ခု တိုးလာတာကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။

ဒီနည်းနဲ့ Test Code တွေ Run တဲ့ကိစ္စတွေ၊ JavaScript Code တွေကို Lint လုပ်တဲ့ကိစ္စတွေ၊ Image Optimize လုပ်တဲ့ ကိစ္စတွေနဲ့ Build လုပ်ငန်းစဉ်မှာလိုအပ်တဲ့ အခြားလုပ်ငန်းတွေကို `Gruntfile.js` ထဲမှာ ကြေငြာသတ်မှတ်ထားနိုင်ပါတယ်။ နောင် Project ကို Build လုပ်လိုတိုင်း `grunt` Command တစ်ကြိမ် Run ယုံနဲ့ လိုအပ်တဲ့လုပ်ငန်းစဉ် တစ်ခုလုံး ပြီးမြောက်သွားမှာပဲဖြစ်ပါတယ်။ အသုံးဝင်တဲ့ Grunt Plugin တွေကို အောက်ပါလိပ်စာမှာ လေ့လာနိုင်ပါတယ်။

http://gruntjs.com/plugins

Yeoman နဲ့ Project တစ်ခုကို Scaffold လုပ်လိုက်တိုင်းမှာ Yeoman က Build Configuration အသင့် ပါဝင်တဲ့ `Gruntfile.js` ကို တစ်ခါတည်း တည်ဆောက်ပေးသွားလေ့ရှိပါတယ်။ လေ့လာလိုရင် `yo webapp` Command နဲ့ Web App Boilerplate တစ်ခု Scaffold လုပ်ကြည့်လိုက်ပါ။ ပြီးရင် သူနဲ့အတူပါလာတဲ့ `Gruntfile.js` ကို လေ့လာကြည့်ရင် Build Process တစ်ခုလုံးအတွက် လိုအပ်တဲ့လုပ်ငန်းအားလုံးကို ကြိုတင် Configure လုပ်ပေးထားတယ် ဆိုတာကို တွေ့ရမှာပဲဖြစ်ပါတယ်။

ပုံ (၈.၁) မှာလေ့လာကြည့်ပါ။ Yeomen, Bower နဲ့ Grunt တို့ကိုအသုံးပြုထားတဲ့ Common Workflow ကိုဖော်ပြထား ပါတယ်။



ပုံ (၈.၁) - Web App Development Workflow with Yeoman, Bower and Grunt

Source – yeoman.io

ပထမဦးဆုံး `yo` Command ကနေတစ်ဆင့် App Boilerplate ကို တည်ဆောက်ယူရပါတယ်။ `yo` က အသုံးဝင်တဲ့ Bower Package တွေကိုပါ တစ်ခါတည်းထည့်သွင်းပေးသွားမှာ ဖြစ်သလို `Gruntfile.js` ကိုလည်း တစ်ခါတည်း တည်ဆောက် ပေးသွားမှာဖြစ်ပါတယ်။ ပြီးတဲ့အခါ ဖြည့်စွက်လိုအပ်တဲ့ Package တွေကို `bower install` နဲ့ ထပ်မံ ဖြည့်စွက်နိုင်ပါတယ်။ ရေးစရာရှိတဲ့ Code တွေရေးပြီးလို့ App ကို စမ်းကြည့်ချင်တယ်ဆို

ရင် `grunt serve` Command ကို Run ပြီ စမ်းကြည့်နိုင်ပါတယ်။ Build မလုပ်သေးပဲ Test Code တွေကို Run စေချင်ရင် `grunt test` Command နဲ့ Test Code တွေ Run နိုင်ပါတယ်။ ဒီအတွက် လိုအပ်တဲ့ Configuration တွေကို `yo` က တစ်ခါ တည်း ထည့်ပေးလိုက်ပြီးသားပါ။ နောက်ဆုံး Build လုပ်ချင်တယ်ဆိုရင်တော့ `grunt` Command ကို Run လိုက်ယုံ ပါပဲ။ JavaScript Lint လုပ်တာတွေ၊ Minify လုပ်တာတွေ၊ Image Optimize လုပ်တာတွေ အားလုံးကို လုပ်ပေးသွားတဲ့ အပြင် ရလာတဲ့ Build Result ရလဒ်ကိုလည်း အသင့်သုံးနိုင်အောင် `dist` Directory ထဲမှာ ထားပေးမှာပဲဖြစ်ပါတယ်။ ဒါကြောင့် `grunt` Command Run ပြီးတဲ့နောက် `dist` Directory ထဲမှာရှိနေတဲ့ Build Result ကိုလက်ရှိ Develop လုပ်နေတဲ့ App ရဲ့ Production Version အဖြစ်မှတ်ယူပြီး လိုအပ်သလို ဆက်လက်ဆောင်ရွက်နိုင်မှာပဲ ဖြစ်ပါတယ်။

8.7 – Deployment

Software တစ်ခုကို Deploy လုပ်ရာမှာ အခြေအနေနှစ်မျိုးရှိနိုင်ပါတယ်။ အကယ်၍ Software က အဖွဲ့အစည်းတစ်ခု အတွင်းမှာသုံးဖို့ ရည်ရွယ်တည်ဆောက်ထားတဲ့ Software ဖြစ်တယ်ဆိုရင်၊ Deploy မလုပ်ခင် အဲ့ဒီအဖွဲ့အစည်းမှာ ဒီ Software ကို အသုံးပြုဖို့အတွက် လိုအပ်တဲ့ Infrastructure တစ်ခုရှိရဲ့လား စမ်းစစ်ဖို့ လိုနိုင်ပါတယ်။ Server, Network နဲ့ System Requirement တွေ ပြည့်စုံရဲ့လားစီစစ်ပြီး၊ လိုအပ်ရင် Infrastructure Architecture Design ကို ပြင်ဆင် တည်ဆောက်ဖို့လိုပါတယ်။ ကိုယ်တိုင် တည်ဆောက်ရတာမျိုး ဖြစ်နိုင်သလို Enterprise IT Architecture ပိုင်း ဝန်ဆောင် မှုပေးသူတွေရဲ့ အကူအညီကို ယူရတာမျိုးလဲ ဖြစ်နိုင်ပါတယ်။

အကယ်၍ Software က အဖွဲ့အစည်းတစ်ခုအတွက် ရည်ရွယ်တဲ့လုပ်ငန်းသုံး Software မဟုတ်ပဲ၊ အများသုံး Consumer Software ဆိုရင်တော့ Build လုပ်ပြီးရလာတဲ့ ရလဒ်ကို User တွေထံရောက်ရှိအောင် ဖြန့်ချီရမယ့် နည်းလမ်းတွေ စဉ်းစားရပါတော့မယ်။ အသုံးများတဲ့ Distribution Method တစ်ခုဖြစ်တဲ့ App Store တွေမှာ တင်ပြီး ဖြန့်ချီမှာလား၊ ကိုယ်ပိုင် Website တစ်ခုနဲ့ Download ပေးမှာလား စသဖြင့် စဉ်းစားရပါမယ်။ Software အများစုက API Back-end ကို အခြေခံကြတဲ့အတွက် Server Environment တစ်ခုလဲ လိုအပ်ပါတယ်။ အဲ့ဒီလို လိုအပ်တဲ့ Server Environment ရရှိဖို့အတွက် Cloud Service Provider တွေထံက လိုအပ်တဲ့ ဝန်ဆောင်မှုတွေကို ရယူနိုင်သလို၊ ကိုယ်တိုင်လည်း Production Server တစ်ခုကို Setup လုပ်ယူနိုင်ပါတယ်။

လိုရင်းကတော့ Deployment & Distribution ဆိုတာဟာ Business Process တစ်ခု Business Decision တစ်ခု ဖြစ်ပါတယ်။ Software Development တစ်ဘက်တည်းကမဟုတ်ပဲ ရှုထောင့်စုံကနေ စဉ်းစားဆုံးဖြတ်ရမယ့် ကိစ္စရပ်တစ်ခု ဖြစ်ပါတယ်။

Business Decision ပိုင်းကိုချန်ပြီး Developer တွေနဲ့ သက်ဆိုင်တဲ့အပိုင်းကို ယေဘုယျခြုံကြည့်လိုက်ရင် နှစ်ပိုင်း ရှိပါတယ်။ လိုအပ်တဲ့ Server နဲ့ Network Environment ပိုင်း Configure ပြုလုပ်ခြင်းတဲ့၊ Build လုပ်ပြီး ရရှိလာတဲ့ ရလဒ် ကို အများအသုံးပြုနိုင်ဖို့အတွက် Production Server မှာ Publish လုပ်ခြင်းတို့ပဲဖြစ်ပါတယ်။

Server နဲ့ Network Environment ပိုင်း Configure ပြုလုပ်ခြင်းဟာ အမှန်တော့ ဒီကဏ္ဍကို အထူးပြုကျွမ်းကျင်တဲ့ sysadmin လို့ခေါ်တဲ့ System Administrator တွေရဲ့အလုပ်ဖြစ်ပါတယ်။ ဒါပေမယ့်လည်း အခုနောက်ပိုင်းမှာ Full-stack Developer ဆိုတဲ့အသုံးအနှုန်းနဲ့အတူ၊ Software ကို တည်ဆောက်နိုင်ယုံသာမက လိုအပ်တဲ့ Server Environment ကိုမှ Configure လုပ်နိုင်တဲ့ ဘက်စုံသုံး Developer တွေ ပေါ်ပေါက်လာခြင်းနဲ့အတူ Server

Environment Configuration ဟာလည်း Developer တွေရဲ့ အလုပ်တစ်ခုဖြစ်လာခဲ့ပါတယ်။ ဒီကဏ္ဍမှာ ပြောစရာတွေ များတဲ့အတွက် ဒီနေရာမှာ မပြောတော့ပဲ၊ **အပိုင်း (၅)** ရောက်တော့မှ ဆက်လက်ဖော်ပြပါမယ်။

ဒုတိယတစ်ပိုင်းဖြစ်တဲ့ အသင့် Setup လုပ်ထားပြီးဖြစ်တဲ့ Server Environment မှာ Build ထားတဲ့ Software ကို Publish လုပ်မယ်ဆိုရင်တော့၊ အလွယ်ဆုံးနည်းလမ်းအနေနဲ့ FTP Software တစ်ခုကိုသုံးပြီး Build လုပ်ထားတဲ့ Software ကို Production Server ပေါ်မှာ Copy ကူးပေးလိုက်နိုင်ပါတယ်။ ဒီနည်းက လက်တွေ့မှာ ပြဿနာတစ်ချို့ပေးတက်ပါတယ်။

၁.) Software မှာပါဝင်တဲ့ ဖိုင်အရေအတွက် အလွန်များပြားတဲ့အခါ အင်တာနက်အဆက်အသွယ် ပြတ်တောက်မှုနဲ့ မမျှော်မှန်းနိုင်တဲ့ Connection ပြဿနာတစ်ချို့ကြောင့် တစ်ချို့ဖိုင်တွေ မတော်တစ်ဆ ကျန်သွားတာမျိုး ဖြစ်နိုင်ပါတယ်။

၂.) Deploy လုပ်တယ်ဆိုတာ ထပ်ခါထပ်ခါလုပ်ရတက်ပါတယ်။ ဥပမာ Version 0.1.0 ကိုတစ်ကြိမ် Deploy လုပ်ပြီး၊ နောက်တော့မှ Version 0.1.1 ထွက်လာလို့ နောက်တစ်ကြိမ် Deploy လုပ်ရတက်ပါတယ်။ အဲ့ဒီအခါမှာ 0.1.0 နဲ့ 0.1.1 အကြား ပြောင်းလဲသွားတဲ့ အပြောင်းအလဲကိုသာမက၊ သေချာအောင် Software တစ်ခုလုံးကို နောက်တစ်ကြိမ် ပြန်ပြီး ကူးတင်ရတက်ပါတယ်။

၃.) လက်တွေ့မှာ တစ်ချို့ဖိုင်တွေကို Compress လုပ်လိုက်ရင် ဖိုင်အမျိုးအစားနဲ့ Compression Algorithm ပေါ်မူတည် ပြီး ဖိုင် Size က 200% ကနေ 400% လာက်ထိကျသွားနိုင်ပါတယ်။ Deployment လုပ်ငန်းမြန်ဆန်ဖို့ဆိုရင် Compress လုပ်ပြီးမှ Server ထံပေးပို့သင့်ပါတယ်။ ဒီအခါမှာ Copy မကူးခင် Compress လုပ်ရတဲ့ကိစ္စတွေ၊ Copy ကူးပြီးနောက် Decompress လုပ်ရတဲ့ကိစ္စတွေ ထပ်လိုလာတက်ပါတယ်။

၄.) ဒီလို Manual Deploy လုပ်ရတဲ့အတွက် နောက်ဆက်တွဲပါလာတဲ့ ပြဿနာတစ်ခုလည်း ရှိပါသေးတယ်။ Deploy လုပ်ပြီးမှ အသေးစားပြင်စရာလေးတစ်ခုတွေ့လာတဲ့အခါ ကိုယ့်စက်ထဲက မူလ Source Code မှာပြင်ပြီး၊ နောက်တစ် ကြိမ် Deploy လုပ်နေရင်ကြာမှာစိုးလို့ဆိုပြီး Production Server ပေါ်မှာ တိုက်ရိုက်ပြင်လိုက်မိတက်ပါတယ်။ ဒီလို ပြင်ဆင်ချက်လေးတွေ များများလာပြီး၊ အဲ့ဒီပြင်ဆင်ချက်တွေကို Local Code Base နဲ့ ညှိနှိုင်းဖို့ နောက်ကျတဲ့အခါ၊ Production Version က Development Version ထက်ရှေ့ရောက်နေတဲ့ပြဿနာတွေ ကြုံရတက်ပါတယ်။

Deployment ဆိုတာ Build လုပ်ထားတဲ့ Software ကို Server ပေါ်ကူးလိုက်တဲ့ကိစ္စလေးပါလို့ ခပ်ပေါ့ပေါ့သဘောထားမိ ရင် ဒီလိုပြဿနာတွေ အနည်းနဲ့အများရှိလာပြီး မလိုအပ်ပဲအချိန်တွေပိုကုန်သွားတက်ပါတယ်။ ဒါကိုဖြေရှင်းဖို့အတွက် Git ရဲ့ အကူအညီကိုရယူနိုင်ပါတယ်။ ဖြစ်သင့်တဲ့ Workflow က ဒီလိုပါ။

၁.) ပထမဆုံးအနေနဲ့ Production Server ပေါ်မှာ git init --bare နဲ့ Git Repo အလွတ်တစ်ခု သတ်မှတ် ပေးရပါမယ်။

၂.) ကိုယ့် Local Git Repo ရဲ့ Remote အဖြစ် အဲ့ဒီ Production Server ပေါ်က Git Repo ကို ညွှန်းပေးရပါမယ်။

```
$ git remote add prod git://path/to/production/repo
```

၃.) Local Code Base ထဲမှာ Build လုပ်လိုက်လို့ dist Directory ရလာပြီးဆိုရင် အဲ့ဒီ `dist` Directory ကို `git push` နဲ့ Production Server ပေါ်ကို ပို့ပေးရပါမယ်။ Code Base ကြီးတစ်ခုလုံးကို Push မလုပ်ပဲ `dist` Directory တစ်ခုတည်းကို ရွေးပြီး Push လုပ်လိုရင် Git Sub Tree ဆိုတဲ့ Git Plugin ကို အသုံးပြုနိုင်ပါတယ်။ အောက်ပါလိပ်စာ မှာ ရယူနိုင်ပါတယ် –

https://github.com/apenwarr/git-subtree

၄.) Git Sub Tree ကို Install လုပ်ဖို့အတွက် သူနဲ့အတူပါလာတဲ့ လမ်းညွှန်ကိုလေ့လာကြည့်လိုက်ပါ။ Git Sub Tree ကို Install လုပ်ပြီးနောက် dist Directory ကို အောက်ပါအတိုင်း Production Server ထံ Push လုပ်နိုင်ပါတယ်။

```
$ git subtree push --prefix dist prod master
```

`--prefix` Option အတူ Push လုပ်လိုတဲ့ Directory အမည်ဖြစ်တဲ့ `dist` ကို တွဲသုံးထားတာကို သတိပြုပါ။ ဒီနည်းနဲ့ Code Base တစ်ခုလုံးကို Push မလုပ်ပဲ `dist` Directory တစ်ခုတည်းကို Push လုပ်နိုင်ပါတယ်။

ဒီနည်းနဲ့ Deployment ပြုလုပ်လိုတိုင်း `git subtree push` ကိုသုံးလိုက်ခြင်းအားဖြင့် Build လုပ်ထားတဲ့ Software လည်း Server ထံ ရောက်ရှိသွားမှာပဲ ဖြစ်ပါတယ်။ Git က ဖိုင်တွေကို Compress လုပ်တဲ့ကိစ္စတွေ၊ ဖိုင်မကျန်ခဲ့အောင်စီစစ်တဲ့ကိစ္စတွေနဲ့၊ နောင်ပိုင်း Push တွေမှာ Software တစ်ခုလုံးကို ထပ်မပို့တော့ပဲ၊ အပြောင်းအလဲကိုသာ ရွေးချယ်ပေးပို့သွားတဲ့ ကိစ္စတွေအားလုံးကို တာဝန်ယူပေးသွားမှာပဲဖြစ်ပါတယ်။ အမှားပြင်စရာရှိရင်လည်း Production Server ပေါ်မှာ ပြင်စရာမလိုပဲ ကိုယ့် Local Code Base ထဲမှာပြင်ပြီး `git subtree push` နောက်တစ်ကြိမ် Run ပေးလိုက်ယုံပဲဖြစ်ပါတယ်။

8.8 – Continues Integration

Development Workflow Automation နဲ့ပက်သက်ပြီး နောက်ဆုံးတစ်ချက်အနေနဲ့ မှတ်သားသင့်တဲ့ နည်းစနစ်ကတော့ Continues Integration (CI) ပဲဖြစ်ပါတယ်။ Continues Integration ဆိုတာကို လိုရင်းပြောရရင် နေ့စဉ် Build လုပ်တဲ့ Daily Build အဆင့်ကိုကျော်ပြီး၊ VCS ထဲကို Commit တစ်ကြိမ်လုပ်တိုင်း အလိုအလျောက် Build လုပ်သွားအောင် စီမံထားရတဲ့ နည်းစနစ်ဖြစ်ပါတယ်။

ဒီနည်းက ပေးတဲ့အဓိက အကျိုးကျေးဇူးနှစ်ခုရှိပါတယ်။ တစ်ခုကတော့၊ Commit တစ်ခုကြောင့် Build ပြဿနာရှိခဲ့ရင် ချက်ခြင်းသိရတဲ့အတွက် ပြင်ဆင်ရလွယ်ကူခြင်းဖြစ်ပါတယ်။ Developer တစ်ယောက်ပြင်ထားတဲ့ Code က သူတာဝန်ယူ တဲ့ အပိုင်းမျာ မှာမှန်နေလို့ Commit လုပ်လိုက်ပေမယ့် အဲ့ဒီ Commit ကြောင့် အခြား Developer တာဝန်ယူနေတဲ့ အပိုင်းမှာသွားပြီး ပြဿနာဖြစ်နေတယ် ဆိုတာမျိုး ဖြစ်တက်ပါတယ်။ Commit တိုင်းမှာ Build လုပ်သွားတဲ့အတွက် ဒီ ပြဿနာမျိုးရှိခဲ့ရင် ချက်ခြင်းသိနိုင်မှာဖြစ်ပါတယ်။

နောက်ထပ်ရရှိတဲ့ အကျိုးကျေးဇူးကတော့ Software ဟာ အမြဲတမ်း Production Ready ဖြစ်နေခြင်းပဲဖြစ်ပါတယ်။ Build လုပ်တယ်ဆိုတာ လက်ရှိရေးသားနေတဲ့ Code ကို Production Code ပြောင်းတဲ့လုပ်ငန်းဖြစ်ပါတယ်။ Commit တိုင်းကို Build လုပ်နေတဲ့အတွက် ရောက်ရှိနေတဲ့နောက်ဆုံးအဆင့်ဟာ အမြဲတမ်း Production သွားဖို့ အသင့်ဖြစ်နေမှာ ပဲဖြစ်ပါတယ်။

ဒီနည်းဟာ Agile နည်းစနစ်တစ်ခုဖြစ်တဲ့ Extreme Programming (XP) ကလာတာပါ။ Continues Integration လုပ် နိုင်ဖို့အတွက် Travis CI (travis-ci.org), BuildBot (buildbot.net), Hudson CI (hudson-ci.org) စတဲ့ Tool တွေရှိ ပါတယ်။ အဲ့ဒီ CI Tool တွေရဲ့ အခြေခံ အလုပ်လုပ်ပုံက ဒီလိုပါ –

ပထမဦးဆုံးအနေနဲ့ Build Configuration ဖိုင်တစ်ခုကို အရင် Setup လုပ်ရပါတယ်။ အဲ့ဒီ Configuration ထဲမှာ လက်ရှိ Software အလုပ်လုပ်ဖို့ ရည်ရွယ်ထားတဲ့ Environment Information အပြည့်အစုံသတ်မှတ်ပေးရတက်ပါတယ်။ Compiler, Compiler Version, အသုံးပြုထားတဲ့ Component တွေနဲ့ Package တွေစာရင်းကိုသတ်မှတ်ပေးရပါတယ်။ ပြီးတဲ့အခါ Build လုပ်ငန်းစဉ်ဆောင်ရွက်ဖို့အတွက် Production Environment မှာ ရှိရမယ့် အခြေအနေအတိုင်း Build Environment တစ်ခုကို Setup လုပ်ရပါတယ်။ Build လုပ်တဲ့အခါ အဲ့ဒီ Build Environment ပေါ်မှာ လုပ်မှာဖြစ်ပါတယ်။ ဒါကြောင့် Build Script ထဲမှာ အသုံးပြုထားတဲ့ Compiler Version တွေ၊ အသုံးပြုထားတဲ့ Package တွေကိုဖော်ပြရ ခြင်းဖြစ်ပါတယ်။ Build Environment ကို Setup လုပ်ဖို့ အတွက် အဲ့ဒီအချက်အလက်တွေကို အသုံးပြုသွားမှာဖြစ်ပါတယ်။ Travis CI မှာ အသုံးပြုရလေ့ရှိတဲ့ Build Configuration နမူနာကို ဖော်ပြလိုက်ပါတယ်။

YAML

```yaml
# File Name: .travis.yml

language: node_js
node_js:
  - 0.10
  - 0.8
  - 0.6

services:
  - mongodb

before_script:
  - npm install
  - bower install

script: npm test
```

```
notifications:
  email:
    - "james@gmail.com"
    - "john@gmail.com"
```

Travis CI Build Configuration ကို YAML လို့ခေါ်တဲ့ Markup ရေးထုံးနဲ့ရေးပေးရပါတယ်။ နမူနာအရ Build Enviro-nment ကို NodeJS 0.10, 0.8 နဲ့ 0.6 Version သုံးမျိုးအတွက် Setup လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ အဲ့ဒီလို Setup လုပ်ရာမှာ MongoDB ကို တွဲဖက်ထည့်သွင်းပေးသွားမှာဖြစ်ပြီး Build Script ကို Run မလုပ်ခင် `npm install` နဲ့ `bower install` တို့ကို အရင် Run ပေးသွားဦးမှာဖြစ်ပါတယ်။ Build Script အနေနဲ့ `npm test` ကို အသုံးပြုဖို့ သတ်မှတ်ထားပါတယ်။ အကယ်၍ `Gruntfile.js` ကို Build Script အနေနဲ့ သုံးစေချင်ရင် NPM Configuration ဖိုင်ဖြစ်တဲ့ `package.json` မှာ အခုလိုထည့်သွင်းပေးထားနိုင်ပါ တယ်။

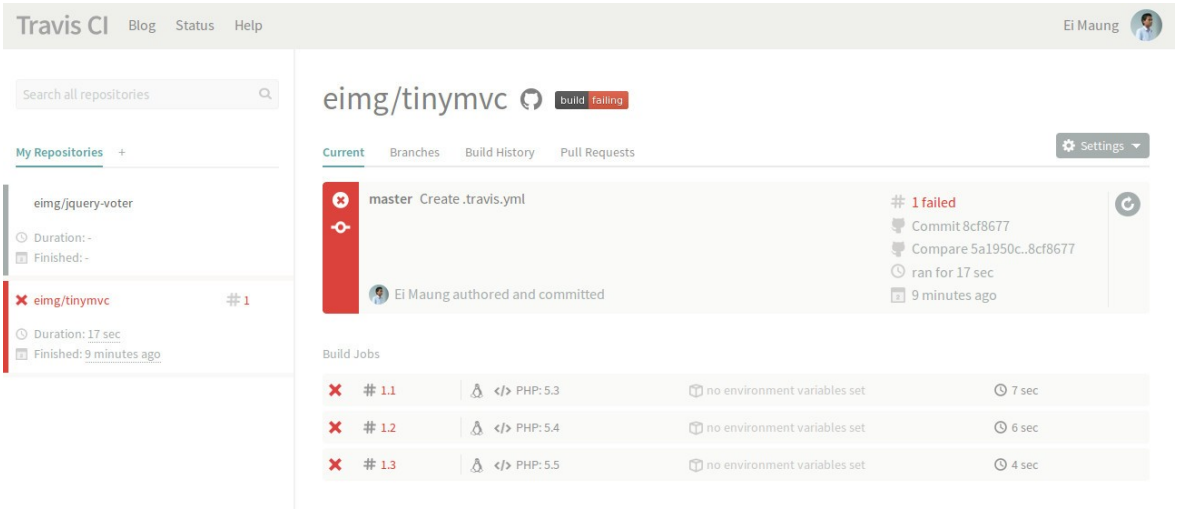
JSON

```
// File Name: package.json

scripts {
  "test": "grunt"
}
```

ဆက်လက်ပြီး Source Code Repo မှာ Commit တစ်ကြိမ်လုပ်တိုင်း Code Base ကို Setup လုပ်ထားတဲ့ သီးခြားစနစ်ပေါ်မူပွားယူခြင်းနဲ့ Build Script ကို Run ပေးခြင်းလုပ်ငန်းတွေ အလိုအလျောက် ဆောင်ရွက်ဖို့ သတ်မှတ်ပေးရပါ တယ်။ Travis CI ကတော့ Github ရဲ့ Web Hook ကို သုံးပါတယ်။ ဒါကြောင့် Github မှာ Host လုပ်ထားတဲ့ Repo မှာ Commit လုပ်လိုက်တိုင်း၊ Github က Travis CI ကို အသိပေးသွားမှာဖြစ်ပြီး၊ Travis CI က Build Environment တည်ဆောက်ခြင်းနဲ့ Build Script Run ခြင်းတို့ကို Commit တိုင်းအတွက် လုပ်ပေးသွားမှာဖြစ်ပါ တယ်။

Build Script ကို Run ပြီးတဲ့အခါ Fail ဖြစ်ခဲ့ရင် အခြေအနေကို Developer အားလုံးထံ Error Log နဲ့ တစ်ကွ ပေးပို့ ပေးရမှာဖြစ်ပါတယ်။ Pass ဖြစ်ခဲ့ရင်တော့ ရရှိလာတဲ့ Build Result ကို Production Environment ထံ ပေးပို့ခြင်း အားဖြင့် Deployment ကိုပါ တစ်ခါတည်း တွဲလုပ်နိုင်ပါတယ်။



ပုံ (၈.၂) - Travis CI - Build Status

ပုံ (၈.၂) မှာဖော်ပြထားတာကတော့၊ Build Fail ဖြစ်တဲ့အခါ Travis CI ကဖော်ပြပေးတဲ့ အခြေအနေဖြစ်ပါတယ်။ Build Pass ဖြစ်ခဲ့ရင်တော့ Amazon AWS, Google Cloud Storage, Heroku, Engine Yard, Appfog စတဲ့ Cloud Service တွေပေါ်မှာအလိုအလျောက် Deploy လုပ်သွားအောင် စီမံထားနိုင်ပါသေးတယ်။ အကယ်၍ Git Push နဲ့ Deploy လုပ်စေလိုရင်လည်း Build Configuration ထဲမှာ အခုလို ထည့်သွင်းရေးသားထားနိုင်ပါတယ်။

YAML

```yaml
# File Name: .travis.yml

after_success:
  - git remote add deploy git:/path/to/repo/
  - git push deploy
```

ဒီနည်းနဲ့ VCS ထဲမှာ Commit လုပ်လိုက်တိုင်း၊ အလိုအလျောက် Build လည်းလုပ်၊ Deploy လည်း လုပ်ပေးသွားအောင် စီမံထားခြင်းအားဖြင့် Continuous Integration လုပ်ဆောင်ချက်ကို ရရှိနိုင်မှာပဲဖြစ်ပါတယ်။

Conclusion

ဒီအခန်းမှာ နမူနာအဖြစ်ဖော်ပြခဲ့တဲ့ Tool တွေဟာ Web App Project တွေအတွက် ရည်ရွယ်ဖန်တီးထားတဲ့ Tool တွေ များပါတယ်။ အခြား Software Project အမျိုးအစားတွေအတွက်လည်း၊ အသုံးပြုထားတဲ့ Language, Framework နဲ့ နည်းပညာတွေ ပေါ်မူတည်ပြီး Scaffolding, Package Management, Build Tool နဲ့ CI Tool တွေ သူ့နေရာနဲ့သူ ရှိတက်ပါတယ်။

Ruby on Rails, Django, ASP.NET MVC, Laravel, Synfony, CakePHP စတဲ့ Framework တွေမှာ Scaffolding လုပ်ဆောင်ချက် တွဲဖက်ပါဝင်လာပြီးဖြစ်ပါတယ်။ Java Project တွေအတွက်ဆိုရင် Spring Roo လို Tool မျိုးကို Scaffolding အတွက် အသုံးပြုနိုင်ပါတယ်။ ဒါ့အပြင် NetBeans, Eclipse, Visual Studio စတဲ့ IDE တွေမှာလည်း

Scaffold နဲ့ အလားသဏ္ဌာန်တူတဲ့ Code Generation လုပ်ဆောင်ချက်တွေ ပါဝင်တက်ပါတယ်။ Plugin Extension အနေနဲ့ ထပ်မံထည့်သွင်းပေးဖို့တော့ လိုကောင်းလိုနိုင်ပါတယ်။

Package Manager အနေနဲ့ဆိုရင်လည်း Microsoft .NET အတွက် NuGet, PHP အတွက် Composer နဲ့ Pear, Python အတွက် pip, Ruby အတွက် Gem နဲ့ Bundler, Java အတွက် Maven စသဖြင့် ကိုယ်စီရှိကြပါတယ်။ Build Tool အနေနဲ့ဆိုရင်တော့ Ruby အတွက် Rake, Java အတွက် Apache Ant, .Net အတွက် NAnt နဲ့ MSBuild, PHP အတွက် Phing, Python အတွက် SCons နဲ့ Paver စသဖြင့်ကိုယ်စီရှိကြပါတယ်။ တစ်ချို့ Tool တွေက Command Line Tool တွေ ဖြစ်ပြီး တစ်ချို့ကိုတော့ IDE တွေနဲ့အတူ တွဲဖက်အသုံးပြုလို့ရတက်ကြပါတယ်။

(၁၀) ဆအလုပ်ပိုတွင်တဲ့ Rockstar Developer တစ်ဦးအနေနဲ့ ဒီလို Workflow Automation Tool တွေကို လေ့လာပြီး ထိရောက်အောင် အသုံးချတက်ခြင်းဟာ လိုအပ်တဲ့အရည်အချင်းတစ်ရပ်ပဲဖြစ်ပါတယ်။

အပိုင်း (၃)

Modern Development Stack

ပြုပြင်ထိမ်းသိမ်းရလွယ်ကူတဲ့ Code ဖြစ်ဖို့အတွက်
စနစ်ကျတဲ့ Code Structure Architecture တစ်ခုကို
လိုက်နာအသုံးပြုဖို့လိုပါတယ်။

Ubuntu – သင့်အတွက် Linux (စာအုပ်)

Ubuntu Linux အသုံးပြုနည်းအား Installation မှ စတင်၍ System Configuration နှင့် Development Environment တည်ဆောက်ခြင်းအထိ အဆင့်လိုက်ရေးသား ဖော်ပြထားသည့်စာအုပ် အောက်ပါလိပ်စာတွင် အခမဲ့ ရယူနိုင်သည်။

http://u4ubook.com

အခန်း(၉) – BackboneJS (Client-Side MVC)

အခန်း (၃) မှာ Service Oriented Architecture (SOA) အကြောင်း ဖော်ပြခဲ့စဉ်က Software တစ်ခုကို တည်ဆောက်ရာမှာ သီးခြားစီအလုပ်လုပ်နိုင်ပြီး၊ အပြန်အလှန်လည်း ဆက်သွယ်ဆောင်ရွက်နိုင်တဲ့ ပရိဂရမ်ငယ်များအဖြစ် တည်ဆောက်ခြင်းကနေ ရရှိနိုင်တဲ့အကျိုးကျေးဇူးတွေကို ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ Mobile Device တွေ အသုံးတွင်ကျယ်လာပြီး Mobile ခေတ်ဖြစ်လာတဲ့ ကနေ့အချိန်မှာ Software Application တစ်ခုကို One Core / Multiple Clients ပုံစံ တည်ဆောက်ခြင်းနည်းဟာ တွင်ကျယ်လာတဲ့ နည်းစနစ်တစ်ခု ဖြစ်လာခဲ့ပါတယ်။ App ရဲ့ Core Logic ကို သီးခြားအလုပ်လုပ် နိုင်တဲ့ Service တစ်ခုအနေနဲ့တည် ဆောက်လိုက်ခြင်းအားဖြင့် Device တစ်မျိုးအတွက် App တစ်ခုစီ သီးခြားထပ်မံတည် ဆောက်နေဖို့မလိုတော့ပဲ၊ UI Service တွေကိုသာ ထပ်မံတည်ဆောက်ဖို့ လိုတဲ့သဘော ဖြစ်ပါတယ်။

Web Browse အတွင်းမှာ အလုပ်လုပ်တဲ့ Web App တွေဟာလည်း အဓိက App Interface အဖြစ်ကနေ Client များစွာထဲက Client တစ်ခုအဖြစ်ကို ရောက်ရှိလာခဲ့ကြပါတယ်။ အရင်က Web Application ဆိုတာ၊ လုပ်ဆောင်ချက်အားလုံးကို Server ပေါ်မှာပဲဆောင်ရွက်ပါတယ်။ အဓိက Core Logic တွေကို Process လုပ်တဲ့ အလုပ်တွေ၊ Data အချက် အလက်တွေ သိမ်းဆည်း ရယူတဲ့လုပ်ငန်းတွေနဲ့ UI Template တွေ ဖန်တီးတဲ့ လုပ်ငန်းတွေ၊ အားလုံးကို Server ပေါ်မှာပဲ လုပ်ပြီး၊ Client ဖြစ်တဲ့ Browser က၊ User Input တွေကို လက်ခံတဲ့ အလုပ်နဲ့၊ Server ပေါ်မှာ အလုပ်လုပ်ပြီး နောက်ဆုံး ရလာတဲ့ရလဒ်ကို ဖော်ပြပေးတဲ့ အလုပ်ကိုသာ လုပ်ပါတယ်။ တစ်နည်းအားဖြင့် Service တစ်ခုအနေနဲ့ သီးခြားအလုပ် လုပ်တာမျိုးမဟုတ်ပဲ Server ပေါ်မှာ မှီခိုအလုပ်လုပ်ကြခြင်းဖြစ်ပါတယ်။ JavaScript ဆိုတာလည်း User Interface ကို ပြည့်စုံအောင် ဖြည့်စွက်တဲ့ ဖြည့်စွက်နည်းပညာ တစ်ခုသာဖြစ်ခဲ့ပါတယ်။

နောက်ပိုင်းမှာတော့ Web App တွေလည်း Server ကို မမှီခိုတော့ပဲ သီးခြားအလုပ်လုပ်နိုင်တဲ့ Service များအဖြစ် တည်ဆောက်လာကြပါတယ်။ Server ကို Core Service တစ်ခုအနေနဲ့ လိုအပ်သလို ဆက်သွယ်ဆောင်ရွက်ပေးမယ့်၊ အဓိက လုပ်ငန်းအများစုကို Server ကို မှီခိုခြင်းမပြုတော့ပဲ ကိုယ်တိုင်ရပ်တည်နိုင်တဲ့ သီးခြားပရိဂရမ်အဖြစ် တည်ဆောက်လာကြ ခြင်းဖြစ်ပါတယ်။ JavaScript ဆိုတာလည်း အရင်လို User Interface ပိုင်းကို ဖြည့်စွက်ပေးတဲ့ Language သက်သက် မဟုတ်တော့ပဲ Client Service တစ်ခုလုံးကို တည်ဆောက်ရာမှာ အသုံးပြုတဲ့ Programming Language တစ်ခုဖြစ်လာပါ တယ်။ တနည်းအားဖြင့် JavaScript ဆိုတာ မူလက App ရဲ့ UI ပိုင်းကိုသာ စီမံဖို့သုံးခဲ့ပေမယ့်၊ ကနေ့အချိန်မှာတော့ UI ရော Data ကိုပါ စီမံဖို့အသုံးပြုလာကြခြင်းပဲ ဖြစ်ပါတယ်။

Software Development မှာ ရေရှည် Maintenance ကောင်းစေဖို့အတွက် Data ပိုင်းလုပ်ဆောင်ချက်နဲ့၊ UI ပိုင်းလုပ် ဆောင်ချက်တို့ကို ခွဲခြားရေးသားကြလေ့ရှိပါတယ်။ ဒီတော့မှ Data နဲ့ UI ကို သီခြားစီခွဲပြီး Maintain လုပ်နိုင်တဲ့အတွက် အမှားနည်းပြီး ပြင်ဆင်ထိမ်းသိမ်းရ လွယ်ကူစေမှာဖြစ်ပါတယ်။ ဒီနေရာမှာ အသုံးများတဲ့

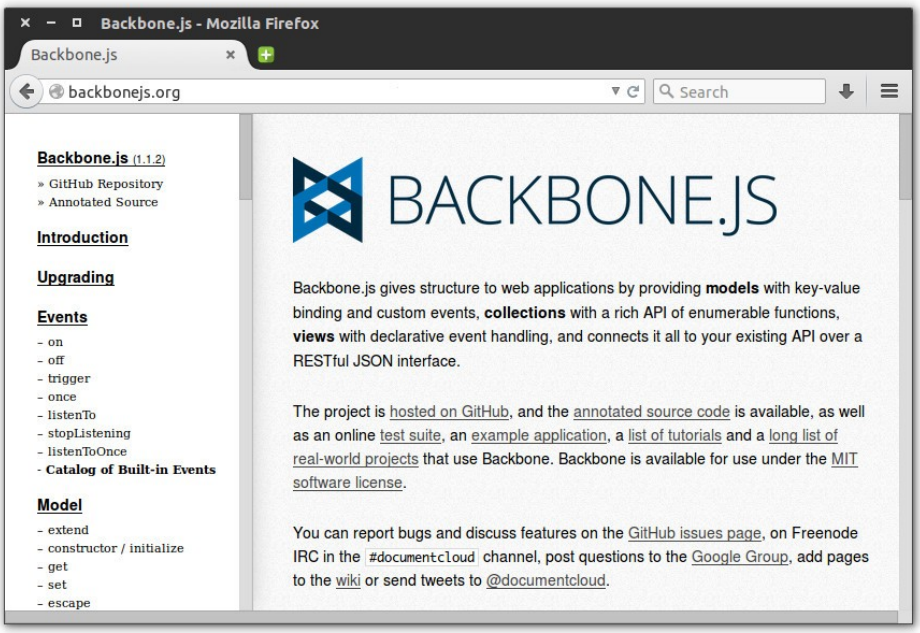
နည်းစနစ်ကတော့ MVC လို့ ခေါ်တဲ့ Model-View-Controller Design Pattern ပဲဖြစ်ပါတယ်။ MVC အကြောင်းကို **Professional Web Developer** ရဲ့ **အခန်း (၁၃)** မှာ ဖော်ပြထားပါတယ်။

BackboneJS ဟာ JavaScript Code တွေကို MVC Pattern နဲ့ ရေးသားနိုင်ဖို့ အကူအညီပေးတဲ့ MVC Framework တစ်ခုဖြစ်ပါတယ်။ မူရင်း MVC ရဲ့ အဓိပ္ပါယ်က Model-View-Controller ဖြစ်ပါတယ်။ အခြေခံအားဖြင့် Model က Data အချက် အလက်စီမံမှုနဲ့ ပက်သက်တဲ့အပိုင်းကို တာဝန်ယူပါတယ်။ View ကတော့ User Interface ဖော်ပြမှုပိုင်းကို တာဝန်ယူပါတယ်။ Controller ကတော့ User Input တွေ စီမံမှုအပိုင်းကို တာဝန်ယူရပါတယ်။ Code ရေးသားတဲ့အခါ Data အချက်အလက် စီမံမှုအပိုင်းတွေကို Model အနေနဲ့ သီးခြားခွဲခြားရေးသားရပြီး၊ UI ဖော်ပြမှုအပိုင်းကို Model Code နဲ့ မရောပဲ View အနေနဲ့ သီးခြားရေးသားခြင်းဖြစ်ပါတယ်။ ဘယ်လို User Input လက်ခံရရှိရင် ဘာအလုပ်လုပ်ရ မလဲဆိုတဲ့ ကိစ္စကို စီမံပေးတဲ့ Code ကို တော့ Model ရော View နဲ့ပါ မရောပဲ Controller အနေနဲ့ သီးခြားရေးသားရ ပါတယ်။

BackboneJS မှာတော့ MVC ဆိုတာ Model-View-**Collection** ဆိုတဲ့အဓိပ္ပါယ်ဖြစ်ပါတယ်။ Data အချက်အလက် စီမံမှုပိုင်းဖြစ်တဲ့ Model နဲ့ UI ဖော်ပြမှုပိုင်းဖြစ်တဲ့ View တို့ကို ခွဲခြားရေးသားနိုင်အောင် အကူအညီပေးပါတယ်။ Collection ဆိုတာကတော့ Model ကို ထောက်ပံ့ဖြည့်စွက်ပေးတဲ့ Model Collection ဖြစ်ပါတယ်။ Controller သဘော မျိုး သီးခြားလုပ်ဆောင်ချက်မပါဝင်ပဲ၊ ဘယ်လို User Event မှာ ဘယ်လုပ်ငန်းလုပ်ရမလဲ စီမံသတ်မှတ်တဲ့အပိုင်းကို View နဲ့အတူ တွဲဖက်ရေးသားရပါတယ်။

9.1 – Getting BackboneJS

BackbonJS ကို backbonejs.org မှာကိုယ်တိုင် Download လုပ်ယူနိုင်သလို၊ Yeoman ကိုသုံးပြီးတော့လည်း Generate လုပ်ယူနိုင်ပါတယ်။



ပုံ (၉.၁) - BackboneJS Website - backbonejs.org

BackboneJS က Model Collection တွေကို စီမံဖို့အတွက် UnderscoreJS လို့ခေါ်တဲ့ JavaScript Library တစ်ခုကို အသုံးပြုထားပြီး၊ DOM Manipulation အတွက် jQuery ကို အသုံးပြုထားပါတယ်။ တစ်နည်းအားဖြင့် Underscore နဲ့ jQuery တို့ဟာ Backbone အတွက် Dependency တွေဖြစ်ကြပါတယ်။ ဒါကြောင့် Yeoman Generator ကို သုံးပြီး Backbone App Boilerplate တစ်ခုကို Generate လုပ်မယ်ဆိုရင်လိုအပ်တဲ့ Dependency တွေ တစ်ခါတည်းပါဝင်သွားမှာ ဖြစ်ပါတယ်။ Yeoman အတွက် Backbone Generator ကို Install မလုပ်ရသေးရင် အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ sudo npm install -g generator-backbone
```

Backbone Generator ကို Install လုပ်ပြီးပြီဆိုရင်တော့ Backbone App Boilerplate တစ်ခုကို အခုလို တည်ဆောက် ယူနိုင်ပါတယ်။

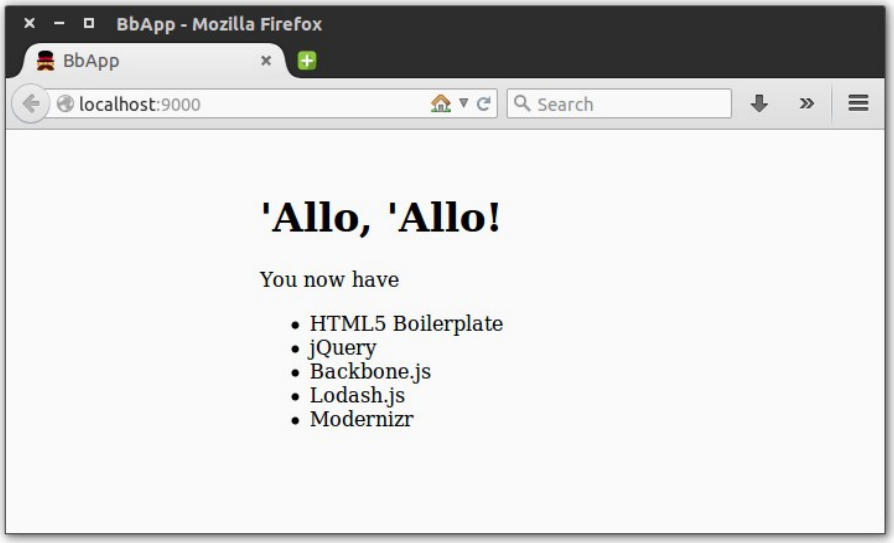
```
$ mkdir bbApp && cd bbApp
$ yo backbone
```

ရရှိလာတဲ့ Directory Structure ကို လေ့လာကြည့်ရင် အခုလိုတွေ့ရမှာ ဖြစ်ပါတယ်။

```
bbApp
├── app
│   ├── bower_components
│   │   ├── backbone
│   │   ├── jquery
│   │   ├── lodash
│   │   ├── modernizr
│   │   └── underscore
│   ├── images
│   ├── scripts
│   │   ├── vendor
│   │   └── main.js
│   ├── styles
│   │   └── main.css
│   ├── 404.html
│   ├── favicon.ico
│   └── index.html
├── node_modules
├── test
├── bower.json
├── Gruntfile.js
└── package.json
```

`bower_components` Directory ထဲမှာ `backbone`, `jquery`, `underscore` စသဖြင့် လိုအပ်တဲ့ Dependency အားလုံးပါဝင်လာတာကို တွေ့ရမှာဖြစ်ပါတယ်။ `bower_components` ထဲမှာ တွေ့နေရတဲ့ `lodash` ဆိုတာ Underscore အစား၊ အစားထိုးသုံးနိုင်တဲ့ Underscore နဲ့ လုပ်ဆောင်ချက်တူညီတဲ့ Library တစ်ခုဖြစ်ပါတယ်။

App Boilerplate Directory ဖြစ်တဲ့ `bbApp` Directory ထဲမှာ `grunt serve` Command ကို Run ကြည့်ရင် ကျွန်တော်တို့ ရဲ့ App ကို Web Browser နဲ့ အခုလို ဖော်ပြလာမှာပဲဖြစ်ပါတယ်။



ပုံ (၉.၂) - Previewing App

Yeoman က Scaffold Generate လုပ်စဉ်မှာ Web Server တစ်ခုကိုပါ တစ်ခါတည်း ထည့်သွင်းပေးသွားတဲ့အပြင် Browser Sync လို့ခေါ်တဲ့ NodeJS Package တစ်ခုကိုပါ တစ်ခါတည်း Setup လုပ်ပေးသွားတဲ့အတွက် အခုနေ App မှာ ပြင်ဆင်မှုတစ်ခုလုပ်လိုက်ရင် Browser မှာဖော်ပြတဲ့ ရလဒ်မှာ တစ်ခါတည်း လိုက်လံပြောင်းလဲသွားမှာ ဖြစ်ပါတယ်။ Development မှာ အလုပ်တွင်စေဖို့အတွက် ကူညီပေးတဲ့ အသုံးဝင်တဲ့ လုပ်ဆောင်ချက်များ ဖြစ်ပါတယ်။

> ဒီအခန်းမှာ ဖော်ပြထားတဲ့ Code တွေဟာ လက်တွေ့ကူးယူစမ်းသပ်နိုင်တဲ့ Code တွေဖြစ်ပါတယ်။

9.2 – Backbone Model

Model ဆိုတာ Data နဲ့ အဲ့ဒီ Data တွေကို စီမံနိုင်တဲ့ Function တွေစုစည်းပါဝင်တဲ့ Object တစ်ခုဖြစ်ပါတယ်။ Backbone မပါပဲနဲ့ JavaScript Model Object တစ်ခုကို အခုလို တည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
var Issue = {
  data: {
         subject: "Drop down menu not working",
         reproduce: " ... ",
         priority: 3,
         status: 1
  },
```

```
 8.   changeStatus: function(value) {
 9.         this.data.status = value;
10.   }
11. }
```

Issue Object အတွင်းမှာ အချက်အလက်တွေစုစည်းထားတဲ့ `data` နဲ့ `changeStatus()` လို့ ခေါ်တဲ့ အဲဒီ Data တွေကို စီမံနိုင်တဲ့ Function တို့ပါဝင်ခြင်းဖြစ်ပါတယ်။ ဒီလို JavaScript သက်သက်နဲ့ Model Object တွေကို တည်ဆောက်နိုင်ပေမယ့် Backbone Model ကို အသုံးပြုပြီး Model တွေတည်ဆောက်မယ်ဆိုရင် အသုံးဝင်တဲ့ လုပ်ဆောင်ချက်တွေ အသင့်ပါဝင်သွားမှာ ဖြစ်ပါတယ်။ Backbone Model Constructor တစ်ခုကို အခုလို တည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
 1. var Issue = Backbone.Model.extend({
 2.
 3.   defaults: function() {
 4.         return {
 5.               subject: "",
 6.               reproduce: "",
 7.               priority: 2,
 8.               status: 1
 9.         };
10.   },
11.
12.   changeStatus: function(value) {
13.         this.save({status: value});
14.      }
15.
16. });
```

Backbone Model Constructor တစ်ခုတည်ဆောက်ဖို့အတွက် `Backbone.Model.extend()` ကို သုံးရပါတယ်။ နမူနာအရ `Issue` ဟာ Model Constructor တစ်ခုဖြစ်သွားပြီဖြစ်ပါတယ်။ ဒါကြောင့် ဒီ Constructor ကို အသုံး ပြုပြီး Model Object တွေ တည်ဆောက်နိုင်မှာဖြစ်ပါတယ်။ Model Object တစ်ခု တည်ဆောက်လိုက်တဲ့အခါ Constructor ရဲ့ `defaults` Property မှာသတ်မှတ်ထားတဲ့ တန်းဖိုးတွေကို Model Object မှာ အလိုအလျောက် တွဲဖက် သတ်မှတ်ပေးသွားမှာ ဖြစ်ပါတယ်။ ဥပမာ – `Issue` Model Constructor ကိုသုံးပြီး Model Object တွေကို အခုလို တည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
1. var issueOne = new Issue();
2.
3. var issueTwo = new Issue({
4.   subject: "Dropdown menu not working",
5.   priority: 3,
6.   status: 1
7. });
```

`issueOne` လို့ခေါ်တဲ့ Model Object တစ်ခုတည်ဆောက်ရာမှာ တန်ဖိုးတွေ ထည့်သွင်းပေးခြင်းမရှိတဲ့အတွက် Constructor ရဲ့ `defaults` တန်ဖိုးတွေကို အသုံးပြုပေးသွားမှာဖြစ်ပါတယ်။ `issueTwo` ကိုတည်ဆောက်ရာမှာတော့ `subject`, `priority`, `status` စတဲ့တန်ဖိုးတွေ ထည့်သွင်းသတ်မှတ်ပေးလိုက်တဲ့အတွက် `defaults` တန်ဖိုး တွေနေရာမှာ သက်ဆိုင်ရာတန်ဖိုးကို အစားထိုးအသုံးပြုပေးသွားမှာဖြစ်ပါတယ်။

Model Object တစ်ခုတည်ဆောက်လိုက်တဲ့အခါ Backbone က ကြိုတင်သတ်မှတ်ထားပေးတဲ့ Function တွေနဲ့ Property တွေကို Object မှာ တစ်ခါတည်း တွဲဖက်ပေးသွားမှာဖြစ်ပါတယ်။ Backbone က Model Object မှာ တွဲဖက် ပေးလိုက်တဲ့ Function နဲ့ Property စာရင်းကို ဖော်ပြလိုက်ပါတယ်။

`get()` – Model ရဲ့ Data တန်ဖိုးတစ်ခုကို ရယူလိုတဲ့အခါ သုံးရပါတယ်။ ဥပမာ –

JavaScript

```
issueTwo.get("priority");       // => 3
```

`set()` – Model ရဲ့ Data တန်ဖိုး သတ်မှတ်ရာမှာသုံးရပါတယ်။ ဥပမာ –

JavaScript

```
issueTwo.set({priority: 2, status: 1});
issueTwo.get("priority");       // => 2
```

`escape()` – `get()` နဲ့ အလုပ်လုပ်ပုံအတူတူပါပဲ။ XSS Filter ပါ တစ်ခါတည်း ပြုလုပ်ပေးပါတယ်။ ဥပမာ –

JavaScript

```
issueTwo.set({detail: "<script>alert(1)</script>"});
issueTwo.escape("detail");      // =>  &lt;script&gt;alert(1)&lt;/script&gt;
```

`has()` – Model မှာ Data တန်ဖိုး သတ်မှတ်ထားခြင်းရှိမရှိ စစ်ပေးပါတယ်။ ဥပမာ –

JavaScript

```
issueTwo.has("assignTo");       // => false
```

`unset()` – Model ရဲ့ Data တန်ဖိုးတစ်ခုကိုဖျက်ပေးပါတယ်။ ဥပမာ –

JavaScript

```
issueTwo.unset("subject");
```

`clear()` – Model ရဲ့ Data တန်ဖိုးအားလုံးကို ပယ်ဖျက်ပေးပါတယ်။

`id` – Model Object တိုင်းမှာ Unique ID တစ်ခုရှိရပါတယ်။ အဲ့ဒီ ID ကို Data နဲ့မရောပဲ `id` Attribute နဲ့ သီးခြား သိမ်းဆည်းပေးထားခြင်းဖြစ်ပါတယ်။

`idAttribute` – Model Data တွေထဲက တန်ဖိုးတစ်ခုကို Model ID အဖြစ် သတ်မှတ်လိုရင် သုံးနိုင်ပါတယ်။

JavaScript

```
var Issue = Backbone.Model.extend({
 idAttribute: "issueID",
 defaults: {
        subject: "",
        status: 1
 }
});

var issueThree = new Issue({ issueID: '001', status: 2 });
console.log(issueThree.id); // => 001
```

`issueThree` လို့ခေါ်တဲ့ Model Object တစ်ခုတည်ဆောက်ရာမှာ `issueID: '001'` ဆိုတဲ့တန်ဖိုးတစ်ခုကို တစ်ခါတည်း တွဲဖက်ပေးထားပါတယ်။ Model Constructor မှာ `idAttribute: "issueID"` လို့သတ်မှတ်ထား တဲ့အတွက် `issueID: '001'` အနေနဲ့ ပါဝင်လာတဲ့တန်ဖိုးကို Backbone က အလိုအလျောက် Model ID အနေနဲ့ သတ်မှတ်ပေးသွားမှာဖြစ်ပါတယ်။ ဒါကြောင့် `issueThree.id` ကို ရယူကြည့်တဲ့အခါ 001 ဖြစ်နေခြင်းဖြစ်ပါတယ်။

တစ်ခုသတိပြုပါ Model Object ရဲ့ Data တွေကို ရယူလိုရင် `get()` Function ကနေတစ်ဆင့် ရယူရပါတယ်။ ဥပမာ – `issueThree.get("status")`။ ID က Model Data နဲ့မဆိုင်ပါဘူး။ Model Property ဖြစ်ပါတယ်။ ဒါကြောင့် `get()` Function မလိုပဲ `issueThree.id` လို့ တိုက်ရိုက်ရယူခြင်းဖြစ်ပါတယ်။

`cid` – Model Object တစ်ခုတည်ဆောက်လိုက်တာနဲ့ Backbone က အလိုအလျောက် Temporary ID တစ်ခုတည် ဆောက်ပေးပြီး `cid` Property နဲ့ သိမ်းဆည်းပေးသွားပါတယ်။ တစ်ကယ့် ID မရှိသေးခင် Unique ID တစ်ခုအနေနဲ့ အသုံးပြုဖို့ လိုအပ်တဲ့အခါ `cid` ကိုယာယီ အသုံးပြုနိုင်ခြင်းဖြစ်ပါတယ်။ ဥပမာ –

JavaScript

```
console.log(issueTwo.cid);      // => c1
```

`toJSON()` – Model Data တွေကို JSON Object တစ်ခုအနေနဲ့ ပြန်ပေးတဲ့ Function ဖြစ်ပါတယ်။ ဥပမာ –

JavaScript

```
issueTwo.toJSON();
// => { subject: "Dropdown menu not working", priority: 3, status: 1 }
```

`fetch()` – Server ကနေ အချက်အလက်တွေကို ဆက်သွယ်ရယူပြီး လက်ရှိ Model Object ကို Update လုပ်ပေး ပါတယ်။ ဆက်သွယ်ရမယ့် Server လိပ်စာ URL အနေနဲ့ Model ပါဝင်တဲ့ Collection မှာ သတ်မှတ်ထားတဲ့ URL ကို အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။ Request Method အနေနဲ့ GET ကို သုံးပေးသွားမှာပါ။

`save()` – Server ကိုဆက်သွယ်ပြီး လက်ရှိ Model Object မှာပါဝင်တဲ့ အချက်အလက်တွေကို ပေးပို့ပေးပါတယ်။ ဆက်သွယ်ရမယ့် Server လိပ်စာ URL အနေနဲ့ Model ပါဝင်တဲ့ Collection မှာ သတ်မှတ်ထားတဲ့ URL ကို အသုံးပြု သွားမှာဖြစ်ပါတယ်။ Request Method အနေနဲ့ Server ကို တစ်ကြိမ်မှ မပေးပို့ရသေးရင် POST ကို သုံးပေးသွားမှာဖြစ် ပြီး၊ တစ်ကြိမ်ပေးပို့ဖူးရင်တော့ PUT ကို အသုံးပြုပေးသွားမှာဖြစ်ပါတယ်။

`destroy()` – Server ကိုဆက်သွယ်ပြီး လက်ရှိ Model Object အတွက် သိမ်းဆည်းထားတဲ့ အချက်အလက်တွေကို ပယ်ဖျက်ပေးပါတယ်။ ဆက်သွယ်ရမယ့် Server လိပ်စာ URL အနေနဲ့ Model ပါဝင်တဲ့ Collection မှာ သတ်မှတ်ထားတဲ့ URL ကို အသုံးပြုသွားမှာ ဖြစ်ပါတယ်။ Request Method အနေနဲ့ DELETE ကို သုံးပေးသွားမှာပါ။

`sync()` – Server ကို ဆက်သွယ်ပြီး ဆက်သွယ်တဲ့ Request Method ပေါ်မူတည်ပြီး Model Object မှာပါဝင်တဲ့ အချက်အလက်တွေကို ရယူခြင်း၊ သိမ်းဆည်းခြင်း၊ ပယ်ဖျက်ခြင်းတို့ကို ဆောင်ရွက်ပေးပါတယ်။ create, read, update, delete စတဲ့ Option တွေကို Request Method အနေနဲ့ တွဲဖက်ပေးရပါတယ်။

`validate()` – Default အနေနဲ့ ဘာမှသတ်မှတ်မထားပါဘူး။ ဒါပေမယ့် `save()` Function ကိုခေါ်တိုင်းမှာ Backbone က validate() Function မှာ ရေးသားသတ်မှတ်ထားတဲ့ သတ်မှတ်ချက်တွေကို အလုပ်လုပ်ပေးပါတယ်။ ဒါကြောင့် validate() Function မှာ Model Object ရဲ့ Data တွေကို စီစစ်တဲ့အလုပ်ကို ရေးသားထားသင့်ပါတယ်။

`validateError` – `validate()` Function က `return` ပြန်ပေးတဲ့ Error Message ကို သိမ်းထားပေးပါတယ်။ Error Message ကို တစ်နေရာမှာ ပြန်လည်ဖော်ပြလိုတဲ့အခါ ဒီ Property ထဲကနေ ရယူဖော်ပြနိုင်လို့ အသုံးဝင် ပါတယ်။

`isValid()` – `validate()` Function ကို Run ပေးပါတယ်။

`url()` – လက်ရှိ Model Object ရဲ့ URL ကို ပြန်ပေးပါတယ်။ `collection.url/id` ဆိုတဲ့ Format နဲ့ ပြန်ပေးမှာဖြစ်ပါတယ်။ Model ပါဝင်တဲ့ Collection မှာ သတ်မှတ်ထားတဲ့ URL တန်းဖိုးကိုရှေ့ကထားပြီး Model ID ကို နောက်ကတွဲပေးပါတယ်။

`parse()` – Server က ပြန်ပေးတဲ့ JSON String ကို JSON Object ဖြစ်အောင် ပြောင်းပေးပါတယ်။ `save()` Function နဲ့ `fetch()` Function တို့ကို Run တိုင်း `parse()` Function ကို အလိုအလျောက် အလုပ်လုပ်ပေးပါတယ်။

`clone()` – လက်ရှိ Model Object ကို မူပွားပေးပါတယ်။

`isNew()` – Model Object ဟာ Server မှာ တစ်ကြိမ်မှသွားမသိမ်းရသေးတဲ့ Object အသစ်တစ်ခု ဟုတ်မဟုတ် စီစစ်ပေးပါတယ်။ ID ရှိမရှိကိုကြည့်လိုက်တာပါ။ ID မရှိသေးရင် Model အသစ်၊ ရှိနေရင် Server မှာ သွားသိမ်းထားပြီး သား Model လို့ ယူဆလိုက်ခြင်း ဖြစ်ပါတယ်။

`hasChanged()` – နောက်ဆုံးတစ်ခေါက် `save()` လုပ်ပြီးနောက်ပိုင်း Model Object မှာပါဝင်တဲ့အချက်အလက်တွေကို ပြင်ဆင်ထားခြင်းရှိမရှိ စီစစ်ပေးပါတယ်။

`changedAttributes()` – နောက်ဆုံးတစ်ခေါက် `save()` ပြီးနောက်ပိုင်း တန်ဖိုးပြောင်းလဲထားတဲ့ Data တွေကို ရွေးချယ်ရယူပေးပါတယ်။

`previous()` – Model Object ရဲ့ တန်ဖိုးတစ်ခုကို `set()` သို့မဟုတ် `save()` နဲ့ ပြောင်းလိုက်ချိန်မှာ ပြောင်း လိုက်တဲ့တန်ဖိုးမတိုင်ခင်က၊ မူလတန်ဖိုးတစ်ခုကို ပြန်လည်ရရှိလိုရင် အသုံးပြုနိုင်ပါတယ်။

`previousAttributes()` – Model Object ရဲ့ တန်ဖိုးတစ်ခုကို `set()` သို့မဟုတ် `save()` နဲ့ ပြောင်းလိုက်ချိန်မှာ ယခင်မူလတန်ဖိုးအားလုံးကို ပြန်လည်ရရှိလိုတဲ့အခါ အသုံးပြုနိုင်ပါတယ်။

Backbone Model တစ်ခုတည်ဆောက်လိုက်တာနဲ့ ဒီ Function တွေနဲ့ Property တွေ အလိုအလျောက် တွဲဖက်ပါဝင် သွားမှာပဲဖြစ်ပါတယ်။

9.3 – Backbone Collection

Collection ဆိုတာဟာ Model တွေကိုစုစည်းထားတဲ့ Model Object Array တစ်ခု ဖြစ်ပါတယ်။ Object Array တစ်ခု ကို JavaScript သက်သက်နဲ့ ကိုယ်တိုင်လည်း တည်ဆောက်နိုင်ပေမယ့် Backbone Collection ကို အသုံးပြုတည်ဆောက်ရင် အသုံးဝင်တဲ့ လုပ်ဆောင်ချက်တွေကို အသင့်ရရှိနိုင်မှာ ဖြစ်ပါတယ်။ `Backbone.Collection.extend()` ကို အသုံးပြုပြီး Collection Constructor တစ်ခုကိုတည်ဆောက်နိုင်ပါတယ်။ အဲ့ဒီလိုတည်ဆောက်ရာမှာ ပါဝင်နိုင်တဲ့ Option တွေအများကြီးရှိပေမယ့် မဖြစ်မနေ ပါဝင်သင့်တဲ့ Option နှစ်ခုကတော့ `model` နဲ့ `url` ဖြစ်ပါတယ်။ ဥပမာ –

JavaScript

```
1. var IssueList = Backbone.Collection.extend({
2.     url: 'http://localhost/api/'
3.     model: Issue
```

```
4. });
```

Model Data တွေကို Database စနစ်တစ်ခုနဲ့ သိမ်းဆည်းလိုတဲ့အခါ Backbone က `url` မှာ သတ်မှတ်ထားတဲ့ လိပ်စာကို အသုံးပြုပြီး Server နဲ့ ဆက်သွယ်ဆောင်ရွက်ပေးသွားမှာ ဖြစ်ပါတယ်။ `model` ကတော့ ဒီ Collection မှာ ပါဝင်မယ့် Model အမျိုးအစားကို သတ်မှတ်ပေးဖို့ ဖြစ်ပါတယ်။ Collection ထဲကို ထည့်သွင်းတဲ့ တန်ဖိုးတွေက Model Object တွေမဟုတ်ပဲ ရိုးရိုး JavaScript Array တစ်ခုဖြစ်နေရင်လည်း Backbone က model မှာ သတ်မှတ်ထားတဲ့ Model Object အမျိုးအစားဖြစ်အောင် အလိုအလျောက်ပြောင်းပြီးမှ ထည့်သွင်းပေးသွားမှာဖြစ်ပါတယ်။ တည်ဆောက် ရရှိထားတဲ့ Collection Constructor ကိုသုံးပြီး Backbone Collection တစ်ခုကို အခုလို တည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
var Issues = new IssueList();
```

အဲဒီလို တည်ဆောက်လိုက်တဲ့အခါ Backbone က ဖြည့်စွက်ပေးလိုက်တဲ့အတွက် Collection Object မှာ တွဲဖက်ပါဝင် သွားတဲ့ Function နဲ့ Property စာရင်းကို ဖော်ပြပေးလိုက်ပါတယ်။

`toJSON()` – Collection ထဲက Model တွေကို JSON Array အနေနဲ့ ပြန်ပေးပါတယ်။

`add()` – Model တစ်ခု (သို့မဟုတ်) Model Array တစ်ခုကို Collection ထဲကို ထည့်သွင်းလိုတဲ့အခါ သုံးပါတယ်။ ဥပမာ –

JavaScript

```
Issues.add([issueOne, issueTwo]);
```

`remove()` – Model တစ်ခုကို Collection ကနေ ထုတ်လိုတဲ့အခါသုံးပါတယ်။

`reset()` – Collection ထဲမှာရှိတဲ့ လက်ရှိ Model တွေကို ပေးလိုက်တဲ့ Model စာရင်းသစ်နဲ့ အစားထိုးပေးပါတယ်။

`set()` – `reset()` နဲ့ သဘောတရားဆင်ပါတယ်။ ဒါပေမယ့် သူက စာရင်းတစ်ခုလုံးကို အစားထိုးတာမျိုးမဟုတ်ပဲ၊ မူလ Model စာရင်းနဲ့၊ ပေးလိုက်တဲ့ Model စာရင်းသစ်ကို တိုက်ကြည့်ပါတယ်။ Model စာရင်းသစ်မှာပါပြီး မူလကမရှိ သေးရင် အဲ့ဒီ Model ကို အသစ်အနေနဲ့ ထည့်ပေးပါတယ်။ Model စာရင်းသစ်မှာပါပြီး မူလကလည်း ရှိတယ်ဆိုရင် အဲ့ဒီ Model ကို Update လုပ်ပေးပါတယ်။ Model စာရင်းသစ်မှာမပါပေမယ့် မူလကရှိနေတယ်ဆိုရင် ပယ်ဖျက်ပေးပါတယ်။

`get()` – Collection ထဲကနေ Model တစ်ခုကို ရယူလိုတဲ့အခါသုံးပါတယ်။ Model ရဲ့ `id` သို့မဟုတ် `cid` ကို အသုံးပြုရယူရပါတယ်။ ဥပမာ –

JavaScript

```
Issues.get("c1");
```

at() – Collection ထဲကနေ Model တစ်ခုကို ရယူခြင်းပဲဖြစ်ပါတယ်။ ဒါပေမယ့် သူက Model ID ကို မသုံးပဲ Model Index ကို အသုံးပြုရယူပေးပါတယ်။ ဥပမာ –

JavaScript

```
Issues.at(2);
```

push() – Model တစ်ခုကို Collection ရဲ့ နောက်ဆုံးကနေ ထည့်ပေးပါတယ်။ `add()` နဲ့ဆင်ပါတယ်။

pop() – Collection ထဲက နောက်ဆုံး Model ကို ဖြုတ်ထုတ်ပေးပါတယ်။ `remove()` နဲ့ဆင်ပါတယ်။

unshift() – Collection ထဲကို Model တစ်ခု ရှေ့ဆုံးကနေ ထည့်ပေးပါတယ်။ `add()` နဲ့ ဆင်ပါတယ်။

shift() – Collection ထဲက ရှေ့ဆုံး Model ကို ဖြုတ်ထုတ်ပေးပါတယ်။ `remove()` နဲ့ ဆင်ပါတယ်။

slice() – Collection ကို စိတ်ပိုင်းရယူလိုတဲ့အခါ သုံးနိုင်ပါတယ်။ အစ Index နဲ့ အဆုံး Index ပေးရပါတယ်။ ဥပမာ – Model တွေထဲက ထိပ်ဆုံး (၅) ခုကိုလိုချင်တယ်ဆိုရင် အခုလိုရယူနိုင်ပါတယ်။

JavaScript

```
var topFive = Issues.slice(0, 5);
```

length – Collection ထဲမှာရှိတဲ့ Model အရေအတွက်ကို ပေးပါတယ်။

comparator – Collection ထဲက Model Object တွေကို Sorting စီတဲ့အခါ အသုံးပြုစေလိုတဲ့ Model Property ကို သတ်မှတ်ပေးနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
Issues.comparator = "priority";
```

`comparator` ကို `priority` လို့သတ်မှတ်လိုက်တဲ့အတွက် Model Object တွေကို `priority` အလိုက် Sorting စီပေးသွားမှာ ဖြစ်ပါတယ်။

`sort()` – Model Object တစ်ခုကို ပြင်ပြီးနောက် Collection ထဲက Model Object တွေကို Sorting ပြန်စီစေလိုတဲ့အခါ သုံးရပါတယ်။

`pluck()` – Model Object တွေရဲ့ Property တစ်ခုကို Array အနေနဲ့ လိုချင်တဲ့အခါသုံးရပါတယ်။ ဥပမာ –

JavaScript

```
Issues.pluck("subject");
```

Model Object တွေအားလုံးရဲ့ `subject` တွေကိုချည်းရွေးချယ်ပြီး Array အနေနဲ့ ပြန်ပေးသွားမှာဖြစ်ပါတယ်။

`where()` – Filter လုပ်ထားတဲ့ Model Object စာရင်းကို ရယူဖို့အတွက် သုံးနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
Issues.where({status: 2});
```

Model Object တွေထဲက `status: 2` တန်ဖိုးရှိတဲ့ Object စာရင်းကိုပဲ ရွေးထုတ်ပေးသွားမှာဖြစ်ပါတယ်။

`findWhere()` – `where()` နဲ့သဘောတရား အတူတူပါပဲ။ ဒါပေမယ့် `findWhere()` က Object စာရင်းပြန်မပေးပဲ ပထမဆုံးတွေ့တဲ့ Object တစ်ခုကိုသာ ပြန်ပေးပါတယ်။

`parse()` – Model ရဲ့ `parse()` နဲ့ သဘောတရားအတူတူပါပဲ။ Server နဲ့ဆက်သွယ်တဲ့လုပ်ငန်းတွေမှာ Server ပြန်ပေးတဲ့ Object String ကို JSON Object ပြောင်းပေးပါတယ်။

`clone()` – လက်ရှိ Collection ကို မူပွားပေးပါတယ်။

`fetch()` – URL မှာ သတ်မှတ်ထားတဲ့ လိပ်စာကိုအသုံးပြုပြီး Server နဲ့ဆက်သွယ်ပြီး Model စာရင်းကို ရယူပေး ပါတယ်။ ရရှိလာတဲ့အခါ `set()` Function ကိုသုံးပြီး လက်ရှိ Collection ကို Update လုပ်ပေးပါတယ်။ GET Request Method ကိုသုံးပါတယ်။

`create()` – Model တစ်ခုတည်ဆောက်ပြီး တည်ဆောက်ရရှိထားတဲ့ Model ကို Collection ထဲမှာထည့်သွင်းပေး သလို URL မှာ သတ်မှတ်ထားတဲ့ လိပ်စာကိုအသုံးပြု ပြီး Server ကိုလည်း တစ်ခါတည်း ပေးပို့ပေးပါတယ်။ POST Request Method ကို သုံးပါတယ်။

`sync()` – Model ရဲ့ `sync()` နဲ့ သဘောသဘာဝတူပါတယ်။ Server နဲ့ဆက်သွယ်ပြီး၊ Collection တွေကို ပေးပို့ ခြင်း၊ ရယူခြင်း၊ ပြင်ဆင်ခြင်း၊ ပယ်ဖျက်ခြင်းတို့ကို ဆောင်ရွက်ပေးပါတယ်။ create, read, update, delete တို့

ကို Request Method အနေနဲ့ တွဲဖက်ပေးပို့နိုင်ပါတယ်။
ဒီ Function တွေနဲ့ Property တွေအပြင် UnderscoreJS ရဲ့ Array တွေစီမံတဲ့ Function တွေကိုလည်း Collection နဲ့ အတူ တွဲဖက်အသုံးပြုနိုင်ပါတယ်။ Underscore Function တွေထဲက မှတ်သားသင့်တာတွေကို ဖော်ပြလိုက်ပါတယ်။

`each()` – Collection တစ်ခုလုံးကို Loop လုပ်စေလိုတဲ့အခါ `each()` ကို သုံးနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
Issues.each(function(model) {
     console.log( model.get("subject") );
});
```

`map()` – `each()` လိုပဲ Collection တစ်ခုလုံးကို Loop ပါတ်ပေးပါတယ်။ ကွာသွားတာကတော့၊ `map()` Function က အဆုံးသတ်ရလဒ် ကို Array အနေနဲ့ ပြန်ပေးမှာ ဖြစ်ပါတယ်။ ဥပမာ –

JavaScript

```
var subjectList = Issues.map(function(model) {
     return model.get("subject");
});
```

Collection ကို Loop လုပ်ပြီး Model တွေရဲ့ subject အားလုံးကို Array အနေနဲ့ ပြန်ပေးမှာပဲဖြစ်ပါတယ်။

`find()` – Collection ကို Loop လုပ်ပြီး ပေးလိုက်တဲ့ Condition နဲ့ ကိုက်ညီတဲ့ ပထမဆုံး Model Object ကို ပြန် ပေးမှာဖြစ်ပါတယ်။ ဥပမာ –

JavaScript

```
var result = Issues.find(function(model) {
     return model.get("subject") == "Dropdown menu not working";
});
```

Collection ထဲမှာရှိတဲ့ Model တွေထဲက `subject` တန်ဖိုး Dropdown menu not working ကို ရှာပြီး တွေ့တယ် ဆိုရင် အဲဒီတန်ဖိုးပါဝင်တဲ့ Model Object ကို ပြန်ပေးမှာဖြစ်ပါတယ်။

`filter()` – `find()` နဲ့ သဘောတရားဆင်ပါတယ်။ `find()` က Condition ကိုက်ညီတဲ့ Model Object တစ်ခုကိုသာ ပြန်ပေးပေမယ့် `filter()` ကတော့ ကိုက်ညီသမျှ Model Object အားလုံးကို Array အနေနဲ့ ပြန် ပေး မှာဖြစ်ပါတယ်။ ဥပမာ –

JavaScript

```
var result = Issues.filter(function(model) {
     return model.get("status") == 2;
});
```

Collection ထဲက `status` တန်ဖိုး 2 နဲ့ညီတဲ့ Model အားလုံးကို Array အနေနဲ့ ပြန်ပေးသွားမှာဖြစ်ပါတယ်။

`reject()` – `filter()` နဲ့ဆန့်ကျင်ဘက်ဖြစ်ပါတယ်။ Condition နဲ့ မကိုက်ညီတဲ့ Model Object စာရင်းကို ပြန်ပေးမှာ ဖြစ်ပါတယ်။

`every()` – Collection ထဲက Model Object အားလုံး Condition နဲ့ ကိုက်ညီခြင်းရှိမရှိ စီစစ်ပေးပါတယ်။ ဥပမာ –

JavaScript

```
var result = Issues.every(function(model) {
     return model.get("status") == 4;
});
```

Collection ထဲက Model Object အားလုံးရဲ့ `status` တန်ဖိုး 4 ဖြစ်နေရင် `true` ရလဒ်ကို ရရှိမှာဖြစ်ပါတယ်။

`some()` – Collection ထဲက Model Object တစ်ခို့ Condition နဲ့ ကိုက်ညီခဲ့ရင် true ရလဒ်ပြန်ပေးမှာပါ။

`shuffle()` – Collection ထဲက Model Object တွေကို အစီအစဉ်အတိုင်းမဟုတ်ပဲ Random ပြန်စီပေးပါတယ်။

`toArray()` – Collection ထဲက Model တွေအားလုံးကို Array အနေနဲ့ ပြန်ပေးပါတယ်။

`first()` – Collection ထဲက ပထမဆုံး Model ကို ပြန်ပေးပါတယ်။

`last()` – Collection ထဲက နောက်ဆုံး Model ကို ပြန်ပေးပါတယ်။

`isEmpty()` – Collection စာရင်းက Model တွေမရှိပဲ အလွတ်ဖြစ်နေသလား စစ်ပေးပါတယ်။

Backbone Collection ကို Database စနစ်တစ်ခုရဲ့ Table နဲ့ အလားသဏ္ဌာန်တူတယ်လို့ယူဆနိုင်ပါတယ်။ Collection ထဲမှာ ပါဝင်တဲ့ Model တွေကတော့ Table Record များဖြစ်ပါတယ်။ RDBMS စနစ်တွေမှာ Table ထဲက Record တွေကို SQL Command တွေနဲ့ စီမံနိုင်သလိုပဲ Backbone Collection ထဲက Model တွေကိုလည်း အခုဖော်ပြခဲ့တဲ့ Function တွေ ပေါင်းစပ်အသုံးပြုခြင်းအားဖြင့် စီမံနိုင်မှာပဲဖြစ်ပါတယ်။

9.4 – Backbone View

Backbone မှာ View ဆိုတာ Optional ပါ။ Backbone View ကို မသုံးပဲ jQuery (သို့မဟုတ်) အခြားနှစ်သက်ရာ JavaScript UI Library တွေ Template Engine တွေနဲ့ User Interface ကို တည်ဆောက်နိုင်ပါတယ်။ ဥပမာ – ရိုးရိုး jQuery ကို Backbone Collection နဲ့တွဲဖက်ပြီး အခုလို အသုံးပြုနိုင်ပါတယ်။

JavaScript

```
var Issue = Backbone.Model.extend({
    defaults: {
        subject: "",
        status: 1
    }
});

var IssueList = Backbone.Collection.extend({
    model: Issue
});

var Issues = new IssueList();

var issueOne = new Issue({ subject: "Issue One", status: 1 });
var issueTwo = new Issue({ subject: "Issue Two", status: 1 });
var issueThree = new Issue({ subject: "Issue Three", status: 1 });

Issues.add([ issueOne, issueTwo, issueThree ]);

Issues.each(function(model) {
    $("ul").append("<li>" + model.get("subject") + "</li>");
});
```

နမူနာကို လေ့လာကြည့်ရင် လိုင်းနံပါတ် (၁) မှာ Issue Model Constructor ကို တည်ဆောက်ထားပါတယ်။ လိုင်းနံပါတ် (၈) မှာ `IssueList` Collection Constructor ကို တည်ဆောက်ထားပါတယ်။ လိုင်းနံပါတ် (၁၂) မှာ `IssueList` Constructor ကို သုံးပြီး `Issues` လို့ခေါ်တဲ့ Collection Object တစ်ခု တည်ဆောက်ထားပါတယ်။ လိုင်းနံပါတ် (၁၄ – ၁၅ – ၁၆) မှာ `Issue` Constructor ကိုသုံးပြီး Model Object သုံးခု တည်ဆောက်ထားပါတယ်။ တည်ဆောက်ထား တဲ့ Model Object တွေကို `add()` Function သုံးပြီး လိုင်းနံပါတ် (၁၈) မှာ `Issues` Collection ထဲ ထည့်သွင်း ထားပါတယ်။ နောက်ဆုံး လိုင်းနံပါတ် (၂၀) မှာတော့ `Issues` ကို `each()` နဲ့ Loop လုပ်ပြီး Model တွေရဲ့ subject ကို jQuery အကူအညီနဲ့ List UI အဖြစ် ဖန်တီးတည်ဆောက်လိုက်ခြင်းပဲဖြစ်ပါတယ်။ သဘောသဘာဝကို မြင်သာအောင် ဖော်ပြခြင်းဖြစ်ပါတယ်။ လက်တွေ့မှာ Backbone View ကပေးနိုင်တဲ့ အားသာချက်တွေရှိပါတယ်။

`Backbone.View.extend()` ကို အသုံးပြု ပြီး Backbone View Constructor တစ်ခုကို တည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
var IssueView = Backbone.View.extend({
    tagName:  "li",

    events: {
       "click .new": "addNew"
    },

    initialize: function() {
        this.listenTo(this.model, 'change', this.render);
    },

    render: function() {
        this.$el.html(this.model.get("status"));
        return this;
    },

    addNew: function() {
        ...
    }
});
```

အဲ့ဒီလို တည်ဆောက်ရာမှာ `tagName` Property ကိုသုံးပြီး View Template ကို ဖော်ပြစေလိုတဲ့ HTML Element Name ကို သတ်မှတ်နိုင်ပါတယ်။ `events` Property နဲ့ Event List ကို ကြေငြာထားနိုင်ပါတယ်။ နမူနာ လိုင်းနံပါတ် (၅) အရ Class မှာ `new` လို့ သတ်မှတ်ထားတဲ့ Element ကို Click လုပ်ရင် `addNew()` Function ကို အလုပ်လုပ်ပေးပါလို့ သတ်မှတ်ထားခြင်း ဖြစ်ပါတယ်။

View Constructor ကိုသုံးပြီး View Object တစ်ခုတည်ဆောက်စဉ်မှာ အလုပ်လုပ်သွားစေလိုတဲ့ လုပ်ဆောင်ချက်တွေကို initialize Property နဲ့ သတ်မှတ်ထားနိုင်ပါတယ်။ နမူနာ လိုင်းနံပါတ် (၉) မှာ `listenTo()` Function ကို သုံးပြီး Model မှာ တစ်ခုခုအပြောင်းအလဲဖြစ်တိုင်း `render()` Function အလုပ်လုပ်စေဖို့ သတ်မှတ်ထားပါတယ်။ ဒီနည်းနဲ့ Model တန်ဖိုးပြောင်းတိုင်း View ရဲ့ဖော်ပြပုံလည်း အလိုအလျောက် လိုက်ပြောင်းနေအောင် သတ်မှတ်ထားလို့ ရတဲ့သဘော ဖြစ်ပါတယ်။

`listenTo()` Function အတွက် Parameter (၃) ခုပေးရပါတယ်။ ပထမတစ်ခုက ပြုပြင်ပြောင်းလဲမှုတွေ ရှိမရှိ စောင့်ကြည့်လိုတဲ့ Model (သို့မဟုတ်) Collection ဖြစ်ပါတယ်။ နမူနာမှာ `this.model` လို့ပြောထားတဲ့အတွက် လက်ရှိ View နဲ့ တွဲဖက်ထားတဲ့ Model ကို ညွှန်းထားခြင်း ဖြစ်ပါတယ်။ အခြား Model (သို့မဟုတ်) Collection တစ်ခုကိုလည်း ညွှန်းထားနိုင်ပါတယ်။ ဒုတိယ Parameter အနေနဲ့ Event ကိုပေးရပါတယ်။ နမူနာမှာ change ကို ပေးထားပါတယ်။ အခြားအသုံးပြုလို့ရတဲ့ Event စာရင်းကို ခဏနေတော့ ဆက်လက်ဖော်ပြပေးပါမယ်။ နောက်ဆုံး Parameter ကတော့ Event ဖြစ်ပေါ်တဲ့အခါ အလုပ်လုပ်စေလိုတဲ့ Function ဖြစ်ပါတယ်။ ဥပမာ – `Issues` Collection ထဲကို Model တစ်ခုထည့်သွင်းလိုက်တိုင်း အလိုအလျောက် အလုပ်လုပ်သွားစေလိုတဲ့ Function ရှိရင် အခု လို သတ်မှတ်ထားနိုင်ပါတယ်။

JavaScript

```
this.listenTo(Issues, 'add', this.doSomething());
```

နမူနာရဲ့ `render()` Function အတွင်းမှာ လက်ရှိ View Element ရဲ့ HTML Content အဖြစ် Model subject ကို ဖော်ပြပေးဖို့သတ်မှတ်ထားပါတယ်။ `$el` ဆိုတာ လက်ရှိ View Element ကို Select လုပ်လိုက်ခြင်းဖြစ်ပါတယ်။ သူနဲ့ တွဲဖက်ပြီး jQuery DOM Function တွေကိုသုံးနိုင်ပါတယ်။ လိုင်းနံပါတ် (၁၃) မှာသုံးထားတဲ့ `html()` Function ဟာ Backbone Function မဟုတ်ပါဘူး jQuery Function တစ်ခုဖြစ်ပါတယ်။ တစ်နည်းအားဖြင့် Backbone View ထဲမှာ jQuery Function တွေကို လိုအပ်သလို တွဲဖက်အသုံးပြုနိုင်ခြင်းဖြစ်ပါတယ်။ jQuery အကြောင်းကို **Professional Web Developer ရဲ့ အခန်း (၆)** မှာ ဖော်ပြခဲ့ပြီး ဖြစ်ပါတယ်။

နမူနာကို လေ့လာကြည့်ရင် Backbone View ကပေးတဲ့ အဓိကအကျဆုံး အားသာချက်က၊ Model မှာ အပြောင်းအလဲ တစ်ခုရှိတာနဲ့ View မှာ အလိုအလျောက် လိုက်လံပြောင်းလဲအောင် စီမံထားနိုင်ခြင်းဖြစ်တယ်ဆိုတာကို တွေ့ရနိုင်ပါတယ်။ နောက်ထပ် အားသာချက်တစ်ခုကတော့ View တစ်ခုနဲ့သက်ဆိုင်တဲ့ Event တွေကို တစ်နေရာထဲမှာ စုစည်းကြေငြာ ထားနိုင်ခြင်းပဲဖြစ်ပါတယ်။ Backbone View နဲ့အတူ တွဲဖက်အသုံးပြုနိုင်တဲ့ Function တွေကို ဖော်ပြလိုက်ပါတယ်။

el – အထက်ကနမူနာမှာ `tagName` ကိုအသုံးပြုပြီး View အတွက် အသုံးပြုရမယ့် HTML Element Name ကို သတ်မှတ် ထားပါတယ်။ အဲ့ဒီလိုမသတ်မှတ်ပဲ ရှိနေပြီးသား HTML Element တစ်ခုကို သုံးစေလိုရင် `el` Property ကို အသုံးပြုနိုင် ပါတယ်။ ဥပမာ –

JavaScript

```
var IssueView = Backbone.View.extend({
     el: "#list"
});
```

ID ကို `#list` လို့သတ်မှတ်ထားတဲ့ HTML Element ကို View Element အနေနဲ့ အသုံးချသွားမှာဖြစ်ပါတယ်။

$el – `tagName` (သို့မဟုတ်) `el` နဲ့ ကြေငြာသတ်မှတ်ထားတဲ့ View Element ကို jQuery Selector သုံးပြီး Select လုပ်ထားပေးခြင်းဖြစ်ပါတယ်။

attributes – View Element အတွက် သတ်မှတ်လိုတဲ့ HTML Attribute တွေကို ဒီ Property နဲ့ သတ်မှတ် ထားနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
var IssueView = Backbone.View.extend({
     el: "#list",
     attributes: {class: 'issue', title: 'click to edit'}
});
```

`$(selector)` – View Element အတွင်းက အခြား Element တွေကို Select လုပ်လိုတဲ့အခါ သုံးပါတယ်။

JavaScript

```
var IssueView = Backbone.View.extend({
    render: function() {
        this.$('.subject').html(this.model.get('subject"));
    }
});
```

View Element အတွင်းက Class `.subject` သတ်မှတ်ထားတဲ့ Element ကို Select လုပ်ထားခြင်းဖြစ်ပါတယ်။

`events` – View Element ပေါ်မှာ ပြုလုပ်တဲ့ User Event တွေအတွက် သက်ဆိုင်ရာ Function တွဲဖက်ကြေငြာဖို့ အသုံးပြုပါတယ်။ `"event element": "function"` ဆိုတဲ့ Format နဲ့ ကြေငြာရပါတယ်။ `event` နေရာမှာ jQuery Event အားလုံး အသုံးပြုနိုင်ပါတယ်။ `element` နေရာမှာ jQuery Selector ပေးရပါတယ်။ Function ကတော့ မိမိနှစ်သက်သလို ကြေငြာသတ်မှတ်ထားနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
var IssueView = Backbone.View.extend({
     el: "#list",
     events: {
            "click .new": addNew,
            "focus input": getUpdate,
            "change #form select": updateStatus
     },
     addNew: function() { … },
     getUpdate: function() { … },
     updateStatus: function() { … }
});
```

`render()` – Default အနေနဲ့ ဘာအလုပ်မှလုပ်မပေးပါဘူး။ ပါဝင်စေလိုတဲ့လုပ်ဆောင်ချက်တွေကို ကိုယ်တိုင် သတ်မှတ်ပေးရပါတယ်။

`remove()` – View ကို ပယ်ဖျက်စေလိုတဲ့အခါသုံးပါတယ်။

View နဲ့အတူ တွဲဖက်အသုံးပြုဖို့အတွက် အရေးပါတဲ့ Function တစ်ခုကတော့ `template()` Function ပဲဖြစ်ပါ တယ်။ Backbone ရဲ့ Function မဟုတ်ပါဘူး။ Underscore ရဲ့ Function ဖြစ်ပါတယ်။ Format လုပ်ထားတဲ့ HTML String အတွင်းမှာ သက်ဆိုင်ရာ တန်ဖိုးတွေကို အစားထိုးပေးနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
var template = _.template("<li><%= subject %> (<%= status %>)</li>");
template({ subject: "Dropdown menu not work", status: 2 });
```

```
// => <li>Dropdown menu not work (2)</li>
```

အရင်ဆုံး `_.template` (Underscore Dot Template) ကို အသုံးပြုပြီး HTML Template Function ကို ကြေငြာရပါတယ်။ HTML Template အတွင်းမှာ Output အနေနဲ့ ဖော်ပြစေလိုတဲ့ တန်ဖိုးတွေကို `<%= %>` Template Tag နဲ့ ထည့်သွင်းသတ်မှတ်နိုင်ပါတယ်။ JavaScript Code တွေ Template ထဲမှာ ထည့်သွင်းလိုရင်တော့ `<% %>` ကို သုံးနိုင် ပါတယ်။ ရရှိလာတဲ့ Template Function ကို Run တဲ့အချိန်မှာ Data ကို ပေးခြင်းအားဖြင့် HTML Template နဲ့ Data တွဲဖက်ထားတဲ့ ရလဒ်ကို ရရှိမှာဖြစ်ပါတယ်။

9.5 – Backbone Events

Backbone Model နဲ့ Collection တွေမှာ တန်ဖိုးအသစ်ထည့်သွင်းလိုက်ခြင်း၊ ပြုပြင်လိုက်ခြင်း၊ ပယ်ဖျက်လိုက်ခြင်း စတဲ့ လုပ်ဆောင်ချက်တစ်ခုပြုလုပ်တိုင်းမှာ အလိုအလျောက်ဆောင်ရွက်သွားစေလိုတဲ့ Function တွေရှိရင် Event အနေနဲ့ သတ်မှတ်ထားနိုင်ပါတယ်။

ဥပမာ –

JavaScript

```
var Issue = Backbone.Model.extend({
    defaults: {
        subject: "",
        status: 2
    },
    initialize: function() {
        this.on("change", function() { console.log("I've changed") });
    }
});

var issueOne = new Issue();
issueOne.set({status: "Dropdown menu not working"});

// => I've changed
```

Model Constructor ရဲ့ initialize မှာ `on()` Function ကို သုံးပြီး Model မှာ အပြောင်းအလဲတစ်ခုခုဖြစ်တိုင်း အလိုအလျောက် လုပ်သွားစေလိုတဲ့ Function ကို သတ်မှတ်ထားပါတယ်။ change ဆိုတာ အပြောင်းအလဲတစ်ခုခုရှိရင် ဆိုတဲ့ အဓိပ္ပါယ်ပဲဖြစ်ပါတယ်။ `change` အစား သုံးလို့ရတဲ့ Event တွေကိုဖော်ပြလိုက်ပါတယ်။

- **`add`** – Model ကို Collection ထဲကိုထည့်လိုက်တဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`remove`** – Model ကို Collection ထဲကနေထုတ်လိုက်တဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`reset`** – Collection ထဲက Model တွေကို စာရင်းအသစ်နဲ့ Reset လုပ်တဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`sort`** – Collection ကို Sorting စီလိုက်တဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`change`** – Model တန်ဖိုးတွေပြောင်းသွားတဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`change:attribute`** – ဥပမာ `change:status` – Model ထဲက `status` တန်ဖိုးပြောင်းသွားတဲ့

အခါဖြစ်ပေါ်တဲ့ Event

- **`destroy`** – Model ကို ပယ်ဖျက်လိုက်တဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`request`** – Server ကို ဆက်သွယ်တဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`sync`** – Server နဲ့ ဆက်သွယ်ဆောင်ရွက်တဲ့လုပ်ငန်း အောင်မြင်တဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`error`** – Server နဲ့ ဆက်သွယ်မှုမအောင်မြင်တဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`invalid`** – Model Validation မအောင်မြင်တဲ့အခါ ဖြစ်ပေါ်တဲ့ Event
- **`all`** – အထက် Event တစ်ခုခု ဖြစ်ပေါ်တိုင်းမှာ ဖြစ်ပေါ်တဲ့ Event

ဘယ် Event မှာ ဘာအလုပ်လုပ်ရမလဲဆိုတာကို အထက်ကနမူနာမှာ ဖော်ပြထားသလို `on()` Function နဲ့ ကြေငြာ သတ်မှတ်ထားနိုင်ပါတယ်။ Event အမှန်တစ်ကယ်ဖြစ်ပေါ်ခြင်း မရှိပေမယ့် လိုအပ်လို့ Event Function ကို Run စေ ချင်ရင် `trigger()` Function ကို သုံးနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
IssueOne.trigger("change");
```

Event Function တွေထဲမှာ အဓိကကျတဲ့ Function ကတော့ `listenTo()` ဖြစ်ပါတယ်။ Model (သို့) Collection (သို့) View တစ်ခုမှာဖြစ်ပေါ်တဲ့ Event တွေကို အခြား တစ်နေရာကနေ စောင့်ကြည့်ပေးခြင်းဖြစ်ပါတယ်။ View အတွက် နမူနာဖော်ပြခဲ့စဉ်က `listenTo()` ကိုသုံးပြီး Model မှာ အပြောင်းအလဲတစ်ခုဖြစ်တိုင်း View ရဲ့ `render()` Function ကို နောက်တစ်ကြိမ် ပြန် Run ပေးတဲ့ လုပ်ဆောင်ချက်ကို ထည့်သွင်းဖော်ပြခဲ့ပါတယ်။ နမူနာမှာ `change` Event ကို နမူနာအနေနဲ့ပေးထားပေမယ့် `add`, `remove`, `reset`, `destroy` စတဲ့ အခြား Event တွေကိုလည်း အသုံးပြုနိုင်ပါတယ်။

အခုဆိုရင် Backbone Model, Collection, View နဲ့ Event တို့အကြောင်းကို ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ ထည့်သွင်းမဖော် ပြရသေးတဲ့ လုပ်ဆောင်ချက်ကတော့ Router ပဲဖြစ်ပါတယ်။ backbonejs.org မှာ ဆက်လက်လေ့လာနိုင်ပါတယ်။

9.6 – Example Backbone App

အခန်း (၁၅) မှာ Backbone ကို အသုံးပြုပြီး Issue Tracking System တစ်ခုအတွက် Web Client တစ်ခု ရေးသားပုံ ကို အပြည့်အစုံဖော်ပြသွားမှာပါ။ ဒီနေရာမှာ Issue တွေ မှတ်တမ်းတင်လို့ရပြီး၊ မှတ်တမ်းတင်ထားတဲ့ Issue စာရင်းကို ပြန်လည်ကြည့်ရှုနိုင်တဲ့ App လေးတစ်ခုကို နမူနာအနေနဲ့ ဖန်တီးကြည့်ပါမယ်။

ပထမဦးဆုံးအနေနဲ့ Backbone App Boilerplate တစ်ခုကို Generate မလုပ်ရသေးရင် လုပ်ဖို့လိုပါမယ်။

```
$ mkdir bbApp && cd bbApp
$ yo backbone
```

Yeoman က Bootstrap, RequireJS စတဲ့ Library တွေကိုရော ထည့်သွင်း Install လုပ်လိုသလားမေးပါလိမ့်မယ်။ ကျွန်တော် တို့နမူနာမှာ CSS တွေကို ကိုယ်တိုင်ရေးနေစရာမလိုအောင် Bootstrap CSS Framework ကို အသုံးပြုလိုတဲ့အတွက် ထည့်သွင်း Install လုပ်ပေးသင့်ပါတယ်။ အကယ်၍ Backbone နဲ့အတူ တစ်ခါတည်း Install မလုပ်ဖြစ် ရင်လည်း အခုလို သီးခြား Install လုပ်နိုင်ပါတယ်။

```
$ bower install bootstrap
```

Backbone, jQuery, Underscore နဲ့ Bootstrap အားလုံးပါဝင်တဲ့ Boilerplate ရုပ်ဆိုရင် နမူနာ App ကို စတင်ရေး သားနိုင်ပါပြီ။ Code နမူနာကို တစ်လုံးမကျန်အကုန် မဖော်ပြတော့ပါဘူး။ အပိုင်းလိုက် ခွဲခြားဖော်ပြသွားပါမယ်။ ပထမဦး ဆုံးအနေနဲ့ Issue List ဖော်ပြဖို့အတွက် Table အလွတ်တစ်ခုကို HTML နဲ့ အခုလို တည်ဆောက်ပေးရပါမယ်။

HTML

```
<table class="table" id="list">
    <tr>
        <th>Subjecct</th>
        <th>Priority</th>
        <th>Severity</th>
        <th>Assigned To</th>
        <th>Status</th>
        <th>#</th>
    </tr>
</table>

<button class="btn btn-primary" data-toggle="modal" data-target="#new">
    <span class="glyphicon glyphicon-plus"></span> New Issue
</button>
```

`#list` လို့ ID သတ်မှတ်ပေးထားတဲ့ `<table>` Element တစ်ခုဖြစ်ပါတယ်။ ခေါင်းစီး Row တစ်ခုပဲပါပြီး Content တော့မပါသေးပါဘူး။ သူ့အောက်မှာ Issue အသစ်တွေထပ်ထည့်နိုင်တဲ့ New Issue Form ကိုဖွင့်ဖို့ `<button>` Element တစ်ခုလည်း ထည့်ထားပါသေးတယ်။ Bootstrap ရဲ့ Modal Dialog လုပ်ဆောင်ချက်ကို သုံးပြီး New Issue Form ကို Dialog Box နဲ့ ဖော်ပြမှာပါ။ ဒါကြောင့် Dialog Box ရဲ့ ID အဖြစ် သတ်မှတ်မယ့် `#new` ကို `data-target` Attribute နဲ့ ညွှန်းထားပေးပါတယ်။ Bootstrap CSS Framework အကြောင်းကို **အခန်း (၁၄)** မှာ ဆက် လက်ဖော်ပြပါမယ်။

ဆက်လက်ပြီး ID ကို `#new` လို့ သတ်မှတ်ထားတဲ့ New Issue Form တစ်ခုကို အခုလိုထည့်သွင်းပေးရပါမယ်။

HTML

```
<div class="modal fade" id="new">
  <div class="modal-dialog">
    <div class="modal-content">

```

```
 5.       <div class="modal-header">
 6.         <button type="button" class="close" data-dismiss="modal">
 7.             <span aria-hidden="true">&times;</span>
 8.         </button>
 9.         <h4 class="modal-title">New Issue</h4>
10.       </div>
11.
12.       <div class="modal-body">
13.
14.         <div class="form-group">
15.           <input class="form-control" type="text"
                     id="subject" placeholder="Subject">
16.         </div>
17.
18.         <div class="form-group">
19.           <select class="form-control" id="priority">
20.             <option value="-1">-- Priority --</option>
21.             <option value="0">Normal</option>
22.             <option value="1">Important</option>
23.             <option value="2">Critical</option>
24.           </select>
25.         </div>
26.
27.         <div class="form-group">
28.           <select class="form-control" id="severity">
29.             <option value="-1">-- Severity --</option>
30.             <option value="0">Minor</option>
31.             <option value="1">Major</option>
32.             <option value="2">Crash</option>
33.           </select>
34.         </div>
35.
36.         <div class="form-group">
37.           <input type="text" class="form-control"
                     id="assigned-to" placeholder="Assigned To">
38.         </div>
39.
40.         <div class="form-group">
41.           <select class="form-control" id="status">
42.             <option value="-1">-- Status --</option>
43.             <option value="0">Assigned</option>
44.             <option value="1">WIP</option>
45.             <option value="2">Done</option>
46.             <option value="3">Close</option>
47.           </select>
48.         </div>
49.
50.       </div>
51.
52.       <div class="modal-footer">
53.         <button class="btn btn-primary" id="submit">Submit Issue</button>
54.       </div>
55.
56.     </div>
57.   </div>
58. </div>
```

Bootstrap Modal Dialog တစ်ခုရရှိဖို့အတွက် `.modal`, `.modal-dialog`, `.modal-content`,

`.modal -header`, `.modal-body`, `.modal-footer` စတဲ့ Element တွေ လိုအပ်တဲ့အတွက် HTML Structure က အဆင့်ဆင့်ဖြစ်နေတာပါ။ လက်တွေ့လို အပ်တာကတော့ `#subject`, `#priority`, `#severity`, `#assigned-to` နဲ့ `#status` ဆိုတဲ့ Input Element တွေသာဖြစ်ပါတယ်။ Input Element တွေရဲ့ အောက်ဆုံးမှာ `<button>` Element တစ်ခု ကို `#submit` ဆိုတဲ့ ID နဲ့ ထည့်သွင်းထားပါသေးတယ်။ Bootstrap က ဒီ Structure တစ်ခုလုံးကို ဖျောက်ထားပြီး `#new` Button ကို နှိပ်လိုက်တော့မှ Modal Dialog Box တစ်ခုအနေနဲ့ ဖော်ပြပေးသွားမှာဖြစ်ပါတယ်။ `#new` Button ရဲ့ `data-target` Attribute မှာ ဒီ Structure ရဲ့ ID ဖြစ်တဲ့ `#list` ကို ညွှန်းထားတဲ့အတွက် ဖြစ်ပါတယ်။ အခုဆိုရင် Issue List ဖော်ပြမယ့် Table အလွတ်တစ်ခုနဲ့ Issue အသစ် ထည့်သွင်းလို့ရတဲ့ Form အတွက် လိုအပ်တဲ့ HTML တွေ ရရှိသွားပါပြီ။ ဆက်လက်ပြီး Issue တစ်ခုစီကို ဖော်ပြရာမှာ အသုံးပြုဖို့အတွက် Template Structure ကို သတ်မှတ် ပေးရပါမယ်။

Template

```
<script type="text/x-template" id="row">
    <td><%= subject %></td>
    <td><%= config.priority[priority] %></td>
    <td><%= config.severity[severity] %></td>
    <td><%= assignedTo %></td>
    <td>
        <select class='form-control input-sm change-status'>
            <% _.each(config.status, function(v, i) { %>
                <% if(i == status) { %>
                    <option value='<%= i %>' selected><%= v %></option>
                <% } else { %>
                    <option value='<%= i %>'><%= v %></option>
                <% } %>
            <% }) %>
        </select>
    </td>
    <td>
        <a href='#' class='del'>
            <span class='glyphicon glyphicon-remove'></span>
        </a>
    </td>
</script>
```

ဒီ Structure ကို `type` မှာ `text/x-template` လို့သတ်မှတ်ထားတဲ့ `<script>` Element တစ်ခုအနေနဲ့ ထည့်သွင်းထားပါမယ်။ `<script>` Element အတွက် `type` အမှန်က `text/javascript` ဖြစ်ပါတယ်။ အခု `type` မှာ `text/x-template` လို့ပြောထားတဲ့အတွက် Browser ကဒီ `type` ကို နားမလည်လို့ JavaScript အနေနဲ့ အလုပ်လုပ်ပေးမှာမဟုတ်သလို HTML အနေနဲ့လည်း ဖော်ပြပေးမှာမဟုတ်ပါဘူး။

JavaScript ကနေ လိုအပ်သလို ယူသုံးဖို့ ရည်ရွယ်ထည့်သွင်းထားခြင်းဖြစ်ပြီး User တွေကို မပြစေလိုတဲ့ Template Structure တွေကို ဒီနည်းနဲ့ ထည့်သွင်းရေးသားကြလေ့ရှိပါတယ်။ ဒီ Template နဲ့ တွဲဖက်အသုံးပြုရမယ့် Data တို့ကို ပေးလိုက်ရင် Underscore Template Function က လက်တွေ့သုံးလို့ရတဲ့ HTML Output ကို ပြန်ပေးမှာဖြစ်ပါတယ်။

Template Structure ကိုလေ့လာကြည့်ရင် `<td>` Element တွေနဲ့ `subject`, `priority`, `severity`,

`assignedTo`, `status` စတဲ့ Variable တွေကို သူနေရာနဲ့သူ အစားဖိုးအသုံးပြုနိုင်ဖို့ သတ်မှတ်ထားတာကို တွေ့နိုင်ပါတယ်။ `priority` တို့ `severity` တို့ကို Model မှာ တန်ဖိုးအနေနဲ့သိမ်းတဲ့အခါ 0, 1, 2 စတဲ့ နံပါတ် တန်ဖိုးတွေနဲ့ သိမ်းမှာပါ။ ဒါပေမယ့် ပြန်လည်ဖော်ပြတဲ့အခါ Priority: 1, Severity: 2 စသဖြင့် နံပါတ်တွေနဲ့ ပြရင်အဆင်မပြေပါဘူး။ ဒါကြောင့် သက်ဆိုင်ရာ 0, 1, 2 နံပါတ်တွေအစား ကြိုတင်သတ်မှတ်ထားတဲ့ တန်ဖိုးတွေနဲ့ အစားထိုးဖော်ပြနိုင်ဖို့အတွက် `config` Object ကို တစ်ဆင့်ခံအသုံးပြုထားခြင်းဖြစ်ပါတယ်။ ဥပမာ – 0 ဆိုရင် Minor, 1 ဆိုရင် Major စသဖြင့် ကြိုတင်ကြေငြာထားတဲ့ တန်ဖိုးနဲ့ ဖော်ပြနိုင်ဖို့အတွက်ပါ။ config Object ကို Global Variable တစ်ခုအနေနဲ့ အခုလို ကြေငြာထားရမှာ ဖြစ်ပါတယ်။

JavaScript

```
var config = {
    priority: ['Normal', 'Important', 'Urgent'],
    severity: ['Minor', 'Major', 'Crash'],
    status: ['Assigned', 'Doing', 'Done', 'Close']
};
```

Template Structure ရဲ့ လိုင်းနံပါတ် (၇) ကိုလေ့လာကြည့်ရင် `config.status` ကို Loop လုပ်ပြီး `<select>` Element တစ်ခုရဲ့ `<option>` တွေအဖြစ် ထည့်သွင်းထားတာကိုတွေ့ရမှာဖြစ်ပါတယ်။ ဒီတော့မှ User က Issue တစ်ခု ရဲ့ Status ကို ပြောင်းချင်ရင် Select Box ကနေတစ်ခါတည်း ရွေးပြီးပြောင်းနိုင်စေဖို့ ရည်ရွယ်ထည့်သွင်းထားခြင်း ဖြစ်ပါ တယ်။ `status` Variable ရဲ့ တန်ဖိုး နဲ့ `<option>` ရဲ့တန်ဖိုးတူတဲ့ အခါ `<option>` မှာ selected Attribute ထည့်ပေးဖို့ကိုလည်း လိုင်းနံပါတ် (၉) မှာ Condition စစ်ပြီး သတ်မှတ်ထားပါသေးတယ်။ Underscore Template တွေမှာ Variable တန်ဖိုးတွေ ဖော်ပြဖို့ `<%= %>` Template Element ကို အသုံးပြုပြီး JavaScript Code တွေ ရောစပ် ရေးသားဖို့အတွက် `<% %>` Template Element ကို သုံးကြောင်း အထက်မှာလည်း ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ နောက်ဆုံး `<td>` အတွင်းမှာ `.del` Class ပေးထားတဲ့ Element တစ်ခုပါဝင်တာ ကိုလည်း သတိပြုပါ။ Issue ကို ဖျက်ချင် တဲ့အခါ အဲ့ဒီ Element ကိုနှိပ်ပြီးဖျက်စေနိုင်ဖို့ ရည်ရွယ်ထည့်သွင်းထားခြင်းဖြစ်ပါတယ်။

အခုဆိုရင် App ရဲ့ အခြေခံ HTML Structure တော့ပြည့်စုံသွားပါပြီ။ Issue List ဖော်ပြဖို့ Table တစ်ခုရပါပြီ။ Issue အသစ်ထည့်သွင်းဖို့ Form တစ်ခုရပါပြီ။ Issue တွေကို ဖော်ပြရာမှာ အသုံးပြုရမယ့် Template တစ်ခုလည်း ရရှိသွားပြီ ဖြစ်ပါတယ်။ ဆက်လက်ပြီး JavaScript Code တွေ ရေးသားပါမယ်။ ပထမဆုံးအနေနဲ့ Issue Model Constructor တစ်ခုကို အခုလို တည်ဆောက်ရပါမယ်။

JavaScript

```
var Issue = Backbone.Model.extend({
    defaults: {
        subject: 'Issue summary',
        priority: 1,
        severity: 1,
        assignedTo: 'none',
        status: 1
    },

    changeStatus: function(value) {
        this.save({status: value});
```

```
12.     }
13. });
```

Model ရဲ့ Default Data အနေနဲ့ `subject`, `priority`, `severity`, `assignedTo` နဲ့ `status` တို့ ပါဝင်ပြီး `status` တန်ဖိုးကို ပြောင်းပေးနိုင်တဲ့ `changeStatus()` Function လည်းပါဝင်ပါတယ်။ ဆက်လက်ပြီး Collection Constructor တစ်ခုကို အခုလို တည်ဆောက်ပေးရပါမယ်။

JavaScript

```
1. var IssueList = Backbone.Collection.extend({
2.     url: '#',
3.     model: Issue
4. });
5.
6. var Issues = new IssueList();
```

Collection Constructor မှာ `url` အနေနဲ့ ဘာမှမသတ်မှတ်လိုတဲ့အတွက် # Sign ကိုပဲပေးထားပါတယ်။ Server နဲ့ ဆက်သွယ်တဲ့အလုပ်တွေ မလုပ်လိုတဲ့အတွက် URL ကို မသတ်မှတ်တော့ခြင်းဖြစ်ပါတယ်။ model အတွက် စောစောက တည်ဆောက်ခဲ့တဲ့ Issue ကို သတ်မှတ်ပေးထားပါတယ်။ Constructor တည်ဆောက်ပြီး Issues အမည်နဲ့ Collection Object တစ်ခုကိုပါ လိုင်းနံပါတ် (၆) မှာ တစ်လက်စတည်း တည်ဆောက်ထားတာကို တွေ့ရမှာဖြစ်ပါတယ်။

ဆက်လက်ပြီး Issue List Table ထဲမှာ ဖော်ပြရမယ့် Issue တစ်ခုစီအတွက် View Constructor ကို ရေးသားပါမယ်။

JavaScript

```
1. var IssueView = Backbone.View.extend({
2.     tagName: 'tr',
3.
4.     events: {
5.         'change .change-status': 'changeStatus',
6.         'click .del': 'delSelf'
7.     },
8.
9.     template: _.template( $('#row').html() ),
10.
11.     initialize: function() {
12.         this.listenTo(this.model, 'change', this.render),
13.         this.listenTo(this.model, 'destroy', this.remove)
14.     },
15.
16.     render: function() {
17.         var modelData = this.model.toJSON();
18.         modelData.config = config;
19.
20.         this.$el.html( this.template(modelData) );
21.         return this;
22.     },
```

```
23.
24.     changeStatus: function() {
25.         this.model.changeStatus( this.$('.change-status').val() );
26.     },
27.
28.     delSelf: function() {
29.         this.model.destroy();
30.     }
31. });
```

Issue တွေကို <tr> Element နဲ့ဖော်ပြချင်တဲ့အတွက် tagName အတွက် tr ကို သတ်မှတ်ပေးထားပါတယ်။ Event အနေနဲ့ Status Select Box မှာတန်ဖိုးပြောင်းတဲ့အခါ changeStatus() Function ကိုအလုပ်လုပ်ပေးဖို့ သတ်မှတ် ပေးထားပါတယ်။ လိုင်းနံပါတ် (၂၅) ကိုလေ့လာကြည့်ရင် ဒီ View က Model ရဲ့ changeStatus() Function ကိုပဲ တစ်ဆင့်ခေါ်ယူအသုံးပြုထား တယ်ဆိုတာကို တွေ့ရနိုင်ပါတယ်။ View ထဲမှာပဲတစ်ခါတည်းရေးလဲ ရပါတယ်။ ဒါပေမယ့် MVC ရေးထုံးအရ View ဟာ User Interface ဖော်ပြမှုပိုင်းကိုသာတာဝန်ယူသင့်ပြီး၊ Logic ပိုင်း လုပ်ဆောင်ချက် တွေကို Model ကသာတာဝန် ယူဆောင်ရွက်ရပါတယ်။

View Object တစ်ခုတည်ဆောက်တဲ့အခါ တွဲဖက်သုံးရမယ့် Model Object ကို သတ်မှတ်ပေးရပါတယ်။ ဥပမာ –

JavaScript

```
var issueOne = new Issue();
var view = new IssueView({model: issueOne});
```

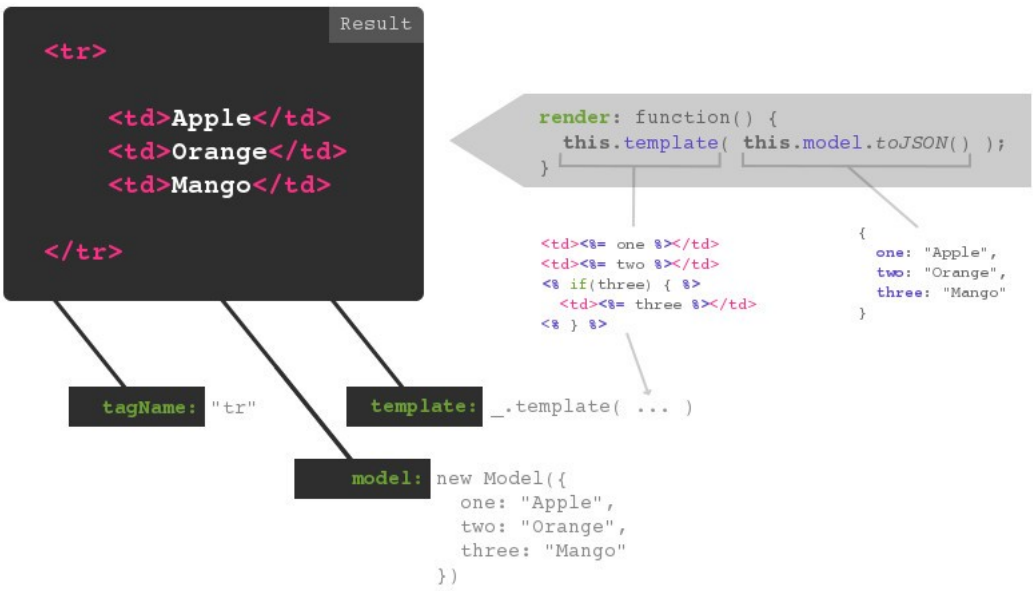
this.model ဆိုတာ အဲ့ဒီလို View နဲ့တွဲသုံးဖို့ သတ်မှတ်ထားတဲ့ Model Object ကို ပြောတာပါ။

Event အနေနဲ့ .del ကိုနှိပ်ရင် delSelf() Function ကို အလုပ်လုပ်ပေးဖို့လည်း သတ်မှတ်ထားပါတယ်။ လိုင်းနံ ပါတ် (၂၉) ကိုလေ့လာကြည့်ပါ။ delSelf() Function က this.model ကို ပယ်ဖျက်လိုက်ခြင်းဖြစ်ပါတယ်။

ဆက်လက်ပြီး လိုင်းနံပါတ် (၉) မှာ template Property အတွက် ကြိုတင်ရေးသားထားခဲ့တဲ့ Template ကို Underscore Template Function အဖြစ်သတ်မှတ်ပေးထားပါတယ်။ လိုင်းနံပါတ် (၁၁) က initialize Function ထဲမှာတော့ listenTo() ကိုသုံးပြီး Model မှာ change တစ်ခုခုရှိတဲ့ အခါ render() Function ကို Run ပေးဖို့ သတ်မှတ်ထားသလို၊ Model ကို destroy လုပ်လိုက်တဲ့အခါ remove() Function ကို Run ပေးဖို့သတ်မှတ် ထားပါတယ်။

ဒီနေရာမှာ remove() ဆိုတာ jQuery remove() Function ကိုပြောတာပါ။ View Object တစ်ခုတည်ဆောက် လိုက်တာနဲ့ အဲ့ဒီ Object နဲ့အတူ jQuery Function တွေ တစ်ခါတည်းတွဲဖက်အသုံးပြုနိုင်အောင် Backbone က စီစဉ်ပေးထားပါတယ်။ ဒါကြောင့် remove() Function ထပ်ရေးစရာမလိုပါဘူး။ jQuery remove() Function က လက်ရှိ View Element ကို DOM Structure ကနေ ဖယ်ထုတ်ပေးသွားမှာဖြစ်ပါတယ်။ change အတွက် သတ်မှတ်ထားတဲ့ render() Function ကိုတော့ ကိုယ်တိုင် ရေးသားရပါတယ်။ လိုင်း

နံပါတ် (၁၆) မှာ `render()` Function ကို သတ်မှတ်ထားပါတယ်။ `render()` နဲ့ Template ပေါင်းစပ် အလုပ်လုပ်ပုံကို **ပုံ (၉.၃)** မှာ ဖော်ပြ ထားပါတယ်။ လေ့လာကြည့်ပါ။



ပုံ (၉.၃) - Backbone View Structure

`template` Property ထဲမှာသိမ်းထားတဲ့ Underscore Template ကို `this.template` နဲ့ပြန်လည်ရယူအသုံးပြု ပါတယ်။ Template နဲ့အတူအသုံးပြုရမယ့် Data ကို `this.model.toJSON()` သုံးပြီးပေးလိုက်တဲ့အခါ ကျွန်တော် တို့ လိုချင်တဲ့ HTML Structure ရလဒ်ကို ရရှိမှာပဲဖြစ်ပါတယ်။

View Constructor Code နမူနာရဲ့ လိုင်းနံပါတ် (၁၈) ကို သွားလေ့လာကြည့်ရင် ထူးခြားချက်အနေနဲ့ ကြိုတင်ကြေငြာထားတဲ့ `config` ကို Model JSON Object နဲ့ တွဲထည့်ပေးလိုက်တာကို တွေ့ရမှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Template ထဲမှာ Model Data တွေသာမက `config` ကိုပါ အသုံးပြုလို့ ရရှိသွားခြင်း ဖြစ်ပါတယ်။ Template Code နမူနာမှာ ပြန်လေ့လာကြည့်ပါ။ `subject`, `priority`, `severity` စတဲ့ Model Data တွေအပြင် `config` ကိုလည်း အသုံးပြုထားတာကို တွေ့ရပါလိမ့်မယ်။ ဆက်လက်ပြီး App View Constructor ကို ဆက်လက်ရေးသားရပါမယ်။

JavaScript

```
1. var AppView = Backbone.View.extend({
2.     el: 'body',
3.
4.     events: {
5.         'click #submit': 'addIssue'
6.     },
7.
8.     initialize: function() {
```

```
 9.          this.listenTo(Issues, 'add', this.addRow);
10.      },
11.
12.      addIssue: function() {
13.          var newIssue = new Issue({
14.              subject: $('#subject').val(),
15.              priority: $('#priority').val(),
16.              severity: $('#severity').val(),
17.              assignedTo: $('#assigned-to').val(),
18.              status: $('#status').val()
19.          });
20.
21.          Issues.add(newIssue);
22.      },
23.
24.      addRow: function(issue) {
25.          var view = new IssueView({ model: issue });
26.          $('#list').append( view.render().el );
27.
28.          $('#subject,  #assigned-to').val('');
29.          $('#priority, #severity, #status').val(0);
30.          $('#new').modal('hide');
31.      }
32. });
```

ဒီ View အတွက်တော့ Element အသစ်မတည်ဆောက်တော့ပါဘူး။ App တစ်ခုလုံးနဲ့သက်ဆိုင်တဲ့ View ဖြစ်တဲ့အတွက် `<body>` Element ကိုပဲ အသုံးပြုပါမယ်။ ဒါကြောင့် `tagName` Property ကို မသုံးပဲ လိုင်းနံပါတ် (၂) မှာ el Property နဲ့ `body` ကို ညွှန်းထားပေးခြင်းဖြစ်ပါတယ်။ Event အနေနဲ့ Form Dialog Box ထဲက `#submit` Button ကိုနှိပ်လိုက်ရင် `addIssue()` Function ကို အလုပ်လုပ်ပေးဖို့ သတ်မှတ်ထားပါတယ်။ `#new` Button ကို နှိပ်လိုက် ရင် Form Dialog Box ပေါ်လာဖို့ကို ကျွန်တော်တို့ ထပ်ရေးစရာမလိုပါဘူး။ Bootstrap က လုပ်ပေးပါလိမ့်မယ်။

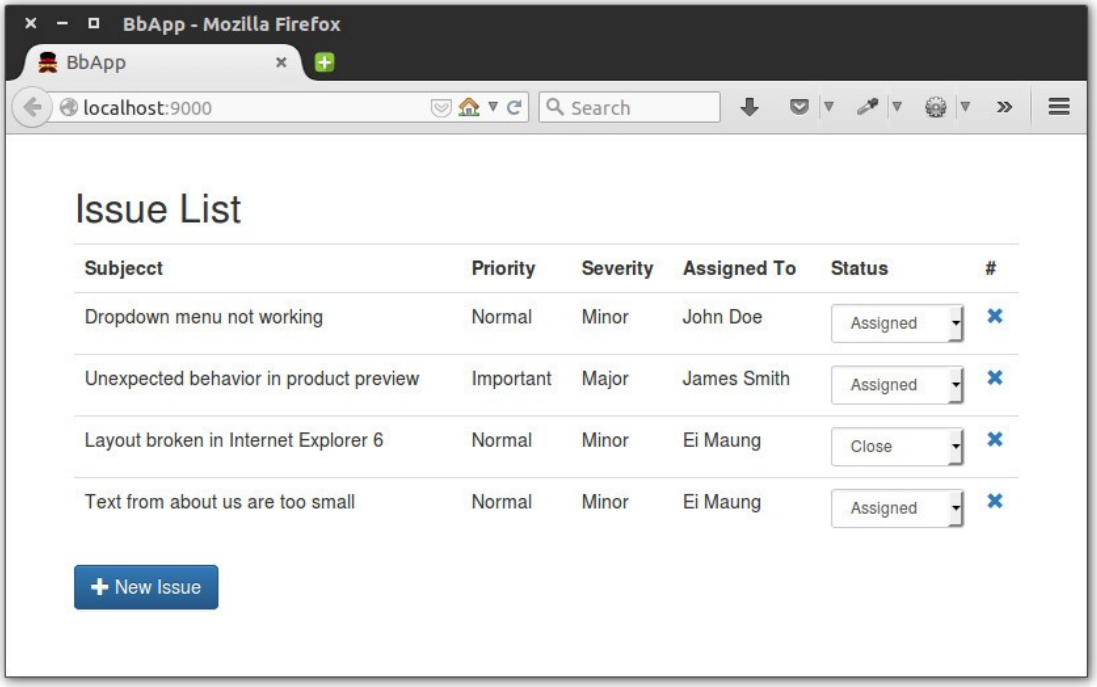
လိုင်းနံပါတ် (၁၂) မှာ `addIssue()` Function ကိုသတ်မှတ်ထားပါတယ်။ Form Input တွေမှာ ရိုက်ထည့်ရွေးချယ် ထားတဲ့ တန်ဖိုးတွေကို အသုံးပြုပြီး `Issue` Model တစ်ခုကို အရင်တည်ဆောက်ပါတယ်။ ပြီးတော့မှ `Issues` Collection ထဲကို `add` နဲ့ ထည့်သွင်းပေးလိုက်ခြင်းဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၉) ကို လေ့လာကြည့်ရင် `listenTo()` နဲ့ Issues Collection မှာ Model တစ်ခုတိုးလာတိုင်း `addRow()` Function ကို အလုပ်လုပ်ပေးဖို့ သတ်မှတ်ထား တာကို တွေ့ရမှာဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၂၄) က `addRow()` Function ကိုလေ့လာကြည့်ရင် `IssueView` Object တစ်ခု တည်ဆောက်ပြီး ရရှိလာတဲ့ရလဒ်ကို jQuery `append()` Function သုံးပြီး `#list` Table ထဲမှာ ထည့်သွင်းပေးထားတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။

ဒီတော့ `#new` ကိုနှိပ်လိုက်ရင် Form Dialog Box ပေါ်လာမယ်။ Input တွေထဲမှာ တန်ဖိုးတွေဖြည့် ပြီး `#submit` ကို နှိပ်လိုက်ရင် Issues Collection ထဲကို Model အသစ်တစ်ခု ရောက်ရှိသွားပါလိမ့်မယ်။ ဒီလို Model အသစ်တစ်ခု ရောက်တိုင်း `IssueView` တစ်ခုတည်ဆောက်ပြီး ရရှိလာတဲ့ရလဒ်ကို `#list` ထဲကို ထည့်သွင်းပေးသွားမှာပဲ ဖြစ်ပါ တယ်။

`IssueView` မှာ Status Select Box ထဲက Status တစ်ခုကို ရွေးလိုက်ရင် Model ထဲက Status တန်ဖိုး လိုက်ပြောင်းပေးတဲ့ လုပ်ဆောင်ချက်က ရေးသားထားပြီးဖြစ်သလို၊ `.del` ကို နှိပ်လိုက်ရင် သက်ဆိုင်ရာ Model ကို ပယ်ဖျက်ပေးတဲ့ လုပ်ဆောင်ချက်ကလည်း ကြိုတင်ရေးသားထားပြီးဖြစ်ပါတယ်။ ဒါကြောင့် အခုဆိုရင် Issue တွေထည့် သွင်းခြင်း၊ ထည့်သွင်းထားတဲ့ Issue တွေ ကို Table တစ်ခုနဲ့ဖော်ပြခြင်း၊ Issue Status ပြောင်းခြင်း၊ Issue တွေကို ပယ်ဖျက်ခြင်း စတဲ့လုပ်ငန်းတွေကို ဆောင်ရွက်ပေး နိုင်တဲ့ App တစ်ခု ရရှိသွားပြီဖြစ်ပါတယ်။ App စတင်အလုပ်လုပ် နိုင်ဖို့အတွက် `AppView` Object တစ်ခုတည်ဆောက်ပေးဖို့ ကျန်ပါသေးတယ်။ အခုလို ရေးသားပေးရမှာ ဖြစ်ပါတယ်။

```
1. $(function() {
2.     new AppView();
3. });
```

အားလုံးပြည့်စုံပြီဖြစ်လို့ `grunt serve` Command နဲ့ App ကို Run ကြည့်လို့ရပါပြီ။ **ပုံ (၉.၄)** မှာ ပြထားသလို ရလဒ်ကို ရရှိမှာဖြစ်ပါတယ်။ New Issue Button ကို နှိပ်ပြီး Issue တွေ စတင်ထည့်သွင်း စမ်းသပ်နိုင်ပါတယ်။



ပုံ (၉.၄) - Example Backbone App - Issue List

Code တွေကို ရေးသားစမ်းသပ်တဲ့အခါ HTML Document ထဲမှာ jQuery, Underscore, Backbone, Bootstrap စတဲ့ Library တွေအတွက် လိုအပ်တဲ့ JavaScript နဲ့ CSS ဖိုင်တွေ ချိတ်ဆက်ထားပေးဖို့ လိုတယ်ဆိုတာကို သတိပြုပါ။ နမူနာ Code အပြည့်အစုံကို အောက်ပါလိပ်စာမှာ ရယူနိုင်ပါတယ်။

https://github.com/eimg/backbone-example/archive/master.zip

အပိုင်း (၄) ရောက်တဲ့အခါ User Registration တွေ၊ Server နဲ့ချိတ်ဆက်ပြီး အချက်အလက်တွေသိမ်းဆည်းပေးတဲ့ လုပ်ဆောင်ချက်တွေ ပါဝင်တဲ့ Issue Management System တစ်ခုကို ဆက်လက်တည်ဆောက်သွားမှာပဲဖြစ်ပါတယ်။

Conclusion

Backbone ဟာ လက်ရှိဒီစာရေးသားနေချိန်မှာ ဒုတိယမြောက်လူသုံးအများဆုံး JavaScript MVC Framework တစ်ခု ဖြစ်ပါတယ်။ Backbone ထက် ပိုအသုံးများတာကတော့ AngularJS လို့ခေါ်တဲ့ Framework ပဲဖြစ်ပါတယ်။ Framework အရွယ်အစားအရ ပိုမိုကြီးမားသလို ပါဝင်တဲ့ လုပ်ဆောင်ချက်တွေလည်း ပိုမိုစုံလင်ပါတယ်။ ဒါပေမယ့် Angular ဟာ Opinionated Framework တစ်ခုဖြစ်ပါတယ်။ Opinionated Framework ဆိုတာ Code ရေးသားရာမှာ လိုက်နာသင့်တဲ့ရေးထုံးတွေ၊ လိုက်နာသင့်တဲ့ နည်းစနစ်တွေကို မဖြစ်မနေလိုက်နာဖို့ သတ်မှတ်ထားတဲ့ Framework အမျိုး အစားဖြစ်ပါတယ်။ Code ဖွဲ့စည်းပုံကောင်းပြီး ရေရှည် Maintenance ကောင်းစေဖို့ဆိုတဲ့ ရည်ရွယ်ချက်နဲ့ဖြစ်ပါတယ်။ Backbone ကတော့ Opinionated Framework မဟုတ်ပါဘူး။ MVC Pattern နဲ့ JavaScript Code တွေ ရေးလို့ရ အောင် ပေးထားတဲ့ ရိုးရှင်းတဲ့အခြေခံ Foundation တစ်ခုသာဖြစ်ပါတယ်။ ဒီ Framework ကိုသုံးပြီး လက်တွေ့ Code တွေ ရေးသားရာမှာ၊ ဘယ်လိုရေးထုံးတွေနဲ့ရေးပြီး ဘယ်လိုနည်းစနစ်တွေကို သုံးမလဲဆိုတာကို ရေးသားသူ Developer ကိုယ်တိုင် ဆုံးဖြတ်ရွေးချယ်ရေးသားနိုင်ပါတယ်။

Opinionated Framework တွေဟာ Code အရည်အသွေးကောင်းဖို့အတွက် အထောက်အကူဖြစ်စေပေမယ့် တစ်ခါ တစ်ရံမှာ သူ့ရဲ့ကန့်သတ်ချက်တွေကြောင့် Code ရေးသားရတာ မလွတ်လပ်ပဲ၊ အလုပ်မတွင်ဖြစ်တက်ပါတယ်။ စိတ်ကူး ရှိသလို ရေးသားဖန်တီးဖို့ထက် သတ်မှတ်ချက်နဲ့ ကိုက်ညီအောင် လိုက်ရေးပေးနေရတက်ပါတယ်။ ဒါကြောင့် ကျွန်တော့် တစ်ကိုယ်ရေအနေနဲ့ Opinionated Framework မဟုတ်တဲ့ လွပ်လပ်ရိုးရှင်းတဲ့ Framework တွေကိုသာ ပိုမိုနှစ်သက် ပါတယ်။ မိမိနှစ်သက်တဲ့ ရေးထုံးနည်းစနစ်ကို သုံးနိုင်လို့ အလုပ်ပိုတွင်ပါတယ်။ ပြုပြင်ထိမ်းသိမ်းရလွယ်စေဖို့၊ ရေးသားသူ Developer ရဲ့ ကျွမ်းကျင်မှုပေါ် မူတည်သွားပေမယ့်၊ SOLID, MVC, TDD စတဲ့ နည်းစနစ်ကောင်းဗဟုသုတတွေ ရှိထား ပြီးသား 10x Programmer တစ်ယောက်အနေနဲ့ မိမိရေးသားတဲ့ Code က ပြုပြင် ထိမ်းသိမ်းရလွယ်အောင် မိမိကိုယ်တိုင် စီမံရေးသားနိုင်မှာပါ။ ဒါကြောင့် ဒီစာအုပ်မှာ JavaScript MVC အတွက် အသုံးအများဆုံးဖြစ်နေတဲ့ Angular ကို မဖော် ပြပဲ ပိုပြီးရိုးရှင်းလွပ်လပ်တဲ့ Backbone ကို ရွေးချယ်ဖော်ပြခဲ့ခြင်းဖြစ်ပါတယ်။ စာဖတ်သူအနေနဲ့ AngularJS, EmberJS, KnockoutJS စတဲ့ အခြား JavaScript MVC တွေကို အခွင့်ရရင်တော့ ဆက်လက်လေ့လာသွားသင့်ကြောင်း အကြံပြုလို ပါတယ်။

Backbone ကို DocumentCloud, Hulu, Wordpress, Foursquare, Bitbucket, Disqus, Delicious, Khan Academy, Basecamp, Airbnb, Pandora, Trello စတဲ့ အသုံးပြုသူ သန်းချီရှိတဲ့ အထင်ကရ App တွေက အသုံးပြု ထားတဲ့အတွက် Scalable ဖြစ်ကြောင်း Proven ဖြစ်ပြီးသား Framework တစ်ခုလည်းဖြစ်ပါတယ်။ ဒါကြောင့် Rockstar Developer တစ်ဦး သိရှိထားသင့်တဲ့ Modern Development Stack ထဲက ပထမဆုံးနည်းပညာတစ်ခုအနေနဲ့ ဖော်ပြပေး လိုက်ရခြင်းပဲ ဖြစ်ပါတယ်။

CPU တစ်ခုဟာ တစ်စက္ကန့်မှာ Instruction ပေါင်းသန်းချီ
အလုပ်လုပ်နိုင်ပေမယ့်၊ Input/Output Device တွေကတော့
အဲ့ဒီလောက်မြန်မြန် အလုပ်မလုပ်နိုင်ကြပါဘူး။ စွမ်းဆောင်ရည်မြင့်
ပရိုဂရမ်တွေရရှိဖို့အတွက် ဒီအချက်ကိုသတိပြုရပါမယ်။

Professional Web Developer Course

HTML5, PHP/MySQL, jQuery/Ajax, Mobile Web စသည့်
Professional Web Developer တစ်ဦး သိရှိထားသင့်သည့်
နည်းပညာများကို စုစည်းသင်ကြားခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၁၀) – NodeJS (Server-side JavaScript)

JavaScript ဟာ မူလအစက Web Browser တစ်ခုအတွင်းမှာ အလုပ်လုပ်တဲ့ Client-side Script Language တစ်ခုသာ ဖြစ်ပါတယ်။ Client-side မှာ JavaScript ကို အဓိကလုပ်ငန်းနှစ်မျိုးအတွက် အသုံးပြုခဲ့ကြပါတယ်။ Document Object Model (DOM) API အကူအညီနဲ့ HTML Element တွေကို စီမံခြင်းအားဖြင့် Interactive User Interface တွေ ဖန်တီးဖို့နဲ့ XMLHttpRequest (XHR) API အကူအညီနဲ့ Server နဲ့ ဆက်သွယ်ပြီး UI Client တွေ ဖန်တီးဖို့ပဲဖြစ်ပါတယ်။ HTML5 နည်းပညာထွက်ပေါ်လာတဲ့အချိန်မှာတော့ Media API, Web Storage API, Web Socket API, File API, History API, Web GL API စသဖြင့် App Development အတွက် အရေးပါတဲ့ JavaScript API များ ပါဝင်လာခြင်းနဲ့အတူ JavaScript ရဲ့ အခန်းကဏ္ဍဟာ ပိုပြီးအရေးပါလာခဲ့ပါတယ်။

၂၀၀၉ ခုနှစ်ထဲမှာ Ryan Dahl လို့ခေါ်တဲ့ System Engineer တစ်ဦးက NodeJS ကို တီထွင်လိုက်ချိန်မှာတော့ JavaScript ဟာ Client-side Scripting Language သာမကတော့ပဲ ကဏ္ဍစုံမှာ အသုံးချလို့ရတဲ့ General Purpose Programing Language တစ်ခုဖြစ်လာခဲ့ပါတယ်။ အဲ့ဒီအချိန်က Web Application ဆိုတာ Application Development အတွက် အဓိက Platform တစ်ခုဖြစ်နေခဲ့တဲ့အတွက် NodeJS ကို Web Application များအတွက် အနာဂါတ် Server-side နည်းပညာအဖြစ် မျှော်မှန်းလာခဲ့ကြပါတယ်။ Web Developer တွေဟာ မူလက မည်သည့် Server-side နည်းပညာကို အသုံးချနေသည်ဖြစ်စေ၊ Client-side အတွက် JavaScript ကိုသာ အသုံးပြုကြရတာပါ။ တစ်နည်းအားဖြင့် Server-side အတွက်သုံးရတဲ့ Language က တစ်ခု၊ Client-side အတွက် သုံးရတဲ့ Language ကတစ်ခု နှစ်ခုခွဲပြီးအသုံးပြုကြရတဲ့သဘော ဖြစ်ပါတယ်။ NodeJS ထွက်ပေါ် လာတဲ့အတွက် Client-side ရော Server-side အတွက်ပါ JavaScript ဆိုတဲ့ Language တစ်မျိုးတည်းကို အသုံးပြုတည်ဆောက်နိုင်တော့မယ်ဆိုတဲ့ အလား အလာဟာ အတော်လေး စိတ်ဝင်စားစရာကောင်းခဲ့ပါတယ်။

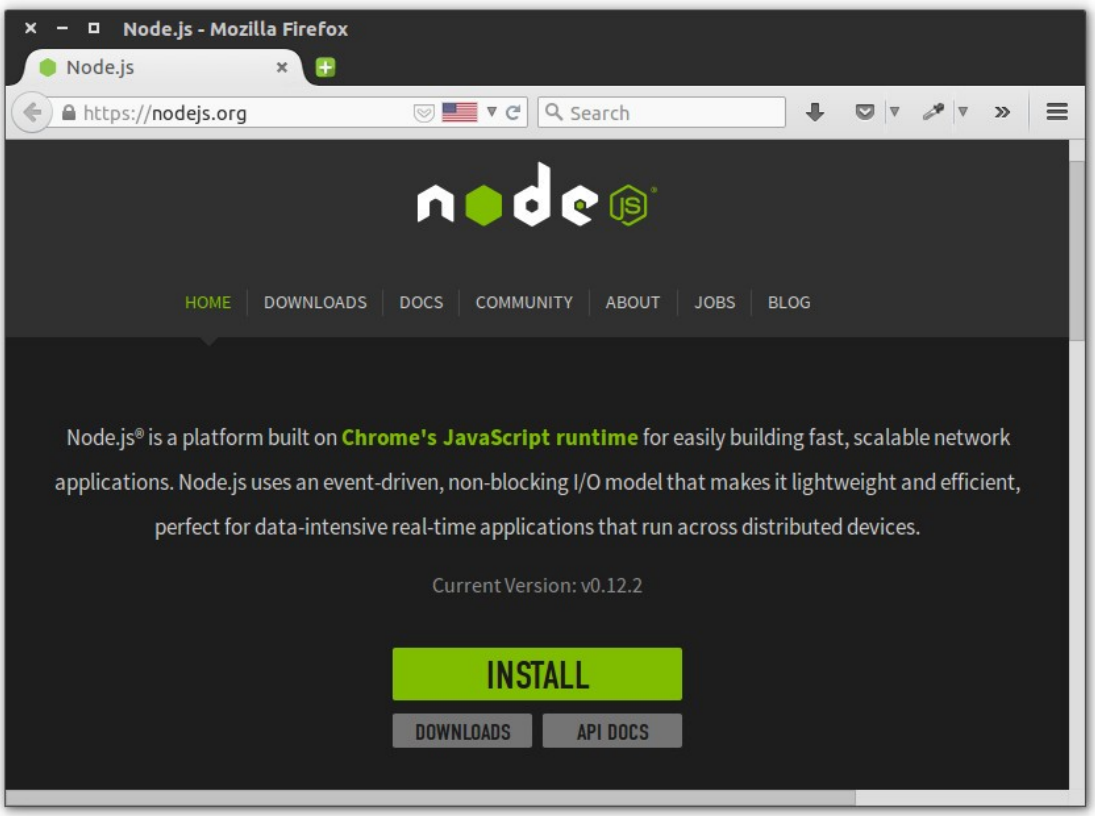
ဒါပေမယ့် လက်တွေ့မှာ NodeJS ဟာ Server-side နည်းပညာတစ်ခုဆိုတာထက် ပိုပါတယ်။ NodeJS ဟာ Event Driven/Non-blocking IO Model နည်းစနစ်ကို အသုံးပြုထားပြီး အခြား Platform တွေနဲ့မတူ တစ်မူထူးခြားတဲ့ နည်းပညာ တစ်ခုဖြစ်ပါတယ်။ Real-time Network App တွေနဲ့ Data ပမာဏများစွာကို ကိုင်တွယ်စီမံရတဲ့ I/O Heavy App တွေ တည်ဆောက်ရာမှာ အထူးကောင်းမွန်တဲ့ နည်းပညာတစ်ခုဖြစ်ပါတယ်။ ဒီအကြောင်းကို မကြာခင် ဆက်လက်ဖော်ပြသွားပါမယ်။ ဒီအခန်းရဲ့ ခေါင်းစဉ်မှာ NodeJS (Server-side JavaScript) ခေါင်းစဉ်တပ်ထားပေမယ့် Server-side ဆိုတာ Web App တွေ အတွက်တင်မကပဲ အခြား App အမျိုးမျိုးတို့အတွက် Back-end Server နဲ့ Socket Server ပရိုဂရမ် တွေ ဖန်တီးရာမှာအသုံးပြုနိုင်တဲ့ နည်းပညာဖြစ်တယ်ဆိုတဲ့ အဓိပ္ပါယ်ပဲဖြစ်ပါတယ်။

NodeJS ကို တီထွင်ခဲ့တဲ့ Ryan Dahl ဆိုတာ Joyent လို့ခေါ်တဲ့ Cloud နည်းပညာလုပ်ငန်းတစ်ခုမှာ အလုပ်လုပ်နေသူဖြစ်ပါတယ်။ တစ်နည်းအားဖြင့် NodeJS ဟာ Open Source နည်းပညာတစ်ခုဖြစ်ပေမယ့် အဓိက Sponsor အဖြစ် Joyent က ပံ့ပိုးပေးနေခြင်းဖြစ်ပါတယ်။ ဒီစာရေးသားနေချိန်မှာ နောက်ဆုံးထွက်ရှိထားတဲ့

NodeJS Version က 0.12.2 ဖြစ်ပါတယ်။ Version 1.0 လို့ Major Release အနေနဲ့ နံပါတ်မတပ်သေးပေမယ့် NodeJS ပေါ်မှာအခြေခံထားတဲ့ ဆက်စပ်နည်းပညာပေါင်းများစွာ ပေါ်ထွက်နေပြီဖြစ်သလို၊ Microsoft, Paypal, Yahoo, SAP စတဲ့ လုပ်ငန်းကြီးတွေက အစ NodeJS ကို စတင်အသုံးပြုနေကြပြီ ဖြစ်ပါတယ်။ **အခန်း (၈)** မှာ ဖော်ပြခဲ့တဲ့ Yeoman, Bower, Grunt စတဲ့ နည်းပညာတွေဟာလည်း NodeJS ကို အသုံးပြု ဖန်တီးထားခြင်း ဖြစ်ပါတယ်။

ဒီစာရေးသားနေချိန်မှာ NodeJS Project ကို ဦးဆောင်နေသူက Ryan Dahl မဟုတ်တော့ပါဘူး။ Timothy J. Fontaine ဆိုသူက ဦးဆောင်နေပါပြီ။ Joyent ဆိုတဲ့ လုပ်ငန်းအဖွဲ့အစည်းက ပံ့ပိုးပေးနေတဲ့ Project ဖြစ်တဲ့အတွက် တစ်ချို့ Project နဲ့ ပက်သက်တဲ့ Decision တွေမှာ ဒီအဖွဲ့အစည်းရဲ့ စွက်ဖက်လွှမ်းမိုးမှု အတိုင်းအတာတစ်ခုထိ ရှိနေပုံ ရပါတယ်။ ဒါကြောင့် ၂၀၁၄ ခုနှစ်ထဲမှာ Fedor Indutny ဆိုသူက NodeJS Project ကို **io.js** အမည်နဲ့ Fork လုပ် ယူပြီး ဆက်လက်ဆောင်ရွက် လာခဲ့တာ အခုဆိုရင် အတော်လေး အရှိန်ရနေပြီဖြစ်ပါတယ်။ ဒါကြောင့် အခုအချိန်မှာ NodeJS နဲ့ **io.js** ဆိုပြီး မူလ Code Base တူတဲ့ Project နှစ်ခု အပြိုင်ရှိနေတယ်လို့ ဆိုနိုင်ပါတယ်။

NodeJS ဟာ V8 လို့ခေါ်တဲ့ JavaScript Engine ပေါ်မှာ အခြေခံ တည်ဆောက်ထားခြင်းဖြစ်ပါတယ်။ V8 ဆိုတာ Google က Google Chrome Browser အတွက် ရည်ရွယ်တီထွင်ထားတဲ့ စွမ်းဆောင်ရည်မြင့် JavaScript Engine တစ်ခုဖြစ်ပါတယ်။



ပုံ (၁၀.၁) - NodeJS official Website - nodejs.org

ဖြည့်စွက်ချက် ။ ။ ဒီစာအုပ်ထွက်ပြီး မကြာခင်မှာပဲ NodeJS Community အတွင်းမှာ ထူးခြားတဲ့ပြောင်းလဲမှုပေါင်းများစွာ ဖြစ်ပေါ်ခဲ့ပါတယ်။ NodeJS နဲ့ io.js တို့ဟာ ပြန်လည်ပေါင်းစပ် သွားကြပြီး ပထမဆုံး NodeJS Major Version အဖြစ် NodeJS 4.0 ကို ကြေငြာနိုင်ခဲ့ပါတယ်။ Joyent ကိုလည်း Samsaung က ဝယ်ယူလိုက်လို့ အခုဆိုရင် Samsaung ပိုင်လုပ်ငန်း ခွဲတစ်ခု ဖြစ်သွားပါပြီ။ ဒီဖြည့်စွက်ချက်ကို ထည့်သွင်းချိန်မှာဆိုရင် NodeJS ဟာ Version 6.2.2 အထိကို ရောက်ရှိသွားခဲ့ပြီ ဖြစ်ပါတယ်။

10.1 – Non-blocking I/O

I/O ဆိုတာ Input/Output ကိုပြောတာပါ။ Hard drive ပေါ်က အချက်အလက်တွေကို ဖတ်ရှုခြင်း၊ ရေးသားခြင်း ဟာ I/O လုပ်ငန်းဖြစ်ပါတယ်။ Database Table ထဲက အချက်အလက်တွေကို ရယူခြင်း၊ ထည့်သွင်းခြင်းဟာ I/O လုပ်ငန်း ဖြစ်ပါတယ်။ Server ကို ဆက်သွယ်ပြီး အချက်အလက်တွေ ရယူခြင်း၊ ပေးပို့ခြင်းဟာ I/O လုပ်ငန်းဖြစ် ပါတယ်။ Finger Print, GPS, Accelerometer, Gyroscope စတဲ့ Sensor Device တွေကနေ အချက်အလက်တွေ ရယူခြင်းဟာ I/O လုပ်ငန်းဖြစ်ပါတယ်။ ပရိုဂရမ်တစ်ခုမှာ ပါဝင်တဲ့ Instruction တွေကို Processes လုပ်ရတာ မြန်ပါတယ်။ I/O က Process လုပ်ငန်းနဲ့ယှဉ်ရင် အများကြီးနှေးကွေးပါတယ်။ ပရိုဂရမ်တစ်ခုဟာ အဆင့်လိုက် အလုပ်လုပ်တဲ့ Serialize ပုံစံ အလုပ်လုပ်တယ်ဆိုရင် Processes က I/O ကို စောင့်နေရတဲ့အတွက် မလိုအပ်ပဲ ပရိုဂရမ်ရဲ့စွမ်းဆောင်ရည် ကျဆင်းရပါတယ်။

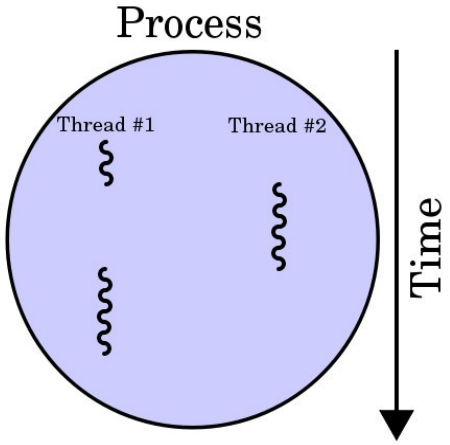
ဥပမာ – ပရိုဂရမ်တစ်ခုဟာ အချက်အလက်တွေအများကြီးပါတဲ့ဖိုင်တစ်ခုကိုဖွင့်ပြီး အလုပ်လုပ်ဖို့လိုတယ်ဆိုကြ ပါစို့။ ဖိုင်ကို ဖွင့်ပြီး အချက်အလက်တွေ ဖတ်ယူတဲ့လုပ်ငန်းမပြီးမချင်း ပရိုဂရမ်က တစ်ခြားအလုပ်တွေကို လုပ် နိုင်မှာမဟုတ်ပါဘူး။ ဖိုင်ကိုဖွင့်ဖတ်တဲ့ အချိန်ဟာ တစ်စက္ကန့်လည်း ဖြစ်နိုင်ပါတယ်။ နှစ်စက္ကန့်လည်း ဖြစ်နိုင်ပါ တယ်။ စက္ကန့်သုံးဆယ်လည်း ဖြစ် နိုင်ပါတယ်။ ဖိုင် Size ပေါ်မူတည်ပါတယ်။ တစ်စက္ကန့်မှာ Instruction ပေါင်း (၁၀)သန်းလောက်ကို အလုပ်လုပ်နိုင်တဲ့ CPU အတွက် တစ်စက္ကန့်၊ နှစ်စက္ကန့် ဆိုတဲ့ အချိန်ဟာ အတော်ကြီးမား တဲ့အဟန့်အတားတစ်ခုပါ။

Process တွေက I/O လုပ်ငန်းတွေကို စောင့်နေစရာမလိုအောင် စီမံရေးသားတဲ့နည်းစနစ်ကို Non-blocking I/O လို့ ခေါ်တာပါ။ Asynchronous I/O လို့လည်းခေါ်ပါတယ်။ တစ်ခုနဲ့တစ်ခု စောင့်စရာမလိုပဲ ပြိုင်တူအလုပ်လုပ် နိုင်တဲ့အတွက် Concurrency လို့လည်း ခေါ်ပါတယ်။

Non-blocking I/O လုပ်ဆောင်ချက်ရဖို့အတွက် Inter-process Communication နည်းစနစ်ကို တစ်ချို့က သုံးကြ ပါ တယ်။ ပရိုဂရမ်တစ်ခုကို Run လိုက်တဲ့အခါ Operating System က Memory Space အပါအဝင် System Resource တွေကို ခွဲဝေပေးထားတဲ့ Process တစ်ခုဖွင့်ပေးပါတယ်။ ပရိုဂရမ်မှာပါတဲ့ Instruction တွေကို အဲ့ဒီ Process ထဲမှာ အစီအစဉ်အတိုင်း အလုပ်လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ Non-blocking I/O လုပ်ဆောင်ချက်ရဖို့ အတွက် Process နှစ်ခု သုံးခုကို အပြိုင်အသုံးပြုနိုင်ပါတယ်။ ဥပမာ – ဖိုင်ကိုဖွင့်ဖတ်ပြီး ပါဝင်တဲ့အချက်အလက် တွေကို Memory ပေါ်တင်ပေးတဲ့ I/O လုပ်ငန်းကို မူလ Process မှာမလုပ်ပဲ သီးခြား Process တစ်ခုဖွင့်ပြီး အလုပ်လုပ်စေနိုင်ပါတယ်။ ဒါပေမယ့် ပြဿနာ က Process ဆိုတာ သီးခြားအလုပ်လုပ်ကြတာပါ။ အခြေခံ အားဖြင့် တစ်ခုနဲ့တစ်ခု ဆက်စပ်မှုမရှိပါဘူး။ ဒါကြောင့် သီးခြား Process တစ်ခုနဲ့ အလုပ်လုပ်နေတဲ့ I/O လုပ်ငန်းရဲ့ ဆောင်ရွက်မှုအခြေအနေကို မူလ Process က အလို အလျောက်မသိနိုင်ပါဘူး။ မူလ Process အနေနဲ့ I/O Process ရဲ့ လုပ်ငန်းပြီးမြောက်မှုအခြေအနေကို သိရဖို့လိုပါတယ်။ ဥပမာ – ဖိုင်ထဲကအချက်အလက်တွေကို

Memory ပေါ် တင်ပြီးပြီဆိုရင်၊ အချက်အလက်တွေရှိနေတဲ့ Memory Space ရဲ့ Reference ကိုသိရမှ မူလ Process က Memory ထဲမှာရောက်ရှိနေပြီးဖြစ်တဲ့ အချက်အလက်တွေကို ပရိုဂရမ်မှာ ဖော်ပြပေးတဲ့ လုပ်ငန်းကို လုပ်နိုင်မှာပါ။ ဒီလိုမျိုး Process တစ်ခုနဲ့တစ်ခု အပြန်အလှန် ဆက်သွယ်အလုပ်လုပ်နိုင် အောင် စီမံရေးသားခြင်းကို Inter-process Communication (IPC) လို့ခေါ်တာပါ။ Non-blocking I/O လုပ်ဆောင်ချက်ရဖို့ IPC နည်းစနစ်ကို တစ်ချို့ကသုံးကြပါတယ်။ Message Passing, Message Queue, Socket, Memory Mapped File စသဖြင့် IPC လုပ်ဆောင်ချက်ရရှိဖို့အသုံးပြုနိုင်တဲ့ နည်းစနစ်အမျိုးမျိုး ရှိပါတယ်။

တစ်ချို့ ပရိုဂရမ်တွေကတော့ Non-blocking I/O လုပ်ဆောင်ချက်ရရှိဖို့အတွက် Time Sharing Thread တွေကို သုံး ကြပါတယ်။ Thread ဆိုတာ Process တစ်ခုအတွင်းမှာပဲ Instruction တွေကို ခွဲခြားအလုပ်လုပ်စေတဲ့ နည်းစနစ် တစ်မျိုးဖြစ်ပါတယ်။ Process နှစ်ခုသုံးခု သုံးဖို့မလိုတော့ပဲ Process တစ်ခုတည်းနဲ့ Non-blocking I/O လုပ်ဆောင်ချက် ရရှိအောင် စီမံသွားနိုင်ခြင်း ဖြစ်ပါတယ်။ Thread တွေက Process တစ်ခုအတွင်းမှာ အလုပ်လုပ်ကြခြင်းဖြစ်လို့ အချင်းချင်း အပြန်အလှန်ဆက်သွယ် ဆောင်ရွက်နိုင်ကြပါတယ်။ IPC လိုနည်းစနစ်တွေထပ်ပြီး Implement လုပ်နေစရာမလိုပဲ သက်ဆိုင်ရာ Language နဲ့ Platform ကပေးထားတဲ့ Thread API တွေကို သုံးပြီး ရေးသားစီမံနိုင်ပါတယ်။



ပုံ (၁၀.၂) - Time Sharing Threads

Source – Wikipedia

ပုံ (၁၀.၂) မှာ Time Sharing Thread စနစ်တစ်ခုရဲ့ အလုပ်လုပ်ပုံကို ဖော်ပြထားပါတယ်။ Process တစ်ခုအတွင်းမှာ Thread နှစ်ခုရှိနေပြီး Instruction တွေကို အပြိုင်အလုပ်လုပ်ကြပါတယ်။ ထူးခြားချက်အနေနဲ့ Thread #2 ကို အလုပ် လုပ်နေစဉ်မှာ Thread #1 ကိုခေတ္တရပ်နားထားပြီး Thread #1 အလုပ်လုပ်နေစဉ်မှာ Thread #2 ကို ခေတ္တရပ်နားထား တဲ့သဘော ဖော်ပြထားတာကို တွေ့နိုင်ပါတယ်။ Process ရဲ့အချိန်ကို ဝေမျှသုံးစွဲပြီး လုပ်ငန်းနှစ်ခုသုံးခုကို ပြိုင်တူအလုပ် လုပ်တဲ့အတွက် Time Sharing Thread လို့ခေါ်တာပါ။ ဒီနည်းနဲ့ I/O လုပ်ငန်းတွေကို Thread တစ်ခုခွဲပြီး ဆောင်ရွက် နေစဉ်မှာ ပရိုဂရမ်ရဲ့ အခြား Instruction တွေက သီးခြား Thread အနေနဲ့ ဆက်လက်အလုပ်လုပ်နိုင်မှာပဲ ဖြစ်ပါတယ်။ Thread တွေစီမံရတာလည်း လွယ်ကူတဲ့အလုပ်

တော့မဟုတ်ပါဘူး။ Thread တစ်ခုနဲ့တစ်ခု အပြန်အလှန်စောင့်ရင်း ပရိဂရမ်က ရှေ့မဆက်နိုင်ပဲ Hang သွားခြင်း၊ Resource တစ်ခုတည်းကို Thread နှစ်ခုက တစ်ပြိုင်တည်း အသုံးပြုမိလို့ မလိုလားအပ်တဲ့ရလဒ်တွေ ထွက်ပေါ်လာခြင်း စတဲ့ပြဿနာတွေ မဖြစ်အောင် စီမံရေးသားနိုင်ဖို့ လိုပါတယ်။

NodeJS မှာတော့ Non-blocking I/O လုပ်ဆောင်ချက်ရရှိဖို့အတွက် Callback နည်းစနစ်ကို အသုံးပြုပါတယ်။ အမှန် တော့ I/O ကို Process တွေက စောင့်ရတယ်ဆိုတာ Process လုပ်ငန်းပြည့်စုံဖို့အတွက် I/O ကပေးတဲ့ ရလဒ်ကို လိုအပ် လို့ဖြစ်ပါတယ်။ IPC လို နည်းစနစ်တွေကို အသုံးပြုရတယ်ဆိုတာလည်း I/O Process တစ်ခု ကပေးတဲ့ ရလဒ်ကို မူလ Process က ရယူအသုံးပြုနိုင်ဖို့ လိုအပ်တဲ့အတွက် ဖြစ်ပါတယ်။ NodeJS မှာတော့ I/O လုပ်ဆောင်ချက်ကပေးတဲ့ ရလဒ် ကိုရရှိဖို့အတွက် Process က စောင့်နေစရာမလိုပါဘူး။ I/O လုပ်ဆောင်ချက် တိုင်းနဲ့အတူ Callback Function တစ်ခု တွဲဖက်သတ်မှတ်ပေးနိုင်ပါတယ်။ I/O လုပ်ဆောင်ချက်ပြီးလို့ ရလဒ်ရရှိ လာတဲ့အခါ Callback Function က အလိုအလျောက်အလုပ်လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Process တွေက I/O ကို စောင့်စရာမလိုပဲ၊ လုပ်စရာရှိတဲ့အလုပ်ကို အပြိုင်လုပ်လို့ ရသွားပါတယ်။ ဥပမာ တစ်ခုဖော်ပြပါ မယ် –

PHP

```php
$data = file("bigfile.csv");
print_r($data);

// other processes
```

ပေးထားတဲ့ PHP Code နမူနာမှာ `file()` Function ကိုသုံးပြီး `bigfile.csv` လို့ခေါ်တဲ့ ဖိုင်တစ်ခုကိုဖတ်ယူထား ပါတယ်။ ဖိုင်ကိုဖတ်နေတဲ့အချိန်မှာ တစ်ခြား Process တွေကို အလုပ်လုပ်ပေးနိုင်မှာမဟုတ်ပါဘူး။ ဖိုင်ကိုဖတ်တဲ့လုပ်ငန်း ပြီးတော့မှသာ အခြား Process တွေကို အလုပ်လုပ်နိုင်မှာဖြစ်ပါတယ်။

JavaScript

```javascript
var fs = require("fs");

fs.readFile("bigfile.csv", "utf8", function(err, data) {
    console.log(data)
});

// other processes
```

ပေးထားတဲ့ NodeJS Code နမူနာမှာ `fs.readFile()` Function ကိုသုံးပြီး `bigfile.csv` ကိုဖတ်ထားပါ တယ်။ ဒါပေမယ့် အဲ့ဒီဖိုင်ကိုဖတ်နေတဲ့အချိန်မှာ အခြား Process တွေက စောင့်စရာမလိုပါဘူး ဆက်လက် အလုပ်လုပ် သွားနိုင်ပါတယ်။ ဖိုင်ဖတ်တဲ့လုပ်ငန်း ပြီးတဲ့အခါ `fs.readFile()` ရဲ့ နောက်ဆုံး Parameter အဖြစ် တွဲဖက်ပေးလိုက်တဲ့ Callback Function က အလုပ်လုပ်သွားပြီး ရရှိလာတဲ့ File Content ကို ရိုက်ထုတ်ဖော်ပြပေးသွားမှာဖြစ်ပါတယ်။

ဒီနည်းနဲ့ Callback Function ကိုသုံးပြီး Non-blocking I/O လုပ်ဆောင်ချက် ရရှိသွားတဲ့အတွက် Thread တွေ Process တွေသုံးစရာမလိုတော့ပါဘူး။ Thread တွေအများကြီးဖွင့်စရာမလိုတဲ့အတွက် ရေးသားစီမံရတာ လွယ်ကူသွားတဲ့အပြင်၊ Resource Consumption နည်းသွားပြီး Process ပေါင်းများစွာ၊ Request ပေါင်းများစွာ ကို လက်ခံစီမံဖို့လိုတဲ့ Network ပရိုဂရမ်တွေ ရေးသားဖန်တီးရာမှာ များစွာအထောက်အကူဖြစ်စေမှာပဲဖြစ်ပါတယ်။

10.2 – Event Driven Architecture

Web App တွေအပါအဝင် Graphical User Interface (GUI) ကို အခြေခံအလုပ်လုပ်တဲ့ ပရိုဂရမ်တစ်ခု ဖန်တီးတိုင်းမှာ Event Driven Programing ကို ကျွန်တော်တို့ သုံးကြလေ့ရှိပါတယ်။ ပရိုဂရမ်ကို ရေးသားတဲ့အခါ ကြိုတင်သတ်မှတ်ထား တဲ့ အစီအစဉ်အတိုင်း မဟုတ်ပဲ၊ Button ကို Click လုပ်လိုက်ရင် ဘယ်အလုပ်လုပ်ရမယ်၊ Image ကို Load လုပ်လိုက်ရင် ဘယ်အလုပ်လုပ်ရမယ်၊ Menu ကို Mouse နဲ့ ထောက်လိုက်ရင် ဘယ်အလုပ်လုပ်ရမယ်၊ စသဖြင့် UI ပေါ်မှာဖြစ်ပေါ်တဲ့ Action ပေါ်မူတည်ပြီး အလုပ်လုပ်အောင် စီစဉ်ရေးသားရတဲ့ Programing နည်းစနစ်ဖြစ်ပါတယ်။

Event Driven Architecture ဆိုတာကတော့ Event Driven Programming လိုပဲ ပရိုဂရမ်ကို ကြိုတင်သတ်မှတ်ထားတဲ့ အစီအစဉ်အတိုင်း အလုပ်လုပ်စေခြင်းမဟုတ်ပဲ ဖြစ်ပေါ်တဲ့ Event များပေါ်မှာ မူတည်အလုပ်လုပ်အောင် စီစဉ်ရေးသားခြင်း ပဲဖြစ်ပါတယ်။ User Interface အတွက်သာမကတော့ပဲ စနစ်တစ်ခုလုံးကို Event များပေါ်မှာ အခြေခံအလုပ်လုပ်အောင် စီစဉ်ရေးသားတဲ့ နည်းစနစ်ဖြစ်ပါတယ်။

ပြီးခဲ့တဲ့အခန်းမှာ BackboneJS အကြောင်း လေ့လာခဲ့ကြစဉ်က `add`, `remove`, `reset`, `sort`, `error` စသဖြင့် Event တွေဖြစ်ပေါ်တဲ့အခါ ဆောင်ရွက်ရမယ့်လုပ်ငန်းတွေ သတ်မှတ်ထားနိုင်ပုံကို တွေ့မြင်ခဲ့ကြရပြီးဖြစ်ပါတယ်။ NodeJS နဲ့လည်း အဲ့ဒီလို Custom Event တွေကိုတည်ဆောက်ပြီး Event များကို အခြေခံအလုပ်လုပ်တဲ့ ပရိုဂရမ်အဖြစ် တည်ဆောက် နိုင်ခြင်းဖြစ်ပါတယ်။ ဥပမာ –

Pseudo-code

```
issue.on('assigned', function() {
     // send notification to assignee
});

issue.on('done', function() {
     // send notification to submitter
});
issue.on('close', function() {
     // archive issue
});
```

UI Event တွေမှာ `onClick`, `onKeyPress`, `onMouseMove` စသဖြင့်ရှိသလိုပဲ ကျွန်တော်တို့ပရိုဂရမ်တွေမှာလည်း နမူနာမှာပေးထားသလို `assigned`, `done`, `close` စသဖြင့် Event တစ်ခုဖြစ်ပေါ်တဲ့အခါ ဆောင်ရွက်ရမယ့် လုပ်ငန်းတွေကို သတ်မှတ်ထားနိုင်အောင် ရေးသားနိုင်မှာပဲဖြစ်ပါတယ်။

10.3 – Running JavaScript with NodeJS

အခန်း (၈) မှာ Yeoman နဲ့ အခြား Tool တွေအသုံးပြုဖို့ NodeJS လိုအပ်တဲ့အတွက် NodeJS Install ပြုလုပ်ပုံကို ထည့်သွင်း ဖော်ပြခဲ့ပါတယ်။ အကယ်၍ Install မလုပ်ရသေးရင်လည်း nodejs.org မှာ Installer ကို Download ရယူ နိုင်ပါတယ်။ Ubuntu မှာ ဆိုရင်တော့ ကိုယ်တိုင်သွားပြီး Download လုပ်မနေပဲ အခုလို Install ပြုလုပ်နိုင်ပါတယ်။

```
$ sudo apt-get install nodejs nodejs-legacy
```

Install လုပ်ပြီးပြီဆိုရင် `node` Command ကနေတစ်ဆင့် NodeJS Console ကိုရရှိနိုင်ပါတယ်။

> ဆက်လက်ဖော်ပြမယ့် Command တွေနဲ့ Code တွေဟာ လက်တွေ့ကူးယူစမ်းသပ်နိုင်တဲ့ Command နဲ့ Code တွေဖြစ်ပါတယ်။

NodeJS Console ထဲမှာ JavaScript Code တွေကို အခုလို တိုက်ရိုက်ရေးသားစမ်းသပ်နိုင်ပါတယ်။

```
$ node
> console.log("Hello, World!");

Hello, World!
undefined

> function area(r) {
... var pi = 3.14;
... return pi * r * r;
... }

undefined

> area(8)

200.96
```

NodeJS Console ကိုမဝင်ပဲ JavaScript Code တွေကို တိုက်ရိုက် Run လိုရင်တော့ `-e` သို့မဟုတ် `-p` Option ကို သုံးနိုင်ပါတယ်။

```
$ node -e 'console.log("Hello, World")'

Hello, World
```

`-e` Option က Evaluate ဆိုတဲ့အဓိပ္ပါယ်ဖြစ်ပြီး `-p` ကတော့ Evaluate and Print ဆိုတဲ့အဓိပ္ပါယ်ပါ။ ဖိုင်တစ်ခု

အနေနဲ့ ရေးသားထားတဲ့ JavaScript Code တွေကိုတော့ အခုလို Run နိုင်ပါတယ်။

```
$ node script.js [args]
```

`[args]` နေရာမှာ `script.js` ကို Run စဉ် တွဲဖက်ပေးလိုတဲ့ Arguments တွေကို ပေးနိုင်ပါတယ်။ Arguments တွေကို NodeJS က `process.args` ဆိုတဲ့ Object နဲ့လက်ခံထားပေးမှာဖြစ်ပါတယ်။ ဥပမာ – `script.js` ထဲမှာ အခုလို ရေးသားစမ်းသပ်နိုင်ပါတယ်။

JavaScript

```
console.log(process.args)
```

`process.args` မှာရှိတဲ့တန်ဖိုးတွေကို ရိုက်ထုတ်ထားခြင်း ဖြစ်ပါတယ်။ ပြီးတဲ့အခါ `script.js` ကို Arguments တွေပေးပြီး Run ကြည့်ပါမယ်။

```
$ node script.js yo bower grunt
[ 'node', '/home/eimg/script.js', 'yo', 'bower', 'grunt' ]
```

`yo`, `bower` နဲ့ `grunt` တို့ကို Argument အနေနဲ့ပေးပြီး Run ကြည့်လိုက်တဲ့အခါ `process.args` ဟာ ကျွန်တော်တို့ပေးလိုက်တဲ့ `yo`, `bower` နဲ့ `grunt` တို့ ပါဝင်တဲ့ Array တစ်ခုဖြစ်တယ်ဆိုတာကို တွေ့ရနိုင်ပါတယ်။ ရှေ့ဆုံးက `node` နဲ့ ဖိုင် Path ဆိုတဲ့ Index နှစ်ခုက `process.args` မှာ အမြဲ ရှိနေမှာဖြစ်ပါတယ်။ ဒါကြောင့် အဲ့ဒီ Index နှစ်ခုကို ရလဒ်မှာမပါဝင်စေလိုရင် အခုလိုပြင်ဆင်ရေးသားနိုင်ပါတယ်။

JavaScript

```
console.log(process.args.slice(2))
```

ပြင်ဆင်ရေးသားထားတဲ့ `script.js` ကိုနောက်တစ်ခေါက်စမ်းကြည့်ရင် ရလဒ်က အခုလိုဖြစ်မှာပါ။

```
$ node script.js yo bower grunt
[ 'yo', 'bower', 'grunt' ]
```

`script.js` ကို Run စဉ်ကပေးလိုက်တဲ့ တန်ဖိုးတွေကို ရရှိတာကိုတွေ့ရမှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ User Input တွေကို Command Argument အနေနဲ့ လက်ခံအလုပ်လုပ်နိုင်တဲ့ Command Line ပရိုဂရမ်တွေကို NodeJS နဲ့ ရေးသား ဖန်တီးနိုင်မှာဖြစ်ပါတယ်။ Yeoman, Bower, Grunt စတဲ့ Tool တွေတာ ဒီနည်းနဲ့ရေးသားထားတဲ့ Command Line ပရိုဂရမ်တွေဖြစ်ပါတယ်။

10.4 – Debugging in NodeJS

JavaScript ပရိုဂရမ်တစ်ခုကို Debug လုပ်လိုရင် `node` Command ကို `debug` Argument နဲ့ တွဲသုံးနိုင်ပါတယ်။

```
$ node debug script.js
```

NodeJS က Debug Console တစ်ခုနဲ့ ပရိုဂရမ်ရဲ့ လိုင်းနံပါတ် (၁) ကနေစပြီး တစ်လိုင်းချင်း Debug လုပ်နိုင်အောင် စီစဉ်ပေးထားပါလိမ့်မယ်။ Code ထဲမှာ Break Point တွေ သတ်မှတ်လိုရင်တော့ debugger Statement ကို အသုံးပြု နိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
1. var pi = 3.14;
2. function area(r) {
3.     var a = pi * r * r;
4.     debugger;
5.     return a;
6. }
7.
8. console.log( area(8) );
9. console.log( area(12) );
```

လိုင်းနံပါတ် (၄) မှာ `debugger` Statement ကိုသုံးထားတဲ့အတွက် Node Debug က ဒီလိုင်းကိုရောက်တိုင်း Break လုပ်ပေးမှာ ဖြစ်ပါတယ်။ ဒီ Code ကို `node debug` နဲ့ Run ကြည့်ရင် အခုလို တွေ့ရပါလိမ့်မယ်။

```
$ node debug script.js

< debugger listening on port 5858
connecting... ok
break in script.js:1
  1 var pi = 3.14;
  2 function area(r) {
  3   var a = pi * r * r;

debug>
```

ပရိုဂရမ်ဟာ လိုင်းနံပါတ် (၁) မှာ ရပ်နေပြီး Debug Console ကိုအသင့်ပြင်ထားပေးပါလိမ့်မယ်။ ပရိုဂရမ်ဆက်အလုပ် လုပ်သွား စေဖို့အတွက် `cont` (သို့မဟုတ်) `c` Command ကို ရိုက်ထည့်ပေးရပါတယ်။

```
debug> c

break in script.js:4
  2 function area(r) {
  3   var a = pi * r * r;
```

```
  4   debugger;
  5   return a;
  6 }

debug>
```

ပရိုဂရမ်က ဆက်အလုပ်လုပ်သွားပေမယ့် `debugger` ကိုတွေ့တဲ့အတွက် လိုင်းနံပါတ် (၄) မှာ နောက်တစ်ခေါက် ရပ် သွားပါလိမ့်မယ်။ အဲ့ဒီအချိန်မှာ `repl` Command ကိုသုံးပြီး JavaScript Code တွေကို Run ကြည့်လို့ ရပါတယ်။ Debug Console ကနေ `repl` Console ကိုပြောင်းသွားပါလိမ့်မယ်။ Variable တွေရဲ့ လက်ရှိတန်ဖိုး ကို စီစစ်ဖို့အသုံးဝင် ပါတယ်။

```
debug> repl

Press Ctrl + C to leave debug repl

> a

200.96

debug>
```

နမူနာမှာ Variable a ထဲက တန်ဖိုးကိုသိချင်တဲ့အတွက် `a` လို့ ရိုက်ထည့် ပေးလိုက်တဲ့အခါ လက်ရှိတန်ဖိုး `200.96` ကို ဖော်ပြလာခြင်းပဲဖြစ်ပါတယ်။ `repl` Console ကနေထွက်ပြီး Debug Console ကို ပြန်သွားဖို့ အတွက် Ctrl+C ကိုနှိပ် ပေးရပါတယ်။ Debug Console ပြန်ပေါ်လာတဲ့အခါ လက်ရှိ Break လုပ်ထားတဲ့ Function ကနေထွက်ပြီး နောက်တစ် ဆင့်ကို ဆက်လက် Debug လုပ်သွားဖို့အတွက် `out` (သို့မဟုတ်) `o` Command ကို သုံးနိုင်ပါတယ်။

```
debug> o

break in script.js:8
  6 }
  7
  8 console.log( area(8) );
  9 console.log( area(12) );
 10

debug>
```

Function အပြင်ဘက်က လိုင်းနံပါတ် (၈) ကိုရောက်သွားပါလိမ့်မယ်။ နောက်တစ်လိုင်းကို ဆက်သွားချင်တယ် ဆိုရင် `next` (သို့မဟုတ်) `n` Command ကို သုံးနိုင်ပါတယ်။

```
debug> n

< 200.96
break in script.js:9
  7
  8 console.log( area(8) );
  9 console.log( area(12) );
 10
 11 });

debug>
```

လိုင်းနံပါတ် (၉) ကိုရောက်သွားပါလိမ့်မယ်။ `n` Command နဲ့တစ်လိုင်းပြီးတစ်လိုင်းဆက်သွားပြီး Debug လုပ်သွားနိုင်ပါတယ်။ တစ်လိုင်းချင်းသွားယုံသာမက၊ ခေါ်ယူအသုံးပြုထားတဲ့ Function ထဲကိုပါ ဝင်လေ့လာတယ်ဆိုရင်တော့ `step` (သို့မဟုတ်) `s` Command ကို သုံးနိုင်ပါတယ်။

```
debug> s

break in node.js:205
 203   startup.globalConsole = function() {
 204     global.__defineGetter__('console', function() {
 205       return NativeModule.require('console');
 206     });
 207   };

debug>
```

လိုင်းနံပါတ် (၉) မှာ `console.log()` Function ကိုခေါ်ထားတဲ့အတွက် `s` Command ကိုသုံးလိုက်တဲ့အခါ NodeJS ရဲ့ `console.log()` Function ရေးသားထားရာနေရာထိ ရောက်ရှိသွားတာကို တွေ့ရမှာဖြစ်ပါတယ်။ အဲ့ဒီနေရာကနေထွက်ပြီး မူလအလုပ်လုပ်နေတဲ့ နေရာကိုပြန်သွားဖို့အတွက် `out` (သို့မဟုတ်) `o` Command ကို သုံးနိုင်ပါ တယ်။

```
debug> o

break in script.js:9
  7
  8 console.log( area(8) );
  9 console.log( area(12) );
 10
 11 });

debug>
```

လက်ရှိပရိုဂရမ်ရဲ့ လိုင်းနံပါတ် (၉) ကိုပြန်လည်ရောက်ရှိ လာတာကိုတွေ့ရမှာပဲဖြစ်ပါတယ်။ ပရိုဂရမ်ကို အဆုံးထိ ဆက်လက် အလုပ်လုပ်သွားစေဖို့အတွက် `cont` (သို့မဟုတ်) `c` ကို သုံးနိုင်ပါတယ်။

```
debug> c

break in script.js:4
  2 function area(r) {
  3   var a = pi * r * r;
  4   debugger;
  5   return a;
  6 }

debug>
```

ပရိုဂရမ်ဆက်အလုပ်လုပ်တဲ့အခါ `area()` Function ကို နောက်တစ်ကြိမ်ထပ်ခေါ်ထားတဲ့အတွက် `area()` Function ထဲကိုရောက်သွားပြန်ပါတယ်။ `area()` Function ထဲမှာ `debugger` Statement ရှိနေတဲ့အတွက် `debugger` Statement ရှိရာလိုင်းမှာ နောက်တစ်ကြိမ် ရပ်နေပေးပါလိမ့်မယ်။ `repl` Console ကိုဝင်ပြီး Variable `a` ရဲ့တန်ဖိုးကို လေ့လာ ကြည့်နိုင်ပါတယ်။

```
debug> repl

Press Ctrl + C to leave debug repl

> a

452.15999999999997

debug> c

< 452.15999999999997
program terminated

debug>
```

`a` ရဲ့ တန်ဖိုး စောစောကနဲ့ မတူတော့ပဲ 452.159 ဖြစ်နေတာကို တွေ့ရမှာဖြစ်ပါတယ်။ Debug Mode ကိုပြန်သွားပြီး `c` Command နဲ့ ပရိုဂရမ်ကို ဆက်လက်အလုပ်လုပ်စေလိုက်တဲ့အခါမှာတော့ ပရိုဂရမ်ရဲ့လုပ်ငန်းဆောင်တာလုံးပြီးမြောက်သွားတဲ့ အတွက် program terminated ဆိုတဲ့ Message ကိုပေးပြီး ရပ်သွားမှာပဲဖြစ်ပါတယ်။

နမူနာမှာ အသုံးပြုခဲ့တဲ့ Debug Console Command စာရင်းကို ထပ်မံဖော်ပြလိုက်ပါတယ်။

- **`cont, c`** – ပရိုဂရမ်ကို ဆက်အလုပ်လုပ်သွားစေလိုတဲ့အခါ သုံးပါတယ်။
- **`next, n`** – နောက်တစ်လိုင်းကို ဆက်လက်လေ့လာလိုတဲ့အခါ သုံးပါတယ်။
- **`step, s`** – Step In ဆိုတဲ့အဓိပ္ပါယ်ပါ။ အခြား Platform တွေမှာရှိတဲ့ Debug စနစ်တွေရဲ့ Step In နဲ့ သဘောတရားအတူတူပါပဲ။ လက်ရှိရောက်ရှိနေတဲ့လိုင်းက ခေါ်ယူအသုံးပြုထားတဲ့ Function ကို ဝင်ရောက်လေ့ လာလိုတဲ့အခါ သုံးပါတယ်။
- **`out, o`** – Step Out ဆိုတဲ့အဓိပ္ပါယ်ပါ။ အခြား Platform တွေမှာရှိတဲ့ Debug စနစ်တွေရဲ့ Step Out နဲ့

သဘောတရားအတူတူပါပဲ။ လက်ရှိရောက်ရှိနေတဲ့ Function ထဲကနေထွက်ပြီး နောက်တစ်လိုင်းကို ဆက်လက် Debug လုပ် သွားစေလိုတဲ့အခါ သုံးပါတယ်။

- **pause** – နမူနာမှာသုံးမထားပါဘူး။ အလုပ်လုပ်နေတဲ့ပရိုဂရမ်ကို ရောက်ရာနေရာမှာ ရပ်တန့်လိုက်စေလိုတဲ့ အခါ သုံးပါတယ်။

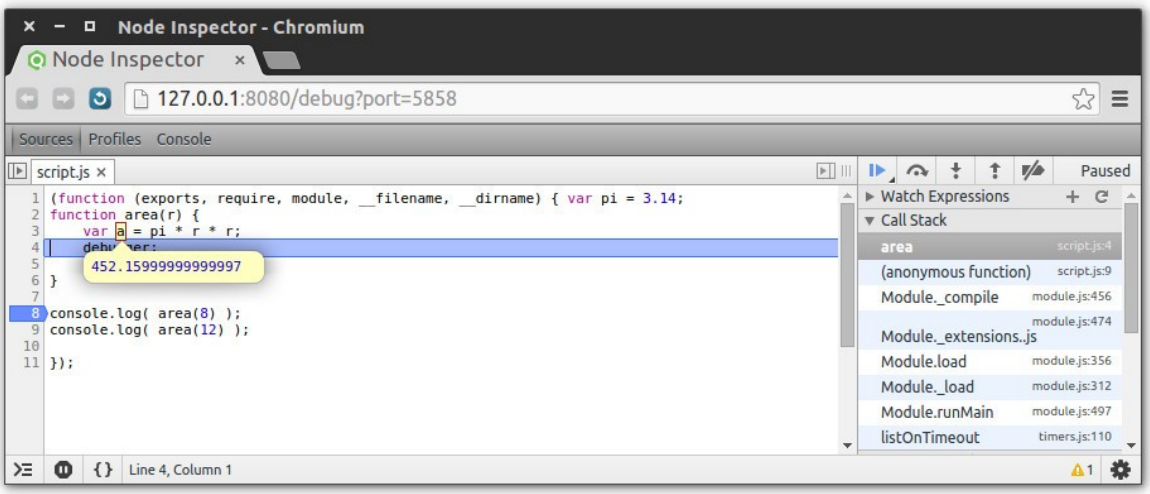
Debug Console ထဲမှာ `help` Command ကိုသုံးပြီး အသုံးပြုလို့ရတဲ့ Command စာရင်းကိုကြည့်ရှုနိုင်ပါတယ်။ ဒီလို Command Line ကနေ Debug လုပ်ရတာ ရှုပ်တယ်ထင်ရင်တော့ Node Inspector ဆိုတဲ့ Package တစ်ခုကို အသုံးပြု နိုင်ပါတယ်။ NPM နဲ့ အခုလို Install လုပ် နိုင်ပါတယ်။

```
$ npm install -g node-inspector
```

Node Inspector ဆိုတာ အမှန်တော့ Chromium Browser ရဲ့ Dev Tool ပဲဖြစ်ပါတယ်။ Firebug, Chrome Dev Tool စတဲ့ Debug Tool တွေနဲ့ ရင်းနှီးပြီးသားသူတွေအတွက် အထူးတလည်ထပ်မံလေ့လာနေစရာမလိုပဲ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။ Node Inspector နဲ့ JavaScript Code တွေကို Debug လုပ်နိုင်ဖို့ `node-debug` Command ကို သုံးရပါတယ်။ ဥပမာ –

```
$ node-debug script.js
debugger listening on port 5858
Node Inspector is now available from http://127.0.0.1:8080/debug?port=5858
Debugging `script.js`
```

Node Inspector က `script.js` ကို Chrome Dev Tool နဲ့ဖော်ပြပေးမှာဖြစ်ပါတယ်။ အကယ်၍ အလိုအလျောက် မဖော်ပြရင်လည်း `node-debug` Command Message ထဲမှာပါတဲ့ 127.0.0.1:8080/debug?port=5858 ဆိုတဲ့ URL ကို Google Chrome (သို့မဟုတ်) Chromium Browser နဲ့ဖွင့်ပြီး Node Inspector UI ကို ရရှိနိုင်ပါတယ်။



ပုံ (၁၀.၃) - Node Inspector

ပုံ (၁၀.၃) မှာလေ့လာကြည့်ပါ။ `script.js` ကို Node Inspector နဲ့ Debug လုပ်နေခြင်းဖြစ်ပါတယ်။ Continue, Step Into, Step Over, Step Out, Pause စတဲ့ Debug Command များအတွက် Button လေးတွေကို ညာဘက် အပေါ်ထောင့်မှာ ပေးထားပါတယ်။ `debugger` Statement တွေမှာ အလိုအလျောက် Break လုပ်ပေးသလို နောက် ထပ် Break Point တွေ ထပ်ထည့်ချင်ရင်လည်း Line Number ကိုနှိပ်ပြီး Break Point သတ်မှတ်ခြင်း၊ ပယ်ဖျက်ခြင်းကို ဆောင်ရွက်နိုင်ပါတယ်။ Variable တွေကို Mouse နဲ့ ထောက်ကြည့်လိုက်ရင် လက်ရှိတန်ဖိုးကိုဖော်ပြပေးသလို၊ ညာ ဘက်ခြမ်းက Side-bar ထဲမှာ လက်ရှိ Variable တွေ Function တွေနဲ့ပက်သက်တဲ့ အချက်အလက်တွေကို ဖော်ပြပေး ထားတာကို တွေ့ရမှာဖြစ်ပါတယ်။ ပါဝင်တဲ့ Function တွေက NodeJS ရဲ့ Build-in Debug လုပ်ဆောင်ချက်နဲ့ သိပ်မကွာပေမယ့် Command Line မဟုတ်ပဲ UI နဲ့ ဖြစ်တဲ့အတွက် အသုံးပြုရ ပိုမိုလွယ်ကူပါတယ်။

10.5 – NodeJS Module

NodeJS ဆိုတာ JavaScript ပဲမို့ JavaScript ရဲ့ Language Feature တွေကိုတော့ ထပ်ပြောမနေတော့ပါဘူး။ **အခန်း (၃)** မှာ JavaScript OOP ရဲ့ ထူးခြားချက်တွေကိုဖော်ပြခဲ့ပြီး ဖြစ်ပါတယ်။ ပိုပြီးအခြေခံကျတဲ့ Language Feature တွေကို **Professional Web Developer** ရဲ့ **အခန်း (၄)** မှာဖော်ပြထားပါတယ်။ ဒီနေရာမှာ NodeJS က ဖြည့်စွက်ပေးထားတဲ့ ထူးခြားချက်တွေကို အဓိကထားဖော်ပြသွားမှာဖြစ်ပါတယ်။ NodeJS က ဖြည့်စွက်ပေးထားတဲ့ ထူးခြားချက်တွေထဲမှာ အဓိကကျတဲ့တစ်ချက်ကတော့ Module ဖြစ်ပါတယ်။

အခြေခံအားဖြင့် JavaScript Code ဖိုင်တစ်ခုကနေ အခြား JavaScript Code ဖိုင်တစ်ခုကို ချိတ်ဆက်အသုံးပြုလို့မရပါ ဘူး။ ဒါကြောင့် Client-side JavaScript မှာ ဆိုရင် JavaScript Module တွေကို စီမံဖို့အတွက် HTML `<script>` Element ကိုပဲ အားကိုးရပါတယ်။ ဥပမာ – JavaScript ဖိုင် (၃) ခုချိတ်ဆက်အသုံးပြုချင်တဲ့အခါ JavaScript ထဲမှာ ကြေငြာချိတ်ဆက်လို့မရပဲ HTML Document တစ်ခုအတွင်းမှာ အခုလိုချိတ်ဆက်ပေးရပါတယ်။

HTML

```
<script src="circle.js"></script>
<script src="square.js"></script>
<script src="area.js"></script>
```

ဒီလိုအပ်ချက်ကို ဖြည့်စွက်ဖို့အတွက် CommonJS, RequireJS စတဲ့ နည်းပညာများကို တီထွင်ဖန်တီးပြီး အသုံးပြုခဲ့ကြ ရပါတယ်။ NodeJS မှာတော့ JavaScript Code ထဲမှာ အခြား JavaScript Module ဖိုင်တွေကို ချိတ်ဆက်အသုံးပြုနိုင် အောင် စီစဉ်ပေးထားပါတယ်။ `require()` Function သုံးပြီး ချိတ်ဆက်နိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
var http = require("http");
var express = require("express");
var measure = require("./circle.js");
```

`require()` Function နဲ့ NodeJS ရဲ့ Build-in Module တွေကို Module အမည်သုံးပြီး တိုက်ရိုက်ချိတ်ဆက်နိုင်ပါ တယ်။ NPM နဲ့ Install လုပ်ထားတဲ့ Package တွေကိုလည်း ချိတ်ဆက်နိုင်ပါတယ်။ ကိုယ်တိုင်ရေးသားထားတဲ့ JavaScript ဖိုင်တွေကိုလည်း ဖိုင်တည်နေရာ Path ပေးပြီး ချိတ်ဆက်နိုင်ပါတယ်။ NodeJS Module တွေကို အခုလို ရေးသားရပါတယ်။

JavaScript

```
exports.circle = function (r) {
    var pi = 3.14;
    return pi * r * r;
}

exports.square = function (w, h) {
    return w * h;
}
```

(သို့မဟုတ်)

JavaScript

```
module.exports = {
    circle: function (r) {
        var pi = 3.14;
        return pi * r * r;
    },

    square: function (w, h) {
        return w * h;
    }
}
```

`exports` Object မှာ Function တွေ Property တွေကို တစ်ခုပြီးတစ်ခု တွဲဖက်ပေးနိုင်သလို၊ `module.exports` Object မှာ Function တွေ Property တွေကို အတွဲလိုက်လည်း သတ်မှတ်ပေးနိုင်ခြင်းဖြစ်ပါတယ်။ `require()` Function ကို အခုလိုရေးသားထားတယ်လို့ မှတ်ယူနိုင်ပါတယ်။

Pseudo-code

```
var require = function(path) {
    // load module or package or path
    return module.exports;
}
```

`require()` Function က ချိတ်ဆက်အသုံးပြုလိုတဲ့ Module (သို့မဟုတ်) Package (သို့မဟုတ်) ဖိုင်ကိုရယူပြီး အဲ့ဒီ ဖိုင်ထဲက `module.exports` Object ကို `return` ပြန်ပေးခြင်းဖြစ်ပါတယ်။ ဒါကြောင့် `require()` Function က ပြန်ပေးတဲ့တန်ဖိုးကို Variable တစ်ခုနဲ့ လက်ခံပေးဖို့လိုအပ်ပါတယ်။ ဥပမာ –

JavaScript

```
var area = require("./area.js");
console.log( area.circle(8) );          // => 200.96
console.log( area.square(2, 4) );       // => 8
```

နမူနာမှာ `area.js` Module ကို `area` Variable နဲ့ လက်ခံချိတ်ဆက်ထားပါတယ်။ ဒါကြောင့် `area.js` Module ရဲ့ လုပ်ဆောင်ချက်တွေကို အသုံးပြုလိုတဲ့အခါ `area` ကနေတစ်ဆင့် အသုံးပြုနိုင်ခြင်းဖြစ်ပါတယ်။

10.6 – NodeJS Build-in Modules

NodeJS မှာ Command Line ပရိုဂရမ်တွေ၊ Back-end Service တွေနဲ့ Network ပရိုဂရမ်တွေ ဖန်တီးဖို့အတွက် လိုအပ်တဲ့ အခြေခံ Module တွေ အသင့်ပါဝင်ပါတယ်။ ဒီနေရာမှာ Module အားလုံးကိုတော့ မဖော်ပြနိုင်ပါဘူး။ အဓိက (၂) ပိုင်းကိုရွေးထုတ်ဖော်ပြလိုပါတယ်။ ဖိုင်တွေစီမံနိုင်တဲ့ File System Module နဲ့ Web Server တစ်ခု အနေနဲ့ အလုပ် လုပ်ပေးနိုင်တဲ့ HTTP Module တို့ပဲဖြစ်ပါတယ်။

File System Module

File System Module ကို အသုံးပြုနိုင်ဖို့အတွက် `require()` Function နဲ့ အခုလိုချိတ်ဆက်ပေးဖို့လိုပါတယ်။

JavaScript

```
var fs = require("fs");
```

File System Object ဖြစ်တဲ့ `fs` ကိုရရှိပြီးနောက် အဲ့ဒီ Object ကနေတစ်ဆင့် File System Module ရဲ့ လုပ်ဆောင် ချက်တွေကို စတင်အသုံးပြုနိုင်ပြီဖြစ်ပါတယ်။ အသုံးဝင်တဲ့ Function စာရင်းကို ဖော်ပြပေးလိုက်ပါတယ်။

`fs.exists()` – ဖိုင်ရှိမရှိစီစစ်လိုတဲ့အခါ သုံးရပါတယ်။ ရလဒ်ကို Callback အနေနဲ့ ပြန်ပေးပါတယ်။

JavaScript

```
fs.exists("path/to/file", function(result) {
    if(result) console.log("File exists!");
});
```

`fs.rename()` – ဖိုင်အမည်ပြောင်းလိုတဲ့အခါသုံးပါတယ်။ Parameter အနေနဲ့ လက်ရှိအမည်၊ ပြောင်းလိုတဲ့ အမည်နဲ့ Callback တို့ကို ပေးရပါတယ်။ ဥပမာ –

JavaScript

```
fs.rename("old.csv", "new.csv", function() {
    console.log("Done renaming file.");
});
```

`fs.unlink()` – ဖိုင်ကိုဖျက်လိုတဲ့အခါသုံးပါတယ်။ ဖျက်လိုတဲ့ဖိုင်နဲ့ Callback Function တို့ကို Parameter အနေနဲ့ ပေးရပါတယ်။

`fs.readdir()` – Directory တစ်ခုမှာပါဝင်တဲ့ ဖိုင်နဲ့ Sub-directory စာရင်းကိုရယူလိုတဲ့အခါ သုံးပါတယ်။ ဖိုင် စာရင်း Array ကို Callback Parameter အနေနဲ့ ပြန်ပေးပါတယ်။ ဥပမာ –

JavaScript

```
fs.readdir("path/to/dir", function(err, files) {
    console.log(files);
});
```

`fs.mkdir()` – Directory တစ်ခုတည်ဆောက်လိုတဲ့အခါသုံးပါတယ်။ တည်ဆောက်လိုတဲ့ Directory အမည်နဲ့ Callback Function တို့ကို Parameter အနေနဲ့ ပေးရပါတယ်။

`fs.rmdir()` – Directory တစ်ခုကို ပယ်ဖျက်လိုတဲ့အခါသုံးပါတယ်။ ပယ်ဖျက်လိုတဲ့ Directory အမည်နဲ့ Callback Function တို့ကို Parameter အနေနဲ့ ပေးရပါတယ်။

`fs.readFile()` – ဖိုင်တစ်ခုမှာပါဝင်တဲ့ Content ကို ဖတ်ယူလိုတဲ့အခါသုံးပါတယ်။ Content ကို Callback Parameter အနေနဲ့ ပြန်ပေးပါတယ်။ ဥပမာ –

JavaScript

```
fs.readFile("users.csv", "utf8", function(err, data) {
    console.log(data);
});
```

ဒုတိယ Parameter အနေနဲ့ Encoding အမျိုးအစားကိုပေးထားတာကို သတိပြုပါ။ နမူနာမှာ `utf8` လို့ပေးထားပါတယ်။ Read လုပ်လိုတဲ့ဖိုင်မှာသုံးထားတဲ့ Encoding က အခြား Encoding ဆိုရင်လည်း မှန်အောင် သတ်မှတ်ပေးဖို့လိုပါတယ်။

`fs.writeFile()` – ဖိုင်တစ်ခုတည်ဆောက်လိုတဲ့အခါ (သို့မဟုတ်) ဖိုင်တစ်ခုအတွင်းမှာ Content တွေ ရေးသားစေ လိုတဲ့အခါသုံးပါတယ်။ ဖိုင်အမည်၊ ရေးသားရမယ့် Content နဲ့ Callback Function တို့ကိုပေးရပါမယ်။ ဥပမာ –

JavaScript

```
fs.writeFile("fruites.csv", "This is file content.", function() {
    console.log("Done saving data to file.");
});
```

`fs.appendFile()` – လက်ရှိဖိုင်အတွင်းမှာ Content တွေ ဖြည့်စွက်လိုတဲ့အခါ သုံးပါတယ်။ `writeFile()` မှာလိုပဲ ဖိုင်အမည်၊ ရေးသားရမယ့် Content နဲ့ Callback Function တို့ကိုပေးရပါတယ်။ `writeFile()` က မူလ Content ရှိနေရင် အစားထိုးသွားမှာဖြစ်ပြီး `appendFile()` ကတော့ အစားမထိုးပဲ၊ ဖြည့်စွက်ပေးသွားမှာဖြစ်ပါတယ်။

အကြောင်းအမျိုးမျိုးကြောင့် File System Function တွေကို ရိုးရိုး Synchronous ပုံစံအလုပ်လုပ်စေလိုရင် `fs.renameSync()`, `fs.chmodSync()`, `fs.unlinkSync()` စတဲ့ နောက်ကနေ Sync တွဲဖက်အမည်ပေးထားတဲ့ Function တွေကို အစားထိုးသုံးနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
var data = fs.readFileSync("users.csv");
console.log(data);
```

File System စီမံရာမှာ အသုံးဝင်တဲ့ နောက်ထပ် Module တစ်ခုကတော့ Path Module ဖြစ်ပါတယ်။ အသုံးပြုနိုင်ဖို့ အတွက် `require()` နဲ့ အရင်ချိတ်ဆက်ပေးဖို့လိုပါတယ်။

JavaScript

```
var path = require("path");
```

Path Module ထဲက အဓိကကျတဲ့ Function တွေကိုဖော်ပြလိုက်ပါတယ်။

`path.dirname()` – ပေးလိုက်တဲ့ Path ထဲက Directory အမည်ကိုရယူပေးပါတယ်။ ဥပမာ –

JavaScript

```
path.dirname("/path/to/file.csv");          // => /path/to
```

`path.basename()` – ပေးလိုက်တဲ့ Path ထဲက ဖိုင်အမည်ကိုရယူပေးပါတယ်။ ဥပမာ –

JavaScript

```
path.basename("/path/to/file.csv");          // => file.csv
path.basename("/path/to/file.csv", ".csv");  // => file
```

`path.extname()` – ပေးလိုက်တဲ့ Path ထဲက ဖိုင် Extension ကို ရယူပေးပါတယ်။ ဥပမာ –

JavaScript

```
path.extname("/path/to/file.csv");          // => .csv
path.extname("/path/to/file.csv.gz");       // => .gz
```

path.parse() – ပေးလိုက်တဲ့ Path ကို Object အဖြစ်ပြောင်းပေးပါတယ်။ ဥပမာ –

JavaScript

```
path.parse("/path/to/file.csv");

{
    root: "/",
    dir: "/path/to",
    base: "file.csv",
    ext: ".csv",
    name: "file"
}
```

path.format() – ပေးလိုက်တဲ့ Object ကို Path ပြန်ပြောင်းပေးပါတယ်။ ဥပမာ –

JavaScript

```
path.format({
    root: "/",
    dir: "/path/to",
    base: "file.csv",
    ext: ".csv",
    name: "file"
});

// => /path/to/file.csv
```

အသုံးများနိုင်တဲ့ Function တွေကို ရွေးထုတ်ဖော်ပြခဲ့ခြင်းဖြစ်ပြီး အောက်ပါလိပ်စာများမှာ File System Module နဲ့ Path Module တို့မှာပါဝင်တဲ့ လုပ်ဆောင်ချက် အပြည့်အစုံကို လေ့လာနိုင်ပါတယ်။

https://nodejs.org/api/fs.html
https://nodejs.org/api/path.html

HTTP Module

NodeJS ရဲ့ HTTP Module မှာ HTTP Server နဲ့ပက်သက်တဲ့ လုပ်ဆောင်ချက်တွေရာ HTTP Client နဲ့ပက်သက်တဲ့လုပ်ဆောင်ချက်တွေပါ ပါဝင်ပါတယ်။ တစ်နည်းအားဖြင့် NodeJS နဲ့ HTTP Web Server တစ်ခုတည်ဆောက်နိုင်သလို HTTP Web Client တစ်ခုလည်း တည်ဆောက်နိုင်ပါတယ်။ HTTP Server တစ်ခုကို အခုလို တည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
1. var http = require("http");
2.
3. http.createServer(function(req, res) {
4.     res.writeHead(200, { "Content-Type": "text/html" });
5.     res.write("Hello, World!");
6.     res.end();
7. }).listen(8080);
```

HTTP Server တစ်ခုတည်ဆောက်ဖို့အတွက် `http.createServer()` Function ကိုသုံးရပါတယ်။ လက်ခံရရှိတဲ့ Request နဲ့ပက်သက်တဲ့အချက်အလက်တွေကို `createServer()` Callback Function ရဲ့ပထမ Parameter နဲ့ လက်ခံထားပေးမှာဖြစ်ပြီး၊ Response အဖြစ်ဆောင်ရွက်ရမယ့် လုပ်ငန်းတွေကိုတော့ ဒုတိယ Parameter နဲ့ သတ်မှတ်နိုင် ပါတယ်။ နမူနာအရ Request နဲ့ပက်သက်တဲ့အချက်အလက်တွေဟာ `req` Object ထဲမှာရှိနေမှာဖြစ်ပြီး Response ကို စီမံဖို့အတွက် `res` Object ကို အသုံးပြုရမှာ ဖြစ်ပါတယ်။

HTTP Response တစ်ခုမှာ Response Header နဲ့ Response Body ဆိုပြီးနှစ်ပိုင်းရှိရမှာဖြစ်တဲ့အတွက် လိုင်းနံပါတ် (၄) မှာ `res.writeHead()` Function နဲ့ Response Header ကို သတ်မှတ်ပေးထားပါတယ်။ အဲ့ဒီလို သတ်မှတ် ရာမှာ ပထမ Parameter အနေနဲ့ Status Code ကိုပေးရပြီး၊ ဒုတိယ Parameter အနေနဲ့ Header Option တွေပေး ရပါမယ်။ နမူနာမှာ Status Code အနေနဲ့ `200 OK` ကို သတ်မှတ်ထားပြီး Response Header Option အနေနဲ့ `content-type: text/html` လို့ သတ်မှတ်ထားပါတယ်။ ပုံမှန်အားဖြင့် Response Header တစ်ခုမှာ Status Code, HTTP Version, Content Type, Content Size, Encoding, Modified Date စ သဖြင့် ပါဝင်သင့်တဲ့အချက် တွေအများကြီးရှိပါတယ်။ နမူနာမှာ Content Type တစ်မျိုးတည်းကိုသာ သတ်မှတ်ထားပေမယ့် လက်တွေ့မှာ Option တွေ အားလုံးကို ထည့်သွင်းပေးသင့်ပါတယ်။

Response Header သတ်မှတ်ပြီးတဲ့အခါမှာ Response Body ကို `res.write()` Function နဲ့သတ်မှတ်ပေးရပါ တယ်။ နမူနာအရ Request တစ်ခုဝင်လာတိုင်းမှာ ကျွန်တော်တို့ရဲ့ HTTP Server က Hello, World! ဆိုတဲ့ Response Body ကို အမြဲပြန်ပေးသွားမှာဖြစ်ပါတယ်။ Response Header နဲ့ Body သတ်မှတ်ပြီးလို့ ပြည့်စုံပြီဆိုရင် `res.end()` Function ကို Run ပေးရပါတယ်။ ဒီတော့မှ Response လုပ်ငန်းပြီးပြတ်သွားပြီဖြစ်ကြောင်း Server ရော Request ပြုလုပ်သူ Client ပါ သိရှိနိုင်မှာဖြစ်ပါတယ်။

Server ကို စတင်အလုပ်လုပ်စေဖို့ `http.listen()` Function ကိုသုံးရပါတယ်။ `listen()` Function နဲ့ အတူ Server အလုပ်လုပ်ရမယ့် Port နံပါတ်ကို တွဲဖက်ပေးရပါတယ်။ နမူနာမှာ `8080` လို့ ပေးထားတဲ့အတွက် အခုနေ ဒီ HTTP Server Code ကို node Command နဲ့ Run လိုက်ရင် HTTP Server တစ်ခုက Port နံပါတ် `8080` မှာ စတင် အလုပ်လုပ် သွားမှာ ဖြစ်ပါတယ်။ Web Browser ကိုဖွင့်ပြီး localhost:8080 ကို Request လုပ်ကြည့်ရင် Hello, World! ဆိုတဲ့ Response Body ကို လက်ခံရရှိမှာပဲဖြစ်ပါတယ်။

NodeJS HTTP Module ဟာ Low Level Module တစ်ခုဖြစ်ပါတယ်။ ဆိုလိုတာက၊ သူက အခြေခံလုပ်ဆောင်ချက်တွေကိုသာ ပေးထားပြီး၊ Request တစ်ခုလက်ခံရရှိတဲ့အခါ ဆောင်ရွက်သွားစေလိုတဲ့ အသေးစိတ် အလုပ်တွေအားလုံးကို ကိုယ်တိုင်ရေးသား သတ်မှတ်ပေးရမှာဖြစ်ပါတယ်။ HTTP Protocol

အကြောင်းကို သိရှိထားမှသာ NodeJS HTTP Module ကို အသုံးချနိုင်မှာဖြစ်ပါတယ်။ HTTP အကြောင်းကို **Professional Web Developer** ရဲ့ **အခန်း (၁)** မှာ ဖော်ပြထားပါတယ်။

Request နဲ့အတူ လက်ခံရရှိတဲ့ URI Path ပေါ်မူတည်ပြီး HTML Document တွေကို Response ပြန်ပေးနိုင်တဲ့ HTTP Server တစ်ခုကို အခုလိုတည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
var http = require("http");
var fs = require("fs");
var docroot = "/path/to/docroot";

http.createServer(function(req, res) {

    var path = docroot + req.url;
    var result = fs.existsSync(path);

    if (result) {
        var body = fs.readFileSync(path, "utf8");
        res.writeHead(200, { "Content-Type": "text/html" });
        res.write(body);
    } else {
        res.writeHead(404);
        res.write("404 Not found");
    };

    res.end();
}).listen(8080);

console.log("HTTP server running at port 8080...");
```

လိုင်းနံပါတ် (၇) မှာ Request URL နဲ့ Document Root Directory ကိုပေါင်းစပ်ပြီး ဖိုင် Path ကို တည်ဆောက်ထား ပါတယ်။ လိုင်းနံပါတ် (၈) မှာတော့ `fs.existsSync()` ကိုသုံးပြီး ဖိုင် ရှိမရှိ စီစစ်ထားပါတယ်။ ဖိုင်ရှိတယ်ဆိုရင် Response Body အဖြစ် ဖိုင် Content ကို ပြန်ပေးပြီး၊ ဖိုင်မရှိရင်တော့ 404 Not found ကို ပြန်ပေးထားပါတယ်။ ဒီနည်းနဲ့ Request URL ပေါ် မူတည်ပြီး ကိုက်ညီတဲ့ HTML Document ကိုပြန်ပေးနိုင်တဲ့ Web Server တစ်ခုကိုရရှိ နိုင်ပါတယ်။ တစ်ကြောင်းချင်း ရှင်းမနေတော့ပါဘူး။ ရေးသားထားတဲ့ Code ကိုသာ လေ့လာကြည့်လိုက်ပါ။

လက်တွေ့ပြည့်စုံဖို့ဆိုရင် ဖိုင် Permission စီစစ်တဲ့ကိစ္စတွေ၊ Directory Index သတ်မှတ်တဲ့ကိစ္စတွေနဲ့ အခြားအထွေထွေ လိုအပ်ချက်တွေရှိသေးတဲ့အတွက် ကိုယ်တိုင်ရေးသားမယ့်အစား အသင့်ရှိနေတဲ့ Package တွေကို အသုံးပြုသင့်ပါတယ်။ Connect နဲ့ Serve Static ဆိုတဲ့ Package တွေကိုအသုံးပြုပြီး Static Web Server တစ်ခုကို ရရှိနိုင်ပါတယ်။ NPM နဲ့ အခုလို Install လုပ်ယူနိုင်ပါတယ်။

```
$ sudo npm install -g connect serve-static
```

Install လုပ်ပြီးပြီဆိုရင် Static Web Server တစ်ခုကို အလွယ်တစ်ကူ ရေးသားတည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
var connect = require('connect');
var serveStatic = require('serve-static');
connect().use(serveStatic("/path/to/docroot")).listen(8080);
```

HTTP Module ကို တိုက်ရိုက်မသုံးတော့ပဲ Connect Package နဲ့ HTTP Server တစ်ခုတည်ဆောက်လိုက်ခြင်းဖြစ်ပါတယ်။ နမူနာအရ `serveStacic()` Function အတွက် Parameter အဖြစ်ပေးလိုက်တဲ့ Directory ကို Document Root အဖြစ် NodeJS Web Server က သုံးပေးသွားမှာဖြစ်ပါတယ်။

NodeJS နဲ့ HTTP Server တစ်ခုကို **Request** ပြုလုပ်လိုတဲ့အခါ `http.get()` ကို သုံးနိုင်ပါတယ်။ Request ပြုလုပ် လိုတဲ့ URL နဲ့ Callback Function တို့ကိုပေးရပါတယ်။ Server က ပြန်ပေးလာတဲ့ Response ကို Callback Parameter အဖြစ် လက်ခံနိုင်ပါတယ်။ jQuery ရဲ့ `$.get()` Function နဲ့ သဘောတရား ဆင်တူပါတယ်။ ဥပမာ –

JavaScript

```
1. http.get("http://www.example.com/", function(res) {
2.     res.setEncoding("utf8");
3.     res.on("data", function (chunk) {
4.         console.log(chunk);
5.     });
6. });
```

Server ပြန်ပေးလာတဲ့ Response ကို `res` Variable နဲ့ Callback Function မှာ လက်ခံထားပါတယ်။ Response Header ကို လိုချင်ရင် `res.headers` Property ကနေ ရယူနိုင်ပါတယ်။ Response Status Code ကိုရယူလိုရင် တော့ `res.statusCode()` Function ကနေ ရယူနိုင်ပါတယ်။ Response Body ကိုတော့ နမူနာလိုင်းနံပါတ် (၃) မှာ ပေးထားသလို `res.on('data')` Event ရဲ့ Callback Function ကနေ ရယူနိုင်ပါတယ်။ သတိပြုရမှာက `http.get()` Function ဟာ Request Method အနေနဲ့ GET ကို အမြဲအသုံးပြုပေးသွားမှာဖြစ်ပါတယ်။ အခြား Request Method တွေနဲ့ Request ပြုလုပ်လိုတယ်ဆိုရင်တော့ `http.request()` ကို အသုံးပြုနိုင်ပါတယ်။

JavaScript

```
1. var options = {
2.     hostname: "www.example.com",
3.     port: 80,
4.     path: "/api",
5.     method: "POST"
6. };
7.
```

```
 8. var req = http.request(options, function(res) {
 9.     res.setEncoding("utf8");
10.     res.on("data", function (chunk) {
11.         console.log(chunk);
12.     });
13. });
14.
15. req.on("error", function(e) {
16.     console.log(e.message);
17. });
18.
19. req.write("foo=bar");
20. req.end();
```

`http.request()` ကိုအသုံးပြုဖို့အတွက် Request Option နဲ့ Callback Function ကိုပေးရပါတယ်။ Option အနေနဲ့ Request Header မှာပါဝင်ရမယ့် အချက်အလက်တွေကို ပေးရပါတယ်။ နမူနာမှာ `hostname`, `port`, `path`, `method` စတဲ့ Request Header Option တွေကို သတ်မှတ်ပေးထားတာကို တွေ့ရနိုင်ပါတယ်။ Request နဲ့အတူ တွဲဖက်ပေးပို့လိုတဲ့ Query String ရှိခဲ့ရင် နမူနာလိုင်းနံပါတ် (၁၉) မှာဖော်ပြထားသလို `req.write()` Function ကို အသုံးပြုနိုင်ပါတယ်။ ထူးခြားချက်အနေနဲ့ Request ပြီးစီးတိုင်းမှာ `req.end()` Function ကို Run ပေးရပါတယ်။ ဒီတော့မှ ဒီ Request ကို လက်ခံမယ့် Server က ဒါဟာ Request ရဲ့ အဆုံးသတ်ဖြစ်ကြောင်းကို သိရှိနိုင် မှာဖြစ်ပါတယ်။ `http.get()` Function ကတော့ `req.end()` ကို အလိုအလျောက်ထည့်ပေးတဲ့အတွက် သီးခြား ထပ် Run စရာမလိုပါဘူး။ `http.request()` သုံးရင်တော့ `req.end()` ကို ထည့်သွင်းရေးသားပေးရပါတယ်။

HTTP Module နဲ့အတူတွဲဖက်အသုံးပြုဖို့ အသုံးဝင်တာကတော့ URL Module နဲ့ Query String Module တို့ပဲဖြစ်ပါတယ်။ URL Module ရဲ့ `parse()` နဲ့ `format()` Function နှစ်ခုက အသုံးဝင်ပါတယ်။

`parse()` Function က ပေးလိုက်တဲ့ URL ကို Object ပြောင်းပေးပါတယ်။ ဥပမာ –

JavaScript

```
var url = require("url");
url.parse("http://example.com:8080/foo/bar.html");

{ protocol: 'http:',
  slashes: true,
  auth: null,
  host: 'example.com:8080',
  port: '8080',
  hostname: 'example.com',
  hash: '#two',
  search: '?q=keywords',
  query: 'q=keywords',
  pathname: '/foo/bar.html',
  path: '/foo/bar.html?q=keywords',
  href: 'http://example.com:8080/foo/bar.html?q=keywords#two' }
```

`format()` Function ကတော့ ပေးလိုက်တဲ့ Object ကို URL ပြန်ပြောင်းပေးပါတယ်။ ဥပမာ –

JavaScript

```
url.format({
  protocol: 'http',
  hostname:'example.com',
  port: 8080,
  pathname: '/foo/bar.html'
});

'http://example.com:8080/foo/bar.html'
```

Query String Module ကတော့ မူလကတည်းက Global Scope မှာရှိနေပြီးသားမို့ `require()` တောင်လုပ်စရာ မလိုပဲ `querystring` Object အနေနဲ့ တန်းသုံးလို့ရပါတယ်။ `querystring.stringify()` Function က Object တွေကို Query String ပြောင်းပေးပါတယ်။ ဥပမာ –

JavaScript

```
querystring.stringify({ foo: 'bar', baz: ['qux', 'quux'] });

'foo=bar&baz=qux&baz=quux'
```

`querystring.parse()` Function ကတော့ Query String တစ်ခုကို Object ပြောင်းပေးပါတယ်။ ဥပမာ –

JavaScript

```
querystring.parse('foo=bar&baz=qux&baz=quux')

{ foo: 'bar', baz: ['qux', 'quux'] }
```

Other Build-in Modules

ဖြည့်စွက်မှတ်သားသင့်တဲ့ NodeJS Build-in Module တစ်ချို့ကို ရွေးထုတ်ဖော်ပြပေးလိုက်ပါတယ်။

Crypto – Hash, Encryption, Description စတဲ့လုပ်ငန်းများအတွက်လိုအပ်တဲ့အခြေခံလုပ်ဆောင်ချက်တွေပါဝင်တဲ့ Module တစ်ခုဖြစ်ပါတယ်။ အသုံးများတဲ့ Hash နဲ့ Encryption Algorithm တွေအားလုံး အသင့်ပါဝင်ပါတယ်။ ဥပမာ – String တစ်ခုရဲ့ MD5 (သို့မဟုတ်) SHA1 Hash Summary ကို အခုလိုရယူနိုင်ပါတယ်။

JavaScript

```
var crypto = require('crypto');

var md5 = crypto.createHash('md5').update('Apple').digest("hex");
// 9f6290f4436e5a2351f12e03b6433c3c

var sha1 = crypto.createHash('sha1').update('Apple').digest("hex");
// 476432a3e85a0aa21c23f5abd2975a89b6820d63
```

Apple နေရာမှာ Hash Summary ရရှိလိုတဲ့ Text နဲ့အစားထိုးပေးနိုင်ပါတယ်။

DNS – OS ရဲ့ Domain Name Resolution စနစ်ကို အသုံးပြုပေးတဲ့ Module ဖြစ်ပါတယ်။ Domain Name တစ်ခုရဲ့ IP address တွေကို ရယူဖို့အတွက် အသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
var dns = require('dns');

dns.lookup('www.example.com', function (err, addresses) {
    console.log('addresses: ' + addresses);
});

// addresses: 93.184.216.34
```

Net – Client-server Model နဲ့အလုပ်လုပ်တဲ့ Network ပရိုဂရမ်တွေတည်ဆောက်ရာမှာ အသုံးပြုနိုင်တဲ့ Module ဖြစ် ပါတယ်။ Net Module ကိုသုံးပြီး Server ပရိုဂရမ်တစ်ခုကို အခုလိုတည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
1. var net = require('net');
2.
3. var server = net.createServer(function(conn) {
4.     console.log('Client connected');
5.
6.     conn.on('end', function() {
7.         console.log('Client disconnected');
8.     });
9.
10.    conn.write('Hello!\n');
11.    conn.pipe(conn);
12. });
13.
14. server.listen(8124, function() {
15.     console.log('Server started');
16. });
```

HTTP Module မှာလိုပဲ `createServer()` ကိုသုံးပြီး Server တစ်ခုတည်ဆောက်နိုင်ခြင်းဖြစ်ပါတယ်။ Client တွေ ချိတ်ဆက်လာတဲ့အခါ ဆောင်ရွက်ရမယ့်လုပ်ငန်းတွေကို Callback Function အနေနဲ့ သတ်မှတ်ထားနိုင်ပါ

တယ်။ Client တွေ ပေးပို့လိုတဲ့အချက်အလက်တွေကို `write()` Function နဲ့ပေးပို့နိုင်ပါတယ်။ လိုင်းနံပါတ် (၁၁) မှာ `conn.pipe(conn)` လို့ရေးသားထားတဲ့အတွက် Client ပေးပို့လာတဲ့ အချက်အလက်တွေကို Client ထံ ပြန်လည် ပေးပို့လိုက်မှာ ဖြစ်ပါတယ်။ ဒါကြောင့် Client က Server ကိုချိတ်ဆက်ပြီး တစ်ခုခုပေးပို့ရင်၊ သူပေးပို့လိုက်တဲ့ အချက် အလက်ကို သူပြန်ရမှာဖြစ်ပါတယ်။ HTTP Module မှာလိုပဲ Server ကိုစတင်ဖို့အတွက် `listen()` Function နဲ့အတူ Port နံပါတ်ကို သတ်မှတ်ပေးနိုင်ပါတယ်။

Net Module ကိုသုံးပြီး Client ပရိုဂရမ်တစ်ခုကို အခုလိုရေးသားနိုင်ပါတယ်။

JavaScript

```
var net = require('net');

var client = net.connect({ port: 8124 }, function() {
    console.log('connected to server!');
    client.write('Hi!\n');
});

client.on('data', function(data) {
    console.log(data.toString());
    client.end();
});

client.on('end', function() {
    console.log('Disconnected from server');
});
```

`connect()` Function ကိုသုံးပြီး Server ပရိုဂရမ်ကို ချိတ်ဆက်ရခြင်းဖြစ်ပါတယ်။ Connect လုပ်တဲ့အခါ အသုံးပြုရ မယ့် Port နံပါတ်ကို Option အနေနဲ့ တွဲဖက်ထည့်သွင်းပေးရပါတယ်။ Server ထံပေးပို့လိုတဲ့ အချက်အလက်တွေကို `write()` Function နဲ့ ပေးပို့နိုင်ပါတယ်။ နမူနာလိုင်းနံပါတ် (၈) မှာ `data` Event နဲ့ Server ပြန်လည်ပေးပို့လာတဲ့ အချက်အလက်တွေကို လက်ခံရရှိတဲ့အခါ ဆောင်ရွက်ရမယ့် Function ကို သတ်မှတ်ထားပါတယ်။

HTTP ဟာ Pull Protocol တစ်ခုဖြစ်ပါတယ်။ ဒါကြောင့် HTTP Client နဲ့ HTTP Server တို့ဆက်သွယ်တဲ့အခါ Client က Request ပြုလုပ်ခြင်းအားဖြင့် ဆက်သွယ်မှုကို အစပြုရပါတယ်။ Server က သူ့သဘောနဲ့သူ အချက်အလက်ပေးပို့ တယ်ဆိုတာ မရှိပါဘူး။ Client Request ရှိမှသာ Server Response ရှိမှာပါ။ Net Module နဲ့တည်ဆောက်ထားတဲ့ Network ပရိုဂရမ်တွေမှာတော့ Client နဲ့ Server ချိတ်ဆက်ပြီးတဲ့နောက်မှာ၊ Client ကရော Server ကပါ အချက် အလက်တွေကို Full Duplex ပုံစံ အပြန်အလှန် ပေးပို့နိုင်မှာဖြစ်ပါတယ်။

OS – CPU Usage, Memory Usage, Server Uptime, Hostname စတဲ့ Operating System နဲ့ပက်သက်တဲ့ အချက်အလက်တွေကို သိရှိလိုရင် OS Module ကို အသုံးပြုရပါတယ်။

Punycode – Unicode Character တွေကို ASCII Character ပြောင်းဖို့သုံးပါတယ်။ Punycode နဲ့ ASCII ပြောင်းထားတဲ့ Character တွေကို Unicode လည်း ပြန်ပြောင်းနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
var punycode = require("punycode");
punycode.encode("ကာ");          // => nid
punycode.decode("nid");         // => ကာ
```

Util – `isArray()`, `isRegExp()`, `isDate()` စတဲ့ Validation Function တစ်ချို့ပါဝင်တဲ့ Module တစ်ခု ဖြစ်ပါတယ်။ အဓိကအသုံးဝင်မှာကတော့ `format()` Function နဲ့ `inherits()` Function တို့ဖြစ်ပါတယ်။ `format()` Function ကို String Format လုပ်ဖို့အတွက် အသုံးပြုနိုင်ပါ တယ်။ ဥပမာ –

JavaScript

```
var util = require('util');
var id = 1, err = "Error", date = "2015-04-15";
util.format("ID: %d, Msg: %s, Date: %s", id, err, date);

// => ID: 1, Msg: Error, Date: 2015-04-15
```

String ထဲမှာ Placeholder တွေထည့်သွင်းပြီး အစားထိုးအသုံးပြုရမယ့် တန်ဖိုးတွေကို `format()` Function နဲ့ တွဲဖက်ပေးနိုင်ခြင်းဖြစ်ပါတယ်။ ကိန်းဂဏန်းတွေအတွက် `%d` Placeholder ကိုသုံးရပြီး Text တွေ အတွက် `%s` Place-holder ကို သုံးရပါတယ်။ ဒီနည်းနဲ့ Variable တွေကို String နဲ့ တွဲဖော်ပြဖို့အတွက် + Operator တွေသုံးပြီး တွဲဆက် နေစရာမလိုတော့ပဲ ပိုပြီးစနစ်ကျသွားမှာဖြစ်ပါတယ်။ `console.log()` Function မှာလည်း `format()` လိုပဲ Placeholder တွေ ထည့်သွင်း အသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
console.log("ID: %d, Msg: %s, Date: %s", id, err, date);

// => ID: 1, Msg: Error, Date: 2015-04-15
```

`inheirts()` Function ကိုတော့ Object Constructor တွေ Inheritance လုပ်ဖို့အတွက် အသုံးပြုနိုင်ပါတယ်။

JavaScript

```
var Person = function() {};
var User = function() {};
uril.inherits(User, Person);
```

နမူနာအရ User Constructor ဟာ Person Constructor ကို Inherit လုပ်သွားမှာဖြစ်ပါတယ်။

Zlib – အချက်အလက်တွေကို Compress/Decompress လုပ်ဖို့သုံးပါတယ်။ Compress လုပ်ဖို့အတွက် Gzip, Deflate စတဲ့ Method တွေကိုအသုံးပြုနိုင်ပါတယ်။ Deflate Method ကိုသုံးပြီး Content တွေကို အခုလို Compress လုပ်နိုင်ပါတယ်။

JavaScript

```
var zlib = require("zlib");
var input = "Hello, World!";

zlib.deflate(input, function(err, buffer) {
    console.log( buffer.toString('base64') );
});

// eJzzSM3JyddRCM8vyklRBAAfngRq
```

Deflate အစား Gzip ကိုသုံးလိုရင် `zlib.gzip()` ကို အစားထိုးသုံးနိုင်ပါတယ်။ Compress လုပ်ထားတဲ့ Content တွေကို Decompress လုပ်လိုရင်တော့ `zlib.unzip()` ကိုသုံးနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
var buffer = new Buffer("eJzzSM3JyddRCM8vyklRBAAfngRq", "base64");

zlib.unzip(buffer, function(err, buffer) {
    console.log(buffer.toString());
});

// Hello, World!
```

Unzip က အသုံးပြုထားတဲ့ Compression Method ကို အလိုအလျောက် Detect လုပ်ပြီး သင့်တော်တဲ့ Decompress Method ကို သုံးပေးသွားမှာဖြစ်ပါတယ်။

Build-in Module တွေထဲမှာ Buffer, Process, Stream, Readline စတဲ့ Performance Optimization အတွက် အရေးပါတဲ့ Low Level Module တွေ ပါဝင်ပါသေးတယ်။ Build-in Module စာရင်းအပြည့်အစုံကို ဒီနေရာမှာ လေ့လာနိုင်ပါတယ်။

https://nodejs.org/api/

Third-party Modules

Build-in Module တွေအပြင် Third-party Module တွေကိုလည်း NPM နဲ့ Install လုပ်ပြီး အသုံးပြုနိုင်ပါတယ်။ ဥပမာ – MySQL Database ကိုအသုံးပြုထားတဲ့ ပရိုဂရမ်တစ်ခုကို NodeJS နဲ့ တည်ဆောက်လိုတယ်ဆိုရင် Node MySQL Module ကို အသုံးပြုနိုင်ပါတယ်။ NPM နဲ့ အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ npm install mysql
```

`-g` Option ထည့်သွင်းပြီး Install မလုပ်ထားတာကို သတိပြုပါ။ Command အနေနဲ့ Run လိုတဲ့ Package တွေကိုသာ Global Package အနေနဲ့ Install လုပ်သင့်ပြီး `require()` နဲ့ချိတ်ဆက်အသုံးပြုလိုတဲ့ Module တွေကို

Local Module အဖြစ်သာ Project Directory ထဲမှာ Install လုပ်ထားသင့်ပါတယ်။ ဒီတော့မှ Project အတွက် လိုအပ်တဲ့ Module တွေဟာ တစ်နေရာ တည်းမှာ စုစည်းပြီးဖြစ်နေမှာဖြစ်ပါတယ်။ Install လုပ်ပြီးရင် MySQL နဲ့ အခုလို ချိတ်ဆက်ဆောင်ရွက်နိုင်ပါတယ်။

JavaScript

```
var mysql = require('mysql');

var conn = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: '' ,
    database: 'dbname'
});

conn.connect();

conn.query('SELECT * FROM table', function(err, rows) {
    console.log(rows.length);
});

conn.end();
```

`mysql.createConnect()` Function ကိုသုံးပြီး Connection တစ်ခုတည်ဆောက်ရပါတယ်။ အဲ့ဒီလိုတည်ဆောက်ရာမှာ Host Name, User Name, Password, Database Name စတဲ့ တန်ဖိုးတွေကို တွဲဖက်ပေးရပါတယ်။ တည်ဆောက်ထားတဲ့ Connection Object ကနေတစ်ဆင့် `connect()` Function နဲ့ MySQL Server ကို ချိတ်ဆက်ရပါတယ်။ Database ပေါ်မှာ အလုပ်လုပ်စေလိုတဲ့ Query တွေကိုတော့ နမူနာလိုင်းနံပါတ် (၁၂) မှာ ဖော်ပြထား သလို `query()` Function နဲ့ ပေးပို့နိုင်ပါတယ်။ Query က SELECT Query ဆိုရင် ရရှိလာတဲ့ Rows တွေကို Callback Function ရဲ့ Parameter အနေ နဲ့ လက်ခံထားပေးမှာ ဖြစ်ပါတယ်။ Query က INSERT, UPDATE, DELETE Query တွေဆိုရင်တော့ Result Object ကို Callback Function ရဲ့ Parameter အနေနဲ့ လက်ခံထားပေးမှာ ဖြစ်ပါတယ်။ ဥပမာ –

JavaScript

```
conn.query('INSERT INTO table (value, value)', function(err, result) {
    console.log(result.insertId);
});

conn.query('DELETE FROM table WHERE id = 123', function (err, result) {
    console.log(result.affectedRows);
})
```

MySQL အကြောင်းကို **Professional Web Developer** ရဲ့ **အခန်း (၉)** မှာ ဖော်ပြထားပါတယ်။ Node MySQL ရဲ့အသုံးပြုပုံ အသေးစိတ်ကို အောက်ပါလိပ်စာမှာ လေ့လာနိုင်ပါတယ်။

https://www.npmjs.com/package/mysql

နောက်အခန်းမှာ အခြား Database စနစ်တစ်ခုဖြစ်တဲ့ MongoDB အကြောင်းကို ဆက်လက်ဖော်ပြသွားမှာပါ။ Third-party Module တွေထဲမှာ လက်ရှိဒီစာရေးနေချိန် အသုံးအများဆုံး Module တွေကတော့ ExpressJS, Grunt, Bower, Yeoman, Cordova, LESS, CoffeeScript, Jade, Karma စတဲ့ Module တွေပဲဖြစ်ပါတယ်။ အဲ့ဒီထဲက Yeoman, Bower နဲ့ Grunt တို့ အကြောင်းကို ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ **အခန်း (၁၂)** မှာ ExpressJS အကြောင်းကို ဖော်ပြသွားမှာဖြစ်ပြီး **အခန်း (၁၆)** မှာ Cordova အကြောင်းကို ဖော်ပြပေးသွားမှာဖြစ်ပါတယ်။

LESS ဟာ CSS Pre-processor နည်းပညာတစ်ခုဖြစ်ပြီး၊ Coffee Script ဟာ JavaScript Pre-processor နည်းပညာဖြစ်ပါတယ်။ Jade ကတော့ HTML Template System တစ်ခုဖြစ်ပါတယ်။ ကျွန်တော့်အနေနဲ့ HTML/CSS/JavaScript တို့ကို မူလအတိုင်း အသုံးပြုရတာကို ပိုမိုနှစ်သက်ပါတယ်။ Pre-processor တွေ တစ်ဆင့်ခံပြီး ရေးလေ့မရှိ တဲ့အတွက် ထည့်သွင်းမဖော်ပြတော့ပါဘူး။ Pre-processor နည်းပညာတွေက ပိုမို သပ်ရပ်တဲ့ရေးထုံးနဲ့ ပြင်ဆင်ထိမ်းသိမ်း ရလွယ်တဲ့ Code တွေရေးနိုင်အောင် ဖြည့်စွက်ပေးထားတဲ့အတွက် လေ့လာထားမယ်ဆိုရင် အကျိုးရှိနိုင်ပါတယ်။

Karma ကတော့ Test Runner ဖြစ်ပါတယ်။ **အခန်း (၂)** မှာ TDD အကြောင်းဖော်ပြခဲ့စဉ်က Qunit Test Framework အကြောင်း ဖော်ပြခဲ့ဖူးတာကို မှတ်မိဦးမှာပါ။ Karma က Qunit လို Test Framework မဟုတ်ပဲ Qunit (သို့မဟုတ်) အခြား Test Framework တစ်ခုခုနဲ့ ရေးသားထားတဲ့ Test တွေကို Run ပေးတဲ့ Test Runner ဖြစ်ပါ တယ်။

Third-party Module တွေ အသုံးပြုပုံဟာ သက်ဆိုင်ရာ Module Author ပေါ်မှာမူတည်တဲ့အတွက် Module Author ပေးထားတဲ့ Documentation တွေကို လေ့လာ အသုံးပြု ကြရမှာပါ။ Third-party Module တွေကို NPM Package အနေနဲ့ npmjs.org မှာ လိုအပ်သလို ရှာဖွေအသုံးပြုနိုင်ပါတယ်။

10.7 – Custom Events

NodeJS မှာ Custom Event တွေဖန်တီးဖို့အတွက် Event Module ရဲ့ EventEmitter လုပ်ဆောင်ချက်ကို အသုံးပြုရပါတယ်။ ဥပမာ – ကျွန်တော်တို့က ဖြစ်ပေါ်တဲ့နေအပြောင်းအလဲတွေကို Event အနေနဲ့ Client ကို အသိပေးနိုင်တဲ့ Radio Station Service တစ်ခု တည်ဆောက်လိုတယ်ဆိုကြပါစို့။ အခုလိုရေးသားနိုင်ပါတယ်။

JavaScript

```
var util = require('util');
var EventEmitter = require('events').EventEmitter;

var Radio = function(station) {
    var self = this;

    setTimeout(function() {
        self.emit('start', station);
    }, 0);

    setTimeout(function() {
        self.emit('next', station);
    }, 5000);
```

```
14.
15.     setTimeout(function() {
16.         self.emit('close', station);
17.     }, 10000);
18. };
19.
20. util.inherits(Radio, EventEmitter);
21. module.exports = Radio;
```

လိုင်းနံပါတ်(၄) မှာ Radio Constructor ကိုကြေငြာထားပါတယ်။ ဒီ Constructor ကိုသုံးပြီး Object တစ်ခု တည်ဆောက်လိုက်တာနဲ့ `start` Event ကို Emit လုပ်ပေးဖို့ လိုင်းနံပါတ် (၇) မှာ သတ်မှတ်ထားပါတယ်။ `setTimeout()` ရဲ့ Timeout တန်ဖိုးကို 0 လို့ပေးထားတဲ့အတွက် Object တည်ဆောက်လိုက်တာနဲ့ ချက်ခြင်းအလုပ် လုပ်သွားမှာဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၁၁) မှာတော့ အချိန် (၅) စက္ကန့်ကြာတဲ့အခါ `next` Event ကို Emit လုပ်ပေးဖို့သတ်မှတ်ထားပါတယ်။ လိုင်းနံ ပါတ် (၁၅) မှာတော့ အချိန် (၁၀) စက္ကန့်ကြာတဲ့အခါ `close` Event ကို Emit လုပ်ပေးဖို့ သတ်မှတ်ပေးထား ပါတယ်။ တစ်ကယ့်လက်တွေ့ ပရိုဂရမ်တွေမှာ၊ အခုလိုပုံသေသတ်မှတ်ထားတဲ့ အချိန်နဲ့ မဟုတ်ပဲပရိုဂရမ်မှာ အမှန်တစ် ကယ် လုပ်ဆောင်ချက်တစ်ခု လုပ်သွားတိုင်းမှာ `this.emit()` ကိုသုံးပြီး Event တွေ လွှတ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၂၀) မှာ `util.inherits()` ကိုသုံးပြီး Radio Constructor က `EventEmitter` ကို Inherit လုပ်ထားတာကို သတိပြုပါ။ ဒီလို Inherit လုပ်ထားတဲ့အတွက် Radio Object မှာ `emit()` Function ကို အသုံးပြု နိုင်ခြင်းဖြစ် ပါတယ်။ ဒီ Module ကိုအသုံးပြုထားတဲ့ ပရိုဂရမ်တစ်ခုကို အခုလို ရေးသားစမ်းသပ်ကြည့်နိုင်ပါတယ်။

JavaScript

```
 1. var Radio = require('./radio.js');
 2.
 3. var station = {
 4.     name: 'City FM Radio',
 5. };
 6.
 7. var radio = new Radio(station);
 8.
 9. radio.on('start', function(s) {
10.     console.log('%s "%s" STARTED', new Date(), s.name);
11. });
12.
13. radio.on('next', function(s) {
14.     console.log('%s "%s" NEXT SONG', new Date(), s.name);
15. });
16.
17. radio.on('close', function(s) {
18.     console.log('%s "%s" CLOSED', new Date(), s.name);
19. });
```

လိုင်းနံပါတ် (၇) မှာ `radio` Object တစ်ခုကို Radio Constructor သုံးပြီး တည်ဆောက်ထားပါတယ်။ လိုင်းနံပါတ် (၉) မှာတော့ `start` Event ဖြစ်ပေါ်တဲ့အခါ လုပ်ရမယ့် Function ကိုသတ်မှတ်ထားပြီး လိုင်းနံပါတ် (၁၃) နဲ့ (၁၇) တို့မှာ `next` နဲ့ `close` Event တွေ ဖြစ်ပေါ်တဲ့အခါ လုပ်ရမယ့် Function တွေကို သတ်မှတ်ပေးထားပါတယ်။ တနည်းအားဖြင့် ကျွန်တော်တို့ သတ်မှတ်ထားတဲ့ Custom Event တွေကို ဒီ Code က အသုံးချနေခြင်းပဲဖြစ်ပါတယ်။

Event Function တွေမှာ သုံးထားတဲ့ s Variable ဟာ Event Emit လုပ်စဉ် Server Module က ပေးလိုက်တဲ့ တန်ဖိုးဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Custom Event တွေကို သတ်မှတ်ယုံသာမက Event ဖြစ်ပေါ်တဲ့အခါ အသုံးပြုရမယ့် တန်ဖိုး တွေကိုပါ Event Emit လုပ်စဉ် တွဲဖက်ပေးပို့နိုင်ခြင်း ဖြစ်ပါတယ်။

ဒီ App ကို စမ်းကြည့်ရင် အခုလိုရလဒ်ကို ရရှိမှာဖြစ်ပါတယ်။

```
$ node app.js

Thu Apr 16 2015 02:09:52 GMT+0630 (MMT) "City FM Radio" STARTED
Thu Apr 16 2015 02:09:57 GMT+0630 (MMT) "City FM Radio" NEXT SONG
Thu Apr 16 2015 02:10:02 GMT+0630 (MMT) "City FM Radio" CLOSED
```

`radio` Object မှာ `on()` Function ကိုအသုံးပြုပြီး Event ဖြစ်ပေါ်တဲ့အခါ အလုပ်လုပ်ရမယ် Function ကို သတ် မှတ်ထားတာကို တွေ့ရမှာဖြစ်ပါတယ်။ ကျွန်တော်တို့ Radio Constructor ကို တည် ဆောက်စဉ်က `on()` Function ထည့်သွင်းကြေငြာမထားပါဘူး။ ဒါကြောင့် `on()` Function ဟာ `EventEmitter` ကနေ ဆက်ခံထားတဲ့ Function ဖြစ်ကြောင်းနားလည်ရမှာ ဖြစ်ပါတယ်။ `on()` အစား `once()` Function ကို အစားထိုးသုံးနိုင်ပါတယ်။ `on()` Function က Event ဖြစ်ပေါ် တိုင်းမှာအလုပ်လုပ်ပြီး `once()` Function ကတော့ Event ဖြစ်ပေါ်တဲ့ ပထမဆုံး တစ်ကြိမ်သာအလုပ်လုပ်မှာဖြစ်ပါတယ်။

ဒီနည်းနဲ့ Event Emitter ကိုသုံးပြီး NodeJS ပရိုဂရမ်တစ်ခုရဲ့ လုပ်ဆောင်ချက်တွေကို Event ပေါ်အခြေခံပြီး အလုပ် လုပ်တဲ့ Architecture နဲ့ တည်ဆောက်ထားနိုင်မှာပဲဖြစ်ပါတယ်။

Conclusion

NodeJS ရဲ့အဓိကအားသာချက် (၃) ချက်ရှိတယ်လို့ ဆိုရပါမယ်။ ပထမတစ်ချက်ကတော့ JavaScript ဖြစ်ခြင်းပါပဲ။

ကျွန်တော် တို့အသုံးပြုနေတဲ့ Back-end နည်းပညာတွေ ဘယ်လိုပဲကွဲပြားပါစေ၊ Client-side Programing အတွက် တော့ JavaScript ကိုပဲသုံးကြရတာပါ။ တနည်းအားဖြင့် JavaScript ကို Developer တိုင်း ရင်းနှီးတက်ကျွမ်းပြီးဖြစ် တယ်လို့ ဆိုနိုင်ပါတယ်။ အဲ့ဒီလို ရင်းနှီးကျွမ်းဝင်ပြီးသား Language ကို အသုံးပြုရေးသားနိုင်တဲ့အတွက် NodeJS ကို အသုံးချနိုင်ဖို့၊ အချိန်တွေ အများကြီးပေးပြီး ထပ်မံလေ့လာနေစရာမလိုတော့ပါဘူး။

နောက်တစ်ချက်ကတော့ Non-blocking I/O ဖြစ်ပါတယ်။ ကျွန်တော်တို့ရဲ့ App တွေမှာ Non-blocking I/O လုပ်ဆောင်ချက်ကို Thread တွေ Process တွေစီမံစရာမလိုပဲ အလွယ်တစ်ကူရရှိနိုင်တဲ့အတွက် စွမ်းဆောင်ရည်မြင့် App တွေကို အလွယ်တစ်ကူ တည်ဆောက်နိုင်ခြင်း ဖြစ်တယ်လို့လည်း ဆိုနိုင်ပါတယ်။

နောက်ဆုံးတစ်ချက်ကတော့ Even Driven Architecture ဖြစ်ပါတယ်။ ဒီ Architecture ရဲ့အကူအညီနဲ့ ပိုပြီး Responsive ဖြစ်တဲ့ Real-time Network App တွေကို NodeJS နဲ့ တစ်ဆောက်နိုင်မှာဖြစ်ပါတယ်။ NodeJS ကို အထူးသဖြင့် I/O Heavy App တွေ၊ JSON API Back-end တွေနဲ့ Real-time App တွေ တည်ဆောက်ရာမှာ အဓိက အသုံးချသင့်ပါတယ်။

ကနေ့ခေတ်မှာ App တွေတည်ဆောက်တဲ့အခါ JSON API Back-end ကို အခြေခံတဲ့ App တွေအဖြစ် တည်ဆောက်ကြခြင်းဟာ Common Practice ဖြစ်နေပါပြီ။ ဒါ့အပြင် Client နဲ့ Server ကြား၊ အချက်အလက်တွေကို အချိန်နဲ့တစ်ပြေးညီ ပေးပို့နိုင်တဲ့ Real-time လုပ်ဆောင်ချက်ဆိုတာလည်း App တိုင်းလိုလိုအတွက် လိုအပ်ချက်တစ်ခု ဖြစ်လာနေပါပြီ။ ဒါကြောင့် NodeJS ဟာ Rockstar Developer တစ်ဦး မဖြစ်မနေ သိရှိထားရမယ့် နည်းပညာတစ်ခုပဲဖြစ်ပါတယ်။

အသုံးပြုသူများလာပြီး၊ စနစ်ရဲ့ သက်တမ်း ရှည်ကြာလာတာနဲ့အမျှ
လက်ခံသိမ်းဆည်းပေးရမယ့် Data တွေလည်း များပြားလာမှာပဲဖြစ်ပါတယ်။
အသုံးပြုထားတဲ့ Database က ရေရှည်မှာ Terabyte ပေါင်းများစွာရှိလာ
နိုင်တဲ့ Data တွေကို လက်ခံသိမ်းဆည်းနိုင်ဖို့လိုပါတယ်။

Rockstar Developer Course

Project Management, Web Service, Server Architecture
NodeJS အစရှိသည် ဤစာအုပ်ပါ အကြောင်းအရာများကို
စာရေးသူကိုယ်တိုင် သင်ကြားပေးခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

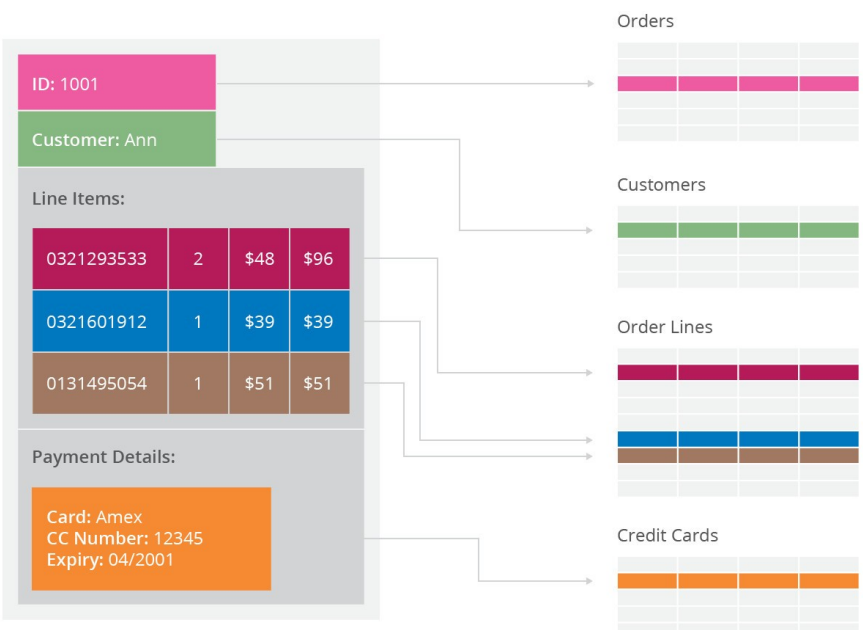
အခန်း(၁၁) – MongoDB (NoSQL Database)

MongoDB ဟာ Document-oriented, NoSQL Database နည်းပညာတစ်ခုဖြစ်ပါတယ်။ Open Source နည်းပညာ တစ်ခုဖြစ်ပြီး NoSQL Database တွေထဲမှာ လက်ရှိလူသုံးအများဆုံး Database ဖြစ်ပါတယ်။

MongoDB ဟာ အချက်အလက်တွေကိုသိမ်းတဲ့အခါ BSON လို့ခေါ်တဲ့ JSON နဲ့ သဘောသဘာဝတူတဲ့ Data Format ကို အသုံးပြုပြီး၊ အချက်အလက်တွေစီမံတဲ့ Administration Shell အတွက်လည်း JavaScript ကို အသုံးပြုထားပါတယ်။ Ajax, HTML5, NodeJS စတဲ့ နည်းပညာများနဲ့အတူ Software Development မှာ JavaScript ရဲ့အခန်းကဏ္ဍ ထူးခြား အရေးပါလာတဲ့အချိန်နဲ့ အချိန်ကိုက် ထွက်ပေါ်လာတဲ့ Database စနစ်တစ်ခုဖြစ်ပြီး၊ ခေတ်သစ် Software Development ရဲ့ အရေးပါတဲ့အစိတ်အပိုင်း တစ်ရပ်ဝဲဖြစ်ပါတယ်။

11.1 – NoSQL

Relational Database Management System (RDBMS) တွေဖြစ်တဲ့ MySQL, Oracle, MSSQL စတဲ့ Database စနစ်တွေမှာ၊ အဓိကသဘောသဘာဝ (၂) ချက်ရှိတယ်လို့ ဆိုနိုင်ပါတယ်။ ပထမတစ်ချက်က Data Model Schema ဖြစ်ပါတယ်။ Data တွေကို သိမ်းဆည်းနိုင်ဖို့အတွက် သိမ်းဆည်းရမယ့် Data Structure ကို ဦးဆုံးတည်ဆောက်ပေးရ ခြင်းဖြစ်ပါတယ်။



ပုံ (၁၁.၁) - Database Schema – RDBMS

Source – thoughtworks.com

ပုံ (၁၁.၁) ကိုလေ့လာကြည့်ပါ။ RDBMS တွေမှာ အချက်အလက်တွေကို သိမ်းဆည်းတဲ့အခါ Orders, Customers, Credit Cards စသဖြင့် အမျိုးအစားအလိုက် Table ကိုယ်စီနဲ့ ခွဲခြားသိမ်းဆည်းရပါတယ်။ Table တစ်ခုစီမှာ ကိုယ်ပိုင် ဖွဲ့စည်းပုံ Data Structure ကိုယ်စီရှိကြပါတယ်။ ဥပမာ – Customers Table မှာ Customer Name, Address, Phone စတဲ့ အချက်အလက်တွေ သိမ်းဆည်းထားနိုင်ပြီး၊ Order Lines Table မှာ Item Name, Category, Price စတဲ့အချက်အလက်တွေ သိမ်းဆည်းထားနိုင်ပါတယ်။ ဒီလိုမျိုး ခွဲခြားသိမ်းဆည်းထားတဲ့ Table တွေကနေ လိုချင်တဲ့ အချက်အလက် Data Set ကိုရရှိဖို့အတွက် SQL JOIN Statement တွေသုံးပြီး ရယူရပါတယ်။ အချက်အလက်တွေရဲ့ ဖွဲ့စည်းပုံ Structure နဲ့ ဆက်စပ်မှု Relationship တို့ကို အရင်ဆုံး Design လုပ်ရသလို၊ လက်တွေ့သိမ်းဆည်းတဲ့အခါ မှာလည်း သတ်မှတ်ထားတဲ့ Structure နဲ့အညီသာ သိမ်းဆည်းရမှာ ဖြစ်ပါတယ်။

RDBMS စနစ်တွေမှာ ရှိလေ့ရှိတဲ့ ဒုတိယတစ်ချက်ကတော့ ACID Compliance ဖြစ်ပါတယ်။ ACID ဆိုတာ Atomicity, Consistency, Isolation နဲ့ Durability တို့ကိုဆိုလိုတာပါ။ လိုရင်းကတော့ အချက်အလက် တိကျသေချာမှုကို အာမခံ ခြင်းပဲဖြစ်ပါတယ်။ Transaction နဲ့ Rollback တို့လိုလုပ်ဆောင်ချက်တွေရဲ့ အကူအညီနဲ့ မမျှော်မှန်းနိုင်တဲ့ ပြဿနာတစ်စုံ တစ်ရာကြောင့် အချက်အလက်တွေ အပြည့်အစုံမဟုတ်ပဲ တစ်ဝက်တစ်ပျက်သာ သိမ်းဆည်းမိခြင်းမျိုး မဖြစ်ရအောင် အာမခံပေးနိုင်ပါတယ်။ Foreign Key Relationship လုပ်ဆောင်ချက်အကူအညီနဲ့ တစ်နေရာမှာအသုံးချထားတဲ့ အချက် အလက်ကို သတိမမူမိပဲပယ် ဖျက်မိခြင်းကနေ ကာကွယ်ပေးပါတယ်။ Locking လုပ်ဆောင်ချက်အကူအညီနဲ့ တစ်ချိန် တည်းတစ်ပြိုင်တည်း အချက်အလက်တွေကို Access လုပ်မိလို့ မလိုလားအပ်တဲ့ ဆုံးရှုံးမှုတွေ မဖြစ်ရအောင် အာမခံပေး ကြပါတယ်။

RDBMS စနစ်တွေကို ဒီလိုအချက်တွေနဲ့ပြည့်စုံတဲ့အတွက် အခုချိန်ထိလည်း အဓိက Database စနစ်များအဖြစ် အားထား အသုံး ပြုနေကြရဆဲဖြစ်ပါတယ်။ ဒါပေမယ့် ခေတ်တွေ ပြောင်းလဲလာတာနဲ့အမျှ Data တွေရဲ့ သဘောသဘာဝကလည်း ပြောင်းလဲလာတဲ့အတွက်၊ အခုနောက်ပိုင်းမှာ RDBMS တွေရဲ့ လုပ်ဆောင်ချက်တွေနဲ့ မလုံလောက်တော့တဲ့ အခြေအနေ တွေ ဖြစ်ပေါ်လာပါတယ်။

Scalability

ကနေ့ခေတ် Software တွေဟာ အင်တာနက်ကိုအခြေခံတဲ့ Distributed Software တွေဖြစ်ကြပါတယ်။ အင်တာနက် အဆက် အသွယ်ကနေတစ်ဆင့် မည်သူမဆိုသုံးလို့ရတဲ့အတွက် တစ်ယောက်စ၊ နှစ်ယောက်စ လူပြောများလာရာကနေ ရုတ်တရက် ထင်ရှားသွားတဲ့ App တစ်ခုဟာ နေ့ချင်းညချင်း အသုံးပြုသူ သန်းချီရှိသွားနိုင်ပါတယ်။ Database စနစ်က အဲ့ဒီလိုအခြေအနေ ဖြစ်ပေါ်လာတဲ့အခါ လက်ခံအလုပ်လုပ်နိုင်ဖို့လိုပါတယ်။

Database တစ်ခုကို 500GB Storage Capacity ရှိတဲ့ Server တစ်ခုမှာ Setup လုပ်ထားတယ် ဆိုပါစို့။ အချက်အလက်တွေများလာလို့ 500GB နဲ့ မလောက်တော့လို့ Storage Capacity ထပ်တိုးရမယ်ဆိုရင် ပုံမှန်အားဖြင့် အဲ့ဒီ Server မှာ Hard Drive တစ်ခုကို သွားတိုးရမယ်သဘောဖြစ်ပါတယ်။ ဒီနည်းကို Vertical Scaling လို့ခေါ်ပါတယ်။ Vertical Scaling ဟာ ကုန်ကျစရိတ်များပြီး စီမံရခက်ခဲပါတယ်။

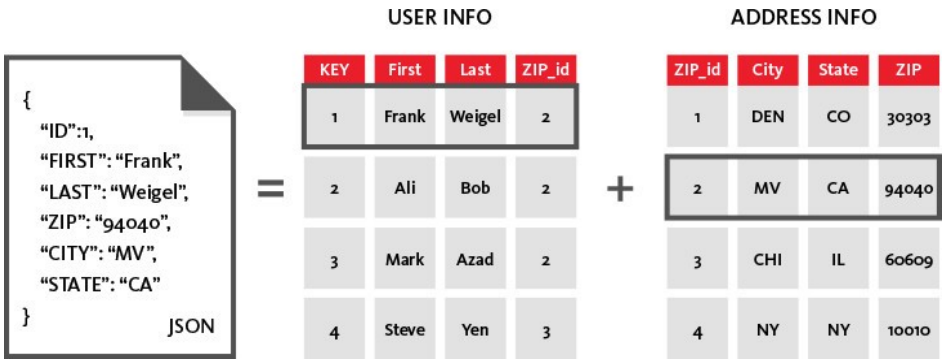
နောက်တစ်နည်းကတော့ Database ကို Network Cluster ပေါ်မှာ Deploy လုပ်နိုင်ပါတယ်။ Capacity တိုးဖို့လိုအပ် လာတဲ့အခါ မူလ Server ကို ထိစရာမလိုပဲ၊ Cluster ထဲမှာ နောက်ထပ် Server Node တစ်ခု ထပ်တိုး

လိုက်ယုံပါပဲ။ ဒီနည်းကို Horizontal Scaling လို့ခေါ်ပါတယ်။ အမှန်တစ်ကယ် Scalable ဖြစ်တဲ့ Database တစ်ခုဖြစ်ဖို့ဆိုရင် Horizontal Scale လုပ်နိုင်တဲ့ စနစ်ဖြစ်ဖို့လိုအပ်ပါတယ်။

RDBMS တွေမှာလည်း Cluster နဲ့ Replication လုပ်ဆောင်ချက်တွေ ရှိတက်ပေမယ့်၊ လက်တွေ့မှာ Storage Capacity တိုးစေဖို့ထက် Performance ကောင်းဖို့အတွက်သာ သုံးတက်ကြပါတယ်။ နည်းပညာသဘာဝအရ RDBMS တစ်ခုကို Cluster ပေါ်မှာ Deploy လုပ်ရတာ ခက်ခဲသလို၊ အကန့်အသတ်တွေလည်း များပါတယ်။ RDBMS ထဲ မှာသိမ်းဆည်းထား တဲ့ အချက်အလက်တွေဟာ တစ်ခုနဲ့တစ်ခု အပြန်အလှန်ဆက်စပ်နေကြတာပါ။ Record တစ်သန်းရှိလို့ဆိုပြီး Node တစ်ခု မှာ ငါးသိန်းထားပြီး နောက်တစ်ခုမှာ ငါးသိန်းထားမယ်လို့ လွယ်လွယ်ပြောလို့ မရနိုင်ပါဘူး။ Table တွေတစ်ခုနဲ့ တစ်ခုဟာ အပြန်အလှန် Relationship တွေရှိနေတဲ့အတွက် အဲ့ဒီ Relationship Structure တစ်ခုလုံးကို Node တွေမှာ ဖြန့်ထား နိုင်ဖို့ စီစဉ်ရတော့မှာပါ။ လုံးဝမရနိုင်တာမျိုး မဟုတ်ပေမယ့် ခက်ခဲတဲ့ကိစ္စတစ်ခုဖြစ်ပါတယ်။

Scale လုပ်တယ်ဆိုတာ လုပ်လို့ရတယ်ဆိုယုံနဲ့ မပြီးပါဘူး။ လွယ်ဖို့လည်း လိုပါတယ်။ အခုမှစပြီး Release လုပ်တဲ့ App တစ်ခုအတွက် လူသန်းချီသုံးနိုင်လောက်တဲ့ Infrastructural တစ်ခုကြိုတင်တည်ဆောက်ထားနိုင်ဖို့ မလွယ်ပါဘူး။ အမှန် တစ်ကယ် လူသန်းချီသုံးလာပြီဆိုတဲ့အချိန်ရောက်တော့မှ ချက်ခြင်း Scale Up လုပ်လိုက်နိုင်ဖို့လိုပါတယ်။ ပြီးတော့ Scale က အမြဲတန်း Up နေရမှာ မဟုတ်ပါဘူး။ လိုအပ်ရင် ပြန် Down ဖို့လည်း လိုပါတယ်။ ရုတ်တရက် လူပြောများသွားလို့ တစ်ပါတ်လောက် လူသန်းချီ သုံးခဲ့ပေမယ့် နောက်ပိုင်းမှာ အသုံးပြုသူ ပြန်နည်းသွားမယ်ဆိုရင် Scale Down ပြန်လုပ်နိုင်ဖို့ လိုပါတယ်။ ဒီလို အလွယ်တစ်ကူ Scale Up / Scale Down လုပ်နိုင်ဖို့အတွက် Cloud နည်းပညာတွေ၊ Visualization နည်းပညာတွေ သုံးနိုင်ပေမယ့် အသုံးပြုတဲ့ Database စနစ်က Network Cluster ပေါ်မှာ Deploy လုပ်ရလွယ်ကူတဲ့ စနစ် ဖြစ်ဖို့လည်း လိုအပ်ပါတယ်။ ဒါဟာ RDBMS စနစ်တွေမှာ ရှိနေတက်တဲ့ အကန့်အသတ်တစ်ခုပဲဖြစ်ပါတယ်။

NoSQL Database စနစ်တွေကတော့၊ စတင်တီထွင်ကတည်းက Scalability ကို ဦးစားပေးစဉ်းစားပြီး Horizontal Scale လုပ်နိုင်အောင် ရည်ရွယ်တည်ဆောက်ထားတဲ့ နည်းပညာတွေ ဖြစ်ကြပါတယ်။ NoSQL Database နဲ့ သိမ်းဆည်းထားတဲ့ အချက်အလက်တွေဟာ အခြေခံအားဖြင့် သီးခြားစီရပ်တည်ကြပါတယ်။ အပြန်အလှန်ဆက်စပ်မှု Relationship ဆိုတာမရှိပါဘူး။



ပုံ (၁၁.၂) - NoSQL Data Set

Source – couchbase.com

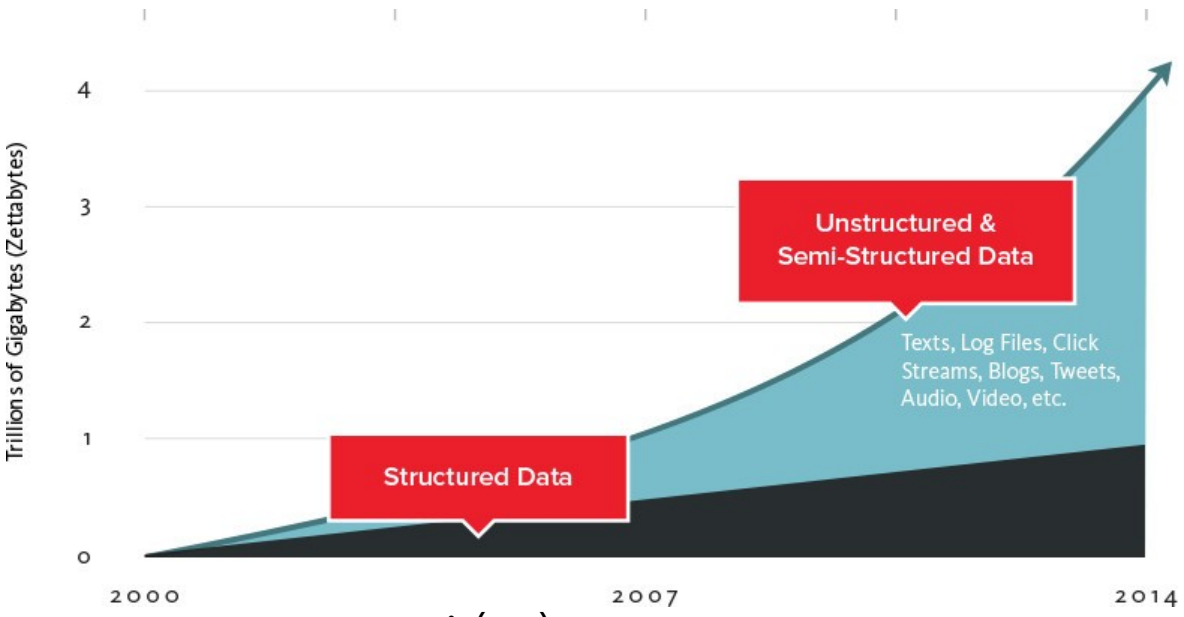
ပုံ (၁၁.၂) ကိုလေ့လာကြည့်ပါ။ ဘယ်ဘက်ခြမ်းက NoSQL ရဲ့ Data သိမ်းဆည်းပုံဖြစ်ပြီး ညာဘက်ခြင်းက RDBMS ရဲ့ Data သိမ်းဆည်းပုံ ဖြစ်ပါတယ်။ RDBMS မှာ User Info ဆိုတဲ့ Table နဲ့ Address Info ဆိုတဲ့ Table နှစ်ခုခွဲခြား သိမ်းဆည်း ထားခြင်းအားဖြင့် User Table မှာ လိပ်စာအပြည့်အစုံကို ထပ်ခါထပ်ခါ သိမ်းနေစရာမ လိုတော့ပဲ သက်ဆိုင်ရာ User ရဲ့ Zip Id ကိုသာ သိမ်းဖို့လိုပါတယ်။ User ရဲ့ လိပ်စာအပြည့်အစုံကို သိချင်ရင် Zip Id ကိုသုံးပြီး Address Info ကနေ ရယူနိုင်မှာပဲဖြစ်ပါတယ်။ NoSQL မှာတော့ အဲ့ဒီလို နှစ်နေရာခွဲသိမ်းမနေပဲ အားလုံးကို စုစည်းသိမ်းဆည်းပါတယ်။ ဒီလိုစုစည်းသိမ်းဆည်းလိုက်တဲ့အတွက် သက်ဆိုင်ရာ User ရဲ့ လိပ်စာ ကိုသိရဖို့၊ နောက်တစ်နေရာကနေ သွားယူနေစရာမလို တော့တဲ့ အားသာချက်ကို ရရှိသွားပါတယ်။ ဒါပေမယ့် Data Duplicate တွေဖြစ်ပြီး Storage နေရာပိုယူသွားတဲ့ အား နည်းချက်တော့ ဖြစ်ပေါ်လာပါတယ်။

NoSQL Database စနစ်တွေက Cluster ပေါ်မှာ Deploy လုပ်နိုင်ဖို့ရည်ရွယ်ထားတဲ့အတွက် Storage နေရာပိုယူ တာကို အမှု မထားပါဘူး။ နေရာပိုယူသွားပေမယ့်၊ Relationship မလိုတော့တဲ့အတွက် Process လုပ်ရတာ သက်သာသွားသလို အချက်အလက်တွေကို Node တွေပေါ်မှာ ဖြန့်သိမ်းရတာ ပိုလွယ်သွားပါတယ်။

ဒါ့အပြင် မူလကတည်းက Scalability အတွက် ရည်ရွယ်ဖန်တီးထားတဲ့စနစ်ဖြစ်တဲ့အတွက် Load Balancing လို လုပ် ဆောင်ချက်တွေ Build-in ပါဝင်ကြလေ့ရှိပါတယ်။ Node တစ်ခု Fail ဖြစ်ခဲ့ရင် စနစ်ရပ်တန့်မသွားစေပဲ နောက် Node တစ်ခုနဲ့ အလိုအလျောက် အစားထိုးပေးနိုင်တဲ့လုပ်ဆောင်ချက်တွေ Build-in ပါဝင်လေ့ရှိကြပါ တယ်။ RDBMS တွေမှာ တော့ ဒီလိုလုပ်ဆောင်ချက်တွေ လိုချင်ရင် ကိုယ်တိုင်အဆင့်ဆင့် Setup လုပ်ယူကြရမှာ ဖြစ်ပါတယ်။

Unstructured Data

ကနေ့အချိန်မှာ ကျွန်တော်တို့ Software တွေနဲ့အတူ အသုံးပြုရမယ့် Data တွေရဲ့ဖွဲ့စည်းပုံကို ကျွန်တော်တို့ ကိုယ်တိုင် အမြဲ သတ်မှတ်ချင်လို့မရတော့ပါဘူး။ Software နဲ့အတူ အသုံးပြုရမယ့် Data တွေဟာ နေရာပေါင်း စုံကလာမှာပါ။ User တွေဆီကလည်းလာပါမယ်။ အဲ့ဒီလိုလာတဲ့အခါ Desktop User ဆီကလာတဲ့ Data ရဲ့ ဖွဲ့စည်းပုံနဲ့ Mobile User ဆီက လာတဲ့ဖွဲ့စည်းပုံ တူချင်မှတူပါလိမ့်မယ်။ Third-party Service တွေထံကလည်း Data တွေကို ရယူအသုံးပြုရနိုင်ပါတယ်။ ဒီလို ရယူအသုံးပြုရာမှာ Service တစ်ခုနဲ့တစ်ခု အသုံးပြုတဲ့ Data Structure တူချင်မှတူပါလိမ့်မယ်။ ပြီးတော့ Data ဆိုတဲ့နေရာမှာ User Input Data တွေချည်းပဲ အသုံးဝင်တာ မဟုတ်ပါဘူး။ Error Log တွေ၊ Traffic Statistics တွေ၊ Usage Behavior တွေ၊ User Location Data တွေဟာ လည်း Software ရဲ့ လုပ်ဆောင်ချက်တွေ ပိုမိုကောင်းမွန်အောင် ဖန်တီးရာမှာ အသုံးဝင်တဲ့ Data တွေပဲ ဖြစ်ပါ တယ်။ ဒီလို နေရာစုံကနေ ပုံစံစုံနဲ့လာတဲ့ Data တွေကို ကြိုတင်သတ်မှတ် ထားတဲ့ Schema တစ်ခုနဲ့ သိမ်းဆည်း ဖို့ဆိုတာ သိပ်အဆင်မပြေတော့ပါဘူး။



ပုံ (၁၁.၃) - Data Evolution

Source – couchbase.com

ပုံ (၁၁.၃) ကိုလေ့လာကြည့်ပါ။ စစ်တမ်းများအရ၊ အခုနောက်ပိုင်းမှာ Unstructured Data တွေက Structured Data တွေထက် ပိုပိုပြီး များလာနေပါတယ်။ ကျွန်တော်တို့ရဲ့ Software တွေအနေနဲ့ Unstructured Data တွေနဲ့ အလုပ်လုပ် နိုင်ဖို့ဟာ လိုအပ် ချက်တစ်ခုဖြစ်လာသလို၊ Database တွေအနေနဲ့လည်း Unstructured Data တွေကို စီမံနိုင်ဖို့ လိုအပ် လာပါတယ်။ ကြိုတင်သတ်မှတ်ထားတဲ့ Structure နဲ့ Relationship ပေါ်မှာ အခြေခံအလုပ်လုပ်တဲ့ RDBMS တွေနဲ့ Unstructured Data တွေကို သိမ်းဆည်းရင် ထိရောက်အဆင်ပြေမှာ မဟုတ်တော့ပါဘူး။

ကနေ့အောင်မြင်နေတဲ့ Web App တွေ Mobile App တွေကိုလေ့လာကြည့်ရင် လုပ်ချက်သစ်တွေနဲ့ အမြဲမပြတ် အဆင့် မြှင့်တင်နေကြတာကို တွေ့ရနိုင်ပါတယ်။ နည်းပညာဟာ အမြဲပြောင်းလဲနေသလို ကျွန်တော်တို့ရဲ့ Software တွေကို လည်း နည်းပညာသစ်တွေနဲ့ အမြဲအဆင့်မြှင့်တင်နေနိုင်ဖို့လိုပါတယ်။ Software တစ်ခုမှာ လုပ်ဆောင်ချက် အသစ်တိုးလာတဲ့အခါ (သို့) လုပ်ဆောင်ပုံ ပြောင်းလဲသွားတဲ့အခါ နောက်ကွယ်ကနေ အချက်အလက်တွေ သိမ်းဆည်းပေးနေတဲ့ Database ထဲက Data Structure လည်း လိုက်လံပြောင်းလဲဖို့ လိုအပ်တက်ပါတယ်။ လက်ရှိ Data တွေရှိနေပြီး၊ အလုပ် လုပ်နေတဲ့ Structure တစ်ခုကို ပြောင်းရတယ် ဆိုတာ အတော်စဉ်းစားပြီးမှ လုပ်ရတဲ့အလုပ်တစ်ခုပါ။ အခန့်မသင့်ရင် ရှိနေတဲ့ Data တွေကို ထိခိုက်သွားနိုင်ပါတယ်။ ပြုလုပ်လိုက်တဲ့ Data Structure အပြောင်းအလဲကြောင့် လက်ရှိအလုပ်လုပ်နေတဲ့ လုပ်ဆောင်ချက်တွေကို ထိခိုက်မသွားစေဖို့လိုပါတယ်။ Data Structure ပိုမိုရှုပ်ထွေးလာလေ ပြောင်းလဲမှုပြု လုပ်ဖို့ ခက်ခဲလာလေပဲဖြစ်ပါတယ်။

တနည်းအားဖြင့်ပြောရရင်၊ ကြိုတင်သတ်မှတ်ထားတဲ့ Structure နဲ့ Relationship ဟာ Iterative Development နဲ့ အံမဝင်ဘူးလို့ ဆိုနိုင်ပါတယ်။ တိကျတဲ့ Requirement Specification ရရှိဖို့ခက်ခဲတဲ့အတွက် Agile နည်းစနစ်

တွေဖြစ်တဲ့ Iterative Development, SCRUM စတဲ့နည်းစနစ်တွေနဲ့ ဖြေရှင်းနိုင်တယ်လို့ ကျွန်တော်တို့ ပြောခဲ့ရာမှာ Coding ဆိုတဲ့ ရှုထောင့်ကနေပဲ အဓိကထား ပြောခဲ့ကြတာပါ။ အမှန်တော့ Data Structure ဆိုတာလည်း Requirement Specification ပေါ်မှာ အခြေခံတည်ဆောက်ရတာပါ။ Coding ကဏ္ဍမှာ တိကျတဲ့ Spec ရရှိဖို့ခက်တဲ့ပြဿနာကို Iterative Development နည်းစနစ်နဲ့ ဖြေရှင်းမယ်ဆိုရင် တိကျတဲ့ Data Structure ရရှိဖို့ ခက်ခဲတဲ့ကိစ္စကို ဘယ်လိုဖြေရှင်းမှာလဲ ဆိုတာ စဉ်းစားစရာဖြစ်လာပါတယ်။ လက်တွေ့မှာ Coding ပြင်ရခက်တယ်ဆိုတာထက် Data Structure ပြင်ရတာက ပိုခက်နိုင်ပါသေးတယ်။

NoSQL Database တွေက ဒီလိုအပ်ချက်တွေကို ဖြည့်စည်းပေးထားပါတယ်။ NoSQL Database တွေမှာ Data တွေကို စီမံဖို့အတွက် RDBMS တွေမှာလို SQL Query တွေကိုမသုံးပေမယ့် Developer တွေ ရင်းနှီးပြီးသားဖြစ်တဲ့ Object-oriented API တွေသုံးပြီး စီမံနိုင်အောင် စီစဉ်ပေးထားတက်ကြပါတယ်။

11.2 – Types of NoSQL Database

အဓိက NoSQL Database အမျိုးအစား (၄) မျိုးရှိပါတယ်။ အဲ့ဒါတွေကတော့ –

- Key-Value Store,
- Column Families Store,
- Document Store နဲ့
- Graph Database – တို့ပဲဖြစ်ပါတယ်။

Key-Value Store ဆိုတာ လက်တွေ့မှာ အချက်အလက်တွေကို Hasg Array ပုံစံ သိမ်းဆည်းတဲ့အရိုးရှင်းဆုံး NoSQL နည်းပညာတစ်ခုဖြစ်ပါတယ်။ အချက်အလက်တစ်ခုကို သိမ်းဆည်းလိုတဲ့အခါ Key တစ်ခုနဲ့ သိမ်းဆည်းပြီး၊ ပြန်လည်ရယူ လိုတဲ့အခါ သတ်မှတ်ထားတဲ့ Key ကိုအသုံးပြုပြီး ပြန်လည်ရယူနိုင်ပါတယ်။ ဥပမာ –

Redis

```
SET foo bar
```

`foo` ဆိုတဲ့ Key နဲ့ `bar` ဆိုတဲ့ တန်ဖိုးကိုသိမ်းဆည်းထားတဲ့အတွက်၊ နောင်လိုအပ်လိုအဲ့ဒီတန်ဖိုးကို ပြန်လည်ရယူလိုတဲ့ အခါ အခုလိုရယူနိုင်မှာဖြစ်ပါတယ်။

Redis

```
GET foo     // => bar
```

တစ်နည်းအားဖြင့် ကျွန်တော်တို့ Code တွေရေးသားရာမှာသုံးလေ့ရှိတဲ့ List တွေ Array တွေကိုပဲ Data Storage အတွက် အသုံးပြုလိုက်ခြင်းဖြစ်ပါတယ်။ Programing Language တွေက Array တွေကိုစီမံဖို့အတွက် Function တွေ ပေးထားကြသလို ပဲ၊ Key-Value Store တွေကလည်း အလားတူစီမံနိုင်တဲ့ API တွေပေးထားတက်ကြပါတယ်။

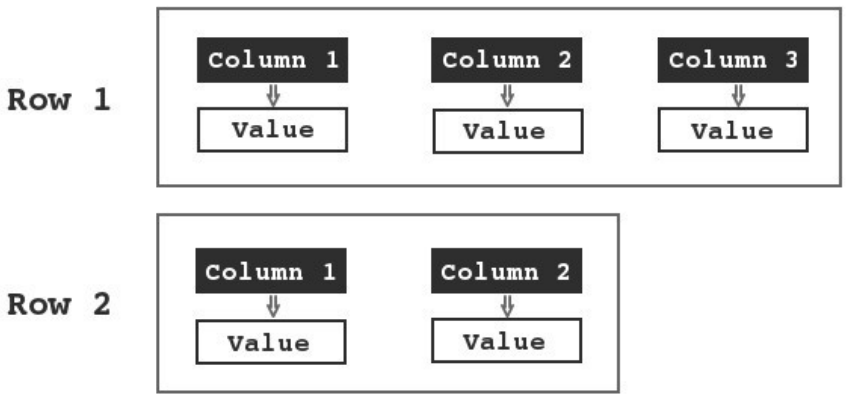
တစ်ချို့ Key-Value Store တွေဟာအချက်အလက်တွေကို Memory ပေါ်မှာပဲ သိမ်းဆည်းအလုပ်လုပ်တဲ့အတွက် အလွန် မြန်ပါတယ်။ လိုအပ်လို့ အချက်အလက်တွေကို File System ပေါ်မှာ အမြဲသိမ်းဆည်းလိုရင်လည်း သိမ်းဆည်း ပေးနိုင်တဲ့ လုပ်ဆောင်ချက်တွေ ပါဝင်ပါတယ်။

ရှုပ်ထွေးတဲ့ဖွဲ့စည်းပုံတွေမရှိပဲ၊ ခပ်ရိုးရိုးပဲ အလုပ်လုပ်တဲ့အတွက် Performance အလွန်ကောင်းကြပါတယ်။ တစ်ချို့ အသုံးများတဲ့ အချက်အလက်တွေကို File System ကနေခဏခဏသွားယူနေစရာမလိုအောင် Memory ပေါ်မှာ Cache လုပ်ထားဖို့နဲ့ App အလုပ်လုပ်နေစဉ် တောက်လျှောက်မှတ်တမ်းတင်နေဖို့ လိုအပ်တဲ့ State Data တွေ၊ Traffic Statistic လို Data တွေ၊ Usage Behavior Data တွေကို သိမ်းဆည်းခြင်းကဲ့သို့ ကိစ္စရပ်များအတွက် အထူးသင့်တော်ပါတယ်။

ထင်ရှားတဲ့ Key-Value Store တွေကတော့ **Redis**, **DynamoDB** နဲ့ **Raik** တို့ပဲဖြစ်ပါတယ်။

Column Families Store တွေကို RDBMS တွေရဲ့ Table တွေနဲ့ ယှဉ်ပြီးကြည့်နိုင်ပါတယ်။ RDBMS Table တွေမှာ Table ရဲ့ ဖွဲ့စည်းပုံ Structure ကို Column တွေနဲ့သတ်မှတ်ပြီး၊ အချက်အလက်တွေကို Row အနေနဲ့ သိမ်းဆည်းပါ တယ်။ Column Families Store တွေမှာတော့ အချက်အလက်တွေကို Column အနေနဲ့ တန်းစီသိမ်းဆည်းသွားနိုင်ပြီး အမျိုးတူရာ အချက်အလက်တွေကို Row အနေနဲ့ စုစည်းထားနိုင်မှာ ဖြစ်ပါတယ်။

တစ်နည်းအားဖြင့် တန်ဖိုးတစ်ခုတိုးလိုတဲ့အခါ Row အနေနဲ့မတိုးပဲ၊ Row တစ်ခုရဲ့ Column အနေနဲ့ တိုးသွားရမှာဖြစ် ပါတယ်။

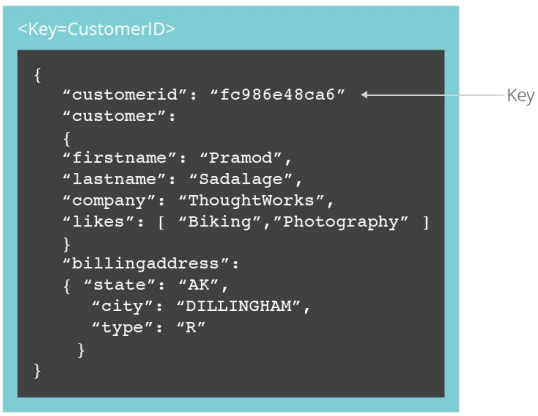


ပုံ (၁၁.၄) – Column Families Store

လက်ရှိထင်ရှားတဲ့ Column Families Store တွေကတော့ **Cassandra** နဲ့ **Hbase** တို့ပဲဖြစ်ပါတယ်။

Cassandra လို Column Families Store က Row တစ်ခုမှာရှိတဲ့ Data အားလုံးကို ဖိုင်တစ်ခုအနေနဲ့ သိမ်းဆည်းထား တက်ပါတယ်။ ဒါကြောင့် အတူတကွ အသုံးပြုဖို့လိုတက်တဲ့ အချက်အလက်တွေကို Row တစ်ခုတည်းမှာ စုစည်းထားခြင်း အားဖြင့်၊ အချက်အလက်တွေ ပြန်လည်ရယူတဲ့အခါ ပိုမိုမြန်ဆန်စေမှာပဲဖြစ်ပါတယ်။

Document Store ဆိုတာကတော့ အချက်အလက်တွေကို Table တွေ Row တွေနဲ့မသိမ်းပဲ Document တွေ အနေနဲ့ စုစည်းသိမ်းဆည်းတဲ့ အမျိုးအစားဖြစ်ပါတယ်။ Document အနေနဲ့ အချက်အလက်တွေကိုသိမ်းဆည်း တဲ့အခါ ကြိုတင် သတ်မှတ်ထားတဲ့ Structure နဲ့ သိမ်းဆည်းဖို့မလိုပေမယ့်၊ အချက်အလက်တွေရဲ့ ဖွဲ့စည်းပုံကို JSON, BSON, XML စတဲ့ Format တွေနဲ့ သတ်မှတ်သိမ်းဆည်းနိုင်ပါတယ်။



ပုံ (၁၁.၅) – Document Store

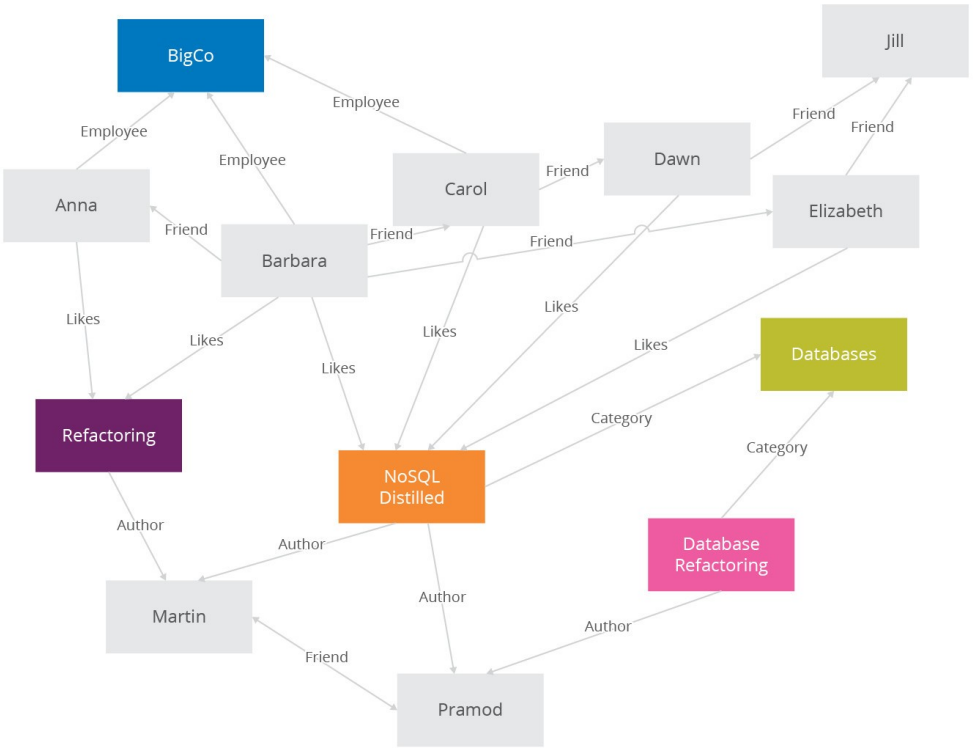
Source – thoughtworks.com

Document Store နဲ့ Key-Value Store သဘောသဘာဝ သိပ်မကွာပါဘူး။ ကွာသွားတာက Key-Value Store မှာ Value က ရိုးရိုးတန်ဖိုးတစ်ခုဖြစ်ပြီး Document Store ရဲ့ Value ကတော့ Key-Value တွေစုစည်းပါဝင်တဲ့ Document တစ်ခုဖြစ်ခြင်းပဲဖြစ်ပါတယ်။

Document Store တွေထဲမှာ ထင်ရှားတာကတော့ **MongoDB** နဲ့ **CouchDB** တို့ပဲဖြစ်ပါတယ်။

Key-Value Store တစ်ခုမှာ အချက်အလက်တွေကို Key ပေါ်အခြေခံပြီး စီမံရပါတယ်။ Value တစ်ခုကိုလိုချင်ရင် Key ကိုသုံး ပြီးရယူရပါတယ်။ ပယ်ဖျက်လိုရင်လည်း Key ကိုသုံးပြီး ပယ်ဖျက်ရပါတယ်။ အခြေခံအားဖြင့် Key-Value Store က Value ကို စိတ်မဝင်စားပါဘူး။ Value ပေါ်အခြေခံပြီး အလုပ်လုပ်မပေးနိုင်ပါဘူး။ Document Store ရဲ့ ထူးခြား ချက်က Key ကိုသာမက Value ကိုပါ အသုံးချပြီး အချက်အလက်တွေကို စီမံနိုင်ခြင်းပဲဖြစ်ပါ တယ်။ ဒါကြောင့် Document Store တွေကို Content အခြေပြု App တွေတည်ဆောက်ရာမှာ အသုံးပြုဖို့ သင့်တော်ပါတယ်။ App အများ စုဟာ Content အခြေပြု App တွေဖြစ်ကြတဲ့အတွက် MongoDB လို Document Store ဟာ NoSQL Database တွေထဲမှာ လူသုံးပိုများတဲ့ အမျိုးအစားဖြစ်နေခြင်းပဲ ဖြစ်ပါတယ်။

Graph Database ဆိုတာကတော့ Graph Theory ပေါ်မှာအခြေခံပြီး အချက်အလက်တွေကို သိမ်းဆည်းတဲ့ Database အမျိုးအစားဖြစ်ပါတယ်။ Graph တစ်ခုမှာ အခြေခံအားဖြင့် Node, Property နဲ့ Edge ဆိုပြီး အစိတ်အပိုင်းသုံးရပ် ပါဝင်ပါတယ်။ Node ဆိုတာ Record Set တစ်ခုလို့ လွယ်လွယ်မှတ်နိုင်ပါတယ်။ Node တိုင်းမှာ Property နဲ့ Edge ဆိုတဲ့ အချက်အလက်အမျိုး အစားနှစ်မျိုးပါဝင်ပါတယ်။ Property က Node Content ဖြစ်ပြီး Edge ကတော့ Node နဲ့ အခြား Node တစ်ခု ဆက်စပ်ပုံကို သတ်မှတ်ထားတဲ့ အချက်အလက် တွေဖြစ်ပါတယ်။



ပုံ (၁၁.၆) - Graph Database

Source – thoughtworks.com

ပုံ (၁၁.၈) လေ့လာကြည့်ရင် ဟိုးဘယ်ဘက်အပေါ်ထောင့်က BigCo ဆိုတာ Node တစ်ခုဖြစ်ပါတယ်။ သူ့မှာ Address, Product, Service စတဲ့ Property တွေပါဝင်နိုင်ပါတယ်။ သူ့အောက်နားက Anna ဆိုတာ နောက်ထပ် Node တစ်ခုဖြစ်ပါတယ်။ အဲ့ဒီ Node မှာလည်း Email, Position, Marital Status စတဲ့ သူ့ကိုယ်ပိုင် Property တွေ ရှိနိုင်ပါတယ်။ ထူးခြားချက်အနေနဲ့ Anna Node မှာ Employee ဆိုတဲ့ Edge တစ်ခုရှိနေပြီး အဲ့ဒီ Edge က BigCo ကိုညွှန်းထားပါတယ်။ Employee Edge မှာလည်း Employee Since လို တွဲဖက်အချက်အလက်တွေ ပါဝင်နိုင်ပါ သေးတယ်။ ဒီ Relationship ကို Unidirectional Relationship လို့ခေါ်ပါတယ်။ Graph ကိုကြည့်လိုက်တာနဲ့ Anna ဟာ BigCo ရဲ့ Employee အနေနဲ့ ဆက်စပ်နေကြောင်းကို သိရနိုင်ပါတယ်။

အောက်နားက Martin နဲ့ Pramod စတဲ့ Node တို့ကို လေ့လာကြည့်ရင် Friend Edge နဲ့ အပြန်အလှန် ဆက်စပ်မှုရှိနေတာကိုတွေ့ရမှာဖြစ်ပါတယ်။ ဒီလိုဆက်စပ်မှုကိုတော့ Bidirectional Relationship လို့ခေါ်ပါတယ်။ လက်တွေ့မှာ ပုံမှာပြထားသလို Edge တစ်ခုနဲ့ အပြန်အလှန်ချိတ်ဆက်မှာ မဟုတ်ပါဘူး။ Martin က Friend Edge နဲ့ Pramod ကိုညွှန်းပြီး၊ Pramod က သူ့ကိုယ်ပိုင် Friend Edge နဲ့ Martin ကို ပြန်ညွှန်းရမှာဖြစ်ပါတယ်။

ဒီတော့ Graph Database တွေမှာ အချက်အလက်တစ်ခုသိမ်းရင် Node အနေနဲ့သိမ်းပြီး ပါဝင်တဲ့ Content တွေကို Property နဲ့ Edge ဆိုပြီး နှစ်ပိုင်းခွဲသိမ်းပေးရတဲ့သဘောပါ။ RDBMS တွေရဲ့ Table Relationship နဲ့ကွာသွားတာက၊ RDBMS တွေမှာ Relationship က Table ပေါ်မှာ သတ်မှတ်ခြင်းဖြစ်ပါတယ်။ Graph Database မှာ

တော့ Node တိုင်းမှာ ကိုယ်ပိုင် Relationship အချက်အလက်တွေ တွဲဖက်ပါဝင်မှာဖြစ်ပါတယ်။

Graph Database တွေဟာ အချက်အလက်ဆက်စပ်မှုပေါ်မှာ အဓိကထားအလုပ်လုပ်တဲ့ App တွေနဲ့ သင့်တော်ပါတယ်။ အမြင်သာဆုံးနမူနာကတော့ Social Network ဖြစ်ပါတယ်။ Account တစ်ခုဟာ ဘယ်သူတွေနဲ့ Friend ဖြစ်နေတယ်၊ Follower ဘယ်လောက်ရှိတယ်၊ ဘယ်သူတွေကို Follow လုပ်ထားတယ် စတဲ့ဆက်စပ်မှုတွေကို စီမံရမှာဖြစ်လို့ Social Network လို App မျိုးဟာ Graph Database တွေကို အသုံးပြုသင့်တဲ့ App အမျိုးအစားပဲ ဖြစ်ပါတယ်။

Graph Database တွေထဲမှာ ထင်ရှားတာကတော့ **Noe4J** ဖြစ်ပါတယ်။

11.3 – MongoDB

MongoDB ဟာ Document Store အမျိုးအစား NoSQL Database တစ်ခုဖြစ်ပါတယ်။ Client-Server ပုံစံအလုပ်လုပ်တဲ့ Database ဖြစ်တဲ့အတွက် အချက်အလက်တွေကို Server မှာစုစည်းသိမ်းဆည်းပြီး Client Software တွေနဲ့ အဲ့ဒီအချက်အလက်တွေကို ရယူစီမံနိုင်မှာဖြစ်ပါတယ်။ MongoDB Installer နောက်ဆုံး Version ကို mongodb.org မှာ Download ရယူနိုင်ပါတယ်။ Installer နဲ့အတူ mongod, mongo, mongoexport, mongoimport, mongodump, mongorestore စတဲ့ Command Line Tool တွေ တစ်ခါတည်း ပါဝင်လာမှာဖြစ်ပါတယ်။



mongod ဟာ MongoDB Server Administration Tool ဖြစ်ပြီး mongo ကတော့ MongoDB Server ကို ချိတ်ဆက်ပြီး Data တွေ စီမံနိုင်တဲ့ Client Tool တစ်ခုဖြစ်ပါတယ်၊ Mongo Shell လို့ခေါ်ပါတယ်။ Mongo Shell ဟာ Mozilla ရဲ့ JavaScript Engine ဖြစ်တဲ့ SpiderMonkey JavaScript Engine ကို အခြေခံထားခြင်း ဖြစ်ပါတယ်။ ဒါကြောင့် Mongo Shell ဆိုတာ အမှန်တော့ NodeJS Console လို JavaScript Console တစ်ခုပဲဖြစ်တယ်လို့ ဆိုနိုင်ပါတယ်။

> ဆက်လက်ဖော်ပြမယ့် Command နဲ့ Mongo Shell နမူနာတွေဟာ လက်တွေ့ကူးယူစမ်းသပ်နိုင်တဲ့ နမူနာတွေဖြစ်ပါတယ်။ ဒါပေမယ့် MongoDB Version တစ်ခုနဲ့တစ်ခု အချို့လုပ်ဆောင်ချက်တွေ ကွဲပြားတက်တာကိုသတိပြုပါ။ နမူနာမှာ အသုံးပြုဖော်ပြမှာကတော့ MongoDB 2.4.9 ဖြစ်ပါတယ်။ စာဖတ်သူရဲ့ MongoDB က အခြား Version ဖြစ်ခဲ့မယ့်ဆိုရင် အချို့ Shell တွေအသုံးပြုပုံ ကွဲပြားသွားနိုင်ပါတယ်။

Ubuntu မှာဆိုရင်တော့ MongoDB ကို အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ sudo apt-get install mongodb
```

တစ်ခုသတိပြုရမှာက Ubuntu Official Repository ထဲက MongoDB ဟာ Version တစ်ချို့ နောက်ကျနေနိုင်ပါတယ်။ ဒါကြောင့် နောက်ဆုံး Version ကိုမှ လိုချင်တာဆိုရင်တော့ mongodb.org ကနေပဲ Download ရယူရမှာ

ဖြစ်ပါတယ်။ ဒီစာရေးနေချိန်မှာ နောက်ဆုံးထွက်ရှိထားတဲ့ MongoDB Version က 3.0.2 ဖြစ်ပြီး Ubuntu 14.04 Repository ထဲက MongoDB Version ကတော့ 2.4.9 ပဲ ရှိပါသေးတယ်။

Ubuntu မှာ `apt-get` နဲ့ MongoDB ကို Install လုပ်ခဲ့မယ်ဆိုရင် Install လုပ်ပြီးနောက် Ubuntu က MongoDB Server ကို အလိုအလျောက် စထားပေးမှာဖြစ်ပါတယ်။ `apt-get` နဲ့ Install မလုပ်ပဲ Manual Install ပြုလုပ်ခဲ့တာဆို ရင်တော့ Install လုပ်ပြီးတဲ့အခါ MongoDB Server ကို အခုလို အစပြုပေးနိုင်ပါတယ်။

```
$ sudo service mongod start
```

Ubuntu မှာ MongoDB နောက်ဆုံး Version ကို Manual Install လုပ်နိုင်ဖို့အတွက် Installation Instruction အပြည့်အစုံကို အောက်ပါလိပ်စာမှာ လေ့လာနိုင်ပါတယ်။

http://docs.mongodb.org/manual/tutorial/install-mongodb-on-ubuntu/

Windows အတွက် Installation Instruction အပြည့်အစုံကိုတော့ အောက်ပါလိပ်စာမှာ လေ့လာနိုင်ပါတယ်။

http://docs.mongodb.org/manual/tutorial/install-mongodb-on-windows/

Install လည်းလုပ်ပြီးပြီ၊ MongoDB Server ကိုလည်း Run ပြီးပြီဆိုရင်တော့ mongo Command နဲ့ Mongo Shell ကို စတင်အသုံးပြုနိုင်ပြီဖြစ်ပါတယ်။ Mongo Shell အတွင်းမှာ help Command ကို Run ကြည့်ရင် အသုံးပြုနိုင်တဲ့ Command စာရင်းကိုတွေ့ရမှာဖြစ်ပါတယ်။

```
$ mongo

MongoDB shell version: 2.4.9
connecting to: test

> help

    db.help()                    help on db methods
    db.mycoll.help()             help on collection methods
    sh.help()                    sharding helpers
    rs.help()                    replica set helpers
    help admin                   administrative help
    help connect                 connecting to a db help
    help keys                    key shortcuts
    help misc                    misc things to know
    help mr                      mapreduce

    show dbs                     show database names
    show collections             show collections in current database
    show users                   show users in current database
    show profile                 show most recent system.profile
    … …
```

MongoDB Database Server မှာရှိနေတဲ့ Database စာရင်းကို ကြည့်ရှုလိုရင် `show dbs` Command ကို သုံးရပါ တယ်။ အဲဒီ Database တွေထဲက အသုံးပြု စီမံလိုတဲ့ Database ကိုရွေးချယ်ဖို့ အတွက် use Command ကို သုံးရပါတယ်။

```
> show dbs

local 0.078125GB

> use mydb

switched to db mydb

> db

mydb

> show dbs

local 0.078125GB

> db.users.insert({"name": "John Doe"});
> show dbs

local 0.078125GB
mydb  0.203125GB

>
```

နမူနာကိုလေ့လာကြည့်ပါ။ ပထမတစ်ကြိမ် `show dbs` နဲ့ လက်ရှိ ရှိနေတဲ့ Database စာရင်းကို ဖော်ပြစေတဲ့ အခါ `local` လို့ခေါ်တဲ့ Database တစ်ခုရှိနေကြောင်းဖော်ပြလာပါလိမ့်မယ်။ `use mydb` Command နဲ့ `mydb` အမည်ရှိ Database ကို ရွေးခိုင်းလိုက်တဲ့အခါ မူလက `mydb` Database ရှိမနေတဲ့အတွက်၊ Mongo Shell က `mydb` အမည်နဲ့ Database အသစ်တစ်ခုကို တည်ဆောက်ပေးသွားမှာ ဖြစ်ပါတယ်။ တနည်းအားဖြင့် `use` Command ကို လက်ရှိရှိနေတဲ့ Database တစ်ခုကို ရွေးချယ်ဖို့သုံးနိုင်သလို၊ Database အသစ်တစ်ခု တည် ဆောက်ဖို့လည်း သုံးနိုင်ခြင်းဖြစ်ပါတယ်။

`db` Command ကိုတော့ လက်ရှိရွေးချယ်ထားတဲ့ Database အမည်ဖော်ပြစေဖို့ သုံးနိုင်ပါတယ်။ နမူနာမှာ `db` Command ကို Run ကြည့်တဲ့အခါ `mydb` လို့ ဖော်ပြပေးတာကို တွေ့ရမှာဖြစ်ပါတယ်။

`show dbs` နဲ့ နောက်တစ်ကြိမ် Database စာရင်းကြည့်တဲ့အခါ ကျွန်တော်တို့ တည်ဆောက်ထားတဲ့ `mydb` မ ပါဝင်သေးတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။ စာရင်းထဲမှာ ဖော်ပြစေဖို့အတွက် Database ထဲမှာ အနည်းဆုံး Document တစ်ခုရှိ ရမှာဖြစ်လို့ `insert()` Function ကိုသုံးပြီး Document တစ်ခု ဆက်လက်ထည့်သွင်း လိုက်ပါတယ်။ `db.users. insert()` လို့ပြောလိုက်တဲ့အတွက် လက်ရှိ Database ဖြစ်တဲ့ `mydb` ထဲမှာ `users` အမည်နဲ့ Collection တစ်ခု တည်ဆောက်ပြီး Document တစ်ခုကို ထည့်သွင်းစေလိုက်ခြင်း ဖြစ်ပါ တယ်။ နမူနာအရ Document Content အနေနဲ့ `{"name": "John Doe"}` ကို သတ်မှတ်ပေးထားပါတယ်။

RDBMS တွေမှာ Record တွေကို Table တွေနဲ့ စုစည်းထားသလိုပဲ MongoDB မှာ Document တွေကို Collection တွေနဲ့ စုစည်းထားရမှာ ဖြစ်ပါတယ်။

Document တစ်ခုထည့်သွင်းပြီးနောက် `show dbs` ကို ထပ် Run ကြည့်တဲ့အခါ Database စာရင်းထဲမှာ `mydb` ပါဝင်လာတာကို တွေ့ရမှာပဲဖြစ်ပါတယ်။

MongoDB က အချက်အလက်တွေကို Document ထဲမှာ သိမ်းတဲ့အခါ BSON Format ကို သုံးပါတယ်။ BSON Format ဟာ JSON နဲ့ ရေးသားပုံ ရေးထုံးအတူတူပါပဲ။ ကွာသွားတာကတော့ Data Type ဖြစ်ပါတယ်။ JSON ဟာ JavaScript ကနေ လာတဲ့အတွက် JavaScript လိုပဲကိန်းဂဏန်း အားလုံးအတွက် Number Data Type ကို သုံးပါတယ်။ ကိန်းပြည့်၊ ဒဿမကိန်း ဆိုပြီးခွဲမထားပါဘူး။ BSON ကတော့ ကိန်းပြည့်တွေအတွက် Integer Data Type ကိုသုံးပြီး ဒဿမကိန်းတွေအတွက် Double Data Type ကို သုံးပါတယ်။ ဒါ့အပြင် ဖြည့်စွက် Data Type တွေအဖြစ် Date နဲ့ Byte Array တို့ပါဝင်လာပါတယ်။ Date ကို Unix Timestamps သိမ်းဖို့သုံးပြီး၊ Byte Array ကိုတော့ Binary Data တွေ သိမ်းဖို့သုံးပါတယ်။

လက်ရှိရွေးချယ်ထားတဲ့ Database ကို ပယ်ဖျက်လိုတဲ့အခါ `db.dropDatabase()` ကို အသုံးပြုနိုင်ပါတယ်။

```
> db.dropDatabase()

{ "dropped" : "mydb", "ok" : 1 }

>
```

11.4 – CRUD Operations in MongoDB

RDBMS တွေမှာ Record တွေကို Table တွေနဲ့စုစည်းထားနိုင်သလိုပဲ MongoDB မှာလည်း Document တွေကို Collection တွေနဲ့ စုစည်းထားနိုင်ပါတယ်။ `db.createCollection()` ကို Collection အသစ်တစ်ခုတည်ဆောက်ဖို့အတွက် သုံးနိုင်ပါတယ်။

```
> use mydb

switched to db mydb

> db.createCollection("issues")

{ "ok" : 1 }

> show collections

issues
system.indexes

>
```

နမူနာအရ `mydb` Database ထဲမှာ `issues` အမည်နဲ့ Collection တစ်ခုကို တည်ဆောက်ပေးသွားမှာဖြစ်ပါတယ်။ Collection တစ်ခုကို ကြိုတင်မတည်ဆောက်ပဲ Document ထည့်သွင်းစဉ်မှာ တစ်ခါတည်းသတ်မှတ်နိုင်ပါသေးတယ်။

```
> db.users.insert({name: "John Doe"})
> show collections

issues
system.indexes
users

>
```

`users` အမည်နဲ့ Collection တစ်ခုကို ကြိုတင်တည်ဆောက်ထားခြင်းမရှိပေမယ့် `db.users.insert()` နဲ့ users Collection ထဲမှာ Document တစ်ခု တည်ဆောက်ပေးဖို့ ပြောလိုက်ခြင်းဖြစ်ပါတယ်။ MongoDB က `users` Collection ကို အလိုအလျောက် တည်ဆောက်ပေးသွားတဲ့အတွက် `show collections` နဲ့ကြည့်လိုက်တဲ့အခါ Collection စာရင်းထဲမှာ `users` လည်း ပါဝင်နေခြင်းပဲဖြစ်ပါတယ်။

Collection တစ်ခုကို ပယ်ဖျက်ဖို့အတွက် `drop()` Function ကိုသုံးနိုင်ပါတယ်။ ဥပမာ –

```
> db.users.drop()

true

> show collections

issues
system.indexes

>
```

Collection ထဲမှာ Document တွေထည့်သွင်းဖို့အတွက် `insert()` Function ကို အသုံးပြုနိုင်ကြောင်း ပြီးခဲ့တဲ့နမူနာမှာ တွေ့ရှိခဲ့ရပြီးဖြစ်ပါတယ်။ နောက်ထပ် `insert()` နမူနာတစ်ခု ထပ်မံဖော်ပြပါမယ်။

```
> db.issues.insert({
...   subject: "Dropdown menu not working",
...   priority: {
...     value: 2,
...     label: "Important"
...   },
...   severity: {
...     value: 1,
...     label: "Minor"
...   },
```

```
...     assignedTo: {
...       name: "John Doe",
...       userId: 1
...     },
...     status: {
...       value: 1,
...       label: "Assigned"
...     }
... })
>
```

`issues` Collection ထဲကို Document တစ်ခုထည့်သွင်းလိုက်ခြင်းဖြစ်ပါတယ်။ အဲ့ဒီလိုထည့်သွင်းရာမှာ လိုအပ်တဲ့ ဆက်စပ် အချက်အလက် အားလုံးကို ထည့်သွင်းထားတာကို သတိပြုပါ။ ဥပမာ – `priority` အတွက် `value: 2` နဲ့ `label: "Import"` လို့ တစ်ခါတည်း အပြည့်အစုံ ထည့်သွင်းထားပါတယ်။ RDBMS တွေမှာ ဆိုရင် `priority: 2` လို့သာထည့်သွင်းပြီး 2 ရဲ့ Label ကိုသိချင်ရင် Priority Table နဲ့ JOIN လုပ်ယူရမှာဖြစ်ပါတယ်။ ဒီနေရာမှာတော့ အဲ့ဒီလို Relationship ကို ရှောင်ချင်တဲ့အတွက် Priority တန်ဖိုး 2 ရော Label ဖြစ်တဲ့ Important ကိုပါ တစ်ခါတည်း တွဲဖက်ထည့်သွင်းထားခြင်းဖြစ်ပါတယ်။ တစ်ပြိုင်တည်း Document နှစ်ခုသုံးခုကိုထည့်သွင်းလိုရင် ထည့်သွင်းလိုတဲ့ Array အနေနဲ့ ထည့်သွင်းနိုင်ပါတယ်။ ဥပမာ –

```
> db.users.insert([
... { name: "John Doe", role: "Admin" },
... { name: "James Smith", role: "Tester" }
... ])
>
```

ထည့်သွင်းထားတဲ့ Document တွေကို ပြန်လည်ကြည့်ရှုလိုရင်တော့ `find()` ကိုအသုံးပြုနိုင်ပါတယ်။

```
> db.users.find()

{ "_id" : ObjectId("55350aea459cc324c59cf1aa"), "name" : "John Doe", "role" :
"Admin" }
{ "_id" : ObjectId("55350aea459cc324c59cf1ab"), "name" : "James Smith", "role"
: "Tester" }

>
```

ရလာတဲ့ရလဒ်ကိုသတိထားကြည့်ရင် Document တွေအတွက် `_id` ဆိုတဲ့ Attribute တစ်ခုပါဝင်နေတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။ Document တိုင်းမှာ `_id` Attribute ရှိရပါတယ်။ ကျွန်တော်တို့ Document တွေထည့်သွင်းစဉ်၊ `_id` ထည့်သွင်း မပေးလိုက်တဲ့အတွက် MongoDB က အလိုအလျောက် Auto Generate လုပ်ပြီး ထည့်သွင်းပေးသွားခြင်းဖြစ် ပါတယ်။ Generate လုပ်လိုက်တဲ့ `_id` ဟာ Unique ဖြစ်နေစေဖို့အတွက် MongoDB က အခုလို Formate ကို အသုံးပြုပါတယ်။

- 4 bytes Timestamps
- 3 bytes Machine ID
- 2 bytes Process ID
- 3 bytes Auto-Increment Counter

`_id` ဟာ ဒီတန်ဖိုး (၄) မျိုးကိုပေါင်းစပ်ထားတဲ့ 12 bytes Hexadecimal String တစ်ခုဖြစ်ပါတယ်။ Document တွေကိုစီမံတဲ့ အခါ ဒီ `_id` ကို Primary Key အနေနဲ့ အသုံးပြုစီမံနိုင်စေမှာဖြစ်ပါတယ်။ MongoDB က Generate လုပ်ပေးတဲ့ ID ကို မသုံးပဲ ကိုယ်တိုင် ID တွေသတ်မှတ်အသုံးပြုလိုတယ်ဆိုရင်လည်း Document ထည့်သွင်းစဉ်မှာ `_id` Attribute တွဲဖက်ထည့်သွင်း ပေးလိုက်ယုံပဲဖြစ်ပါတယ်။ ပေးလိုက်တဲ့တန်ဖိုးဟာ Unique ဖြစ်ဖို့တော့လိုပါတယ်။

ပြီးခဲ့တဲ့ `find()` ရလဒ်ကို ပြန်လည်လေ့လာကြည့်ရင် `_id` တန်ဖိုးကို `ObjectId()` Function နဲ့ တွဲဖက်ပေးထား တာကို သတိပြုမိနိုင်ပါတယ်။ `ObjectId()` Function နဲ့ တွဲဖက်ထားတဲ့အတွက် ID ကနေ Timestamps ကို ပြန်လည်ထုတ်ယူနိုင်တဲ့ အားသာချက်နဲ့၊ Sorting စီတဲ့အခါ Timestamps ကိုအခြေခံစီပေးသွားတဲ့အတွက် ပိုမိုမှန်ကန်တဲ့ အားသာချက်တို့ကို ရရှိနိုင်ပါတယ်။ ဥပမာ –

```
> db.users.find()

{ "_id" : ObjectId("55350aea459cc324c59cf1aa"), "name" : "John Doe", "role" :
"Admin" }
{ "_id" : ObjectId("55350aea459cc324c59cf1ab"), "name" : "James Smith", "role"
: "Tester" }

> ObjectId("55350aea459cc324c59cf1aa").getTimestamp()

ISODate("2015-04-20T14:19:22Z")

> ObjectId("55350aea459cc324c59cf1aa").str

55350aea459cc324c59cf1aa

>
```

`ObjectId()` Function ကနေတစ်ဆင့် `getTimestamp()` ကိုသုံးပြီး ID ထဲကအချိန်ကို ပြန်လည်ရယူနိုင် ခြင်း ဖြစ်သလို၊ `str` Attribute ကိုသုံးပြီး ID ကို String တန်ဖိုးအဖြစ် ပြောင်းယူနိုင်မှာဖြစ်ပါတယ်။

Collection ထဲက Document တစ်ခုကို ပြု ပြင်လိုရင် `save()` ကို အသုံးပြုရပါတယ်။ ဥပမာ –

```
> db.users.find()

{ "_id" : ObjectId("55350aea459cc324c59cf1aa"), "name" : "John Doe", "role" :
"Admin" }
{ "_id" : ObjectId("55350aea459cc324c59cf1ab"), "name" : "James Smith", "role"
: "Tester" }

> db.users.save({
... _id : ObjectId("55350aea459cc324c59cf1aa"),
... name: "John Doe",
... role: "Developer"})

> db.users.find()

{ "_id" : ObjectId("55350aea459cc324c59cf1aa"), "name" : "John Doe", "role" :
"Developer" }
{ "_id" : ObjectId("55350aea459cc324c59cf1ab"), "name" : "James Smith", "role"
: "Tester" }

>
```

နမူနာမှာ User John Doe ရဲ့ `role` ကို Admin ကနေ Developer လို့ `save()` Function သုံးပြီးပြောင်းထားပါတယ်။ `save()` Function က `_id` ကိုကြည့်ပြီး မူလကအဲ့ဒီ ID နဲ့ရှိနေတဲ့ Document ကို Update လုပ်ပေးသွားမှာ ဖြစ်ပါတယ်။

`find()` Function ကပြန်ပေးတဲ့ရလဒ်ကို Format လုပ်ပြီး သပ်သပ်ရပ်ရပ်ဖော်ပြစေလိုရင် `pretty()` နဲ့ တွဲသုံးရပါ တယ်။

```
> db.users.find().pretty()

{
      "_id" : ObjectId("55350aea459cc324c59cf1aa"),
      "name" : "John Doe",
      "role" : "Developer"
}
{
      "_id" : ObjectId("55350aea459cc324c59cf1ab"),
      "name" : "James Smith",
      "role" : "Tester"
}

>
```

`find()` Function နဲ့အတူ Query တွေလည်းတွဲဖက် ပေးနိုင်ပါတယ်။ ဥပမာ - `users` Collection ထဲက `role: "Tester"` တွေကို ရရှိလိုတဲ့အခါ အခုလိုအသုံးပြုနိုင်ပါတယ်။

```
> db.users.find({ role: "Tester" } )
```

တစ်ချို့တန်ဖိုးတွေက Nested Object အနေနဲ့ Document ထဲမှာသိမ်းဆည်းထားတာပါ။ ကျွန်တော်တို့ `insert()` နမူနာအနေနဲ့ ထည့်သွင်းခဲ့တဲ့ Document မှာဆိုရင် `priority`, `status` စတဲ့ တန်ဖိုးတွေဟာ ရိုးရိုးတန်ဖိုးမဟုတ်ပဲ Object တွေဖြစ်ပါတယ်။ ဒီလို Nested Object ထဲက တန်ဖိုးတစ်ခုခုနဲ့လည်း Query လုပ်နိုင်ပါတယ်။ ဥပမာ – Status ရဲ့ Nested Object မှာ Value 1 တန်ဖိုးရှိတဲ့ Document တွေကို အခုလို Query လုပ်ယူနိုင်ပါတယ်။

```
> db.issues.find({ "status.value": 1 })
```

AND ပုံစံနဲ့ တန်ဖိုးတစ်ခုထက်ပိုသုံးပြီးတော့လည်း Query လုပ်နိုင်ပါတယ်။ ဥပမာ –

```
> db.issues.find({ "status.value": 1, "severity.value": 3 })
```

OR ပုံစံ Query လုပ်လိုရင်တော့ အခုလို အသုံးပြုနိုင်ပါတယ်။

```
> db.issues.find({
... $or: [
...   { "status.value": 1 },
...   { "severity.value": 3 }
... ]
... });
>
```

Greater Than, Less Than စတဲ့ Comparison တွေပြုလုပ်လိုရင် အခုလိုပြုလုပ်နိုင်ပါတယ်။

```
> db.issue.find({
...   "status.value": { $gt: 1 }
... })
>
```

`$gt` က Greater Than ဆိုတဲ့အဓိပ္ပါယ်ဖြစ်ပြီး နမူနာအရ `status.value` တန်ဖိုး 1 ထက်ကြီးတဲ့ Document တွေကို ရယူပေးမှာ ဖြစ်ပါတယ်။ `$gt` အစား `$lt`, `$gte`, `$lte`, `$ne` တို့ကို အစားထိုးအသုံးပြုနိုင်ပါတယ်။ `$gte` က Greater Than or Equal ဖြစ်ပြီး `$ne` ကတော့ Not Equal ဖြစ်ပါတယ်။

RDBMS တွေရဲ့ LIKE Statement လိုသဘောမျိုး Filter လိုချင်ရင်တော့ Regular Expression ကိုအသုံးပြုရပါတယ်။

```
> db.issues.find({ subject: /menu/ })
```

နမူနာအရ `subject` Attribute ထဲမှာ `menu` ဆိုတဲ့စာလုံးပါဝင်တဲ့ Document အားလုံးကို ပြန်ပေးသွားမှာဖြစ်ပါတယ်။ Regular Expression အကြောင်းအသေးစိတ်ကိုတော့ ဒီနေရာမှာ မဖော်ပြနိုင်ပါတယ်။ ပို့ပြီး အသေးစိတ်တဲ့ Filter တွေ ဆောင်ရွက်နိုင်ဖို့ အတွက် Regular Express ကို လေ့လာပြီး `find()` Function နဲ့ အတူ တွဲဖက်အသုံးချလို့ရပါတယ်။

RDBMS တွေရဲ့ SELECT Query မှာ Field List ကို သတ်မှတ်နိုင်သလိုပဲ `find()` Function နဲ့ Query လုပ်တဲ့အခါ Document တစ်ခုလုံးမဟုတ်ပဲ၊ လိုအပ်တဲ့ Attribute တွေကိုသာ ရွေးချယ်ရယူနိုင်ပါတယ်။ ဥပမာ –

```
> db.users.find({}, { name: 1 })

{ "_id" : ObjectId("5537b3036831dbd2140f548e"), "name" : "John Doe" }
{ "_id" : ObjectId("5537ba0b6831dbd2140f548f"), "name" : "James Smith" }

>
```

`find()` Function ရဲ့ ဒုတိယ Parameter အနေနဲ့ Attribute List ကိုပေးရခြင်းဖြစ်ပါတယ်။ ရွေးချယ်လိုတဲ့ Attribute တန်ဖိုးကို 1 လို့ သတ်မှတ်ပေးရခြင်းဖြစ်ပါတယ်။ ထူးခြားချက်အနေနဲ့၊ `_id` က Field List ထဲမှာ အမြဲပါဝင် နေမှာဖြစ်ပါတယ်။

`find()` Function ကပြန်ပေးတဲ့ ရလဒ်မှာပါဝင်တဲ့ Document အရေအတွက်ကို ကန့်သတ်လိုရင် `limit()` Function နဲ့ တွဲသုံးနိုင်ပါတယ်။ ဥပမာ –

```
> db.users.find().limit(2)
```

`limit(2)` လို့ပြောထားတဲ့အတွက် `find()` Function ကပြန်ပေးလာတဲ့ Document တွေထဲက ပထမဆုံး (၂) ခုကိုပဲ ဖော်ပြပေးသွားမှာဖြစ်ပါတယ်။ RDBMS တွေမှာ Offset အနေနဲ့ စတင်ဖော်ပြရမယ့် Index ကိုသတ်မှတ်နိုင်သလိုပဲ MongoDB မှာလည်း သတ်မှတ်နိုင်ပါတယ်။ `skip()` Function ကို သုံးရပါတယ်။ ဥပမာ –

```
> db.users.find().skip(5).limit(10)
```

နမူနာအရ `users` Collection ထဲက Document (၁၀) ခုကိုရယူပေးမှာဖြစ်ပါတယ်။ အဲ့ဒီလိုရယူရာမှာ ပထမဆုံး (၅) ခုကိုကျော်ပြီး (၆) ခုမြောက်ကနေ စတင်ရယူပေးသွားမှာဖြစ်ပါတယ်။ `find()` Function ကပြန်ပေးတဲ့ရလဒ်ကို Sorting စီပေးစေလိုရင် `sort()` Function နဲ့ တွဲသုံးနိုင်ပါတယ်။ ဥပမာ –

```
> db.issues.find().sort({ priority: 1, status: 1 })
```

နမူနာအရ `issues` Collection ထဲက Document တွေကို `priority` နဲ့ ငယ်စဉ်ကြီးလိုက်စီပေးသွားမှာဖြစ်ပါတယ်။ `priority` တူနေတဲ့ Document တွေကိုတော့ `status` နဲ့စီပေးသွားမှာဖြစ်ပါတယ်။ ကြီးစဉ်ငယ်လိုက်ပြောင်းပြန်စီ စေလိုရင်တော့ 1 အစား -1 ကို သုံးနိုင်ပါတယ်။

Collection ထဲက Document တစ်ခုကို ပယ်ဖျက်လိုရင်တော့ `remove()` Function ကိုသုံးနိုင်ပါတယ်။ ပယ်ဖျက်လိုတဲ့ Document အတွက် Query ထည့်သွင်းပေးဖို့လိုပါတယ်။ ဥပမာ – `users` Collection ထဲက `role: "Tester"` သတ်မှတ်ထားတဲ့ Document အားလုံးကို ပယ်ဖျက်လိုရင် အခုလိုပယ်ဖျက်နိုင်ပါတယ်။

```
> db.users.remove({ role: "Tester" })
```

Query နဲ့ကိုက်ညီတဲ့ Document တွေထဲက ပထမဆုံးတစ်ခုကိုသာ ပယ်ဖျက်လိုရင် အခုလိုပယ်ဖျက်နိုင်ပါတယ်။

```
> db.users.remove({ role: "Tester"}, 1);
```

`remove()` ကို Query မပေးပဲ Run ရင်တော့ ရှိသမျှ Document အားလုံးကို ပယ်ဖျက်သွားမှာဖြစ်ပါတယ်။

`save()` Function ကိုသုံးပြီး လက်ရှိ Document ကို Overwrite လုပ်နိုင်ပုံကို အထက်မှာဖော်ပြခဲ့ပါတယ်။ အဲ့ဒီလို Overwrite မလုပ်ပဲ တန်ဖိုးတစ်ချို့ကိုသာ ပြင်လိုရင်တော့ `update()` ကိုသုံးရပါတယ်။ ဥပမာ –

```
> db.users.update(
...    { role: "Developer" },
...    { $set: { role: "Programmer" } }
...    { multi: true }
... )
>
```

နမူနာအရ `role: "Developer"` သတ်မှတ်ထားတဲ့ Document အားလုံးကို `role: "Programmer"` တန်ဖိုး နဲ့ Update လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ `update()` Function အတွက် Parameter (၃) ခုပေးရပါတယ်။ ပထမ တစ်ခုက Query ပါ။ ဒုတိယတစ်ခုက အစားထိုးလိုတဲ့ Value Set ဖြစ်ပြီး၊ နောက်ဆုံးတစ်ခုကတော့ Option ဖြစ်ပါတယ်။ အကယ်၍ Query နဲ့ ကိုက်ညီသမျှ Document အားလုံးကို Update မလုပ်စေလိုပဲ၊ ပထမဆုံးတစ်ခုကိုသာ Update ပြုလုပ်စေလိုရင်တော့ နောက်ဆုံးက `multi: true` Option ကို `false` ပြောင်းထားနိုင်ပါတယ်။

RDBMS စနစ်တွေမှာ SQL Query တွေကိုသုံးပြီး Create, Read, Update, Delete (CRUD) လုပ်ငန်းတွေကို

ဆောင်ရွက်နိုင်သလိုပဲ MongoDB မှာလည်း `insert()`, `find()`, `update()`, `remove()` Function များကို အသုံးပြုပြီး CRUD လုပ်ငန်းများကို ဆောင်ရွက်နိုင်ခြင်းပဲ ဖြစ်ပါတယ်။

11.5 – Indexing in MongoDB

RDBMS စနစ်များမှာ Read Performance ကောင်းမွန်ဖို့အတွက် Index တွေသတ်မှတ်ထားနိုင်သလိုပဲ၊ MongoDB မှာလည်း Index တွေသတ်မှတ်ထားနိုင်ပါတယ်။ `createIndex()` Function ကိုသုံးရပါတယ်။ ဥပမာ –

```
> db.issues.createIndex({ status: 1 })
```

`issues` Collection ထဲမှာ Index တည်ဆောက်လိုက်ခြင်းဖြစ်ပြီး Index ပြုလုပ်လိုတဲ့ Attribute အဖြစ် `status` ကို ရွေးချယ်လိုက်ခြင်းဖြစ်ပါတယ်။။ ဒါကြောင့် `status` နဲ့ Query လုပ်တဲ့အခါ Document တစ်ခုစီ လိုက်ရှာနေဖို့မလိုတော့ ပဲ Index ကို ကြည့်ပြီး ရလဒ်ပြန်ပေးနိုင်မှာဖြစ်လို့ Performance ပိုကောင်းသွားမှာပဲဖြစ်ပါတယ်။ Index သတ်မှတ်ရာမှာ Option တစ်ချို့ တွဲပေးနိုင်ပါတယ်။ ဥပမာ –

```
> db.issues.createIndex({ _id: 1 }, {
... name: "PrimaryKey",
... unique: true,
... sparse: true
})
>
```

`name` Option ကိုအသုံးပြုပြီး Index Name သတ်မှတ်နိုင်ပါတယ်။ အကယ်၍ Index Name မသတ်မှတ်ပေးခဲ့ရင်လည်း MongoDB က Index Name တစ်ခု Generate လုပ်ပြီး အလိုအလျောက်သတ်မှတ်ပေးသွားမှာ ဖြစ်ပါတယ်။ `unique` Option ကို `true` လို့သတ်မှတ်ထားရင်တော့၊ ရိုးရိုး Index မဟုတ်တော့ပဲ Unique Index ဖြစ်သွားမှာ ဖြစ်ပါတယ်။ Index လုပ်လိုတဲ့ Attribute မပါဝင်တဲ့ Document တွေလည်း Collection ထဲမှာ ရှိနေနိုင်ပါတယ်။ အဲ့ဒီလို Document တွေကို Index ထည့်မလုပ်စေလိုရင် `sparse` Option ကို `true` လို့သတ်မှတ်ပေးရပါတယ်။

Index တွေကိုပြန်လည်ပယ်ဖျက်စေလိုတဲ့အခါ `dropIndex()` Function နဲ့ပယ်ဖျက်နိုင်ပါတယ်။ ပယ်ဖျက်လိုတဲ့ Index Name (သို့မဟုတ်) Index လုပ်ထားတဲ့ Attribute ကို ပေးရပါတယ်။ ဥပမာ –

```
> db.issues.dropIndex("PrimaryKey")
> db.issues.dropIndex({ status: 1 })
```

ရှိသမျှ Index အားလုံးကို ပယ်ဖျက်စေလိုရင်တော့ `dropIndexes()` ကို သုံးနိုင်ပါတယ်။ တည်ဆောက်ထားတဲ့ Index တွေရဲ့ စာရင်းကိုတော့ `getIndexes()` နဲ့ ရယူနိုင်ပါတယ်။

```
> db.issues.getIndexes()
```

MySQL MyISAM Storage Engine မှာ Text Search အတွက် Full-text Index လုပ်ဆောင်ချက်ပါဝင်သလိုပဲ MongoDB မှာလည်း Text Index လုပ်ဆောင်ချက်ပါဝင်ပါတယ်။ Text Index က Index ပြုလုပ်ယုံသာမက၊ Search Engine တစ်ခုရဲ့ အခြေခံလုပ်ဆောင်ချက်တွေဖြစ်တဲ့ Noise တွေ၊ Stop Words တွေဖယ်ထုတ်ပေးခြင်းနဲ့ Stemming လို လုပ်ဆောင်ချက်မျိုး ၏တွေကို တစ်ခါတည်း ဆောင်ရွက်ပေးနိုင်ပါတယ်။ Stemming ဆိုတာ၊ ဥပမာပြောရရင် wait လို့ရှာရင် waiting, waited, waits စတဲ့စာလုံးတွေကိုပါ အလိုအလျောက် ထည့်သွင်းရှာဖွေပေးတဲ့လုပ်ဆောင်ချက်မျိုး ဖြစ်ပါတယ်။ Text Index တစ်ခုကို အခုလို သတ်မှတ်နိုင်ပါတယ်။

```
> db.issues.createIndex({ subject: "text" })
```

ရိုးရိုး Index အတွက် 1 ကိုအသုံးပြု ရပြီး Text Index အတွက် text ကို အသုံးပြုရခြင်းဖြစ်ပါတယ်။ အကယ်၍ Text Index သတ်မှတ်ရာမှာ { "ok": 0, "errmsg": "text search not enabled" } ဆိုတဲ့ Error Message ရခဲ့ရင် Text Index လုပ်ဆောင်ချက်ကို အခုလို Enable လုပ်ပေးဖို့လိုပါတယ်။

```
> db.adminCommand({ setParameter: "*", textSearchEnabled: true })

{ "was" : false, "ok" : 1 }
```

Text Search ကိုလည်း Enable လုပ်ပြီးပြီး Text Index လည်းလုပ်ပြီးပြီဆိုရင် Text Index ထဲကနေ ရှာဖွေခြင်းလုပ် ငန်းကို `runCommand()` နဲ့ ဆောင်ရွက်နိုင်မှာ ဖြစ်ပါတယ်။ ဥပမာ –

```
> db.issues.runCommand("text", { search: "menu" })

{
      "queryDebugString" : "menu||||||",
      "language" : "english",
      "results" : [
            {
                  "score" : 0.6666666666666666,
                  "obj" : {
                        "_id" : ObjectId("553508f5459cc324c59cf1a9"),
                        "subject" : "Dropdown menu not working",
                        "priority" : {
                              "value" : 2,
                              "label" : "Important"
                        },
                        "assignedTo" : {
                              "name" : "John Doe",
                              "id" : 1
                        },
                        "status" : {
                              "value" : 1,
                              "label" : "Assigned"
```

```
                        }
                }
        }
    ],
    "stats" : {
            "nscanned" : 1,
            "nscannedObjects" : 0,
            "n" : 1,
            "nfound" : 1,
            "timeMicros" : 197
    },
    "ok" : 1
}

>
```

`runCommand()` Function ကိုသုံးပြီး `menu` ဆိုတဲ့ Keyword ကို Text Index ထဲမှာ ရှာကြည့်လိုက်တဲ့အခါ၊ `menu` Keyword ပါဝင်တဲ့ Document တွေကို ဖော်ပြပေးလာမှာဖြစ်ပါတယ်။ `result` ထဲက `score` ကို သတိပြု ကြည့်သင့်ပါတယ်။ `score` မှာဖော်ပြနေတဲ့တန်ဖိုးဟာ Relevant Value ဖြစ်ပြီး ရှာဖွေလိုတဲ့ Keyword နဲ့ လက်ရှိ Document ဘယ်လောက်ထိ Relevant ဖြစ်သလဲဆိုတာကို ဖော်ပြနေခြင်းဖြစ်ပါတယ်။ ဒီ `score` ကို သုံးပြီး Sorting စီလိုက်မယ်ဆိုရင် Relevant အဖြစ်ဆုံးရလဒ်ကို အရင်ဖော်ပြပေးနိုင်တဲ့ Search Result ကို ရရှိမှာပဲဖြစ်ပါတယ်။

ဒီနည်းနဲ့ RDBMS တွေမှာ ရရှိလေ့ရှိတဲ့ Index, Unique Index နဲ့ Full Text Index တို့ကို MongoDB မှာလည်း ရရှိနိုင်မှာဖြစ်ပါတယ်။

11.6 – Access Control in MongoDB

ကျွန်တော်တို့ လက်ရှိဆောင်ရွက်နေတဲ့ ကိစ္စတွေအားလုံးအတွက် MongoDB က ဘာ Authentication Credential မှ မတောင်းတာကို သတိပြုမိပါလိမ့်မယ်။ Default အနေနဲ့ MongoDB မှာ Access Control ကို ကန့်သတ်မထားပါဘူး။ Access Control သတ်မှတ်လိုတယ်ဆိုရင်၊ ပထမဦးဆုံးအနေနဲ့ `admin` User တစ်ဦးကို စတင်သတ်မှတ်သင့်ပါတယ်။ အခုလို သတ်မှတ်နိုင်ပါတယ်။

```
> use admin

switched to db admin

> db.addUser("root", "root")

{
    "user" : "root",
    "readOnly" : false,
    "pwd" : "2a8025f0885adad5a8ce0044070032b3",
    "_id" : ObjectId("5536899f1d331f653a28893e")
}

>
```

ပထမဦးဆုံး `admin` Database ကိုရွေးထားပေးရပါတယ်။ ပြီးတော့မှ `db.addUser()` နဲ့ Admin User Account တစ်ခုကို တည်ဆောက်နိုင်ခြင်းဖြစ်ပါတယ်။ နမူနာအရ Username: root နဲ့ Password: root လို့ သတ်မှတ်ထား ပါတယ်။

သတ်မှတ်ထားတဲ့ Access Control စတင်အလုပ်လုပ်စေဖို့အတွက် `/etc/mongodb.conf` ဖိုင်ထဲမှာ `auth=true` ဆိုတဲ့ Setting တစ်ခု သတ်မှတ်ပေးရပါတယ်။ ပြီးတဲ့အခါ MongoDB ကို Restart လုပ်ပေးရပါတယ်။

```
$ sudo service mongodb restart
```

အခုနေ Mongo Shell ကိုပြန်ဝင်ပြီး စမ်းကြည့်ရင် Access Control Error ကိုတွေ့ရမှာဖြစ်ပါတယ်။

```
> use admin

switched to db admin

> show collections

Wed Apr 22 00:11:07.267 error: {
     "$err" : "not authorized for query on admin.system.namespaces",
     "code" : 16550
} at src/mongo/shell/query.js:128

>
```

သတ်မှတ်ထားတဲ့ Username, Password မှန်အောင်ပေးပြီးမှသာ အချက်အလက်တွေကို စတင်စီမံနိုင်တော့မှာ ဖြစ်ပါ တယ်။ `db.auth()` Function ကိုသုံးပြီး Authenticate လုပ်နိုင်ပါတယ်။

```
> db.auth("root", "root")

1

> show collections

system.indexes
system.users
```

Admin User မဟုတ်တဲ့ ရိုးရိုး User Account ကိုတော့ သက်ဆိုင်ရာ Database ပေါ်မှာ အခုလိုတည်ဆောက်ပေးနိုင်ပါ တယ်။

```
> use mydb

switched to db mydb

> db.addUser("user", "user")

{
      "user" : "user",
      "readOnly" : false,
      "pwd" : "fa26a506aa0f786a447bbd6d1caaa8b5",
      "_id" : ObjectId("55368c6c3e0d2164a31a2ec8")
}

>
```

Admin User နဲ့ Database အားလုံးကိုစီမံနိုင်ပေမယ့် ရိုးရိုး User ကတော့ သူ့အတွက်သတ်မှတ်ပေးထားတဲ့ Database ကိုသာ စီမံခွင့်ရမှာဖြစ်ပါတယ်။

```
> use admin

switched to db admin

> db.auth("user", "user")

Error: 18 { code: 18, ok: 0.0, errmsg: "auth fails" }
0

>
```

`mydb` Database အတွက် တည်ဆောက်ထားတဲ့ User နဲ့ `admin` Database ကို Authenticate လုပ်ဖို့ကြိုးစားတဲ့အခါ auth fails Error ကို ရရှိခြင်းဖြစ်ပါတယ်။ Data တွေကို Access လုပ်လို့ရပေမယ့် အသစ်တည်ဆောက်ခွင့်မရှိတဲ့ Readonly User Account တွေလည်းတည်ဆောက်နိုင်ပါတယ်။ `addUser()` Function ရဲ့ တတိယ Parameter ကို true လို့သတ်မှတ်ပေး ရပါတယ်။

```
> db.addUser("user", "user", true)

{
      "user" : "user",
      "readOnly" : 1,
      "pwd" : "fa26a506aa0f786a447bbd6d1caaa8b5",
      "_id" : ObjectId("55368d46d3bc6702df267302")
}

>
```

Readonly User ဟာ အချက်အလက်တွေကို ရယူနိုင်ပေမယ့်၊ အသစ်တည်ဆောက်ခွင့် ရမှာမဟုတ်ပါဘူး။

```
> use mydb

switched to db mydb

> db.auth("user", "user)
> show collections

issues
system.indexes
system.users
users

> db.createCollection('comments')

{ "ok" : 0, "errmsg" : "unauthorized" }

>
```

အခုဖော်ပြခဲ့တဲ့ Access Control လုပ်ဆောင်ချက်ဟာ MongoDB Version 2.4 အတွက်ဖြစ်ပါတယ်။ MongoDB Version 2.6 ကနေနောက်ပိုင်းမှာ Access Control သတ်မှတ်ပုံ ပြောင်းလဲသွားပါတယ်။ ဥပမာ – `addUser()` Function ကိုမသုံးတော့ပဲ User Account တွေရော၊ Role တွေကိုပါ တစ်ခါတည်း သတ်မှတ်နိုင်တဲ့ `createUser()` Function ကို ပြောင်းသုံးသွားပါတယ်။ ပြောင်းလဲသွားတဲ့ Access Control သတ်မှတ်ပုံကို အောက်ပါလိပ်စာမှာ ဆက်လက်လေ့လာနိုင်ပါတယ်။

http://docs.mongodb.org/manual/tutorial/add-user-to-database/

11.7 – MongoDB Backup and Restore

MongoDB Database ထဲက အချက်အလက်တွေကို `mongodump` Tool အသုံးပြုပြီး Backup ပြုလုပ်နိုင်ပါတယ်။ ဥပမာ -

```
$ mongodump -d mydb -o /path/to/output
```

`-d` Option နဲ့ Backup လုပ်လိုတဲ့ Database ကိုပေးရပြီး `-o` Option နဲ့ Backup ဖိုင်တွေသိမ်းပေးရမယ့် Directory ကို သတ်မှတ်ပေးရပါတယ်။ `-d` Option နဲ့ Database မသတ်မှတ်ခဲ့ရင် ရှိသမျှ Database အားလုံးကို Backup လုပ်ပေးသွားမှာ ဖြစ်ပါတယ်။ `-o` Option နဲ့ Output Directory သတ်မှတ်မပေးရင်တော့ လက်ရှိ Command ကို Run တဲ့ Directory ထဲမှာ Backup ဖိုင်တွေကို သိမ်းပေးသွားမှာဖြစ်ပါတယ်။

Database မှာ Access Control သတ်မှတ်ထားရင် Username, Password တစ်ခါတည်းပေးရပါတယ်။ မပေးရင် auth fails Error Message ကိုပေးပါလိမ့်မယ်။ Username ကို `-u` Option နဲ့ပေးရပြီး Password ကို `-p` Option နဲ့ ပေးရပါတယ်။ ဥပမာ –

```
$ mongodump -u username -p password -d mydb -o /path/to/output
```

အကယ်၍ Collection တစ်ခုတည်းကိုရွေးပြီး Backup ပြုလုပ်လိုရင်တော့ `-c` Option ကို သုံးနိုင်ပါတယ်။ ဥပမာ –

```
$ mongodump -d mydb -c issues -o /path/to/output
```

နမူနာအရ `mydb` Database ထဲက `issues` Collection တစ်ခုတည်းကို Backup ထုတ်ယူလိုက်ခြင်းပဲဖြစ်ပါတယ်။

`mongodump` နဲ့ Backup လုပ်ထားတဲ့ ဖိုင်တွေကို ပြန်ပြီး Restore လုပ်လိုရင်တော့ `mongorestore` ကို အသုံးပြုရ ပါတယ်။ Backup ဖိုင်တွေရှိနေတဲ့ Directory ထဲမှာ Run ပေးသင့်ပါတယ်။ အကယ်၍ တစ်ခြား Director ထဲမှာ Run မယ်ဆိုရင် Backup ဖိုင်တွေတည်ရှိရာနေရာကို တွဲပြီးညွှန်းပေးရပြီး Access Control တွေ ရှိနေရင်လည်း တွဲဖက်ပေးဖို့လို ပါတယ်။

```
$ mongorestore /path/to/backup -u username -p password
```

Restore လုပ်တဲ့အခါ မူလရှိနေတဲ့ Collection တွေကို Drop အရင်လုပ်ပြီးမှ Restore လုပ်စေလိုတဲ့အခါ `--drop` Option တွဲဖက်သက်မှတ်ပေးရပြီး Collection တွေအတွက် သုံးရမယ့် Database ကို `-d` Option နဲ့ တွဲဖက်ပေးရပါ တယ်။ ဥပမာ –

```
$ mongorestore /path/to/backup -d mydb --drop
```

နမူနာအရ Backup ဖိုင်ထဲက Collection တွေကို `mydb` Database ထဲမှာ Restore လုပ်ပေးသွားမှာဖြစ်ပြီး၊ `mydb` Database ထဲမှာ မူလကအမည်တူ Collection ရှိနေရင် အဲ့ဒီ Collection ကိုအရင် Drop လုပ်ပြီးမှ Restore လုပ်ပေး သွားမှာဖြစ်ပါတယ်။

Backup လုပ်ဖို့အတွက် `mongoexport` ကိုလည်း သုံးနိုင်ပါသေးတယ်။ `mongodump` က အချက်အလက်တွေကို Binary Data အနေနဲ့ Backup လုပ်ပေးပြီး `mongoexport` ကတော့ JSON (သို့မဟုတ်) CSV အနေနဲ့ Backup လုပ်ပေးပါတယ်။

```
$ mongoexport -d mydb -c issues -o export.json
```

နမူနာအရ `mydb` Database ထဲက `issues` Collection ကို JSON Format နဲ့ Export လုပ်ပေးသွားမှာဖြစ်ပါ

တယ်။ `-o` Option နဲ့ ရရှိလာတဲ့ ရလဒ်ကို `export.json` ဖိုင်ထဲမှာ ထည့်သွင်းပေးဖို့ သတ်မှတ်ထားပါတယ်။ `-o` Option သတ်မှတ်မထားရင် ဖိုင်အနေနဲ့မသိမ်းပဲ Terminal မှာ Output အနေနဲ့ ဖော်ပြလာမှာပါ။ CSV Format နဲ့ လိုချင်ရင် တော့ `--csv` Option နဲ့တွဲပြီး Export ထုတ်ယူရပါတယ်။

```
$ mongoexport -d mydb -c issues -f "subject,status" --csv -o
export.csv
```

ထူးခြားချက်အနေနဲ့ Export ထုတ်ယူလိုတဲ့ Field List ကို `-f` Option နဲ့ တွဲပေးရပါတယ်။ ထုံးစံအတိုင်း Access Control တွေသတ်မှတ်ထားရင်လည်း သက်ဆိုင်ရာ Username, Password ကို `-u`, `-p` Option တွေနဲ့ ထည့်သွင်း ပေးဖို့လိုပါတယ်။

`mongoexport` နဲ့ထုတ်ယူထားတဲ့ Backup ဖိုင်ကို ပြန်ပြီး Restore လုပ်လိုရင်တော့ `mongoimport` ကို သုံးရပါ တယ်။

```
$ mongoimport -d mydb -c issues < export.json
```

နမူနာအရ `mydb` Database ရဲ့ `issues` Collection ထဲကို `export.json` ဖိုင်ထဲက အချက်အလက်တွေနဲ့ Restore လုပ်ပေးသွားမှာ ဖြစ်ပါတယ်။ `mongorestore` မှာလိုပဲ နဂိုရှိနေတဲ့ Collection ကိုအရင် Drop လုပ်စေလို ရင် `--drop` Option တွဲပေးနိုင်ပါတယ်။

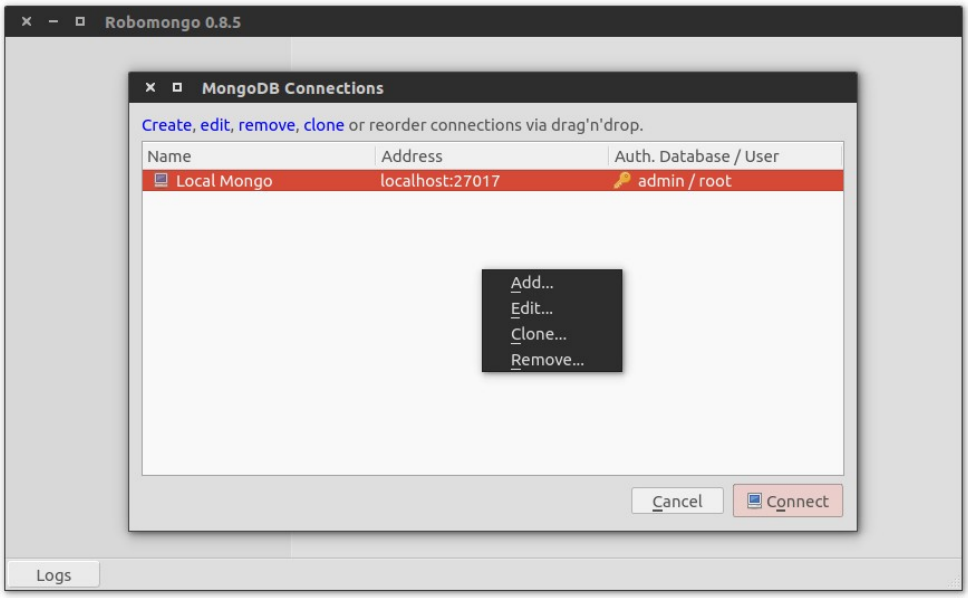
11.8 – MongoDB Administration Tool

MongoDB ကို Command Line ကနေစီမံမယ့်အစား Admin Tool တွေသုံးပြီးတော့လည်း စီမံနိုင်ပါတယ်။ အသုံးများ တဲ့ Admin Tool စာရင်းကို အောက်ပါလိပ်စာမှာ လေ့လာနိုင်ပါတယ်။

http://docs.mongodb.org/ecosystem/tools/administration-interfaces/

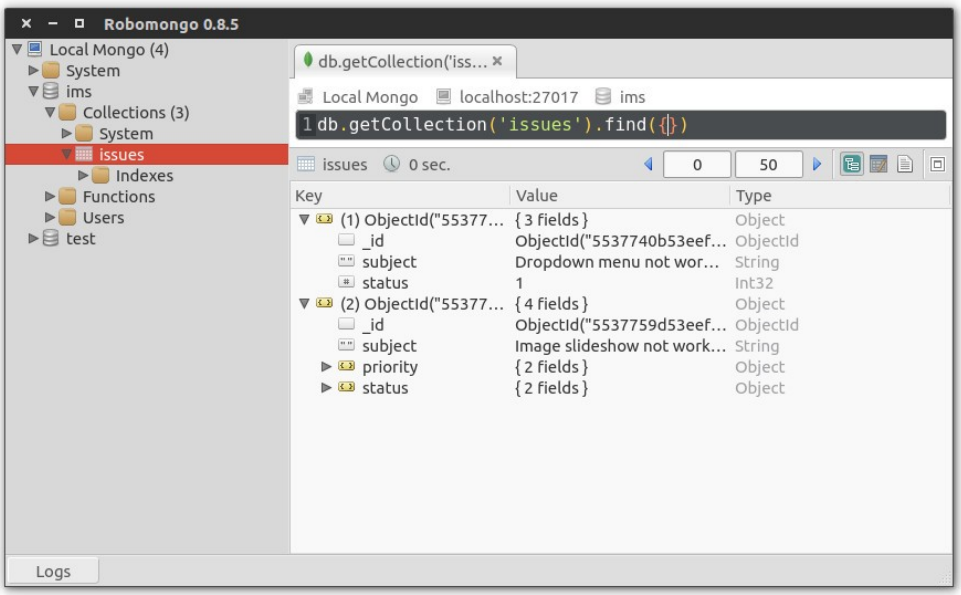
ဒီနေရာမှာတော့ နမူနာအနေနဲ့ Robomongo လို့ခေါ်တဲ့ Admin Tool တစ်ခုကို နမူနာအနေနဲ့ ဖော်ပြလိုပါတယ်။ Robomongo ဟာ Windows, Linux, Mac စတဲ့ Platform အားလုံးမှာ အလုပ်လုပ်တဲ့ Tool တစ်ခုဖြစ်ပါတယ်။ robomongo.org မှာ Installer ကို Download ရယူနိုင်ပါတယ်။

Robomongo ကို Install လုပ်ပြီးတဲ့အခါ ပထမဆုံးအနေနဲ့ Connection Setting သတ်မှတ်ပေးရပါတယ်။ **File → Connect** Menu ကို နှိပ်လိုက်ရင် အခုလို Connection Setting Manager ကိုတွေ့ရမှာဖြစ်ပါတယ်။



ပုံ (၁၁.၇) - Robomongo - Connection Manager

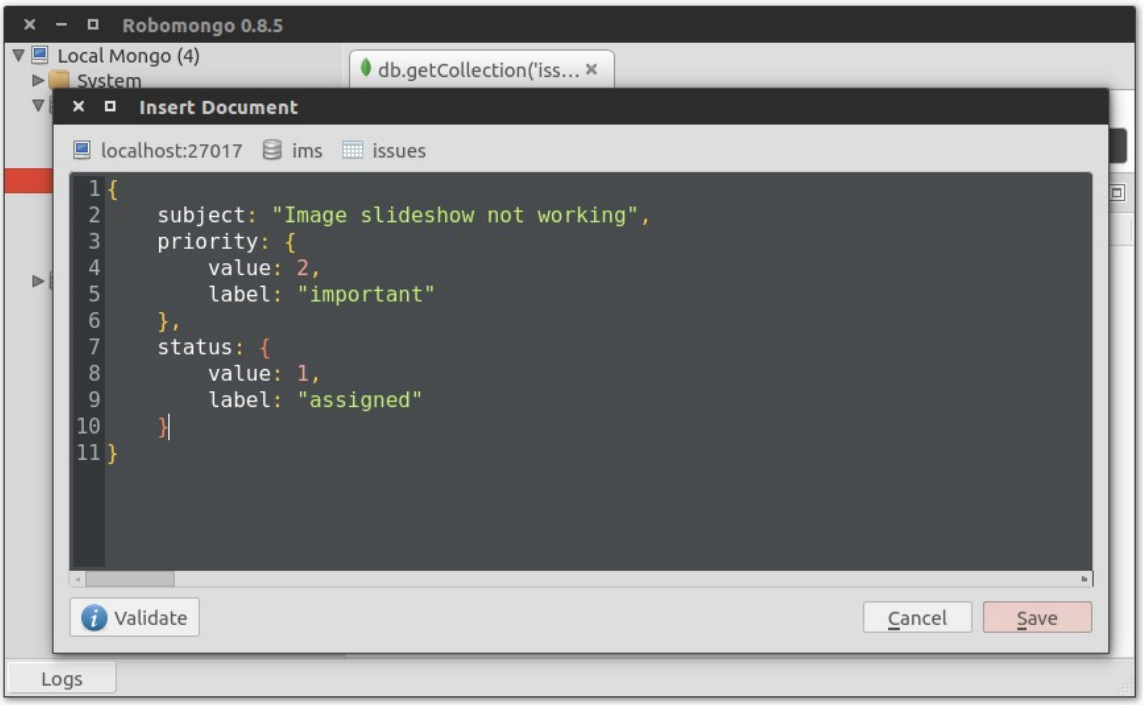
Right Click → Add ကို ရွေးပြီး Connection အသစ်တစ်ခုတည်ဆောက်နိုင်ပါတယ်။ ပေါ်လာတဲ့ Dialog Box ထဲမှာ Connection Name အနေနဲ့ နှစ်သက်ရာ အမည်ကိုပေးနိုင်ပါတယ်။ Address နေရာမှာ localhost နဲ့ Port နေရာမှာ 27017 ကိုပေးရပါတယ်။ အကယ်၍ Database မှာ Access Control သတ်မှတ်ထားတယ်ဆိုရင် Username, Password ကိုသတ်မှတ်ပေးရပါတယ်။ လိုအပ်တဲ့အချက်အလက်တွေသတ်မှတ်ပြီး ရရှိလာတဲ့ Connection ကိုရွေးပြီး Connect နှိပ်လိုက်ရင် အခုလိုရလဒ်ကို ရရှိမှာဖြစ်ပါတယ်။



ပုံ (၁၁.၈) - Robomongo - Main Interface

ပုံ (၁၁.၈) မှာပြထားသလို ဘယ်ဘက်ခြမ်းမှာ Database Explorer ဖော်ပြစေဖို့အတွက် **View → Explorer** ကို ရွေး ပေးဖို့လိုနိုင်ပါတယ်။ Database အသစ်ဆောက်ဖို့အတွက် Database Explorer ထဲက Connection ကို Right Click နှိပ်ပြီး Create Database ကိုရွေးခြင်းအားဖြင့် တည်ဆောက်နိုင်ပါတယ်။ Collection အသစ်တည်ဆောက်ဖို့အတွက် သက်ဆိုင်ရာ Database ကို Right Click နှိပ်ပြီး တည်ဆောက်နိုင်ပါတယ်။

ညာဘက်ခြမ်းမှာ လက်ရှိရွေးထားတဲ့ Collection ထဲက Document တွေကိုဖော်ပြပေးမှာဖြစ်ပါတယ်။ Document တွေ ကို Tree View, Table View, Document View စသဖြင့် View အမျိုးမျိုးနဲ့ကြည့်လို့ရပါတယ်။ Toolbar ထဲက Button တွေကို နှိပ်ပြီး View ကို အမျိုးမျိုးပြောင်းကြည့်နိုင်ပါတယ်။



ပုံ (၁၁.၉) - Robomongo - Document Editor

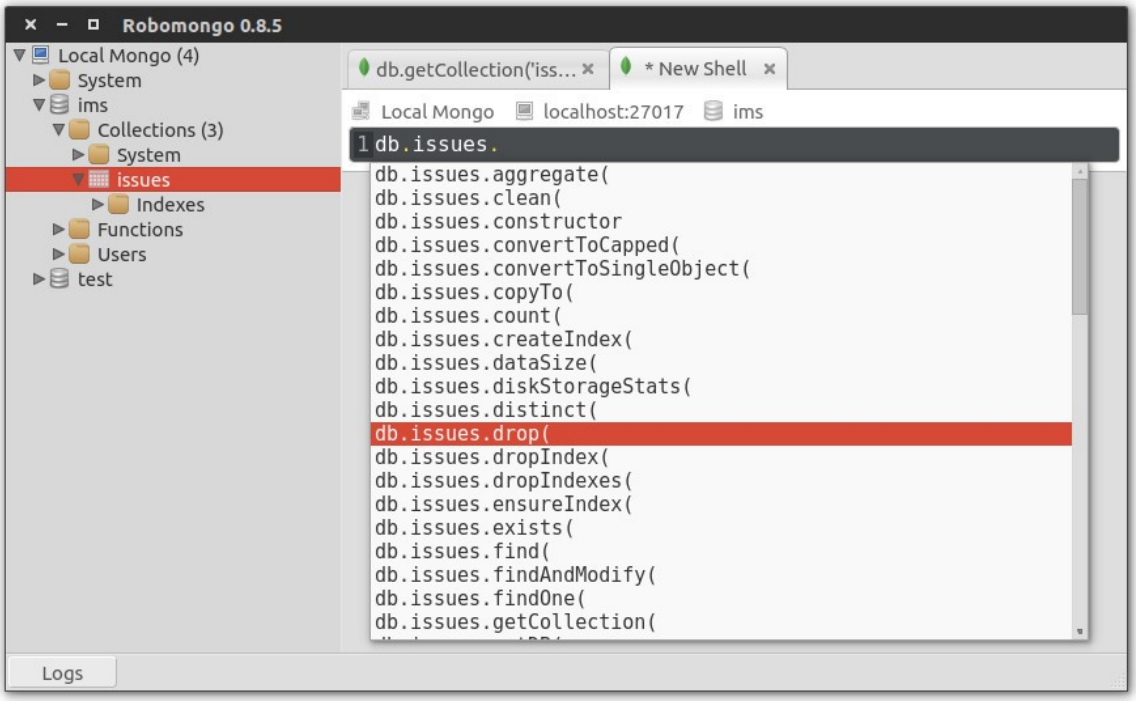
Document တွေကို Create, Read, Update, Delete လုပ်ဖို့အတွက် သက်ဆိုင်ရာ Document ကို Right Click လုပ်ပြီးဆောင် ရွက်နိုင်ပါတယ်။ Document အသစ်ထည့်ဖို့အတွက် Document စာရင်းနေရာလွတ်မှာ (သို့မဟုတ်) Database Explorer ထဲက Collection ပေါ်မှာ Right Click နှိပ်ပြီး **Insert Document** ကိုရွေးခြင်း အားဖြင့် တည် ဆောက်နိုင်ပါတယ်။ Robomongo က Document Editor နဲ့ Document အသစ်တစ်ခုထည့်သွင်းနိုင်အောင် စီမံပေး မှာဖြစ်လို့ Command Line Mongo Shell မှာထက် အသုံးပြုရတာ ပိုမိုလွယ်ကူအဆင်ပြေစေမှာပဲဖြစ်ပါတယ်။

Index တွေစီမံဖို့အတွက်လဲ သက်ဆိုင်ရာ Collection အောက်က Index Folder ကို Right Click နှိပ်ပြီးစီမံနိုင်ပါတယ်။ လက်တွေ့မှာ Robomongo က Mongo Shell ကိုပဲ အသုံးပြုထားခြင်းဖြစ်ပါတယ်။ ဆိုလိုတာက Mongo

Shell မှာ လုပ်လို့ ရသမျှ Robomongo မှာ လည်း ရနိုင်မှာပဲဖြစ်ပါတယ်။

Shell Command တွေ ကိုယ်တိုင်ရေးသားလိုရင် Tab Bar မှာ Right Click လုပ်ပြီး New Shell ကိုရွေးခြင်းအားဖြင့် Mongo Shell အသစ်တစ်ခုကို ဖွင့်နိုင်ပါတယ်။ ထူးခြားချက်အနေနဲ့ Robomono ရဲ့ Shell Window ထဲမှာ **ပုံ (၁၁.၁၀)** မှာ ပြထားသလို Auto-complete လုပ်ဆောင်ချက်ကို ရရှိနိုင်မှာပဲဖြစ်ပါတယ်။

ရေးသားထားတဲ့ Shell Command တွေကို Run ဖို့အတွက် Ctrl+R ကိုနှိပ်ပြီး Run နိုင်ပါတယ်။ ဒါမှမဟုတ် Tab ကို Right Click လုပ်ပြီး **Re-execute Query** ကိုရွေးပြီးတော့လည်း Run နိုင်ပါတယ်။



ပုံ (၁၁.၁၀) - Robomongo - Auto-Complete in Shell

လက်တွေ့မှာ Mongo Shell Command Line ကနေ စီမံနေမယ့်အစား အခုလို Admin Tool တွေကိုအသုံးပြုလိုက်တာ ပိုပြီးအလုပ်တွင်နိုင်ပါတယ်။ NoSQL Database တွေရဲ့ အဓိကအားသာချက်ဟာ အလွယ်တစ်ကူ Scale လုပ်နိုင်ခြင်း ဖြစ်ကြောင်းဖော်ပြခဲ့ပါတယ်။ MongoDB ကို Scale လုပ်ရာမှာ လိုအပ်တဲ့နည်းစနစ်တွေဖြစ်တဲ့ Replication, Shard စတဲ့ အကြောင်းအရာတွေကို မဖော်ပြရသေးပါဘူး။ **အခန်း (၁၉)** ရောက်တော့မှ ဆက်လက်ဖော်ပြပါမယ်။ တစ်ချို့ MongoDB Admin Tool တွေက Database တွေ Collection တွေကိုသာမက Replication နဲ့ Shard တွေကိုပါ စီမံပေးနိုင်ကြပါသေးတယ်။ ဒါကြောင့် အထက်မှာပေးထားတဲ့ Admin Tool စာရင်းလိပ်စာမှာ သွားရောက်လေ့လာပြီး အခြား Admin Tool တွေကိုလဲ လေ့လာစမ်းသပ်ကြည့်သင့်ပါတယ်။

11.9 – Using MongoDB with NodeJS

MongoDB ကို NodeJS နဲ့ တွဲဖက်အသုံးပြုနိုင်ဖို့အတွက် Official JavaScript Driver ကို အခုလို Install လုပ်ယူနိုင်ပါတယ်။

```
$ npm install mongodb
```

အဲ့ဒီ Official Driver ကိုပဲ Mongo Shell Command တွေနဲ့ အနီးစပ်ဆုံးတူအောင် ဖြည့်စွက်ထားတဲ့ `mongojs` ဆိုတဲ့ NPM Package တစ်ခုလည်းရှိပါသေးတယ်။ လက်တွေ့မှာ Mongo Shell တွေနဲ့ ပိုမိုနီးစပ်တဲ့အတွက် `mongojs` က Official Driver ထက် သုံးရတာပိုအဆင်ပြေပါတယ်။ နမူနာအနေနဲ့ `mongojs` ကိုအသုံးပြုဖော်ပြပေးသွားပါမယ်။ `mongojs` ကို အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ npm install mongojs
```

Project မှာအသုံးချလိုတဲ့ NPM Package တွေကို Local Module အဖြစ်နဲ့ Project Director ထဲမှာ Install လုပ်သင့် ကြောင်းကို ပြီးခဲ့တဲ့အခန်းမှာ ဖော်ပြခဲပြီးဖြစ်ပါတယ်။ ဒါကြောင့် `mongojs` ကို Install လုပ်တဲ့အခါ Local Module အနေနဲ့သာ Install လုပ်ထားတာကို သတိပြုပါ။ `mongojs` ကိုအသုံးပြုပြီး MongoDB ကို အခုလို ချိတ်ဆက်နိုင်ပါတယ်။

JavaScript

```
1. var mongojs = require("mongojs");
2. var db = mongojs('mydb', ['issues', 'users']);
3.
4. db.users.find(function(err, data) {
5.     console.log(data);
6.     db.close();
7. });
```

လိုင်းနံပါတ် (၂) မှာ `mongojs` ကိုသုံးပြီး MongoDB Server ထဲက `mydb` Database ကိုချိတ်ဆက်ရွေးချယ်ထားပါတယ်။ အဲ့ဒီလိုချိတ်ဆက်ရာမှာ Collection စာရင်းကိုပါ တစ်ခါတည်း Array အနေနဲ့ တွဲပေးရပါတယ်။ နမူနာအရ `issues` နဲ့ `users` ဆိုတဲ့ Collection နှစ်ခုတွဲပေးထားပါတယ်။ ဒီလိုချိတ်ဆက်ပြီးတဲ့နောက်မှာ `db` ကနေတစ်ဆင့် Mongo Shell Command တွေကို စတင်အသုံးပြု နိုင်မှာ ဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၄) မှာ `users` Collection ထဲက Document တွေကို လိုချင်တဲ့အတွက် `db.users.find()` ကို သုံးထားပါတယ်။ MongoDB ကပြန်ပေးတဲ့ `users` Collection ထဲက Document တွေဟာ Callback Function ရဲ့ `data` Variable ထဲမှာ Array အနေနဲ့ ရှိနေမှာဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၆) မှာ `db.close()` နဲ့ MongoDB Server နဲ့ချိတ်ဆက်ထားမှုကို ပိတ်ပေးထားတာကိုသတိပြုပါ။ Database နဲ့ ဆက်သွယ်ဆောင်ရွက်တဲ့ကိစ္စတွေပြီးတိုင်းမှာ Connection ကို ပြန်ပိတ်ပေးဖို့လိုပါတယ်။ အကယ်၍ Database မှာ Access Control သတ်မှတ်ထားရင်တော့ လိုင်းနံပါတ် (၂) ကို အခုလိုပြင်ဆင်

ချိတ်ဆက်ပေးရမှာပါ။

JavaScript

```
var db = mongojs('username:password@localhost/mydb', ['issues', 'users']);
```

သက်ဆိုင်ရာ Username, Password ကို MongoDB နဲ့ချိတ်ဆက်စဉ် တစ်ခါတည်း ပေးလိုက်ရတဲ့သဘော ဖြစ်ပါတယ်။

ကျန် Create, Read, Update, Delete လုပ်ငန်းအားလုံးဟာ Mongo Shell မှာ အသုံးပြုတဲ့အတိုင်းပဲ ဆက်လက် အသုံးပြု ရမှာဖြစ်ပါတယ်။ ဥပမာ – `find()` Function အတွက် Query ကို အခုလိုပေးနိုင်ပါတယ်။

JavaScript

```
1. db.issues.find({ status: { $gt: 2 } }, function(err, data) {
2.   console.log(data);
3.   db.close();
4. });
```

ဒါဟာ status တန်ဖိုး 2 ထက်ကြီးတဲ့တန်ဖိုးရှိတဲ့ Document များကို Query လုပ်ယူလိုက်ခြင်းဖြစ်ပါတယ်။ Document အသစ်တစ်ခုကို အခုလို ထည့်သွင်းနိုင်ပါတယ်။

JavaScript

```
1. db.users.insert({
2.   name: "Ei Maung",
3.   role: "Developer"
4. }, function() {
5.   db.close();
6. });
```

Document တစ်ခုကို ပယ်ဖျက်လိုရင် အခုလို ပယ်ဖျက်နိုင်ပါတယ်။

JavaScript

```
1. db.users.remove({
2.     _id: mongojs.ObjectId("5537b76a292e04f67d2af4c1")
3. }, function() {
4.   db.close();
5. });
```

`ObjectId()` Function ကို တိုက်ရိုက်သုံးလို့မရပဲ `mongojs` ကနေတစ်ဆင့် သုံးရတာကို သတိပြုပါ။ Document တွေကို Update ပြုလုပ်လိုရင်လည်း အခုလိုပြုလုပ်နိုင်ပါတယ်။

JavaScript

```
db.users.update(
     { role: "Admin" },
     { $set: { role: "Developer" } },
     { multi: true },
     function() {
      db.close();
  }
);
```

Callback Function တွေ ပါသွားတာကလွဲရင်၊ Mongo Shell အသုံးပြုပုံနဲ့ အားလုံးအတူတူပဲဆိုတာကို တွေ့ရမှာ ဖြစ်ပါ တယ်။ Index တည်ဆောက်ပုံတွေ `runCommand()` အသုံးပြုပုံတွေလည်း အတူတူပဲဖြစ်ပါတယ်။ ဥပမာ – Text Index တစ်ခုအခုလို တည်ဆောက်နိုင်ပါတယ်။

JavaScript

```
db.issues.createIndex({ subject: "text" },  {}, function() {
  db.close();
});
```

Text Index ထဲကနေ ပြန်လည်ရှာဖွေလိုတဲ့အခါ၊ အခုလို ပြည်လည်ရှာဖွေနိုင်ပါတယ်။

JavaScript

```
db.issues.runCommand("text", { search: "menu" }, function(err, result) {
  console.log(result);
  db.close();
});
```

ဒီနည်းအတိုင်း NodeJS ရဲ့ Event Driven, Non-blocking I/O လုပ်ဆောင်ချက်တွေနဲ့ MongoDB ရဲ့ Scalability အားသာ ချက်တို့ကို တွဲဖက်ပြီး စွမ်းဆောင်ရည်မြင့် Database အခြေပြု App တွေကို ဖန်တီးတည်ဆောက်နိုင် မှာပဲဖြစ် ပါတယ်။

တစ်ချို့လည်း RDBMS တွေနဲ့အတူ ORM Library တွေကို တွဲဖက်အသုံးပြုတက်ကြပါတယ်။ NodeJS အတွက် လည်း mongoose လို့ခေါ်တဲ့ ODM Module တစ်ခုကို လူသုံးများပါတယ်။ Relational Database မဟုတ်တဲ့ အတွက် ORM လို့ မခေါ်ပဲ ODM လို့ခေါ်ခြင်းဖြစ်ပါတယ်။ သဘောသဘာဝကတော့အတူတူပါပဲ။ ဒီနေရာမှာ ORM နဲ့ mongoose အကြောင်းကို ထည့်သွင်းမဖော်ပြနိုင်တော့ပါဘူး။ mongoose အကြောင်းကို mongoosejs.com မှာ ဆက်လက်လေ့လာ နိုင်ပါတယ်။

Conclusion

ဒီအခန်းရဲ့အစမှာ RDBMS တွေမှာ ရှိတက်တဲ့ အခက်အခဲတွေကို ဖြေရှင်းနိုင်ဖို့အတွက် NoSQL Database တွေ ပေါ် ပေါက်လာတယ်လို့ဖော်ပြခဲ့ပါတယ်။ ဒါပေမယ့် RDBMS တွေမှာရှိနေတဲ့ ACID Compliance ဆိုတဲ့ အချက်အလက်တိကျ သေချာမှု အာမခံအားသာချက်ကိုလည်း မေ့မထားသင့်ပါဘူး။ NoSQL ကို Not Only SQL လို့ ဆိုကြပါတယ်။ RDBMS တွေကို အစားထိုးဖို့ ရည်ရွယ်ချက်နဲ့ ပေါ်လာခြင်းမဟုတ်ပဲ၊ RDBMS တွေနဲ့ အံမဝင်တဲ့ Software အမျိုးအစားတွေမှာ အသုံးချနိုင်ဖို့ဆိုတဲ့ ရည်ရွယ်ချက်နဲ့ ပေါ်ပေါက်လာခြင်းဖြစ်ပါတယ်။ တစ်ချို့ အဖွဲ့အစည်းတွေမှာဆိုရင် RDBMS နဲ့ NoSQL Database တွေကို အတူတစ်ကွပူးတွဲအသုံးချကြတာတွေလည်း ရှိပါတယ်။ ဥပမာ – အချက်အလက်တိကျ သေချာဖို့လိုတဲ့ အပိုင်းတွေကို RDBMS တွေသုံးပြီး၊ Cache လုပ်ဖို့နဲ့ Real-time Data, State Data စတဲ့ Unstructured Data တွေကို စီမံဖို့အတွက် NoSQL Database တွေကို တွဲဖက်အသုံးပြုကြခြင်းဖြစ်ပါတယ်။

ကိုယ်ရဲ့ Software Project အတွက်ဘယ် Database အမျိုးအစားကို ရွေးချယ်ရမလဲလို့ စဉ်းစားစရာဖြစ်လာတိုင်း၊ ACID Compliance နဲ့ Scalability နှစ်ခုမှာ ဘာပိုအရေးကြီးလဲဆိုတဲ့ အချက်ပေါ်အခြေခံဆုံးဖြတ်ကြရမှာဖြစ်ပါတယ်။ အချက် အလက် တိကျသေချာမှုအာမခံချက် မဖြစ်မနေလိုအပ်တဲ့ Software တွေမှာ (ဥပမာ – Finical Solution) RDBMS တွေကို ရွေးချယ်အသုံးပြုရမှာဖြစ်ပြီး၊ အချက်အလက်ပေါင်းများစွာကို မြန်ဆန်ထိရောက်စွာစီမံနိုင်မှုပိုအရေးကြီးတက်တဲ့ Software တွေမှာ (ဥပမာ – Communication App) NoSQL Database တွေကို ရွေးချယ်အသုံးပြု ကြရမှာပဲဖြစ်ပါ တယ်။

MongoDB ရဲ့ အဓိကကျတဲ့ လုပ်ဆောင်ချက်နှစ်ခုဖြစ်တဲ့ Replication နဲ့ Sharded Cluster အကြောင်းကို **အခန်း (၁၉)** မှာ ဆက်လက်ဖော်ပြပေးသွားမှာပဲ ဖြစ်ပါတယ်။

တစ်ချို့ Framework တွေဟာ၊ အစစအရာအရာကြိုတင်
ဆုံးဖြတ်သတ်မှတ်ပေးထားလေ့ရှိတဲ့ Opinionated Framework
တွေဖြစ်ကြပါတယ်။ တစ်ချို့ Framework တွေကတော့ လိုအပ်တဲ့
အခြေခံကိုသာပေးထားပြီး၊ အသေးစိတ်ကအစ ကိုယ်တိုင်ဆုံးဖြတ်
ရေးသားခွင့်ပြုတဲ့ Unopinionated Framework တွေဖြစ်ကြပါတယ်။
သူ့အားသာချက်နဲ့သူပဲမို့ ရွေးချယ်မှု မှန်ကန်ဖို့လိုပါတယ်။

Professional Web Developer (စာအုပ်)
Web Standard, jQuery, PHP, MySQL, Ajax, CMS, MVC,
HTML5, Mobile Web, Web Application Security စသည့်
အကြောင်းအရာများကို ရေးသားဖော်ပြထားသည့်စာအုပ်
အောက်ပါလိပ်စာတွင် အခမဲ့ Download ရယူနိုင်သည်။

http://pwdbook.com

အခန်း(၁၂) – ExpressJS (REST Framework)

ExpressJS ဟာ NodeJS ပေါ်မှာအခြေခံထားတဲ့ REST Framework တစ်ခုဖြစ်ပါတယ်။ အခြား Full-scaled App Development Framework တွေလို ရှုပ်ထွေးတဲ့လုပ်ဆောင်ချက်တွေ မပါဝင်ပဲ REST API အခြေပြု Service တွေ တည်ဆောက်ရာမှာ လိုအပ်နိုင်တဲ့ အခြေခံလုပ်ဆောင်ချက်များဖြစ်တဲ့ Routing, Request/Response Handling စတဲ့ ကိစ္စတွေသာ အဓိကပါဝင်တဲ့ ရိုးရှင်းတဲ့ Unopinionated Framework တစ်ခုဖြစ်ပါတယ်။

အခန်း (၄) မှာ Service Oriented Architecture (SOA) အကြောင်းလေ့လာခဲ့ကြစဉ်က၊ HTTP ဟာ REST နည်းပညာသတ်မှတ်ချက်နဲ့အညီ ဖန်တီးတီထွင်ထားတဲ့ Protocol တစ်ခုဖြစ်တဲ့အကြောင်း ဖော်ပြခဲ့ပါတယ်။ ဒါကြောင့် REST Web Service တွေ တည်ဆောက်တဲ့အခါ HTTP ကို ထိရောက်အောင်အသုံးချနိုင်ဖို့လိုပါတယ်။ **အခန်း (၁၀)** မှာ NodeJS အကြောင်း ဖော်ပြခဲ့ရာမှာတော့၊ NodeJS ရဲ့ HTTP Module ဟာ Low Level Module တစ်ခုဖြစ်ပြီး အသေးစိတ်လုပ်ဆောင်ချက်တွေကို ကိုယ်တိုင်ရေးသားရမှာ ဖြစ်တဲ့အကြောင်း ဖော်ပြခဲ့ပါတယ်။ ဒီနေရာမှာ ExpressJS ဝင်လာခြင်းဖြစ်ပြီး NodeJS HTTP Module မှာ မပါဝင်တဲ့ Function တွေကို ကြိုတင်ဖန်တီး ပေးထားခြင်းပဲဖြစ်ပါ တယ်။

12.1 – Simple ExpressJS App

ExpressJS ဟာ NodeJS Module တစ်ခုဖြစ်တဲ့အတွက် NPM နဲ့ အခုလို Install လုပ်ယူနိုင်ပါတယ်။

```
$ npm install express
```

Install လုပ်ပြီးတဲ့အခါ ExpressJS ရဲ့ `listen()`, `get()` နဲ့ `res.send()` Function တို့ကို အသုံးပြုပြီး Web Service တစ်ခုကို စတင်တည်ဆောက်နိုင်ပြီဖြစ်ပါတယ်။

> ဒီအခန်းမှာ ဖော်ပြထားတဲ့ Code တွေဟာ လက်တွေ့ကျကျူးယူစမ်းသပ်နိုင်တဲ့ Code တွေဖြစ်ပါ တယ်။

`listen()` Function ကို Web Server Run ဖို့အတွက် အသုံးပြုပြီး `get()` Function ကို Client Request တွေ လက်ခံဖို့သုံးရပါတယ်။ `send()` Function ကိုတော့ Client ထံ Response ပြန်ပေးပို့ဖို့ သုံးရပါတယ်။ ဥပမာ –

JavaScript

```
var express = require("express");
var app = express();

var server = app.listen(3000, function(){
    console.log("ExpressJS server running on port 3000");

    app.get('/', function(req, res){
        res.send("Home Page");
    });

    app.get('/about', function(req, res){
        res.send("About Us Page");
    });
});
```

နမူနာလိုင်းနံပါတ် (၄) မှာ `listen()` Function ကိုသုံးပြီး Port နံပါတ် 3000 မှာ Web Server တစ်ခုကို Run ထားပါတယ်။ လိုင်းနံပါတ် (၇) မှာ `get()` Function ကိုသုံးပြီး၊ URI အလွတ်နဲ့ Request တစ်ခုလက်ခံရရှိတဲ့အခါ `res.send()` ကိုသုံးပြီး Home Page ဆိုတဲ့ Text Response တစ်ခုကို Respond ပြန်ပေးဖို့ သတ်မှတ်ထားပါ တယ်။ လိုင်းနံပါတ် (၁၁) မှာ တော့ `/about` URI ကိုလက်ခံရရှိတဲ့အခါ About Us Page ဆိုတဲ့ Text Response တစ်ခုကို ပြန်လည်ပေးပို့ထားပါတယ်။ ဒီ Code ကို `server.js` အမည်နဲ့ ရေးသားပြီး Run ကြည့်နိုင်ပါတယ်။

```
$ node server.js

ExpressJS server running on port 3000
```

Web Browser ကိုဖွင့်ပြီး localhost:3000 ကိုသွားကြည့်ရင် Home Page ဆိုတဲ့ Response ကို လက်ခံရရှိမှာဖြစ်ပြီး localhost:3000/about ကိုသွားကြည့်ရင်တော့ About Us Page ဆိုတဲ့ Response ကို လက်ခံရရှိမှာပဲဖြစ်ပါတယ်။

URI ပေါ်သာမက Request Method ပေါ်မူတည်ပြီးတော့လည်း Response တွေကို စီမံနိုင်ပါသေးတယ်။ နမူနာမှာ `get()` Function ကိုသုံးထားတဲ့အတွက် GET Request Method တွေကို လက်ခံစီမံပေးနေခြင်းဖြစ်ပါတယ်။ POST, PUT, DELETE အစရှိတဲ့ Request Method တွေကိုလက်ခံစီမံဖို့အတွက် `get()` အစား `post()`, `put()`, `delete()` စတဲ့ Function တွေကို အစားထိုး အသုံးပြုနိုင်ပါတယ်။

ပြီးခဲ့တဲ့နမူနာမှာလို Text တွေကို Response မပြန်ပဲဖိုင်တွေကို Response ပြန်ပေးလိုတယ်ဆိုရင် `res.send()` အစား `res.sendFile()` ကို အစားထိုးအသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
var express = require("express");
var app = express();

var server = app.listen(3000, function(){
    console.log("ExpressJS server running on port 3000");

    app.get('/', function(req, res){
        res.sendFile( __dirname + "/index.html" );
    });

    app.get('/about', function(req, res){
        res.sendFile( __dirname +  "/about.html" );
    });
});
```

နမူနာအရ URI အလွတ်နဲ့ Request လုပ်လာခဲ့ရင် `index.html` ကို Response ပြန်ပေးသွားမှာဖြစ်ပြီး၊ `/about` URI နဲ့ Request လုပ်လာခဲ့ရင်တော့ `about.html` ကို Response ပြန်ပေးသွားမှာဖြစ်ပါတယ်။ `__dirname` Constant ကိုလက်ရှိ Directory ရဲ့ Absolute Path ကိုလိုချင်တဲ့အခါ သုံးရပါတယ်။

တစ်ခုသတိပြုရမှာကတော့၊ `res.sendFile()` ဟာ Static ဖိုင်တွေကိုသာ Response လုပ်ပေးနိုင်ပါတယ်။ အကယ် ၍ Dynamic Template တွေကို Response လုပ်လိုရင်တော့ `res.render()` Function ကို အသုံးပြုနိုင်ပါတယ်။ `res.render()` Function ကို Jade, Ejs, Handlebar စတဲ့ JavaScript Template Engine တစ်ခုခုနဲ့ တွဲသုံး ပေးရပါတယ်။ နမူနာအနေနဲ့ Ejs ကို အသုံးပြု ဖော်ပြပေးပါမယ်။ Ejs Template Engine ကို အခုလို Install လုပ်နိုင်ပါ တယ်။

```
$ npm install ejs
```

ပြီးတဲ့အခါ၊ စောစောက Code နမူနာကို အခုလို ပြင်ဆင်ဖြည့်စွက်ပေးရပါမယ်။

JavaScript

```
var express = require("express");
var app = express();

app.set('views', './views');
app.engine('html', require('ejs').renderFile);

var server = app.listen(3000, function(){
    console.log("ExpressJS server running on port 3000");

    app.get('/', function(req, res){
        res.render("index.html");
    });

    app.get('/about', function(req, res){
        res.render("about.html", { greeting: "Hello, World!" });
```

```
16.     });
17. });
```

လိုင်းနံပါတ် (၄) မှာ `app.set()` Function ကိုသုံးပြီး HTML Template တွေရှိရာ Directory ကို သတ်မှတ်ပေးထားပါတယ်။ ဒါကြောင့် HTML Template ဖိုင်တွေကို `views` အမည်ရှိ Directory တစ်ခုနဲ့ စုစည်းထားရမှာဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၅) မှာတော့ `app.engine()` Function ကိုသုံးပြီး `html` Extension နဲ့ ဖိုင်တွေအတွက် Ejs Template Engine ကိုသုံးဖို့ သတ်မှတ်ထားပါတယ်။ လိုင်းနံပါတ် (၁၅) မှာ `res.render()` Function နဲ့ `about.html` ကို Response ပြန်ခိုင်းထားပါတယ်။ အဲ့ဒီလို Response ပြန်ရာမှာ တွဲဖက်အသုံးပြုရမယ့် တန်ဖိုးတွေကို JSON Object အနေနဲ့ တွဲဖက်ပေးနိုင်ပါတယ်။

Ejs ဟာ ကျွန်တော်တို့ **အခန်း (၉)** မှာ BackboneJS အကြောင်းလေ့လာခဲ့စဉ်က ထည့်သွင်းအသုံးပြုခဲ့တဲ့ Underscore ရဲ့ Template နဲ့ ရေးသားပုံ ရေးထုံးဆင်ပါတယ်။ JavaScript Code တွေ HTML ထဲမှာရောရေးဖို့အတွက် **<% %>** Tag ကို အသုံးပြုပြီး Output တစ်ခါတည်း ရိုက်ထုတ်စေလိုရင်တော့ **<%= %>** Tag ကို သုံးရပါတယ်။ `about.html` ထဲမှာ အခုလို ရေးသားထားတယ်ဆိုကြပါစို့။

Template

```
<h1>About Us</h1>
<p><%= greeting %></p>
```

localhost:3000/about လို့ Request လုပ်ကြည့်ရင် `about.html` ကို Response အနေနဲ့လက်ခံရရှိမှာဖြစ်ပြီး ရရှိ မယ့်ရလဒ်က အခုလိုဖြစ်နေမှာပါ။

HTML

```
<h1>About Us</h1>
<p>Hello, World!</p>
```

`greeting` Variable အစား Hello, World! ဆိုတဲ့တန်ဖိုးကို Ejs က အစားထိုးထည့်သွင်းပေးလိုက်ခြင်းပဲဖြစ်ပါတယ်။

ဒီနည်းနဲ့ Text တွေ Static ဖိုင်တွေသာမက Dynamic Template တွေကိုပါ Response ပြန်ပေးနိုင်တဲ့ App တွေတည် ဆောက်နိုင်မှာပဲဖြစ်ပါတယ်။ ဒါပေမယ့် လက်တွေ့မှာ User Interface နဲ့ Template လုပ်ဆောင်ချက်တွေကို ExpressJS နဲ့မလုပ်ပဲ BackboneJS လို နည်းပညာမျိုးကိုသုံးပြီး သီးခြား UI Client အနေနဲ့သာ တည်ဆောက်သင့်ပါတယ်။ ExpressJS ကိုတော့ UI Client က ဆက်သွယ်လာတဲ့အခါ လက်ခံဆောင်ရွက်ပေးနိုင်တဲ့ Service တွေ တည်ဆောက်ဖို့ အတွက် အသုံးပြုသင့်ပါတယ်။ BackboneJS နဲ့ ExpressJS တို့ကိုပူးတွဲအသုံးချထားတဲ့ နမူနာ App ကို **အပိုင်း (၄)** မှာ ဆက်လက်ဖော်ပြပေးသွားပါမယ်။

`res.send()`, `res.sendFile()` နဲ့ `res.render()` တို့အပြင် သုံးလို့ရတဲ့အခြား Response Function

တွေရှိပါသေးတယ်။ `res.download()`, `res.json()`, `res.jsonp()` တို့ပဲဖြစ်ပါတယ်။ `res.download()` Function က `res.sendFile()` လိုပဲ ဖိုင်ကို Response ပြန်ပေးခြင်းဖြစ်ပါတယ်။ ဒါပေမယ့် ဖော်ပြဖို့မဟုတ်ပဲ Client အနေနဲ့ Download လုပ်ယူဖို့ ပေးပို့ခြင်းဖြစ်ပါတယ်။ `res.json()` နဲ့ `res.jsonp()` Function တို့ကတော့ `send()` လိုပဲ Text တွေကို Response ပြန်ပေးခြင်းဖြစ်ပါတယ်။ ဒါပေမယ့် Response ရဲ့ Content-Type Header ကို JSON လို့ တစ်ခါတည်း သတ်မှတ်ပေးသွားမှာ ဖြစ်ပါတယ်။

12.1 – Routing

REST API အခြေပြု Web Service တွေတည်ဆောက်တဲ့အခါ အခြေခံအကျဆုံးလိုက်နာရမယ့် အချက်နှစ်ချက် ရှိပါတယ်။ HTTP Request Method တွေကို ထိရောက်အောင် အသုံးချဖို့နဲ့ နားလည်ရလွယ်ကူတဲ့ URL တွေကို အသုံးပြုဖို့ပဲဖြစ်ပါတယ်။ ExpressJS က အဲ့ဒီလုပ်ဆောင်ချက်တွေရရှိဖို့အတွက် အသုံးပြုနိုင်တဲ့ Routing Function တွေကို အသင့်ပေးထားပါတယ်။

ဥပမာအားဖြင့် ကျွန်တော်တို့ App အတွက် အခုလို URL Structure ကို ရရှိလိုတယ်ဆိုပါစို့။

Request Method	URL	Purpose
GET	users/	User စာရင်းရယူရန်
GET	users/123	User အမှတ် 123 ၏ အချက်အလက်များရယူရန်
POST	users/	User အသစ်တည်ဆောက်ရန်
PUT	users/123	User အမှတ် 123 ၏ အချက်အလက်များပြင်ဆင်ရန်
DELETE	users/	User အားလုံးအား ပယ်ဖျက်ရန်
DELETE	users/123	User အမှတ် 123 အား ပယ်ဖျက်ရန်

ExpressJS နဲ့ လိုချင်တဲ့ URL Structure ရအောင် အခုလိုသတ်မှတ်ပေးနိုင်ပါတယ်။

JavaScript

```
app.get("/users", function(req, res) {
     console.log("Getting user list");
});

app.get("/users/:id", function(req, res) {
     var userId = req.params.id;
     console.log("Getting user %s", userId);
});

app.post("/users", function(req, res) {
     console.log("Creating a new user");
});
```

```
app.put("/users/:id", function(req, res) {
     var userId = req.params.id;
     console.log("Updating user %s", userId);
});

app.delete("/users", function(req, res) {
     console.log("Removing all users");
});

app.delete("/users/:id", function(req, res) {
     var userId = req.params.id;
     console.log("Removing user %s", userId);
});
```

`/users` URL ကို GET Method နဲ့ Request ပြုလုပ်လာခဲ့ရင် `app.get("/users")` ရဲ့ Callback Function က အလုပ်လုပ်သွားမှာဖြစ်ပြီး `/users` URL ကိုပဲ POST Method နဲ့ Request ပြုလုပ်ခဲ့ရင် `app.post("/users")` ရဲ့ Callback Function က အလုပ်လုပ်သွားမှာပဲဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Request Method ပေါ်မူတည်ပြီး အလုပ်လုပ် နိုင်တဲ့ REST API အခြေပြု Web Service တွေကို ExpressJS နဲ့ အလွယ်တစ်ကူ တည်ဆောက်နိုင်မှာ ဖြစ်ပါတယ်။

နမူနာမှာ `:id` ကို URL Parameter အနေနဲ့အသုံးပြုထားတာကို သတိပြုကြည့်ပါ။ `:id` ကဲ့သို့ အလားတူ Parameter တွေကို URL Pattern ထဲမှာ လိုအပ်သလို ထည့်သွင်းအသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
app.get("/users/:name/:id", function(req, res) {
    var name = req.params.name;
    var id = req.params.id;
});
```

ဒီလိုသတ်မှတ်ထားတဲ့အတွက်၊ `users/john/profile/123` ဆိုတဲ့ URL ကိုလက်ခံရရှိတဲ့အခါ `name` အတွက် တန်ဖိုး `john` ကိုလက်ခံရရှိမှာဖြစ်ပြီး `id` အတွက်တန်ဖိုးအနေနဲ့ 123 ကို လက်ခံရရှိမှာပဲဖြစ်ပါတယ်။ Method ကို မကြည့်ပဲ ဘယ်လို Method နဲ့ပဲ Request ပြုလုပ်ခဲ့သည့်ဖြစ်စေ ဆောင်ရွက်စေလိုတဲ့ လုပ်ငန်းတွေကိုတော့ `all()` Function နဲ့ သတ်မှတ်ထားနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
app.all("/users", function(req, res) {
    console.log("Request received");
    next();
});
```

ပုံမှန်အားဖြင့် `all()` Function အတွင်းမှာ `next()` Function ကို ခေါ်ယူပေးရလေ့ရှိပါတယ်။ ဒီတော့မှ အဲ့ဒီ URL အတွက်သတ်မှတ်ထားတဲ့ `get()`, `post()` စတဲ့ သီးခြား Function တွေ ရှိနေသေးရင် ဆက်လက် အလုပ်လုပ် ဆောင် ပေးသွားမှာဖြစ်ပါတယ်။

URL တစ်ခုတည်းအတွက် Method အမျိုးမျိုးကို တစ်ခါတည်းအတွဲလိုက် သတ်မှတ်လိုရင်တော့ `route()` Function ကို သုံးနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
app.route("users/")
   .get(function(req, res) {
       console.log("Getting user list");
   })
   .delete(function(req, res) {
       console.log("Removing all users");
   });
```

App ရဲ့ အစိတ်အပိုင်းတစ်ခုစီအတွက် Routing Pattern တွေကို သီးခြားစီ ခွဲခြားရေးသားလိုတယ်ဆိုရင် Router Constructor ကို အသုံးပြု နိုင်ပါတယ်။ ဥပမာ - `home-router.js` ဆိုတဲ့ ဖိုင်မှာ အခုလို ရေးသားထားပါမယ်။

JavaScript

```
1. var express = require("express");
2. var router = express.Router();
3.
4. router.get("/", function(req, res) {
5.     res.send("Home");
6. });
7.
8. router.get("/about", function(req, res) {
9.     res.send("About Us");
10. });
11.
12. module.exports = router;
```

ဆက်လက်ပြီး `user-router.js` ဆိုတဲ့ဖိုင်မှာ အခုလိုရေးသားထားပါမယ်။

JavaScript

```
1. var express = require("express");
2. var router = express.Router();
3.
4. router.get("/", function(req, res) {
5.     res.send("User List");
6. });
7.
8. router.get("/:id", function(req, res) {
9.     var id = req.params.id;
10.    res.send("View user %s", id);
11. });
12.
13. module.exports = router;
```

ဒီတော့ Home Sub App အတွက် Route Pattern တစ်ခုနဲ့ User Sub App အတွက် Route Pattern တစ်ခု ကိုယ်စီ ရရှိသွားတဲ့သဘောဖြစ်ပါတယ်။ App ထဲမှာ ဒီ Router နှစ်ခုကို Sub App အနေနဲ့ အခုလို ကြေငြာ အသုံးချနိုင်ပါတယ်။

JavaScript

```
var express = require("express");
var homeRouter = require("./home-router.js");
var userRouter = require("./user-router.js");
var app = express();

app.use("/home", homeRouter);
app.use("/users", userRouter);

var server = app.listen(3000, function(){
    console.log("ExpressJS server running on port 3000");
});
```

လိုင်နံပါတ် (၆) မှာ `/home` အတွက် `homeRouter` ကို Sub App အနေနဲ့သုံးဖို့ သတ်မှတ်ထားပြီး၊ လိုင်းနံပါတ် (၇) မှာ `/users` အတွက် `userRouter` ကို Sub App အနေနဲ့သုံးဖို့ သတ်မှတ်ထားပါတယ်။ ဒါကြောင့် `/home` နဲ့ စတဲ့ URL တိုင်းအတွက် `home-router.js` Sub App က အလုပ်လုပ်ပေးသွားမှာဖြစ်ပြီး၊ `/users` နဲ့ စတဲ့ URL တိုင်းအတွက် `user-router.js` Sub App ကအလုပ်လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ နမူနာအရ URL က `/home/about` ဆိုရင် **About Us** ကို Response ပြန်ပေးသွားမှာဖြစ်ပြီး URL က `/users/123` ဆို ရင်တော့ **Viewing user 123** ကို Response ပြန်ပေးသွားမှာပါ။

12.2 – Serving Statistic Files

NodeJS ရဲ့ HTTP Module အကြောင်းလေ့လာခဲ့စဉ်က Request ပြုလုပ်လာတဲ့ Path လမ်းကြောင်းအတိုင်း Static HTML ဖိုင်တွေကို အလိုအလျောက် Response ပြုလုပ်ပေးပုံကို ဖော်ပြခဲ့ပါတယ်။ အဲ့ဒီလိုဖော်ပြခဲ့ရာမှာ NodeJS နဲ့ ကိုယ်တိုင်ရေးသားပုံ နမူနာကိုပေးခဲ့သလို `connect` နဲ့ `serveStatic` လို့ခေါ်တဲ့ NPM Module တွေကိုအသုံး ပြုနိုင်ပုံကိုလည်း ဖော်ပြခဲ့ပါတယ်။ ExpressJS ကိုလည်း အဲ့ဒီလို URL Path လမ်းကြောင်းနဲ့ကိုက် ညီတဲ့ Static ဖိုင်တွေ အလိုအလျောက် Response ပြန်ပေးနိုင်တဲ့ Static Web Server တစ်ခုအနေနဲ့ အသုံးပြု နိုင်ပါတယ်။ အခုလို ရေးသား ပေးရမှာဖြစ်ပါတယ်။

JavaScript

```
var express = require("express");
var app = express();

app.use(express.static("static"));

var server = app.listen(3000, function(){
    console.log("ExpressJS server running on port 3000");
});
```

`express.static()` Function ကိုသုံးပြီး HTML ဖိုင်တွေတည်ရှိရာ Directory ကို သတ်မှတ်ပေးရပါတယ်။ ဒါကြောင့် localhost:3000/about.html ဆိုတဲ့ Request ကိုလက်ခံရရှိရင် ExpressJS က `static` Director ထဲက `about.html` ဆိုတဲ့ဖိုင်ကို Response ပြန်ပေးသွားမှာပဲဖြစ်ပါတယ်။

Static ဖိုင် Server လုပ်ဆောင်ချက်ကို Sub App အနေနဲ့ တွဲသုံးလိုတယ်ဆိုရင် လိုင်းနံပါတ် (၄) ကို အခုလို ပြင်ဆင် ရေးသားနိုင်ပါတယ်။

JavaScript

```
app.use('/static', express.static('static'));
```

ဒီတော့ ExpressJS က `/static` နဲ့စတဲ့ URL တွေကိုလက်ခံရရှိတော့မှသာ Static ဖိုင် Server လုပ်ဆောင်ချက်ကို အလုပ်လုပ် ပေးသွားတော့မှာဖြစ်ပါတယ်။

12.3 – Request Handling

Web Service တစ်ခုအနေနဲ့ Request တွေကိုလက်ခံရရှိတဲ့အခါ၊ အဲ့ဒီ Request တွေနဲ့အတူတွဲဖက်ပါဝင်လာတဲ့ Query String တွေ၊ Request Data တွေ၊ ဖိုင်တွေနဲ့ Cookie တွေကို လက်ခံစီမံနိုင်ဖို့လိုပါတယ်။ ExpressJS ရဲ့အရင် Version တွေမှာ Request Data တွေစီမံနိုင်တဲ့ လုပ်ဆောင်ချက်တွေ တစ်ခါတည်းပါဝင်ပေမယ့်၊ နောက်ပိုင်း Version တွေမှာ Query String တွေလက်ခံစီမံတဲ့ လုပ်ငန်းတစ်ခုသာ Build-in ပါဝင်ပြီး၊ ကျန်လုပ်ငန်းတွေအတွက် သီးခြား Module တွေကို ထပ်မံထည့်သွင်း ပေး ရပါတယ်။

Request နဲ့အတူပါဝင်လာတဲ့ Query String Data တွေကို ExpressJS က `req.query` Property ထဲမှာ JSON Object တစ်ခုအနေနဲ့ လက်ခံထားပေးပါတယ်။ ဒါကြောင့် Query String Data တွေကိုစီမံဖို့လိုတဲ့အခါ `req.query` ကနေ အသင့်ရယူအသုံးပြုနိုင်မှာပဲ ဖြစ်ပါတယ်။ ဥပမာ –

JavaScript

```
var express = require("express");
var app = express();

var server = app.listen(3000, function() {
    console.log("ExpressJS server running on port 3000");

    app.get("/", function(req, res) {
        res.send(req.query);
    });
});

// URL => http://localhost:3000/?foo=apple&bar=orange
// Result => {foo: "apple", bar: "orange"}
```

Request နဲ့အတူပါဝင်လာတဲ့ Form Data တွေကို စီမံလိုရင်တော့ **body-parser** ဆိုတဲ့ Module တစ်ခုကို

အသုံးပြုရ ပါတယ်။ အရင်ဆုံး `body-parser` ကို အခုလို Install လုပ်ထားပေးဖို့လိုပါတယ်။

```
$ npm install body-parser
```

`body-parser` က `req.body` ဆိုတဲ့ Property ထဲမှာ Form Data တွေကို JSON Object တစ်ခုအနေနဲ့ အသင့် ထည့်သွင်း ပေးထားမှာဖြစ်ပါတယ်။ အခုလို ရေးသားအသုံးပြုရပါတယ်။

JavaScript

```
var express = require('express');
var parser = require('body-parser');
var app = express();

app.use(parser.urlencoded({ extended: true }));

app.post('/', function (req, res) {
    res.send(req.body);
});
```

လိုင်းနံပါတ် (၅) မှာ `app.use()` Function ကိုသုံးပြီး `body-parser` ရဲ့ `urlencoded()` Function ကို အသုံးပြု မယ်လို့ ကြေငြာသတ်မှတ်ထားပါတယ်။ Form Data တွေကို Object ပြောင်းတဲ့အခါ NodeJS Build-in `querystring` Module ကို အသုံးပြုလိုရင် `extended: false` Parameter ကိုပေးရပြီး၊ Third-party Module တစ်ခုဖြစ်တဲ့ `qs` ကို အသုံးပြုလိုရင်တော့ နမူနာမှာ ဖော်ပြထားသလို `extended: true` Parameter ကိုပေးရမှာဖြစ်ပါတယ်။

Request နဲ့ပါဝင်လာတဲ့ Data တွေထဲမှာ Form Data အပြင် JSON Data တွေလည်းပါဝင်တက်ပါတယ်။ တစ်နည်းအားဖြင့် Request Body ဟာ Form Data မဟုတ်ပဲ JSON Data ဖြစ်နေနိုင်တဲ့သဘောဖြစ်ပါတယ်။ Request Body ဟာ JSON Data ဖြစ်နေရင်လည်း အလိုအလျောက်ရယူပြီး `req.body` ထဲမှာ ထည့်သွင်းပေးထားစေဖို့အတွက် အောက်ပါနမူနာရဲ့ လိုင်းနံပါတ် (၅) မှာဖော်ပြထားသလို ဖြည့်စွက်ကြေငြာပေးရမှာ ဖြစ်ပါတယ်။

JavaScript

```
var express = require('express');
var parser = require('body-parser');
var app = express();

app.use(parser.json());
app.use(parser.urlencoded({ extended: true }));

app.post('/', function (req, res) {
    res.send(req.body);
});
```

Request နဲ့အတူ ပါဝင်လာတဲ့ ဖိုင်တွေကို လက်ခံစီမံဖို့အတွက် **multer** လို့ခေါ်တဲ့ Module တစ်ခုကို အသုံးပြုရပါ တယ်။ NPM နဲ့ အခုလို Install ပြုလုပ်နိုင်ပါတယ်။

```
$ npm install multer
```

အောက်ပါနမူနာရဲ့ လိုင်းနံပါတ် (၅) မှာဖော်ပြထားသလို `multer` Module ကို အသုံးပြုမယ့်အကြောင်း ကြေငြာရာမှာ Request နဲ့အတူ ပါဝင်လာတဲ့ ဖိုင်တွေကို သိမ်းဆည်းထားပေးရမယ့် Directory Location ကို တစ်ခါတည်း သတ်မှတ် ပေးရပါတယ်။

JavaScript

```
1. var express = require("express");
2. var multer = require("multer");
3. var app = express();
4.
5. app.use(multer({ dest: './upload/'}));
6.
7. var server = app.listen(3000, function() {
8.
9.     app.post("/", function(req, res) {
10.         res.send(req.files);
11.     });
12.
13. });
```

နမူနာအရ Request နဲ့အတူ လက်ခံရရှိတဲ့ဖိုင်တွေကို `upload` ဆိုတဲ့ Directory တစ်ခုနဲ့ စုစည်းထားမှာဖြစ်ပါ တယ်။ ExpressJS က `req.files` Property နဲ့လည်း ဖိုင် Information တွေကို အသင့်သုံးလို့ရအောင် သိမ်းထားပေး ပါသေးတယ်။ ဖိုင်တစ်ခုကိုလက်ခံရရှိတဲ့အခါ `req.files` Property မှာရှိနေမယ့် အချက်အလက်တွေက အခုလိုပုံစံ ဖြစ်မှာပါ။

JSON

```
{
  "name": {
    "fieldname": "file",
    "originalname": "express.js",
    "name": "4f4ac9f6ed1ba0e8515f6144a6efb183.js",
    "encoding": "7bit",
    "mimetype": "application/javascript",
    "path": "upload/4f4ac9f6ed1ba0e8515f6144a6efb183.js",
    "extension": "js",
    "size": 471,
    "truncated": false,
    "buffer": null
  }
}
```

ဖိုင်ရဲ့မူလအမည်၊ လက်ရှိသိမ်းဆည်းထားတဲ့ Path, ဖိုင် Size, ဖိုင် Type စတဲ့အချက်အလက်တွေကို `req.files` ထဲမှာ စုစည်းပေးထားခြင်းပဲဖြစ်ပါတယ်။

Professional Web Developer ရဲ့ **အခန်း (၈)** မှာ PHP နဲ့ Request Data တွေ Upload File တွေ စီမံပုံကို ဖော်ပြထားပါတယ်။ Language မတူနဲ့အတွက် ရေးထုံးကွဲသွားပေမယ့် PHP ရော ExpressJS ကပါ HTTP သတ်မှတ် ချက်များအတိုင်း အလုပ်လုပ်နေခြင်းဖြစ်တဲ့အတွက် အခြေခံသဘောသဘာဝကတော့ အတူတူပဲဖြစ်ပါတယ်။ PHP က Request Data တွေ ကို $_GET, $_POST စတဲ့ Variable တွေနဲ့လက်ခံပေးပြီး ExpressJS က `req.body` နဲ့ လက်ခံထားပေးခြင်းဖြစ်ပါတယ်။ PHP က Request နဲ့အတူပါဝင်လာတဲ့ ဖိုင် Information တွေကို $_FILES Variable နဲ့ လက်ခံထားပေးသလို ExpressJS ကလည်း `req.files` နဲ့ လက်ခံထားပေးခြင်းပဲဖြစ်ပါတယ်။

အလားတူပဲ PHP က Request နဲ့အတူ ပါဝင်လာတဲ့ Cookie Data တွေကို $_COOKIE Variable နဲ့လက်ခံထားပေးသလို ExpressJS က `req.cookies` နဲ့ လက်ခံထားပေးမှာ ဖြစ်ပါတယ်။ ဒီလိုလက်ခံပေးနိုင်ဖို့အတွက် **`cookie-parser`** လို့ခေါ်တဲ့ Module တစ်ခုကိုတော့ အသုံးပြုပေးရပါတယ်။ `cookie-parser` ကို အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ npm install cookie-parser
```

ရေးသားအသုံးပြုပုံ Code နမူနာက ဒီလိုပါ –

JavaScript

```
var express = require("express");
var cookie = require("cookie-parser");
var app = express();

app.use(cookie());

var server = app.listen(3000, function() {

    app.get("/", function(req, res) {
        res.send(req.cookies);
    });

});
```

ထုံးစံအတိုင်း `cookie-parser` ကိုအသုံးပြုမယ့်အကြောင်း `app.use()` နဲ့ လိုင်းနံပါတ် (၅) မှာကြေငြာပေးထားပါတယ်။ အဲ့ဒီလိုကြေငြာပေးလိုက်တာနဲ့ ExpressJS က Request နဲ့အတူပါဝင်လာတဲ့ Cookie Data တွေကို `req.cookies` ထဲမှာ JSON Object အနေနဲ့ လက်ခံထားပေးမှာပဲဖြစ်ပါတယ်။

Session Data တွေဟာ Request နဲ့အတူ ပါဝင်လာတဲ့ Data တော့မဟုတ်ပါဘူး။ ဒါပေမယ့် ExpressJS မှာ Session Data တွေကို စီမံဖို့အတွက် `req.session` Property ကိုသုံးပါ။ **`express-session`** လို့ခေါ်တဲ့

Module နဲ့ တွဲသုံးဖို့ လိုပါတယ်။ `express-session` ကို အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ npm install express-session
```

Session တွေစီမံပုံ Code နမူနာကို ဖော်ပြပေးလိုက်ပါတယ်။

JavaScript

```
var express = require("express");
var session = require("express-session");
var app = express();

app.use(session({
    secret: 'secret'
}));

var server = app.listen(3000, function() {
    console.log("ExpressJS server running on port 3000");

    app.get("/", function(req, res) {
        if(req.session.view) {
            req.session.view += 1;
        } else {
            req.session.view = 1;
        }

        res.send(req.session);
    });

});
```

`app.use()` နဲ့ `express-session` ကို အသုံးပြုမယ့်အကြောင်း ကြေငြာရာမှာ `secret` Option ကို မဖြစ်မနေထည့် သွင်းပေးရပါတယ်။ Value အနေနဲ့နှစ်သက်ရာတန်ဖိုးကို ပေးနိုင်ပါတယ်။ `express-session` က အဲ့ဒီ Secret Value ကို သုံးပြီး Session Data တွေကို Encrypt လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ နမူနာလိုင်းနံပါတ် (၁၃) မှာ `req.session.view` ရှိမရှိစီစစ်ပြီး ရှိရင်တစ်တိုးပေးပါတယ်။ မရှိရင် 1 လို့ သတ်မှတ်ပေးထားပါတယ်။ ဒီနည်းနဲ့ Request တစ်ကြိမ်လုပ်တိုင်း `view` Session ရဲ့ တန်ဖိုး တစ်တိုးတိုးသွားမှာဖြစ်ပါတယ်။ တနည်းအားဖြင့် `req.session` ကို Session Data တွေရယူဖို့သာမက၊ Session တန်ဖိုးတွေ သတ်မှတ်ခြင်း၊ ပြုပြင်ခြင်း စတဲ့ကိစ္စတွေအတွက်ပါ အသုံးပြုနိုင် ခြင်းဖြစ်ပါတယ်။

Session နဲ့ပက်သက်ရင် တစ်ခုသတိပြုရမှာရှိပါတယ်။ ExpressJS က Session Data တွေကို Memory ပေါ်မှာပဲ သိမ်းပေးပါတယ်။ ဒါကြောင့် Production အတွက် ဒီအတိုင်းသုံးလို့မရပါဘူး။ Request တွေများလာတာနဲ့အမျှ Memory ပေါ်မှာ Session Data တွေများလာမှာဖြစ်တဲ့အတွက် Memory Leak ဖြစ်သွားနိုင်ပါတယ်။ Development အဆင့်မှာ စမ်းသပ်ယုံအတွက်သာ ဒီအတိုင်းအသုံးပြုသင့်ပါတယ်။ လက်တွေ့မှာ Memory ပေါ်မှာသိမ်းမယ့်အစား Redis တို့ MongoDB တို့လို Database တွေနဲ့ Session Data တွေကိုသိမ်းပေးဖို့ လိုပါတယ်။ ဒါမှမဟုတ်ရင်လည်း PHP က စီမံသလို Session Data တွေကို ဖိုင်အနေနဲ့ သိမ်းပေးဖို့လိုပါတယ်။ ဒီအတွက်

`connect-redis`, `connect-mongo`, `session-file-store` စတဲ့ Module တွေကို အသုံးပြုနိုင်ပါတယ်။ အသေးစိတ်ကို အောက်ပါလိပ်စာမှာ ဆက်လက်လေ့ လာနိုင်ပါတယ်။

https://github.com/expressjs/session

နောက်ဆုံးအနေနဲ့ Request တွေစီမံရာမှာ ဖြည့်စွက်မှတ်သားသင့်တဲ့ Property နှစ်ခုကတော့ `req.ip` နဲ့ `req.xhr` တို့ပဲ ဖြစ်ပါတယ်။ ExpressJS က `req.ip` Property ထဲမှာ Request ပြုလုပ်သူ Client ရဲ့ IP Address ကို သိမ်းဆည်းထားပြီး၊ အကယ်၍ ပြုလုပ်လာတဲ့ Request က Ajax Request ဆိုရင် `req.xhr` ထဲမှာ `true` တန်ဖိုးကို သိမ်းဆည်းထားပေးမှာ ဖြစ်ပါတယ်။

12.4 – Response Handling

Response တွေ ပြန်လည်ပေးပို့ရာမှာ အသုံးပြုရတဲ့ `res.send()`, `res.sendFile()`, `res.render()`, `res.download()`, `res.json()`, `res.jsonp()` စတဲ့ Function တွေအသုံးပြုပုံကို ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ အခုဆက်လက်ပြီး Response Header တွေ စီမံဖို့အတွက် အသုံးပြုရတဲ့ Function တွေကို ဖော်ပြပေးပါမယ်။

`res.send()` နဲ့ `res.sendFile()` Function တွေဟာ Response Body ကို ပေးပို့တဲ့ Function တွေဖြစ်တဲ့အတွက် သူ့ချည်းသက်သက် အသုံးမပြုသင့်ပါဘူး။ **`res.status()`** နဲ့ တွဲဖက်အသုံးပြုသင့်ပါတယ်။ ဥပမာ –

JavaScript

```
res.status(200).send("Response Body");
res.status(404).sendFile("/path/to/not-found.png");
```

ဒီတော့မှ Response Body မပေးဖို့ခင် Status Code ကို အရင်ပေးပို့သွားတဲ့အတွက် ပိုပြီးပြည့်စုံစနစ်ကျသွားမှာပဲဖြစ်ပါတယ်။ Response Body မပေးပို့တော့ပဲ Status Code ချည်းသက်သက် ပေးပို့လိုတဲ့အခါ **`res.sendStatus()`** ကို သုံးနိုင်ပါတယ်။ `res.sendStatus()` က ပေးလိုက်တဲ့ Status Code ပေါ် မူတည်ပြီး သင့်တော်တဲ့ Status Message ကို Response Body အဖြစ် အလိုအလျောက် တွဲဖက်ပေးပို့သွားမှာဖြစ်ပါတယ်။ ဥပမာ –

JavaScript

```
res.sendStatus(200);            // Response Body => OK
res.sendStatus(403);            // Response Body => Forbidden
res.sendStatus(600);            // Response Body => 600
```

တစ်ကယ်မရှိတဲ့ Status Code တစ်ခုကိုပေးပို့ရင်တော့ Response Body အတွက်လည်း အဲ့ဒီ Code ကိုပဲ ပေးပို့သွားမှာ ဖြစ်ပါတယ်။ Response Header အပြည့်အစုံ သတ်မှတ်လိုရင်တော့ **`res.set()`** Function ကို သုံးနိုင်ပါတယ်။

JavaScript

```
res.set({
  'Content-Type': 'text/plain',
  'Content-Length': '123'
});
```

လက်ရှိ Response Header မှာ နောက်ထပ်တန်ဖိုးတွေ ထပ်တိုးလိုရင်တော့ **res.append()** ကိုသုံးရပါတယ်။ ဥပမာ

JavaScript

```
res.append('Content-Encoding', 'gzip');
```

Respond နဲ့အတူ Download လုပ်ရမယ့်ဖိုင်တွေ တွဲဖက်လိုတဲ့အခါ Content-Disposition: attachment ဆိုတဲ့ Header ကို အသုံးပြုရပါတယ်။ Content-Disposition Header ကို အလိုအလျောက်သတ်မှတ်ပေးစေလိုရင် **res.attachment()** ကို သုံးနိုင်ပါတယ်။ Response နဲ့အတူတွဲဖက်ပေးပို့လိုတဲ့ ဖိုင်တည်ရှိရာ Path ကို Parameter အနေနဲ့ပေးရပါတယ်။ ဥပမာ –

JavaScript

```
res.attachment('path/to/logo.png');

// => Content-Disposition: attachment; filename="logo.png"
// => Content-Type: image/png
```

`res.attachment()` က Content-Disposition Header ကိုသာမက သင့်တော်တဲ့ Content-Type Header ကိုပါ တွဲဖက်သတ်မှတ်ပေးသွားမှာပဲဖြစ်ပါတယ်။ Content-Type Header သတ်မှတ်ဖို့ အတွက်လည်း ကိုယ်တိုင် သတ်မှတ်နေမယ့်အစား **res.type()** ကို သုံးနိုင်ပါ တယ်။ ဥပမာ –

JavaScript

```
res.type('html');              // => Content-Type: text/html
res.type('json');              // => Content-Type: application/json
```

Cookie တန်ဖိုးတွေကို Response Header မှာ တွဲဖက်ပေးလိုက်ခြင်းအားဖြင့် Cookie တန်ဖိုးတွေ သတ်မှတ်လိုတဲ့ အခါ **res.cookie()** ကိုသုံးနိုင်ပါတယ်။ Cookie Name, Value နဲ့ Expire Time တို့ကိုပေးရပါတယ်။

JavaScript

```
res.cookie('foo', 'Apple', { expires: new Date().now() + 3600 });
res.cookie('bar', 'Orange', { maxAge: 3600 });
```

Expire Time ကို `expires` (သို့မဟုတ်) `maxAge` Option နဲ့ သတ်မှတ်နိုင်ခြင်းဖြစ်ပါတယ်။ နမူနာမှာ စက္ကန့်ပေါင်း ၃၆၀၀ လို့သတ်မှတ်ထားတဲ့အတွက် Cookie နှစ်ခုလုံးဟာ သက်တမ်း (၁) နာရီကြာသာ တည်ရှိနေမှာဖြစ်ပါတယ်။

Cookie တွေပယ်ဖျက်စေလိုရင်တော့ **`res.clearCookie()`** ကိုသုံးနိုင်ပါတယ်။ ပယ်ဖျက်လိုတဲ့ Cookie အမည်ကို Parameter အနေနဲ့ ပေးရပါတယ်။ ဥပမာ –

JavaScript

```
res.clearCookie("foo");
```

နောက်ဆုံးတစ်ခုအနေနဲ့ ဖြည့်စွက်မှတ်သားရမှာကတော့ Redirect ပဲဖြစ်ပါတယ်။ URL တစ်ခုကို အလိုအလျောက် Redirect လုပ်သွားစေလိုတဲ့အခါ **`res.redirect()`** ကိုသုံးနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
app.get("/about", function(req, res) {
    res.redirect("/home/about-us");
});
```

နမူနာအရ `/about` URL နဲ့ပြုလုပ်လာတဲ့ Request တစ်ခုကို လက်ခံရရှိတဲ့အခါ `/home/about-us` ကို အလို အလျောက် Redirect ပြုလုပ်ပေးသွားမှာပဲဖြစ်ပါတယ်။

12.5 – CRUD Service Example

ExpressJS ဟာ အခြား Full-scaled Framework တွေလို MVC Pattern တွေ၊ Database Abstraction တွေ Validation Helper တွေ စတဲ့လုပ်ဆောင်ချက်တွေ တစ်ခါတည်း Build-in မပါဝင်ပါဘူး။ လိုချင်တဲ့ လုပ်ဆောင်ချက်တွေ ကို NPM Module တွေအနေနဲ့ ထည့်သွင်းအသုံးပြုရမှာဖြစ်ပါတယ်။ ဒါကြောင့် MongoDB လို Database မျိုးနဲ့ ချိတ်ဆက်အလုပ်လုပ်လိုရင်လည်း နှစ်သက်ရာ MongoDB Module ကို ရွေးချယ်ထည့်သွင်းပြီး အဲ့ဒီ Module က ပေးထားတဲ့လုပ်ဆောင်ချက်တွေကိုသာ အသုံးပြုရမှာဖြစ်ပါမယ်။ နမူနာအနေနဲ့ `mongojs` Module ကိုအသုံးပြုပြီး MongoDB Database မှာ Create, Read, Update, Delete (CRUD) လုပ်ဆောင်ချက်ကို ဆောင်ရွက်ပေးနိုင်တဲ့ Web Service တစ်ခု တည်ဆောက်ပုံကို ဖော်ပြပေးလိုပါတယ်။

ပထမဆုံးအနေနဲ့ Project Directory တစ်ခုတည်ဆောက်ပြီး `express`, `body-parser` နဲ့ `mongojs` တို့ကို Install လုပ်ပါမယ်။

```
$ npm install express body-parser mongojs
```

ပြီးတဲ့အခါ လိုအပ်တဲ့ Directory Structure ရအောင် ဆက်လက်တည်ဆောက်ရပါမယ်။

```
myApp
├── node_modules
│   ├── body-parser
│   ├── express
│   └── mongojs
├── static
│   ├── about.html
│   ├── contact.html
│   └── index.html
├── api.js
└── index.js
```

`node_modules` Directory ထဲမှာ NPM နဲ့ Install လုပ်ထားတဲ့ Module တွေရှိနေမှာဖြစ်ပြီး `static` Directory ထဲမှာ Static HTML ဖိုင်တွေ ရှိနေမှာဖြစ်ပါတယ်။ Project Root Folder ထဲမှာတော့ `api.js` နဲ့ `index.js` ဆိုတဲ့ ဖိုင်နှစ်ခု ရှိနေမှာဖြစ်ပါတယ်။ `index.js` ဟာ ပင်မ App ဖြစ်ပါတယ်။ `api.js` ကတော့ Database နဲ့ချိတ်ဆက်ပြီး CRUD လုပ်ဆောင်ချက်များ ဆောင်ရွက်ပေးတဲ့ Sub App တစ်ခုဖြစ်ပါတယ်။

ဆက်လက်ပြီး `api.js` ထဲမှာ MongoDB နဲ့ချိတ်ဆက်ပြီး CRUD လုပ်ငန်းတွေဆောင်ရွက်ပေးတဲ့ လုပ်ဆောင်ချက်ကို အခုလို ရေးသားထည့်သွင်းပါမယ်။

JavaScript

```
var express = require("express");
var mongojs = require("mongojs");
var router = express.Router();

var db = mongojs('username:password@localhost/mydb', ['users']);

router.get("/users", function(req, res) {
    db.users.find(function(err, users) {
        res.status(200).json(users);
   });
});

router.post("/users", function(req, res) {
    db.users.insert(req.body, function(err, result) {
        res.status(200).send(result._id);
    });
});

router.get("/users/:id", function(req, res) {
    var id = req.params.id;
    db.users.find({ _id: mongojs.ObjectId(id) }, function(err, user) {
        if(user) res.status(200).json(user);
        else res.status(404).json({msg: "Not Found"});
    });
});

router.put("/users/:id", function(req, res) {
    var id = mongojs.ObjectId(req.params.id);
    var data = req.body;

```

```
    data._id = id;

    db.users.save(data, function(err, result){
        res.status(200).send(result.nModified);
    });
});

router.delete("/users/:id", function(req, res) {
    var id = mongojs.ObjectId(req.params.id);

    db.users.remove({ _id: id }, function(err, result) {
        res.status(200).send(result.nRemoved);
    });
});

module.exports = router;
```

အောက်ပါ Request Handler (၅) မျိုးကို ရေးသားသတ်မှတ်ထားခြင်း ဖြစ်ပါတယ်။

- **GET** /users (User စာရင်းရယူရန်)
- **POST** /users (User အသစ်ထည့်သွင်းရန်)
- **GET** /users/:id (:id နှင့်ကိုက်ညီသည့် User ၏ အချက်အလက်များအားရယူရန်)
- **PUT** /users/:id (:id နှင့်ကိုက်ညီသည့် User ၏ အချက်အလက်များပြင်ဆင်ရန်)
- **DELETE** /users/:id (:id နှင့်ကိုက်ညီသည့် User အားပယ်ဖျက်ရန်)

ဆက်လက်ပြီး `index.js` မှာ အခုလို ရေးသားပါမယ်။

JavaScript

```
var express = require("express");
var parser = require("body-parser");
var api = require("./api.js");
var app = express();

app.use(parser.urlencoded({ extended: true }));

app.use(express.static("./static"));
app.use("/api", api);

var server = app.listen(3000, function() {
    console.log("ExpressJS server running on port 3000");
});
```

လိုင်းနံပါတ် (၃) မှာ `api.js` ကို `require()` လုပ်ထားတာကို သတိပြုပါ။ ထူးခြားချက်အနေနဲ့ လိုင်းနံပါတ် (၈) မှာ ပင်မ App အတွက် `static` Directory ထဲက ဖိုင်တွေကို အသုံးပြုပေးဖို့ သတ်မှတ်ထားပြီး လိုင်းနံပါတ် (၉) မှာ `api.js` ကို `/api` Sub App အနေနဲ့ အသုံးပြုပေးဖို့ သတ်မှတ်ထားပါတယ်။ ဒါကြောင့် ဒီ App က Request တွေကို အခုလိုပုံစံ လက်ခံစီမံပေးသွားမှာဖြစ်ပါတယ် –

```
GET localhost:3000/                    → static/index.html
GET localhost:3000/about.html          → static/about.html
GET localhost:3000/contact.html        → static/contact.html
GET localhost:3000/api/users           → api.js GET /users Handler
GET localhost:3000/api/users/:id       → api.js GET /users/:id Handler
PUT localhost:3000/api/users/:id       → api.js PUT /users/:id Handler
```

လိုအပ်တဲ့ Code တွေရေးသားပြီးပြီဖြစ်လို့ အောက်ပါ API URL များကို အသုံးပြုပြီးစမ်းသပ်နိုင်ပြီဖြစ်ပါတယ်။

- **GET** /api/users (User စာရင်းရယူခြင်း)
- **POST** /api/users (User အသစ်ထည့်သွင်းခြင်း)
- **GET** /api/users/:id (:id နှင့်ကိုက်ညီသည့် User ၏ အချက်အလက်များအားရယူခြင်း)
- **PUT** /api/users/:id (:id နှင့်ကိုက်ညီသည့် User ၏ အချက်အလက်များပြင်ဆင်ခြင်း)
- **DELETE** /api/users/:id (:id နှင့်ကိုက်ညီသည့် User အားပယ်ဖျက်ခြင်း)

GET Request တွေကို URL သက်သက်နဲ့ စမ်းသပ်နိုင်ပေမယ့် POST, PUT, DELETE စတဲ့ Request တွေကိုတော့ URL သက်သက်နဲ့ စမ်းလို့မရတော့ပါဘူး။ ဒီလို Request တွေပြုလုပ်နိုင်အောင် စီစဉ်ရေးသားထားတဲ့ Client App တွေကနေတစ်ဆင့် စမ်းစစ်ရမှာဖြစ်ပါတယ်။ API လုပ်ဆောင်ချက်မှန်ကန်မှုရှိမရှိ စမ်းသပ်ယုံအတွက် Client App တွေရေးနေ ဖို့မလိုပါဘူး။ `cURL` လို့ခေါ်တဲ့ Command Line Tool ကို အသုံးပြုနိုင်ပါတယ်။ တနည်းအားဖြင့် `cURL` ဟာ HTTP Client တစ်ခုဖြစ်ပါတယ်။ ဒါကြောင့် Client App တစ်ခု တည်ဆောက်မနေတော့ပဲ `cURL` နဲ့ပဲ စမ်းသပ်ကြည့်ပါမယ်။ `cURL` ကို Ubuntu မှာ အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ sudo apt-get install curl
```

Windows အတွက် Installer ကိုတော့ အောက်ပါလိမ်စာမှာ Download ရယူနိုင်ပါတယ်။

http://curl.haxx.se/download.html

`cURL` ကို Install လုပ်ပြီးပြီဆိုရင် API URL တစ်ခုချင်းကို အခုလို စမ်းသပ်ကြည့်နိုင်ပါပြီ။

```
$ curl -X GET http://localhost:3000/api/users

[{"_id":"553b7f726ab0b99f0664b180","name":"Jonh Doe","role":"Developer"},
{"_id":"553b7f7f6ab0b99f0664b181","name":"Jame Smith","role":"Tester"}]
```

`/api/users` URL ကို GET Method နဲ့ Request လုပ်ကြည့်လိုက်ခြင်းဖြစ်ပါတယ်။ `/api/users` ကို GET Method နဲ့ Request လုပ်လာခဲ့ရင် တာဝန်ယူအလုပ်လုပ်ရမယ့် Handler ကို အခုလိုရေးသားထားခဲ့ပါတယ်။

```
 7. router.get("/users", function(req, res) {
 8.     db.users.find(function(err, users) {
 9.         res.status(200).json(users);
10.     });
11. });
```

`db.users.find()` နဲ့ users Collection ထဲကရှိသမျှ Document တွေကို ရယူပြီး `res.json()` နဲ့ ပြန်ပေးထားတဲ့အတွက် နမူနာမှာ `users` Collection ထဲမှာ လက်ရှိ ရှိနေတဲ့ Document တွေကို JSON Array အနေနဲ့ ဖော်ပြနေခြင်းပဲ ဖြစ်ပါတယ်။ နောက်တစ်ခု ထပ်မံစမ်းသပ်ကြည့်ပါမယ်။

```
$ curl -X POST http://localhost:3000/api/users -d
"name=Bob&role=Tester"

"553bd0d215349a0864c5df60"
```

`/api/users` ကို POST Method နဲ့ Request လုပ်ပြီး `name=Bob&role=Tester` ဆိုတဲ့ Data ကိုပါ တွဲဖက်ပေး လိုက်ခြင်းဖြစ်ပါတယ်။ `/api/users` ကို POST နဲ့ Request ပြုလုပ်လာခဲ့ရင် တာဝန်ယူအလုပ်လုပ်မယ့် Handler ကို အခုလို ရေးသားထားပါတယ်။

```
13. router.post("/users", function(req, res) {
14.     db.users.insert(req.body, function(err, result) {
15.         res.status(200).send(result._id);
16.     });
17. });
```

`db.users.insert()` နဲ့ `req.body` အဖြစ်ပါဝင်လာတဲ့ Data ကို `users` Collection ထဲကို ထည့်သွင်းခိုင်းလိုက် ခြင်းဖြစ်ပါတယ်။ ထည့်သွင်းလိုက်တဲ့အခါ ရရှိလာတဲ့ Generated ID ကို `res.send()` နဲ့ Response ပြန်ပေးဖို့ သတ်မှတ်ထားတဲ့ အတွက် နမူနာမှာ Generated ID ကို တွေ့မြင်ရခြင်း ဖြစ်ပါတယ်။ အခုနေ `/api/users` ကို GET Request နဲ့ ပြန်စမ်းကြည့်ရင် Document တစ်ခုတိုးနေတာကို အခုလို တွေ့ရမှာ ဖြစ်ပါတယ်။

```
$ curl -X GET http://localhost:3000/api/users

[{"_id":"553b7f726ab0b99f0664b180","name":"Jonh Doe","role":"Developer"},
{"_id":"553b7f7f6ab0b99f0664b181","name":"Jame Smith","role":"Tester"},
{"name":"Bob","role":"Tester","_id":"553bd0d215349a0864c5df60"}]
```

ဆက်လက်ပြီး GET `/api/users/:id` ကို စမ်းသပ်ကြည့်ပါမယ်။

```
$ curl -X GET http://localhost:3000/api/users/553bd0d215349a0864c5df60

[{"name":"Bob","role":"Tester","_id":"553bd0d215349a0864c5df60"}]
```

`/api/users/:id` ကို GET နဲ့ Request ပြုလုပ်လိုက်တဲ့အခါ `:id` နဲ့ကိုက်ညီတဲ့ Document တစ်ခုကိုပဲ ပြန်လည် ရရှိတာကို တွေ့ရမှာဖြစ်ပါတယ်။ PUT နဲ့ စမ်းသပ်ကြည့်ပါဦးမယ်။

```
$ curl -X PUT http://localhost:3000/api/users/553bd0d215349a0864c5df60
-d "name=Alice&role=Tester"

$ curl -X GET http://localhost:3000/api/users/553bd0d215349a0864c5df60

[{"name":"Alice","role":"Tester","_id":"553bd0d215349a0864c5df60"}]
```

PUT Method ကိုသုံးပြီး စောစောကထည့်သွင်းထားတဲ့ Bob ဆိုတဲ့ User အမည်ကို Alice လို့ Update ပြုလုပ်လိုက်ခြင်း ပဲ ဖြစ်ပါတယ်။ နောက်ဆုံးတစ်ခုဖြစ်တဲ့ DELETE Method ကို စမ်းသပ်ကြည့်ပါဦးမယ်။

```
$ curl -X DELETE http://localhost:3000/api/users/553bd0d215349a0864c5df60

$ curl -X GET http://localhost:3000/api/users/

[{"_id":"553b7f726ab0b99f0664b180","name":"Jonh Doe","role":"Developer"},
{"_id":"553b7f7f6ab0b99f0664b181","name":"Jame Smith","role":"Tester"}]
```

`/api/users/:id` ကို DELETE Method နဲ့ Request လုပ်လိုက်တဲ့အတွက် သက်ဆိုင်ရာ Document ကို ပယ်ဖျက် ပေးသွားခြင်းဖြစ်ပါတယ်။ `/api/users` ကို GET နဲ့ နောက်တစ်ကြိမ် Request လုပ်ကြည့်တဲ့အခါမှာ ကျွန်တော် တို့စမ်းသပ် ပယ်ဖျက်ထားတဲ့ Document ပါဝင်လာခြင်း မရှိတော့တာကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။

ဒီနည်းနဲ့ cURL ကိုသုံးပြီး ကျွန်တော်တို့ဖန်တီးတည်ဆောက်ထားတဲ့ Web Service တွေ မှန်ကန်စွာအလုပ်လုပ်ခြင်း ရှိမရှိ ကို စီစစ်နိုင်မှာပဲ ဖြစ်ပါတယ်။

12.6 – ExpressJS Generator

ExpressJS App တစ်ခုအတွက် လိုအပ်တဲ့ Module တွေ၊ Directory Structure တွေနဲ့ Code Structure တွေ အသင့်ပါဝင်တဲ့ Boilerplate တစ်ခုကို Generate လုပ်ပေးနိုင်တဲ့ **`express-generator`** လို့ခေါ်တဲ့ Scaffold Generator တစ်ခုရှိပါတယ်။ NPM နဲ့ အခုလို Install လုပ်ယူနိုင်ပါတယ်။

```
$ sudo npm install -g express-generator
```

Install လုပ်ပြီးတဲ့အခါ `express` Command ကိုသုံးပြီး ExpressJS Boilerplate ကို Generate လုပ်ယူနိုင်ပါပြီ။

```
$ express myApp

   create : myApp
   create : myApp/package.json
   create : myApp/myApp.js
   … … …
   install dependencies:
     $ cd myApp && npm install

   run the myApp:
     $ DEBUG=myApp:* ./bin/www
   … … …
```

Generate လုပ်ပြီး ရရှိလာတဲ့ App Director ထဲကိုသွားပြီး npm install Command ကို Run ပေးလိုက်ရင် NPM က လိုအပ်တဲ့ Dependency Package တွေကို အလိုအလျောက် Install လုပ်ပေးသွားပါလိမ့်မယ်။ အားလုံးပြီးစီးသွားတဲ့အခါ ရရှိလာတဲ့ Directory Structure ကို လေ့လာကြည့်ရင် အခုလိုတွေ့ရမှာ ဖြစ်ပါတယ်။

```
myApp
├── bin
│   └── www
├── node_modules
│   ├── body-parser
│   ├── cookie-parser
│   ├── debug
│   ├── express
│   ├── jade
│   ├── morgan
│   └── serve-favicon
├── public
│   ├── images
│   ├── javascripts
│   └── stylesheets
│       └── style.css
├── routes
│   ├── index.js
│   └── users.js
├── views
│   ├── error.jade
│   ├── index.jade
│   └── layout.jade
├── app.js
└── package.json
```

`body-parser`, `cookie-parser` စတဲ့ Module တွေကိုတစ်ခါတည်း ထည့်သွင်းပေးထားတာကို တွေ့ရမှာ ဖြစ်သလို၊ Template Engine တစ်ခုဖြစ်တဲ့ `jade` ကိုလည်း ထည့်သွင်းပေးထားပါတယ်။ အကယ်၍ Template

Engine အနေနဲ့ `jade` အစား `ejs` ကို ထည့်သွင်းပေးစေလိုရင် `-e` Option ကို အသုံးပြုနိုင်ပါတယ်။

```
$ express -e myApp
```

`-e` Option ထည့်သွင်းပေးလိုက်ရင် `express-generator` က `jade` အစား `ejs` ကို Template Engine အနေနဲ့ ထည့် သွင်းပေးသွားမှာဖြစ်ပါတယ်။

`express-generator` အစား ကျွန်တော်တို့ **အခန်း (၈)** မှာဖော်ပြခဲ့တဲ့ Yeoman ကိုလည်း ExpressJS App Boilerplate တွေ Generate လုပ်ဖို့အသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

```
$ sudo npm install -g generator-express
$ yo express
```

Yeoman Generator က လက်တွေ့မှာ `express-generator` ကိုပဲ ပြန်သုံးထားတာပါ။ ဒါပေမယ့် သူက Boilerplate နဲ့အတူ Bower Config ဖိုင်တွေ၊ Grunt Config ဖိုင်တွေကိုပါ တစ်ခါတည်း ထည့်သွင်းပေးသွားမှာ ဖြစ်တဲ့အတွက် ပိုမို ပြည့်စုံတယ်လို့ ဆိုနိုင်ပါတယ်။ မိမိနှစ်သက်ရာကို အသုံးပြုနိုင်ပါတယ်။

Conclusion

Web Application Development အတွက် အဓိက Development Stack အဖြစ် ဆယ်စုနှစ်တစ်ခုစာ ထင်ရှားခဲ့တဲ့ နည်းပညာကတော့ **LAMP** Stack ပဲဖြစ်ပါတယ်။ Linux OS, Apache Web Server, MySQL Database, နဲ့ PHP တို့ အတွဲလိုက် စုစည်းပါဝင်တဲ့ Development Environment တစ်ခုဖြစ်ပါတယ်။ ကျွန်တော်ရဲ့ ပထမစာအုပ်နှစ်အုပ်ဖြစ်တဲ့ **Professional Web Developer** နဲ့ **Ubuntu – သင့်အတွက် Linux** တို့ဟာ LAMP Stack တစ်ခုလုံးကို Covered ဖြစ်စေဖို့ ရည်ရွယ်ရေးသားခဲ့တဲ့ စာအုပ်နှစ်အုပ် ဖြစ်ပါတယ်။

အခုနောက်ပိုင်းမှာတော့ JavaScript နည်းပညာတွေရဲ့ အခန်းကဏ္ဍဟာ များစွာတိုးတက်လာခြင်းနဲ့အတူ **MEAN** လို့ ခေါ် တဲ့ Development Stack အသစ်တစ်ခုက LAMP Stack ကို အနာဂါတ်မှာ အစားထိုးသွားလိမ့်မယ်ဆိုတဲ့ မျှော်မှန်းချက် တွေ ရှိလာခဲ့ပါတယ်။ MEAN ဆိုတာ MongoDB, ExpressJS, AngularJS နဲ့ NodeJS တို့ စုစည်းပါဝင်တဲ့ Develop-ment Stack ကို ခေါ်တာပါ။

မူလက၊ ဒီစာအုပ်ရဲ့ **အဝိုင်း (၃)** အဖြစ်၊ MEAN Stack အကြောင်းဖော်ပြမယ်လို့ ရည်ရွယ်ထားခဲ့တာပါ။ ဒါပေမယ့် လက်တွေ့မှာ AngularJS အစား ပိုပြီးရိုးရှင်း အခြေခံကျတဲ့ BackboneJS ကိုပဲ ရွေးချယ်ဖော်ပြဖြစ်ခဲ့ပါတယ်။ ဒါကြောင့် ခေါင်းစဉ်ကို MEAN Stack လို့မတပ်တော့ပဲ Modern Development Stack လို့ပဲ တပ်ထားခဲ့တာပါ။ BackboneJS, NodeJS, MongoDB နဲ့ ExpressJS တို့ဟာ JavaScript အခြေပြု အနာဂါတ် Development Stack အတွက် အရေးပါတဲ့ အစိတ်အပိုင်းတွေဖြစ်ပါတယ်။

အပိုင်း (၄)

Building an Issue Tracking System

အကောင်းဆုံး User Experience ရရှိဖို့အတွက် User Interface
နဲ့ ပက်သက်တဲ့အပိုင်းတွေကို အသေးစိတ်ကအစ ကိုယ်တိုင်လုပ်နိုင်ရင်
ကောင်းပေမယ့်၊ Authentication အပါအဝင် အခြေခံခိုင်မာဖို့လိုတဲ့ကိစ္စ
တွေမှာတော့ Proven ဖြစ်ပြီးသား Component တွေကို အသုံးပြု
လိုက်တာက ကိုယ်တိုင်လုပ်နေတာထက် ပိုမိုထိရောက်ပြီး
ပိုမိုစိတ်ချရစေမှာ ဖြစ်ပါတယ်။

Ubuntu – သင့်အတွက် Linux (စာအုပ်)
Ubuntu Linux အသုံးပြုနည်းအား Installation မှ စတင်၍ System
Configuration နှင့် Development Environment တည်ဆောက်
ခြင်းအထိ အဆင့်လိုက်ရေးသား ဖော်ပြထားသည့်စာအုပ်
အောက်ပါလိပ်စာတွင် အခမဲ့ ရယူနိုင်သည်။

http://u4ubook.com

အခန်း(၁၃) – Building Back-end Service

ပြီးခဲ့တဲ့အပိုင်းမှာ လေ့လာခဲ့တဲ့ BackboneJS, NodeJS, ExpressJS နဲ့ MongoDB တို့နဲ့ပက်သက်ပြီး လက်တွေ့နမူနာ တစ်ခုရသွားစေဖို့ရည်ရွယ်ပြီး ဒီအပိုင်းမှာ Issue Tracking System တစ်ခုကို အခဲ့ဒီနည်းပညာတွေသုံးပြီး တည်ဆောက်ပုံ နမူနာကို ဖော်ပြသွားမှာဖြစ်ပါတယ်။

အခန်း (၄) မှာ Service-oriented Architecture (SOA) အကြောင်းဖော်ပြခဲ့စဉ်က၊ Software တစ်ခုကို သီးခြားစီ အလုပ်လုပ်နိုင်တဲ့ Service များအဖြစ်တည်ဆောက်ခြင်းအားဖြင့် ရရှိနိုင်တဲ့ အကျိုးရလဒ်တွေကို ဖော်ပြခဲ့တာကို မှတ်မိဦးမှာပါ။ ဒီအပိုင်းမှာ Issue Tracking System တစ်ခုကို နမူနာအနေနဲ့ ဖန်တီးရာမှာလည်း Back-end Service နဲ့ Client App တို့ကို Code တွေရောစပ်ရေးသားခြင်းမပြုပဲ သီးခြားစီတည်ဆောက်သွားမှာဖြစ်ပါတယ်။ ExpressJS နဲ့ MongoDB တို့ကိုသုံးပြီး Back-end Service တည်ဆောက်ပုံကို ဦးစွာဖော်ပြချင်ပါတယ်။

> ဒီအခန်းမှာ နမူနာဖော်ပြမယ့် Code တွေဟာ လက်တွေ့ကူးယူစမ်းသပ်လိုက စမ်းသပ်နိုင်တဲ့ Code တွေ ဖြစ်ပါတယ်။ ဒါပေမယ့် တစ်လုံးမကျန်ကူးယူစမ်းသပ်မယ့်အစား ရေးသားထားတဲ့ Code ကိုအရင် နားလည်အောင်လေ့လာပြီး၊ ကိုယ်တိုင် ရေးသားဖို့ကို အကြံပြုလိုပါတယ်။ ဒါကြောင့် Code နမူနာ အပြည့်အစုံကို ဖတ်ရှုနိုင်ဖို့အတွက် အောက်ပါလိပ်စာမှာ ကြိုတင် Download ရယူထားသင့်ပါတယ်။
>
> https://github.com/eimg/issues-api/archive/master.zip

13.1 – Scaffolding

ပထမဦးဆုံးအနေနဲ့ `issues-api` ဆိုတဲ့အမည် (သို့မဟုတ် မိမိနှစ်သက်ရာအမည်) နဲ့ Directory တစ်ခု တည်ဆောက်ပြီး အဲ့ဒီ Directory ထဲမှာ လိုအပ်တဲ့ Package တွေကို NPM နဲ့ Install လုပ်ခြင်းအားဖြင့် စတင်ပါမယ်။

```
$ mkdir issues-api && cd issues-api
$ npm install express \
              mongojs \
              express-session \
              body-parser \
              cookie-parser \
              express-validator
              passport \
              passport-local
```

နမူနာမှာ `npm install` နဲ့ Package (၈) ခုကို Install လုပ်ထားပါတယ်။ ExpressJS နဲ့ MongoDB တို့ကို အသုံးပြုရေးသားမှာမို့ `express` နဲ့ `mongojs` Package တွေကို Install လုပ်ထားခြင်းပါတယ်။ `express-session, body-parser` နဲ့ `cookie-parser` တို့ကတော့ User Request တွေလက်ခံခြင်းလုပ်ငန်းနဲ့ Cookie, Session တွေ စီမံခြင်းလုပ်ငန်းအတွက်လိုအပ်လို့ ထည့်သွင်းထားရတာပါ။ User Request တွေကို Validation လဲ စစ်ချင်သေးတဲ့ အတွက် `express-validator` ကိုလည်း ထည့်သွင်းထားပါတယ်။ User Authentication နဲ့ Authorization လုပ်ငန်းတွေ အတွက် PassportJS လို့ခေါ်တဲ့ Framework ကို အသုံးပြုချင်တဲ့ အတွက် `passport` နဲ့ `passport-local` ဆိုတဲ့ Package နှစ်ခုကို တွဲဖက် ထည့်သွင်းထားခြင်းဖြစ်ပါတယ်။ ဒီအကြောင်းကို မကြာခင်ဆက်လက်ဖော်ပြပေးပါမယ်။

ဒီ Package တွေထည့်သွင်းပြီးတဲ့အခါ Directory Structure က အခုလိုရရှိသွားမှာဖြစ်ပါတယ်။

```
issues-api
└── node_modules
    ├── body-parser
    ├── cookie-parser
    ├── express
    ├── express-session
    ├── express-validator
    ├── mongojs
    ├── passport
    └── passport-local
```

ဒီ Structure ထဲမှာ လိုအပ်တဲ့ ဖိုင်တွေကိုအောက်ပါအတိုင်း ဆက်လက်ဖြည့်စွက်ပေးပါ။

```
issues-api
├── app.js
├── auth.js
├── config.js
├── node_modules
└── routes
    ├── issues.js
    └── users.js
```

`app.js, auth.js` နဲ့ `config.js` တို့ကို Project Root Directory အောက်မှာ ထည့်သွင်းပြီး `issues.js` နဲ့ `users.js` တို့ကို တော့ `routes` ဆိုတဲ့ Directory တစ်ခုနဲ့ ဖြည့်စွက်ထည့်သွင်းထားခြင်းဖြစ်ပါတယ်။

13.2 – Basic Config – `config.js`

ဆက်လက်ပြီး `config.js` အတွင်းမှာ အောက်ပါအတိုင်း ရေးသားရပါမယ်။

JavaScript

```
exports.role = [
  "User",
  "Tester",
  "Developer",
  "Manager"
];

exports.status = [
  "New",
  "Assigned",
  "Doing",
  "Done",
  "Closed"
];

exports.type = [
  "Bug",
  "Feature",
  "Other"
];

exports.priority = [
  "Low",
  "High",
  "Urgent"
];
```

`role, status, type` နဲ့ `priority` တန်ဖိုးတွေကို `exports` Object ရဲ့ Property တွေအနေနဲ့ ကြိုတင် သတ်မှတ်လိုက်ခြင်း ဖြစ်ပါတယ်။ **အခန်း (၁၀)** မှာ NodeJS အကြောင်းဖော်ပြခဲ့စဉ်က Module အဖြစ် အခြား Code တွေကနေ ရယူအသုံးပြုလိုတဲ့အခါ `module.exports` (သို့မဟုတ်) `exports` Object မှာ အသုံးပြုလိုတဲ့ Function နဲ့ Property တွေကို သတ်မှတ်ပေးရကြောင်း ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။

13.3 – Authentication and Authorization Strategies – `auth.js`

ဆက်လက်ပြီး အဓိကလုပ်ဆောင်ချက်များ မရေးသားမီ PassportJS ကို အသုံးပြုပြီး Authentication နဲ့ပက်သက်တဲ့ အပိုင်းတွေကို အရင်ရေးသားပါမယ်။ `auth.js` အတွင်းမှာ အခုလိုရေးသားပေးရပါမယ်။

JavaScript

```
var mongojs = require("mongojs");
var crypto = require('crypto');
var passport = require("passport");
var LocalStrategy = require("passport-local").Strategy;

var db = mongojs('mydb', ['users', 'issues']);

```

```
 8. passport.serializeUser(function(user, done) {
 9.   done(null, user);
10. });
11.
12. passport.deserializeUser(function(user, done) {
13.   done(null, user);
14. });
15.
16. // Authentication strategy
17. passport.use(new LocalStrategy(
18.  function(username, password, done) {
19.    var pwdHash = crypto.createHash('sha1').update(password).digest("hex");
20.
21.    db.users.findOne({
22.       loginId: username,
23.       password: pwdHash
24.     }, function(err, data) {
25.      if(data)
26.        return done(null, data);
27.      else
28.        return done(null, false);
29.      });
30.   }
31. ));
32.
33. exports.login = function() {
34.  return passport.authenticate("local");
35. }
36.
37. exports.ensureAuth = function() {
38.  return function(req, res, next) {
39.    if(req.isAuthenticated())
40.      next()
41.    else res.sendStatus(401);
42.  }
43. }
```

App သို့မဟုတ် API တစ်ခုအတွက် Authentication ပိုင်းစဉ်းစားတဲ့အခါ ဘယ်လို Authentication Strategy ကို အသုံးပြုမလဲဆိုတာကို အရင်းဆုံးစဉ်းစားရပါတယ်။ ပုံမှန်အားဖြင့် ရိုးရိုး Point to Point API Service တစ်ခုအတွက်ဆိုရင် သိပ်ပြီးရှုပ်ရှုပ်ထွေးထွေးတွေ စဉ်းစားနေစရာမလိုပဲ HTTP Basic Authentication ကို အသုံးပြုနိုင်ပါတယ်။ Point to Point ဆိုတာ Client နဲ့ Server တိုက်ရိုက်ဆက်သွယ်ပြီး ကြားခံ Third-party Service တွေ မပါဝင်တဲ့ အခြေအနေ ကို ဆိုလိုတာပါ။

HTTP Basic Authentication ရဲ့အလုပ်လုပ်ပုံက ရိုးရှင်းပါတယ်။ Request တိုင်းမှာ Access Control Credential ဖြစ်တဲ့ Username နဲ့ Password ပါဝင်ဖို့လိုပြီး၊ Request နဲ့အတူပါဝင်လာတဲ့ Credential မှန်မှသာ Access လုပ်ခွင့် ပြုမှာဖြစ်ပါတယ်။ HTTP ရဲ့ Stateless သဘာဝအတိုင်းသာ အလုပ်လုပ်ပြီး၊ တစ်ကြိမ်ပေးဖူးတဲ့အတွက် နောက် Request တွေမှာ Credential ထပ်မပေးတော့ဘူးလို့ သဘောထားလို့မရတော့ပါဘူး။ Request တိုင်းမှာ Credential ပါရမှာဖြစ်သလို Service ကလည်း Credential ပါလာမှသာ Access လုပ်ခွင့်ပြုမှာပဲဖြစ်ပါတယ်။ ရိုးရိုး API Service တွေအတွက် သင့်တော်ပါတယ်။

နောက်အသုံးများတဲ့ Authentication Strategy ကတော့ Session Based Authentication Model ဖြစ်ပါတယ်။ သူ့ကို Local Strategy လို့လည်း ခေါ်ပါတယ်။ အခြေခံအလုပ်လုပ်ပုံကတော့၊ Access Control Credential ကို တစ်ကြိမ် မှန်အောင်ပေးပြီးရင် Authentication Token ကို Session နဲ့ သိမ်းဆည်းထားမှာဖြစ်ပါတယ်။ ဒါကြောင့် နောက် Access လုပ်ဖို့လိုတဲ့အခါ Credential ကို ထပ်ပေးစရာမလိုတော့ပဲ Session Token ကိုကြည့်ပြီး Access လုပ်ခွင့်ပြု မပြုကို ဆုံးဖြတ်သွားမှာ ဖြစ်ပါတယ်။ Web App တွေမှာ အဓိကအသုံးများတဲ့ Strategy ဖြစ်ပါတယ်။

နောက်တနည်းကတော့ API Key (သို့မဟုတ်) Token ကို အခြေခံတဲ့ Strategy ဖြစ်ပါတယ်။ သူလည်းပဲ လက်တွေ့မှာ HTTP Basic Authentication နဲ့ သိပ်မကွာပါဘူး။ HTTP Basic Authentication က Username နဲ့ Password ကို Access Control Credential အဖြစ်သုံးပြီး သူကတော့ Generate လုပ်ထားတဲ့ API Key (သို့မဟုတ်) Random Token ကို Access Control Credential အဖြစ် အသုံးပြုခြင်းဖြစ်ပါတယ်။ အခြေခံအားဖြင့်၊ ပထမတစ်ကြိမ်မှာ သတ်မှတ်ထားတဲ့ Access Control Credential တွေကို မှန်ကန်အောင်ပေးရပါတယ်။ Credential မှန်တယ်ဆိုရင် Service က Random Token တစ်ခုကို ပြန်ပေးမှာဖြစ်ပါတယ်။ နောင် Service ကို Access လုပ်ဖို့လိုတဲ့အခါ Credential ကို ထပ်ပေးနေစရာမလိုတော့ပဲ Request နဲ့အတူ Token ကို ထည့်သွင်းပေးပို့ခြင်းအားဖြင့် Access လုပ်နိုင် မှာဖြစ်ပါတယ်။

Token ဆိုတာ ပုံမှန်အားဖြင့် မှန်းဆလို့မရအောင် Random Generate လုပ်ထားတဲ့ တန်ဖိုးတစ်ခုဖြစ်ပါတယ်။ ဒါပေမယ့် အခုနောက်ပိုင်းမှာ Random မဟုတ်တော့ပဲ User Information တွေကို Token ထဲမှာ တစ်ခါတည်း ထည့်ပြီး Encrypt လုပ်ပေးနိုင်တဲ့ JSON Web Tokens လို နည်းပညာမျိုးတွေလည်း ပေါ်ပေါက်လာပါတယ်။ Token က Random မဟုတ် တော့ပဲ User Information တွေပါဝင်လာတဲ့အတွက် Token ကနေရရှိတဲ့ User Information ကိုလိုအပ်သလို အသုံးချ နိုင်မှာဖြစ်တဲ့အတွက် Authorization လုပ်ယုံသက်သက်ထက် ပိုအသုံးဝင်သွားပါတယ်။ ဒီလို Token ထဲမှာ ရှိနေတဲ့ User Information ကို မသမာသူတွေ ကြားဖြတ်ခိုးယူလို့ မရစေဖို့ အတွက်တော့ Encrypt လုပ်ခြင်းအားဖြင့် ကာကွယ်ထားတက် ကြပါတယ်။

နောက်တစ်နည်းကတော့ OAuth2 Protocol ကို အသုံးပြုခြင်းဖြစ်ပါတယ်။ ဒီ Strategy ကတော့၊ Service က သပ်သပ်၊ Client ကသပ်သပ်၊ Authentication Provider သပ်သပ်၊ သီးခြားစီ အလုပ်လုပ်တဲ့ အခြေအနေမျိုးနဲ့ သင့်တော်ပါတယ်။ တစ်နည်းအားဖြင့်၊ Access ပြုလုပ်လိုတာက Service A ဖြစ်ပေမယ့်၊ Access လုပ်ခွင့်ပြုမပြု ဆုံးဖြတ်ပေးမယ့်သူက Service B ဖြစ်နေတယ်ဆိုရင် OAuth2 ကို အသုံးပြုသင့်ပါတယ်။ ဥပမာ – Web App တစ်ခုကို Login ဝင်နိုင်ဖို့အတွက် Account အသစ်မတည်ဆောက်ပဲ Facebook Account ကို အသုံးပြု Login ဝင်နိုင်ပုံကို တွေ့ဖူးကြမှာပါ။ အဲ့ဒီလို အခြေ အနေမှာ အသုံးပြုလိုတဲ့ Web App အတွက် Authentication ကို သီးခြား Service ဖြစ်တဲ့ Facebook က ဆုံးဖြတ်ပေး တဲ့သဘော ဖြစ်ပါတယ်။

ကျွန်တော်တို့ရဲ့ နမူနာမှာတော့၊ နောင်တစ်ချိန်မှာ Web Client နဲ့ တွဲဖက်အလုပ်လုပ်ဖို့ ရည်ရွယ်ထားတဲ့ Service တစ်ခုကို တည်ဆောက်မှာဖြစ်တဲ့အတွက် Session ကို အခြေခံတဲ့ Local Strategy ကို အသုံးပြုသွားမှာဖြစ်ပါတယ်။ ဒါကြောင့် လိုင်းနံပါတ် (၃) နဲ့ (၄) မှာ passport နဲ့ passport-local Package တွေကို အသုံးပြုဖို့ ကြေငြာထားခြင်းပဲဖြစ်ပါတယ်။ အကယ်၍ Local Strategy အစား အခြား Strategy ကို အသုံးပြုလိုတယ်ဆိုရင် သက်ဆိုင်ရာ Passport Strategy Package ကို ထည့်သွင်းအသုံးပြုနိုင်ပါတယ်။

အသုံးပြုသူပေးလာတဲ့ Username နဲ့ Password ကို Database ထဲမှာရှိတဲ့ Record (Document) နဲ့ တိုက်ဆိုင်စစ်ဆေးပြီး မှန်တယ်ဆိုရင် အဲ့ဒီ User ရဲ့ အချက်အလက်တွေအားလုံးကို Session ထဲမှာ သိမ်းပေးခြင်းအားဖြင့် Authorize လုပ်ပေးမှာဖြစ်ပါတယ်။ ဒါကြောင့် လိုင်းနံပါတ် (၆) မှာ mydb လို့ခေါ်တဲ့ MongoDB Database ကို mongojs နဲ့ ချိတ်ဆက်ပေးထားပါတယ်။ အကယ်၍ စာဖတ်သူရဲ့ MongoDB Database က Username, Password နဲ့ Protect လုပ်ထားရင် ဒီလိုင်းကို အခုလိုပြင်ပေးရမှာပါ။

JavaScript

```
6. var db = mongojs('username:password@localhost/mydb', ['users', 'issues']);
```

`username` နဲ့ `password` နေရာမှာ မှန်ကန်တဲ့ Database Username နဲ့ Password တို့ကို အစားထိုးထည့်သွင်း ပေးရမှာပဲဖြစ်ပါတယ်။ ဒီ Connection String ထဲမှာ `users` နဲ့ `issues` ဆိုတဲ့ Collection နှစ်ခုတို့ကို အသုံးပြုမှာ ဖြစ်ကြောင်း တစ်ခါတည်းထည့်သွင်းပေးထားတာကိုလည်း သတိပြုပါ။

ဆက်လက်ပြီး လိုင်းနံပါတ် (၈) မှာ `passport.serializeUser()` Function ကို ကြေငြာထားပါတယ်။ ဒီ Function ရဲ့ ရည်ရွယ်ချက်က JSON User Data ကို Session ထဲမှာ Text အဖြစ်ပြောင်းပြီး သိမ်းဖို့အတွက် ဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၁၂) က `passport.deserializeUser()` ကတော့ သူနဲ့ပြောင်းပြန်၊ Session ထဲက User Data ကို JSON ဖြစ်အောင် ပြန်ပြောင်းပေးဖို့အတွက် ဖြစ်ပါတယ်။ တနည်းအားဖြင့် လက်တွေ့အသုံးပြုတဲ့ အချိန်မှာ User Data တွေကို JSON နဲ့အသုံးပြုနိုင်ပြီး၊ Session ထဲမှာ သိမ်းတဲ့အခါ Text အနေနဲ့ သိမ်းနိုင်ဖို့ ဒီ Function နှစ်ခုကို ကြေငြာပေးရခြင်းဖြစ်ပါတယ်။ Serialize လုပ်တဲ့အချိန်နဲ့ Deserialize လုပ်တဲ့အချိန်တို့မှာ ဖြည့်စွက်လုပ်ဆောင် ချက်တွေရှိရင် သက်ဆိုင်ရာ Callback Function ထဲမှာ ရေးသားသတ်မှတ်ထားနိုင်ပါတယ်။ နမူနာမှာတော့ ဘာမှဖြည့်စွက် ရေးသားထားခြင်းမရှိပဲ Serialize/Deserialize လုပ်ငန်းပြီးစီးကြောင်း ကြေငြာတဲ့ `done()` Function ကိုပဲ အလုပ်လုပ် စေထားပါတယ်။

လိုင်းနံပါတ် (၁၇) မှာတော့ `passport.use()` ကိုသုံးပြီး Local Strategy ကို အသုံးပြုမှာဖြစ်ကြောင်း ကြေငြာ သတ်မှတ်ထားပါတယ်။ တစ်လက်စတည်း Strategy Definition ကိုလည်း တွဲဖက်သတ်မှတ်ပေးထားပါတယ်။ နမူနာမှာ ပေးထားတဲ့ Strategy Definition အရ၊ ပေးလာတဲ့ Username, Password နဲ့ ကိုက်ညီတဲ့ User ရှိမရှိ `db.users.find()` နဲ့ စီစစ်ထားပါတယ်။ ကိုက်ညီတဲ့ User ရှိတယ်ဆိုရင် `done()` Function နဲ့အတူ ရရှိလာတဲ့ User Data တွေကို ပြန်ပေးပြီး၊ ကိုက်ညီတဲ့ User မရှိဘူးဆိုရင်တော့ `done()` Function နဲ့အတူ `false` တန်ဖိုးကို return ပြန်ပေးထားပါတယ်။

Password တွေကိုသိမ်းတဲ့အခါ SHA1 Hash လုပ်ပြီးမှ သိမ်းဖို့ရည်ရွယ်ထားတဲ့အတွက်၊ ပေးလာတဲ့ Password ကိုလည်း SHA1 Hash လုပ်ပြီးမှ တိုက်ဆိုင်စစ်ဆေးထားတာကိုလည်း သတိပြုပါ။

အသုံးပြုမယ့် Strategy ကြေငြာသတ်မှတ်ပြီးတဲ့အခါ Authentication နဲ့ Authorization တို့အတွက် သတ်မှတ်ချက်တွေ ကိုဆက်လက်ရေးသားပါတယ်။ Authentication အတွက် လိုင်းနံပါတ် (၃၃) မှာ `login()` Function ကို ကြေငြာ သတ်မှတ်ထားပါတယ်။ ဒီ Function က ထူးထူးခြားခြား ဘာမှမလုပ်ပဲ `passport`

`.authenticate()` Function ကို ခေါ်ယူပေးထားခြင်းဖြစ်ပါတယ်။ ဒီလိုခေါ်ယူရာမှာ Strategy အမည် အဖြစ် `local` ကိုပေးလိုက်တဲ့အတွက် Request နဲ့အတူပါဝင်လာတဲ့ Username, Password ကိုအသုံးပြုပြီး Credential မှန်မမှန်စီစစ်တဲ့လုပ်ငန်းကို ကျွန်တော်တို့ လိုင်းနံပါတ် (၁၇) မှာ ကြေငြာခဲ့တဲ့ Local Strategy ကို သုံးပြီး အလုပ်လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ Credential မှန်ရင် User Data တွေ ကို Session ထဲမှာ သိမ်းပေးတဲ့ လုပ်ငန်းကို ကျွန်တော်တို့ ကိုယ်တိုင်ရေးစရာမလိုပါဘူး။ PassportJS နဲ့ Passport Local တို့က ကျွန်တော်တို့ ကိုယ်စား လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ တနည်းအားဖြင့် ကျွန်တော်တို့က Strategy Definition ကိုသာ သတ်မှတ်ပေးဖို့လိုပြီး၊ ကျန်လုပ်ငန်းတွေကို PassportJS က လုပ်ပေးသွားတဲ့သဘော ဖြစ်ပါတယ်။

Authorization အတွက် လိုင်းနံပါတ် (၃၇) မှာ ensureAuth() ဆိုတဲ့ Function ကိုရေးသားထားပါတယ်။ လက်ရှိ Request ဟာ Login လုပ်ထားပြီး Access လုပ်ခွင့်ရှိသူ ဟုတ်မဟုတ် စီစစ်တဲ့ Function ဖြစ်ပါတယ်။ ပုံမှန်အားဖြင့် Login လုပ်ထားပြီးသူ ဟုတ်မဟုတ် Session ထဲမှာ ရှိတဲ့ User Data ကို ကြည့်ပြီးစစ်ရမှာပါ။ ဒါပေမယ့် ဒီကိစ္စကိုလည်း ကိုယ်တိုင်လုပ်စရာ မလိုပါဘူး။ PassportJS က လုပ်ပေးတဲ့ အတွက်၊ `req.isAuthenticated()` Function ကို အသင့်ခေါ်ယူ အသုံးပြုပြီး Authenticated User ဟုတ်မဟုတ် စီစစ်နိုင်မှာဖြစ်ပါတယ်။ နမူနာအရ Authenticated User ဆိုရင် Next Route ကို ဆက်သွားခွင့်ပြုတဲ့အနေနဲ့ `next()` Function ကို ခေါ်ယူထားပြီး၊ Authenticated User မဟုတ်ရင်တော့ 401 Status Code ကို ပြန်ပေးဖို့ သတ်မှတ်ထားပါတယ်။

အခြေခံ Authentication နဲ့ Authorization လုပ်ဆောင်ချက်တွေကို ရရှိသွားပြီဖြစ်ပါတယ်။ ဆက်လက်ပြီး User Role နဲ့ ပက်သက်တဲ့ Authorization လုပ်ငန်းတွေ လုပ်ချင်သေးတဲ့အတွက် `auth.js` ထဲမှာ အခုလို ဆက်လက် ဖြည့်စွက် ပေးပါ။

JavaScript

```
45. exports.ensureSuper = function() {
46.   return function(req, res, next) {
47.     if (req.isAuthenticated() && req.user.role === 3)
48.       next();
49.     else
50.       res.sendStatus(401);
51.   }
52. };
53.
54. exports.ensureRole = function(role) {
55.   return function(req, res, next) {
56.     if (req.isAuthenticated() && req.user.role >= role)
57.       next();
58.     else
59.       res.sendStatus(401);
60.   }
61. };
62.
63. exports.ensureOwner = function() {
64.   return function(req, res, next) {
65.     if (req.isAuthenticated() && (req.user._id == req.params.id ||
66.           req.user.role === 3))
67.       next();
```

```
68.     else
69.       res.sendStatus(401);
70.   }
71. };
72.
73. exports.ensureAssignee = function() {
74.   return function(req, res, next) {
75.     var iid = req.params.id;
76.     db.issues.findOne({ _id: mongojs.ObjectId(iid) }, function(err,data){
77.       if(err) {
78.         res.sendStatus(401);
79.       } else {
80.         if (req.isAuthenticated() &&
81.           (req.user._id.str==(data.assignedTo && data.assignedTo.str) ||
82.            req.user.role === 3))
83.           next();
84.         else
85.           res.sendStatus(401);
86.       }
87.     });
88.   }
89. };
90.
91. exports.ensureSubmitter = function() {
92.   return function(req, res, next) {
93.     var iid = req.params.id;
94.     db.issues.findOne({ _id: mongojs.ObjectId(iid) }, function(err,data){
95.       if(err) {
96.         res.sendStatus(401);
97.       } else {
98.         if (req.isAuthenticated() && (req.user._id == data.submittedBy ||
99.             req.user.role === 3))
100.          next();
101.        else
102.          res.sendStatus(401);
103.      }
104.    });
105.  }
106. };
```

PassportJS က Session ထဲမှာ User Data တွေကို ထည့်သွင်းပေးထားယုံသာမက၊ လိုအပ်ရင် အသင့်အသုံးပြုနိုင်အောင် `req.user` ဆိုတဲ့ Property နဲ့လည်း အသင့်ပြင်ဆင်ထားပေးပါတယ်။ ဒါကြောင့် လိုင်းနံပါတ် (၄၅) မှာ `req.user` ကို အသုံးချပြီး `ensureSuper()` Function ကို ကြေငြာသတ်မှတ်ထားပါတယ်။ သတ်မှတ်ချက်အရ User ဟာ Authenticated User ဖြစ်ယုံသာမက `role` တန်ဖိုးလည်း 3 ဖြစ်ရမယ်လို့ သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။ `config.js` မှာ ပြန်လည် လေ့လာကြည့်ရင် Role အနေနဲ့ User, Tester, Developer, Manager လို့ အဆင့်ဆင့် သတ်မှတ်ထားပြီး အမြင့်ဆုံး Manager ရဲ့ Index တန်ဖိုးက 3 ဖြစ်ပါတယ်။ ဒါကြောင့် `ensureSuper()` သတ်မှတ် ချက်အရဆိုရင် Manager Level User မှသာ Access လုပ်ခွင့်ရှိမှာဖြစ်ပါတယ်။ ဒါဟာ သတ်မှတ်ချက်သက်သက်သာ ဖြစ်ပါတယ်။ ဒီသတ်မှတ်ချက်ကို ဘယ်နေရာမှာသုံးမလဲဆိုတာကို မဆုံးဖြတ်ရသေးပါဘူး။

ဆက်လက်ပြီး (၅၄) မှာ `ensureRole()` Function ကို ကြေငြာသတ်မှတ်ထားပါတယ်။ ဒီ Function ရဲ့ ရည်ရွယ် ချက်ကတော့ Access Right ကို Role နဲ့တွဲဖက်သတ်မှတ်ထားဖို့ဖြစ်ပါတယ်။ ဥပမာ – `ensureRole(2)` လို့ပြော လိုက်ရင် User Role Level တန်ဖိုး 2 ကိုသာ Access လုပ်ခွင့်ပြုမှာဖြစ်ပါတယ်။ သတိပြုသင့်တာကတော့ Manager Level ဖြစ်တဲ့ 3 ဆိုရင်လည်း Access လုပ်ခွင့်ပြုကြောင့်သတ်မှတ်ထားပါ တယ်။ ဒါကြောင့် `ensureRole(2)` လို့ ပြောလိုက်ရင် User Role Level တန်ဖိုး 2 ရှိတဲ့ User (သို့မဟုတ်) Manager Level User တွေကို Access လုပ်ခွင့်ပြု တယ်ဆိုတဲ့ အဓိပ္ပါယ်ဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၆၃) မှာတော့ `ensureOwner()` Function ကို ကြေငြာသတ်မှတ်ထားပါတယ်။ ဒီ Function ရဲ့ ရည်ရွယ်ချက်က၊ လက်ရှိ Access လုပ်ဖို့ကြိုးစားတဲ့ User Information က လက်ရှိ Authenticated ဖြစ်နေတဲ့ User ရဲ့ ကိုယ်ပိုင် Information ဟုတ်သလားစီစစ်ပေးထားခြင်းဖြစ်ပါတယ်။ ပုံမှန်အားဖြင့် အရေးကြီးတဲ့ User Information ပြုပြင် တဲ့လုပ်ငန်းတွေကို Manager Level ကိုသာခွင့်ပြုပေးမှာဖြစ်ပါတယ်။ ဒါပေမယ့် User တစ် ယောက်က ကိုယ့် Information ကို ကိုယ်တိုင်ပြုပြင်ခြင်းကို ခွင့်ပြုသင့်တဲ့အတွက်၊ ခွင့်ပြုနိုင်အောင် ဒီသတ်မှတ် ချက်ကို ထည့်သွင်းသတ်မှတ်ထားခြင်းဖြစ် ပါတယ်။ Access လုပ်ဖို့ကြိုးစားတဲ့ User ရဲ့ ID နဲ့ Session ထဲမှာ သိမ်းထားတဲ့ လက်ရှိ Authenticated User ရဲ့ ID တို့ တိုက်ဆိုင် စစ်ဆေးခြင်းအားဖြင့် Owner ဟုတ်မဟုတ် သိရှိ နိုင်မှာဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၇၃) မှာတော့ `ensureAssignee()` Function ကို ကြေငြာသတ်မှတ်ထားပါတယ်။ ပုံမှန် အားဖြင့် Issue တစ်ခုရဲ့ Status ကို Manager မှသာ ပြောင်းခွင့်ရှိမှာဖြစ်ပါတယ်။ ဒါပေမယ့် Assign လုပ်ခြင်းခံရ တဲ့ User ကလည်း Status ကို ပြောင်းလိုတဲ့အခါ ပြောင်းခွင့်ပြုနိုင်ဖို့အတွက် ဒီသတ်မှတ်ချက်ကို ထည့်သွင်း သတ်မှတ်ထားခြင်း ဖြစ်ပါတယ်။ လက်ရှိ ပြင်ဆင်ဖို့ကြိုးစားတဲ့ Issue ရဲ့ Assigned To တန်ဖိုးဟာ၊ Authenticated User ရဲ့ ID နဲ့ တူညီမှုရှိမရှိ တိုက်ဆိုင်စစ်ဆေးခြင်းအားဖြင့် Assign လုပ်ခြင်းခံထားရတဲ့ User ဟုတ်မဟုတ်သိနိုင်ပါတယ်။ Issue ရဲ့ Assigned To တန်ဖိုးကို သိရဖို့အတွက် Database ကနေ သွားရောက်စီစစ် ဖို့လိုတဲ့အတွက် ဒီနေရာမှာတော့ Database နဲ့ ချိတ်ဆက်အသုံးပြုထားရပါတယ်။

လိုင်းနံပါတ် (၉၁) က `ensureSubmitter()` ကတော့ `ensureAssignee()` နဲ့ သဘာသဘာဝဆင်တူ ပါ တယ်။ Issue ကို မူလစတင် ထည့်သွင်းခဲ့သူကိုလည်း ပြင်ဆင်ခွင့်ကို ပေးလိုကပေးနိုင်စေဖို့အတွက် ထည့်သွင်း သတ်မှတ် ထားခြင်းပဲဖြစ်ပါတယ်။

အခုဆိုရင် Authorization နဲ့ Authentication ပိုင်းဆိုင်ရာ သတ်မှတ်ချက်တွေ ပြည့်စုံသွားပြီပဲဖြစ်ပါတယ်။

13.4 – User Router – `routes/users.js`

ဆက်လက်ပြီး User Login, Register နဲ့ Create, Read, Update, Delete လုပ်ငန်းတွေကို `routes/users.js` ထဲမှာ ဆက်လက်ရေးသားပါမယ်။

JavaScript

```
var mongojs = require("mongojs");
var crypto = require('crypto');
var express = require("express");
var validator = require('express-validator');
var router = express.Router();

var auth = require("../auth");
var config = require("../config");

var db = mongojs('mydb', ['users']);

router.post("/login", auth.login(), function(req, res) {
  if(req.user) res.status(200).json(req.user);
  else res.sendStatus(401);
});

router.delete("/logout", function(req, res) {
  // req.logout();
  req.session.destroy();
  res.status(200).json({"logout": "done"});
});

router.get("/verify", auth.ensureAuth(), function(req, res) {
  res.status(200).json(req.user);
});
```

ပထမဆုံးအနေနဲ့ သတိပြုသင့်တာကတော့၊ လိုင်းနံပါတ် (၄) မှာ ချိတ်ဆက်ထားတဲ့ `express-validator` ပါ။ Request နဲ့ အတူပါဝင်လာတဲ့ Data တွေကို Validate လုပ်ဖို့အတွက် ဒီ Package ကို အသုံးပြုမှာဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၇) နဲ့ (၈) တို့မှာ ပြီးခဲ့တဲ့အဆင့်တွေက ရေးသားထားခဲ့တဲ့ `config.js` နဲ့ `auth.js` တို့ကို ချိတ်ဆက်ထားပါတယ်။ `.js` Extension က မထည့်လည်းရတဲ့အတွက် နမူနာမှာတော့ `.js` Extension တွေကို ထည့်မထားပါဘူး။

Login လုပ်ငန်း ဆောင်ရွက်နိုင်စေဖို့အတွက် လိုင်းနံပါတ် (၁၂) မှာ `/login` URI နဲ့ POST Route တစ်ခုကို သတ်မှတ် ထားပါတယ်။ `auth.js` ထဲမှာရေးသားထားခဲ့တဲ့ `login()` Function အလုပ်လုပ်သွားစေဖို့အတွက် `auth.login()` ကို Route Function အနေနဲ့ သတ်မှတ်ပေးထားပါတယ်။ `auth.login()` က ပါလာတဲ့ Username, Password စစ်ပြီး မှန်တယ်ဆိုမှ Session ထဲမှာ User Data တွေကို သိမ်းပေးမှာဖြစ်ပါတယ်။ ဒါကြောင့် ဒုတိယ Route Function ထဲမှာ `req.user` ရှိ မရှိကို စီစစ်ပြီး ရှိတော့မှ 200 Status Code နဲ့ အတူ User Data တွေကို Response ပြန်ပေးပြီး မရှိရင် 401 ကိုသာ Response ပြန်ပေးဖို့ သတ်မှတ်ထားပါတယ်။

လိုင်းနံပါတ် (၁၇) မှာတော့ Logout လုပ်ငန်းအတွက် `/logout` URI နဲ့ DELETE Route တစ်ခု သတ်မှတ်ထားပါတယ်။ Logout လုပ်ဖို့အတွက်နည်းလမ်းနှစ်မျိုးရှိပါတယ်။ တစ်နည်းကတော့ `req.logout()` Function ကို ခေါ်ယူ လိုက်ခြင်းဖြစ်ပြီး၊ နောက်တစ်နည်းကတော့ Session ထဲက တန်ဖိုးတွေကို ပယ်ဖျက်လိုက်ခြင်းပဲ ဖြစ်ပါတယ်။ နမူနာမှာ တော့ Session ထဲက တန်ဖိုးတွေကို `req.session.destory()` နဲ့ ပယ်ဖျက်ထားပါတယ်။

Authenticated User ဟုတ်မဟုတ်စီစစ်တဲ့လုပ်ငန်းအတွက် `/verify` URI နဲ့ GET Route တစ်ခုကို လိုင်းနံပါတ် (၂၃) မှာသတ်မှတ်ထားပါတယ်။ ဒီ Route မှာတော့ `auth.js` ထဲက `ensureAuth()` Function ကိုအသုံးပြုထား ပါတယ်။ `ensureAuth()` က Login ဝင်ထားတဲ့အတွက် Session တန်ဖိုးရှိရင် `next()` Function နဲ့ ရှေ့ဆက် သွားခွင့်ပြုထားတဲ့အတွက်၊ နောက် Route Function ထဲမှာ သတ်မှတ်ထားတဲ့ 200 OK ကို Response လုပ်ပေးသွား မှာဖြစ်ပြီး၊ Login ဝင်မထားတဲ့အတွက် Session ရှိမရှိနေရင် 401 Status Code ကို ပြန်ပေးသွား မှာဖြစ်ပါတယ်။

ဆက်လက်ပြီး User အသစ်ထည့်သွင်းတဲ့ လုပ်ဆောင်ချက်အတွက် အောက်ပါအတိုင်း ဖြည့်စွက်ပေးရပါမယ်။

JavaScript

```
26. router.post("/", auth.ensureSuper(), function(req, res) {
27.   // Validation
28.   req.checkBody('fullName', 'Invalid Full Name').notEmpty();
29.   req.checkBody('loginId', 'Invalid Login ID').isAlphanumeric();
30.   req.checkBody('email', 'Invalid Email Address').isEmail();
31.   req.checkBody('role', 'Invalid Role').isInt();
32.   req.checkBody('password', 'Password required 6 characters').isLength(6);
33.
34.   var errors = req.validationErrors();
35.   if(errors) {
36.     res.status(400).json(errors);
37.     return false;
38.   }
39.
40.   req.body.password = crypto.createHash('sha1')
41.                             .update(req.body.password)
42.                             .digest("hex");
43.   req.body.roleLabel = config.role[req.body.role];
44.
45.   db.users.insert(req.body, function(err, data) {
46.     if(err) res.status(500).json(err);
47.     else res.status(200).json(data);
48.   });
49. });
```

User အသစ်ထည့်သွင်းဖို့အတွက် / URL မှာ POST Rout တစ်ခုသတ်မှတ်ထားပါတယ်။ User အသစ်ထည့်သွင်းသူ ဟာ Manager Role ရှိသူဖြစ်သင့်တဲ့အတွက် `ensureSuper()` Function ကို တစ်ဆင့်ခံပြီး စီစစ်ထားပါတယ်။ Manager မဟုတ်သူက ဒီ Route ကို အသုံးပြုဖို့ကြိုးစားရင် 401 Status Code ပြန်ပေးသွားမှာဖြစ်ပါတယ်။

User တစ်ယောက် ထည့်သွင်းတဲ့အခါ ပါဝင်သင့်တဲ့ အချက်အလက်တွေ အားလုံးပါဝင်တာ သေချာအောင်လို့ လိုင်းနံပါတ် (၂၈) ကနေ (၃၂) ထိ Validation စစ်ထားပါတယ်။ `express-validator` က `req.checkBody()` Function နဲ့ Request Body မှာပါဝင်လာတဲ့ တန်ဖိုးတွေကို Validate စစ်နိုင်အောင် ဆောင်ရွက်ပေးထားတဲ့အတွက် `req.checkBody()` ကို အသုံးပြု စီစစ်ထားခြင်းပဲ ဖြစ်ပါတယ်။ `checkBody()` Function အတွက်၊ စီစစ်လိုတဲ့ Field နဲ့ ဖော်ပြ စေလိုတဲ့ Error Message ကိုပေးရပါတယ်။ နမူ

နာအရ `isEmpty()` နဲ့ နောက်ဆုံးကနေ စစ်ထားတဲ့အတွက် `fullName` ပါဝင်လာခြင်းမရှိရင် Invalid Full Name ဆိုတဲ့ Error Message ကို ရရှိမှာဖြစ်ပါတယ်။

ဆက်လက်ပြီး `loginId` ကို Alpha-Numeric ပဲ ဖြစ်ရမယ်လို့ စီစစ်သတ်မှတ်ထားပါတယ်။ `email` ကိုတော့ `isEmail()` နဲ့ Email Format မှန်ရမယ်လို့ စီစစ်ထားပြီး၊ `role` ကိုတော့ `isInt()` နဲ့ Integer ဖြစ်ရမယ်လို့ စီစစ်ထားပါတယ်။ password အတွက်ကိုတော့ `isLength(6)` နဲ့ အနည်းဆုံး (၆) လုံးရှိရမယ်လို့ သတ်မှတ်ထားပါ တယ်။

`express-validator` က စီစစ်ယုံပဲ စီစစ်ပေးတာပါ။ ရလာတဲ့လဒ်ပေါ်မူတည်ပြီး ဘာလုပ်ရမလဲဆိုတာကိုတော့ ကိုယ်တိုင် သတ်မှတ်ပေးရမှာဖြစ်ပါတယ်။ ဒါကြောင့် လိုင်းနံပါတ် (၃၅) မှာ `req.validationErrors()` Function နဲ့ Error ရှိမရှိ စစ်ဆေးပြီး Error ရှိတဲ့အခါ 400 Status Code နဲ့အတူ ရရှိလာတဲ့ Error Message တွေကို Response ပြန်ပေးဖို့ ဆက်လက်သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။

ပြီးတဲ့အခါ၊ အချက်အလက်တွေကို Database ထဲမှာ မထည့်သွင်းမီ Password ကို SHA1 Hash ပြောင်းပါတယ်။ ပါဝင်လာတဲ့ `role` ကိုအသုံးပြုပြီး User, Developer, Manager စတဲ့ `roleLable` ကို `req.body` မှာ တစ်ခါတည်း တွဲပေးပါတယ်။ ပြီးတော့မှ လိုင်းနံပါတ် (၄၅) မှာ တန်ဖိုးတွေကို Database ထဲက `users` Collection မှာ ထည့်သွင်း ပေးလိုက်ခြင်းအားဖြင့် User အသစ် ထည့်သွင်းတဲ့ လုပ်ဆောင်ချက်ကို ရရှိသွားမှာပဲ ဖြစ်ပါတယ်။

ဆက်လက်ပြီး ကျန်လုပ်ငန်းတွေဖြစ်တဲ့ User စာရင်းရယူခြင်း၊ User ကိုပယ်ဖျက်ခြင်း၊ User Role ပြုပြင်ခြင်းစတဲ့ လုပ်ငန်းတွေအတွက် အခုလို ဆက်လက်ရေးသားပေးရပါမယ်။

JavaScript

```
51. // Get all users
52. router.get("/", auth.ensureAuth(), function(req, res) {
53.   db.users.find({}, function(err, data) {
54.     res.status(200).json(data);
55.   });
56. });
57.
58. // Get user roles
59. router.get("/roles", auth.ensureAuth(), function(req, res) {
60.   res.status(200).json(config.role);
61. });
62.
63. // Get one user
64. router.get("/:id", auth.ensureAuth(), function(req, res) {
65.   var uid = req.params.id;
66.
67.   // Validation
68.   req.checkParams("id", "Invalid User ID").isMongoId();
69.   var errors = req.validationErrors();
70.   if(errors) {
71.     res.status(400).json(errors);
72.     return false;
```

```
  }

  db.users.findOne({_id: mongojs.ObjectId(uid)}, function(err, data) {
    if(data) res.status(200).json(data);
    else res.sendStatus(404);
  });
});

// Update a user
router.put("/:id", auth.ensureOwner(), function(req, res) {
  var uid = req.params.id;

  // Validation
  req.checkParams("id", "Invalid User ID").isMongoId();
  req.checkBody('fullName', 'Invalid Full Name').isLength(3);
  req.checkBody('loginId', 'Invalid Login ID').isAlphanumeric();
  req.checkBody('email', 'Invalid Email Address').isEmail();
  req.checkBody('role', 'Invalid Role').isInt();

  var errors = req.validationErrors();
  if(errors) {
    res.status(400).json(errors);
    return false;
  }

  var newData = {
    fullName: req.body.fullName,
    loginId: req.body.loginId,
    email: req.body.email,
    role: req.body.role,
    roleLabel: config.role[req.body.role]
  };

  if(req.body.password) {
    var pwdHash = crypto.createHash('sha1')
                        .update(req.body.password)
                        .digest("hex");
    newData.password = pwdHash;
  }

  db.users.update(
    { _id: mongojs.ObjectId(uid) },
    { $set: newData },
    { multi: false },
    function(err, data) {
      if(err) res.status(500).json(err);
      else res.status(200).json(data);
    }
  );
});

router.delete("/:id", auth.ensureSuper(), function(req, res) {
  var uid = req.params.id;

  // Validation
  req.checkParams("id", "Invalid User ID").isMongoId();
  var errors = req.validationErrors();
  if(errors) {
    res.status(400).json(errors);
```

```
133.     return false;
134.   }
135.
136.   db.users.remove({_id: mongojs.ObjectId(uid)}, function(err, data) {
137.     if(data) res.status(200).json(data);
138.     else res.sendStatus(404);
139.   });
140. });
141.
142. module.exports = router;
```

ဒီ Code တွေကတော့ Create, Read, Update, Delete လုပ်ငန်းတွေသာဖြစ်လို့ ရှင်းပြနေစရာမလိုတော့ဘူးလို့ ထင်ပါတယ်။ ရေးထားတဲ့ Code ကို လေ့လာကြည့်လိုက်ရင် အလုပ်လုပ်ပုံကို နားလည်မှာပါ။ ထူးခြားချက်အနေနဲ့ လိုင်းနံပါတ် (၆၈) `express-validate` ရဲ့ `req.checkParams()` ကို အသုံးပြုပြီး URL Parameter ကို Validation စစ်ထား ချက်တစ်ခုရှိပါတယ်။ Validation Function အနေနဲ့ `isMongoId()` ကိုအသုံးပြုထားပါတယ်။ URL Parameter အဖြစ်ပါဝင်လာတဲ့ `id` ဟာ MongoDB ID Format ဟုတ်မှန်ရဲ့လား စစ်ဆေးထားခြင်းဖြစ်ပါတယ်။

နောက်ဆုံးရရှိမယ့် Route Pattern ကတော့ အခုလိုဖြစ်မှာပါ –

Method	Route	Description	Access Control
POST	/login	User Login ပြုလုပ်ရန်	မည်သူမဆိုအသုံးပြုနိုင်
DELETE	/logout	User Logout ပြုလုပ်ရန်	မည်သူမဆိုအသုံးပြုနိုင်
GET	/verify	Authenticated User ဟုတ်မဟုတ်စီစစ်ရန်	မည်သူမဆိုအသုံးပြုနိုင်
POST	/	User အသစ်ထည့်သွင်းရန်	Manager Only
GET	/	User စာရင်းရယူရန်	Authenticated User Only
GET	/roles	User Role စာရင်းရယူရန်	Authenticated User Only
GET	/:id	User တစ်ဦးအတွက် Data ရယူရန်	Authenticated User Only
PUT	/:id	User တစ်ဦး၏ Data ကို Update ပြုလုပ်ရန်	Account Owner & Manager
DELETE	/:id	User တစ်ဦးအား ပယ်ဖျက်ရန်	Manager Only

13.5 – Issues Router – `routes/issues.js`

ဆက်လက်ပြီး Issue နဲ့ပက်သက်တဲ့ Code တွေကို ဆက်လက်ရေးသားဖို့အတွက် `routes/issues.js` ထဲမှာ အခုလိုရေး သားပေးရပါမယ်။

JavaScript

```
var mongojs = require("mongojs");
var express = require("express");
var validator = require('express-validator');
var router = express.Router();

var auth = require("../auth");
var config = require("../config");

var db = mongojs('mydb', ['issues']);

// Get all issues
router.get("/", auth.ensureAuth(), function(req, res) {
  db.issues.find({}, function(err, data) {
    res.status(200).json(data);
  });
});

// Get one issue
router.get("/:id", auth.ensureAuth(), function(req, res) {
  var iid = req.params.id;

  // Validation
  req.checkParams("id", "Invalid Issue ID").isMongoId();
  var errors = req.validationErrors();
  if(errors) {
    res.status(400).json(errors);
    return false;
  }

  db.issues.findOne({_id: mongojs.ObjectId(iid)}, function(err, data) {
    if(data) res.status(200).json(data);
    else res.sendStatus(404);
  });
});

// Create a new issue
router.post("/", auth.ensureRole(1), function(req, res) {
  // Validation
  req.checkBody('summary', 'Invalid Summary').notEmpty();
  req.checkBody('priority', 'Invalid Priority').isInt();
  req.checkBody('type', 'Invalid Type').isInt();

  var errors = req.validationErrors();
  if(errors) {
    res.status(400).json(errors);
    return false;
  }

  var newIssue = {
    summary: req.body.summary,
    detail: req.body.detail,
```

```
    priority: req.body.priority,
    type: req.body.type,
    typeLabel: config.type[req.body.type],
    priorityLabel: config.priority[req.body.priority],
    status: 0,
    statusLabel: config.status[0],
    assignedTo: null,
    assignedToLabel: null,
    submittedAt: new Date(),
    submittedBy: mongojs.ObjectId(req.user._id),
    submittedByLabel: req.user.fullName
  }

  db.issues.insert(newIssue, function(err, data) {
    if(err) res.status(500).json(err);
    else res.status(200).json(data);
  });
});

// Update an issue
router.put("/:id", auth.ensureRole(1), function(req, res) {
  var iid = req.params.id;

  // Validation
  req.checkParams("id", "Invalid Issue ID").isMongoId();
  req.checkBody("summary", "Invalid Summary").notEmpty();

  var errors = req.validationErrors();
  if(errors) {
    res.status(400).json(errors);
    return false;
  }

  var newData = {
    summary: req.body.summary,
    detail: req.body.detail,
    modifiedAt: new Date()
  };

  db.issues.update(
    { _id: mongojs.ObjectId(iid) },
    { $set: newData },
    { multi: false },
    function(err, data) {
      if(err) res.status(500).json(err);
      else res.status(200).json(data);
    }
  );
});

// Delete an issue
router.delete("/:id", auth.ensureSubmitter(), function(req, res) {
  var iid = req.params.id;

  // Validation
  req.checkParams("id", "Invalid Issue ID").isMongoId();
  var errors = req.validationErrors();
  if(errors) {
    res.status(400).json(errors);
```

```
    return false;
  }

  db.issues.remove({_id: mongojs.ObjectId(iid)}, function(err, data) {
    if(data) res.status(200).json(data);
    else res.sendStatus(404);
  });
});
```

ထူးခြားတဲ့လုပ်ဆောင်ချက် မပါဝင်ပဲ၊ issues Collection ပေါ်မှာ Create, Read, Update, Delete လုပ်ငန်းတွေ ဆောင်ရွက်ထားခြင်းသာဖြစ်ပါတယ်။ ဆက်လက်ပြီး Issue တစ်ခုရဲ့ Status, Priority နဲ့ Assignment တို့ကို သီးခြားစီ ပြင်ဆင်နိုင်ဖို့အတွက် အခုလို ရေးသားပေးရပါမယ်။

JavaScript

```
// Update issue type
router.patch("/type/:id", auth.ensureRole(1), function(req, res) {
  var iid = req.params.id;

  // Validation
  req.checkParams("id", "Invalid Issue ID").isMongoId();
  req.checkBody("type", "Invalid Type").isInt();

  var errors = req.validationErrors();
  if(errors) {
    res.status(400).json(errors);
    return false;
  }

  var newData = {
    type: req.body.type,
    typeLabel: config.type[req.body.type],
    modifiedAt: new Date()
  };

  db.issues.update(
    { _id: mongojs.ObjectId(iid) },
    { $set: newData },
    { multi: false },
    function(err, data) {
      if(err) res.status(500).json(err);
      else res.status(200).json(data);
    }
  );
});

// Update issue priority
router.patch("/priority/:id", auth.ensureRole(2), function(req, res) {
  var iid = req.params.id;

  // Validation
  req.checkParams("id", "Invalid Issue ID").isMongoId();
  req.checkBody("priority", "Invalid Priority").isInt();

```

```
160.   var errors = req.validationErrors();
161.   if(errors) {
162.     res.status(400).json(errors);
163.     return false;
164.   }
165.
166.   var newData = {
167.     priority: req.body.priority,
168.     priorityLabel: config.priority[req.body.priority],
169.     modifiedAt: new Date()
170.   };
171.
172.   db.issues.update(
173.     { _id: mongojs.ObjectId(iid) },
174.     { $set: newData },
175.     { multi: false },
176.     function(err, data) {
177.       if(err) res.status(500).json(err);
178.       else res.status(200).json(data);
179.     }
180.   );
181. });
182.
183. // Update issue assign to
184. router.patch("/assign/:id", auth.ensureSuper(), function(req, res) {
185.   var iid = req.params.id;
186.
187.   // Validation
188.   req.checkParams("id", "Invalid Issue ID").isMongoId();
189.   req.checkBody("assignedTo", "Invalid User ID").isMongoId();
190.   req.checkBody("assignedToLabel", "Invalid User Name").notEmpty();
191.
192.   var errors = req.validationErrors();
193.   if(errors) {
194.     res.status(400).json(errors);
195.     return false;
196.   }
197.
198.   var newData = {
199.     assignedTo: req.body.assignedTo,
200.     assignedToLabel: req.body.assignedToLabel,
201.     status: 1,
202.     statusLabel: config.status[1],
203.     modifiedAt: new Date()
204.   };
205.
206.   db.issues.update(
207.     { _id: mongojs.ObjectId(iid) },
208.     { $set: newData },
209.     { multi: false },
210.     function(err, data) {
211.       if(err) res.status(500).json(err);
212.       else res.status(200).json(data);
213.     }
214.   );
215. });
216.
217. // Update issue status
```

```
218. router.patch("/status/:id", auth.ensureAssignee(), function(req, res) {
219.   var iid = req.params.id;
220.
221.   // Validation
222.   req.checkParams("id", "Invalid Issue ID").isMongoId();
223.   req.checkBody("status", "Invalid Status").isInt();
224.
225.   var errors = req.validationErrors();
226.   if(errors) {
227.     res.status(400).json(errors);
228.     return false;
229.   }
230.
231.   var newData = {
232.     status: req.body.status,
233.     statusLabel: config.status[req.body.status],
234.     modifiedAt: new Date()
235.   };
236.
237.   db.issues.update(
238.     { _id: mongojs.ObjectId(iid) },
239.     { $set: newData },
240.     { multi: false },
241.     function(err, data) {
242.       if(err) res.status(500).json(err);
243.       else res.status(200).json(data);
244.     }
245.   );
246. });
247.
248. // Close an issue
249. router.patch("/close/:id", auth.ensureSuper(), function(req, res) {
250.   var iid = req.params.id;
251.
252.   // Validation
253.   req.checkParams("id", "Invalid Issue ID").isMongoId();
254.
255.   var errors = req.validationErrors();
256.   if(errors) {
257.     res.status(400).json(errors);
258.     return false;
259.   }
260.
261.   var newData = {
262.     status: 4,
263.     statusLabel: config.status[4],
264.     modifiedAt: new Date()
265.   };
266.
267.   db.issues.update(
268.     { _id: mongojs.ObjectId(iid) },
269.     { $set: newData },
270.     { multi: false },
271.     function(err, data) {
272.       if(err) res.status(500).json(err);
273.       else res.status(200).json(data);
274.     }
275.   );
```

```
276. });
```

ဒီနေရာမှာ သတိပြုသင့်တာကတော့ PATCH Request Method ကို အသုံးပြုထားခြင်းပဲဖြစ်ပါတယ်။ အခြေခံအားဖြင့် အချက်အလက်တွေ ပြင်ဆင်ဖို့အတွက် PUT Request Method သုံးပါတယ်။ ဒါပေမယ့် လက်တွေ့မှာ HTTP သတ်မှတ်ချက်အရ PUT ကို အချက်အလက်တစ်ချို့ ပြင်ဆင်ဖို့ဆိုမှာ မသုံးသင့်ပဲ၊ အချက်အလက်အားလုံးကို အစားထိုးဖို့သာ အသုံးပြုသင့်ပါတယ်။ တနည်းအားဖြင့် Record Set ကြီးတစ်ခုလုံးကို သီးခြား Record Set တစ်ခုနဲ့ အစားထိုးလိုက်တဲ့ ပြင်ဆင်မှုမျိုးအတွက် PUT ကို အသုံးပြုရခြင်းဖြစ်ပြီး၊ Record Set တစ်ခုရဲ့ အစိတ်အပိုင်းတစ်ချို့ကိုသာ ပြင်ဆင်လိုရင်တော့ PATCH Request Method ကို အစားထိုးအသုံးပြုသင့်ပါတယ်။ Issue တစ်ခုရဲ့ Priority ပြင်ဆင်ခြင်း၊ Status ပြင်ဆင် ခြင်း စတဲ့လုပ်ငန်းတွေဟာ Issue တစ်ခုလုံးကို အစားထိုးပြင်ဆင်ခြင်းမဟုတ်ပဲ အစိတ်အပိုင်းတစ်ခုကိုသာ ပြင်ဆင်ခြင်းဖြစ်တဲ့အတွက် ဒီနေရာမှာ PATCH Request Method ကို အသုံးပြုခြင်းပဲဖြစ်ပါတယ်။

ဆက်လက်ပြီး Issue Comment ထည့်သွင်းခြင်းနဲ့၊ ပယ်ဖျက်ခြင်းတို့အတွက် အခုလိုရေးသားပေးရပါမယ်။

JavaScript

```
277. // Comments
278. router.post("/comments", auth.ensureAuth(), function(req, res) {
279.   req.checkBody('issueId', 'Invalid Issue ID').isMongoId();
280.   req.checkBody('authorId', 'Invalid Author ID').isMongoId();
281.   req.checkBody('authorName', 'Invalid Author Name').notEmpty();
282.   req.checkBody('comment', 'Invalid Comment Body').notEmpty();
283.
284.   var errors = req.validationErrors();
285.   if(errors) {
286.     res.status(400).json(errors);
287.     return false;
288.   }
289.
290.   db.issues.findOne({
291.     _id: mongojs.ObjectId(req.body.issueId)
292.   }, function(err, data) {
293.     if(data) {
294.       var comments = data.comments || [];
295.       var comment = {
296.         comment: req.body.comment,
297.         authorId: req.body.authorId,
298.         authorName: req.body.authorName,
299.         submittedAt: new Date()
300.       }
301.
302.       comments.push(comment);
303.
304.       db.issues.update(
305.         { _id: mongojs.ObjectId(req.body.issueId) },
306.         { $set: { comments: comments } },
307.         { multi: false },
308.         function(err, data) {
309.           if(err) res.status(500).json(err);
310.           else res.status(200).json(data);
311.         }
312.       );
```

```
313.     }
314.     else {
315.       res.sendStatus(404);
316.     }
317.   });
318. });
319.
320. router.delete("/comments", auth.ensureAuth(), function(req, res) {
321.   req.checkBody('issueId', 'Invalid Issue ID').isMongoId();
322.   req.checkBody('commentId', 'Invalid Comment ID').isInt();
323.
324.   var errors = req.validationErrors();
325.   if(errors) {
326.     res.status(400).json(errors);
327.     return false;
328.   }
329.
330.   db.issues.findOne({
331.     _id: mongojs.ObjectId(req.body.issueId)
332.   }, function(err, data) {
333.     if(data) {
334.       var comments = data.comments || [];
335.       comments.splice(req.body.commentId, 1);;
336.
337.       db.issues.update(
338.         { _id: mongojs.ObjectId(req.body.issueId) },
339.         { $set: { comments: comments } },
340.         { multi: false },
341.         function(err, data) {
342.           if(err) res.status(500).json(err);
343.           else res.status(200).json(data);
344.         }
345.       );
346.     }
347.     else {
348.       res.sendStatus(404);
349.     }
350.   });
351. });
```

ဒီနေရာမှာ ထူးခြားချက်အနေနဲ့ Comment တွေကို သီးခြား Collection နဲ့မသုံးပဲ၊ Issue နဲ့အတူ သက်ဆိုင်ရာ Comment တွေကို တစ်ခါတည်း တွဲဖက်သိမ်းဆည်းမှာဖြစ်ပါတယ်။ Comment တွေကို သီးခြား Collection နဲ့ သိမ်းမယ် ဆိုရင် `issue` နဲ့ `comment` Collection နှစ်ခုအကြား Relationship တစ်ခုရှိသွားတော့မှာဖြစ်ပါတယ်။ MongoDB ရဲ့ NoSQL သဘာဝအရ ဖြစ်နိုင်ရင် Relationship ကိုရှောင်သင့်တဲ့အတွက် သီးခြားမသိမ်းပဲ Issue နဲ့အတူ တွဲဖက်သိမ်း ဆည်းခြင်းဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Issue တစ်ခုကို ရယူလိုက်ရင်၊ သက်ဆိုင်ရာ Comment တွေကို သီးခြားထပ်မံရယူစရာ မလိုပဲ တစ်ခါတည်းပါဝင်သွားမှာဖြစ်ပါတယ်။ ဒါကြောင့် Comment တည်ဆောက်ခြင်းနဲ့၊ ပယ်ဖျက်ခြင်းတို့ကိုသာ ရေးသား ထားခြင်းဖြစ်ပါတယ်။ Comment တွေရယူခြင်းကို ထည့်သွင်းရေးသားမထားပါဘူး။ Issue တစ်ခုရယူလိုက်ရင် Comment တွေက တစ်ခါတည်း ပါဝင်သွားမှာပဲ ဖြစ်ပါတယ်။

ဆက်လက်ပြီး Issue Router ရဲ့နောက်ဆုံးပိုင်းအနေနဲ့ Issue Status, Priority နဲ့ Type တို့ကို ရယူနိုင်တဲ့ Route

တွေကို ဆက်လည်ထည့်သွင်းပါမယ်။

JavaScript

```
352. // Get statuses
353. router.get("/statuses", auth.ensureAuth(), function(req, res) {
354.   res.status(200).json(config.status);
355. });
356.
357. // Get priorities
358. router.get("/priorities", auth.ensureAuth(), function(req, res) {
359.   res.status(200).json(config.priority);
360. });
361.
362. // Get types
363. router.get("/types", auth.ensureAuth(), function(req, res) {
364.   res.status(200).json(config.type);
365. });
367.
368. module.exports = router;
```

အားလုံးပြီးစီးသွားတဲ့အခါမှာ ရရှိမယ့် Route Pattern ကတော့ အခုလိုဖြစ်မှာပဲဖြစ်ပါတယ်။

Method	URI	Description	Access Control
GET	/	Issue စာရင်းရယူရန်	Authenticated User Only
GET	/:id	Issue တစ်ခုအတွက် Data ရယူရန်	Authenticated User Only
POST	/	Issue အသစ်တစ်ခုထည့်သွင်းရန်	Tester & Above
PUT	/:id	Issue တစ်ခု၏ Data ပြုပြင်ရန်	Tester & Above
DELETE	/:id	Issue တစ်ခုအား ပယ်ဖျက်ရန်	Submitter & Manager
PATCH	/type/:id	Issue Type ပြောင်းရန်	Tester & Above
PATCH	/priority/:id	Issue Priority ပြောင်းရန်	Developer & Above
PATCH	/assign/:id	Issue အတွက် တာဝန်ကျသူပြောင်းရန်	Manager Only
PATCH	/status/:id	Issue Status ပြောင်းရန်	Assignee & Manager
PATCH	/close/:id	Issue Status ကို Closed ပြောင်းရန်	Manager Only
POST	/comments	Comment တစ်ခုထည့်သွင်းရန်	Authenticated User Only
DELETE	/comments	Comment တစ်ခုပယ်ဖျက်ရန်	Authenticated User Only
GET	/statuses	Issue Status စာရင်းရယူရန်	Authenticated User Only
GET	/priorities	Issue Priority စာရင်းရယူရန်	Authenticated User Only
GET	/types	Issue Type စာရင်းရယူရန်	Authenticated User Only

အခုဆိုရင် လိုအပ်တဲ့ လုပ်ဆောင်ချက်တွေ ပြည့်စုံပြီမို့ Main App ကို စတင်ရေးသားနိုင်ပြီဖြစ်ပါတယ်။

13.6 – Main App – `app.js`

ဆက်လက်ပြီး `app.js` မှာ အခုလိုရေးသားပေးရမှာဖြစ်ပါတယ်။

JavaScript

```
var express = require("express");
var validator = require('express-validator');
var cookieParser = require("cookie-parser");
var bodyParser = require("body-parser");

var issues = require("./routes/issues");
var users = require("./routes/users");

var app = express();

var passport = require("passport");

app.use(require('express-session')({
  secret: 'secret',
  resave: false,
  saveUninitialized: false
}));

app.use(passport.initialize());
app.use(passport.session());

app.use(bodyParser.json());
app.use(bodyParser.urlencoded({
  extended: true
}));
app.use(cookieParser());

app.use(validator());

app.use(function(req, res, next) {
  res.set("Access-Control-Allow-Origin", "http://localhost:8080/");
  res.set("Access-Control-Allow-Credentials", "true");
  res.set("Access-Control-Allow-Methods",
            "GET, POST, PUT, DELETE, PATCH, OPTIONS");
  res.set("Access-Control-Allow-Headers", "X-CSRF-Token, X-Requested-With,
            Accept, Accept-Version, Content-Length, Content-MD5,
            Content-Type, Date, X-Api-Version");
  next();
});

app.use(express.static("./static"));
app.use("/api/issues", issues);
app.use("/api/users", users);

app.listen(3000, function() {
  console.log("API server running at port 3000");
});
```

အဓိကလုပ်ဆောင်ချက်များ မပါဝင်ပဲ `require()` နဲ့ `app.use()` တို့ကိုသုံးပြီး အသုံးပြုဖို့လိုအပ်တဲ့ Package များကို ချိတ်ဆက်ကြေငြာထားခြင်းသာ ဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၄၁) မှာ Static File Server အဖြစ် အလုပ်လုပ်ဖို့ သတ်မှတ်ထားတဲ့အတွက် Static ဖိုင်အဖြစ် Response ပြန်ပေးစေလိုတဲ့ဖိုင်တွေကို `./static` Directory မှာထည့် သွင်းထားနိုင်တာကို သတိပြုပါ။ လိုင်းနံပါတ် (၄၂) နဲ့ (၄၃) မှာတော့ ကျွန်တော်တို့ ကြိုတင် ရေးသားထားတဲ့ Issue Router နဲ့ User Router တို့ကို Sub-App အဖြစ် တွဲဖက်ပေးထားပါတယ်။ `/api/issues` နဲ့ `/api/users` တို့ ကို API URL အဖြစ် အသီးသီး အသုံးပြုရမှာဖြစ်ပါတယ်။

ထူးခြားချက်အနေနဲ့ လိုင်းနံပါတ် (၃၈) မှာ `app.use()` ကိုသုံးပြီး Default Response Header တွေ တန်းစီ သတ်မှတ်ထားပါတယ်။ ExpressJS Server က Response ပြန်ပေးတိုင်းမှာ ဒီ Header တွေကို အလိုအလျောက် တွဲဖက်ထည့်သွင်းပေးသွားမှာဖြစ်ပါတယ်။

ဥပမာ – Web Client က example.com မှာ အလုပ်လုပ်နေပြီး API Service က api.example.com မှာ Host လုပ် ထားတယ်ဆိုပါစို့။ Browser တွေမှာ လုံခြုံရေးအရ သတ်မှတ်ထားတဲ့ Cross-Origin Policy ဆိုတာရှိပါတယ်။ အဲ့ ဒီ ပေါ်လစီအရ Web Browser က example.com မှာ အလုပ်လုပ်နေတဲ့ Web App ကနေ api.example.com ကို Ajax နဲ့ Request ပြုလုပ်ခြင်းကို ခွင့်ပြုမှာမဟုတ်ပါဘူး။ ဒီပြဿနာကို ဖြေရှင်းဖို့အတွက် Cross-Origin Resource Sharing (CORS) လို့ခေါ်တဲ့ နည်းစနစ်ကို အသုံးပြုရပါတယ်။ CORS နည်းစနစ်အရ Server က Response Header ထဲမှာ Access-Control-Allow-Origin ဆိုတဲ့ Header ကို ထည့်သွင်းပေးပို့ခြင်းအားဖြင့် Cross-Origin Request တွေကို ခွင့်ပြုကြောင်း ပြောပေးရပါတယ်။

လိုင်းနံပါတ် (၃၁) မှာ Access-Control-Allow-Origin အတွက်တန်ဖိုးကို http://localhost:8080 လို့ပြောထားတဲ့ အတွက် ဒီ Server ဟာ localhost:8080 ကပြုလုပ်တဲ့ Cross-Origin Request တွေကိုလက်ခံမယ် လို့ အသိပေး လိုက်ခြင်းဖြစ်ပါတယ်။ အကယ်၍ Web Client ရဲ့ လိပ်စာက example.com ဆိုရင် ဒီနေရာမှာ http://example.com လို့ ထည့်သွင်းသတ်မှတ်ပေးရမှာဖြစ်ပါတယ်။ အကယ်၍ မည်သည့် Client ကမဆိုပြုလုပ် တဲ့ Cross-Origin Request တွေကိုလက်ခံလိုရင်တော့ * သင်္ကေတကို အသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

JavaScript

```
res.set("Access-Control-Allow-Origin", "*");
```

လိုင်းနံပါတ် (၃၂) မှာ Access-Control-Allow-Credentials အတွက်တန်ဖိုးကို `true` လို့ သတ်မှတ်ထား ရတဲ့ ရည်ရွယ်ချက်ကတော့၊ Cross-Origin Request တွေနဲ့အတူ Cookie တွေထည့်သွင်းပေးရင်လည်း လက်ခံအလုပ် လုပ်စေလိုတဲ့အတွက် ထည့်သွင်းထားရခြင်းဖြစ်ပါတယ်။ အကယ်၍ ဒီ Header တန်ဖိုးထည့်သွင်းမထားရင် Cross-Origin Request တွေကို လက်ခံပေမယ့် Cookie တွေထည့်သွင်းပေးပို့ခြင်းကိုတော့ ခွင့်ပြုမှာမဟုတ်ပါ ဘူး။

တစ်ခုသတိပြုရမှာက၊ Access-Control-Allow-Credentials တန်ဖိုးကို `true` သတ်မှတ်ထားရင် Access-Control-Allow-Origin တန်ဖိုးကို ခွင့်ပြုလိုတဲ့ Origin Domain Name အတိအကျနဲ့ သတ်မှတ်ပေးရပါတယ်။ *

ကို အသုံးပြုလို့ မရတော့ပါဘူး။ ဒါကြောင့်လည်း ကျွန်တော်တို့ Code ထဲမှာ Access-Control-Allow-Origin တန်ဖိုးကို * လို့ပေးမထားပဲ http://localhost:8080 လို့ အတိအကျသတ်မှတ်ပေးထားခြင်းဖြစ်ပါ တယ်။ ဘာကြောင့်မရတာလဲဆိုတဲ့ နောက်ကွယ်ကအသေးစိတ်ကိုတော့ ဒီနေရာမှာ ထည့်သွင်းမဖော်ပြနိုင်တော့ပါဘူး။ Access-Control-Allow-Credentials တန်ဖိုး `true` သတ်မှတ်လိုရင် Access-Control-Allow-Origin တန်ဖိုးကို * သုံးလို့မရဘူးလို့သာ အလွယ်မှတ်ပေးပါ။

လိုင်းနံပါတ် (၃၃) မှာတော့ Access-Control-Allow-Methods Header နဲ့ Cross-Origin Request တွေနဲ့ အတူ ခွင့်ပြုလိုတဲ့ Request Method စာရင်းကို သတ်မှတ်ပေးထားပါတယ်။ ဒီနေရာမှာလည်း Request Method အားလုံးကိုခွင့်ပြုလိုရင် * သင်္ကေတကို အသုံးပြုနိုင်ပါတယ်။

လိုင်းနံပါတ် (၃၅) မှာတော့ Access-Control-Allow-Headers နဲ့ Cross-Origin Request တွေနဲ့အတူ ထည့်သွင်းပေးပို့ခြင်းကို ခွင့်ပြုမယ့် Header စာရင်းကို သတ်မှတ်ထားပါတယ်။ API Server တွေကို ပေးပို့ဖို့ လိုအပ်တက်လေ့ရှိတဲ့ Header စာရင်းကို ပေးထားခြင်းဖြစ်ပါတယ်။ ဒီနေရာမှာလည်း Header အားလုံးကို ခွင့်ပြုလိုရင် * သင်္ကေတကို အသုံးပြုနိုင်ပါတယ်။

CORS နဲ့ပက်ပြီး အသေးစိတ်ဆက်လက်လေ့လာလိုရင် အောက်ပါလိပ်စာမှာ လေ့လာနိုင်ပါတယ်။

https://developer.mozilla.org/en-US/docs/Web/HTTP/Access_control_CORS

အခုဆိုရင် ကျွန်တော်တို့ရဲ့ Issue Tracking System API Service အတွက် လိုအပ်ချက်တွေအားလုံးပြည့်စုံပြီမို့ စတင် စမ်းသပ်နိုင်ပြီဖြစ်ပါတယ်။

13.7 – Testing the API

ရေးသားထားတဲ့ Service ကို စမ်းသပ်နိုင်ဖို့အတွက် ပထမဆုံး User Account တစ်ခုကို mydb Database ရဲ့ users Collection ထဲမှာ ကြိုတင်တည်ဆောက်ထား ပေးရမှာဖြစ်ပါတယ်။ User Account တည်ဆောက်ရာမှာ အသုံးပြုဖို့ လိုအပ်တဲ့အတွက် Password Hash တစ်ခုကို Node နဲ့ အရင် Generate လုပ်ယူပါမယ်။

```
$ node
> var crypto = require("crypto");
> var password =
crypto.createHash("sha1").update("123456").digest("hex");
> console.log(password);

7c4a8d09ca3762af61e59520943dc26494f8941b
```

123456 ဆိုတဲ့တန်ဖိုးအတွက် SHA1 Hash တစ်ခုကို Generate လုပ်ယူလိုက်ခြင်းဖြစ်ပါတယ်။ ပြီးတဲ့အခါ Mongo Shell ကနေတစ်ဆင့် Manager Account တစ်ခုထည့်သွင်းပါမယ်။

```
$ mongo
> use mydb;

switched to db mydb

> db.users.insert({
... fullName: "John Doe",
... loginId: "manager",
... email: "john@example.com",
... role: 3,
... password: "7c4a8d09ca3762af61e59520943dc26494f8941b"
... });
> db.users.find().pretty();

{
     "_id" : ObjectId("55b703a9d2141ae9733ac0eb"),
     "fullName" : "John Doe",
     "loginId" : "manager",
     "email" : "john@example.com",
     "role" : 3,
     "password" : "7c4a8d09ca3762af61e59520943dc26494f8941b"
}
```

`loginId` ကို `manager` လို့ပေးထားပြီး `password` ကို `123456` လို့ပေးထားတဲ့ User Account တစ်ခုရရှိသွား ပြီဖြစ်ပါတယ်။ Issue Tracking System API ကို စတင်စမ်းသပ်နိုင်ဖို့အတွက် Server ကို Run ပေးရပါမယ်။

```
$ node app.js

API server running at port 3000
```

အခုဆိုရင် ကျွန်တော်တို့ရေးသားထားတဲ့ Service က Port 3000 မှာ အလုပ်လုပ်နေပြီဖြစ်လို့ CURL နဲ့ ဆက်သွယ်စမ်း သပ်နိုင်ပြီးဖြစ်ပါတယ်။ ပထမဆုံးအနေနဲ့ Verify လုပ်ကြည့်ပါမယ်။

```
$ curl -X GET localhost:3000/api/users/verify

Unauthorized%
```

`/users/verify` ကို Request လုပ်ကြည့်တဲ့အခါ 401 Unauthorized ကို လက်ခံရရှိခြင်းဖြစ်ပါတယ်။ ဆက်လက် ပြီး Login ဝင်ကြည့်ပါမယ်။

```
$ curl -X POST localhost:3000/api/users/login -d
"username=manager&password=123456"

{"_id":"55b70503d2141ae9733ac0ec","fullName":"John
Doe","loginId":"manager","email":"john@example.com","role":3,"password":"7c4a8
d09ca3762af61e59520943dc26494f8941b"}%

$ curl -X GET localhost:3000/api/users/verify

Unauthorized%
```

`/users/login` ကို POST Request ပြုလုပ်ပြီး Username, Password အမှန်ပေးလိုက်နိုင်တဲ့အတွက် User Data တွေ ကို API Server က Response ပြန်ပေးခြင်းဖြစ်ပါတယ်။ ဒါပေမယ့် `/users/verify` ကို ပြန်စစ်ကြည့်တဲ့အခါ Unauthorized ပဲဆက်ပြနေတာကို တွေ့ရမှာဖြစ်ပါတယ်။ ပြဿနာက Login Success ဖြစ်ပြီး User Data တွေကို Session ထဲမှာ သိမ်းသွားပေမယ့် CURL က အဲ့ဒီ Session ရဲ့ ID ကို မသိဘူးဖြစ်နေတာပါ။ Session ID က Cookie တစ်ခုနဲ့ တွဲသိမ်းပေးဖို့လိုပါတယ်။ ဒါကြောင့် အခုလို Login ဝင်မှ မှန်မှာပါ။

```
$ curl -X POST localhost:3000/api/users/login \
                -d "username=manager&password=123456" -c cookie.txt

{"_id":"55b70503d2141ae9733ac0ec","fullName":"John
Doe","loginId":"manager","email":"john@example.com","role":3,"password":"7c4a8
d09ca3762af61e59520943dc26494f8941b"}%
```

နောက်ဆုံးက `-c` Option နဲ့ Session ID ကို `cookie.txt` ဖိုင်ထဲမှာ ရေးသားစေလိုက်ခြင်းဖြစ်ပါတယ်။ နောင် Request ပြုလုပ်တဲ့အခါ Username, Password ကို ထပ်ပေးဖို့မလိုတော့ပေမယ့် အဲ့ဒီ Cookie ဖိုင်ကိုတော့ တွဲထည့်ပေး ဖို့လိုပါတယ်။ `-b` Option ကို သုံးရပါတယ်။

```
$ curl -X GET localhost:3000/api/users/verify -b cookie.txt

{"_id":"55b70503d2141ae9733ac0ec","fullName":"John
Doe","loginId":"manager","email":"john@example.com","role":3,"password":"7c4a8
d09ca3762af61e59520943dc26494f8941b"}%
```

နောက်တစ်ကြိမ် `/users/verify` ကို Request လုပ်တဲ့အခါ `-b` Option နဲ့ Cookie ဖိုင်ကိုတွဲဖက်ပေးလိုက်တဲ့အခါ Authorized ဖြစ်ပြီးဖြစ်တဲ့အတွက် လက်ရှိ Authorized User ရဲ့ အချက်အလက်တွေကို Response အနေနဲ့ လက်ခံရရှိ တာကို တွေ့ရမှာဖြစ်ပါတယ်။

ဆက်လက်ပြီး လက်ရှိရှိနေတဲ့ User စာရင်းကို ရယူကြည့်နိုင်ပါတယ်။

```
$ curl -X GET localhost:3000/api/users -b cookie.txt

[{"_id":"55b70503d2141ae9733ac0ec","fullName":"John
Doe","loginId":"manager","email":"john@example.com","role":3,"password":"7c4a8
d09ca3762af61e59520943dc26494f8941b"}]%
```

လိုအပ်တဲ့အချက်အလက်တွေ မှန်ကန်အောင်မပေးပဲ User အသစ်တစ်ယောက်ကို တည်ဆောက်ဖို့ကြိုးစားရင်တော့ အခုလို ပြန်လည်ရရှိမှာဖြစ်ပါတယ်။

```
$ curl -X POST localhost:3000/api/users -b cookie.txt

[{"param":"fullName","msg":"Invalid Full Name"},
{"param":"loginId","msg":"Invalid Login ID"},{"param":"email","msg":"Invalid
Email Address"},{"param":"role","msg":"Invalid Role"},
{"param":"password","msg":"Password should have at least 6 characters"}]%
```

`express-validate` နဲ့ `fullName`, `loginId`, `email` စတဲ့အချက်အလက်တွေပါဝင်ဖို့လိုကြောင်း စစ်ဆေးထားတဲ့အတွက် ဖြစ်ပါတယ်။ ဒါကြောင့် လိုအပ်တဲ့အချက်အလက်အပြည့်အစုံနဲ့ အခုလို Request လုပ်ပေးဖို့ လိုပါတယ်။

```
$ curl -X POST localhost:3000/api/users \
        -d "fullName=Bob&loginId=bob&email=bob@gmail.com&role=2&password=123456" \
        -b cookie.txt

{"fullName":"Bob","loginId":"bob","email":"bob@gmail.com","role":"2","password
":"7c4a8d09ca3762af61e59520943dc26494f8941b","roleLabel":"Developer","_id":"55
b70b342468830b671da93b"}%
```

ဒီတစ်ကြိမ်မှာတော့ လိုအပ်တဲ့အချက်အလက်တွေ အပြည့်အစုံပါဝင်တဲ့အတွက် ထည့်သွင်းလိုက်တဲ့ User ရဲ့ အချက် အလက်တွေကို Response အနေနဲ့ ပြည်လည်လက်ခံ ရရှိမှာဖြစ်ပါတယ်။

အသစ်တည်ဆောက်လိုက်တဲ့ User ဖြစ်တဲ့ Bob ရဲ့ Role ကို 2 လို့သတ်မှတ်ထားပါတယ်။ ဒါကြောင့် သူဟာ Developer Role ဖြစ်ပြီး User တွေကိုပယ်ဖျက်လို့မရပါဘူး။ Bob နဲ့ Login ဝင်ပြီး Manager ကို Delete လုပ်ကြည့်ပါမယ်။

```
$ curl -X POST localhost:3000/api/users/login \
                -d "username=bob&password=123456" -c cookie.txt

{"fullName":"Bob","loginId":"bob","email":"bob@gmail.com","role":"2","password
":"7c4a8d09ca3762af61e59520943dc26494f8941b","roleLabel":"Developer","_id":"55
b70b342468830b671da93b"}%
```

```
$ curl -X GET localhost:3000/api/users -b cookie.txt

[{"_id":"55b70503d2141ae9733ac0ec","fullName":"John
Doe","loginId":"manager","email":"john@example.com","role":3,"password":"7c4a8
d09ca3762af61e59520943dc26494f8941b"},
{"fullName":"Bob","loginId":"bob","email":"bob@gmail.com","role":"2","password
":"7c4a8d09ca3762af61e59520943dc26494f8941b","roleLabel":"Developer","_id":"55
b70b342468830b671da93b"}]%

$ curl -X DELETE localhost:3000/api/users/55b70503d2141ae9733ac0ec -b
cookie.txt

Unauthorized%
```

Bob နဲ့ Login ဝင်ပြီး Manager Account ကိုပယ်ဖျက်ဖို့ DELETE Request ပြုလုပ်တဲ့အခါ ပယ်ဖျက်ခွင့် မရှိတဲ့အတွက် Unauthorized ကိုပဲ လက်ခံရရှိမှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ကျန် Rout တွေကို ဆက်လက်စမ်းသပ်ကြည့်နိုင်ပြီဖြစ်ပါတယ်။

Code နမူနာ အပြည့်အစုံကို အောက်ပါလိပ်စာမှာ ရယူနိုင်ပါတယ်။

https://github.com/eimg/issues-api/archive/master.zip

Conclusion

ဖော်ပြခဲ့တဲ့နမူနာဟာ REST API နည်းစနစ်ကို လိုက်နာပြီး HTTP Request Method တွေနဲ့ HTTP Status Code တွေ ကို သူ့နေရာနဲ့သူ အသုံးချထားတဲ့ Back-end Service တစ်ခုဖြစ်ပါတယ်။ နားလည်မှတ်သားရလွယ်တဲ့ URL Routing ကိုလည်း အသုံးပြုထားပါတယ်။ Session Based Authentication စနစ်ကိုလည်း အသုံးပြုထားပါတယ်။ ဒါပေမယ့် စွမ်း ဆောင်ရည်ကောင်းမွန်စေဖို့အတွက် Compression နဲ့ Cache လို လုပ်ဆောင်ချက်တွေဖြည့်စွက်ပေးသင့်ပါတယ်။ Response တွေကို Plain Text အနေနဲ့ပြန်ပေးတာမျိုးမဟုတ်ပဲ Compress လုပ်ပြီးမှ Response ပြန်ပေးသင့်ပါတယ်။ တစ်ချို့ Cache လုပ်သင့်တဲ့ Content တွေကို Cache လုပ်စေဖို့နဲ့၊ Cache မလုပ်သင့်တဲ့ Content တွေကို Cache မလုပ်စေဖို့အတွက် Cache-Control Header တွေ Response နဲ့အတူ မှန်ကန်အောင်သတ်မှတ်ပေးသင့်ပါ သေးတယ်။

ဒီ Back-end Service ကို ကိုယ်တိုင် အသုံးပြုယုံသာမက၊ အများကိုလည်း အသုံးပြုခွင့်ပေးတော့မယ်ဆိုရင် ဖြည့်စွက်စဉ်း စားစရာတွေ ထပ်ရှိလာနိုင်ပါသေးတယ်။ ဥပမာ – API Version ကိစ္စ စဉ်းစားရပါမယ်။ လက်ရှိ Service ကပေးထားတဲ့ လုပ်ဆောင်ချက်တွေကို အသုံးချထားတဲ့ Client တွေရှိနေတယ်ဆိုပါစို့။ အဲ့ဒီအချိန်မှာ ကိုယ့် Service ရဲ့ လုပ်ဆောင်ချက် တွေကို အဆင့်မြှင့်ဖို့ လိုအပ်လာတဲ့အခါ ပြုလုပ်လိုက်တဲ့ အပြောင်းအလဲကြောင့် လက်ရှိအသုံးပြုနေဆဲ Client တွေအတွက် အဆင်မပြေ မဖြစ်ရအောင် စီစဉ်ထားရမှာဖြစ်ပါတယ်။

ပေးထားတဲ့ Service ကို လိုတာထက်ပိုအသုံးပြုထားတဲ့ Client တွေလည်းရှိနိုင်ပါတယ်။ ဥပမာ – မလိုအပ်ပဲနဲ့ တစ်စက္ကန့် တစ်ခါ Request လုပ်နေတဲ့အတွက် Server Resource တွေ ဖြုန်းတီးသလိုဖြစ်နေတဲ့ Client တွေ ရှိနိုင်ပါ တယ်။ အဲ့ဒီလို Client တွေကို ဘယ်လိုကန့်သတ်မလဲဆိုတာ စဉ်းစားဖို့လိုအပ်လာနိုင်ပါတယ်။ ကန့်သတ်ထားတဲ့ Request အရေအတွက် ပြည့်တဲ့အခါ အဲ့ဒီ Client ပြုလုပ်တဲ့ Request တွေကို ထပ်လက်မခံတော့တဲ့ လုပ်ဆောင်ချက်မျိုးတွေ ထည့်သွင်းရေးသားဖို့ လိုနိုင်ပါတယ်။

ပြီးတဲ့အခါ Client ရေးသားသူ Developer တွေအတွက် အဆင်ပြေစေဖို့အတွက် API Documentation တွေ ရေးသားပေးဖို့လည်း လိုအပ်နိုင်ပါသေးတယ်။ လက်တွေ့ Production အဆင့် သွားနိုင်ဖို့အတွက် ဖြည့်စွက်စဉ်းစားရမှာတွေ ရှိနေပေမယ့်၊ ဖော်ပြထားတဲ့နမူနာပေါ်မှာအခြေခံပြီး ကိုယ်တိုင်ဆက်လက်ဖြည့်စွက်သွားဖို့ လမ်းစတစ်ခုကို ရရှိသွားနိုင်လိမ့်မယ်လို့ မျှော်လင့်ပါတယ်။

ပြုပြင်ထိမ်းသိမ်းရလွယ်တဲ့ Maintainable Code
ဖြစ်ဖို့အတွက် သီးခြားစီပြင်ဆင်နိုင်တဲ့ အစိတ်အပိုင်းငယ်များ
အဖြစ် ခွဲခြားရေးသားထားရမှာဖြစ်ပါတယ်။

Professional Web Developer Course

HTML5, PHP/MySQL, jQuery/Ajax, Mobile Web စသည့်
Professional Web Developer တစ်ဦး သိရှိထားသင့်သည့်
နည်းပညာများကို စုစည်းသင်ကြားခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၁၄) – Building Clients (HTML Templates)

Issue Tracking System အတွက် Back-end Service ရရှိပြီးနောက်မှာ အဲ့ဒီ Back-end Service နဲ့ဆက်သွယ်အလုပ် လုပ်နိုင်တဲ့ Client App တည်ဆောက်ပုံကို ဆက်လက်ဖော်ပြပါမယ်။ BackboneJS ကို သုံးပြီး Web Client တစ်ခု တည်ဆောက်မှာဖြစ်ပါတယ်။ App တည်ဆောက်ပုံအကြောင်း မပြောခင် BackboneJS နဲ့အတူ တွဲဖက်အသုံးပြုမယ့် Bootstrap နဲ့ DataTable တို့အကြောင်းကို အရင်ဖော်ပြချင်ပါတယ်။

14.1 – Bootstrap 3

Bootstrap ဟာ လူသုံးများတဲ့ CSS Framework တစ်ခုဖြစ်ပါတယ်။ Device အမျိုးမျိုး၊ Screen Size အမျိုးမျိုးနဲ့သင့်တော်တဲ့ Layout ကို အလိုက်သင့်ပြောင်းလဲ အလုပ်လုပ်ပေးနိုင်တဲ့ Responsive Layout လုပ်ဆောင်ချက်ပါဝင်ပြီး အသင့်အသုံးပြုနိုင်တဲ့ CSS Style နဲ့ JavaScript Component တွေကို စုစည်းပေးထားတဲ့ Framework တစ်ခုဖြစ်ပါ တယ်။ ဒီစာရေးသားနေစဉ် နောက်ဆုံးထွက်ရှိထားတဲ့ Version ကတော့ Version 3.2.0 ပဲဖြစ်ပါတယ်။

Bootstrap ကို bower install နဲ့ရယူနိုင်သလို getbootstrap.com မှာကိုယ်တိုင် Download ရယူလို့လည်းရပါတယ်။ ဒီနေရာမှာ Bootstrap မှာပါဝင်တဲ့ Style နဲ့ Components အားလုံးကို ဖော်ပြမှာမဟုတ်ပါဘူး။ နမူနာ Client App မှာ အသုံးပြုမယ့်အပိုင်းတွေကို အဓိကထား ဖော်ပြသွားမှာဖြစ်ပါတယ်။ Bootstrap ရဲ့ Style နဲ့ Component အားလုံးကိုလေ့လာလိုရင် getbootstrap.com မှာပဲ တစ်ပါတည်းလေ့လာနိုင်ပါတယ်။ သူပေးထားတဲ့ Document က ရိုးရှင်းနားလည်ရတဲ့အတွက်၊ HTML/CSS နဲ့ JavaScript/jQuery အခြေခံရှိတဲ့ မည်သူမဆို အခက်အခဲမရှိ လေ့လာနိုင် မှာပါ။

Layout

Bootstrap နဲ့ပက်သက်ပြီး ပထမဆုံးလေ့လာသင့်တာကတော့ Layout ဖြစ်ပါတယ်။ Bootstrap က Column (၁၂) ခုပါ ဝင်တဲ့ Layout ကိုအသုံးပြုပြီး အခြေခံ Layout Structure က အခုလို ဖြစ်မှာပါ။

HTML

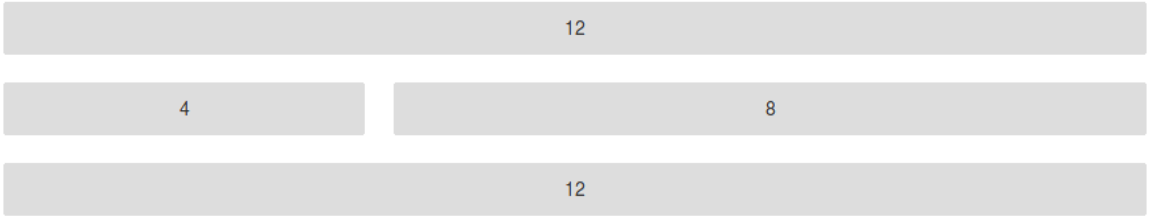
```
<div class="container">
  <div class="row">
    <div class="col-md-12"></div>
  </div>
  <div class="row">
    <div class="col-md-4"></div>
    <div class="col-md-8"></div>
  </div>
</div>
```

Layout မှာပါဝင်တဲ့ Row တွေကို `.row` Class သတ်မှတ်ပေးဖို့လိုပြီး `.row` အားလုံးဟာ `.container` (သို့) `.container-fluid` အတွင်းမှာ ရှိရပါတယ်။ `.container` ဟာ Size ပုံသေသတ်မှတ်ထားတဲ့ Fixed-width Element ဖြစ်ပြီး `.container-fluid` ဆိုရင်တော့ Screen Size ရှိသလောက်နေရာယူမယ့် Fluid-width Element ဖြစ်မှာဖြစ်ပါတယ်။

Bootstrap က Screen Size (၄) မျိုးအတွက် ကြိုတင်သတ်မှတ်ပေးထားပါတယ်။ Extra-Small Screen, Small Screen, Medium Screen နဲ့ Large Screen တို့ဖြစ်ပါတယ်။ အခြေခံအားဖြင့် Extra-Small Screen ဆိုတာ Smart Phone Screen Size ကို ဆိုလိုပြီး Small Screen ကတော့ Tablet Screen Size ကို ဆိုလိုပါတယ်။ Medium Screen ကတော့ Desktop/Laptop Screen Size ဖြစ်ပြီး Large Screen ကတော့ Big Screen TV ကို ဆိုလိုပါတယ်။

`.container` ရဲ့ Fixed-width Size ဟာ Desktop/Laptop မှာ Width 970px ရှိမှာဖြစ်ပါတယ်။ Screen Size ပြောင်းသွားရင် Container လည်းလိုက်ပြောင်းသွားပြီး Smart Phone Screen မှာဆိုရင် Width Auto ဖြစ်သွားမှာဖြစ် ပါတယ်။ `.container-fluid` ကတော့ ဘယ် Screen Size မှာမဆို Width Auto နဲ့ အလုပ်လုပ်ပေးမှာဖြစ်ပါတယ်။ အလုပ်လုပ်ပုံကိုသာ ဖော်ပြခြင်းဖြစ်ပြီး လက်တွေ့မှာ Fixed-Width Layout လိုချင်ရင် `.container` သုံးရပြီး၊ Fluid-Width Layout လိုချင်ရင် `.container-fluid` သုံးလိုက်ယုံပါပဲ။ Screen Size အမျိုးမျိုးမှာ အဆင်ပြေပြေ ပြောင်းလဲ အလုပ်လုပ်အောင် Bootstrap က ကြိုတင်သတ်မှတ်ထားပေးပြီးဖြစ်ပါတယ်။

`.row` အတွင်မှာတော့ Column တွေကို လိုချင်တဲ့ Layout ရရှိအောင်သတ်မှတ်နိုင်ပါတယ်။ အထက်ကနမူနာအရ ပထမ Row ထဲမှာ Size အပြည့် (၁၂) ကွက်စာနေရာယူထားတဲ့ Column တစ်ခုရှိနေပြီး၊ ဒုတိယ Row ထဲမှာတော့ (၄) ကွက် စာနေရယူထားတဲ့ Column တစ်ခုနဲ့ (၈) ကွက်စာနေရာယူထားတဲ့ Column တစ်ခု နှစ်ခုရှိနေမှာဖြစ်ပါတယ်။ Column တွေ Layout ပုံစံဖော်ပြစေဖို့ သီးခြားသတ်မှတ်ပေးစရာမလိုပဲ သတ်မှတ်လိုတဲ့ Column Size ကိုသာ သတ်မှတ်ပေးယုံပဲ ဖြစ်ပါတယ်။ ရရှိမယ်ရလဒ် Layout က အခုလိုဖြစ်မှာပါ။



ပုံ (၁၄.၁) - Bootstrap Layout

အထက်ကနမူနာမှာ Column တွေကိုသတ်မှတ်ဖို့အတွက် `.col-md-[n]` ဆိုတဲ့ Format ကို အသုံးပြုခဲ့ပါတယ်။ `md` ဆိုတာ Medium Device ရဲ့ အတိုကောက်ပါ။ Small Screen Device တွေအတွက် `sm` ကို အသုံးပြုရပြီး Extra Small Screen Device တွေအတွက် `xs` ကို အသုံးပြုရပါတယ်။ Large Screen Device တွေအတွက်ဆိုရင်တော့ `lg` ကို အသုံးပြုရပါတယ်။ အထက်ကနမူနာကို အခုလိုပြင်ဆင်ရေးသားနိုင်ပါတယ်။

HTML

```
<div class="container">
  <div class="row">
    <div class="col-md-12"></div>
  </div>
  <div class="row">
    <div class="col-md-8 col-sm-6 col-xs-12"></div>
    <div class="col-md-4 col-sm-6 col-xs-12"></div>
  </div>
</div>
```

ဒုတိယ Row ထဲက Column မှာသတ်မှတ်ထားတဲ့ `col-md-8` ရဲ့အဓိပ္ပါယ်က၊ Medium Device ဆိုရင် (၈) ကွက်စာ နေရာယူပါဆိုတဲ့ အဓိပ္ပါယ်ဖြစ်ပါတယ်။ `col-sm-6` ရဲ့ အဓိပ္ပါယ်ကတော့ Small Screen Device ဆိုရင် (၆) ကွက်စာ နေရာယူပါဆိုတဲ့ အဓိပ္ပါယ်ပါ။ `col-xs-12` ရဲ့ အဓိပ္ပါယ်ကတော့ Extra Small Screen ဆိုရင် (၁၂) ကွက်စာ နေရာယူပါလို့ သတ်မှတ်လိုက်ခြင်းဖြစ်ပါတယ်။ ဒါကြောင့် ဒီ Column ဟာ Screen Size ပေါ်မူတည်ပြီး သတ်မှတ်ထားတဲ့အတိုင်း အလိုအလျောက် လိုက်လံပြောင်းလဲပေးနိုင်တဲ့ Responsive Column တစ်ခုဖြစ်သွားမှာပဲဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Responsive Layout တွေကို Bootstrap နဲ့အလွယ်တစ်ကူ သတ်မှတ်ထားနိုင်ပါတယ်။

Styles

Bootstrap က Basic Element တွေဖြစ်ကြတဲ့ h1-h6, p, a စတဲ့ Element တွေအတွက် သင့်တော်တဲ့ Default Style တွေ ကြိုတင်သတ်မှတ်ထားပေးပြီးဖြစ်ပါတယ်။ အဲ့ဒီ Element တွေကို Bootstrap Style နဲ့ ဖော်ပြစေဖို့ အတွက် Class တွေ သတ်မှတ်ပေးနေစရာမလိုပါဘူး။ Table လို Element မျိုးကို Bootstrap Style နဲ့ ဖော်ပြစေလိုရင်တော့ .table Class သတ်မှတ်ပေးဖို့လိုပါတယ်။ ဥပမာ –

HTML

```
<table class="table">
  <tr>
    <th> … </th>
    <th> … </th>
  </tr>
  <tr>
    <td> … </td>
    <td> … </td>
  </tr>
</table>
```

ရရှိမယ့်နမူနာရလဒ်ကတော့ အခုလိုဖြစ်ပါတယ်။

#	First Name	Last Name	Username
1	Mark	Otto	@mdo
2	Jacob	Thornton	@fat
3	Larry	the Bird	@twitter

ပုံ (၁၄.၂) - Bootstrap Style Table

Table ဘေးပါတ်လည်မှာ Border ပါဝင်သွားစေလိုရင် `.table-bordered` ကိုအသုံးပြုနိုင်ပြီး Row တွေကို Zebra Stripping နဲ့ဖော်ပြစေလိုရင် `.table-striped` ကို အသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

HTML

```
<table class="table table-bordered table-striped">
  <tr>
    <th> … </th>
    <th> … </th>
  </tr>
  <tr>
    <td> … </td>
    <td> … </td>
  </tr>
</table>
```

Input, Select, Textarea စတဲ့ Form Element တွေကို Bootstrap Style နဲ့ ဖော်ပြစေလိုရင်တော့ `.form-control` ကို သတ်မှတ်ပေးရပါတယ်။ အကယ်၍ Input Element တစ်ခု ကို Label Element နဲ့တွဲဖက်ဖော်ပြစေလို ရင်တော့ `.form-group` Element တစ်ခုနဲ့ စုစည်းရေးသားပါရပါတယ်။ ဥပမာ –

HTML

```
<div class="form-group">
  <label for="email">Email address</label>
  <input type="email" class="form-control" id="email">
</div>
```

Button တစ်ခု (သို့မဟုတ်) Link တစ်ခုကို Bootstrap Button ပုံစံဖော်ပြစေလိုရင် `.btn` Class ကို အသုံးပြုရပါတယ်။ `.btn` နဲ့အတူ `.btn-default`, `.btn-primary`, `.btn-success`, `.btn-info`, `.btn-warning`, `.btn-danger` စတဲ့ Class တွေကိုလည်း တွဲဖက်အသုံးပြုနိုင်ပါတယ်။ အကယ်၍ Button ကို Link ပုံစံ ဖော်ပြစေလိုတယ်ဆိုရင်လည်း `.btn-link` ကို သုံးနိုင်ပါသေးတယ်။



ပုံ (၁၄.၂) - Bootstrap Style Buttons

တစ်ခုထက်ပိုတဲ့ Button တွေကို အတွဲလိုက်စုစည်းထားချင်ရင် `.btn-group` Class သတ်မှတ်ထားတဲ့ Element တစ်ခုတည်းမှာ ထည့်သွင်းစုစည်းထားနိုင်ပါတယ်။

စာလုံးအရောင်အတွက်လည်း သက်ဆိုင်ရာ Element မှာ `.text-primary`, `.text-success`, `.text-info`, `.text-warning`, `.text-danger` စသဖြင့် Bootstrap Text Color တွေကို သတ်မှတ်ပေးနိုင်ပါ တယ်။ အရောင်မှိန်ပြီးဖော်ပြစေလိုတဲ့ Text တွေအတွက်လည်း `.text-muted` ကို အသုံးပြုနိုင်ပါတယ်။

အလားတူပဲ Element တစ်ခုရဲ့ Background Color ကို Bootstrap Color နဲ့ သတ်မှတ်လိုရင်လည်း `.bg-primary`, `.bg-success`, `.bg-info` စသဖြင့် သတ်မှတ်ပေးနိုင်ပါသေးတယ်။ ဖြည့်စွက်မှတ်သားသင့်တဲ့ Style ကတော့ `.pull-left` နဲ့ `.pull-right` တို့ပဲဖြစ်ပါတယ်။ Element တစ်ခုကို ညာဘက်ကပ်စေလိုရင် (`float: right` ပြုလုပ်စေလိုရင်) `.pull-right` ကို သုံးနိုင်ပြီး၊ ဘယ်ဘက်ကပ်စေလိုရင်တော့ `.pull-left` ကို သုံးနိုင်ပါတယ်။ Drop-down Arrow လေးတစ်ခု (▼) ဖော်ပြစေလိုရင်တော့ အောက်ပါအတိုင်းသတ်မှတ်နိုင်ပါတယ်။

HTML

```
<span class="caret"></span>
```

နောက်ထပ်ထူးခြားချက်အနေနဲ့၊ Element တွေကို တစ်ချို့ Screen Size တွေမှာဖော်ပြစေလိုပြီး၊ တစ်ချို့ Screen Size တွေမှာ မဖော်ပြစေလိုရင် သတ်မှတ်နိုင်တဲ့ Class တွေလည်းရှိပါသေးတယ်။ ဥပမာ – Element တစ်ခုမှာ `.hidden-xs` လို့ သတ်မှတ်ထားရင် အဲ့ဒီ Element ကို Extra Small Device တွေမှာ မဖော်ပြပဲဖျောက်ထားပေးမှာ ဖြစ်ပါတယ်။ အလားတူ Class တွေဖြစ်တဲ့ `.hidden-sm`, `.hidden-md` နဲ့ `.hidden-lg` တို့ကိုလည်း လိုအပ်သလို အသုံး ပြုနိုင်ပါတယ်။

ပြောင်းပြန်အားဖြင့် Element တစ်ခုကို ကျန် Screen Size တွေမှာ မဖော်ပြစေလိုပဲ၊ Extra Small Device တွေမှာသာ ဖော်ပြစေလိုရင် `.visible-xs-*` ကို အသုံးပြုနိုင်ပါတယ်။ ဒီလိုသတ်မှတ်ပေးလိုက်ရင် Element ကို Extra Small Device မှာသာဖော်ပြပြီး ကျန် Screen Size တွေမှာ ဖျောက်ထားပေးမှာဖြစ်ပါတယ်။ အလားတူ Responsive Style Class တွေကို အောက်ပါဇယားမှာ လေ့လာနိုင်ပါတယ်။

	Extra small devices Phones (<768px)	Small devices Tablets (≥768px)	Medium devices Desktops (≥992px)	Large devices Desktops (≥1200px)
`.visible-xs-*`	Visible	Hidden	Hidden	Hidden
`.visible-sm-*`	Hidden	Visible	Hidden	Hidden
`.visible-md-*`	Hidden	Hidden	Visible	Hidden
`.visible-lg-*`	Hidden	Hidden	Hidden	Visible
`.hidden-xs`	Hidden	Visible	Visible	Visible
`.hidden-sm`	Visible	Hidden	Visible	Visible
`.hidden-md`	Visible	Visible	Hidden	Visible
`.hidden-lg`	Visible	Visible	Visible	Hidden

ပုံ (၁၄.၄) - Bootstrap Responsive Helper Classes

Components

Bootstrap Component တွေထဲမှာ အခြေခံအကျဆုံးကတော့ Icon ဖြစ်ပါတယ်။ Icon တွေအသုံးပြုဖို့အတွက် သီးခြား Image တွေထည့်သွင်းနေစရာမလိုပဲ Bootstrap နဲ့အတူပါဝင်လာတဲ့ Icon တွေကို အသုံးပြုနိုင်ပါတယ်။ ဥပမာ – Button တစ်ခုမှာ Add Icon ကိုထည့်သွင်းဖော်ပြစေလိုရင် အခုလို သတ်မှတ်နိုင်ပါတယ်။

HTML

```
<button class="btn btn-default">
  <span class="glyphicon glyphicon-plus-sign"></span>
  Add User
</button>
```

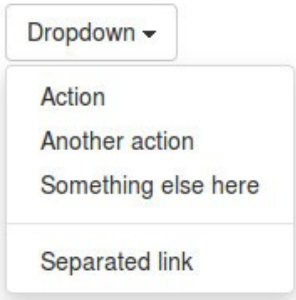


Icon ထည့်သွင်းဖို့အတွက် `<span>` Element ကို အသုံးပြုပြီး `.glyphicon` Class နဲ့အတူ တွဲဖက်အသုံးပြုလိုတဲ့ Icon Type ကို တွဲဖက်ပေးရပါတယ်။ နမူနာမှာ `.glyphicon-plus-sign` ကို အသုံးပြုထားတဲ့အတွက် ရလဒ်မှာ Plus သင်္ကေတ Icon ပါဝင်လာတာကိုတွေ့ရမှာဖြစ်ပါတယ်။ getbootstrap.com/components မှာ Icon စာရင်းအပြည့် အစုံကို ဆက်လက်လေ့လာနိုင်ပါတယ်။

Component တွေထဲမှာ အရေးပါတဲ့ လုပ်ဆောင်ချက်တစ်ခုကတော့ Drop-down Menu ဖြစ်ပါတယ်။ Bootstrap Style Drop-down Menu တစ်ခုရရှိဖို့ အခုလို ရေးသားနိုင်ပါတယ်။

HTML

```
<div class="dropdown">
  <button class="btn btn-default" data-toggle="dropdown">
    Dropdown <span class="caret"></span>
  </button>
  <ul class="dropdown-menu">
    <li> ... </li>
    <li> ... </li>
  </ul>
</div>
```



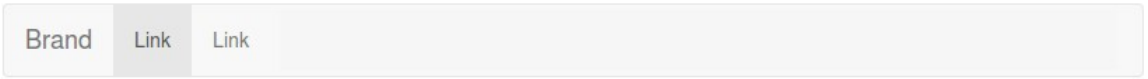
Drop-down Trigger လုပ်ပေးမယ့် Link (သို့မဟုတ်) Button တစ်ခုနဲ့ `.dorpdown-menu` Class အသုံးပြုထားတဲ့ `<ul>` Element တို့ကို Element တစ်ခုအတွင်းမှာ အတူတစ်ကွထည့်သွင်းပေးရပြီး Trigger Link (သို့မဟုတ်) Button မှာ `data-toggle` Attribute နဲ့ `dorpdown` လို့ပြောလိုက်ယုံနဲ့ Bootstrap Style Drop-down Menu တစ်ခုကို ရရှိနိုင်မှာပဲဖြစ်ပါတယ်။ `<li>` Element တွေထဲက Divider (ကန့်လန့်ဖြတ်လိုင်း) အဖြစ်အသုံးပြုလိုတဲ့ Element မှာ `.divider` ကို အသုံးပြုနိုင်ပါတယ်။ တစ်ချက်သတိပြုသင့်တာကတော့၊ Drop-down Menu အပါအဝင် Bootstrap ရဲ့ အချို့လုပ်ဆောင်ချက်တွေက jQuery အသုံးပြု ရေးသားထားခြင်းဖြစ်တဲ့အတွက် jQuery ပါမှ အလုပ်လုပ်မှာဖြစ်ပါတယ်။ နောက်ထပ်အသုံးဝင်တဲ့ Component ကတော့ Navigation Menu Bar ဖြစ်ပါတယ်။ အခုလို ထည့်သွင်းနိုင်ပါတယ်။

HTML

```
<nav class="navbar navbar-default">
  <div class="container-fluid">
    <div class="navbar-header">
      <a class="navbar-brand" href="#">Brand</a>
    </div>

    <ul class="nav navbar-nav">
      <li class="active"><a href="#">Link</a></li>
      <li><a href="#">Link</a></li>
    </ul>

  </div>
</nav>
```



Navigation Bar အတွက် `.navbar` `.navbar-default` Class တို့ကိုသုံးရပြီး၊ Menu Item တွေအတွက် `.nav` `.navbar-nav` Class တို့သတ်မှတ်ထားတဲ့ `<ul>` Element တစ်ခုကို အသုံးပြုရခြင်းဖြစ်ပါတယ်။ ဖြည့်စွက်ချက်အနေနဲ့ Brand Name ကို Menu Bar ထဲမှာ ထည့်သွင်းဖော်ပြစေလိုတဲ့အခါ `.navbar-header` Element ထဲက `.navbar-brand` Element နဲ့ သတ်မှတ်ပေးနိုင်ပါတယ်။ Menu Item တွေထဲက လက်ရှိ ရွေးချယ်ထားကြောင်း ပေါ်လွင်စေလိုတဲ့ Item မှာ `.active` Class ကို သတ်မှတ်ပေးနိုင်ပါတယ်။

Drop-down Menu တွေ Button တွေနဲ့ Form Element တွေလည်း လိုအပ်ရင် Navbar ထဲမှာ တွဲဖက်ထည့်သွင်း အသုံးပြုနိုင်သလို၊ ညာဘက်ကပ်ဖော်ပြစေလိုတဲ့ Menu ရှိနရင်လည်း `.pull-right` Class ကို တွဲဖက် အသုံးပြုနိုင်ပါတယ်။ ဒါ့အပြင် Navbar ကိုနမူနာမှာပြထားသလို Light Color နဲ့ မဟုတ်ပဲ Dark Color နဲ့ ဖော်ပြ စေလိုရင် `.navbar-inverse` ကို `.navbar-default` အစား အစားထိုးအသုံးပြုနိုင်ပါတယ်။

အရောင်လေးတွေနဲ့ဖော်ပြတဲ့ Tag Label တွေအသုံးပြုလိုရင် `.label` Class နဲ့အတူ `.label-success`, `.label-info`, `.label-warning`, `.label-danger` စသဖြင့် တွဲဖက်အသုံးပြုနိုင်ပါ တယ်။

HTML

```
<span class="label label-default">Default</span>
<span class="label label-primary">Primary</span>
<span class="label label-success">Success</span>
<span class="label label-info">Info</span>
<span class="label label-warning">Warning</span>
<span class="label label-danger">Danger</span>
```



Menu Item (သို့မဟုတ်) အခြားလိုအပ်တဲ့နေရာတွေမှာ Item Count Notification ကို တွဲဖက်ဖော်ပြဖို့အတွက် Badge ကို အသုံးပြုနိုင်ပါတယ်။

HTML

```
<a href="#">Inbox <span class="badge">42</span></a>
```



Error Message (သို့မဟုတ်) အလားတူ Information တွေဖော်ပြဖို့အတွက် `.alert` ကို အသုံးပြုနိုင်ပါတယ်။

HTML

```
<div class="alert alert-success">...</div>
```

Well done! You successfully read this important alert message.

`.alert-success` အစား `.alert-info`, `.alert-warning`, `.alert-danger` စတဲ့ Class ကို လိုအပ်သလိုအစားထိုးအသုံးပြုနိုင်ပါတယ်။ Component တွေထဲမှာ နောက်ဆုံးမှတ်သားသင့်တဲ့ Component ကတော့ Panel ဖြစ်ပါတယ်။ အခုလိုရေးသားရပါတယ်။

HTML

```
1. <div class="panel panel-primary">
2.   <div class="panel-header">Panel Title</div>
3.   <div class="panel-body">Panel Content</div>
4. </div>
```

Panel title

Panel content

နမူနာမှာ `.panel` နဲ့အတူ `.panel-primary` ကို တွဲဖက်အသုံးပြုထားပါတယ်။ ထုံးစံအတိုင်း `.panel-primary` အစား `.panel-success`, `.panel-info`, `.panel-warning`, `.panel-danger` တို့ကို တွဲဖက်အသုံးပြုနိုင်ပါတယ်။ Panel အတွင်းမှာ `.panel-header`, `.panel-body` နဲ့ `.panel-footer` သတ်မှတ်ထားတဲ့ Element တွေကို အသီးသီးထည့်သွင်းအသုံးပြုနိုင်ပါတယ်။

ကျွန်တော်တို့ရဲ့ Issue Tracking System Client App ကိုတည်ဆောက်တဲ့အခါ ဒီလုပ်ဆောင်ချက်တွေအားလုံးကို အသုံး ပြုသွားမှာဖြစ်ပါတယ်။

14.2 – jQuery DataTables

DataTables ဟာ jQuery Plugin တစ်ခုဖြစ်ပြီး ရိုးရိုး HTML Table တစ်ခုကို၊ Sorting, Filter, Paging လုပ်ဆောင်ချက်တွေပါဝင်တဲ့ Data Grid တစ်ခုဖြစ်အောင် ပြောင်းပေးနိုင်ပါတယ်။ `bower install` နဲ့ ရယူနိုင်သလို datatables.net မှာလည်း ကိုယ်တိုင် Download ရယူနိုင်ပါတယ်။

Show 10 entries Search:

Name	Position	Office	Age	Start date	Salary
Airi Satou	Accountant	Tokyo	33	2008/11/28	$162,700
Angelica Ramos	Chief Executive Officer (CEO)	London	47	2009/10/09	$1,200,000
Ashton Cox	Junior Technical Author	San Francisco	66	2009/01/12	$86,000
Bradley Greer	Software Engineer	London	41	2012/10/13	$132,000
Brenden Wagner	Software Engineer	San Francisco	28	2011/06/07	$206,850
Brielle Williamson	Integration Specialist	New York	61	2012/12/02	$372,000
Bruno Nash	Software Engineer	London	38	2011/05/03	$163,500
Caesar Vance	Pre-Sales Support	New York	21	2011/12/12	$106,450
Cara Stevens	Sales Assistant	New York	46	2011/12/06	$145,600
Cedric Kelly	Senior Javascript Developer	Edinburgh	22	2012/03/29	$433,060
Name	Position	Office	Age	Start date	Salary

Showing 1 to 10 of 57 entries Previous 1 2 3 4 5 6 Next

ပုံ (၁၄.၅) - DataTables Example

ပုံ (၁၄.၅) မှာဖော်ပြထားသလို လုပ်ဆောင်ချက်အပြည့်အစုံပါဝင်တဲ့ Data Grid ရရှိဖို့အတွက် DataTables ရဲ့ အကူ အညီနဲ့ အခုလို အလွယ်တစ်ကူရေးသားနိုင်ပါတယ်။

JavaScript

```
$("table").dataTables();
```

Data Grid ပြောင်းလိုတဲ့ Table ကို Select လုပ်ပြီး `dataTables()` Function တွဲဖက်ပေးလိုက်ခြင်းဖြစ်ပါတယ်။ DataTables ရဲ့ လုပ်ဆောင်ချက်အပြည့်အစုံကိုတော့ datatables.net မှာ ကိုယ်တိုင်ပဲ ဆက်လက်လေ့လာလိုက်ပါ။ ဒီနေရာမှာတော့ ကျွန်တော်တို့ ထည့်သွင်းအသုံးပြုမယ့် ထူးခြားချက်နှစ်ချက်ကို ဖော်ပြလိုပါတယ်။

ပထမတစ်ချက်ကတော့ Sorting ဖြစ်ပါတယ်။ Sorting စီတဲ့အခါ ဖော်ပြထားတဲ့အတိုင်း ကြီးစဉ်ငယ်လိုက် (သို့မဟုတ်) ငယ်စဉ်ကြီးလိုက်စီစဉ်တဲ့အလုပ်ငန်းကို DataTables က အလိုအလျောက်လုပ်ပေးနိုင်ပါတယ်။ ပြဿနာက ဖော်ပြတဲ့ တန်ဖိုးကတစ်ခု၊ အသုံးပြုစီစဉ်စေလိုတဲ့တန်ဖိုးကတစ်ခုဆိုတဲ့ လိုအပ်ချက်မျိုး ရှိတက်ပါတယ်။ ဥပမာ – Issue Status ကို အသုံးပြုသူ ဖတ်ရှုရလွယ်ကူစေဖို့အတွက် New, Assigned, Doing, Done, Close စသဖြင့် ဖော်ပြပေမယ့်၊ Sorting စီ တဲ့အခါမှာတော့ ငယ်စဉ်ကြီးလိုက်စီမယ်ဆိုရင် Assigned, Close, Doing, Done, New ဆိုပြီး A-Z စီပေးသွားမှာပါ။ လက်တွေ့မှာ ဖော်ပြထားတဲ့တန်ဖိုးအတိုင်း စီလို့မရပဲ New (0), Assigned (1), Doing (2) စသဖြင့် သက်ဆိုင်ရာ တန်ဖိုးအလိုက်စီပေးဖို့လိုအပ်ပါတယ်။ ဒီပြဿနာကို ပြေလည်စေဖို့အတွက် DataTables က `order-value` ဆိုတဲ့ Attribute တစ်ခုကို အသုံးပြုပါတယ်။ ဥပမာ –

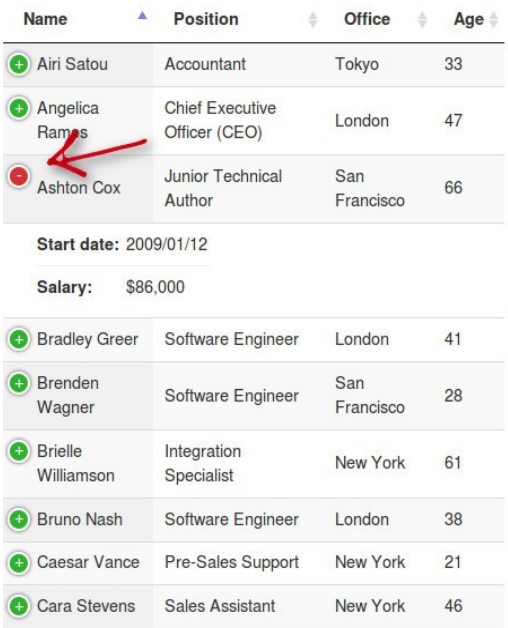
HTML

```
<table>
  <tr><td order-value="1">Assigned</td></tr>
  <tr><td order-value="4">Close</td></tr>
  <tr><td order-value="0">New</td></tr>
  <tr><td order-value="3">Done</td></tr>
  <tr><td order-value="2">Doing</td></tr>
</table>
```

နမူနာ Table မှာ ဖော်ပြမယ့်တန်ဖိုးတွေက Assigned, Close, New, Done, Doing စတဲ့တန်ဖိုးတွေဖြစ်ပေမယ့် လက်တွေ့ Sorting စီတဲ့အခါမှာတော့ DataTables က အဲ့ဒီတန်ဖိုးတွေနဲ့မစီပဲ `order-value` မှာပေးထားတဲ့ တန်ဖိုးကို အသုံးပြု စီပေးသွားမှာပဲဖြစ်ပါတယ်၊

နောက်တစ်ခုကတော့ Device အမျိုးမျိုးမှာ အလုပ်လုပ်စေနိုင်တဲ့ Responsive Table Design ဖြစ်ပါတယ်။ DataTables နဲ့ Bootstrap ဟာ သီးခြား Library တွေဖြစ်ကြတဲ့အတွက် တစ်ခုနဲ့တစ်ခု အသုံးပြုတဲ့ Style တွေမတူကြပါဘူး။ ဒါကြောင့် DataTables ရဲ့ ဖော်ပြပုံကို Bootstrap Table Style ကိုသုံးပြီး ဖော်ပြစေခြင်းနဲ့ Screen Size ပေါ်မူတည်ပြီး အဆင်ပြေအောင် အလိုက်သင့်ပြောင်းလဲဖော်ပြနိုင်စေခြင်းဟာလည်း လိုအပ်ချက်တစ်ခုဖြစ်ပါတယ်။ ဒီပြဿနာပြေလည်စေ ဖို့အတွက် DataTables Bootstrap 3 Plugin နဲ့ DataTables Responsive Plugin တို့ကို အသုံးပြုပေးဖို့လိုအပ်ပါတယ်။

- DataTables Bootstrap - https://datatables.net/examples/styling/bootstrap.html
- DataTables Responsive - https://datatables.net/extensions/responsive/



ပုံ (၁၄.၆) - DataTables Responsive Example

ဒီ Plugin နှစ်ခုကိုထည့်သွင်းလိုက်ရင်တော့ DataTables ရဲ့ ဖော်ပြပုံအသွင်အပြင်ကို Bootstrap Style နဲ့ဖော်ပြပေးသွားမှာဖြစ်ပြီး၊ Screen Size သေးသွားတဲ့အခါ **ပုံ (၁၄.၆)** မှာ နမူနာပြထားသလို တစ်ချို့ Table Column တွေကို ဖျောက်ထားပြီး၊ Button တစ်ခုကို နှိပ်လိုက်မှသာ ဖော်ပြအောင်ပေးမှာဖြစ်တဲ့အတွက် Screen Size သေးတဲ့ Device တွေမှာလည်း Data Grid က အဆင်ပြေပြေ ဆက်လက်အလုပ်လုပ်ပေးနိုင်မှာပဲဖြစ်ပါတယ်။

> ဒီအခန်းမှာ နမူနာဖော်ပြမယ့် Code တွေဟာ မပြည့်စုံသေးတဲ့အတွက် လက်တွေ့ကူးယူစမ်းသပ်လို့ ရဦးမှာမဟုတ်ပါဘူး။ နောက်တစ်ခန်းမှာ ဆက်လက်ဖော်ပြတဲ့ Code တွေနဲ့ ပေါင်းစပ်ပြီးမှသာ လက်တွေ့ စမ်းသပ်လို့ရမှာ ဖြစ်ပါတယ်။ Code နမူနာ အပြည့်အစုံကို အောက်ပါ လိပ်စာမှာ ကြိုတင် Download ရယူထားနိုင်ပါတယ်။

https://github.com/eimg/issues-backbone/archive/master.zip

14.3 – Directory Structure for Web Client

အခုဆိုရင် လိုအပ်တဲ့နည်းပညာနှစ်ခုကို ဖြည့်စွက်ဖော်ပြပြီးပြီမို့ Web Client တည်ဆောက်တဲ့အပိုင်းကို သွားနိုင်ပါပြီး ဒီလို တည်ဆောက်ရာမှာ ထမဦးဆုံးအနေနဲ့ လိုအပ်တဲ့ Directory Structure နဲ့ HTML Template တွေကို အရင် တည်ဆောက်ပါမယ်။ Directory တစ်ခုကို issues-backbone အမည်နဲ့ (သို့) နှစ်သက်ရာအမည်နဲ့ တည်ဆောက် ပြီး၊ လိုအပ်တဲ့ Package တွေကို Bower နဲ့ အခုလိုရယူပါမယ်။

```
$ mkdir issues-backbone && cd issues-backbone
$ bower install backbone jquery underscore bootstrap datatables
datatables-responsive datatables-bootstrap3 bootstrap3-wysiwyg jquery-
validate moment
```

အသုံးပြုဖို့လိုအပ်တဲ့ Backbone, jQuery, Undersocre, Bootstrap, DataTables, DataTables Responsive Plugin, DataTables Bootstrap Plugin စတဲ့ Library တွေကို ရယူထားပါတယ်။ ဒါ့အပြင် Rich Text Editor လုပ်ဆောင်ချက် ရရှိနိုင်ဖို့အတွက် Bootstrap3 Wysiwyg, Form Validation ပြုလုပ်နိုင်ဖို့အတွက် jQuery Validate နဲ့ ရက်စွဲတွေကို Format လုပ်ဖော်ပြနိုင်ဖို့အတွက် MomentJS တို့ကိုလည်း တွဲဖက်ထည့်သွင်းထားပါတယ်။

ရရှိလာတဲ့ Directory Structure က အခုလိုဖြစ်မှာပါ –

```
issues-backbone
└── bower_components
    ├── backbone
    ├── bootstrap
    ├── bootstrap3-wysiwyg
    ├── datatables
    ├── datatables-bootstrap3
    ├── datatables-responsive
```

```
    ├── jquery
    ├── jquery-validate
    ├── moment
    └── underscore
```

ဆက်လက်ပြီး လိုအပ်တဲ့ ဖိုင်တွေကို အခုလိုဖြည့်စွက်ပေးရပါမယ်။

```
issues-backbone
├── app.js
├── index.html
├── issues.js
├── users.js
├── bower_components
├── templates
│   ├── issue-detail.html
│   ├── issue-edit.html
│   ├── issue.html
│   ├── issue-list.html
│   ├── issue-new.html
│   ├── login.html
│   ├── nav.html
│   ├── user-edit.html
│   ├── user.html
│   ├── user-list.html
│   ├── user-new.html
│   └── user-profile.html
└── views
    ├── appView.js
    ├── issueDetailView.js
    ├── issueEditView.js
    ├── issueListView.js
    ├── issueNewView.js
    ├── loginView.js
    ├── navView.js
    ├── userEditView.js
    ├── userListView.js
    ├── userNewView.js
    └── userProfileView.js
```

`app.js` မှာ Client App အတွက်လိုအပ်တဲ့ အခြေခံ Function တွေကို ရေးသားမှာဖြစ်ပြီး `issues.js` နဲ့ `users.js` တို့မှာ သက်ဆိုင်ရာ Model နဲ့ Collection တွေကို ရေးသားမှာဖြစ်ပါတယ်။ `index.html` ကတော့ Main Template ဖိုင်အဖြစ်လိုအပ်တဲ့ Script နဲ့ Style တွေကိုစုစည်းချိတ်ဆက်ထားမှာ ဖြစ်ပါတယ်။ `views` Directory ထဲမှာတော့ လိုအပ်တဲ့ Backbone View တွေကို စုစည်းထားမှာဖြစ်ပြီး `templates` Directory ထဲမှာ View နဲ့အတူ တွဲဖက်အသုံးပြုရမယ့် HTML Template တွေကို ကြိုတင်ရေးသားထားမှာဖြစ်ပါတယ်။ ဒီအခန်းမှာ အဲ့ဒီ Template တွေ ကိုဖော်ပြပြီး နောက်တစ်ခန်းမှာ Model-View-Collection တွေတည်ဆောက်ပုံကို ဆက်လက်ဖော်ပြသွားမှာဖြစ်ပါတယ်။

14.4 – Issue Detail Template – `templates/issue-detail.html`

ဆက်လက်ပြီး Issue တစ်ခုစီရဲ့ အသေးစိတ်အချက်အလက်တွေကို ဖော်ပြပေးတဲ့ Template ဖြစ်တဲ့ Issue Template တစ်ခုကို `templates/issue-detail.html` မှာ ရေးသားပါမယ်။

Template

```
<div class="row">
  <div class="col-md-8 col-sm-12 col-xs-12">
    <div class="panel
      <% if(priority == 0) { %>
        panel-info
      <% } else if (priority == 1) { %>
        panel-warning
      <% } else { %>
        panel-danger
      <% } %>
      issue-detail">
      <div class="panel-heading">
        <h2>
          <% if(type == 0) { %>
             <span class="glyphicon glyphicon-fire"></span>
          <% } else if (type == 1) { %>
            <span class="glyphicon glyphicon-heart-empty"></span>
          <% } else { %>
            <span class="glyphicon glyphicon-asterisk"></span>
          <% } %>
          <%= summary %>
          <span class="pull-right small">#<%= _id %></span>
        </h2>
      </div>
      <div class="panel-body">
        <%= detail %>
      </div>
      <div class="panel-footer clearfix">
        <span class="text-muted"><%= moment(submittedAt).fromNow() %></span>
        <div class="btn-group pull-right">
          <button class="btn btn-default btn-sm" id="close-issue">
            <span class="glyphicon glyphicon-ok text-success"></span>
            Close
          </button>
          <button class="btn btn-default btn-sm" id="edit-issue">
            <span class="glyphicon glyphicon-edit text-info"></span>
            Edit
          </button>
          <button class="btn btn-default btn-sm" id="delete-issue">
            <span class="glyphicon glyphicon-trash text-danger"></span>
            Remove
          </button>
        </div>
      </div>
    </div>

    <hr>
    <% if(typeof comments !== "undefined") { %>
    <% _.each(comments, function(comment, id) { %>

      <div class="panel panel-default small comment">
```

```
52.         <div class="panel-heading">
53.            <span class="glyphicon glyphicon-comment"></span>
54.           <b class="text-primary"><%= comment.authorName %></b>
55.           <time class="text-muted">
56.              <%= moment(comment.submittedAt).fromNow() %>
57.           </time>
58.
59.           <a href="#" title="" class="pull-right
60.                                       text-danger
61.                                       delete-comment"
62.                                data-index="<%= id %>">
63.             <span class="glyphicon glyphicon-remove"></span>
64.           </a>
65.
66.           <span class="pull-right text-muted">#<%= id %></span>
67.         </div>
68.         <div class="panel-body">
69.           <p>
70.             <%= comment.comment %>
71.           </p>
72.         </div>
73.       </div>
74.     <% }) %>
75.     <% } %>
76.
77.     <div class="comment-form">
78.       <div class="form-group">
79.         <label>Comment</label>
80.         <textarea class="form-control"
81.                     placeholder="Comment" id="comment"></textarea>
82.       </div>
83.       <div class="form-group">
84.         <button type="submit" class="btn btn-default" id="post-comment">
85.           <span class="glyphicon glyphicon-comment"></span>
86.           Add Comment
87.         </button>
88.       </div>
89.     </div>
90.    </div>
91.
92.   <div class="col-md-4 col-sm-12 col-xs-12 sidebar">
93.     <div class="btn-group btn-breadcrumb">
94.       <a href="#" class="btn btn-danger">
95.         <% if(type == 0) { %>
96.           <span class="glyphicon glyphicon-fire"></span>
97.         <% } else if (type == 1) { %>
98.           <span class="glyphicon glyphicon-heart-empty"></span>
99.         <% } else { %>
100.          <span class="glyphicon glyphicon-asterisk"></span>
101.        <% } %>
102.      </a>
103.      <a href="#" class="btn btn-danger">Type</a>
104.
105.      <a href="#" class="btn btn-default" data-toggle="dropdown">
106.        <%= typeLabel %> <span class="caret"></span>
107.      </a>
108.      <ul class="dropdown-menu dropdown-menu-right">
109.        <% _.each(app.issueType, function(label, type) { %>
110.          <li>
```

```
            <a href="#" class="change-type" data-value="<%= type %>">
              <%= label %>
            </a>
          </li>
        <% }) %>
      </ul>
    </div>


    <div class="btn-group btn-breadcrumb">
      <a href="#" class="btn btn-warning">
        <span class="glyphicon glyphicon-sort-by-attributes-alt"></span>
      </a>
      <a href="#" class="btn btn-warning">Priority</a>
      <a href="#" class="btn btn-default" data-toggle="dropdown">
        <%= priorityLabel %> <span class="caret"></span>
      </a>
      <ul class="dropdown-menu dropdown-menu-right">
        <% _.each(app.issuePriority, function(label, priority) { %>
          <li>
            <a href="#" class="change-priority"
                 data-value="<%= priority %>"><%= label %></a>
          </li>
        <% }) %>
      </ul>
    </div>

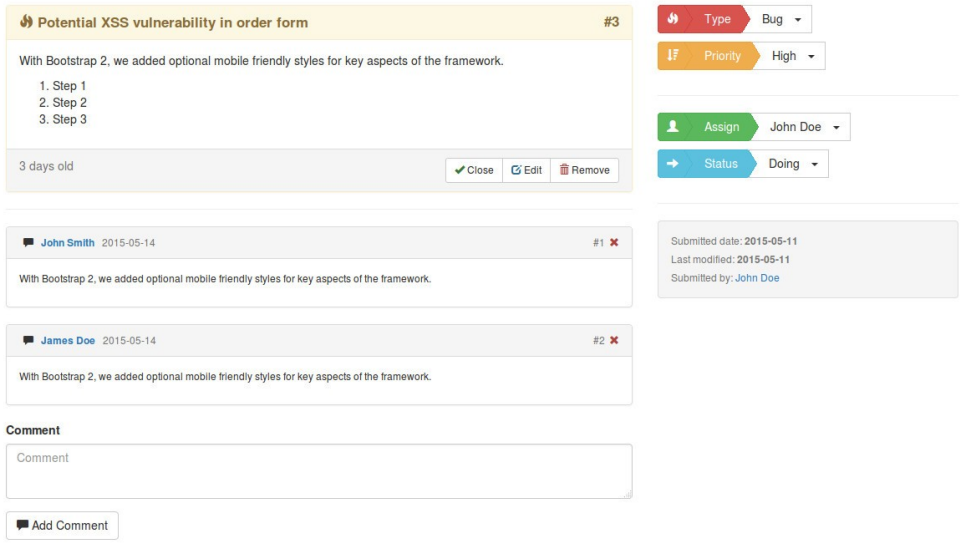
    <hr>

    <div class="btn-group btn-breadcrumb">
      <a href="#" class="btn btn-success">
        <span class="glyphicon glyphicon-user"></span>
      </a>
      <a href="#" class="btn btn-success">Assign</a>
      <a href="#" class="btn btn-default" data-toggle="dropdown">
        <%= assignedToLabel %> <span class="caret"></span>
      </a>
      <ul class="dropdown-menu dropdown-menu-right">
        <% _.each(users, function(user, id) { %>
          <li>
            <a href="#" class="change-assign"
                 data-value="<%= user._id %>"><%= user.fullName %></a>
          </li>
        <% }) %>
      </ul>
    </div>

    <div class="btn-group btn-breadcrumb">
      <a href="#" class="btn btn-info">
        <span class="glyphicon glyphicon-arrow-right"></span>
      </a>
      <a href="#" class="btn btn-info">Status</a>
      <a href="#" class="btn btn-default" data-toggle="dropdown">
       <%= statusLabel %> <span class="caret"></span>
      </a>
      <ul class="dropdown-menu dropdown-menu-right">
        <% _.each(app.issueStatus, function(label, status) { %>
          <li>
            <a href="#" class="change-status" data-value="<%= status %>">
```

```
171.                 <%= label %></a>
172.             </li>
173.           <% }) %>
174.         </ul>
175.       </div>
176.
177.       <hr>
178.
179.       <div class="alert text-muted meta">
180.         <p>
181.           Submitted date: <b>
182.             <%= moment(submittedAt).format("DD-MM-YYYY h-mm-ss a") %>
183.           </b>
184.         </p>
185.         <p>Last modified: <b>
186.           <% if(typeof modifiedAt !== 'undefined') { %>
187.             <%= moment(modifiedAt).format("DD-MM-YYYY h-mm-ss a") %>
188.           <% } %>
189.         </b></p>
190.         <p>Submitted by: <b>
191.           <% var submitter = app.userList.get({ _id: submittedBy }) %>
192.           <%= submitter.get("fullName") %>
193.         </b></p>
194.       </div>
195.     </div>
196. </div>
```

HTML Structure အပြည့်စုံထည့်သွင်းထားခြင်းမရှိသလို၊ အသုံးပြုလိုတဲ့ Library တွေကို ချိတ်ဆက်ထားခြင်းလည်း မရှိ တာကိုသတိပြုပါ။ ဒီ Template ဟာ သူ့ချည်းသက်သက် အလုပ်လုပ်တဲ့ Page တစ်ခုမဟုတ်ပဲ နောက်တော့မှ Backbone View ကနေ ရယူအသုံးပြုနိုင်ဖို့ ကြိုတင်ရေးသားထားပေးတဲ့ Template တစ်ခုဖြစ်ပါတယ်။ Backbone View နဲ့ ချိတ်ဆက်ပြီးရင် ရရှိမယ့် ရလဒ်နမူနာကို **ပုံ (၁၄.၇)** မှာလေ့လာနိုင်ပါတယ်။



ပုံ (၁၄.၇) - Issue Detail Template

Column နှစ်ခုပါဝင်တဲ့ Layout ဖြစ်ပြီး၊ ဘယ်ဘက် Column ထဲမှာ Bootstrap Panel တစ်ခုနဲ့ Issue Summary နဲ့ Detail တို့ကို ဖော်ပြထားပါတယ်။ Panel ရဲ့ အရောင်ဟာ `.panel-info`, `.panel-warning`, `.panel-danger` စသဖြင့်လက်ရှိ Issue ရဲ့ Priority ပေါ်မူတည်ပြီး ဖော်ပြအောင် လိုင်းနံပါတ် (၄) မှာ စီစစ် ဖော်ပြထားစေတာကို သတိပြုကြည့်ပါ။

Issue Summary ရှေ့ကဖော်ပြတဲ့ Icon ကိုလည်း Issue Type ပေါ်မူတည်ပြီးပြောင်းလဲဖော်ပြအောင် လိုင်းနံပါတ် (၁၄) မှာ စီစစ်ထားပါသေးတယ်။ ပြီးတဲ့အခါ Panel Footer ထဲမှာ `.btn-group` နဲ့ Close, Edit, Delete Button တွေကို တန်းစီထည့်သွင်းထားပါတယ်။

Main Panel ရဲ့အောက်မှာတော့ နောက်ထပ် Panel များနဲ့ Comment တွေကို (ရှိရင်) တန်းစီဖော်ပြသွားမှာဖြစ်ပါ တယ်။ အောက်ဆုံးမှာတော့ Comment အသစ်ထည့်သွင်းနိုင်ဖို့အတွက် Textarea တစ်ခုနဲ့ Button တစ်ခုကို ပေး ထားပါတယ်။

ညာဘက် Column ထဲမှာတော့ Issue ရဲ့ Type, Priority, Assigned To နဲ့ Status တို့ကို ပြောင်းလဲသတ်မှတ်တိုင် ဖို့ အတွက် Bootstrap Dropdown တွေကို တန်းစီနေရာချထားပါတယ်။ Column နှစ်ခုကို သတ်မှတ်တဲ့အခါ Medium Screen မှာဆိုရင် 8-4 အချိုးနဲ့ဖော်ပြပြီး Small Screen နဲ့ Extra-Small Screen တွေမှာဆိုရင် 12 Column အပြည့် ဖော်ပြဖို့သတ်မှတ်ထားတဲ့အတွက် ဒီ Layout ဟာ Screen Size အမျိုးမျိုးမှာ သင့်တော်အောင် ဖော်ပြပေးနိုင်တဲ့ Responsive Layout တစ်ခုလည်းဖြစ်ပါတယ်။

14.5 – Issue Edit Template – `templates/issue-edit.html`

Issue တစ်ခုကို ပြင်ဆင်တဲ့အခါ အသုံးပြုနိုင်ဖို့အတွက် Form တစ်ခုကို ဆက်လက်ရေးသား ပါမယ်။ `templates/issue-edit.html` မှာ အခုလိုရေးသားပေးပါ။

Template

```
<form action="#" id="issue-edit-form">
  <div class="alert alert-info">
    Issue ID: <b><%= _id %></b>
  </div>
  <div class="form-group">
    <label for="">Summary *</label>
    <input type="text" class="form-control"
       placeholder="Summary" id="summary" value="<%= summary %>">
  </div>

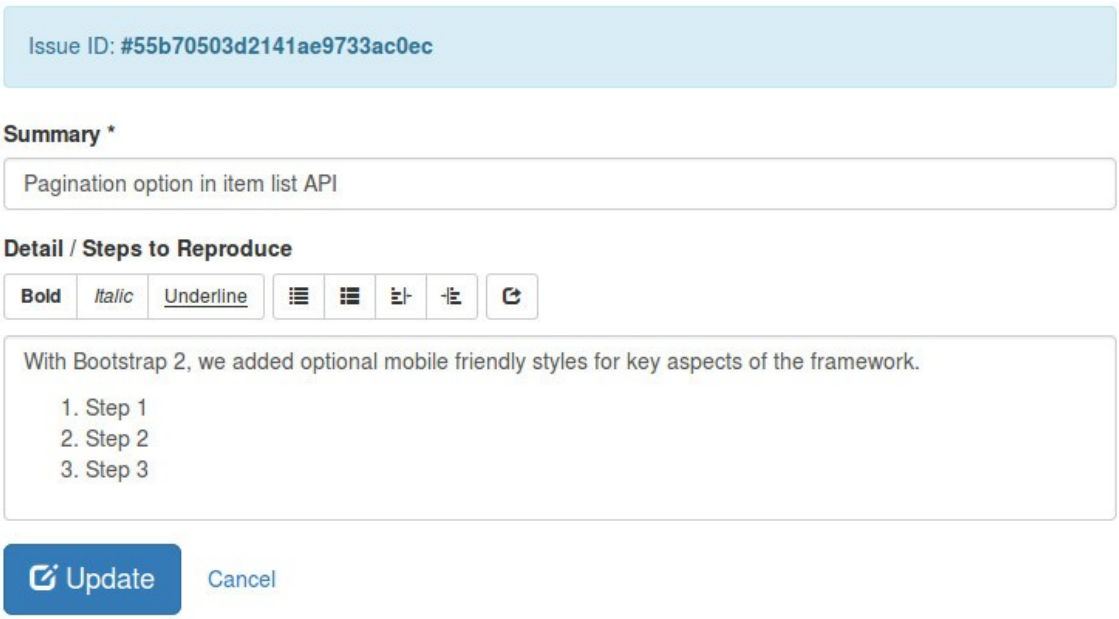
  <div class="form-group">
    <label for="">Detail / Steps to Reproduce</label>
    <div class="form-control editor"
       placeholder="Detail / Step to reproduce" id="detail">
       <%= detail %></div>
  </div>

  <button type="submit" class="btn btn-primary btn-lg" id="update-issue">
    <span class="glyphicon glyphicon-edit"></span>
    Update
```

```
  </button>
  <button class="btn btn-link" id="close-edit">Cancel</button>
</form>

<script>
  $(function() {
    $('.editor').wysihtml5({
      toolbar: {
        "font-styles": false,
        "emphasis": true,
        "lists": true,
        "html": false,
        "link": true,
        "image": false,
        "color": false,
        "blockquote": false,
        "size": "sm"
      }
    });
  });
</script>
```

View နဲ့ချိတ်ဆက်ပြီးတဲ့အခါ ရရှိလာမယ့် ရလဒ်က အခုလိုဖြစ်မှာပါ။



ပုံ (၁၄.၈) - Issue Edit Template

ဒီ Template မှာတော့ Column တစ်ခုသာရှိပြီး Issue Summary, နဲ့ Issue Detail ပြင်ဆင်ရာမှာအသုံးပြုနိုင်ဖို့ အတွက် Form Input တွေကို သတ်မှတ်ထားခြင်း ဖြစ်ပါတယ်။ Wysihtml5 Plugin အကူအညီနဲ့ Issue Detail Textarea ကို Rich Text Editor ပြောင်းလဲထားတာကို သတိပြုပါ။

14.6 – Issue List Templates – `templates/issue-list.html, issue.html`

Issue List ဖော်ပြတဲ့ Template ကိုတော့ နှစ်ပိုင်းခွဲထားပါတယ်။ Issue List Table Template နဲ့ Issue List Item Table Row Template တို့ဖြစ်ပါတယ်။ Issue List Table Template ကို `templates/issue-list.html` မှာ အခုလိုရေးသားရပါမယ်။

Template

```
<table id="issue-list-table"
    class="table table-bordered" cellspacing="0" width="100%">
  <thead>
    <tr>
      <th>#</th>
      <th width="50%">Summary</th>
      <th>Type</th>
      <th>Assigned To</th>
      <th>Priority</th>
      <th>Status</th>
    </tr>
  </thead>

  <tbody>

  </tbody>
</table>

<script>
  $(function() {
    $("#issue-list-table").dataTable({
      responsive: true
    });
  });
</script>
```

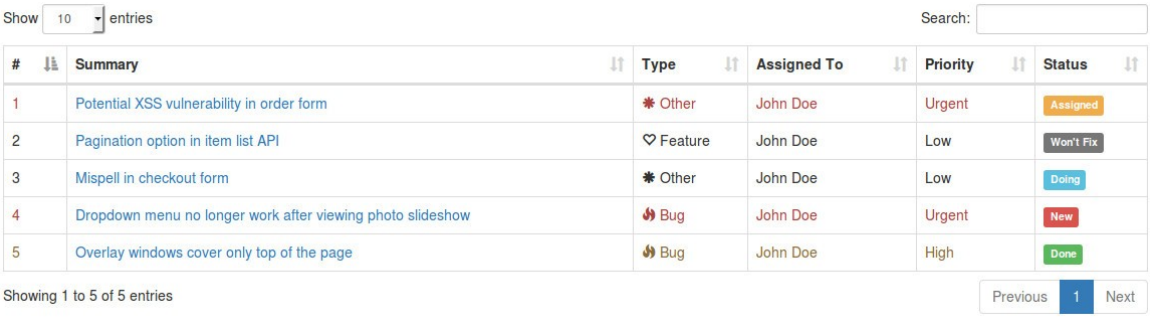
ဘာမှထူးထူးထွေထွေလုပ်ဆောင်ချက်မပါဝင်တဲ့ Table Template အလွတ်တစ်ခုသာဖြစ်ပါတယ်။ DataTables ကိုသုံးပြီး Table ကို Data Grid ပြောင်းပေးဖို့ကိုတော့ ထည့်သွင်းသတ်မှတ်ထားပါတယ်။ ဆက်လက်ပြီး Issue List Item Table Row Template ကို `templates/issue.html` မှာ အခုလိုရေးသားရပါမယ်။

Template

```
<td><%= index %></td>
<td>
  <% if (status == 4) { %>
    <a href="#" data-id="<%= _id %>" class="issue-summary">
    <del><%= summary %></del></a>
  <% } else { %>
    <a href="#" data-id="<%= _id %>" class="issue-summary">
    <%= summary %></a>
  <% } %>
</td>
<td data-order="<%= type %>">
  <% if(type == 0) { %>
    <span class="glyphicon glyphicon-fire"></span>
```

```
14.    <% } else if (type == 1) { %>
15.      <span class="glyphicon glyphicon-heart-empty"></span>
16.    <% } else { %>
17.      <span class="glyphicon glyphicon-asterisk"></span>
18.    <% } %>
19.    <%= typeLabel %>
20.  </td>
21.  <td><%= assignedToLabel %></td>
22.  <td data-order="<%= priority %>"><%= priorityLabel%></td>
23.  <td data-order="<%= status %>">
24.    <% if(status == 0) { %>
25.      <span class="label label-danger"><%= statusLabel %></span>
26.    <% } else if (status == 1) { %>
27.      <span class="label label-warning"><%= statusLabel %></span>
28.    <% } else if (status == 2) { %>
29.      <span class="label label-info"><%= statusLabel %></span>
30.    <% } else if (status == 3) { %>
31.      <span class="label label-success"><%= statusLabel %></span>
32.    <% } else if (status == 4) { %>
33.      <span class="label label-default"><%= statusLabel %></span>
34.    <% } %>
35.  </td>
```

Issue Type အလိုက် သင့်တော်တဲ့ Icon လေးတွေ ဖော်ပြပေးဖို့နဲ့ Issue Status ပေါ်မူတည်ပြီး သင့်တော်တဲ့ Label Tag လေးတွေဖော်ပြပေးဖို့ စီစစ်ရေးသားထားတဲ့ Template တစ်ခုဖြစ်ပါတယ်။ သက်ဆိုင်ရာ View နဲ့ ချိတ်ဆက်ပြီးရင် ရရှိလာမယ့်ရလဒ်က ဒီလိုပါ။



ပုံ (၁၄.၉) - Issue List Template

DataTables ရဲ့ အကူအညီနဲ့ Sorting, Paging နဲ့ Filter လုပ်ဆောင်ချက်တွေ ပါဝင်တဲ့ Issue List Table ကို Bootstrap Table Style နဲ့ ဖော်ပြပေးမှာ ဖြစ်ပါတယ်။

14.7 – New Issue Template – `templates/issue-new.html`

ဆက်လက်ပြီး Issue အသစ်ထည့်သွင်းတဲ့အခါမှာ အသုံးပြုရမယ့် Form အတွက် Template တစ်ခုကို `templates/issue-new.html` မှာ အခုလိုရေးသားပေးပါ။

Template

```
<form action="#" id="new-issue-form">
  <div class="form-group">
    <label for="">Summary *</label>
    <input type="text" class="form-control"
      placeholder="Summary" id="summary">
  </div>

  <div class="form-group">
    <label for="">Detail / Steps to Reproduce</label>
    <div class="form-control editor" id="detail"></div>
  </div>

  <div class="form-group">
    <label for="">Category</label>
    <select class="form-control" id="type">
      <% _.each(types, function(type, val) { %>
        <option value="<%= val %>"><%= type %></option>
      <% }); %>
    </select>
  </div>

  <div class="form-group">
    <label for="">Priority</label>
    <select class="form-control" id="priority">
      <% _.each(priorities, function(type, val) { %>
        <option value="<%= val %>"><%= type %></option>
      <% }); %>
    </select>
  </div>

  <button type="submit" class="btn btn-primary btn-lg" id="add-button">
    <span class="glyphicon glyphicon-plus-sign"></span>
    Add Issue
  </button>
</form>

<script>
  $(function() {
    $('.editor').wysihtml5({
      toolbar: {
        "font-styles": false,
        "emphasis": true,
        "lists": true,
        "html": false,
        "link": true,
        "image": false,
        "color": false,
        "blockquote": false,
        "size": "sm"
      }
    });
```

```
53.   });
54. </script>
```

Issue Summary, Issue Detail နဲ့ Issue Priority တို့ သတ်မှတ်လို့ရတဲ့ Form Template တစ်ခုဖြစ်ပြီး သူလည်းပဲ Issue Edit လိုပဲ Issue Detail Textarea ကို Rich Text Editor အဖြစ် ပြောင်းလဲသတ်မှတ်ထားပါတယ်။

14.8 – Login Form Template – `templates/login.html`

ဆက်လက်တည်ဆောက်မယ့် Template ကတော့ Login Form အတွက်ဖြစ်ပါတယ်။ `teamplates/login.html` မှာ အခုလိုရေးသားရပါမယ်။

Template

```
1. <form action="#" id="login-form">
2.   <div class="alert alert-warning" id="login-alert"
3.     style="display: none">
4.     <span class="glyphicon glyphicon-warning-sign"></span>
5.     Login ID or Password incorrect
6.   </div>
7.
8.   <div class="form-group">
9.     <label for="login-id">Login ID</label>
10.    <input type="text" class="form-control" placeholder="Login ID"
11.      id="login-id">
12.  </div>
13.  <div class="form-group"  placeholder="Email">
14.    <label for="password">Password</label>
15.    <input type="password" class="form-control" id="password"
16.       placeholder="Password">
17.  </div>
18.
19.  <button type="submit" class="btn btn-primary btn-lg" id="login">
20.    Login
21.    <span class="glyphicon glyphicon-log-in"></span>
22.  </button>
23. </form>
```

LoginID နဲ့ Password တို့ထည့်သွင်းလို့ရတဲ့ Form Template တစ်ခုဖြစ်ပြီး အပေါ်ဆုံးမှာ Login Error ဖြစ်ရင် ဖော်ပြ ရမယ့် Bootstrap Style Alert Box တစ်ခုကိုပါ ထည့်သွင်းသတ်မှတ်ထားပါတယ်။

14.9 – Navigation Menu Template – `templates/nav.html`

Navigation Menu ကိုလည်းသီးခြား Template တစ်ခုနဲ့ ခွဲခြားရေးသားထားပါဦးမယ်။ `templates/nav.html` မှာ အခုလိုရေးသားရပါမယ်။

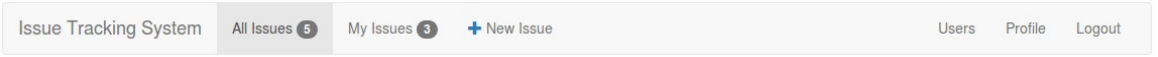
Template

```
<div class="container-fluid">
  <div class="navbar-header">
    <button type="button" class="navbar-toggle"
        data-toggle="collapse" data-target=".navbar-collapse">
      <span class="icon-bar"></span>
      <span class="icon-bar"></span>
      <span class="icon-bar"></span>
    </button>
    <a class="navbar-brand" href="#">Issue Tracking System</a>
  </div>
  <div class="navbar-collapse collapse">
    <% if(auth) { %>
    <ul class="nav navbar-nav">
      <li class="active">
        <a href="#/issues" id="issue-list">
          All Issues <span class="badge"><%= issueCount %></span>
        </a>
      </li>
      <li><a href="#" id="my-issues">My Issues  <span class="badge">
         <%= myIssues %></span></a></li>
      <li>
        <a href="#" id="new-issue">
          <span class="text-primary glyphicon glyphicon-plus"></span>
          New Issue
        </a>
      </li>
    </ul>
    <ul class="nav navbar-nav navbar-right">
      <li><a href="#/users" id="user-list">Users</a></li>
      <li><a href="#" id="profile"><%= userName %></a></li>
      <li><a href="#" id="logout">Logout</a></li>
    </ul>
    <% } %>
  </div>
</div>
```

Navbar ရဲ့ ရှေ့နားမှာ Issue Tracking System ဆိုတဲ့အမည်ကို Brand Name အဖြစ်ထည့်သွင်းထားပြီး၊ Navbar Menu အနေနဲ့ နှစ်ခုပါဝင်ပါတယ်။ ဘယ်ဘက်ခြမ်းက Menu မှာ All Issues, My Issues နဲ့ New Issue တို့ကို သွားလို့ရတဲ့ Button တွေဘာဝင်ပြီး ညာဘက်ခြမ်းက Menu မှာတော့ User List, User Profile နဲ့ Logout Button တို့ ပါဝင်ပါတယ်။ Menu Item တွေကို Authenticated User မှသာ ဖော်ပြဖို့ကိုလည်း ထည့်သွင်းစီစစ်ထားပါတယ်။

Navbar ရဲ့ ရှေ့ဆုံးမှာ Button တစ်ခုကိုထည့်သွင်းထားတဲ့ရည်ရွယ်ချက်ကတော့ Responsive လုပ်ဆောင်ချက် ရရှိနိုင်ဖို့ အတွက်ဖြစ်ပါတယ်။ အခြေခံအလုပ်လုပ်ပုံကတော့၊ ပုံမှန်ဆိုရင် Navbar Menu တွေကိုဖော်ပြပြီး Screen သေးလို့ Menu ကို မဖော်ပြနိုင်တဲ့အခါ Menu ကို ဖျောက်လိုက်ပြီး Navbar ရဲ့ ရှေ့ဆုံးက Toggle Button

ပေါ်လာမှာဖြစ်ပါတယ်။ Button ကို နှိပ်လိုက်မှ Menu ကိုဖော်ပြမှာဖြစ်တဲ့အတွက် Screen Size သေးတဲ့ Device တွေမှာလည်း Menu ရဲ့ အလုပ်လုပ်ပုံက အလိုက်သင့် အဆင်ပြေနေစေမှာ ဖြစ်ပါတယ်။



ပုံ (၁၄.၁၀) - Navbar Template

14.10 – User Edit Template – `templates/user-edit.html`

ဆက်လက်ပြီး User တစ်ဦးချင်းစီရဲ့ အချက်အလက်တွေနဲ့ Password တို့ကို ပြုပြင်လို့ရတဲ့ User Edit Template တစ်ခုကို တည်ဆောက်ပါမယ်။ `templates/user-edit.html` မှာ အခုလိုရေးသားရပါမယ်။

Template

```
<h2>Edit User</h2>

<form action="#" id="edit-user-form">
  <div class="form-group">
    <label for="">Full Name *</label>
    <input type="text" class="form-control"
      placeholder="Full Name" id="full-name" value="<%= fullName %>">
  </div>
  <div class="form-group">
    <label for="">Login ID *</label>
    <input type="text" class="form-control"
      placeholder="Login ID" id="login-id" value="<%= loginId %>">
    <span class="help-block">No space, small letters</span>
  </div>
  <div class="form-group"  placeholder="Email">
    <label for="">Email *</label>
    <input type="email" class="form-control" id="email"
      placeholder="Email" value="<%= email %>">
  </div>
  <div class="form-group">
    <label for="">Role *</label>
    <select class="form-control" id="role">
      <% _.each(app.userRole, function(name, value) { %>
        <option value="<%= value %>"
          <%= value == role ? 'selected' : '' %>><%= name %></option>
      <% }) %>
    </select>
  </div>
  <div class="form-group">
    <label for="">Password (twice)</label>
    <div class="row">
      <div class="col-md-6">
        <input type="password" class="form-control"
          placeholder="Password" id="password">
        <span class="help-block">Leave blank if you don't
          want to change password</span>
      </div>
      <div class="col-md-6">
        <input type="password" class="form-control"
          placeholder="Password Again" id="password-again">
```

```
      </div>
    </div>
  </div>

  <button type="submit" class="btn btn-primary btn-lg" id="update-user">
    <span class="glyphicon glyphicon-check"></span>
    Update User
  </button>
</form>
```

User ရဲ့ Full Name, Login ID, Email, Role နဲ့ Password တို့ကို ပြုပြင်လို့ရတဲ့ Form Template တစ်ခုဖြစ်ပါတယ်။

14.11 – User List Templates – `templates/user-list.html, user.html`

User စာရင်းဖော်ပြတဲ့ List Template ကိုတော့ Issue List လိုပဲ နှစ်ပိုင်းခွဲတည်ဆောက်ပါတယ်။ User List Table Template နဲ့ User Row Template တို့ဖြစ်ပါတယ်။ `templates/user-list.html` အခုလိုရေးသားရမှာပါ။

Template

```
<div class="table-responsive">
  <table id="issue-list" class="table table-bordered"
    cellspacing="0" width="100%">
    <thead>
      <tr>
        <th>#</th>
        <th>Full Name</th>
        <th>Login ID</th>
        <th>Email</th>
        <th>Role</th>
        <th width="160">Manage</th>
      </tr>
    </thead>

    <tbody>

    </tbody>
  </table>
</div>

<button class="btn btn-primary" id="new-user">
  <span class="glyphicon glyphicon-plus-sign"></span>
  New User
</button>
```

ထူးခြားတဲ့လုပ်ဆောင်ချက်မပါဝင်ပဲ Table အလွတ်တစ်ခုထည့်သွင်းထားခြင်းဖြစ်ပါတယ်။ `templates/user.html` မှာ အခုလိုဆက်လက်ရေးသားပေးရပါမယ်။

Template

```
<td><%= index %></td>
<td><a href="#" class="username"><%= fullName %></a></td>
<td><%= loginId %></td>
<td><%= email %></td>
<td><%= roleLabel %></td>
<td>
  <a href="#" class="btn btn-default btn-xs edit-user">
    <span class="glyphicon glyphicon-edit"></span>
    Edit
  </a> -
  <a href="#" class="btn btn-danger btn-xs delete-user">
    <span class="glyphicon glyphicon-trash"></span>
    Remove
  </a>
</td>
```

Table Row Template တစ်ခုဖြစ်ပြီး သက်ဆိုင်ရာနေရာမှာ Full Name, Login ID, Email စတဲ့တန်ဖိုးတွေကို သူ့နေ ရာနဲ့သူ အစားထိုးဖော်ပြဖို့ သတ်မှတ်ထားပါတယ်။

14.12 – User New Template – `templates/user-new.html`

User အသစ်တွေထည့်သွင်းရာမှာ အသုံးပြုနိုင်ဖို့အတွက် New User Form Template တည်ဆောက်ပေးရပါဦး မယ်။ `templates/user-new.html` မှာ အခုလိုရေးသားပေးပါမယ်။

Template

```
<h2>New User</h2>

<form action="#" id="new-user-form">
  <div class="form-group">
    <label for="">Full Name *</label>
    <input type="text" class="form-control"
      placeholder="Full Name" id="full-name">
  </div>
  <div class="form-group">
    <label for="">Login ID *</label>
    <input type="text" class="form-control"
      placeholder="Login ID" id="login-id">
    <span class="help-block">No space, small letters</span>
  </div>
  <div class="form-group"  placeholder="Email">
    <label for="">Email *</label>
    <input type="email" class="form-control" id="email"
      placeholder="Email">
  </div>
  <div class="form-group">
    <label for="">Role *</label>
    <select class="form-control" id="role">
      <% _.each(app.userRole, function(name, value) { %>
        <option value="<%= value %>"><%= name %></option>
      <% }) %>
    </select>
```

```
  </div>
  <div class="form-group">
    <label for="">Password (twice) *</label>
    <div class="row">
      <div class="col-md-6">
        <input type="password" class="form-control"
          placeholder="Password" id="password">
      </div>
      <div class="col-md-6">
        <input type="password" class="form-control"
          placeholder="Password Again" id="password-again">
      </div>
    </div>
  </div>

  <button type="submit" class="btn btn-primary btn-lg" id="add-user">
    <span class="glyphicon glyphicon-plus-sign"></span>
    Add User
  </button>
</form>
```

User Edit မှာလိုပဲ Full Name, Login ID, Email, Role နဲ့ Password တို့ကို ထည့်သွင်းနိုင်တဲ့ Form Template တစ်ခုဖြစ်ပါတယ်။

14.13 – User Profile Template – `templates/user-profile.html`

နောက်ဆုံး Template တစ်ခုအနေနဲ့ User တစ်ယောက်ရဲ့ အချက်အလက်တွေကိုဖော်ပြပေးတဲ့ Profile Template တစ်ခုတည်ဆောက်ဖို့အတွက် `templates/user-profile.html` မှာ အခုလို ရေးသားပေးပါမယ်။

Template

```
<div class="row">
  <div class="col-md-8 col-sm-12 col-xs-12">
    <div class="panel panel-info issue-detail">
      <div class="panel-heading">
        <h2><span class="glyphicon glyphicon-user"></span>
          <%= fullName %></h2>
      </div>
      <div class="panel-body">
        <div class="form-horizontal" role="form">
          <div class="form-group">
            <label class="col-sm-2 control-label">Login ID</label>
            <div class="col-sm-10">
              <p class="form-control-static"><%= loginId %></p>
            </div>
          </div>
          <div class="form-group">
            <label class="col-sm-2 control-label">Email</label>
            <div class="col-sm-10">
              <p class="form-control-static"><%= email %></p>
            </div>
          </div>
```

```
        </div>
      </div>
      <div class="panel-footer clearfix">
        <div class="btn-group pull-right">
          <button class="btn btn-default btn-sm" id="edit-user">
            <span class="glyphicon glyphicon-edit text-info"></span>
            Edit
          </button>
        </div>
      </div>
    </div>
  </div>

  <div class="col-md-4 col-sm-12 col-xs-12 sidebar">
    <div class="btn-group btn-breadcrumb">
      <a href="#" class="btn btn-info">
        <span class="glyphicon glyphicon-star"></span></a>
      <a href="#" class="btn btn-info">Role</a>

      <a href="#" class="btn btn-default"><%= roleLabel %></a>
      <span></span>
    </div>

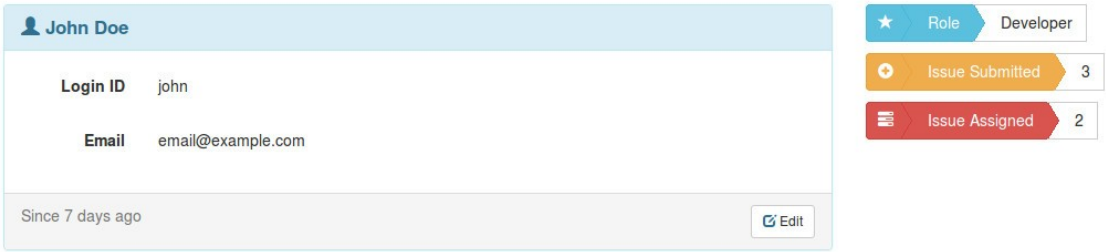
    <div class="btn-group btn-breadcrumb">
      <a href="#" class="btn btn-warning">
        <span class="glyphicon glyphicon-plus-sign"></span></a>
      <a href="#" class="btn btn-warning">Issue Submitted</a>

     <a href="#" class="btn btn-default">
        <%= submitCount %>
      </a>
      <span></span>
    </div>

    <div class="btn-group btn-breadcrumb">
      <a href="#" class="btn btn-danger">
        <span class="glyphicon glyphicon-tasks"></span></a>
      <a href="#" class="btn btn-danger">Issue Assigned</a>

      <a href="#" class="btn btn-default">
        <%= assignCount %>
      </a>
      <span></span>
    </div>
  </div>
</div>
```

သူလည်းပဲ Two Column Layout တစ်ခုဖြစ်ပြီး သက်ဆိုင်ရာ View နဲ့ချိတ်ဆက်ပြီးတဲ့အခါ ရရှိလာမယ့်ရလဒ်ကို **ပုံ (၁၄.၁၁)** မှာလေ့လာနိုင်ပါတယ်။



ပုံ (၁၄.၁၁) - User Profile Template

ဘယ်ဘက်ခြမ်းမှာ User ရဲ့ Personal Information ကိုဖော်ပြထားပြီး ညာဘက်ခြမ်းမှာတော့ User Role နဲ့အတူ အဲ့ဒီ User ထည့်သွင်းထားခဲ့တဲ့ Issue စာရင်းနဲ့ အဲ့ဒီ User ကို တာဝန်ပေးအပ်ထားတဲ့ Issue စာရင်းတို့ကို တွေ့ရမှာပဲဖြစ်ပါ တယ်။

လိုအပ်တဲ့ Template တွေ ပြည့်စုံပြီဖြစ်တဲ့အတွက် BackboneJS နဲ့ လိုအပ်တဲ့ Code တွေကို ဆက်လက်ရေးသားနိုင်ပြီ ဖြစ်ပါတယ်။ နောက်တစ်ခန်းမှာ ဆက်လက်ဖော်ပြပါမယ်။

လက်တမ်းချရေးလိုက်ရင် (၂) နာရီလောက်နဲ့ ပြီးနိုင်တဲ့
အလုပ်ကို၊ (၄) နာရီလောက် အချိန်ယူပြီး Code Structure
စနစ်ကျအောင် ပြင်ဆင်တယ်ဆိုတာ၊ ပြုပြင်ထိမ်းသိမ်းရလွယ်တဲ့
အရည်အသွေးကောင်း Code ရရှိဖို့ဖြစ်ပါတယ်။

Rockstar Developer Course

Project Management, Web Service, Server Architecture
NodeJS အစရှိသည် ဤစာအုပ်ပါ အကြောင်းအရာများကို
စာရေးသူကိုယ်တိုင် သင်ကြားပေးခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၁၅) – Building Clients (Web App)

Issue Tracking System Client App တည်ဆောက်ဖို့အတွက် လိုအပ်တဲ့ Directory Structure နဲ့ Template တွေကို ပြီးခဲ့တဲ့အခန်းမှာ တည်ဆောက်ခဲ့ပြီးဖြစ်လို့ လိုအပ်တဲ့ Backbone Model-View-Collection တွေနဲ့ JavaScript Code တွေကို ဆက်လက်ရေးသားပါမယ်။

> ဒီအခန်းမှာ ဖော်ပြမယ့် Code တွေကို ပြီးခဲ့တဲ့အခန်းမှာ ဖော်ပြခဲ့ပြီးဖြစ်တဲ့ Code တွေနဲ့ တွဲဖက်အသုံး ပြုရမှာဖြစ်ပါတယ်။ အားလုံးပါဝင်တဲ့ Code နမူနာ အပြည့်အစုံကို အောက်ပါ လိပ်စာမှာ ကြိုတင် Download ရယူထားသင့်ပါတယ်။

https://github.com/eimg/issues-backbone/archive/master.zip

ပထမဦးဆုံးအနေနဲ့၊ App တစ်ခုလုံး နဲ့သက်ဆိုင်တဲ့ Login, Logout အပါအဝင် အခြား Helper Function တွေကို `app.js` ထဲမှာ အခုလို ရေးသားပါမယ်။

JavaScript

```
var app = {
  host: "http://localhost:3000/api",

  api: function(path) {
    return app.host + path;
  },

  templateCache: {},

  loadTemplates: function(names, callback) {
    var that = this;
    var load = function(index) {
      var name = names[index];
      console.log('Loading template: ' + name);
      $.get('templates/' + name + '.html?cache=false', function(data) {
        that.templateCache[name] = data;
        index++;
        if (index < names.length) {
          load(index);
        } else {
          callback();
        }
      });
    }

    load(0);
```

```
27.   },
28.
29.   hookTemplate: function(name, data) {
30.     var tmpl = _.template(this.templateCache[name]);
31.     return tmpl(data);
32.   },
33.
34.   login: function(user, pass, ok, notok) {
35.     $.ajax({
36.       type: "post",
37.       url: this.api("/users/login"),
38.       data: {
39.         username: user,
40.         password: pass
41.       },
42.       statusCode: {
43.         400: notok,
44.         401: notok,
45.         200: ok
46.       }
47.     });
48.   },
49.
50.  logout: function(callback) {
51.     $.ajax({
52.       type: "delete",
53.       url: this.api("/users/logout"),
54.       complete: callback
55.     });
56.
57.     localStorage.removeItem("user");
58.   },
59.
60.   verify: function(ok, notok) {
61.     $.ajax({
62.       url: this.api("/users/verify"),
63.       statusCode: {
64.         401: notok,
65.         200: ok
66.       }
67.     });
68.   },
69.
70.   saveUserInfo: function(user) {
71.     localStorage.setItem("user", JSON.stringify(user));
72.   },
73.
74.   getUserInfo: function() {
75.     return JSON.parse(localStorage.getItem("user"));
76.   },
77.
78.   issueType: [],
79.   issuePriority: [],
80.   issueStatus: [],
81.   userRole: [],
82.
83.   loadIssueTypes: function() {
84.     var that = this;
```

```
    $.ajax({
      url: this.api("/issues/types"),
      success: function(data) {
        that.issueType = data;
      }
    });
  },

  loadIssuePriorities: function() {
    var that = this;
    $.ajax({
      url: this.api("/issues/priorities"),
      success: function(data) {
        that.issuePriority = data;
      }
    });
  },

  loadIssueStatuses: function() {
    var that = this;
    $.ajax({
      url: this.api("/issues/statuses"),
      success: function(data) {
        that.issueStatus = data;
      }
    });
  },

  loadUserRoles: function() {
    var that = this;
    $.ajax({
      url: this.api("/users/roles"),
      success: function(data) {
        that.userRole = data;
      }
    });
  },

  init: function() {
    this.loadTemplates(['nav', 'login', 'issue-list',
      'issue', 'issue-new', 'issue-detail', 'issue-edit',
      'user-list', 'user', 'user-profile', 'user-edit', 'user-new'
    ], function() {
      new appView();
    });
  }
}

$.ajaxPrefilter( function( options, originalOptions, jqXHR ) {
  options.crossDomain = {
    crossDomain: true
  };

  options.xhrFields = {
    withCredentials: true
  };
});
```

လိုင်းနံပါတ် (၂) မှာ API Server Location ကို host Property နဲ့ ကြေငြာထားပါတယ်။ လိုင်းနံပါတ် (၄) မှာတော့ ပေးလိုက်တဲ့ URI ကို API Server Location နဲ့ တွဲဖက်ပြီးပြန်ပေးတဲ့ `api()` Function ကိုကြေငြာထားပါတယ်။ လိုင်းနံ ပါတ် (၈) က `templateCache` ဆိုတဲ့ Property အလွတ်တစ်ခုကြေငြာထားပါတယ်။ လိုင်းနံပါတ် (၁၀) က `loadTemplates()` Function ကတော့ ပြီးခဲ့တဲ့အခန်းမှာ ရေးသားခဲ့တဲ့ Template တွေကိုရယူပြီး `template` Cache Property ထဲမှာ စုစည်းထားပေးမှာဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၂၉) က `hookTemplate()` Function ကတော့ ပေးလိုက်တဲ့ Data နဲ့ Template ကို UnderscoreJS Template Function သုံးပြီး Compile လုပ်ပေးမှာ ဖြစ်ပါတယ်။

ဆက်လက်ရေးသားထားတဲ့ `login()`, `logout()`, `verify()` Function တွေကတော့ API Server ထံ Login, Logout နဲ့ Verify Request တွေ ပေးဖို့အတွက်ဖြစ်ပါတယ်။ Login အောင်မြင်တဲ့အခါ API Server က User Data တွေကို ပြန်လည်ပေးပို့မှာဖြစ်ပါတယ်။ လက်ခံရရှိတဲ့ User Data တွေကို HTML5 LocalStorage နဲ့ သိမ်းဆည်းခြင်း၊ ပြန်လည်ရယူခြင်းလုပ်ငန်းများအတွက် `saveUserInfo()` နဲ့ `getUserInfo()` တို့ကို ဆက်လက်ရေးသား ထားပါတယ်။

ဆက်လက်ပြီး လိုင်းနံပါတ် (၈၁) ကနေစတင်ပြီး `loadIssueTypes()`, `loadIssuePriorities()`, `loadIssueStatuses()` နဲ့ `loadUserRoles()` Function တို့ကို အသီးသီးသတ်မှတ်ထားပါတယ်။ ဒီ Function တွေက API Server ထံ Request တွေပေးပို့ပြီး ပြန်လည်လက်ခံရရှိတဲ့ Issue Types, Priorities, Statuses နဲ့ User Roles တို့ကို သက်ဆိုင်ရာ Property ထဲမှာ ထည့်သွင်းသိမ်းဆည်းထားပေးထားမှာ ဖြစ်ပါ တယ်။

`app` Object ရဲ့ နောက်ဆုံး အနေနဲ့ လိုင်းနံပါတ် (၁၂၄) မှာသတ်မှတ်ထားတဲ့ `init()` Function ကတော့ `loadTemplates()` Function အကူအညီနဲ့ Template ဖိုင်တွေအားလုံးကို ရယူပေးပါတယ်။ Template ရယူခြင်း လုပ်ငန်းပြီးစီးတဲ့အခါ `appView()` Object တစ်ခု တည်ဆောက်ပေးခြင်းအားဖြင့် App ကို စတင်ပေးမှာဖြစ်ပါ တယ်။ တနည်းအားဖြင့် App ကို စတင်အလုပ်လုပ်စေဖို့ အတွက် `app.init()` ကို ခေါ်ယူပေးရမှာဖြစ်ပါ တယ်။

ပုံမှန်အားဖြင့် Client App ကနေ Back-end API Service ကို Cross-Domain Request တွေပြုလုပ်တဲ့အခါ CORS Header တွေကို Request နဲ့အတူ မှန်ကန်အောင်ထည့်သွင်းပေးပို့ဖို့လိုပါတယ်။ ဒါပေမယ့်လက်တွေ့မှာ ဒီ Header တွေကို တစ်ခုချင်း ကိုယ်တိုင်လိုက်ထည့်နေစရာမလိုပါဘူး။ jQuery ရဲ့ Ajax Options မှာ `crossDomain: true` နဲ့ `withCredentials: true` လို့သတ်မှတ်ပေးလိုက်ရင် jQuery က လိုအပ်တဲ့ CORS Header တွေကို အလို အလျောက် ထည့်သွင်းပေးသွားမှာဖြစ်ပါတယ်။ ဒီ Options တွေကို Ajax Request ပြုလုပ်တိုင်း ထည့်ပေးနေစရာ မလိုအောင် `$.ajaxPrefilter()` နဲ့ ကြိုတင်သတ်မှတ်ထားနိုင်ပါတယ်။ `$.ajaxPrefilter()` မှာ သတ်မှတ် ထားတဲ့ Options တွေကို jQuery က Ajax Request တိုင်းမှာ အလို အလျောက် ထည့်သွင်းပေးသွားမှာဖြစ်ပါတယ်။ ဒါကြောင့် လိုင်းနံပါတ် (၁၃၄) မှာ `$.ajaxPrefilter()` Function ကို သတ်မှတ်ပေးထားခြင်းဖြစ်ပါတယ်။

15.1 – Issue Model and Collection – `issues.js`

ဆက်လက်ပြီး `issues.js` ထဲမှာ Issue Model နဲ့ Collection တို့ကို အခုလိုရေးသားပေးပါမယ်။

JavaScript

```
app.issueModel = Backbone.Model.extend({
  urlRoot: app.api("/issues"),
  defaults: {
    "assignedTo" : null,
    "detail" : null,
    "priority" : 0,
    "priorityLabel" : "Low",
    "status" : 0,
    "statusLabel" : "New",
    "summary" : null,
    "type" : 0,
    "typeLabel" : "Bug"
  },

  idAttribute: "_id"
});

app.issueCollection = Backbone.Collection.extend({
  model: app.issueModel,
  url: app.api("/issues")
});

app.issueList = new app.issueCollection();
```

`issueModel` ကို `app` Object ရဲ့ Property အနေနဲ့ ကြေငြာထားတာကို သတိပြုပါ။ အလားတူပဲ လိုင်းနံပါတ် (၁၈) မှာ ကြေငြာထားတဲ့ `issueCollection` နဲ့ လိုင်းနံပါတ် (၂၃) မှာ ကြေငြာထားတဲ့ Collection Object တို့ကိုလည်း `app` Object ရဲ့ Property အနေနဲ့ပဲ ကြေငြာထားပါတယ်။ ဒီနည်းနဲ့ Namespace တစ်ခုကွဲသွားတဲ့အတွက် အခြား တစ်နေရာမှာ `issueModel`, `issueList` စတဲ့ Variable တွေလိုအပ်လို့ ထပ်မံကြေငြာအသုံးပြုလိုတဲ့အခါ နာမည်တိုက်နေမှာကို စိတ်ပူစရာမလိုပဲ လွပ်လွပ်လပ်လပ် ကြေငြာအသုံးပြုနိုင်မှာဖြစ်ပါတယ်။

`issueModel` ရဲ့ URL Root ကို `app.api()` Function ရဲ့ အကူအညီနဲ့ http://localhost:3000/api/issues လို့ သတ်မှတ်ပေးထားပါတယ်။ ဒါကြောင့် ဒီ Model ပေါ်မှာ `save()`, `destroy()` စတဲ့လုပ်ငန်း တွေအတွက် BackboneJS က Data Sync Request တွေပေးပို့တဲ့အခါ http://localhost:3000/api/issues ကို ပေးပို့သွားမှာဖြစ်ပါတယ်။

Model ID အနေနဲ့ Property တွေထဲက `_id` ကို အသုံးပြုဖို့ သတ်မှတ်ပေးထားတာကိုလည်းသတိပြုပါ။ သီးခြား ID အသုံးပြုစရာမလိုပဲ Issue တစ်ခုချင်းစီကို MongoDB Database ထဲမှာ သိမ်းလိုက်ရင်ရရှိလာမယ့် `_id` ကို Model ID အဖြစ် အသုံးချလိုက်ခြင်းဖြစ်ပါတယ်။

ဆက်လက်ပြီး လိုင်းနံပါတ် (၁၈) မှာကြေငြာထားတဲ့ `issueCollection` အတွက် URL ကိုလည်း

`app.api()` Function အကူအညီနဲ့သတ်မှတ်ထားပါတယ်။ ဒါကြောင့် ဒီ Collection ပေါ်မှာပြုလုပ်တဲ့ `fetch()`, `remove()` စတဲ့လုပ်ငန်းတွေ အတွက် Data Sync Request တွေပေးပို့တဲ့အခါမှာလည်း http://localhost:3000/api/issues ကိုပဲ ပေးပို့သွားမှာ ဖြစ်ပါတယ်။

15.2 – User Model and Collection – `users.js`

ဆက်လက်ပြီး `users.js` ထဲမှာ User Model နဲ့ Collection တို့ကို အခုလိုရေးသားပေးပါမယ်။

JavaScript

```
app.userModel = Backbone.Model.extend({
  urlRoot: app.api("/users"),
  idAttribute: "_id"
});

app.userCollection = Backbone.Collection.extend({
  model: app.userModel,
  url: app.api("/users")
});

app.userList = new app.userCollection();
```

ဒီနေရာမှာလည်း Model နဲ့ Collection တို့အတွက် Back-end Server အဖြစ် http://localhost:3000/api/users ကို သတ်မှတ်ပေးထားပါတယ်။

လိုအပ်တဲ့ Model နဲ့ Collection တွေရရှိပြီမို့ View တွေဆက်လက်တည်ဆောက်ပါမယ်။

15.3 – Navigation Menu View – `views/navView.js`

ပထမဦးဆုံးအနေနဲ့ Navigation Menu View တစ်ခုတည်ဆောက်ပါမယ်။ Navigation ဟာ ပုံမှန်အားဖြင့် View တစ်ခု ဖြစ်ဖို့မလိုပဲ Static HTML အနေနဲ့ Main Template ထဲမှာ အသေရေးထားလို့ရပါတယ်။ ဒါပေမယ့် ကျွန်တော်တို့က Navigation Menu ထဲမှာ Issue Count လို အချက်အလက်မျိုးကို ထည့်သွင်းဖော်ပြလိုတဲ့ အတွက် အသေရေးမထားပဲ အခြေအနေပေါ်မူတည်ပြီး ပြောင်းလဲဖော်ပြနိုင်စေဖို့အတွက် Backbone View အနေနဲ့ တည်ဆောက်ထားမှာဖြစ်ပါတယ်။ `views/navView.js` မှာ အခုလို ရေးသားရပါမယ်။

JavaScript

```
var navView = Backbone.View.extend({
  tagName: "div",
  className: "navbar navbar-default",

  events: {
    "click #new-issue": "showNewIssueForm",
    "click #issue-list": "showIssueList",
    "click #my-issues": "showMyIssues",
    "click #user-list": "showUserList",
    "click #profile": "showProfile",
```

```
    "click #logout": "logout"
  },

  render: function() {
    var user = app.getUserInfo();
    if( user ) {
      var myIssues = app.issueList.where({
        assignedTo: user._id
      });

      var data = {
        auth: true,
        userName: user.fullName,
        issueCount: app.issueList.length,
        myIssues: myIssues.length
      };
    } else {
      data = {
        auth: false
      }
    }

    this.$el.html( app.hookTemplate("nav", data) );
    return this;
  },

  showNewIssueForm: function(e) {
    $(".nav li").removeClass('active');
    $(e.currentTarget).parent().addClass('active');

    var form = new issueNewView();
    $("#main").html( form.render().el );
  },

  showIssueList: function(e) {
    $(".nav li").removeClass('active');
    $(e.currentTarget).parent().addClass('active');

    var list = new issueListView();
    $("#main").html( list.render().el );
  },

  showMyIssues: function(e) {
    $(".nav li").removeClass('active');
    $(e.currentTarget).parent().addClass('active');

    var list = new myIssueListView();
    $("#main").html( list.render().el );
  },

  showUserList: function(e) {
    $(".nav li").removeClass('active');
    $(e.currentTarget).parent().addClass('active');

    var users = new userListView();
    $("#main").html( users.render().el );
  },

  showProfile: function(e) {
```

```
70.       $(".nav li").removeClass('active');
71.       $(e.currentTarget).parent().addClass('active');
72.
73.       var user = new app.userModel(app.getUserInfo());
74.       var profile = new userProfileView({ model: user });
75.       $("#main").html( profile.render().el );
76.     },
77.
78.     logout: function() {
79.       var that = this;
80.       app.logout(function() {
81.         var login = new loginView();
82.         $("#main").html( login.render().el );
83.
84.         that.render();
85.       });
86.     }
87. });
```

ဒီ View ရဲ့ အဓိကလုပ်ငန်းတာဝန်ကတော့ နှိပ်လိုက်တဲ့ Menu Item ပေါ်မူတည်ပြီး သင့်တော်တဲ့ Main Content View ကို ဖော်ပြပေးဖို့ပဲဖြစ်ပါတယ်။ All Issue Menu ကို နှိပ်လိုက်ရင် Main Content အနေနဲ့ Issue List View ကို ဖော်ပြ ပေးပြီး၊ Users Menu ကို နှိပ်လိုက်ရင် User List View ကို ဖော်ပြပေးတဲ့သဘောမျိုးဖြစ်ပါတယ်။

ဒီနေရာမှာ ပိုပြီးသဘာဝကျတဲ့ နည်းလမ်းကတော့ Backbone Router ကိုအသုံးပြုခြင်းဖြစ်ပါတယ်။ Router ရဲ့ URL Pattern ပေါ်မူ တည်ပြီး သင့်တော်တဲ့ Main Content View ကို ဖော်ပြသင့်ပါတယ်။ ဥပမာ – URL က `#/issue-list` ဆိုရင် Issue List View ကိုပြပြီး URL က `#/user-list` ဆိုရင် User List View ကိုပြအောင် Backbone Router နဲ့ရေးသားထားနိုင်ပါတယ်။ ဒါဆိုရင် Menu Item တွေအတွက် သက်ဆိုင်ရာ Link သတ်မှတ်ပေးထားလိုက်ယုံနဲ့ နှိပ်လိုက်တဲ့ Link ပေါ်မူတည်ပြီး View လည်း ပြောင်းလဲသွားမှာဖြစ်ပါတယ်။ ဒါပေမယ့် ကျွန်တော်တို့က Backbone Router ကို ထည့်သွင်း ဖော်ပြမထားတဲ့အတွက် Router မသုံးပဲ Navigation View ရဲ့ Event အနေနဲ့ပဲ ဘယ် Menu Item ကိုနှိပ်ရင် ဘယ် Main Content View ကို ဖော်ပြရမလဲဆိုတာကို Event အနေနဲ့ သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၁၄) က `render()` ကိုလေ့လာကြည့်ရင်၊ `app.getUserInfo()` အကူအညီနဲ့ User Information ကို ရယူထားပါတယ်။ ပုံမှန်အားဖြင့် Login ဝင်ထားမှသာ User Information က ရှိမှာပါ။ ဒါကြောင့် User Information ရှိနေရင် Nav View အတွက် Data အနေနဲ့ `auth: true` တန်ဖိုးကို ထည့်သွင်းပေးထားပါတယ်။ Navigation Menu အတွက် Template ရေးသားစဉ်က `auth: true` ဖြစ်မှသာ Menu Item တွေကို ဖော်ပြဖို့ ထည့်သွင်းရေး သားထားပြီးဖြစ်ပါတယ်။ ပြီးခဲ့တဲ့အခန်းမှာ ရေးသားခဲ့တဲ့ tempates/nav.html မှာ ပြန်လည်လေ့လာနိုင်ပါတယ်။

ပြီးတဲ့အခါ Menu ထဲမှာ ထည့်သွင်းဖော်ပြဖို့လိုတဲ့ User Name, Issue Count, My Issue Count တန်ဖိုးတွေကိုလည်း Data အဖြစ် တွဲဖက်ထည့်သွင်းပေးထားပါတယ်။ User Name ကို `app.getUserInfo()` ကနေရရှိပြီး Issue Count ကိုတော့ Issue Collection ရဲ့ Length ကို ရယူခြင်းအားဖြင့် ရရှိနိုင်ပါတယ်။ My Issue Count အနေနဲ့ Issue Collection ထဲက Issue တွေရဲ့ Assigned To Field တန်ဖိုးကို User ID နဲ့ Filter လုပ်ယူခြင်းအားဖြင့် ရရှိ

ပါတယ်။ Filter လုပ်တဲ့လုပ်ငန်းကို လိုင်းနံပါတ် (၁၇) မှာ ရေးသားထားပါတယ်။

ဒီ View မှာပါဝင်တဲ့ ကျန် Function တွေကတော့ သက်ဆိုင်ရာ Main View ကို Render လုပ်ပေးခြင်းဖြစ်ပါတယ်။ အသေးစိတ်ကို ဒီနေရာမှာ ရေးစရာမလိုပါဘူး။ သက်ဆိုင်ရာ View အတွင်းမှာ ရေးသားမှာဖြစ်ပါတယ်။ နမူနာအနေနဲ့ လိုင်းနံပါတ် (၄၅) မှာရေးသားထားတဲ့ `showIssueList()` ကိုလေ့လာကြည့်နိုင်ပါတယ်။ jQuery ကို အသုံးပြုပြီး အခြား Menu Item တွေမှာ ရှိနေတဲ့ `.active` Class ကိုဖြုတ်ပြီး လက်ရှိ Menu Item အတွက် `.active` Class ကို ထည့်သွင်းထားပါတယ်။ ပြီးတဲ့အခါ `issueListView()` Object တစ်ခု တည်ဆောက်ပြီး၊ အဲ့ဒီ Object ရဲ့ `render()` Function က ပေးတဲ့ `el` (Element) ကို `#main` အတွင်းမှာ `app.hook()` Function အကူအညီနဲ့ ဖော်ပြစေလိုက်ခြင်းဖြစ်ပါတယ်။ `#main` ဆိုတာ Main Content View ကို ဖော်ပြဖို့အတွက် `index.html` အတွင်း မှာ ကြိုတင်သတ်မှတ်ထားတဲ့ Element ဖြစ်ပါတယ်။ ဒီ အကြောင်းကို မကြာခင်ဆက်လက်ဖော်ပြပါမယ်။

15.4 – Login View – `views/loginView.js`

ဆက်လက်ပြီး Login Form ဖော်ပြပေးနိုင်တဲ့ Login View တစ်ခုတည်ဆောက်ပါမယ်။

JavaScript

```
var loginView = Backbone.View.extend({
  tagName: "div",

  events: {
    "submit #login-form": "login"
  },

  initialize: function() {
    //
  },

  render: function() {
    this.$el.html( app.hookTemplate("login") );
    return this;
  },

  login: function() {
    var loginId = $("#login-id").val();
    var password = $("#password").val();

    app.login(loginId, password, function(user) {

      app.saveUserInfo(user);

      app.issueList.fetch({
        success: function() {
          var list = new issueListView();
          $("#main").html( list.render().el );

          var nav = new navView();
          $("#nav").html( nav.render().el );

          app.userList.fetch();
```

```
34.             app.loadIssueTypes();
35.             app.loadIssuePriorities();
36.             app.loadIssueStatuses();
37.             app.loadUserRoles();
38.           }
39.         });
40.
41.       }, function() {
42.         $("#login-alert").show();
43.       });
44.
45.       return false;
46.     }
47. });
```

Login View ရဲ့ တာဝန်က Login Form ကို ဖော်ပြယုံသာမကပဲ၊ Login လုပ်ငန်းစဉ်ကိုပါ ဆောင်ရွက်ပေးဖို့ဖြစ်ပါတယ်။ လိုင်းနံပါတ် (၁၂) က `render()` Function က `app.hook()` Function အကူအညီနဲ့ ကြိုတင် ရေးသားထားတဲ့ Login Template ကို အသုံးပြုပြီး Login Form ကို ဖော်ပြစေပါတယ်။ လိုင်းနံပါတ် (၅) မှာ Event အနေနဲ့ Login Button ကို နှိပ်ရင် `login()` Function ကို အလုပ်လုပ်ပေးဖို့ သတ်မှတ်ထားပါတယ်။

`login()` Function က Form မှာ ရိုက်ထည့်လိုက်တဲ့ Login ID နဲ့ Password ကို ရယူပြီး ကြိုတင်ရေးသားထား တဲ့ `app.login()` ကို အသုံးပြုပြီး Login ဝင်ပေးပါတယ်။ Login Success ဖြစ်တဲ့အခါ `app .saveUserInfo()` နဲ့ User Information ကို သိမ်းပေးပါတယ်။ ပြီးတဲ့အခါ `app.issueList` Collection ပေါ်မှာ `fetch()` ကို Run ခြင်းအားဖြင့် API Server က Issue List ကို ရယူပေးပါတယ်။ `issueList` Collection မှာ URL ကို localhost:3000/api/issues/ လို့ သတ်မှတ်ထားတဲ့အတွက် `fetch()` Function ကို Run တဲ့အခါ Backbone က localhost:3000/api/issues/ ကို GET Request တစ်ခုပေးပို့သွားမှာဖြစ်ပါတယ်။ ကျွန်တော်တို့က API Server မှာ `api/issues` URI ကို GET Request လက်ခံရရှိတဲ့အခါ Issue List ကို ပြန် ပေးဖို့ ကြိုတင်ရေးသားထားပြီးဖြစ်တဲ့အတွက် API Server က Issue List ကို ပြန်ပေး ပါလိမ့်မယ်။ Backbone က လက်ခံရရှိတဲ့ Issue List ကို Collection ထဲမှာ ထည့်သွင်းပေးသွားမှာဖြစ်ပါတယ်။

ဆိုလိုတာက Server ကနေ Issue List ကို ရရှိဖို့ Request ပြုလုပ်တဲ့ကိစ္စတွေ၊ ရရှိလာတဲ့ ရလဒ်ကို Format မှန် မ မှန် စီစစ်ပြီး Collection ထဲထည့်သွင်းတဲ့ကိစ္စတွေကို ကိုယ်တိုင်လုပ်စရာမလိုပဲ Collection ပေါ်မှာ `fetch()` Function ကို Run ပေးလိုက်ယုံနဲ့ Server က ပြန်ပေးတဲ့ List က Collection ထဲမှာ အလိုအလျောက်ရောက်ရှိ သွားအောင် Backbone က နောက်ကွယ်ကနေ ဆောင်ရွက်ပေးသွားခြင်းပဲဖြစ်ပါတယ်။

Issue List ကို ရရှိပြီးတဲ့အခါ Nav View ကို `#nav` Element အတွင်းမှာ Render လုပ်ပေးပြီး Issue List View ကို `#main` Element အတွင်းမှာ Render လုပ်ပေးထားပါတယ်။ ဆက်လက်ပြီး Issue Type, Priority, Status နဲ့ User List, User Role တွေကို Load လုပ်ပြီး သက်ဆိုင်ရာ Property အတွင်းမှာ သိမ်းဆည်းပေးပါသေးတယ်။

ဒီနည်းအတိုင်း Login Success ဖြစ်သွားတာနဲ့ လိုအပ်တဲ့ Data အားလုံးကို တန်းစီရယူပေးသွားမှာဖြစ်လို့၊ နောက်အဆင့်တွေမှာ ဒီ Data တွေကို အသင့်အသုံးပြုနိုင်မှာပဲဖြစ်ပါတယ်။ Login Fail ဖြစ်ရင်တော့ Template ထဲမှာ ကြိုတင်ရေး သားထားတဲ့ `#login-alert` ကို ဖော်ပြပေးမှာဖြစ်ပါတယ်။

15.5 – Issue List View – `views/issueListView.js`

ဆက်လက်ပြီး Issue List View တစ်ခုကို တည်ဆောက်ပါမယ်။

JavaScript

```
var issueListView = Backbone.View.extend({
  tagName: "div",

  initialize: function() {
    //
  },

  render: function() {
    this.$el.html( app.hookTemplate("issue-list") );

    var that = this;
    app.issueList.each(function(issue) {
      var view = new issueView({ model: issue });
      $("tbody", that.$el).append( view.render().el );
    });

    return this;
  }
});

var myIssueListView = Backbone.View.extend({
  tagName: "div",

  initialize: function() {
    //
  },

  render: function() {
    this.$el.html( app.hookTemplate("issue-list") );

    var that = this;
    var issues = app.issueList.where({
      assignedTo: app.getUserInfo()._id
    });

    _.each(issues, function(issue) {
      var view = new issueView({ model: issue });
      $("tbody", that.$el).append( view.render().el );
    });

    return this;
  }
});

var issueView = Backbone.View.extend({
  tagName: "tr",

  events: {
    "click .issue-summary": "viewDetail"
  },

```

```
52.   initialize: function() {
53.     //
54.   },
55.
56.   render: function() {
57.     var index = app.issueList.indexOf(this.model);
58.     this.model.set("index", index + 1);
59.
60.     this.$el.html( app.hookTemplate("issue", this.model.toJSON() ) );
61.     return this;
62.   },
63.
64.   viewDetail: function(e) {
65.     var detail = new issueDetailView({model: this.model});
66.     $("#main").html( detail.render().el );
67.   }
68. });
```

ဒီဖိုင်ထဲမှာ View (၃) ခုပါဝင်ပါတယ်။ Issue List Table View, My Issue List Table View နဲ့ Table တွေထဲက Row တစ်ခုချင်းအတွက်သတ်မှတ်ထားတဲ့ Issue View တို့ပဲဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၄၅) မှာ သတ်မှတ်ထားတဲ့ Issue View မှာ `app.hookTemplate()` အကူအညီနဲ့ ကြိုတင်ရေးသား ထားတဲ့ Issue Template ကို Render လုပ်ထားပါတယ်။ Event အနေနဲ့ Issue Summary ကိုနှိပ်ရင် Issue Detail View ကို ဖော်ပြပေးဖို့ သတ်မှတ်ထားပါတယ်။

Issue List View နဲ့ My Issue List View တို့ကတော့ အလုပ်လုပ်ပုံအတူတူပါပဲ။ Issue List Collection ကို Loop လုပ်ပြီး Table အတွင်းမှာ Issue View တွေကို Row အနေနဲ့ တန်းစီထည့်သွင်းပေးလိုက်ခြင်းပဲဖြစ်ပါတယ်။ Issue List View အတွက် Issue List Collection တစ်ခုလုံးကိုသုံးပြီး၊ My Issue List View အတွက်တော့ Login လုပ်ထားတဲ့ User ရဲ့ ID နဲ့ Issue ရဲ့ Assigned To Field တို့ကိုက်ညီတဲ့ Issue စာရင်းကိုသာ အသုံးပြုထားပါတယ်။

15.6 – Issue Detail View – `views/issueDetailView.js`

ဆက်လက်ပြီး Issue Summary ကို နှိပ်လိုက်တဲ့အခါ ဖော်ပြမယ့် Issue Detail အတွက် View တစ်ခုကို တည်ဆောက်ပါမယ်။

JavaScript

```
1. var issueDetailView = Backbone.View.extend({
2.   tagName: "div",
3.
4.   events: {
5.     "click #delete-issue": "deleteIssue",
6.     "click #close-issue": "closeIssue",
7.     "click #edit-issue": "editIssue",
8.     "click .change-type": "changeType",
9.     "click .change-priority": "changePriority",
10.    "click .change-assign": "changeAssign",
```

```
    "click .change-status": "changeStatus",
    "click #post-comment": "postComment",
    "click .delete-comment": "deleteComment"
  },

  initialize: function() {
    //
  },

   render: function() {
    this.model.set("users", app.userList.toJSON());
    this.$el.html(app.hookTemplate("issue-detail",this.model.toJSON()));
    return this;
  },

  deleteIssue: function() {
    var urlRoot = this.model.urlRoot;
    var id = this.model.id;
    this.model.url = function() {
      return urlRoot + "/" + id;
    };

    this.model.destroy({
      wait: true,
      success: function() {
        var list = new issueListView();
        $("#main").html( list.render().el );
      }
    });
  },

  closeIssue: function() {
    var urlRoot = this.model.urlRoot;
    var id = this.model.id;
    this.model.url = function() {
      return urlRoot + "/status/" + id;
    };

    this.model.save({
      status: 4,
      statusLabel: app.issueStatus[4]
     }, {
      patch: true,
      wait: true,
      success: function() {
        var list = new issueListView();
        $("#main").html( list.render().el );
      }
    });
  },

  editIssue: function() {
    var edit = new issueEditView({ model: this.model });
    $("#main").html( edit.render().el );
  },

  changeType: function(e) {
    var type = $(e.currentTarget).data("value");
    var urlRoot = this.model.urlRoot;
```

```
71.     var id = this.model.id;
72.     var that = this;
73.
74.     this.model.url = function() {
75.       return urlRoot + "/type/" + id;
76.     };
77.
78.     this.model.save({
79.       type: type,
80.       typeLabel: app.issueType[type]
81.     }, {
82.       patch: true,
83.       wait: true,
84.       success: function() {
85.         var detail = new issueDetailView({ model: that.model });
86.         $("#main").html( detail.render().el );
87.       }
88.     });
89.   },
90.
91.   changePriority: function(e) {
92.     var priority = $(e.currentTarget).data("value");
93.     var urlRoot = this.model.urlRoot;
94.     var id = this.model.id;
95.     var that = this;
96.
97.     this.model.url = function() {
98.       return urlRoot + "/priority/" + id;
99.     };
100.
101.    this.model.save({
102.      priority: priority,
103.      priorityLabel: app.issuePriority[priority]
104.    }, {
105.      patch: true,
106.      wait: true,
107.      success: function() {
108.        var detail = new issueDetailView({ model: that.model });
109.        $("#main").html( detail.render().el );
110.      }
111.    });
112.  },
113.
114.  changeStatus: function(e) {
115.    var status = $(e.currentTarget).data("value");
116.    var urlRoot = this.model.urlRoot;
117.    var id = this.model.id;
118.    var that = this;
119.
120.    this.model.url = function() {
121.      return urlRoot + "/status/" + id;
123.    };
124.
125.    this.model.save({
126.      status: status,
127.      statusLabel: app.issueStatus[status]
128.    }, {
129.      patch: true,
130.      wait: true,
```

```
131.       success: function() {
132.         var detail = new issueDetailView({ model: that.model });
133.         $("#main").html( detail.render().el );
134.       }
135.     });
136.   },
137.
138.   changeAssign: function(e) {
139.     var assign = $(e.currentTarget).data("value");
140.     var label = $(e.currentTarget).html();
141.
142.     var urlRoot = this.model.urlRoot;
143.     var id = this.model.id;
144.     var that = this;
145.
146.     this.model.url = function() {
147.       return urlRoot + "/assign/" + id;
148.     };
149.
150.     this.model.save({
151.       assignedTo: assign,
152.       assignedToLabel: label
153.     }, {
154.       patch: true,
155.       wait: true,
156.       success: function() {
157.         var detail = new issueDetailView({ model: that.model });
158.         $("#main").html( detail.render().el );
159.       }
160.     });
161.   },
162.
163.   postComment: function() {
164.     var comment = $("#comment").val();
165.     if(!comment) return;
166.
167.     var user = app.getUserInfo();
168.     var authorId = user._id;
169.     var authorName = user.fullName;
170.
171.     var newComment = {
172.       comment: comment,
173.       authorId: authorId,
174.       authorName: authorName,
175.       submittedAt: new Date(),
176.       issueId: this.model.id
177.     }
178.
179.     var comments = this.model.get("comments") || [];
180.     comments.push(newComment);
181.
182.     var urlRoot = this.model.urlRoot;
183.     var that = this;
184.     $.post(urlRoot + "/comments", newComment, function() {
185.       var detail = new issueDetailView({ model: that.model });
186.       $("#main").html( detail.render().el );
187.     });
188.   },
189.
```

```
190.    deleteComment: function(e) {
191.      var index = $(e.currentTarget).data("index");
192.
193.      var comments = this.model.get("comments") || [];
194.      comments.splice(index, 1);
195.      this.model.set("comments", comments);
196.
197.      var urlRoot = this.model.urlRoot;
198.      var that = this;
199.      $.ajax({
200.        type: "delete",
201.        url: urlRoot + "/comments",
202.        data: {
203.          issueId: that.model.id,
204.          commentId: index
205.        },
206.        complete: function() {
207.          var detail = new issueDetailView({ model: that.model });
208.          $("#main").html( detail.render().el );
209.        }
210.      });
211.    }
212. });
```

ဒီ View မှာတော့ ပါဝင်ဝင်တဲ့လုပ်ဆောင်ချက် နည်းနည်းများပါတယ်။ Issue ကို Delete လုပ်နိုင်တဲ့ လုပ်ဆောင်ချက်တွေ၊ Issue ကို Close လုပ်လို့ရတဲ့ လုပ်ဆောင်ချက်တွေ၊ Issue Status, Priority, Assigned To တန်ဖိုးတွေကို ပြောင်းလို့ ရတဲ့လုပ်ဆောင်ချက်တွေနဲ့ Comments လုပ်ဆောင်ချက်တွေ ပါဝင်ပါတယ်။

လိုင်းနံပါတ် (၂၆) က `deleteIssue()` ကိုလေ့လာကြည့်ပါ။ Issue Model တည်ဆောက်ခဲ့စဉ်က Model URL ကို ထည့်သွင်းသတ်မှတ်ထားခဲ့ပါတယ်။ ဒါပေမယ့် URL Structure က ဆောင်ရွက်လိုတဲ့ လုပ်ဆောင်ချက်ပေါ် မူတည်ပြီး ပြောင်းလဲမှာဖြစ်တဲ့အတွက် Default URL ကို မသုံးပဲ Model URL တစ်ခုကို အရင်တည်ဆောက်ပါ တယ်။ ဥပမာ – Issue Delete လုပ်လိုတဲ့အခါ အသုံးပြုရမယ့် URL က localhost:3000/api/issues/:id ဖြစ်ပေ မယ့် Issue Status ကို ပြောင်းလိုတဲ့အခါ အသုံးပြုရမယ့် URL က localhost:3000/api/issues/status/:id ဖြစ်ပါ တယ်။ Action အားလုံးအတွက် URL Pattern တစ်ခုတည်းကို သုံးလို့မရတော့ပဲ URL Pattern ကို လိုသလို သတ်မှတ်အသုံးပြုပေးဖို့ လိုတဲ့သဘောဖြစ်ပါတယ်။

Delete လုပ်ဆောင်ချက်ဖြစ်တဲ့အတွက် Model ပေါ်မှာ `destroy()` Function ကို Run ထားပါတယ်။ Destroy Function က Model ကို ပယ်ဖျက်ပေးလိုက်တဲ့အပြင် Server ထံကိုလည်း DELETE Request ပေးပို့သွားမှာဖြစ်ပါ တယ်။ ဒါကြောင့် `destroy()` Function Run လိုက်တာနဲ့ လက်ရှိ Model Client-side မှာပျက်သွားသလို၊ Server-side မှာလည်း ပယ်ဖျက်ပြီးဖြစ်သွားမှာပဲ ဖြစ်ပါတယ်။

သတိပြုသင့်တာကတော့၊ လိုင်းနံပါတ် (၃၄) က `wait: true` ဖြစ်ပါတယ်။ ဒီသတ်မှတ်ချက်ကြောင့် Backbone က Model ကို ချက်ခြင်းပယ်ဖျက်မပြစ်ပဲ Server-side ကနေ Success Respond လက်ခံရရှိမှာ ပယ်ဖျက်ပေးမှာဖြစ် ပါတယ်။ တနည်းအားဖြင့် Model က Client-side မှာပျက်သွားပေမယ့် Server-side မှာ အမှားအယွင်းတစ်ခုကြောင့် ကျန်သွားတယ်ဆိုတာမျိုး မဖြစ်နိုင်တော့ပါဘူး။ Server-side မှာ အမှန်တစ်ကယ်

ပျက်သွားမှ Backbone က Client-side မှာ ပယ်ဖျက်ပေးစေလိုတဲ့အတွက် `wait: true` ကို ထည့်သွင်းထားရခြင်းပဲဖြစ်ပါတယ်။

နောက်ထပ်နမူနာအနေနဲ့ လိုင်းနံပါတ် (၄၂) က `closeIssue()` ကို လေ့လာကြည့်စေလိုပါတယ်။ Issue ကို Close လုပ်ဖို့ အတွက် Server ကို ပေးပို့ရမယ့် API URL က localhost:3000/api/issues/status/:id ဖြစ်တဲ့အတွက် URL Pattern မှာ `status` ပါဝင်သွားအောင် အရင်ပြုပြင်ထားပါတယ်။ ပြီးတဲ့အခါ လိုအပ်တဲ့တန်ဖိုးတွေ သတ်မှတ်ပြီး Model ပေါ်မှာ `save()` Function ကို Run လိုက်ပါတယ်။

`save()` Function က Model အသစ်ဆိုရင် Server ထံ POST Request တစ်ခုကို ပေးပို့ပေးပါတယ်။ မူလရှိနေတဲ့ Model ပေါ်မှာဆိုရင်တော့ Model ကို ပြုပြင်ခြင်းဖြစ်တယ်လို့ နားလည်ပြီး PUT Request တစ်ခုကို ပေးပို့ပေးပါတယ်။ Model အသစ်ဟုတ်မဟုတ်ကို Model မှာ ID Attribute (`_id`) ရှိမရှိနဲ့ဆုံးဖြတ်ပါတယ်။ Model အသစ်ဆိုရင် ID Attribute က ရှိမှာမဟုတ်ပါဘူး။ Server မှာ တစ်ကြိမ်သိမ်းဖူးတဲ့ Model မှသာ ID Attribute က ရှိနေမှာပါ။

လက်ရှိ `save()` Function ကို Run တဲ့ Model က `_id` ရှိပြီး Model ဖြစ်တဲ့အတွက် Backbone က PUT Request တစ်ခုကို Server ထံ ပေးပို့ပေးသွားမှာဖြစ်ပါတယ်။ ဒီတော့မှ Client-side မှာ Model ကို ပြုပြင်လိုက်တဲ့ တန်ဖိုးက Server-side မှာပါ သက်ရောက်သွားစေမှာဖြစ်ပါတယ်။

ပြဿနာက ကျွန်တော်တို့ရဲ့ API Server မှာ Status ပြောင်းတဲ့လုပ်ငန်းအတွက် PUT Request ကို အသုံးမပြုစေပဲ PATCH ကို အသုံးပြုဖို့ သတ်မှတ်ထားပါတယ်။ ဒါကြောင့် Backbone ကို Request Method အနေနဲ့ PATCH ကိုသုံးပေးဖို့ ပြောဖို့လိုပါတယ်။ ဒါကြောင့် လိုင်းနံပါတ် (၅၃) မှာ patch: true ဆိုတဲ့တန်ဖိုးကို ထည့်သွင်းပေးထားခြင်းပဲ ဖြစ်ပါတယ်။

ကျွန်လုပ်ဆောင်ချက်တွေကတော့ သဘာတရားခပ်ဆင်ဆင်တွေပဲမို့ စာဖတ်သူကိုယ်တိုင်ပဲ ရေးသားထားတဲ့ Code ကို ဖတ် ပြီး ဆက်လက်လေ့လာကြည့်လိုက်ပါ။

15.7 – App View – `views/appView.js`

ဆက်လက်ပြီး Main App View ကို `views/appView.js` ထဲမှာ ရေးသားပေးပါမယ်။

JavaScript

```
var appView = Backbone.View.extend({
  el: "#app",

  initialize: function() {
    this.render();
  },

  render: function() {
    var nav = new navView();
    $("#nav").html( nav.render().el );
```

```
11.
12.     app.verify(function() {
13.       app.issueList.fetch({
14.         success: function() {
15.           var list = new issueListView();
16.           $("#main").html( list.render().el );
17.
18.           var nav = new navView();
19.           $("#nav").html( nav.render().el );
20.
21.           app.userList.fetch();
22.           app.loadIssueTypes();
23.           app.loadIssuePriorities();
24.           app.loadIssueStatuses();
25.           app.loadUserRoles();
26.         }
27.       });
28.     }, function() {
29.       var login = new loginView();
30.       $("#main").html( login.render().el );
31.     });
32.   }
33. });
```

ဒီ View အတွက် Element အနေနဲ့ Main Template ဖြစ်တဲ့ `index.html` မှာ ကြိုတင်သတ်မှတ်ထားတဲ့ `#app` Element ကို အသုံးပြုထားပါတယ်။ ထူးခြားချက်အနေနဲ့ `initialize()` Function အတွင်းမှာ `render()` Function ကို ခေါ်ယူထားပါတယ်။ အဓိပ္ပါယ်က၊ ဒီ View ကိုသုံးပြီး View Object တစ်ခု တည်ဆောက်လိုက်တာနဲ့ `render()` Function က တစ်ခါတည်း အလုပ်လုပ်သွားမယ်ဆိုတဲ့ သဘောပဲဖြစ်ပါတယ်။

`render()` Function အတွင်းမှာတော့ `app.verify()` ကိုသုံးပြီး လက်ရှိ User ဟာ Authenticated User ဟုတ် မဟုတ်စီစစ်ထားပါတယ်။ Authenticated User ဆိုရင် Issue List View ကို ဖော်ပြပြီး၊ Authenticated User မဟုတ်ရင် တော့ Login View ကို ဖော်ပြဖို့ သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။

15.8 – Main Template – `index.html`

တစ်ခြား View တွေလိုအပ်ပေမယ့် App စတင်အလုပ်လုပ်ဖို့အတွက် လိုအပ်တဲ့အခြေခံ View တွေ ပါဝင်သွားပြီမို့ ကျန် View တွေကိုခဏထားပြီး Main Template ကို အရင်တည်ဆောက်ပါမယ်။ `index.html` မှာ အခုလို ရေးသား ပေးရမှာဖြစ်ပါတယ်။

HTML

```
1. <!DOCTYPE html>
2. <html lang="en">
3. <head>
4.   <meta charset="UTF-8">
5.   <title>Issue Tracking System</title>
6.   <link rel="stylesheet" href="bower_components/bootstrap/dist/
                 css/bootstrap.min.css">
7.   <link rel="stylesheet" href="bower_components/bootstrap3-wysiwyg/dist/
```

```
                                        bootstrap3-wysihtml5.min.css">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <div class="container" id="app">
    <div id="nav">

    </div>
    <div id="main">

    </div>
  </div>

  <script src="bower_components/jquery/dist/jquery.min.js"></script>
  <script src="bower_components/bootstrap/dist/js/
                bootstrap.min.js"></script>
  <script src="bower_components/bootstrap3-wysiwyg/dist/
                bootstrap3-wysihtml5.all.min.js"></script>
  <script src="bower_components/DataTables/media/js/
                jquery.dataTables.min.js"></script>
  <script src="bower_components/datatables-responsive/js/
                dataTables.responsive.js"></script>

  <script src="bower_components/underscore/underscore-min.js"></script>
  <script src="bower_components/backbone/backbone-min.js"></script>

  <script src="bower_components/moment/min/moment.min.js"></script>

  <script src="app.js"></script>
  <script src="issues.js"></script>
  <script src="users.js"></script>

  <script src="views/appView.js"></script>
  <script src="views/navView.js"></script>
  <script src="views/loginView.js"></script>
  <script src="views/issueListView.js"></script>
  <script src="views/issueNewView.js"></script>
  <script src="views/issueDetailView.js"></script>
  <script src="views/issueEditView.js"></script>
  <script src="views/userListView.js"></script>
  <script src="views/userProfileView.js"></script>
  <script src="views/userEditView.js"></script>
  <script src="views/userNewView.js"></script>

  <script>
    app.init();
  </script>
</body>
</html>
```

ထူးခြားတဲ့လုပ်ဆောင်ချက်တွေမပါဝင်ပဲ App မှာအသုံးပြုဖို့လိုတဲ့ Library ဖိုင်တွေ View ဖိုင်တွေနဲ့ Model, Collection ဖိုင်တွေကို တန်းစီချိတ်ဆက်ထားခြင်းသာဖြစ်ပါတယ်။ ထူးခြားချက်အနေနဲ့ `#app` Element အတွင်းထဲမှာ `#nav` Element အလွတ်တစ်ခုနဲ့ `#main` Element အလွတ်တစ်ခုတို့ပါဝင်တာကို သတိပြုပါ။ `#nav` Element အတွင်းမှာ Nav View ကို Render လုပ်မှာဖြစ်ပြီး `#main` Element အတွင်းမှာတော့ Main Content View ကို အခြေအနေ ပေါ်မူတည် ဖော်ပြပေးသွားမှာပဲ ဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၄၈) မှာ `app.init()` Function ကို ခေါ်ယူခြင်းအားဖြင့် App ကို အစပြုထားပါတယ်။ `app.js` ထဲမှာ ပြန်လည်လေ့လာကြည့်ရင် `app.init()` က Template ဖိုင်တွေကို Load လုပ်ပြီး App View တစ်ခု တည်ဆောက်ပေးထားတယ်ဆိုတာကို တွေ့ရမှာဖြစ်ပါတယ်။

ဒီအဆင့်ထိ ရေးသားပြီးစီးပြီဆိုရင် App ကို စတင်စမ်းသပ်လို့ရပါပြီ။ စတင်စမ်းသပ်နိုင်ဖို့အတွက် လက်ရှိ ရေးသားနေတဲ့ Client ကို Web Server တစ်ခုနဲ့ Run ပေးဖို့လိုပါတယ်။ ဒါကြောင့် `http-server` ဆိုတဲ့ NPM Package တစ်ခုကို Install လုပ်ပြီး Project Directory ထဲမှာ `http-server` Command ကို Run ပေးခြင်းအားဖြင့် စမ်းသပ်နိုင်ပါတယ်။

```
$ sudo npm install -g http-server
$ http-server

Starting up http-server, serving ./ on: http://0.0.0.0:8080
Hit CTRL-C to stop the server
```

`http-server` က Default Port အနေနဲ့ Port နံပါတ် 8080 ကို သုံးပြီးအလုပ်လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ ဒါကြောင့် အခုနေ Browser ကိုဖွင့်ပြီး localhost:8080 ကိုသွားကြည့်လိုက်ရင် လက်ရှိရေးသားနေတဲ့ Client App ကို တွေ့ရမှာပဲ ဖြစ်ပါတယ်။ စစခြင်းမှာ Login View ကိုတွေ့ရမှာ ဖြစ်ပြီး Login ဝင်ပြီးတဲ့အခါမှာတော့ Issue List View ကို ရရှိမှာ ဖြစ်ပါတယ်။

တစ်ခုသတိပြုရမှာကတော့၊ ဒီ App ဟာ ကျွန်တော်တို့ **အခန်း (၁၃)** မှာ ရေးသားထားခဲ့တဲ့ Back-end Service ကို ဆက်သွယ်အလုပ်လုပ်မှာဖြစ်လို့ အဲ့ဒီ Service ကိုလည်း Run ထားပေးဖို့လိုပါတယ်။ Back-end Service ကို ရေးသားခဲ့ စဉ်ကလက်ရှိ Client Run နေတဲ့ localhost:8080 ကို Access-Control-Allow-Origin နဲ့ Cross-domain Request ပြုလုပ်ခွင့်ကို ခွင့်ပြုထားခဲ့ပြီးဖြစ်တယ်ဆိုတာကိုလည်း သတိပြုသင့်ပါတယ်။

ကျန် View တွေကိုတော့ Code ကိုပဲအဓိကဖော်ပြပါတော့မယ်။ တစ်ခုချင်းလိုက်ရှင်းမနေတော့ပါဘူး။ စာဖတ်သူကိုယ်တိုင် Code ကိုကြည့်ပြီး အဓိပ္ပါယ်ကို နားလည်နိုင်လိမ့်မယ်လို့ ယူဆပါတယ်။

15.9 – New Issue View – `views/issueNewView.js`

Issue အသစ်ထည့်လို့ရတဲ့ New Form ကို ဖော်ပြပေးတဲ့ View ဖြစ်ပါတယ်။ Form ကို Submit လုပ်လိုက်ရင် Issue Model တစ်ခုတည်ဆောက်ပြီး အဲ့ဒီ Model ကို Issue List Collection ထဲမှာထည့်သွင်းထားပါတယ်။ ထူးခြားချက်အနေ နဲ့ လိုင်းနံပါတ် (၉) မှာ `listenTo()` ကို သုံးပြီး Issue List Collection မှာ Issue တစ်ခု ထည့်သွင်းလိုက်တဲ့အခါ Show Issue List လုပ်ဆောင်ချက်ကို လုပ်ပေးဖို့ သတ်မှတ်ထားတာကို သတိပြုသင့်ပါတယ်။

```
var issueNewView = Backbone.View.extend({
  tagName: "div",

  events: {
    "submit #new-issue-form": "create"
  },

  initialize: function() {
    this.listenTo(app.issueList, 'add', this.showIssueList);
  },

  render: function() {
    var data = {
      priorities: app.issuePriority,
      types: app.issueType
    };

    this.$el.html( app.hookTemplate("issue-new", data) );
    return this;
  },

  create: function() {
    var summaryInput = $("#summary");
    if(!summaryInput.val()) {
      summaryInput.parent().addClass('has-error');
      summaryInput.focus();
      return false;
    }

    var summary = summaryInput.val();
    var detail = $("#detail").html();
    var type = $("#type").val();
    var priority = $("#priority").val();

    var model = new app.issueModel({
      "detail" : detail,
      "summary" : summary,
      "type" : type,
      "typeLabel" : app.issueType[type],
      "priority" : priority,
      "priorityLabel" : app.issuePriority[priority]
    });

    model.save(null, {
      wait: true,
      success: function(res) {
        app.issueList.add(res);
      }
    });

    return false;
  },

  showIssueList: function(data) {
    var list = new issueListView();
    $("#main").html( list.render().el );
  }
});
```

15.10 – Edit Issue View – `views/issueEditView.js`

Issue Summary နဲ့ Detail ကို ပြင်ဆင်လို့ရတဲ့ Edit Form ကို ဖော်ပြပေးတဲ့ View ဖြစ်ပါတယ်။ Issue တစ်ခု အတွက် ပါဝင်တဲ့အချက်အလက်အများအပြားရှိပေမယ့် ဒီ Form ကိုတော့ Issue Summary နဲ့ Detail ကို ပြင်ဆင်ဖို့အတွက်သာ အသုံးပြုမှာဖြစ်ပါတယ်။ ကျန်အချက်အလက်တွေဖြစ်တဲ့ Priority, Status, Assigned To စတဲ့ အချက်အလက်တွေကို ပြင်ဆင်နိုင်တဲ့ လုပ်ဆောင်ချက်ကိုတော့ Issue Detail မှာ ထည့်သွင်းရေးသားခဲ့ပြီး ဖြစ်ပါတယ်။

```
var issueEditView = Backbone.View.extend({
  tagName: "div",

  events: {
    "submit #issue-edit-form": "updateIssue",
    "click #close-edit": "showIssueDetail"
  },

  initialize: function() {
    //
  },

  render: function() {
    this.$el.html( app.hookTemplate("issue-edit", this.model.toJSON()));
    return this;
  },

  updateIssue: function() {
    var summaryInput = $("#summary");
    if(!summaryInput.val()) {
      summaryInput.parent().addClass('has-error');
      summaryInput.focus();
      return false;
    }

    var summary = summaryInput.val();
    var detail = $("#detail").html();

    var that = this;
    this.model.save({
      "detail" : detail,
      "summary" : summary
    }, {
      wait: true,
      success: function(res) {
        that.showIssueDetail();
      }
    });

    return false;
  },

  showIssueDetail: function(data) {
    var detail = new issueDetailView({ model: this.model });
    $("#main").html( detail.render().el );
  }
});
```

15.11 – User List View – `views/userListView.js`

User စာရင်းကို ဖော်ပြပေးတဲ့ View ဖြစ်ပါတယ်။ ထူးခြားချက်အနေနဲ့ Delete လုပ်ဆောင်ချက်ကို List နဲ့အတူ တစ်ခါ တည်း တွဲဖက်သတ်မှတ်ထားပါတယ်။ ဒါကြောင့် User တစ်ယောက်ကို ပယ်ဖျက်လိုရင် သက်ဆိုင်ရာ User နဲ့အတူ တွဲဖက်ဖော်ပြထားတဲ့ Delete Button ကို နှိပ်ပြီး ပယ်ဖျက်နိုင်မှာ ဖြစ်ပါတယ်။

JavaScript

```
var userView = Backbone.View.extend({
  tagName: "tr",

  events: {
    "click .username": "viewProfile",
    "click .edit-user": "editUser",
    "click .delete-user": "deleteUser"
  },

  render: function() {
    var index = app.userList.indexOf(this.model);
    this.model.set("index", index + 1);

    this.$el.html( app.hookTemplate("user", this.model.toJSON()) );
    return this;
  },

  viewProfile: function() {
    var profile = new userProfileView({ model: this.model });
    $("#main").html( profile.render().el );
  },

  editUser: function() {
    var user = new userEditView({ model: this.model });
    $("#main").html( user.render().el );
  },

  deleteUser: function() {
    this.model.destroy({
      wait: true,
      success: function() {
        var users = new userListView();
        $("#main").html( users.render().el );
      }
    });
  }
});

var userListView = Backbone.View.extend({
  tagName: "div",

  events: {
    "click #new-user": "newUser"
  },

  render: function() {
    this.$el.html( app.hookTemplate("user-list") );

    var that = this;
```

```
    app.userList.each(function(user) {
      var view = new userView({ model: user });
      $("tbody", that.$el).append( view.render().el );
    });

    return this;
  },

  newUser: function() {
    var view = new userNewView();
    $("#main").html( view.render().el );
  }
});
```

15.12 – User New View – `views/userNewView.js`

User အသစ်ထည့်သွင်းလို့ရတဲ့ Form ကို ဖော်ပြပေးတဲ့ View ဖြစ်ပါတယ်။ New Issue View နဲ့ လုပ်ဆောင်ချက်သိပ် မကွာပါဘူး။ ပြီးခဲ့တဲ့ Form တွေမှာရော၊ ဒီ Form မှာပါ ကျွန်တော်တို့ အလွယ်လုပ်ထားမိတာ တစ်ခုတော့ ရှိပါတယ်။ Form Validation လုပ်ဆောင်ချက်ကို Form နဲ့အတူ တွဲဖက်ထားခြင်းဖြစ်ပါတယ်။ လက်တွေ့မှာ Validate လို လုပ်ဆောင်ချက်မျိုးကို Model နဲ့တွဲပြီးသတ်မှတ်တာကို ပိုပြီး စနစ်ကျစေမှာဖြစ်ပါတယ်။ BackboneJS ကလည်း ဒီရည် ရွယ်ချက်နဲ့ပဲ `validate()` Function ကို Model မှာ ထည့်သွင်းပေးထားပါတယ်။ ဒါပေမယ့် နမူနာမှာတော့ Validation လုပ်ငန်းကို View ထဲမှာပဲ ထည့်သွင်းထားပါတယ်။ လက်တွေ့မှာ Model ထဲမှာ စစ်သင့်တယ်ဆိုတာကို သတိ ပြုစေလိုပါတယ်။

JavaScript

```
var userNewView = Backbone.View.extend({
  tagName: "div",

  events: {
    "submit #new-user-form": "create"
  },

  initialize: function() {
    this.listenTo(app.userList, 'add', this.showUserList);
  },

  render: function() {
    this.$el.html( app.hookTemplate("user-new") );
    return this;
  },

  create: function() {
    var fullName = $("#full-name").val();
    var loginId = $("#login-id").val();
    var email = $("#email").val();
    var role = $("#role").val();
    var password = $("#password").val();

    var err = 0;
    if(!fullName) {
      err++;
      $("#full-name").parent().addClass('has-error');
```

```
28.       }
29.
30.       if(!loginId) {
31.         err++;
32.         $("#login-id").parent().addClass('has-error');
33.       }
34.
35.       if(!email) {
36.         err++;
37.         $("#email").parent().addClass('has-error');
38.       }
39.
40.       if(!password) {
41.         err++;
42.         $("#password").parent().addClass('has-error');
43.       }
44.
45.       if(password != $("#password-again").val()) {
46.         err++;
47.         $("#password").parent().addClass('has-error');
48.         $("#password-again").parent().addClass('has-error');
49.       }
50.
51.       if(err) return false;
52.
53.       var model = new app.userModel({
54.         "fullName" : fullName,
55.         "loginId" : loginId,
56.         "email" : email,
57.         "role" : role,
58.         "password" : password
59.       });
60.
61.       model.save(null, {
62.         wait: true,
63.         success: function(res) {
64.           app.userList.add(res);
65.         }
66.       });
67.
68.       return false;
69.     },
70.
71.     showUserList: function() {
72.       var list = new userListView();
73.       $("#main").html( list.render().el );
74.     }
75. });
```

15.13 – User Edit View – `views/userEditView.js`

User ရဲ့ Password အပါအဝင် အချက်အလက်တွေကို ပြင်ဆင်နိုင်တဲ့ Form ကို ဖော်ပြပေးတဲ့ View ဖြစ်ပါတယ်။ ပိုကောင်းစေလိုရင်တော့ Password ပြင်လို့ရတဲ့ Form နဲ့ အခြားအချက်အလက်တွေကို ပြင်လို့ရတဲ့ Form ခွဲခြားထား သင့်ပါတယ်။ နမူနာမှာတော့ Password အပါအဝင် အချက်အလက်အားလုံးကို ပြင်ဆင်ဖို့အတွက် Form တစ်ခုတည်းကို သာ သုံးထားပါတယ်။

JavaScript

```
var userEditView = Backbone.View.extend({
  tagName: "div",

  events: {
    "submit #edit-user-form": "updateUser"
  },

  initialize: function() {
    //
  },

  render: function() {
    this.$el.html( app.hookTemplate("user-edit", this.model.toJSON()) );
    return this;
  },

  updateUser: function() {
    var fullName = $("#full-name").val();
    var loginId = $("#login-id").val();
    var email = $("#email").val();
    var role = $("#role").val();
    var password = $("#password").val();

    var err = 0;
    if(!fullName) {
      err++;
      $("#full-name").parent().addClass('has-error');
    }

    if(!loginId) {
      err++;
      $("#login-id").parent().addClass('has-error');
    }

    if(!email) {
      err++;
      $("#email").parent().addClass('has-error');
    }

    if(err) return false;

    var that = this;
    this.model.save({
      "fullName" : fullName,
      "loginId" : loginId,
      "email" : email,
      "role" : role,
      "roleLabel" : app.userRole[role],
      "password": password
   }, {
      wait: true,
      success: function() {
        that.showUserList();
      }
    });

    return false;
  },
```

```
59.
60.   showUserList: function() {
61.     var list = new userListView();
62.     $("#main").html( list.render().el );
63.   }
64. });
```

15.14 – User Profile View – `views/userProfileView.js`

User တစ်ဦးရဲ့ အချက်အလက်တွေကို ဖော်ပြပေးတဲ့ Profile View ဖြစ်ပါတယ်။ User ရဲ့ အမည်၊ Email စတဲ့ အချက် အလက်တွေနဲ့အတူ၊ အဲ့ဒီ User ထည့်သွင်းထားတဲ့ Issue အရေအတွက်နဲ့ အဲ့ဒီ User ကို Assigned လုပ်ထားတဲ့ Issue အရေအတွက်ကိုပါ ထည့်သွင်းဖော်ပြပေးမှာ ဖြစ်ပါတယ်။

JavaScript

```
1. var userProfileView = Backbone.View.extend({
2.   tagName: "div",
3.
4.   events: {
5.     "click #edit-user": "userEdit"
6.   },
7.
8.   initialize: function() {
9.     //
10.  },
11.
12.  render: function() {
13.    var that = this;
14.    this.model.set("submitCount", app.issueList.where({
15.      "submittedBy": that.model.id
16.    }).length);
17.
18.    this.model.set("assignCount", app.issueList.where({
19.      "assignedTo": that.model.id
20.    }).length);
21.
22.    this.$el.html(app.hookTemplate("user-profile", this.model.toJSON()));
23.    return this;
24.  },
25.
26.  userEdit: function() {
27.    var user = new userEditView({ model: this.model });
28.    $("#main").html( user.render().el );
29.  }
30. });
```

ဒါဟာ ကျွန်တော်တို့ရဲ့ Issue Tracking System Web Client အတွက် လိုအပ်တဲ့ Code အပြည့်အစုံပဲဖြစ်ပါတယ်။ အထက်မှာပြောခဲ့သလိုပဲ၊ လက်တွေ့မှာ တစ်လုံးချင်းကူးယူရေးသားမယ့်အစား၊ ရေးသားထားတဲ့ Code ကိုသာ နားလည် အောင်လေ့လာသင့်ပါတယ်။ သဘောသဘာဝကို နားလည်ပြီဆိုတော့မှသာ ကိုယ်တိုင်ရေးသားဖန်တီးကြည့်သင့်ပါတယ်။ Code အပြည့်အစုံကို အောက်ပါလိပ်စာမှာ Download ရယူနိုင်ပါတယ်။

https://github.com/eimg/issues-backbone/archive/master.zip

Conclusion

App တစ်ခုတည်ဆောက်ရာမှာ BackboneJS လို M-V-C Code Architecture Framework တွေကို အသုံးပြုရတဲ့ ရည်ရွယ်ချက်ကတော့ ရေရှည်မှာ ပြုပြင်ထိမ်းသိမ်းရလွယ်ကူစေဖို့ ဖြစ်ပါတယ်။ BackboneJS ရဲ့ အကူအညီနဲ့ App ရဲ့ အစိတ်အပိုင်းတွေကို ကဏ္ဍအလိုက်ခွဲခြားထားနိုင်တဲ့အတွက် ပြုပြင်ထိမ်းသိမ်းရလွယ်ကူမှုအတွက် အထောက်အကူဖြစ်စေမှာ ဖြစ်ပါတယ်။ ဥပမာ – အသွင်အပြင်ဖော်ပြပုံ Structure ကို ပြုပြင်လိုရင် သက်ဆိုင်ရာ Template ကိုပြင်နိုင်ပါတယ်။ အခြားလုပ်ဆောင်ချက်တွေ မပါဝင်ပဲ Template သက်သက်ဖြစ်လို့ ပြင်ဆင်ဖြည့်စွက်ရတာ လွယ်ကူပြီး၊ ပြင်လိုက်တဲ့ အတွက်လည်း အခြားလုပ်ဆောင်ချက်တွေကို သက်ရောက်ထိခိုက်သွားစရာမရှိပါဘူး။ အလားတူပဲ View တစ်ခုအတွက် သတ်မှတ်ထားတဲ့ Event တွေကို ပြင်ဆင်လိုရင် သက်ဆိုင်ရာ View မှာပြင်နိုင်ပါတယ်။ ဒီလိုပြင်ဆင်မှုကြောင့် အခြား View တွေအတွက် သက်မှတ်ထားတဲ့ Event တွေကို သက်ရောက်ထိခိုက်သွားမှာ မဟုတ်ပါဘူး။

ဖော်ပြခဲ့တဲ့နမူနာမှာ File Name တွေ၊ Model, View, Collection Name တွေ၊ Function Name တွေနဲ့ Variable Name တွေကိုပေးတဲ့အခါ၊ သက်ဆိုင်ရာလုပ်ဆောင်ချက်အဓိပ္ပါယ်ကိုပေါ်လွင်အောင် ပေးထားတာကိုလည်း သတိပြုစေလိုပါ တယ်။ ဒီလိုပေးထားတဲ့အတွက် File Name (သို့မဟုတ်) Function Name ကို ကြည့်လိုက်ယုံနဲ့ ဒါဘာအတွက်လည်း ဆိုတာကိုနားလည်နိုင်မှာဖြစ်လို့ ပြင်ဆင်ထိမ်းသိမ်းရလွယ်ကူမှုပိုင်းမှာ အများကြီး အထောက်အကူဖြစ်စေမှာ ဖြစ်ပါတယ်။ ဒါကြောင့် မိမိကိုယ်တိုင် ရေးသားတဲ့အခါမှာလည်း အမည်ပေးပုံလေးတွေကို အထူးဂရုပြုသင့်ပါတယ်။ လုပ်ဆောင်ချက် အဓိပ္ပါယ်ကို ပေါ်လွင်စေတဲ့အမည်တွေ ပေထားမှသာ ပြင်ဆင်ထိမ်းသိမ်းရလွယ်ကူတဲ့ Code ဖြစ်မှာ ဖြစ်ပါတယ်။

နမူနာကို တက်နိုင်သမျှရိုးရှင်းစေလိုတဲ့အတွက် မဖြစ်မနေပါဝင်သင့်တဲ့ လုပ်ဆောင်ချက်တွေကိုသာ ထည့်သွင်းထားပါတယ်။ လက်တွေ့မှာထပ်မံဖြည့်စွက်စရာတွေရှိပါသေးတယ်။ ဥပမာ – Issue တွေကို Status အလိုက် သော်လည်ကောင်း Priority အလိုက်သော်လည်ကောင်း Filter လုပ်ပေးနိုင်တဲ့ လုပ်ဆောင်ချက်တွေ ထည့်သွင်းသင့်ပါတယ်။ Email Notification လို လုပ်ဆောင်ချက်တွေ ထည့်သွင်းသင့်ပါတယ်။ ကျွန်တော်တို့ **အခန်း (၆)** မှာဖော်ပြခဲ့တဲ့ Issue Tracking System တစ်ခု မှာပါဝင်သင့်တဲ့ လုပ်ဆောင်ချက်တွေ အပြည့်အစုံပါဝင်အောင် ဆက်လက်ဖြည့်စွက်သင့်ပါတယ်။

စာဖတ်သူအနေနဲ့ နမူနာ Source Code ကိုရယူပြီး လိုအပ်သလို ပြင်ဆင်ဖြည့်စွက်ပြီး စမ်းသပ်ကြည့်စေလိုပါ တယ်။

တစ်ကြိမ်ရေးသားလိုက်ယုံနဲ့ Platform အမျိုးမျိုးမှာအလုပ်
လုပ်နိုင်တဲ့ Cross-platform လုပ်ဆောင်ချက်ရရှိဖို့ဆိုတာ
Software Development မှာ ယခင်ကတည်းကရှိခဲ့တဲ့
အဓိကစိန်ခေါ်မှုတစ်ရပ်ဖြစ်ပါတယ်။

Professional Web Developer (စာအုပ်)
Web Standard, jQuery, PHP, MySQL, Ajax, CMS, MVC,
HTML5, Mobile Web, Web Application Security စသည့်
အကြောင်းအရာများကို ရေးသားဖော်ပြထားသည့်စာအုပ်
အောက်ပါလိပ်စာတွင် အခမဲ့ Download ရယူနိုင်သည်။

http://pwdbook.com

အခန်း(၁၆) – Building Clients (Hybrid Mobile App)

အခန်း (၁၃) မှာ ExpressJS ကိုအသုံးပြုပြီး ကျွန်တော်တို့တည်ဆောက်ထားခဲ့တဲ့ Issue Tracking System ဟာ API Service တစ်ခုဖြစ်တဲ့အတွက် HTTP အသုံးပြုဆက်သွယ်နိုင်တဲ့ မည်သည့် Client နဲ့မဆို ချိတ်ဆက်အလုပ်လုပ်နိုင်မှာ ဖြစ် ပါတယ်။ ပြီးခဲ့တဲ့အခန်းမှာ နမူနာအနေနဲ့ Web Client တစ်ခုကို အဲ့ဒီ API Service နဲ့ချိတ်ဆက်တည်ဆောက်ပုံကို ဖော်ပြ ခဲ့ပါတယ်။ အကယ်၍ အဲ့ဒီ API နဲ့ချိတ်ဆက်ပြီး Andorid, iOS စတဲ့ Mobile Platform တွေမှာအလုပ်လုပ်တဲ့ Mobile Client တွေကို တည်ဆောက်မယ်ဆိုရင်လည်း တည်ဆောက်နိုင်မှာဖြစ်ပါတယ်။

16.1 – Types of Mobile Apps

Mobile App တွေကို အုပ်စုသုံးစုခွဲနိုင်ပါတယ်။ Native App, Mobile Web App နဲ့ Hybrid App တို့ပဲဖြစ်ပါတယ်။

Native App ဆိုတာကတော့ သက်ဆိုင်ရာ Mobile Platform အတွက်ပေးထားတဲ့ SDK တွေနဲ့ ဆက်စပ်နည်းပညာတွေ ကို အသုံးပြုတည်ဆောက်ထားတဲ့ App များဖြစ်ပါတယ်။ Mobile Device များမှာ Install လုပ်ထည့်သွင်းထားနိုင်တဲ့ App အမျိုးအစားဖြစ်ပါတယ်။ Native App ရဲ့ ထူးခြားချက်ကတော့ Android အတွက် ဖန်တီးထားတဲ့ App ကို Android Device တွေမှာသာ အသုံးပြုနိုင်မှာဖြစ်ပြီး၊ iOS အတွက် ရည်ရွယ်ဖန်တီးထားတဲ့ App ကို iOS မှာသာ အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

ပြီးခဲ့တဲ့ အခန်းမှာကျွန်တော်တို့တည်ဆောက်ခဲ့တဲ့ Web Client ဟာ Bootstrap ရဲ့ Responsive Layout ကို အသုံးပြု ထားတဲ့အတွက် Mobile Device တွေအပါအဝင် မည်သည့် Device မှာမဆို အလုပ်လုပ်မှာ ဖြစ်ပါတယ်။ ဒါကြောင့် အဲ့ဒီ Web Client ကို Mobile Web Client လို့ဆိုနိုင်ပါတယ်။ Mobile Web Client ရဲ့ ထူးခြားချက်ကတော့ Browser ပါဝင်တဲ့ မည်သည့် Device မှာမဆိုအသုံးပြုနိုင်မှာဖြစ်ပါတယ်။ ဒါပေမယ့် Mobile Web Client ရဲ့ အားနည်းချက် ကတော့ Device က ပေးနိုင်တဲ့စွမ်းဆောင်ရည်ကို အပြည့်အဝရရှိမှာ မဟုတ်ပါဘူး။ ဥပမာ၊ Device ရဲ့ Storage နဲ့ GPS, Camera စတဲ့လုပ်ဆောင်ချက်တွေကို Web Client ကနေ အပြည့်အဝရယူအသုံးချနိုင်ဖို့ မလွယ်ပါဘူး။ ဒါကြောင့် လက်ရှိအချိန်ထိ Mobile Web Client တွေဟာ Native Client နဲ့ယှဉ်ရင် စွမ်းဆောင်ရည်ပိုင်း အားနည်းနေပါသေးတယ်။

ဒီနှစ်ခုကို ပေါင်းစပ်အသုံးပြုထားတဲ့ နည်းစနစ်ကိုတော့ Hybrid App လို့ခေါ်ပါတယ်။ App ကို တည်ဆောက်ရာမှာ HTML5, CSS3, JavaScript စတဲ့ Web Technology များနဲ့ တည်ဆောက်ရပါတယ်။ Mobile Web Client တွေရဲ့ အားသာချက်ဖြစ်တဲ့ တစ်ကြိမ်ရေးလိုက်ယုံနဲ့ Mobile Platform အမျိုးမျိုးမှာ အလုပ်လုပ်နိုင်တဲ့ အားသာချက်ကို ဆက်ခံ ရရှိပါတယ်။ ဒီနည်းစနစ်က Web Technology နဲ့ Native Technology ကို ပေါင်းစပ်အသုံးပြုခြင်းဖြစ်လို့ Device ရဲ့ စွမ်းဆောင်ရည်ကိုလည်း အတိုင်းအတာတစ်ခုထိအသုံးချနိုင်ပါတယ်။ ဒီနည်းနဲ့ဖန်တီးထားတဲ့ App တွေကို Web Browser ဖွင့်စရာမလိုပဲ အသင့်အသုံးပြုနိုင်အောင်၊ Native App ကဲ့သို့ Install လုပ်ထားလို့လည်း ရပါသေးတယ်။

နည်းစနစ် (၃) မျိုးလုံးမှာ သူ့ကိုယ်ပိုင် အားသာချက်အားနည်းချက်တွေ ကိုယ်စီရှိကြပြီး ဒီနေရာမှာတော့ ကျွန်တော်တို့ ပြီးခဲ့တဲ့အခန်းမှာ တည်ဆောက်ခဲ့တဲ့ Web Client ကို Cordova လို့ခေါ်တဲ့ နည်းပညာရဲ့အကူအညီနဲ့ Mobile Device တွေမှာ Install လုပ်အသုံးပြုနိုင်တဲ့ Hybrid App ဖြစ်အောင် တည်ဆောက်ပုံကို ဖော်ပြသွားမှာ ဖြစ်ပါတယ်။

16.2 – Apache Cordova

Apache Cordova ဆိုတာဟာ Hybrid App တွေတည်ဆောက်ရာမှာ အသုံးပြုနိုင်တဲ့ Mobile Development Framework တစ်ခုဖြစ်ပါတယ်။ App တည်ဆောက်ဖို့အတွက် HTML5, CSS3, JavaScript စတဲ့ Web Technology များကိုအသုံးပြုရပြီး၊ App ကို သက်ဆိုင်ရာ Mobile Platform နဲ့အတူပါဝင်လာတဲ့ Webview လို့ခေါ်တဲ့ Embedded Browser ထဲမှာ အလုပ်လုပ်ပေးမှာဖြစ်ပါတယ်။ Mobile Device ရဲ့ Native လုပ်ဆောင်ချက်တွေကို JavaScript နဲ့ ရယူအသုံးပြုနိုင်စေဖို့ API တွေပေးထားတဲ့အတွက် App ကနေ Device ရဲ့ Native လုပ်ဆောင်ချက်တွေကို ရယူအသုံးပြု နိုင်မှာဖြစ်ပါတယ်။

ဒီနေရာမှာ Cordova ရဲ့ လုပ်ဆောင်ချက်အားလုံးကို ဖော်ပြသွားမှာမဟုတ်ပဲ Cordova ကိုအသုံးပြုပြီး ကျွန်တော်တို့ တည်ဆောက်ထားတဲ့ Web Client ကို Hybrid App တစ်ခုဖြစ်အောင် ပြောင်းလဲပုံကိုသာဖော်ပြသွားမှာဖြစ်ပါတယ်။ Cordova အကြောင်းအသေးစိတ်ကို cordova.apache.org မှာ လေ့လာနိုင်ပါတယ်။

Cordova ကို NPM နဲ့ အခုလို Install ပြုလုပ်နိုင်ပါတယ်။

```
$ sudo npm install -g cordova
```

Install လုပ်ပြီးတဲ့အခါ Project တစ်ခုစတင်ဖို့အတွက် `cordova create` ကို အသုံးပြုနိုင်ပါတယ်။

```
$ cordova create issues com.yourname.issues 'Issue Tracking System'
```

`cordova create` အတွက် Parameter (၃) ခုပေးလိုက်တာကို တွေ့ရမှာဖြစ်ပါတယ်။ ရှေ့ဆုံးက `issues` က Project Directory Name ဖြစ်ပါတယ်။ သူ့နောက်က `com.yourname.issues` ကတော့ Reverse Domain Style ID ဖြစ်ပါတယ်။ Domain Name ကိုပြောင်းပြန်ပေးလိုက်ခြင်းဖြစ်ပြီး Unique Package Name အနေနဲ့ သုံးပါတယ်။ `yourname` နေရာမှာ ကိုယ့်ရဲ့ကိုယ်ပိုင် Domain Name နဲ့အစားထိုးပေးလို့ရပါတယ်။ နောက်ဆုံးက Parameter ကတော့ App Title ဖြစ်ပါတယ်။

`cordova create` က တည်ဆောက်ပေးလိုက်တဲ့ `issues` Directory ကိုဝင်ကြည့်ရင်အခုလိုတွေ့ရမှာဖြစ်ပါတယ်။

```
issues
├── config.xml
├── hooks
│   └── README.md
├── platforms
├── plugins
└── www
    ├── css
    ├── img
    ├── index.html
    └── js
```

`config.xml` ထဲမှာ App Setting တွေပါဝင်မှာဖြစ်ပါတယ်။ ဖွင့်ကြည့်လိုက်ရင် အခုလို တွေ့ရနိုင်ပါတယ်။

XML

```xml
<?xml version='1.0' encoding='utf-8'?>
<widget id="com.yourname.issues" version="0.0.1"
    xmlns="http://www.w3.org/ns/widgets"
    xmlns:cdv="http://cordova.apache.org/ns/1.0">
    <name>Issues Tracking System</name>
    <description>
        …
    </description>
    <author email="dev@cordova.apache.org" href="http://cordova.io">
        Apache Cordova Team
    </author>
    <content src="index.html" />
    <plugin name="cordova-plugin-whitelist" version="1" />
    <access origin="*" />
    <allow-intent href="http://*/*" />
    <allow-intent href="https://*/*" />
    <allow-intent href="tel:*" />
    <allow-intent href="sms:*" />
    <allow-intent href="mailto:*" />
    <allow-intent href="geo:*" />
    <platform name="android">
        <allow-intent href="market:*" />
    </platform>
    <platform name="ios">
        <allow-intent href="itms:*" />
        <allow-intent href="itms-apps:*" />
    </platform>
</widget>
```

`id` အနေနဲ့ ကျွန်တော်တို့ပေးလိုက်တဲ့ Reverse Domain ကို အသုံးပြုထားတာကို တွေ့ရမှာဖြစ်ပြီး `<name>`, `<description>`, `<author>` စတဲ့ Element တွေရဲ့ တန်ဖိုးကိုလည်း သင့်တော်သလို ပြုပြင်ထားနိုင်ပါတယ်။ ထူးခြားချက်အနေနဲ့ လိုင်းနံပါတ် (၁၂) က `<access origin="*" />` ကို သတိပြုကြည့်ပါ။ ပုံမှန်

ဆိုရင် Web Browser တွေက Cross-domain Request တွေကို ခွင့်ပြုလေ့မရှိကြပါဘူး။ Cordova ကတော့ Default အနေနဲ့ `<access origin="*" />` လို့သတ်မှတ်ပေးထားတဲ့အတွက် Cross-domain Request တွေကို ခွင့်ပြုပေးသွားမှာ ဖြစ်ပါတယ်။ ဒါကြောင့် API Service ကို ဘယ်မှာပဲထားထား ဒီ App ကနေ Access လုပ်အသုံးပြုနိုင်မှာဖြစ်ပါတယ်။

Project Directory ထဲက `www` Directory ကတော့ App Directory ဖြစ်ပါတယ်။ ကျွန်တော်တို့ ရရေးသားပြီးဖြစ်တဲ့ BackboneJS Client Code တွေကို အဲ့ဒီ Directory ထဲမှာ ကူးယူထည့်သွင်းပေးထားရမှာ ဖြစ်ပါတယ်။

ဆက်လက်ပြီး ရည်ရွယ်အသုံးပြုလိုတဲ့ Platform ကို `cordova add platform` နဲ့ထည့်သွင်းပေးနိုင်ပါတယ်။ ဥပမာ –

```
$ cordova platform add android

npm http GET https://registry.npmjs.org/cordova-android/4.0.0
npm http 200 https://registry.npmjs.org/cordova-android/4.0.0
npm http GET https://registry.npmjs.org/cordova-android/-/cordova-android-
4.0.0.tgz
npm http 200 https://registry.npmjs.org/cordova-android/-/cordova-android-
4.0.0.tgz
Adding android project...
Creating Cordova project for the Android platform:
     Path: platforms/android
     Package: com.eimaung.issues
     Name: Issues Tracking System
     Activity: MainActivity
     Android target: android-22
Copying template files...
Android project created with cordova-android@4.0.0
Discovered plugin "cordova-plugin-whitelist" in config.xml. Installing to the
project
Fetching plugin "cordova-plugin-whitelist@1" via npm
npm http GET https://registry.npmjs.org/cordova-plugin-whitelist
npm http 304 https://registry.npmjs.org/cordova-plugin-whitelist
Installing "cordova-plugin-whitelist" for android
```

ဒီလိုထည့်သွင်းနိုင်ဖို့အတွက် စက်ထဲမှာ Android SDK တော့အဆင်သင့်ရှိရပါတယ်။ Cordova က Android SDK နဲ့အတူ ပါဝင်လာတဲ့ Platform Tools တွေကို အသုံးပြုမှာဖြစ်တဲ့အတွက်၊ အကယ်၍ Android SDK မရှိသေးရင် အောက်ပါလိပ်စာမှာ ကြိုတင်ရယူ ထည့်သွင်းထားပေးရမှာ ဖြစ်ပါတယ်။

http://developer.android.com/sdk/index.html

Android အပြင် အခြား Platform တွေကို ထည့်သွင်းလိုတယ်ဆိုရင်လည်း အခုလို ထည့်သွင်းပေးနိုင်ပါတယ်။

```
cordova platform add ios
cordova platform add amazon-fireos
cordova platform add blackberry10
cordova platform add firefoxos
cordova platform add wp8
cordova platform add windows
```

ထည့်သွင်းထားတဲ့ Platform တစ်ခုကို ပြန်ထုတ်လိုရင်တော့ `cordova platform remove` ကို သုံးနိုင်ပါတယ်။

`www` Directory ထဲမှာ Web Client Source Code ကိုကူးယူထည့်သွင်းပြီးရင် `index.html` ကို အခုလိုပြုပြင်ပေးဖို့ လိုအပ်ပါတယ်။

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  …
</head>
<body onLoad="onLoad()">
  …
  <script>
    function onLoad() {
      document.addEventListener("deviceready", function(){
        app.init();
      }, false);
    }
  </script>
</body>
</html>
```

ပြောင်းလဲဖို့လိုတဲ့အပိုင်းကိုပဲ ရွေးချယ်ဖော်ပြထားခြင်းဖြစ်ပါတယ်။ ကျန်အပိုင်းတွေ အပြောင်းအလဲမရှိပါဘူး။ Webview ထဲမှာ JavaScript Code တွေ မှန်မှန်ကန်ကန်အလုပ်လုပ်ဖို့ဆိုရင် `deviceready` Event ထဲမှာ ထည့်သွင်းပေးဖို့လိုတဲ့ အတွက် `app.init()` ကို `deviceready` Event ထဲမှာ အလုပ်လုပ်စေခြင်းဖြစ်ပါတယ်။ တနည်းအားဖြင့် ကျွန်တော်တို့ရဲ့ Web Client Code ကို Device နဲ့ Webview အသင့်ဖြစ်တော့မှ စတင်အလုပ်လုပ်ဖို့ သတ်မှတ်လိုက်ခြင်းပဲ ဖြစ်ပါတယ်။

App ကို Android Device တစ်ခုမှာ Run ကြည့်နိုင်ဖို့အတွက် Device ကို ကွန်ပျူတာမှာ USB ကြိုးနဲ့ချိတ်ပေးရပါမယ်။ ပြီးရင် Device Setting တွေထဲက USB Debugging ကို ဖွင့်ထားပေးရပါမယ်။ USB ကြိုးချိတ်ပြီး USB Debugging ကို ဖွင့်ပြီးတဲ့အခါ `cordova run` နဲ့ App ကို စမ်းသပ်နိုင်ပြီဖြစ်ပါတယ်။

```
$ cordova run android
```

ကျွန်တော်တို့ရဲ့ App က USB နဲ့ ချိတ်ဆက် ထားတဲ့ Android Device ပေါ်မှာ အလုပ်လုပ်သွားမှာ ဖြစ်ပါတယ်။။ `cordova run` က `.apk` Installer ဖိုင်တစ်ခုကိုလည်း တစ်ခါတည်းတည်ဆောက်ပေးသွားပါတယ်။ `platforms/android/build` Directory ထဲမှာ ရှိနေမှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Hybrid App တစ်ခုကို Android အတွက်သာမက အခြား Mobile Platform တွေအတွက်ပါ အလွယ်တစ်ကူ တည်ဆောက်ယူနိုင်မှာ ဖြစ်ပါတယ်။

Conclusion

နမူနာအရ အခြားဖြည့်စွက်မှုတွေမပါဝင်ပဲ ကြိုတင်ရေးသားထားတဲ့ Web App တစ်ခုကို Install လုပ်လို့ရတဲ့ App ဖြစ် အောင် ပြောင်းလိုက်ခြင်းသာဖြစ်ပါတယ်။ လက်တွေ့မှာ Device ရဲ့ လုပ်ဆောင်ချက်တွေကို ထိရောက်အောင် ထည့်သွင်း အသုံးချနိုင်မှသာ Hybrid App ဖြစ်သွားခြင်းရဲ့ အကျိုးရလဒ်ကိုရရှိမှာဖြစ်ပါတယ်။

Device ရဲ့ Battery Status, Network Status, Motion Sensor, Orientation, Geolocation, Camera, File, Vibration စတဲ့ Hardware ပိုင်းဆိုင်ရာ လုပ်ဆောင်ချက်တွေနဲ့ Contact, Dialog စတဲ့ Mobile OS ရဲ့ လုပ်ဆောင်ချက် တွေကို App ကနေ ရယူအသုံးပြုလိုတယ်ဆိုရင် Plugin တွေကို အသုံးပြုနိုင်ပါတယ်။ အသုံးပြုနိုင်တဲ့ Cordova Plugin စာရင်းကို အောက်ပါလိပ်စာမှာ ဆက်လက်လေ့လာနိုင်ပါတယ်။

http://plugins.cordova.io/

အပိုင်း (၅)

Deployment and Scaling

Software တစ်ခုရဲ့ လုပ်ဆောင်ချက်တွေ တိုးတက်
ပြောင်းလဲနေတာနဲ့အမျှ အဲ့ဒီ Software ကို Run ပေးရမယ့်
Server Environment ကလည်း အလိုက်သင့်
လိုက်လံပြောင်းလဲပေးနိုင်ဖို့လိုပါတယ်။

Ubuntu – သင့်အတွက် Linux (စာအုပ်)
Ubuntu Linux အသုံးပြုနည်းအား Installation မှ စတင်၍ System
Configuration နှင့် Development Environment တည်ဆောက်
ခြင်းအထိ အဆင့်လိုက်ရေးသား ဖော်ပြထားသည့်စာအုပ်
အောက်ပါလိပ်စာတွင် အခမဲ့ ရယူနိုင်သည်။

http://u4ubook.com

အခန်း(၁၇) – Deployment with Docker

Software တစ်ခုအတွက် လိုအပ်တဲ့ Production Server Environment တစ်ခု တည်ဆောက်ရာမှာ အဓိက ကြုံတွေ့ရ နိုင်တဲ့ Challenge (၃) ခု ရှိပါတယ်။ အဲ့ဒါတွေကတော့ –

1. လက်ရှိ Development Machine ပေါ်မှာ အလုပ်လုပ်နေတဲ့ Software ဟာ Production Server ပေါ် ရောက်တဲ့အခါ၊ လိုအပ်တဲ့ Package Dependencies မကိုက်ညီတဲ့အတွက် အလုပ်မလုပ်တော့တဲ့ ပြဿနာ ရှိတက်ပါတယ်။
2. Software ကို စဉ်ဆက်မပြတ်အဆင့်မြှင့်တင်နေတဲ့အတွက် အဲ့ဒီ Software ကို လက်ခံအလုပ်လုပ် ပေးရမယ့် Production Server ကလည်း အလိုက်သင့်လိုက်လံပြောင်းလဲပေးဖို့ လိုနိုင်ပါတယ်။
3. SOA နည်းစနစ်ကိုသုံးပြီး၊ Software တစ်ခုကို Service ငယ်များအဖြစ် ခွဲခြားတည်ဆောက် တဲ့အခါ၊ Service တွေအလုပ်လုပ်ဖို့အတွက် လိုအပ်တဲ့ Environment မတူတဲ့အတွက်၊ Production Server တစ်ခု ပေါ်မှာ Service နှစ်ခုသုံးခု ပူးတွဲအလုပ်မလုပ်နိုင်တဲ့ပြဿနာ ရှိနိုင်ပါတယ်။

17.1 – Development Machine vs. Production Server

Software တစ်ခုမှာ လက်ရှိရေးသားစမ်းသပ်နေတဲ့ Development Code နဲ့ အများအသုံးပြုနိုင်ဖို့ Build လုပ် ထားတဲ့ Production Code ဆိုပြီး ရှိတဲ့ကြောင်းကို **အပိုင်း (၂)** မှာ ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ Code မှာ Development နဲ့ Production ဆိုပြီးနှစ်ပိုင်းရှိသလိုပဲ၊ အဲ့ဒီ Code ကို Run မယ့် ကွန်ပျူတာတွေမှာလည်း Development Machine နဲ့ Production Server ဆိုပြီးနှစ်မျိုးရှိမှာ ဖြစ်ပါတယ်။ Development Machine ဆို တာ ကျွန်တော်တို့ Code ရေးသား ဖို့ နေ့စဉ်အသုံးပြုနေတဲ့ Laptop ကွန်ပျူတာဖြစ်နိုင်ပါတယ်။ Production Server ဆိုတာကတော့ အများ ဆက်သွယ် ရယူ အသုံးပြုနိုင်အောင် Public IP Address (သို့မဟုတ်) Domain Name တစ်ခုနဲ့ စီစဉ်ပေးထားတဲ့ စွမ်းဆောင်ရည် မြင့် ကွန်ပျူတာဖြစ်နိုင်ပါတယ်။

ရေးသားထားတဲ့ Software ဟာ ကျွန်တော်တို့ရဲ့ Development Machine မှာ အလုပ်လုပ်နေသလိုပဲ၊ Production Server မှာလဲ ပုံမှန် ဆက်လက်အလုပ်လုပ်နေဖို့လိုပါတယ်။ တနည်းအားဖြင့် Production Server ကို ကျွန်တော်တို့ရဲ့ Development Machine နဲ့ တူညီတဲ့ Environment ရှိအောင် စီစဉ်ထားဖို့ လိုအပ် မှာဖြစ်ပါတယ်။ Software အတွက် လိုအပ်တဲ့ Operating System, Web Server, Database Server, Programming Language နဲ့ အခြားလိုအပ်တဲ့ Package တွေကို Production Server ထည့်သွင်းထားဖို့ လိုနိုင် ပါတယ်။

ဒီလိုထည့်သွင်းရာမှာ Software တွေ Package တွေရဲ့ Version ကိုလည်း လိုအပ်ချက်နဲ့အညီ အတိအကျ ထည့်သွင်း ထားဖို့ လိုနိုင်ပါတယ်။ ဥပမာ – Software ကို ရေးသားစဉ်က MongoDB 2.4 နဲ့ တွဲဖက်ရေးသား ထားတယ်ဆိုရင် Production Server မှာလဲ MongoDB 2.4 ကို ထည့်သွင်းထားပေးဖို့ လိုအပ်မှာဖြစ်ပါတယ်။

ပိုကောင်းတဲ့နောက်ဆုံး Version မို့လို့ဆိုပြီး MongoDB 3.0 ကို Production Server မှာထည့်သွင်းထားမယ်ဆိုရင် Software က အဆင် ပြေပြေ အလုပ်လုပ်ဖို့မသေချာတော့ပါဘူး။ Development Machine မှာ အလုပ်လုပ်နေပြီး၊ Production Server ကျတော့မှ အလုပ်မလုပ်တော့ဘူးဆိုတဲ့ ပြဿနာမျိုး ကြုံတွေ့ရနိုင်ပါတယ်။

အမြဲအဆင့်မြှင့်တင်မှုတွေ ပြုလုပ်နေတဲ့ Software တစ်ခုဟာ အသုံးပြုထားတဲ့ Package တွေလည်း အမြဲ ပြောင်းလဲနေ နိုင်ပါတယ်။ ဥပမာ – လက်ရှိ Software ရဲ့ Version 1.0 ကို NodeJS 0.12.2 ကိုအသုံးပြု တည်ဆောက်ထားတယ် ဆိုပါစို့။ ဒီ Software အဆင်ပြေပြေအလုပ်လုပ်စေဖို့အတွက် Production Server မှာလည်း NodeJS 0.12.2 ကို အတိ အကျထည့်သွင်းထားဖို့ လိုနိုင်ပါတယ်။ နောက်တော့မှ Software ရဲ့ Version 2.0 ကို NodeJS 1.0 နဲ့ ပြောင်းလဲတည် ဆောက်တယ်ဆိုရင် Production Server မှာလည်း NodeJS 1.0 ကို ပြောင်းလဲထည့်သွင်းထားပေးဖို့ လိုနိုင်ပါတယ်။ ဒါပေမယ့် ချက်ခြင်းထည့်သွင်းလို့ မရသေးပါဘူး။ Version 2.0 ကို Release လုပ်တော့မယ်ဆိုမှ ထည့်သွင်းပေးရမှာပါ။ကာလတစ်ခုထိ၊ Production Server မှာ Run နေတာက Software ရဲ့ Version 1.0 ဖြစ်ပြီး၊ လက်ရှိ Development Machine ပေါ်မှာ ရောက်ရှိနေတာက Version 2.0 ဖြစ် နေတယ်ဆိုတာမျိုးလည်း ဖြစ်နိုင်ပါသေးတယ်။ ဒီလိုအခြေအနေ တွေကို ထိရောက်အောင် စီမံနိုင်ဖို့ဆိုတာ Challenge တစ်ခုဖြစ်ပါတယ်။

နောက်ပြဿနာတစ်ခုကတော့ အခြေခံလိုအပ်ချက်မတူတဲ့ Service တွေကို Server တစ်ခုတည်းပေါ်မှာ ယှဉ်တွဲ အလုပ်လုပ် စေဖို့ကလည်း လိုအပ်နိုင်ပါသေးတယ်။ ဥပမာ – Service တစ်ခုက NodeJS 0.12.2 နဲ့တည်ဆောက် ထားပြီး နောက် Service တစ်ခုက NodeJS 1.0 နဲ့တည်ဆောက်ထားတယ်ဆိုတဲ့အခြေအနေမှာ၊ Service နှစ်ခု လုံးကို Server တစ်လုံး တည်းနဲ့ အလုပ်လုပ်နိုင်စေဖို့ဆိုရင် NodeJS Version နှစ်မျိုး ထည့်သွင်းထားရတော့မယ့် သဘော ဖြစ်ပါတယ်။ တစ်ချို့ Package တွေက Version နှစ်မျိုးသုံးမျိုးကို ယှဉ်တွဲထည့်သွင်းထားလို့ ရတက်ပေ မယ့်၊ Package အများစုကတော့ အဲ့ဒီ လို ယှဉ်တွဲထည့်သွင်းထားလို့ ရမှာ မဟုတ်ပါဘူး။ ဒါလည်းပဲ နောက်ထပ် Challenge တစ်ခုပဲဖြစ်ပါတယ်။

ဒီ ပြဿနာတွေကို ဖြေရှင်းဖို့အတွက် အသုံးဝင်တဲ့နည်းပညာကတော့ Docker ဆိုတဲ့ နည်းပညာပဲ ဖြစ်ပါတယ်။

17.2 – Docker

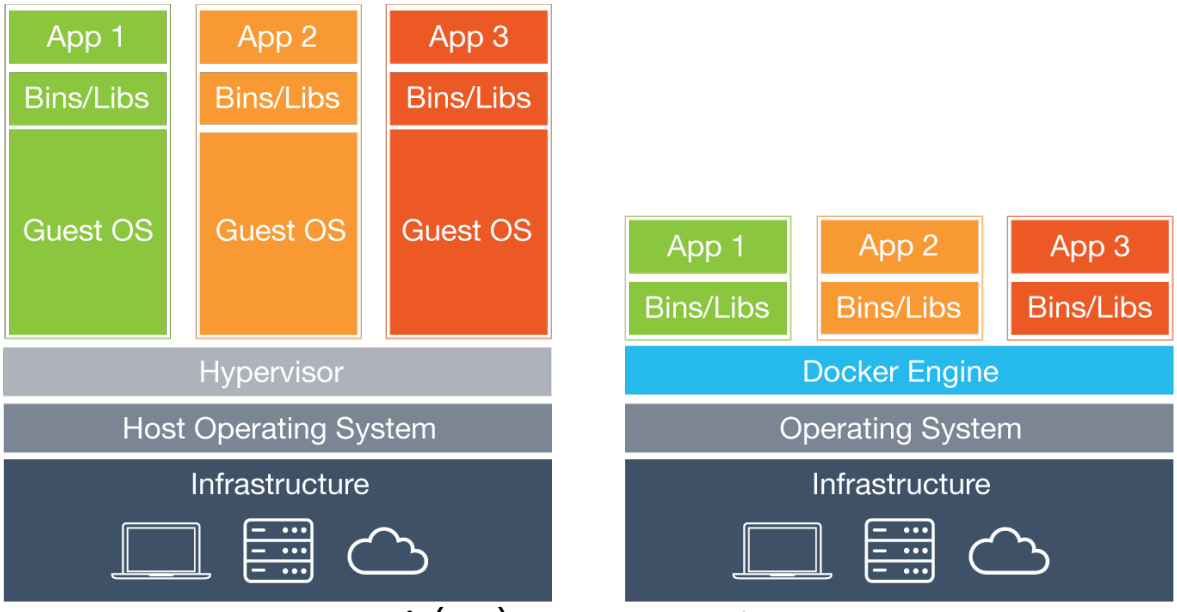
Docker ဟာ လက်ရှိမှာ အများရဲ့စိတ်ဝင်စားမှုကို အထူးရရှိနေတဲ့ Tool တစ်ခုဖြစ်ပါတယ်။ LXC လို့ အတိုကောက်ခေါ်တဲ့ Linux Container နည်းပညာပေါ်မှာ အခြေခံထားခြင်း ဖြစ်ပါတယ်။ LXC ဆိုတာဟာ Virtualization နည်းပညာတစ်မျိုး ဖြစ်ပါတယ်။ ဒါပေမယ့် ကျွန်တော်တို့ အသုံးပြုကြလေ့ရှိတဲ့ VirtualBox, VMWare စတဲ့ Hypervisor အခြေပြု Virtual Machine တွေနဲ့ သဘောသဘာဝ ကွဲပြားပါတယ်။

VirtualBox တို့ VMWare တို့လို Hypervisor အခြေပြု Virtual Machine တွေဟာ Host Operating System ပေါ်မှာ Guest Operating System ကို ထပ်မံထည့်သွင်း အသုံးပြုနိုင်တဲ့ နည်းပညာဖြစ်ပါတယ်။ အခြေခံအားဖြင့် Host Operating System ဟာ Linux, Windows, Mac မည်သည့် OS မဆို ဖြစ်နိုင်သလို၊ Guest OS အနေနဲ့လည်း မည် သည့် OS မဆိုဖြစ်နိုင်ပါတယ်။ ဥပမာ – Ubuntu OS ပေါ်မှာ VirtualBox နဲ့ Windows 7 ကို ထည့်သွင်းအသုံးပြု နိုင်ပါတယ်။ ဒီအခြေအနေမှာ Ubuntu OS ဟာ Host OS ဖြစ်ပြီး Windows 7 ဟာ Guest OS ဖြစ်ပါတယ်။ ဒီ အကြောင်းကို **Ubuntu – သင့်အတွက် Linux** ရဲ့ **အခန်း (၁၁)** မှာ ဖော်ပြထားပါတယ်။ Host OS နဲ့ Guest OS

အမျိုးအစား တူစရာမလိုသလို Host OS တစ်ခု ပေါ်မှာလည်း Guest OS အများအပြားရှိနိုင်ပါတယ်။ **ပုံ (၁၇.၁)** ရဲ့ ဘယ်ဘက်ခြမ်းမှာ Hypervisor အခြေပြု Virtual Machine တွေ အလုပ်လုပ်ပုံကို ဖော်ပြထားပါတယ်။



Docker Container တွေကတော့ Virtual Machine နဲ့ အနည်းငယ်ကွဲပြားပါတယ်။ Docker ဟာ Linux OS ပေါ်မှာ အလုပ်လုပ်တဲ့ Linux အခြေပြု နည်းပညာ ဖြစ်ပါ တယ်။ Docker ကိုအသုံးပြုပြီး Linux Kernel တစ်ခု ပေါ်မှာ သီးခြား Environment နဲ့အလုပ်လုပ်တဲ့ Linux OS တွေကို ထပ်မံထည့်သွင်းနိုင်ပါတယ်။ Virtual Machine အခေါ်အဝေါ်တွေကို သုံးပြီးပြောရမယ်ဆိုရင် Linux Host OS ပေါ်မှာ နောက်ထပ် Linux Guest OS တွေ ထပ်မံထည့် သွင်းနိုင်တဲ့ သဘော ပါပဲ။ Docker မှာတော့ ထပ်မံထည့် သွင်းထားတဲ့ OS တွေကို Guest OS လို့မခေါ်ပဲ OS Image လို့ခေါ်ပါတယ်။ OS Image တွေ Run နိုင်တဲ့ လုပ်ဆောင် ချက်ကိုလည်း Virtual Machine လို့မခေါ်ပါဘူး။ Container လို့ခေါ်ပါတယ်။ **ပုံ (၁၇.၁)** ရဲ့ ညာဘက်ခြမ်းမှာ Docker အခြေပြု Container နည်းပညာရဲ့ အလုပ်လုပ်ပုံကို ဖော်ပြထားပါတယ်။



ပုံ (၁၇.၁) - Hypervisor vs. Docker

Source – docker.com

ပုံ (၁၇.၁) မှာယှဉ်တွဲဖော်ပြထားတဲ့ နည်းပညာနှစ်ခုကို လေ့လာကြည့်ရင် ထူးခြားချက်ကို သတိပြုမိမှာပါ။ Hypervisor နည်းပညာမှာ၊ Host OS က Kernel, Library နဲ့ လိုအပ်တဲ့အစိတ်အပိုင်း အပြည့်အစုံပါဝင်တဲ့ OS ဖြစ်သလို၊ Guest OS တွေကလည်း Kernel, Library နဲ့ လိုအပ်တဲ့အစိတ်အပိုင်း အပြည့်အစုံပါဝင်တဲ့ OS တွေဖြစ်ကြပါတယ်။ Docker နည်းပညာမှာတော့၊ Host OS ကသာ Kernel, Library နဲ့ လိုအပ်တဲ့အစိတ်အပိုင်း အပြည့်အစုံပါဝင်တဲ့ OS ဖြစ်ပြီး Container ပေါ်က OS Image တွေကတော့ သီးခြား OS မဟုတ်ပဲ Host OS ကိုပဲ ပြန်လည်အသုံးပြုထားခြင်း ဖြစ်ပါ တယ်။ Host OS ကို ပြန်လည်အသုံးပြုထားပေမယ့် ကိုယ်ပိုင် Library နဲ့

Configuration တွေကိုယ်စီရှိကြတဲ့အတွက် မတူကွဲပြားတဲ့ OS များကဲ့သို့ သီးခြား အလုပ်လုပ်ပေးနိုင်ကြခြင်း ဖြစ်ပါတယ်။

Virtual Machine ရဲ့ Guest OS တွေက ကိုယ်ပိုင် Kernel နဲ့ OS တွေဖြစ်ကြတဲ့အတွက် ပိုမိုပြည့်စုံလွပ်လပ်တယ်လို့ ဆိုနိုင်ပါတယ်။ Host OS နဲ့ Guest OS လည်း အမျိုးအစား တူစရာမလိုပါဘူး။ ဒါပေမယ့် Host OS ကို ထည့်သွင်း ထားတဲ့ ကွန်ပျူတာကတော့ အတော်လေး စွမ်းဆောင်ရည်မြင့်မားဖို့လိုပါတယ်။ စွမ်းဆောင်ရည်မြင့်မှသာ Guest OS အများအပြားကို လက်ခံအလုပ်လုပ်ပေးနိုင်မှာ ဖြစ်ပါတယ်။ CPU, Memory အစရှိတဲ့ System Resource တွေကို Host OS ရော၊ Guest OS တွေကပါ စိတ်ပိုင်းရယူအသုံးပြုကြမှာဖြစ်ပါတယ်။ ဥပမာ – RAM 4 GB ရှိတဲ့ ကွန်ပျူတာပေါ်မှာ Host OS အတွက် 2 GB အသုံးပြုမယ်ဆိုရင် RAM 1 GB အသုံးပြုတဲ့ Guest OS နှစ်ခု သာ ထပ်မံထည့်သွင်း အသုံးပြုနိုင်မှာ ဖြစ်ပါတယ်။

Container တွေကတော့ သီးခြား Kernel နဲ့အလုပ်လုပ်တဲ့ OS မဟုတ်တဲ့အတွက် စွမ်းဆောင်ရည်လိုအပ်ချက် သိပ်မရှိပါ ဘူး။ အသင့်အတင့် စွမ်းဆောင်ရည်ရှိတဲ့ Host OS ပေါ်မှာ Container နဲ့ OS Image အမြောက်အများ ထည့်သွင်းအသုံး ပြုနိုင်မှာဖြစ်ပါတယ်။ မတူကွဲပြားတဲ့ Environment လိုချင်ယုံသက်သက်နဲ့ Guest OS ကြီးတစ်ခု ထပ်မံထည့်သွင်းနေစရာ မလိုတော့ပဲ၊ မူလ Host OS ပေါ်မှာပဲ လိုအပ်သလို Setting လုပ်ထားတဲ့ OS Image တွေကို ခွဲခြားအသုံးပြုနိုင်တဲ့ အတွက်ဖြစ်ပါတယ်။ Docker ကို Ubuntu Linux မှာ အခုလို Install ပြုလုပ်နိုင်ပါတယ်။

```
$ sudo apt-get install docker.io
```

boot2docker လို့ခေါ်တဲ့ နည်းပညာအကူအညီနဲ့ Windows, Mac နဲ့ အခြား Operating System တွေမှာလည်း Docker ကို Install ပြုလုပ်နိုင်ပါတယ်။ အောက်ပါလိပ်စာမှာ လေ့လာနိုင်ပါတယ်။

https://docs.docker.com/installation/

> ဆက်လက်ဖော်ပြထားတဲ့ Command နမူနာတွေဟာ လက်တွေ့ စမ်းသပ်လိုရင် ကူးယူ စမ်းသပ်နိုင်တဲ့ နမူနာတွေ ဖြစ်ပါတယ်။

17.3 – Getting Docker Images

Docker ကို Install လုပ်ပြီးတဲ့အခါ အသင့်သုံးလို့ရတဲ့ Docker Image တွေကို Download ရယူထားနိုင်ပါတယ်။ docker pull Command ကို အသုံးပြုရပါတယ်။ Ubuntu Docker Image ကို Install အခုလို ရယူနိုင်ပါတယ်။

```
$ sudo docker pull ubuntu
```

Git နဲ့ ကျွန်တော်တို့ရဲ့ Source Code ကို Version မှတ်တမ်း တင်ထားနိုင်သလိုပဲ Docker Image တွေကိုလဲ

Version အလိုက် မှတ်တမ်းတင် ထားနိုင်ပါတယ်။ နမူနာအရ Version အမျိုးမျိုးပါဝင်တဲ့ Ubuntu Image ကို ရယူပေးသွားမှာ ဖြစ်ပါတယ်။ သက်ဆိုင်ရာ Version တစ်ခုတည်းကိုပဲလိုချင်ရင် အခုလို ရယူရပါတယ်။

```
$ sudo docker pull ubuntu:latest
$ sudo docker pull ubuntu:14.04
```

နမူနာအရ Ubuntu Image ထဲက latest လို့ Tag လုပ်ထားတဲ့ Version နဲ့ 14.04 လို့ Tag လုပ်ထားတဲ့ Version တွေကိုသာ ရယူပေးသွားမှာဖြစ်ပါတယ်။ အသင့်ရယူနိုင်တဲ့ Docker Image စာရင်းကို registry.hub.docker.com မှာ လေ့လာနိုင်သလို `docker search` နဲ့လည်း ရှာဖွေနိုင်ပါတယ်။ ဥပမာ –

```
$ sudo docker search mongo

NAME                            DESCRIPTION                                     STARS     OFFICIAL  AUTOMATED mongo
MongoDB document databases provide high av...   753       [OK]
tutum/mongodb                   MongoDB Docker image - listens in port 2...     33                  [OK]
torusware/speedus-mongo         Always updated official MongoDB docker ima...   7                   [OK]
jacksoncage/mongo               Instant MongoDB sharded cluster                 6                   [OK]
mongooseim/mongooseim-docker    MongooseIM server the latest stable version     2                   [OK]
knickers/mongo-express          Graphically manage your mongoDB container ...   2                   [OK]
19hz/mongo-container            Mongodb replicaset for coreos                   1                   [OK]
… …
asteris/apache-php-mongo        Apache2.4 + PHP + Mongo + mod_rewrite           0                   [OK]
appertly/hhvm-mongo             Docker image of HHVM with compiled Mongofi...   0                   [OK]
ulexus/mongo                    DEPRECATED - please use the official mongo...   0                   [OK]
razmo/mongo                     MongoDB base                                    0                   [OK]
rzani/php-mongo                 PHP and MongoDB (drive)                         0                   [OK]
```

နမူနာမှာ `docker search` နဲ့ `mongo` Image ကို ရှာဖွေလိုက်တဲ့အခါ MongoDB Install လုပ်ထားပြီး အသင့်အသုံးပြုနိုင်တဲ့ Image စာရင်းကို ဖော်ပြပေးတာကို တွေ့ရမှာ ဖြစ်ပါတယ်။ OFFICIAL Column မှာ [OK] ပြထားတဲ့ Image ဟာ Docker Team က ဖန်တီးပေးထားတဲ့ Official Image ဖြစ်ပြီး [OK] မပြထားတဲ့ Image တွေကတော့ အခြားသူတွေ ဖန်တီးပေးထားတဲ့ Unofficial Image တွေဖြစ်ပါတယ်။ Unofficial Image တွေရဲ့အမည်ကို username/image ပုံစံနဲ့ ဖော်ပြတာကိုလည်းသတိပြုကြည့်ပါ။ ကျွန်တော်တို့ကိုယ်တိုင်တည်ဆောက်ထားတဲ့ Image တွေကိုလည်း ဆန္ဒရှိရင် DockerHub မှာ တင်ထားပေးနိုင်မှာဖြစ်ပါတယ်။

လက်ရှိ စက်ထဲမှာရှိနေတဲ့ Image စာရင်းကိုရယူလိုရင် `docker images` ကို အသုံးပြုနိုင်ပါတယ်။

```
$ sudo docker images

REPOSITORY          TAG                 IMAGE ID            CREATED             VIRTUAL SIZE
ubuntu              nginx               906bf9539e9d        4 days ago          206.9 MB
ubuntu              14.04               d2a0ecffe6fa        7 days ago          188.4 MB
hello-world         latest              91c95931e552        3 months ago        910 B
```

နမူနာမှာ `ubuntu` Image နှစ်ခုရှိနေပြီး `hello-world` Image တစ်ခုရှိနေတာကို တွေ့ရမှာဖြစ်ပါတယ်။ `ubuntu` Image နှစ်ခုထဲက တစ်ခုကို `nginx` လို့ TAG လုပ်ထားပြီး နောက်တစ်ခုကိုတော့ 14.04 လို့ TAG လုပ်ထားပါတယ်။

17.4 – Running Image in Docker Container

Image တစ်ခုကို Docker Container ထဲမှာ Run လိုရင် `docker run` ကို အသုံးပြုရပါတယ်။

```
$ sudo docker run ubuntu:14.04 echo 'hello world'

hello world
```

နမူနာအရ `ubuntu:14.04` Image ကို Docker Container ထဲမှာ Run သွားပြီး echo 'hello world' ဆိုတဲ့ Command ကို အဲ့ဒီ Container ထဲမှာ အလုပ်လုပ်စေလိုက်ခြင်း ဖြစ်ပါတယ်။ ရရှိလာတဲ့ hello world Message ဟာ Host OS ပေါ်မှာ အလုပ်လုပ်သွားလို့ ရရှိလာခြင်းမဟုတ်ပဲ Container ထဲမှာ echo Command အလုပ် လုပ်သွားတဲ့အတွက် ရရှိလာခြင်းဖြစ်ပါတယ်။

Container ထဲမှာ Run နေတဲ့ Image ကို Terminal နဲ့ ဝင်ရောက် စီမံလိုရင်တော့ `-t` Option နဲ့ `-i` Option တို့ကို တွဲဖက် အသုံးပြုရပါတယ်။ `-t` က Terminal ဆိုတဲ့အဓိပ္ပါယ်ဖြစ်ပြီး `-i` ကတော့ Interactive ဆိုတဲ့အဓိပ္ပါယ်ပဲ ဖြစ်ပါ တယ်။ `docker run` နဲ့အတူ `-t -i` (သို့မဟုတ်) `-ti` လို့ တွဲဖက်အသုံးပြုနိုင်ပါတယ်။

```
$ sudo docker run -t -i ubuntu:14.04 /bin/bash

root@090b21537ff8:/#
```

နမူနာအရ `ubuntu:14.04` Image ကို Container ထဲမှာ Run ပြီး Command တွေ Run လို့ရတဲ့ Prompt တစ်ခုကို Terminal မှာ ဖော်ပြလာမှာပဲဖြစ်ပါတယ်။ နမူနာအနေနဲ့ NodeJS ကို အခုလို Install လုပ်နိုင်ပါတယ်။

```
root@090b21537ff8:/# apt-get update && apt-get install -y nodejs
```

`apt-get` update နဲ့ Package Repo ကို Update လုပ်လိုက်ပြီး `apt-get` install နဲ့ NodeJS ကို Install လုပ်လိုက်ခြင်းဖြစ်ပါတယ်။ အခုဆိုရင် ကျွန်တော်တို့ရဲ့ `ubuntu:14.04` Image ဟာ မူလ Image နဲ့ မတူတော့ပဲ NodeJS ဆိုတဲ့ Package အသစ်တစ်ခုနဲ့ လိုအပ်တဲ့ Dependency တွေ ဖြည့်စွက်ပါဝင်သွားပြီ ဖြစ်ပါတယ်။

လက်ရှိ အလုပ်လုပ်နေတဲ့ Container စာရင်းကို ရယူလိုရင်တော့ `docker ps` ကို အသုံးပြုနိုင်ပါတယ်။ နောက် Terminal အသစ်တစ်ခုဖွင့်ပြီး အခုလို စမ်းသပ်ကြည့်နိုင်ပါတယ်။

```
$ sudo docker ps

CONTAINER ID  IMAGE          COMMAND       CREATED         STATUS         NAMES
090b21537ff8  ubuntu:14.04   /bin/bash     4 minutes ago   Up 4 minutes   berserk_bohr
```

နမူနာအရ လက်ရှိ `ubuntu:14.04` Image ကို Run နေကြောင်းတွေ့ရမှာဖြစ်ပါတယ်။ ဖော်ပြလာတဲ့စာရင်းရဲ့ ရှေ့ဆုံးမှာ Container Id နဲ့ နောက်ဆုံးမှာ Container Name တို့ပါဝင်လာတာကို သတိပြုပါ။ နမူနာအရ လက်ရှိ Container ကို `berserk_bohr` လို့ Docker က အလိုအလျောက် အမည်တစ်ခုပေးထားပါတယ်။

17.5 – Saving Image Update

NodeJS Install လုပ်ထားတဲ့ လက်ရှိ Container ကို သီးခြား Image တစ်ခုအနေနဲ့ သိမ်းထားလို့ရပါတယ်။ `docker commit` Command ကို အသုံးပြုရပါတယ်။ Save လုပ်လိုတဲ့ Container ရဲ့ ID သို့မဟုတ် Name ကို တွဲဖက်ပေးရပါတယ်။

```
$ sudo docker commit -m 'Installed NodeJS' -a 'Ei Maung' berserk_bohr ubuntu:node

ecc87910db51819c68414d2400e5a2a88b7090c6efb35eb1e7ee681502f8384a

$ sudo docker images

REPOSITORY          TAG                 IMAGE ID            CREATED             VIRTUAL SIZE
ubuntu              node                ecc87910db51        29 seconds ago      221 MB
ubuntu              nginx               906bf9539e9d        4 days ago          206.9 MB
ubuntu              14.04               d2a0ecffe6fa        7 days ago          188.4 MB
hello-world         latest              91c95931e552        3 months ago        910 B
```

အခန်း (၅) မှာ Git အကြောင်းလေ့လာခဲ့စဉ်က Version မှတ်တမ်းတင်ပုံနဲ့ ခပ်ဆင်ဆင်ပါပဲ။ `-m` Option နဲ့ Commit Message ကိုပေးရပြီး `-a` Option နဲ့ Commit ပြုလုပ်သူအမည်ကို ပေးရပါတယ်။ ဆက်လက်ပြီး Container Name (သို့မဟုတ်) ID ကို ပေးရပြီး နောက်ဆုံးက Image အသစ်အတွက် ပေးလိုတဲ့ အမည်ကို ပေးရပါတယ်။

နမူနာအရ Image ကို `ubuntu` ဆိုတဲ့အမည်နဲ့သာသိမ်းပြီး နောက်ကနေ `node` လို့ TAG လုပ်ပေးထားပါတယ်။ ဒါကြောင့် NodeJS Install လုပ်ထားပြီးဖြစ်တဲ့ Ubuntu Image တစ်ခုကို `ubuntu:node` ဆိုတဲ့အမည်နဲ့ ရရှိသွားမှာ ဖြစ်ပါတယ်။

Image တစ်ခုကို အမည်ပြောင်းလိုရင် (သို့မဟုတ်) Tag ပြောင်းလိုရင် `docker tag` ကို အသုံးပြုနိုင်ပါတယ်။

```
$ sudo docker tag ubuntu:node ubuntu:nodejs
$ sudo docker images

REPOSITORY          TAG                 IMAGE ID            CREATED             VIRTUAL SIZE
ubuntu              node                ecc87910db51        4 minutes ago       221 MB
ubuntu              nodejs              ecc87910db51        4 minutes ago       221 MB
ubuntu              14.04               d2a0ecffe6fa        7 days ago          188.4 MB
hello-world         latest              91c95931e552        3 months ago        910 B
```

နမူနာအရ `ubuntu:node` Image ကို `ubuntu:nodejs` အနေနဲ့ အမည်ပြောင်းပေးထားတာကို တွေ့ရမှာဖြစ်ပါ တယ်။ ထူးခြားချက်အနေနဲ့ Docker က `ubuntu:node` ကိုလည်း ဒီအတိုင်း ချန်ထားပေးတာကို တွေ့ရ

နိုင်ပါတယ်။ Image တစ်ခုကို ပယ်ဖျက်လိုရင်တော့ `docker rmi` ကို အသုံးပြုရပါတယ်။

```
$ sudo docker rmi ubuntu:nodejs

Untagged: ubuntu:nodejs

$ sudo docker images

REPOSITORY          TAG                 IMAGE ID            CREATED             VIRTUAL SIZE
ubuntu              node                ecc87910db51        6 minutes ago       221 MB
ubuntu              14.04               d2a0ecffe6fa        7 days ago          188.4 MB
hello-world         latest              91c95931e552        3 months ago        910 B
```

17.6 – Creating Image with Dockerfile

ဖြည့်စွက်ပြင်ဆင်မှုတွေပြုလုပ်ထားတဲ့ Image တစ်ခုကို `docker commit` နဲ့ သီးခြား Image အဖြစ် သိမ်းဆည်းနိုင်သလို Docker Image အသစ်တွေ တည်ဆောက်ဖို့အတွက် Dockerfile လို့ခေါ် တဲ့ Configuration ဖိုင်တစ်မျိုးကို လည်း အသုံးပြုနိုင်ပါတယ်။ ရေးသားပုံနမူနာက ဒီလိုပါ –

Config

```
1. FROM ubuntu:14.04
2. RUN apt-get update && apt-get install -y mongodb
3. RUN mkdir -p /data/db
4. EXPOSE 27017
5. CMD ['/usr/bin/mongod']
```

FROM Keyword ကိုသုံးပြီး အခြေခံရမယ့် Image ကို သတ်မှတ်ပါတယ်။ နမူနာအရ `ubuntu:14.04` Image ပေါ်မှာ Image အသစ်ကို အခြေခံတည်ဆောက်သွားမှာ ဖြစ်ပါတယ်။ RUN Keyword ကို သုံးပြီး Image တည်ဆောက် စဉ်မှာ အလုပ်လုပ်စေလိုတဲ့ Command စာရင်းကို ပေးရပါတယ်။ နမူနာအရ `apt-get` နဲ့ Package Repository ကို Update လုပ်ပြီး `mongodb` ကို Install လုပ်ခိုင်းထားပါတယ်။ MongoDB အလုပ်လုပ်ဖို့အတွက် လိုအပ်နိုင်တဲ့ `/data/db` Directory ကို နောက်ထပ် RUN Keyword တစ်ခုနဲ့ ဆက်လက် တည်ဆောက်ထားပါတယ်။

EXPOSE Keyword ကိုတော့ Container ထဲက Service တွေကို Host OS ကနေ ရယူအသုံးပြုနိုင်အောင် ခွင့်ပြုပေးဖို့ အသုံးပြုရပါတယ်။ နမူနာအရ MongoDB ရဲ့ Default Port ဖြစ်တဲ့ 27017 ကို Host OS ကနေ ရယူအသုံးပြုခွင့် ပြုထားပါတယ်။

CMD Command ကတော့ Image ကို Run တဲ့အခါ အလိုအလျောက် အလုပ်လုပ် သွားစေလိုတဲ့ Command ကို သတ်မှတ်ဖို့ သုံးနိုင်ပါတယ်။ နမူနာအရ MongoDB Server ကို စတင်ပေးနိုင်တဲ့ `mongod` Command ကို Image အလိုအလျောက် Run ပေးဖို့ သတ်မှတ်ထားပါတယ်။

RUN နဲ့ CMD မတူပါဘူး။ RUN က Image တည်ဆောက်စဉ်မှာ အလုပ်လုပ်စေလိုတဲ့ Command တွေသတ်မှတ်ဖို့

အသုံး ပြုရပြီး CMD ကိုတော့ Image ကို Container ထဲမှာ စတင်စဉ် အလုပ်လုပ်စေလိုတဲ့ Command ကို သတ်မှတ်ရခြင်းဖြစ်ပါတယ်။ ဒါကြောင့် Dockerfile တစ်ခု ထဲမှာ RUN Keyword ကို လိုသလောက်အသုံးပြုနိုင်ပေမယ့် CMD Keyword ကိုတော့ တစ်ကြိမ်သာ အသုံးပြုသင့်ပါတယ်။ CMD Keyword တစ်ခုထက် ပိုသုံးထားရင် Docker က နောက်ဆုံးတစ်ခုကိုသာ အသုံးပြုပေးမှာဖြစ်ပါတယ်။

ရရှိလာတဲ့ Dockerfile ကို အသုံးပြုပြီး Image တည်ဆောက်နိုင်ဖို့အတွက် `docker build` ကို သုံးရပါတယ်။ `-t` Option နဲ့ Image ရဲ့ အမည်ကို သတ်မှတ်ပေးနိုင်ပါတယ်။

```
$ sudo docker build -t ubuntu:mongo .
```

နမူနာမှာ နောက်ဆုံးက Dot ကို သတိပြုပါ။ Dot နေရာမှာ Dockerfile တည်ရှိရာ Directory ကို ပေးရမှာဖြစ်ပါတယ်။ နမူနာမှာ `docker build` Command ကို Run တဲ့ Directory ထဲမှာ Dockerfile တည်ရှိတယ်လို့ ယူဆပြီး Dot ကို Location အနေနဲ့ အသုံးပြုထားခြင်း ဖြစ်ပါတယ်။ Build လုပ်ငန်းစဉ်ပြီးမြောက်သွားတဲ့အခါ MongoDB အသင့် Install လုပ်ထားပြီးဖြစ်တဲ့ `ubuntu:mongo` ဆိုတဲ့ Image ကို စတင်အသုံးပြုနိုင်ပြီပဲဖြစ်ပါတယ်။ Dockerfile အတွင်းမှာအသုံးပြုနိုင်တဲ့ ကျန် Keyword တွေကို အောက်ပါလိပ်စာ မှာ ဆက်လက်လေ့လာနိုင်ပါတယ်။

http://docs.docker.com/reference/builder/

Image ကို Build လုပ်ယူပြီးနောက်မှ Dockerfile ကို ပြင်ဆင်ဖြည့်စွက်မယ်ဆိုရင် `docker build` ကို နောက်တစ်ကြိမ် ထပ် Run လိုက်ယုံပါပဲ။ `docker build` က အစအဆုံး Build ပြန်လုပ်မနေပဲ Dockerfile မှာ ပြုလိုက်လိုက်တဲ့ပြင်ဆင်မှုတွေနဲ့အညီ Image ကို Update လုပ်ပေးသွားမှာဖြစ်ပါတယ်။

17.7 – Running Container as Daemon

Image တစ်ခုကို Container ထဲမှာ Run လိုက်ယုံမဟုတ်ပဲ အဲ့ဒီ Container ကို Background Service (Daemon) အနေနဲ့ ဆက်လက် အလုပ်လုပ်စေလိုရင် `-d` Option ကို အသုံးပြုနိုင်ပါတယ်။ ဥပမာ -

```
$ sudo docker run -d -p 27017:27017 ubuntu:mongo
```

နမူနာအရ `ubuntu:mongo` Image ကို Background Service အနေနဲ့ Container ထဲမှာ Run ပေးသွားမှာ ဖြစ်ပါတယ်။ `-p` Option ကတော့ Port Binding အတွက်ဖြစ်ပါတယ်။ `27017:27017` လို့ပေးထားတဲ့အတွက် Host OS ရဲ့ Port 27017 နဲ့ Container ရဲ့ Port 27017 တို့ကို Bind လုပ်ပေးသွားမှာဖြစ်ပါတယ်။ တနည်းအားဖြင့် Container ထဲက Port 27017 မှာအလုပ်လုပ်နေတဲ့ MongoDB Server ကို Host OS ကနေ ရယူအသုံးပြုနိုင်သွားမှာ ဖြစ်ပါတယ်။

လက်ရှိ `docker ps` နဲ့ လက်ရှိအလုပ်လုပ်နေတဲ့ Container စာရင်းကိုရယူလိုက်တဲ့အခါ Background Service အနေနဲ့ Run ထားတဲ့ Container ပါဝင်လာတာကို တွေ့ရမှာပဲဖြစ်ပါတယ်။ Background Service အနေနဲ့ အလုပ်လုပ် နေတဲ့ Container တစ်ခုကို ရပ်လိုရင်တော့ `docker stop` ကို အသုံးပြုနိုင်ပါတယ်။

```
$ sudo docker ps

CONTAINER ID        IMAGE               COMMAND                  PORTS               NAMES
959db209f4cb        ubuntu:mongo        /usr/bin/mongod          27017->27017        silly_archimedes
73f4d6ae4a1a        ubuntu:14.04        /bin/sh -c 'while tr                         trusting_pare

$ sudo docker stop silly_archimedes
```

`docker stop` နဲ့အတူ Container အမည် သို့မဟုတ် ID ကိုပေးပြီး Background Service အဖြစ် Run နေတဲ့ Container ကို ရပ်နိုင်ပါတယ်။ `dock stop` က Container ကို ရပ်ယုံပဲရပ်လိုက်တာပါ၊ အပြီး ပယ်ဖျက်မပြစ်ပါဘူး။ ဒါကြောင့် လိုအပ်လို့ ပြန်စလိုရင် `docker start` နဲ့ ပြန်စနိုင်ပါသေးတယ်။

```
$ sudo docker start silly_archimedes
```

Container တွေထဲက တစ်ခုကို အပြီးဖျက်လိုရင်တော့ `docker rm` ကို သုံးရမှာဖြစ်ပါတယ်။

```
$ sudo docker stop silly_archimedes
$ sudo docker rm silly_archimedes
```

ဖြည့်စွက်မှတ်သားသင့်တာကတော့ `--name` Option ဖြစ်ပါတယ်။ Container တစ်ခုကို Run တဲ့အခါ Container အမည်ကို Docker က အလိုအလျောက် သတ်မှတ်ပေးသွားလေ့ ရှိပါတယ်။ အဲ့ဒီလို Docker ကပေးတဲ့ အမည်ကို မသုံးလိုပဲ ကိုယ်တိုင် Container Name သတ်မှတ်လိုရင် `--name` Option ကို အသုံးပြုနိုင်ခြင်းပဲဖြစ်ပါတယ်။ ဥပမာ –

```
$ sudo docker run -ti --name mymongo ubuntu:14.04 /bin/bash
```

အခုနေ၊ နောက် Terminal တစ်ခုမှာ `docker ps` ကို Run ကြည့်ရင်၊ ကျွန်တော်တို့ သတ်မှတ်ပေးလိုက်တဲ့ `mymongo` ဆိုတဲ့ အမည်ကို Container အမည်အဖြစ် အသုံးပြုပေးထားတာကို တွေ့ရမှာဖြစ်ပါတယ်။

```
$ sudo docker ps

CONTAINER ID        IMAGE               COMMAND             PORTS               NAMES
959db209f4cb        ubuntu:14.04        /bin/bash                               mymongo
```

လက်ရှိ အလုပ်လုပ်နေတဲ့ Container တွေသာ မက Run ခဲ့တဲ့ Container အားလုံးစာရင်းကိုလိုချင်ရင်တော့ `docker ps` ကို `-a` Option နဲ့ တွဲသုံးနိုင်ပါတယ်။

17.8 – Data Volume in Container

Container ထဲမှာ ဖိုင်တွေ Data တွေ သိမ်းဆည်းဖို့အတွက် Volume တွေ သတ်မှတ်ပေးနိုင်ပါတယ်။ ဥပမာ – `/data` ဆိုတဲ့ Volume တစ်ခု Container ထဲမှာ အသုံးပြုလိုရင် အခုလို သတ်မှတ်နိုင်ပါတယ်။

```
$ sudo docker run -d -v /data ubuntu:14.04
```

`-v` Option နဲ့အတူ သတ်မှတ်လိုတဲ့ Volume ကို တွဲဖက်ပေးရခြင်းဖြစ်ပါတယ်။ လက်ရှိ Host OS ထဲက Directory တစ်ခုကို Data Volume အနေနဲ့ အသုံးပြုစေလိုရင်တော့ အခုလို အသုံးပြုနိုင်ပါတယ်။

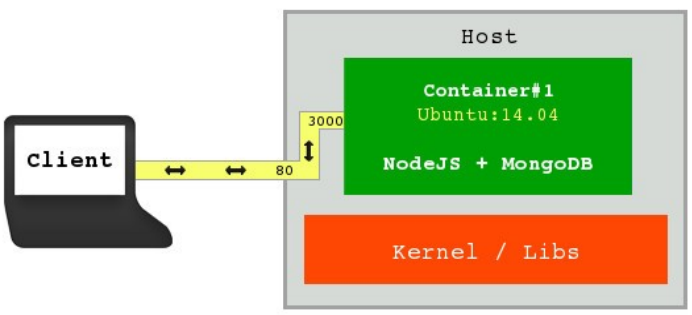
```
$ sudo docker run -d -v /var/www:/data ubuntu:14.04

root@cef44bf69c38:/# cd data
root@cef44bf69c38:/data# ls
sample.txt logo.png
```

နမူနာမှာပေးထားတဲ့ `/var/www` နေရာမှာ အသုံးပြုလိုတဲ့ Host OS Directory တည်နေရာကို ထည့်သွင်းပေးရမှာ ဖြစ်ပါတယ်။ နမူနာအရ Host ရဲ့ `/var/www` ထဲက ဖိုင်တွေဟာ Container ထဲက Image ရဲ့ `/data` မှာ ပါဝင် သွားမှာဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Build လုပ်ထားတဲ့ Project တည်ရှိရာ Directory ကို Docker Container Run တဲ့ အချိန်မှာ Image နဲ့အတူ တခါတည်း ပါဝင်သွားအောင် စီစဉ်ထားနိုင်စေမှာဖြစ်ပါတယ်။

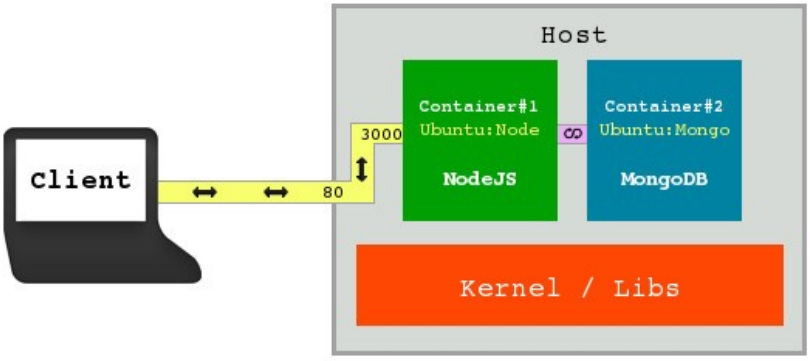
17.9 – Linking container

Service တစ်ခုအလုပ်လုပ်ဖို့အတွက် လိုအပ်တဲ့ Package တွေကို Docker Image တစ်ခုအနေနဲ့စုစည်းထားနိုင်ပါတယ်။ ဥပမာ – NodeJS နဲ့ MongoDB ကို အသုံးပြုထားတဲ့ App တစ်ခုအတွက် NodeJS ရော MongoDB ကိုပါ Image တစ်ခုထဲမှာ တစ်ခါတည်း ထည့်သွင်းထားနိုင်ပါတယ်။



ပုံ (၁၇.၂) - Docker Image with NodeJS + MongoDB

နောက်တစ်နည်းအနေနဲ့၊ လိုအပ်တဲ့ Package တွေကို သီးခြား Image အနေနဲ့ခွဲထားနိုင်ပါတယ်။ ဥပမာ – NodeJS ပါဝင်တဲ့ Image တစ်ခု၊ MongoDB ပါဝင်တဲ့ Image တစ်ခု၊ နှစ်ခုခွဲထားပြီး အဲဒီ Image တွေကို Run ပေးတဲ့ Container တွေကို Link လုပ်ပြီး အသုံးပြုနိုင်ခြင်းဖြစ်ပါတယ်။



ပုံ (၁၇.၃) - Linking Docker Containers

ဥပမာအနေနဲ့ ကျွန်တော်တို့ **အပိုင်း (၄)** မှာ ExpressJS နဲ့ တည်ဆောက်ခဲ့တဲ့ Issue Tracking System API အတွက် Image တစ်ခုကို Dockerfile သုံးပြီး အခုလိုတည်ဆောက်ပါမယ်။

Config

```
1. FROM ubuntu:14.04
2. RUN apt-get update && apt-get install -y nodejs nodejs-legacy
3. RUN mkdir -p /src
4. COPY . /src
5. EXPOSE 3000
6. CMD ["node", "/src/index.js"]
```

`ubuntu:14.04` ကို အခြေခံတဲ့ Image တစ်ခု တည်ဆောက်မှာဖြစ်ပါတယ်။ `nodejs` နဲ့ `nodejs-legacy` ဆိုတဲ့ Package နှစ်ခုကို Install လုပ်ထားပါတယ်။ ပြီးတဲ့အခါ `/src` လို့ခေါ်တဲ့ Directory လည်း တည်ဆောက်ထားပါသေးတယ်။ လိုင်းနံပါတ် (၄) မှာ COPY Keyword နဲ့ လက်ရှိ Host Directory ထဲက ဖိုင်အားလုံးကို Image ရဲ့ `/src` Directory ထဲမှာ ကူးယူထည့်သွင်းပေးဖို့ သတ်မှတ်ထားပါတယ်။ ဒါကြောင့် လက်ရှိ Directory ထဲမှာ ကြိုတင် ရေးသားထားတဲ့ Issue Tracking API Source Code ကို ထားပေးရမှာပါ။ Image ကို Build လုပ်စဉ်မှာ Source Code တွေကို Image ထဲ Docker က ကူးယူထည့်သွင်းပေးသွားမှာ ဖြစ်ပါတယ်။

ဆက်လက်ပြီး Port 3000 ကို Expose လုပ်ဖို့သတ်မှတ်ထားပါတယ်။ Image ဧ Run တဲ့အချိန်မှာ node Command နဲ့ `/src/app.js` ကို Run ပေးဖို့လဲ သတ်မှတ်ထားပါတယ်။ ဒီ Dockerfile ကို docker build နဲ့ အခုလို Build လုပ်ပေးရပါမယ်။

```
$ sudo docker build -t ubuntu:node .
```

ဒီနည်းနဲ့ NodeJS ကို Install လုပ်ထားယုံသာမက Issue Tracking API Source Code ပါ အတစ်ခါတည်းပါဝင်တဲ့ `ubuntu:node Image` ကို ရရှိမှာဖြစ်ပါတယ်။ ရရှိလာတဲ့ Image ကို အခုလို Run ပြီး စတင်အသုံးပြုနိုင်ပြီဖြစ် ပါတယ်။

```
$ sudo docker run -d -p 80:3000 ubuntu:node
```

ဒါပေမယ့် Issue Tracking API က MongoDB ကို အသုံးပြုထားပါတယ်။ လက်ရှိ Image မှာ MongoDB ကို ထည့်သွင်းမထားပါဘူး။ ဒါကြောင့် ပြီးခဲ့တဲ့နမူနာမှာ MongoDB ကို Install လုပ်ထားတဲ့ Image နဲ့ Link လုပ်ပေးဖို့လိုပါ တယ်။ Link လုပ်နိုင်ဖို့အတွက် MongoDB Image ကို အရင် Run ပေးရပါမယ်။

```
$ sudo docker run -d ubuntu:mongo
$ sudo docker ps

CONTAINER ID        IMAGE               COMMAND             PORTS               NAMES
959db209f4cb        ubuntu:mongo        /usr/bin/mongod     27017->27017        silly_archimedes
```

ရရှိလာတဲ့ Container Name ကိုသုံးပြီး `--link` Options နဲ့ ချိတ်ဆက်ပေးနိုင်ပါတယ်။

```
$ sudo docker run -d -p 80:3000 --link silly_archimedes ubuntu:node
```

နမူနာမှာ `ubuntu:node` Image ကို Run တဲ့အခါ `--link` Option နဲ့ စောစောက Run ထားတဲ့ `ubuntu:mongo` ကို ချိတ်ဆက်ထားပါတယ်။ ဒီနည်းနဲ့ `ubuntu:node` ထဲက Issue Tracking API က သီးခြား Container ထဲမှာ Run နေတဲ့ MongoDB ကို အသုံးပြု အလုပ်လုပ်ပေးသွားမှာပဲ ဖြစ်ပါတယ်။

Conclusion

Production Server Environment တစ်ခုတည်ဆောက်ရာမှာ Challenge (၃) ခုရှိနိုင်တယ်လို့ အစပိုင်းမှာ ဖော်ပြခဲ့ပါ တယ်။ ဒီပြဿနာတွေကို Docker အကူအညီနဲ့ အောက်ပါအတိုင်း ဖြေရှင်းနိုင်ပါတယ်။

1. Software Project အတွက် Development Environment တည်ဆောက်ဖို့အတွက် လိုအပ်တဲ့ Package စာရင်းကို Dockerfile တစ်ခုအနေနဲ့ ကြိုတင်သတ်မှတ်ထားသင့်ပါတယ်။
2. ရရှိလာတဲ့ Dockerfile ကို သုံးပြီး Image တစ်ခု Build လုပ်ထားနိုင်ပါတယ်။ ဆောင်ရွက်ရမယ့် Development လုပ်ငန်းတွေကို အဲ့ဒီ Image ပေါ်မှာ ဆက်လက်ဆောင်ရွက်ရမှာ ဖြစ်ပါတယ်။
3. လိုအပ်လို့ Package တွေ Upgrade ပြုလုပ်ရမယ် (သို့မဟုတ်) အသစ်ထည့်သွင်းရမယ်ဆိုရင်လည်း Manual မထည့်သွင်းပဲ Dockerfile ထဲမှာသာ ဖြည့်စွက်ပြင်ဆင်သင့်ပါတယ်။ ဒီနည်းနဲ့ လက်ရှိ Software Project အတွက် လိုအပ်တဲ့ Package စာရင်းက Dockerfile တစ်ခုအနေနဲ့ အမြဲအသင့်ရှိနေမှာ ဖြစ်ပါတယ်။

4. Development လုပ်ငန်းပြီးလို့ Production Environment တည်ဆောက်ဖို့လိုတဲ့အခါ၊ အဲ့ဒီ Dockerfile ကို Server ပေါ်မှာ Build လုပ်ခြင်းအားဖြင့် Software Project အတွက် လိုအပ်တဲ့ Production Environment ကို တစ်ဆင့်တည်းနဲ့ အလွယ်တစ်ကူရရှိသွားမှာဖြစ်ပါတယ်။
5. Development ရော Production ကပါ၊ Dockerfile တစ်ခုတည်းပေါ်မှာ အခြေခံထားတဲ့အတွက် Environ-ment မတူတဲ့ပြဿနာ ပြေလည်သွားမှာဖြစ်ပါတယ်။
6. Development Machine မှာပြုလုပ်တဲ့ Package အပြောင်းအလဲရှိခဲ့ရင်လည်း ရရှိလာတဲ့ Update Dockerfile ကို Production Server မှာ ထပ်မံ Build လုပ်ပေးခြင်းအားဖြင့် Production Environment လည်း လိုအပ် သလို လိုက်လံပြောင်းလဲသွားမှာပဲ ဖြစ်ပါတယ်။

SOA နည်းစနစ်နဲ့ ခွဲခြားတည်ဆောက်ထားတဲ့ Service တွေကိုလည်း၊ ဘယ်လို Environment ပေါ်မှာ အခြေခံ အလုပ်လုပ် သည်ဖြစ်စေ၊ လိုအပ်တဲ့ Package တွေ တစ်ခါတည်းပါဝင်တဲ့ Docker Image များအဖြစ် ဖန်တီး ထား ခြင်းအားဖြင့် Server တစ်ခုတည်းပေါ်မှာ Environment မတူတဲ့ Service တွေကို အတူတစ်ကွ ပူးတွဲအလုပ် လုပ်စေနိုင် မှာပဲဖြစ်ပါ တယ်။ ဥပမာ – Service တစ်ခုက NodeJS 0.12.2 ပေါ်မှာ အခြေခံအလုပ်လုပ်တယ်ဆို ရင် သူရဲ့ Docker Image ထဲ မှာ NodeJS 0.12.2 ကို ထည့်သွင်းထားနိုင်ပါတယ်။ နောက်တစ်ခုက NodeJS 1.0 လိုအပ်တယ်ဆိုရင်လည်း သူ့ရဲ့ Docker Image ထဲမှာ NodeJS 1.0 ကို ထည့်သွင်းထားနိုင်ပါတယ်။ Image နှစ်ခု ဟာ သီးခြား Container တွေထဲ မှာ သီးခြားအလုပ်လုပ်ကြတာဖြစ်လို့ ဘာပြဿနာမှ ရှိမှာမဟုတ်ပဲ Server တစ် ခုတည်းပေါ်မှာ ပူးတွဲ အလုပ်လုပ်နိုင်သွားမှာပဲ ဖြစ်ပါတယ်။

ဒီလိုအားသာချက်တွေကြောင့်ပဲ Docker ဟာ အခုနောက်ပိုင်းမှာ Developer တွေအကြား အထူးထင်ရှားလာ ခြင်းဖြစ်ပါတယ်။ Docker ရဲ့အောင်မြင်မှုကြောင့်၊ RedHat, VMWare, Ubuntu, Microsoft စတဲ့ လုပ်ငန်းတွေ လည်းပဲ Docker ကဲ့သို့ LXC အခြေပြု Container နည်းပညာတွေကို လိုက်လံတီထွင်လာကြပြီဖြစ်တဲ့အတွက် အနာဂါတ်မှာ အခုထက်ပိုမို ဖွံ့ဖြိုးတိုးတက်လာပြီး ပိုမိုအသုံးဝင်လာဦးမယ့် နည်းပညာတစ်ခုပဲဖြစ်ပါတယ်။

ရုတ်တရက်နာမည်ရသွားတဲ့ App တစ်ခုဟာ နေ့ချင်းညချင်း
အသုံးပြုသူ ဆယ်ဆလည်းတိုးသွားနိုင်တယ်။ အဆတစ်ရာလည်း
တိုးသွားနိုင်တယ်။ ဒီလိုအခြေအနေ ကြုံလာခဲ့ရင် လက်ခံအလုပ်လုပ်
ပေးနိုင်တဲ့ Server Architecture ကို ကြိုတင်
ပြင်ဆင်ထားရပါမယ်။

Professional Web Developer Course

HTML5, PHP/MySQL, jQuery/Ajax, Mobile Web စသည့်
Professional Web Developer တစ်ဦး သိရှိထားသင့်သည့်
နည်းပညာများကို စုစည်းသင်ကြားခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၁၈) – Load Balancing with Nginx

ဘယ်လောက်ပဲ Performance ကောင်းတဲ့ Hardware တွေကို အသုံးပြုထားပါစေ၊ Server တစ်လုံးရဲ့ လက်ခံအလုပ်လုပ် ပေးနိုင်တဲ့ ပမာဏကတော့ အကန့်အသတ် ရှိကြတာချည်းပါပဲ။ Host လုပ်ထားတဲ့ Software Application (သို့မဟုတ်) Service ရဲ့ အနေအထားပေါ်မူတည်ပြီး တစ်စက္ကန့်ကို Request ပေါင်း ထောင်ဂဏန်းလည်း ဖြစ်နိုင်ပါတယ်။ သောင်း ဂဏန်းလည်း ဖြစ်နိုင်ပါတယ်။ ကနေ့ခေတ်မှာ ရုတ်တရက် နာမည်ရသွားတဲ့ App တစ်ခုဟာ နေ့ချင်းညချင်း အသုံးပြုသူ သန်းချီရရှိသွားနိုင်ပါတယ်။ ဒီလိုအခြေအနေမျိုး ကြုံလာတဲ့အခါ Server က လက်ခံအလုပ်လုပ်ပေးနိုင်တဲ့ ပမာဏကို ကျော် နေတဲ့အတွက် အလုပ်မလုပ်နိုင်တော့ဘူးဆိုတာမျိုး မဖြစ်ရလေအောင် App ကို တစ်လုံးထက်ပိုတဲ့ Server တွေမှာခွဲထား ပြီး မျှသုံးပေးဖို့လိုအပ်လာမှာဖြစ်ပါတယ်။ ဒီအခန်းမှာ၊ အဲ့ဒီလို တစ်လုံးထက်ပိုတဲ့ Server တွေကနေ မျှသုံးပြီး အလုပ်လုပ် ပေးနိုင်တဲ့ Load Balance နည်းပညာတစ်ခုဖြစ်တဲ့ Nginx အကြောင်းကို ဖော်ပြပေးသွားမှာပဲဖြစ်ပါတယ်။



Nginx ဟာ လက်ရှိမှာ Apache ပြီးရင် ဒုတိယမြောက် လူသုံးအများဆုံး Web Server ဖြစ်ပါတယ်။ Engine X လို့ အသံထွက်ရပါတယ်။ ရုရှားလူမျိုး ပညာရှင် တစ်ဦးဖြစ်တဲ့ Igor Sysoev က ၂၀၀၄ ခုနှစ်မှာ စတင်တီထွင်ခဲ့ခြင်းဖြစ်ပြီး Open Source နည်းပညာတစ်ခုဖြစ်ပါတယ်။ Resource လိုအပ်ချက် နည်းနည်းနဲ့ စွမ်းဆောင်ရည် ကောင်းမွန်မှုပိုင်းကို အဓိကရည်ရွယ်ဖန်တီးထားတဲ့ Web Server ဖြစ်တဲ့အတွက် သူ့ကို Web Server အဖြစ်သာမက၊ Reverse Proxy, Load Balancer, Cache Server စတဲ့ လုပ်ငန်းတွေအတွက်ပါ အသုံးပြုကြပါတယ်။

Ubuntu မှာ Nginx ကို အခုလို Install လုပ်နိုင်ပါတယ်။

```
$ sudo apt-get install nginx
```

Source Code (သို့မဟုတ်) Windows အတွက် Nginx Installer ကိုတော့ အောက်ပါလိပ်စာမှာ ရယူနိုင်ပါတယ်။

http://nginx.org/en/download.html

Install လုပ်ပြီးတဲ့အခါ Nginx Web Server ကို အခုလို Run နိုင်ပါတယ်။

```
$ sudo nginx
```

Ubuntu မှာ `apt-get` နဲ့ Install လုပ်ထားတာဆိုရင် Ubuntu က အလိုအလျောက် Run ပေးထားမှာဖြစ်လို့

ကိုယ်တိုင် Run ပေးစရာမလိုပါဘူး။ Nginx ကို ရပ်စေလိုရင်တော့ `nginx -s stop` (သို့မဟုတ်) `nginx -s quit` ကို အသုံးပြုနိုင်ပါတယ်။ `stop` က Nginx ကို ချက်ခြင်းရပ်လိုက်မှာဖြစ်ပြီး၊ `quit` Signal ကတော့ လက်ရှိအလုပ်လုပ်နေတဲ့ Request တွေ ပြီးစီးအောင် စောင့်ပြီးတော့မှ ရပ်ပေးမှာဖြစ်ပါတယ်။ ဥပမာ –

```
$ sudo nginx -s quit
```

Nginx Server Configuration ဖိုင်တွေကို ပြင်ဆင်ပြီးနောက် Nginx ကို Restart မလုပ်ပဲ Configuration တွေကိုပဲ Reload လုပ်စေလိုရင်တော့ reload Signal ကို အသုံးပြုရပါတယ်။

```
$ sudo nginx -s reload
```

18.1 – Nginx Configuration

Ngix Configuration တွေကို Simple Directive နဲ့ Block Directive ဆိုပြီး ပုံစံနှစ်မျိုးနဲ့ ရေးသားနိုင်ပါတယ်။ Simple Directive တွေကို `name parameter` Format နဲ့ရေးရပြီး လိုင်းအဆုံးမှာ Semi-colon နဲ့ ပိတ်ပေးရပါတယ်။ Block Directive တွေကိုတော့ `name` နောက်က တွန့်ကွင်း အဖွင့်အပိတ် Block နဲ့ ရေးပေးရပါတယ်။ Block ထဲမှာ လိုအပ်တဲ့ Instruction တွေကို စုစည်းထည့်သွင်းနိုင်ပါတယ်။ # Sign နဲ့စတဲ့လိုင်းတွေကတော့ Comment တွေဖြစ်ပါတယ်။

> ဒီအခန်းမှာ ဖော်ပြထားတဲ့ Configuration နမူနာတွေဟာ တိုက်ရိုက်ကူးယူ စမ်းသပ်ဖို့မဟုတ်ပါ။ Nginx ရဲ့ သဘောသဘာဝကို ပေါ်လွင်အောင် Simplify လုပ်ထားတဲ့ နမူနာများဖြစ်ပါတယ်။ ဒါကြောင့် သဘောသဘာဝကိုသာ နားလည်အောင်လေ့လာပြီး လက်တွေ့ကိုတော့ ကိုယ်တိုင်ဆက်လက် လေ့လာ သွားရမှာ ဖြစ်ပါတယ်။

Directive တွေထဲမှာ အဓိကသတိပြုသင့်တာတွေကတော့ `http`, `server` နဲ့ `location` တို့ပဲဖြစ်ပါတယ်။ ပုံမှန် အားဖြင့်၊ ဒီ Directive (၃) ခုရဲ့ဖွဲ့စည်းပုံက အခုလိုရှိတက်ပါတယ်။

Config

```
1. http {
2.  gzip on;
3.  server {
4.          listen 80
5.          server_name example.com
6.          location / {
7.                  # Configuration
8.          }
9.  }
10.
```

```
  server {
        listen 80
        server_name example.net
        location / {
              # Configuration
        }

        location /images {
              # Configuration
        }
  }
}
```

နမူနာကိုလေ့လာကြည့်ရင် `http` က Main Block ဖြစ်တယ်ဆိုတာကို တွေ့ရမှာဖြစ်ပါတယ်။ `http` Block ထဲမှာ အခြား Setting တွေနဲ့အတူ `server` Block တွေရှိနိုင်ပါတယ်။ `server` Block တွေထဲမှာလည်း အခြား Setting တွေနဲ့အတူ `location` Block တွေရှိတက်ပါတယ်။ `server` Block တွေကို Virtual Host တွေ သတ်မှတ်ဖို့အတွက် အသုံးပြုနိုင်ပြီး `location` Block တွေကိုတော့ Routing အတွက် အသုံးပြုနိုင်ပါတယ်။ Nginx Server တစ်ခုလုံးနဲ့ သတ်ဆိုင်တဲ့ Setting တွေကို `http` Block ထဲမှာ ရေးရပြီး သက်ဆိုင်ရာ Virtual Web Server အတွက် Setting တွေကိုတော့ `server` Block ထဲမှာ ရေးပေးရပါတယ်။ အလားတူပဲ `location` တစ်ခုနဲ့သာ သက်ဆိုင်စေလိုတဲ့ Setting တွေကိုတော့ `location` Block ထဲမှာ ရေးပေးရပါတယ်။

Ubuntu Linux မှာဆိုရင်၊ Nginx ရဲ့ Main Configuration တွေဟာ `/etc/nginx/nginx.conf` ဖိုင်ထဲမှာ တည်ရှိနေမှာ ဖြစ်ပါတယ်။ အဲ့ဒီဖိုင်ကို Text Editor နဲ့ ဖွင့်ကြည့်ရင် အခုလိုတွေ့ရနိုင်ပါတယ်။

Config

```
user www-data;
worker_processes 4;
pid /run/nginx.pid;

events {
  worker_connections 768;
  # multi_accept on;
}

http {

  ##
  # Basic Settings
  ##

  sendfile on;
  tcp_nopush on;
  tcp_nodelay on;
  keepalive_timeout 65;
  types_hash_max_size 2048;

  include /etc/nginx/mime.types;
  default_type application/octet-stream;

  access_log /var/log/nginx/access.log;
```

```
26.   error_log /var/log/nginx/error.log;
27.
28.   gzip on;
29.   gzip_disable "msie6";
30.
31.   include /etc/nginx/conf.d/*.conf;
32.   include /etc/nginx/sites-enabled/*;
33. }
```

`http` Block ရဲ့ အပြင်ဖက်မှာ မဖြစ်မနေပါဝင်ဖို့လိုအပ်တဲ့ အခြေခံ Setting တွေ ရှိပါတယ်။ `http` Block ထဲက နမူနာလိုင်းနံပါတ် (၃၂) မှာတော့ `include` Directive နဲ့ `/etc/nginx/sites-enabled` Directory ထဲက Configuration ဖိုင်တွေကို ချိတ်ဆက်ပေးထားတာကို တွေ့ရမှာဖြစ်ပါတယ်။ အဓိပ္ပါယ်ကတော့၊ `server` Block တွေကို Main Configuration ဖိုင်ထဲမှာ ရောမရေးပဲ `sites-enabled` Directory ထဲမှာ၊ သီးခြားဖိုင်များအဖြစ် ခွဲရေးထား ခြင်းဖြစ်ပါတယ်။ `sites-enabled` Directory က `default` ဖိုင်ကို ဖွင့်ကြည့်ရင်အခုလို တွေ့ရ နိုင်ပါတယ်။

Config

```
1. server {
2.   listen 80 default_server;
3.   listen [::]:80 default_server ipv6only=on;
4.
5.   root /usr/share/nginx/html;
6.   index index.html index.htm;
7.
8.   server_name localhost;
9.
10.  location / {
11.         try_files $uri $uri/ =404;
12.  }
13. }
```

နမူနာမှာ Comment တွေဖယ်ပြီးဖော်ပြထားပါတယ်။ စာဖတ်သူဖွင့်ကြည့်တဲ့အခါမှာတော့ Comment တွေ လည်း ပါဝင်နေနိုင်ပါတယ်။ `server` Block ထဲက လိုင်းနံပါတ် (၅) မှာ `root` Directive ကိုသုံးပြီး /usr/share/nginx/html Directory ကို Document Root အဖြစ် သတ်မှတ်ထားတာကို တွေ့ရမှာဖြစ်ပါတယ်။ ဒါ ကြောင့် HTML နဲ့ အခြား Static Document တွေကို /user/share/nginx/html Directory ထဲမှာ ထားပေးရမှာပါ။ လိုင်းနံပါတ် (၆) မှာ index Directive နဲ့ index.html နဲ့ index.htm တို့ကို Directory Index အနေနဲ့ အသုံးပြုပေးဖို့ သတ်မှတ်ထားပါတယ်။

Document Root ကို Location အလိုက် ခွဲခြားသတ်မှတ်လိုရင် အခုလိုသတ်မှတ်နိုင်ပါတယ်။

Config

```
location /static {
  root /usr/share/nginx/html
}

location /images {
  root /usr/share/nginx/html/images
}
```

URI မှာ `/static` စတဲ့ Request တွေအတွက် အသုံးပြုရမယ့် Document Root ကတစ်ခုဖြစ်ပြီး URI မှာ `/images` နဲ့ စတဲ့ Request တွေအတွက် Document Root တစ်ခုကို သီးခြားသတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။

`default` Configuration ရဲ့ လိုင်းနံပါတ် (၁၁) မှာသတ်မှတ်ထားတဲ့ `try_files $uri $uri/ =404` ရဲ့ အဓိပ္ပါယ်ကတော့ Document Root ထဲမှာ URI Path နဲ့ ကိုက်ညီတဲ့ဖိုင် (သို့မဟုတ်) Directory ရှိရင် Response ပြန်ပေးပြီး မရှိနေရင်တော့ 404 ကို Response ပြန်ပေးဖို့ သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။ တနည်းအားဖြင့် Nginx ကို Static File Server တစ်ခုအနေနဲ့ အလုပ်လုပ်စေဖို့ ဒီ Directive ကို ထည့်သွင်းထားခြင်းပဲဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၈) မှာတော့ `server_name` ကို `localhost` လို့ သတ်မှတ်ထားပါတယ်။ အကယ်၍ Server မှာ Domain Name ရှိတယ်ဆိုရင် `localhost` အစား သတ်ဆိုင်ရာ Domain Name ကို သတ်မှတ်ပေးရမှာ ဖြစ်ပါတယ်။ အကယ်၍ Domain Name နှစ်ခုရှိတယ်ဆိုရင် သက်ဆိုင်ရာ Domain Name အတွက် Virtual Host တွေကို အခုလို ခွဲခြားသတ်မှတ် ပေးနိုင်ပါတယ်။

Config

```
server {
  server_name example.com;
  root /var/www/example.com;
  index index.html index.htm;
  location / {
        try_files $uri $uri/ =404
  }
}

server {
  server_name example.net;
  root /var/www/example.net;
  index index.html index.htm;
  location / {
        try_files $uri $uri/ =404
  }
}
```

ဒီနည်းနဲ့ example.com အတွက် Document Root တစ်ခုနဲ့ exmaple.net အတွက် Document Root တစ်ခု၊ သီးခြားစီ ခွဲခြားသတ်မှတ်ထားနိုင်ပါတယ်။

ကျွန်တော်တို့ **အခန်း (၁၃)** မှာ ExpressJS နဲ့ API Server တည်ဆောက်ခဲ့စဉ်က Cross Origin Resource Sharing နဲ့ပက်သက်တဲ့ Header တွေ သတ်မှတ်ခဲ့ရတာကို မှတ်မိဦးမှာပါ။ Nginx မှာလည်း အလားတူ Default Header တွေ သတ်မှတ်လိုတယ်ဆိုရင် `add_header` Directive ကို အသုံးပြုနိုင်ပါတယ်။

Config

```
location / {
  add_header 'Access-Control-Allow-Origin' '*';
  add_header 'Access-Control-Allow-Credentials' 'true';

  try_files $uri $uri/ =404
}
```

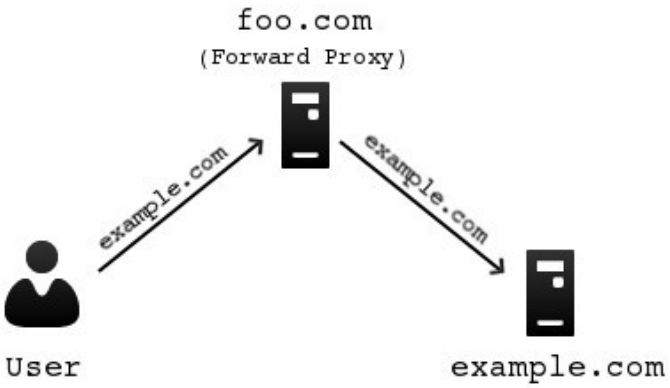
နမူနာမှာပေးထားသလို `location` Block ထဲမှာရေးရင်တော့ သက်ဆိုင်ရာ Location အတွက်ပဲ အလုပ်လုပ်မှာပါ။ အကယ်၍ Web Server တစ်ခုလုံးအတွက် အလုပ်လုပ်စေလိုရင်တော့ `server` Block ထဲမှာ ရေးသားပေးသင့်ပါတယ်။ Nginx Directive တစ်ခုချင်းစီရဲ့ အဓိပ္ပါယ်နဲ့ အသုံးပြုပုံတွေကို အောက်ပါလိပ်စာမှာ ဆက်လက်လေ့လာနိုင် ပါတယ်။

http://nginx.org/en/docs/http/ngx_http_core_module.html

18.2 – Proxy

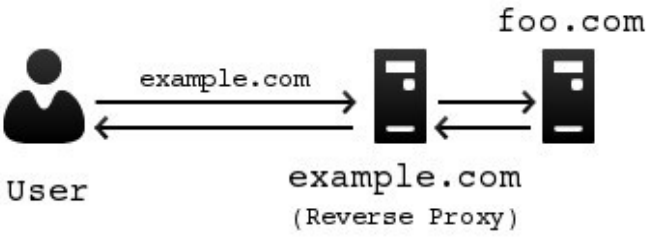
Ngix ရဲ့ လုပ်ဆောင်ချက်တွေထဲက အရေးပါတဲ့ လုပ်ဆောင်ချက်ကတော့ Proxy ဖြစ်ပါတယ်။ Proxy Server တစ်ခုရဲ့ အခြေခံ အလုပ်လုပ်ပုံကတော့၊ အသုံးပြုသူ User တွေပေးပို့တဲ့ Request ကို ကြားထဲကနေလက်ခံပြီး App Server ထံ လက်ဆင့်ကမ်းပေးပို့ခြင်းပဲဖြစ်ပါတယ်။

Proxy မှာ Forward Proxy နဲ့ Reverse Proxy ဆိုပြီး နှစ်မျိုးရှိပါတယ်။ Forward Proxy ဆိုတာ အသုံးပြုသူ "Client ကိုယ်စား" ကြားကနေ တစ်ဆင့်ဆောင်ရွက်ပေးတဲ့ Proxy ဖြစ်ပါတယ်။ ဥပမာ – Client က example.com ကို သွားချင် ပေမယ့် တိုက်ရိုက်မသွားပဲ foo.com ကို တစ်ဆင့်သွားမယ်၊ foo.com က Client ရဲ့ Request ကို example.com ထံ လက်ဆင့်ကမ်းပေးမယ်ဆိုရင် foo.com ကို Forward Proxy လို့ခေါ်ပါတယ်။ တစ်ကယ်အလုပ်လုပ်တာက example.com ဖြစ်ပြီး foo.com က ကြားခံတစ်ခုသာဖြစ်ပါတယ်။



ပုံ (၁၈.၁) - Forward Proxy

Reverse Proxy ဆိုတာကတော့ Request ကို လက်ခံအလုပ်လုပ်မယ့် "Server ကိုယ်စား" ကြားကနေ ကြားဖြတ် လက်ခံ ပေးတဲ့ Proxy အမျိုးအစားဖြစ်ပါတယ်။ ဥပမာ – Client က example.com ကို Request လုပ်ပေမယ့်၊ example.com လက်ခံရရှိတဲ့ Request ကို foo.com ထံ လက်ဆင့်ကမ်းပေးလိုက်တယ်ဆိုရင် example.com ဟာ Reverse Proxy ဖြစ် ပါတယ်။ တစ်ကယ်အလုပ်လုပ်တဲ့ App Server က foo.com ဖြစ်ပြီး example.com က ကြားခံတစ်ခုသာ ဖြစ်ပါတယ်။ ထူးခြားတာက၊ User အနေနဲ့ foo.com ရှိနေတာကို သိစရာမလိုပဲ example.com ကိုသာ App Server အမှတ်နဲ့ ဆက်သွယ်အသုံးပြုရမှာဖြစ်ပါတယ်။ ကျွန်တော်တို့ ဒီနေရာမှာ ဖော်ပြလိုတာ ကတော့ Reverse Proxy အကြောင်းပဲဖြစ်ပါ တယ်။



ပုံ (၁၈.၂) - Reverse Proxy

ဒီလို Proxy Server တစ်ခုနဲ့ ကြားကနေ Request တွေကို လက်ခံပေးခြင်းအားဖြင့် ရရှိလာတဲ့ အကျိုးရလဒ် ကတော –

၁.) လုံခြုံရေးနဲ့ပက်သက်တဲ့ ကိစ္စတွေကို App Server မှာဝန်မပိစေပဲ ကြားဖြတ်ဆောင်ရွက်နိုင်ပါတယ်။ ဥပမာ – Access လုပ်ခွင့်မရှိတဲ့ Client တွေက ပြုလုပ်လာတဲ့ Request တွေကို App Server ထံရောက်စရာမလိုပဲ ကြား ကနေ Proxy Server က ဖြတ်တောက်လိုက်နိုင်ပါတယ်။

၂.) စွမ်းဆောင်ရည် ပိုမိုကောင်းမွန်စေဖို့အတွက် Proxy Cache အဖြစ် ကြားခံဆောင်ရွက်ပေးနိုင်ပါတယ်။ အကယ်၍ Client လိုချင်တဲ့ Resource က Proxy Cache မှာရှိနေတယ်ဆိုရင် App Server ထံကနေ ရယူနေစရာ မလိုပဲ Proxy Cache ကနေ ပြန်ပေးနိုင်ပါတယ်။ ဒီနည်းနဲ့ ထပ်ခါထပ်ခါ ရယူဖို့လိုတဲ့တူညီတဲ့ Content တွေ အတွက်နဲ့ App Server ပေါ် ဝန်ပိစရာမလိုတော့ပါဘူး။

၃.) App Server အများအပြားရှိတဲ့ထဲက လက်ရှိအချိန်မှာ Request ကို လက်ခံဆောင်ရွက်ဖို့ အဆင်အပြေဆုံးနဲ့ အသင့် တော်ဆုံး Server ထံ လက်ဆင့်ကမ်းပေးပို့ခြင်းအားဖြင့်၊ App Server အများအပြားကို မျှသုံးပေးတဲ့ Load Balance လုပ်ဆောင်ချက်ကို ရရှိနိုင်မှာ ဖြစ်ပါတယ်။

Nginx ရဲ့ Proxy လုပ်ဆောင်ချက်ကို အသုံးပြုဖို့အတွက် `proxy_pass` Directive ကို အသုံးပြုနိုင်ပါတယ်။ ဥပမာ –

Config

```
1. location /api {
2.  proxy_pass http://foo.com:3000/api;
3. }
```

နမူနာအရ `/api` URI ကို လက်ခံရရှိတဲ့အခါ Nginx က foo.com:3000/api ကို အလိုအလျောက် လက်ဆင့်ကမ်းပေးသွားမှာဖြစ်ပါတယ်။ အကယ်၍ URI က `/api/issues` ဖြစ်ခဲ့မယ်ဆိုရင် လက်ဆင့်ကမ်းပေးသွားမယ့် URL က http://foo.com:3000/api/issues ဖြစ်မှာဖြစ်ပါတယ်။

App Server ထံ Request ကို Forward မလုပ်ခင်၊ တစ်ချို့ လုံခြုံရေး စစ်ဆေးမှုတွေ ပြုလုပ်လိုတယ်ဆိုရင် အခုလို ဆောင်ရွက်နိုင်ပါတယ်။

Config

```
1. if ($http_user_agent ~* (wget|crawler) ) {
2.  return 403;
3. }
4.
5. location /api {
6.  deny 192.168.1.2;
7.  proxy_pass http://foo.com:3000/api;
8. }
```

နမူနာအရ User Agent က Search Engine Crawler (သို့မဟုတ်) Wget ပရိုဂရမ် ဖြစ်နေမယ်ဆိုရင် 403 Forbidden ကို Response လုပ်ပေးမှာဖြစ်ပြီး IP Address 192.168.1.2 ကနေ Request ပြုလုပ်လာရင်လည်း Deny လုပ်သွားမှာဖြစ်ပါတယ်။ နမူနာအဖြစ်သာ ဖော်ပြထားခြင်းဖြစ်ပြီး လက်တွေ့မှာ လုံခြုံရေးအရ စီစစ်လိုတဲ့ IP Address တွေနဲ့ User Agent တွေကို Black List စာရင်းတစ်ခုလုပ်ထားပြီး အဲ့ဒီစာရင်းနဲ့ တိုက်ဆိုင်စီစစ်သွားရမှာ ဖြစ်ပါတယ်။

Request တွေကို လက်ဆင့်ကမ်းပေးပို့တဲ့အခါ Request Header ကို အနည်းငယ်သတိထားရပါမယ်။ ဥပမာ – example.com မှာ Proxy Server ကို Setup လုပ်ထားပြီး API Server ကို foo.com မှာ Setup လုပ်ထားတယ်ဆိုကြပါစို့။ User က example.com ကို Request ပြုလုပ်လိုက်တဲ့အခါ example.com Proxy Server က foo.com ကို အလိုအလျောက် Request ကို လက်ဆင့်ကမ်းပေးသွားမှာဖြစ်ပါတယ်။ ဒီတော့ စဉ်းစားကြည့်ရင် foo.com ကို အမှန်တစ်ကယ် Request ပြုလုပ်သူက User မဟုတ်တော့ပဲ example.com ဖြစ်နေမှာဖြစ်ပါတယ်။ လိုအပ်ချက်အရ Request ပြုလုပ်သူ စာရင်းကို မှတ်တမ်းလုပ်ထားချင်ရင် မရတော့ပါဘူး။ foo.com ကို Request ပြုလုပ်သူဟာ အမြဲတမ်း example.com ပဲဖြစ်နေမှာဖြစ်ပါတယ်။ ဒီပြဿနာမျိုးကို ဖြေရှင်းနိုင်ဖို့ အတွက် Proxy Request နဲ့အတူ Request Header ကို ပြင်ဆင်ပေးဖို့ဖို့ လိုတက်ပါတယ်။ ပြင်ဆင်ပေးဖို့ လိုလေ့ရှိတဲ့ Header အချို့ကို ဖော်ပြလိုက်ပါတယ်။

Config

```
1. proxy_set_header HOST $host;
2. proxy_set_header X-Forwarded-Proto $scheme;
3. proxy_set_header X-Real-IP $remote_addr;
4. proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
5.
6. location /api {
7.  proxy_pass http://foo.com/api;
8. }
```

Nginx က `$host` Variable ထဲမှာ မူလ Request ပြုလုပ်ခဲ့တဲ့ Host အမည်ကို သိမ်းထားပေးပါတယ်။ အထက်က နမူနာအရ `$host` Variable ထဲကတန်ဖိုးက example.com ဖြစ်မှာပါ။ Proxy က Request ကို လက်ဆင့်ကမ်း ပေးပို့တဲ့အခါ HOST Header အတွက် `proxy_pass` မှာပေးလိုက်တဲ့ Host Name ကို အသုံးပြုပေးမှာပါ။ ဒါကြောင့် HOST Header တန်ဖိုးက Client ပေးပို့စဉ်မှာ example.com ဖြစ်ပေမယ့် Proxy က foo.com လို့ပြောင်းပြီး လက်ဆင့် ကမ်းသွားမှာဖြစ်ပါတယ်။ ဒါကြောင့် HOST Header တန်ဖိုးကို မပြောင်းပဲ မူလအတိုင်းပဲ လက်ဆင့်ကမ်းစေလိုတဲ့အခါ နမူနာ လိုင်းနံပါတ် (၁) မှာပေးထားသလို `proxy_set_header` Directive ကိုသုံးပြီး HOST `$host` လို့ သတ်မှတ်ပေးနိုင်ပါတယ်။ လိုင်းနံပါတ် (၂) မှာ သတ်မှတ်ထားတဲ့ X-Forwarded-Proto Header ကတော့ Standard HTTP Header မဟုတ်ပဲ ကိုးကားနိုင်ဖို့အတွက် ကိုယ့်သဘောနဲ့ကိုယ် ထည့်သွင်းပေးလိုက်တဲ့ Custom Header ဖြစ်ပါတယ်။ Nginx က $scheme Variable ထဲမှာ မူလပြုလုပ်ခဲ့တဲ့ Protocol (HTTP သို့ HTTPS) ကို သိမ်းထားပေးပါတယ်။ Proxy Server နဲ့ App Server တို့ဆက်သွယ်တဲ့အခါ သုံးတဲ့ Protocol နဲ့ Client က အမှန် တစ်ကယ်ဆက်သွယ်စဉ်သုံးတဲ့ Protocol ကွာနိုင်တဲ့အတွက် ဒီလိုတွဲဖက်ပေးဖို့ သင့်ခြင်း ဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၃) မှာလည်း X-Real-IP Custom Header ကိုသုံးပြီး Client ရဲ့ IP Address ကို App Server ထံ တွဲဖက်ပေးပို့ထားပါတယ်။ အကယ်၍များ Proxy က တစ်ဆင့်တည်းမဟုတ်ပဲ၊ နှစ်ဆင့်သုံးဆင့်ဆင့်ပြီး Configure လုပ်ထားရင် Nginx က Proxy Server အဆင့်ဆင့်ရဲ့ IP Address တွေကို `$proxy_add_x_forwarded_for` Variable ထဲမှာ သိမ်းထားပေးပါတယ်။ ဒါကြောင့် လိုင်းနံပါတ် (၄) မှာ အဲ့ဒီ Proxy Server တွေရဲ့ IP စာရင်းကိုလည်း တွဲဖက်ပေးပို့ထားပါသေးတယ်။

ဒီနည်းနဲ့ Request ကို အမှန်တစ်ကယ်တာဝန်ယူဆောင်ရွက်မယ့် App Server သိသင့်တဲ့ ဖြည့်စွက် အချက်အလက်တွေကို Proxy Server က Header ထဲမှာ ထည့်သွင်းပေးပို့နိုင်မှာဖြစ်ပါတယ်။

18.2 – Load Balancing

Nginx Proxy ကို App Server အများအပြားရှိတဲ့ထဲက လက်ရှိမှာ လက်ခံဆောင်ရွက်ဖို့ အဆင်အပြေဆုံး Server ကို အလိုအလျောက်ရွေးချယ်ပြီး Request ကို လက်ဆင့်ကမ်းပေးသွားအောင် စီစဉ်ထားနိုင်ပါတယ်။ upstream Directive ကို အသုံးပြုရပါတယ်။

Config

```
upstream api_servers {
  server api.server1.com;
  server api.server2.com;
  server api.server3.com;
}

server {
  # Other server configs
  location /api {
        proxy_pass http://api_servers;
  }
}
```

နမူနာကိုလေ့လာကြည့်ရင် `upstream` Directive ကိုသုံးပြီး `api_servers` အမည်နဲ့ Block တစ်ခုသတ်မှတ်ထား ပါတယ်။ Block ထဲမှာ Server စာရင်းကို တန်းစီပြီးထည့်သွင်းပေးထားပါတယ်။ လိုင်းနံပါတ် (၁၀) ကို လေ့လာကြည့်ရင် `proxy_pass` အတွက် Upstream Server စာရင်းအဖြစ် ကြိုတင်သတ်မှတ်ထားတဲ့ `api_servers` ကို ပေးထား တာကိုတွေ့ရမှာဖြစ်ပါတယ်။ ဒီလိုပေးလိုက်တဲ့အခါ Nginx က Upstream Server တွေထဲက လက်ရှိ Request ကို လက်ခံဆောင်ရွက်ဖို့ အဆင်အပြေဆုံး Server ကို အလိုအလျောက် ရွေးချယ်ပြီး တော့မှ Request ကို လက်ဆင့်ကမ်းပေး သွားမှာပဲ ဖြစ်ပါတယ်။

Load Balancing Algorithm အမျိုးမျိုးရှိပါတယ်။ Nginx က Default အနေနဲ့ အသုံးပြုပေးမယ့် Algorithm ကတော့ Round Robin Algorithm ပဲဖြစ်ပါတယ်။

Round Robin - Round Robin Algorithm ရဲ့ သဘောသဘာဝက ရှင်းရှင်းလေးပါ။ Request တွေကို ပေးထားတဲ့ Server စာရင်းထဲက Server တစ်ခုပြီးတစ်ခု အလှည့်ကျ ပေးပို့ပေးသွားမှာဖြစ်ပါတယ်။ ဥပမာ – Request နှစ်ခုကို လက်ခံရရှိတဲ့အခါ ပထမတစ်ခုကို api.server1.com ထံပေးပို့သွားမှာဖြစ်ပြီး ဒုတိယတစ်ခုကို api.server2.com ထံ ပေးပို့သွားမှာပဲ ဖြစ်ပါတယ်။

Least Connection – Nginx က အသုံးပြုပေးနိုင်တဲ့ နောက်ထပ် Load Balancing Algorithm ကတော့ least_conn ပဲဖြစ်ပါတယ်။ Server တွေထဲလက် လက်ရှိ Active Connection အနည်းဆုံးရှိနေတဲ့ Server ကို ရွေးချယ်ပြီး Request ကို ပေးပို့သွားမှာဖြစ်ပါတယ်။ Round Robin လို ရိုးရိုးအလှည့်ကျ ပေးပိတာမျိုး မဟုတ်

တော့ပဲ Server တွေထဲက လက်ရှိအနည်းဆုံး အလုပ်လုပ်နေရတဲ့ Server ကို ရွေးချယ်ပေးပို့ပေးမယ့် Algorithm ပဲဖြစ်ပါတယ်။

IP Hash – နောက် Algorithm တစ်ခုကတော့ ip_hash ဖြစ်ပါတယ်။ ဒီ Algorithm ကတော့ Client ရဲ့ IP Address ပေါ်မူတည်ပြီး Server ကို ရွေးချယ်ပေးခြင်းဖြစ်ပါတယ်။ ပုံမှန်အားဖြင့် Client တစ်ခုက Request လုပ်လာတဲ့ အခါ ဘယ် Server ထံ ရောက်ရှိအလုပ်လုပ်မယ်ဆိုတာ ပုံသေပြောလို့မရပါဘူး။ သင့်တော်ရာ Server ကို Proxy က ရွေးချယ်ပေးပို့သွားမှာဖြစ်တဲ့အတွက် ဖြစ်ပါတယ်။ ip_hash Algorithm ကတော့ Client ရဲ့ IP ကို မှတ်ထားပြီး တစ်ကြိမ်ပေးပို့ ဖူးတဲ့ Server ထံကိုသာ နောက်တစ်ကြိမ်ထပ်မံပေးပို့မှာဖြစ်ပါတယ်။

Least Connection Algorithm ကို အသုံးပြုလိုတယ်ဆိုရင် Upstream Setting ကို အခုလို ပြုပြင်နိုင်ပါတယ်။

Config

```
upstream api_servers {

  least_conn;

  server api.server1.com;
  server api.server2.com;
  server api.server3.com;
}
```

IP Hash ကို အသုံးပြုလိုတယ်ဆိုရင်လည်း အလားတူပဲ သတ်မှတ်နိုင်ပါတယ်။

Config

```
upstream api_servers {

  ip_hash;

  server api.server1.com;
  server api.server2.com;
  server api.server3.com;
}
```

Server တွေရဲ့ စွမ်းဆောင်ရည် တစ်ခုနဲ့တစ်ခုမတူလို့ တစ်ချို့ Server တွေကို ပိုပြီးအသုံးပြုစေလိုတယ်ဆိုရင် Upstream Setting မှာ Weight ကိုလည်း ထည့်သွင်းသတ်မှတ်နိုင်ပါတယ်။ ဥပမာ –

Config

```
upstream api_servers {
  server api.server1.com weight=3;
  server api.server2.com;
  server api.server3.com;
}
```

နမူနာအရ api.server1.com ဟာ တစ်ခြား Server တွေထက် Request တွေကို (၃) ဆပိုပြီး လက်ခံရရှိတော့မှာ ဖြစ်ပါတယ်။ တစ်ခါတစ်ရံမှာ Server တွေခွဲပြီး Load Balance လုပ်နေပေမယ့်၊ အကြောင်းအမျိုးမျိုးကြောင့် Upstream Server တွေထဲက Server တစ်ခုဟာ လက်တစ်လော Request တွေကို လက်ခံဆောင်ရွက်နိုင်ခြင်းမရှိသေးတာမျိုး ဖြစ်တက်ပါတယ်။ ဒီလိုအခြေအနေရှိလာတဲ့အခါ အဆင်ပြေစေဖို့အတွက် အခုလို သတ်မှတ်ထားနိုင်ပါတယ်။

Config

```
1. upstream api_servers {
2.   server api.server1.com max_fails=3 fail_timeout=25s;
3.   server api.server2.com;
4.   server api.server3.com;
5. }
```

နမူနာမှာ `max_fails` ကို 3 လို့ သတ်မှတ်ထားတဲ့အတွက် api.server1.com ဟာ Request တွေလက်ခံဆောင်ရွက်ဖို့ (၃) ကြိမ် Fail ဖြစ်ခဲ့တယ်ဆိုရင် `fail_timeout` မှာ သတ်မှတ်ထားတဲ့ အချိန်ကာလဖြစ်တဲ့ (၂၅) စက္ကန့် အဲ့ဒီ Server ကို Nginx Proxy က အသုံးမပြုပဲ၊ အခြား Server တွေကို အစားထိုး အသုံးပြုပေးသွားမှာ ဖြစ်ပါတယ်။

18.3 – Proxy Cache

Nginx Proxy ရဲ့ စွမ်းဆောင်ရည် ပိုမိုကောင်းမွန်စေဖို့အတွက် Content တွေကို Cache လုပ်ထားနိုင်ပါတယ်။ Nginx Proxy က တစ်ကြိမ်ရယူထားဖူးတဲ့ Content ကို ခေတ္တသိမ်းထားပြီး၊ အလားတူ Request မျိုး ထပ်မံရရှိတဲ့အခါ Upstream Server တွေထံ လက်ဆင့်ကမ်းမနေတော့ပဲ သူသိမ်းထားတဲ့ Cache Content ကို Response ပြန်ပေးနိုင်တဲ့ လုပ်ဆောင်ချက်ဖြစ်ပါတယ်။ မလိုအပ်ရင် Upstream Server တွေထံ သွားနေစရာမလိုတော့တဲ့အတွက် စွမ်းဆောင်ရည် လည်း တိုးတက်လာမှာပဲဖြစ်ပါတယ်။

Cache လုပ်ဆောင်ချက်ကို အသုံးပြုနိုင်ဖို့အတွက်၊ ပထမအဆင့်အနေနဲ့ `http` Block အတွင်းမှာ အောက်ပါအတိုင်း သတ်မှတ်ပေးဖို့လိုပါတယ်။

Config

```
1. proxy_cache_path /var/lib/nginx/cache levels=1:2 keys_zone=backcache:8m max_size=50m;
2. proxy_cache_key "$scheme$request_method$host$request_uri$is_args$args";
3. proxy_cache_valid 200 10m;
4. proxy_cache_valid 404 1m;
```

လိုင်းနံပါတ် (၁) မှာ `proxy_cache_path` Directive ကိုအသုံးပြုပြီး Cache Content ကို သိမ်းဆည်းရမယ့် Directory ကို သတ်မှတ်ပေးထားခြင်းဖြစ်ပါတယ်။ Nginx က Cache ကိုသိမ်းတဲ့အခါ Cache Key နဲ့ Cache Data ဆိုပြီး နှစ်ပိုင်းခွဲသိမ်းပါတယ်။ Cache Key အနေနဲ့ Scheme, Request Method, Host Name, Request URI စတဲ့ Request Variable တွေကို အတွဲလိုက်သုံးဖို့ လိုင်းနံပါတ် (၂) မှာ သတ်မှတ်ထားပါတယ်။ ဥပမာ – Request က GET http://example.com/api ဆိုရင် Cache Key က httpgetexample.com/api0 ဖြစ်မှာ ဖြစ်ပါတယ်။ Nginx

က အဲ့ဒီ Key ကို Hash လုပ်ပြီး အသုံးပြုမှာပါ။

လိုင်းနံပါတ် (၁) က `levels=1:2` ရဲ့အဓိပ္ပါယ်ကတော့ Cache တွေကို Cache Key Hash ရဲ့ နောက်ဆုံး Character တစ်လုံးကို အမည်အဖြစ်အသုံးပြုထားတဲ့ Directory အတွင်းမှာ Hash ရဲ့ နောက်ဆုံး Character နှစ်လုံးကို အမည်အဖြစ် အသုံးပြုထားတဲ့ Directory အနေနဲ့ စုစည်းသိမ်းဆည်းဖို့ သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။ ဥပမာ – Key Hash က abcd123ef34abc ဆိုရင် c Directory ထဲမှာ ab Directory နဲ့ Cache Content ကို သိမ်းဆည်းပေးမှာ ဖြစ်ပါတယ်။

`keys_zone=backcache:8m` ရဲ့ အဓိပ္ပါယ်ကတော့ Cache Key ကို `backcache` လို့ခေါ်တဲ့အမည်နဲ့ သိမ်းဆည်းပြီး Cache Key သိမ်းဆည်းဖို့အတွက် Storage ကို 8 MB ထိ ယူပါလို့သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။ `max_size=50m` ကတော့ Cache Content သိမ်းဆည်းဖို့ အမြင့်ဆုံးခွင့်ပြုတဲ့ Storage ပမာဏက 50 MB ဖြစ်တယ် လို့ သတ်မှတ်ထားခြင်းပဲဖြစ်ပါတယ်။

လိုင်းနံပါတ် (၃) နဲ့ (၄) က `proxy_cache_valid` ရဲ့ အဓိပ္ပါယ်ကတော့ Status Code 200 OK နဲ့ လက်ခံရရှိတဲ့ Content တွေကို (၁၀) မိနစ် Cache လုပ်ပြီး Status Code 404 Not Found နဲ့ လက်ခံရရှိတဲ့ Content တွေကိုတော့ (၁) မိနစ်သာ Cache လုပ်ပါလို့ သတ်မှတ်ထားခြင်းပဲဖြစ်ပါတယ်။

ဒါ Cache သိမ်းဆည်းရမယ့် Setting ပဲရှိပါသေးတယ်။ သိမ်းဆည်းထားတဲ့ Cache ကို အသုံးပြုဖို့အတွက် သက်ဆိုင်ရာ server (သို့မဟုတ်) location Block တွေထဲမှာ အခုလို သတ်မှတ်ပေးနိုင်ပါတယ်။

Config

```
1. location /api {
2.     proxy_cache backcache;
3.     proxy_cache_bypass $http_cache_control;
4.
5.     proxy_pass http://api_servers;
6. }
```

`proxy_cache` Directive ကိုသုံးပြီး အသုံးပြုရမယ့် Cache Key ကို သတ်မှတ်ပေးရပါတယ်။ အကယ်၍ Client က Cache-Control Header ထည့်သွင်း ပေးပို့လာရင်၊ လိုက်နာအသုံးပြုဖို့အတွက် `proxy_cache_bypass` Directive နဲ့ သက်မှတ်ပေးထားပါတယ်။ ဒါကြောင့် Client က Cache ကို မလိုချင်ဘူး၊ App Server ကသွားယူတာကိုပဲ လိုချင်တယ်ဆိုရင် "Cache-Control: no cache" ဆိုတဲ့ Header ကို Request နဲ့အတူ ထည့်သွင်းပေးပို့နိုင်ပါတယ်။ no cache Header ပါဝင်လာရင် Nginx က Cache ကို အသုံးမပြုပဲ Upstream Server ထံ Request ကို လက်ဆင့်ကမ်းပေးသွားမှာဖြစ်ပါတယ်။

Cache ကို အသုံးပြုတဲ့အခါ အတော်လေးသတိထားဖို့တော့လိုပါတယ်။ အခြေခံအားဖြင့် အမြဲပြောင်းလဲနေခြင်းမရှိတဲ့ Image, Icon, Style, JavaScript Library စတဲ့ Static Content တွေကိုသာ Cache လုပ်သင့်ပါတယ်။ ပြောင်းလဲနေတတ်တဲ့ Content တွေကိုတော့ Cache မလုပ်သင့်ပါဘူး။ ဥပမာ – Database ထဲက User List ကို ပြန်ပေးတဲ့အခါ Cache မလုပ်သင့်ပါဘူး။ Cache လုပ်ထားရင် Database ထဲမှာ လက်တလော Data အသစ်တိုး

နေပေမယ့် Nginx က ပြီးခဲ့တဲ့ (၁၀) မိနစ်က Cache ကိုပဲ ပြန်ပေးတဲ့အခါ Client က Data အသစ်ပါဝင်ခြင်းမရှိတဲ့ Content အဟောင်းကိုပဲ လက်ခံရရှိနေမှာဖြစ်ပါတယ်။ ဒီထက်ပိုဆိုးတာက User A Request လုပ်ထားတဲ့ Data ကို Cache လုပ်ထားပြီး User B ရဲ့ Request ကို လက်ခံရရှိတဲ့အခါ Cache ကြီးကိုပြန်ပေးနေရင် User B က User A ရဲ့ Content ကို လက်ခံရရှိသွား မှာဖြစ်ပါတယ်။ ဒါကြောင့် ပြောင်းလဲမှာမဟုတ်တဲ့ Static Content တွေကိုသာ Cache လုပ်သင့်ပါတယ်။ တစ်ချို့ Response တွေကို Cache မလုပ်စေလိုရင် App Server မှာ အခုလို သတ်မှတ်ထားနိုင်ပါတယ်။

JavaScript

```
1. app.use(function(req, res, next) {
2.    res.set("expire", "-1");
3.    res.set("Cache-Control", "no store");
4. });
```

ExpressJS App Server က Cache-Control Header နဲ့အတူ no store ကို ထည့်သွင်းပေးတဲ့အခါ Nginx Proxy က အဲ့ဒီ Header ကို အသိအမှတ်ပြုပြီး လက်ခံရရှိတဲ့ Content ကို Cache အနေနဲ့ မသိမ်းဆည်းပဲ ထားပေးမှာပဲ ဖြစ်ပါတယ်။

ဒီနည်းနဲ့ Nginx ကို အသုံးပြုသူအနည်းအများ လိုအပ်ချက်ပေါ်မူတည်ပြီး App Server တွေ အချိန်မရွေး အတိုးအလျှော့ လုပ်နိုင်တဲ့ Load Balancer အနေနဲ့သာမက၊ App ရဲ့ စွမ်းဆောင်ရည် မြင့်တက်စေဖို့အတွက် Cache Proxy အနေနဲ့ပါ အသုံးပြုနိုင်မှာပဲဖြစ်ပါတယ်။

Conclusion

Nginx ကို စတင်ဖန်တီးစဉ်ကတည်းက Reverse Proxy အနေနဲ့ အလုပ်လုပ်ဖို့ ဦးစားပေး ဖန်တီးခဲ့ခြင်း ဖြစ်ပါတယ်။ ဒါကြောင့်လည်း Nginx အတွက် Proxy Setting နဲ့ Load Balancing Setting တွေ သတ်မှတ်ရတာ ရှုပ်ထွေးခြင်းမရှိပဲ ရိုးရှင်းလွယ်ကူနေခြင်း ဖြစ်ပါတယ်။ Nginx အပြင် အခြားလူသုံးများတဲ့ နည်းပညာတွေလည်း ရှိပါသေးတယ်။ Load Balancing အတွက် HAProxy လို့ခေါ်တဲ့ နည်းပညာကိုလည်း လူသုံးများကြပါတယ်။ ထူးခြားချက်ကတော့၊ HAProxy ဟာ Nginx ကို Application Layer Proxy မဟုတ်ပဲ Network Layer Proxy ဖြစ်ပါတယ်။ Cache Server အနေနဲ့ လည်း Varnish လို့ခေါ်တဲ့ နည်းပညာကို လူသုံးများကြပါသေးတယ်။ Nginx ကတော့ Web Server အနေနဲ့သာမက၊ Proxy Server အနေနဲ့ရော၊ Cache Server အနေနဲ့ပါ အားလုံးကို ဆောင်ရွက်ပေးနိုင်တဲ့အတွက် Nginx ကို ဒီနေရာမှာ ရွေးထုတ် ဖော်ပြလိုက်ရခြင်းပဲဖြစ်ပါတယ်။

Database Server Architecture စဉ်းစားတဲ့အခါ Hardware Failure ကြောင့် Data မဆုံးရှုံးစေဖို့နဲ့ Data ပမာဏများလာတဲ့အခါ Storage Capacity ကို အလွယ်တစ်ကူ တိုးမြှင့်နိုင်ဖို့ဆိုတဲ့ အချက်နှစ်ချက်ကို အဓိကစဉ်းစားရပါတယ်။

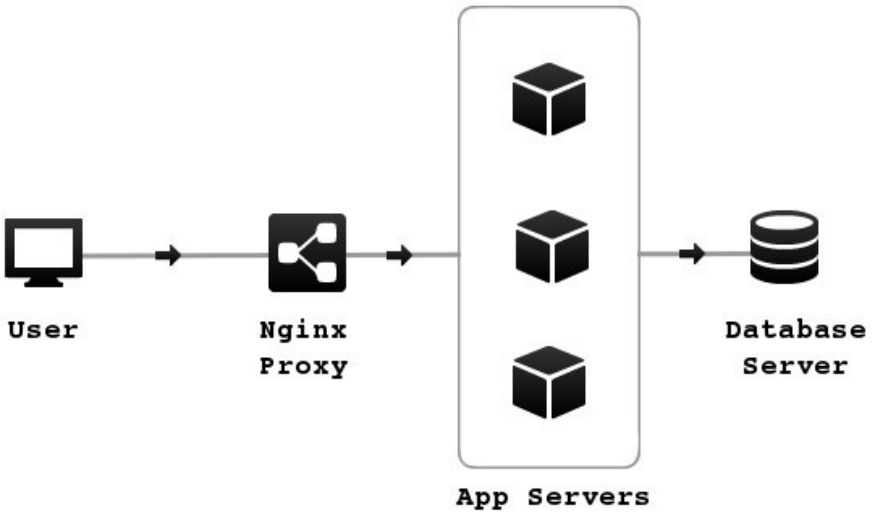
Rockstar Developer Course

Project Management, Web Service, Server Architecture
NodeJS အစရှိသည် ဤစာအုပ်ပါ အကြောင်းအရာများကို
စာရေးသူကိုယ်တိုင် သင်ကြားပေးခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၁၉) – MongoDB Replication and Sharded Cluster

ပြီးခဲ့တဲ့အခန်းမှာ Nginx ကိုသုံးပြီး Load Balance ပြုလုပ်နိုင်ပုံကို ဖော်ပြခဲ့ပါတယ်။ အဲ့ဒီလို Load Balance ပြုလုပ်ရာမှာ App Server ကိုသာ Balance ပြုလုပ်ခြင်းဖြစ်ပြီး အချက်အလက်တွေ သိမ်းဆည်းထားတဲ့ Database ကိုတော့ ထည့် သွင်း စဉ်းစားထားခြင်း မရှိသေးပါဘူး။ ဒါကြောင့် ဒီနည်းနဲ့ Server Architecture ကို ဖန်တီးထားမယ်ဆိုရင်၊ ရရှိမယ့် ပုံသဏ္ဍာန်က အခုလိုဖြစ်နေမှာပါ။



ပုံ (၁၉.၁) - Server Architecture without Database Load Balancing

အသုံးပြုသူများလာတဲ့ ဒါဏ်ကိုခံနိုင်အောင် App Server တွေကို Load Balance လုပ်ထားပေမယ့် Database Server ကတော့ တစ်ခုတည်းသာဖြစ်နေဦးမှာပါ။ ဒါကြောင့် Database Server မှာ လာပြီး နှေးနေတာမျိုးဖြစ်နိုင်ပါတယ်။ Data ပမာဏများလာတဲ့အခါ လက်ခံသိမ်းဆည်းပေးနိုင်ခြင်းမရှိ ဖြစ်လာနိုင်ပါတယ်။

အခန်း (၁၁) မှာ MongoDB အကြောင်း လေ့လာခဲ့စဉ်က MongoDB ဟာ ဒီလိုအခြေအနေမျိုးမှာ Scale လုပ်ရလွယ်အောင်ဖန်တီးထားခြင်း ဖြစ်တယ်ဆိုတဲ့အကြောင်းကိုဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ ဒီအခန်းမှာတော့ MongoDB ရဲ့ စွမ်းဆောင်ရည် Performance နဲ့ Availability ကောင်းမွန်စေဖို့အတွက် Replication ပြုလုပ်ပုံနဲ့၊ Sharded Cluster လုပ်ဆောင် ချက်ကိုသုံးပြီး Scale ပြုလုပ်ပုံတို့အကြောင်းကို ဆက်လက်ဖော်ပြသွားမှာ ဖြစ်ပါတယ်။

19.1 – MongoDB Replica Sets

Replication လုပ်ဆောင်ချက်ဟာ Database စနစ်တိုင်းလိုလိုမှာပါဝင်တဲ့ လုပ်ဆောင်ချက်တစ်ခုဖြစ်ပါတယ်။ သူ့ရဲ့အခြေခံ အလုပ်လုပ်ပုံကတော့ Master Database Server မှာသိမ်းဆည်းထားတဲ့ အချက်အလက်တွေကို Slave Database Server တွေမှာ မူပွားယူထားခြင်းဖြစ်ပါတယ်။ ဒီလိုမူပွားယူထားတဲ့အတွက် ရရှိလာတဲ့ အကျိုးရလဒ်တွေကတော့ –

Read Performance – အချက်အလက်တွေ ရယူဖို့ Request ပြုလုပ်လာတဲ့အခါ Master Database Server က အမြဲ အကြောင်းပြန်နေဖို့ မလိုအပ်တော့ပဲ Slave Database Server တွေက ကိုယ်စားအကြောင်းပြန်ပေးနိုင်တဲ့အတွက် Database ရဲ့ Read Performance စွမ်းဆောင်ရည် မြင့်တက်လာမှာပဲဖြစ်ပါတယ်။

Data Safety – Master Database Server အကြောင်းအမျိုးမျိုးနဲ့ အလုပ် မလုပ်တော့တဲ့အခါ၊ Slave Database Server တွေမှာ အချက်အလက်တွေ ကျန်ရှိနေမှာဖြစ်တဲ့အတွက် အချက်အလက်လုံခြုံရေးကိုလည်း အထောက်အကူဖြစ်စေမှာဖြစ်ပါတယ်။

Availability – အကြောင်းအမျိုးမျိုးကြောင့် Master Database Server အလုပ်မလုပ်နိုင်တဲ့အခါ Slave Database Server တစ်ခုက Master အဖြစ် ချက်ခြင်းပြောင်းလဲအလုပ်လုပ်ပေးခြင်းအားဖြင့် လက်ရှိအလုပ်လုပ်နေတဲ့ App ကို ရပ်တန့်သွားစရာမလိုပဲ ဆက်လက်အလုပ်လုပ်နိုင်မှာဖြစ်ပါတယ်။

Replication ရဲ့ထူးခြားချက်ကတော့ Write လုပ်ငန်းစဉ်အားလုံးဟာ Master Database Server မှာသာ လုပ်ရပါတယ်။ Slave Database Server တွေက Master ရဲ့ Data ကို မူပွားယူခြင်းမဟုတ်ပဲ၊ Master ရဲ့ Operation Log (oplogs) ကို ဖတ်ပြီး Master မှာလုပ်ခဲ့တဲ့ အလုပ်တွေကို Slave ပေါ်မှာ ထပ်တူလုပ်ပေးခြင်းအားဖြင့် အချက်အလက်တွေကို ကူးယူခြင်းဖြစ်ပါတယ်။ အကယ်၍ Slave တွေမှာပါ Write လုပ်ငန်းစဉ်ကို ခွင့်ပြုလိုက်ရင် မလိုလားအပ်တဲ့ပြဿနာတွေ တက်နိုင်တဲ့အတွက် Write လုပ်ငန်းစဉ်အားလုံးကတော့ Master မှာပဲ လုပ်ရလေ့ရှိပါတယ်။ Read လုပ်ငန်းစဉ်ကိုတော့ Master နဲ့ Slave Server တွေ မျှပြီး ခွဲဝေလက်ခံ ဆောင်ရွက်ကြလေ့ရှိပါတယ်။

MongoDB မှာ Replication လုပ်ဆောင်ချက်ကို Replica Sets လို့ခေါ်ပါတယ်။ Master Database Server ကိုတော့ Primary Member လို့ခေါ်လေ့ရှိပြီး၊ Slave Database Server ကိုတော့ Secondary Member လို့ ခေါ်လေ့ရှိပါတယ်။ MongoDB Replica Set တစ်ခုမှာ Member (၅၀) ထိ ပါဝင်နိုင်ပါတယ်။

> ဒီအခန်းမှာ ဖော်ပြထားတဲ့ Mongo Shell နမူနာတွေဟာ တိုက်ရိုက်ကူးယူစမ်းသပ်ဖို့မဟုတ်ပါ။ သဘော သဘာဝ ပေါ်လွင်အောင် Simplify လုပ်ထားတဲ့ နမူနာသက်သက်သာ ဖြစ်ပါတယ်။ ဒါကြောင့် သဘော သဘာဝကိုသာ နားလည်အောင်လေ့လာပြီး အသေးစိတ်ကို ကိုယ်တိုင် ဆက်လက် လေ့လာသွားရမှာ ဖြစ်ပါတယ်။

19.2 – Setting Up Replica Sets

MongoDB Replica Sets သတ်မှတ်ပုံ နမူနာဖော်ပြနိုင်ဖို့အတွက် ကျွန်တော်တို့မှာ အောက်ပါ Domain Name များနဲ့ MongoDB ကို အသင့် Install လုပ်ထားပြီးသား Database Server တွေရှိတယ်လို့ မှတ်ယူပေးပါ။

- db1.example.com
- db2.example.com
- db3.example.com

ပထမဦးဆုံးအနေနဲ့ db1.example.com ရဲ့ MongoDB Configuration မှာ replSet = rs0 ဆိုတဲ့ လိုင်းတစ်လိုင်း ထည့်ပေးရပါမယ်။ rs0 ဆိုတာ Replica Sets အမည်ဖြစ်ပြီး၊ အခြားမှတ်ရလွယ်တဲ့ အမည်တစ်ခုခုကို ပေးမယ် ဆိုလည်း ပေးနိုင်ပါတယ်။ Ubuntu မှာဆိုရင် /etc/mongodb.conf ဆိုတဲ့အမည်နဲ့ MongoDB Configuration ဖိုင်က တည်ရှိမှာဖြစ်ပါတယ်။

ပြီးတဲ့အခါ MongoDB Server ကို Restart လုပ်ပေးဖို့လိုပါတယ်။

```
$ sudo service mongodb restart
```

ဆက်လက်ပြီး Mongo Shell ကို `mongo` Command နဲ့ ဝင်ရောက်ပြီး `rs.initiate()` Function ကို Run ပေးရပါမယ်။

```
$ mongo
> rs.initiate()
```

`rs.initiate()` Function က လက်ရှိ Server ကို Replica Sets ရဲ့ ပထမဦးဆုံး Member အနေနဲ့ ထည့်သွင်း ပေးသွားမှာဖြစ်ပါတယ်။ Replica Sets ရဲ့ လက်ရှိအခြေအနေကို `rs.conf()` နဲ့ ခေါ်ယူကြည့်ရှုနိုင်ပါတယ်။

```
{
    "_id" : "rs0"
    "version" : 1,
    "members" : [
        {
            "_id" : 0,
            "host" "db1.example.com:27017"
        }
    ]
}
```

ဆက်လက်ပြီး ကျန် MongoDB Server တွေကို `rs.add()` နဲ့ Replication Sets ထဲမှာ ထည့်ပေးနိုင်ပါတယ်။

```
> rs.add("db2.example.com");
> rs.add("db3.example.com");
```

ဒီနည်းနဲ့ Server (၃) လုံးပါဝင်တဲ့ MongoDB Replica Sets တစ်ခုကို အလွယ်တစ်ကူ တည်ဆောက်နိုင်ပါတယ်။ MonboDB Replica Sets တစ်ခုမှာ အနည်းဆုံး Member (၃) လုံး ပါဝင်သင့်ပါတယ်။

Replica Sets ထဲက Member တစ်လုံးကို ပြန်ထုတ်လိုရင်တော့ `rs.remove()` ကို အသုံးပြုနိုင်ပါတယ်။

```
> rs.remove("db3.example.com");
```

အကယ်၍ Remove လုပ်လိုက်တဲ့ Member က Primary ဖြစ်နေရင် MongoDB က ကျန် Member တွေထဲကတစ်ခုကို Primary အဖြစ် အလိုအလျောက် ရွေးချယ်သတ်မှတ်သွားမှာဖြစ်ပါတယ်။

19.3 – Arbiter Member in Replica Sets and Member Priority

MongoDB ဟာ Automatic Failover လုပ်ဆောင်ချက်နဲ့ Primary Member အလုပ်မလုပ်တော့တဲ့အခါ Replica Sets ထဲက Secondary Member တစ်ခုကို အလိုအလျောက် Primary အဖြစ် ပြောင်းပေးသွားမှာဖြစ်ပါတယ်။ အဲ့ဒီလို ပြောင်း ဖို့အတွက် Primary ရွေးချယ်ရေး ရွေးကောက်ပွဲ (Election) ကို ဆောက်ရွက်ပါတယ်။ ဒီနေရာမှာ MongoDB Election Process ကို အသေးစိတ်မပြောတော့ပါဘူး။ လိုရင်းအနေနဲ့ မှတ်သားစေလိုတာကတော့ Election လုပ်ငန်းဆောင်ရွက်ရာမှာ အထောက်အကူဖြစ်စေဖို့အတွက် Arbiter Member လို့ခေါ်တဲ့ MongoDB Server တစ်မျိုးကို Replica Sets ထဲမှာ ထည့်သွင်းပေးနိုင်ပါတယ်။

Arbiter Member ဟာ Replica Sets ရဲ့ အစိတ်အပိုင်းတစ်ခုဖြစ်ပေမယ့် Secondary Member တွေလို Data တွေ မူပွားယူသိမ်းဆည်းခြင်းမရှိပါဘူး။ အကြောင်းအမျိုးမျိုးနဲ့ Primary Member အလုပ်မလုပ်တော့လို့ Secondary Member ထဲကတစ်ခုကို Primary ပြောင်းဖို့ ရွေးချယ်ရတဲ့အချိန်မှာ ဘယ်သူကို Primary ပြောင်းပေးသင့်သလဲဆိုတဲ့ Vote ကို လုပ် ပေးပါတယ်။ Replica Sets ထဲမှာ Arbiter Member တစ်ခုကို အခုလို ထည့်သွင်းနိုင်ပါတယ်။

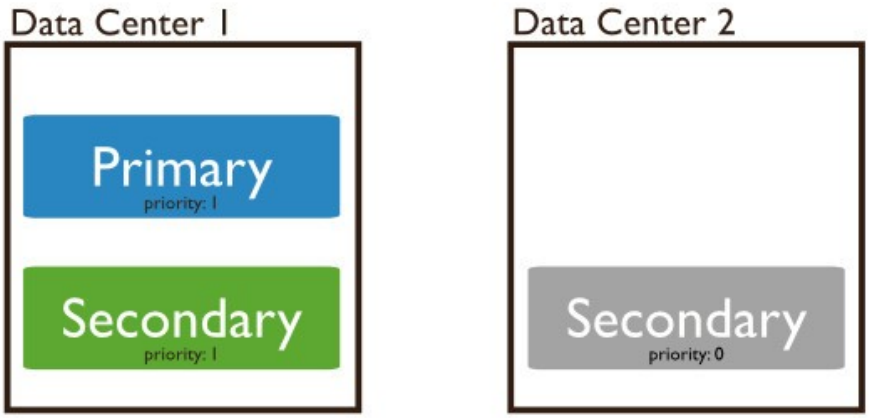
```
> rs.addArb("adb.example.com");
```

အခြေခံအားဖြင့် Replica Set ထဲမှာ Member အရေအတွက် စုံကိန်းဖြစ်နေရင် Arbiter Member ထည့်သွင်းသင့်ပါ တယ်။ မကိန်းဆိုရင်တော့ ထည့်သွင်းစရာမလိုပါဘူး။ ဥပမာ – စုစုပေါင်း Member (၄) လုံးဆိုရင် Arbiter ထည့်ပြီး (၅) လုံးဖြစ်သွားအောင် ဆောက်ရွက်ပေးသင့်ပါတယ်။ စုစုပေါင်း Member (၃) လုံးဆိုရင်တော့ မကိန်းဖြစ်နေတဲ့အတွက် Arbiter ထပ်ထည့်ဖို့ မလိုအပ်တော့ပါဘူး။

ဖြည့်စွက်သတိပြုသင့်တာကတော့၊ Replica Sets တစ်ခုမှာ Member (၅၀) ထိပါဝင်နိုင်ပေမယ့် Election လုပ်တဲ့အခါ အမြင့်ဆုံး (၇) လုံးထိသာ Vote လုပ်မှာဖြစ်ပါတယ်။ ဒါကြောင့် Member (၇) လုံးကျော်သွားရင်တော့၊ စုံပဲ

ဖြစ်ဖြစ်၊ မပဲ ဖြစ်ဖြစ်၊ Arbiter ကို သီးခြားထပ်ထည့်စရာ မလိုတော့ပါဘူး။

နောက်တစ်ချက်အနေနဲ့ သတိပြုသင့်တာကတော့၊ Primary ဘယ်တော့မှမဖြစ်စေလိုတဲ့ Member ကို Priority 0 အဖြစ် သတ်မှတ်ထားနိုင်ပါတယ်။ အထူးသဖြင့် Member Server တွေကို နေရာဒေသမတူတဲ့ Data Center မှာခွဲထားတဲ့အခါ သီးခြားဖြစ်နေတဲ့ Server တွေကို Primary အဖြစ် မသတ်မှတ်သင့်ပါဘူး။



ပုံ (၁၉.၂) - Multi-Data Center Replica Sets

source: mongodb.org

သီးခြား Data Center မှာ ရှိနေတဲ့ Member က Primary ဖြစ်သွားရင် Replica Sets ရဲ့ အလုပ်လုပ်ပုံ မလိုအပ်ပဲ နှေးသွားနိုင်တဲ့ အတွက်ဖြစ်ပါတယ်။ ဒါကြောင့် အဲ့ဒီလို Member မျိုးကို Priority 0 သတ်မှတ်ပေးထားသင့်ပါတယ်။ အခုလို သတ်မှတ်နိုင်ပါတယ်။

```
> rs.conf()

{
    "_id" : "rs0"
    "version" : 1,
    "members" : [
        {
            "_id" : 0,
            "host" "db1.example.com:27017"
        },
        {
            "_id" : 1,
            "host" "db2.example.com:27017"
        },
        {
            "_id" : 2,
            "host" "db3.example.com:27017"
        }
    ]
}

> cfg = rs.conf();
```

```
> cfg.members[2].priority = 0;
> rs.reconfig(cfg);
```

လက်ရှိ Replica Sets Configuration ထဲက Member တစ်ခုရဲ့ Priority တန်ဖိုးကို 0 လို့ သတ်မှတ်ပေးလိုက်ခြင်း ဖြစ်ပါ တယ်။ ဒါကြောင့်၊ အဲ့ဒီ Member ကို Vote လုပ်ချိန်မှာ Primary အဖြစ်မပြောင်းပဲ ချန်လှပ်ပေးသွားမှာပဲ ဖြစ်ပါတယ်။

19.4 – Hidden and Delayed Member in Replica Sets

Replica Sets ထဲက တစ်ချို့ Member တွေကို Hidden Member အနေနဲ့ သတ်မှတ်ထားနိုင်ပါတယ်။ Hidden Member တွေဟာ၊ Data တွေကို အခြား Secondary Member များနည်းတူ မူပွားယူပေမယ့်၊ Client Request တွေကို လက်ခံအလုပ်လုပ်မှာ မဟုတ်ပါဘူး။ Data Backup သက်သက်ပြုလုပ်လိုတဲ့အခါ အသုံးပြုနိုင်ပါတယ်။ ဒါမှမဟုတ် လက်ရှိ Application Data ကို မထိပဲ Statistics နဲ့ Report တွေကို သီးခြားထုတ်ယူချင်တဲ့အခါ Hidden Member ထဲက မူပွား ယူထားတဲ့ Data တွေကို အသုံးပြုနိုင်မှာဖြစ်ပါတယ်။

Member တစ်ခုကို Hidden Member အဖြစ်အခုလို သတ်မှတ်နိုင်ပါတယ်။

```
> cfg = rs.conf();
> cfg.members[2].priority = 0;
> cfg.members[2].hidden = true;
> rs.reconfig(cfg);
```

Hidden လုပ်လိုတဲ့ Member ရဲ့ hidden ကို true လို့ သတ်မှတ်ပေးလိုက်ခြင်းဖြစ်ပါတယ်။ သတိပြုသင့်တာကတော့၊ Hidden Member ကို Priority 0 မဖြစ်မနေ သတ်မှတ်ထားပေးဖို့ လိုအပ်ပါတယ်။

Replica Sets ထဲက Member တစ်ခုကို Delay Member အနေနဲ့လည်း သတ်မှတ်ထားနိုင်ပါတယ်။ Delay Member ဆိုတာ၊ မူပွားကူယူတဲ့လုပ်ငန်းကို ချက်ခြင်းမလုပ်ပဲ သတ်မှတ်ထားတဲ့အချိန်တစ်ခုစောင့်ပြီးမှ ကူးယူတဲ့ Member ဖြစ်ပါ တယ်။ Backup လုပ်ဖို့အတွက် အသုံးဝင်ပါတယ်။ Delay Member က Data တွေကို ချက်ခြင်းကူးမယူတဲ့အတွက် Primary Member ပေါ်မှာ မတော်တဆ အမှားအယွင်းတစ်ခုခုလုပ်မိခဲ့ရင် အဲ့ဒီအမှားက Delay Member ကို ချက်ချင်း မရောက်တဲ့အတွက် အမှားမပါဝင်သေးတဲ့ မူလအချက်အလက်တွေကို Delay Member ကနေ ပြန်လည်ရယူနိုင်မှာပဲ ဖြစ် ပါတယ်။

Delay Member ရဲ့ Priority ကို 0 သတ်မှတ်ထားဖို့လိုသလို Hidden ကိုလည်း True သတ်မှတ်ထားပေးဖို့လိုပါ တယ်။ Member တစ်ခုကို Delay Member အဖြစ် အခုလိုသတ်မှတ်နိုင်ပါတယ်။

```
> cfg = rs.conf();
> cfg.members[2].priority = 0;
> cfg.members[2].hidden = true;
```

```
> cfg.members[2].slaveDelay = 3600;
> rs.reconfig(cfg);
```

Member ရဲ့ `slaveDelay` Property အတွက် မူပွားမယူမီ စောင့်ရမယ့် အချိန်ကာလစက္ကန့်ကို ပေးခြင်းအားဖြင့် Delay Member အဖြစ် သတ်မှတ်နိုင်ခြင်းဖြစ်ပါတယ်။ နမူနာအရ `member[2]` ဟာ အချိန်စက္ကန့် ၃၆၀၀ (၁ နာရီ) စောင့်ဆိုင်းပြီးတော့မှ Primary က အချက်အလက်တွေကို မူပွားကူးယူမှာဖြစ်ပါတယ်။ တနည်းအားဖြင့် `member[2]` မှာ အမြဲတမ်း တစ်နာရီနောက်ကျတဲ့ အချက်အလက်တွေ ရှိနေမှာပဲဖြစ်ပါတယ်။

19.5 – Forcing a Member to be Primary in Replica Sets

ပုံမှန်အားဖြင့် Replica Sets တစ်ခုမှာ ဘယ်သူ Primary Member ဖြစ်သင့်သလဲဆိုတာကို MongoDB က Election လုပ်ပြီးဆုံးဖြတ်ပါတယ်။ Member တွေထဲက တစ်ခုကို Primary ဖြစ်အောင် ကိုယ်တိုင်သတ်မှတ်လိုရင်တော့ သတ်မှတ်လို တဲ့ Member ရဲ့ Priority ကို ကျန် Member များထက် မြင့်အောင်ပေးခြင်းအားဖြင့် သတ်မှတ်နိုင်ပါတယ်။ Priority တန်ဖိုးကို 0 ကနေ 1000 အထိ ကိန်းပြည့်အနေနဲ့ရော၊ ဒဿမကိန်းအနေနဲ့ပါ ပေးနိုင်ပါတယ်။ Priority ကို 0 သတ်မှတ် ထားရင်တော့ Primary မဖြစ်နိုင်တာကို အထက်မှာ ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။

Member တွေထဲက `members[2]` ကို Primary Member ဖြစ်စေလိုတယ်ဆိုရင် အခုလိုသတ်မှတ်နိုင်ပါတယ်။

```
> cfg = rs.conf();
> cfg.members[0].priority = 0.5;
> cfg.members[1].priority = 0.5;
> cfg.members[2].priority = 1;
> rs.reconfig(cfg);
```

ဒီနည်းနဲ့ `members[2]` ရဲ့ Priority ကျန် Member များထက်မြင့်သွားတဲ့အခါ `member[2]` ကို Primary အဖြစ် MongoDB က ရွေးချယ်ပေးလိုက်မှာပဲဖြစ်ပါတယ်။

လက်ရှိ ဘယ်သူက Primary ဖြစ်ပြီး ဘယ်သူတွေက Secondary လည်းဆိုတာကို အခုလိုလေ့လာနိုင်ပါတယ်။

```
> db.runCommand("ismaster");

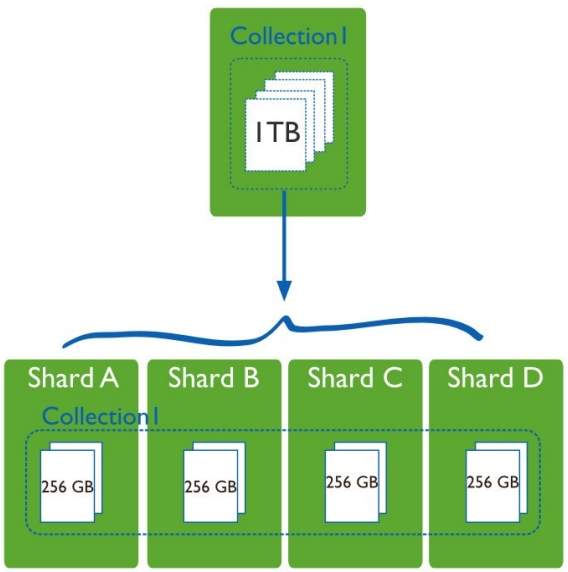
{
     "ismaster" : false,
     "secondary" : true,
     "hosts" : [
             "db1.example.com",
             "db2.example.com"
     ],
     "passives" : [
          "db3.example.com"
     ],
     "arbiters" : [
         "db4.example.com",
```

```
        ]
        "primary" : "db2.example.com",
        "ok" : true
}
```

နမူနာအရ db2.example.com က Primary ဖြစ်နေတာကို တွေ့ရမှာပါ။ ရလဒ်ထဲမှာ Priority 0 သတ်မှတ်ထားတဲ့ Passive Server စာရင်းနဲ့ Arbiter စာရင်းကိုလည်း ရှိနေရင် ထည့်သွင်းဖော်ပြပေးမှာဖြစ်ပါတယ်။

19.6 – Sharding in MongoDB

Replica Sets လုပ်ဆောင်ချက်ဟာ Reliability နဲ့ Read Performance အတွက် အသုံးဝင်ပါတယ်။ ဒါပေမယ့် Data အများအပြားကို Server တွေခွဲသိမ်းလိုရင်တော့ Sharding လုပ်ဆောင်ချက်ကို အသုံးပြုရမှာဖြစ်ပါတယ်။ အခြေခံအားဖြင့် Sharding ဆိုတာ Data တွေကို တစ်လုံးထက်ပိုတဲ့ Database Server တွေမှာ ခွဲခြားသိမ်းဆည်းပေးခြင်းဖြစ်ပါတယ်။ ဒီနည်းနဲ့ Storage လိုအပ်လာတိုင်း လက်ရှိစနစ်ကို မထိခိုက်စေပဲ Database Server တွေကို လိုအပ်သလို ထပ်မံထည့် သွင်း တိုးချဲ့နိုင်မှာပဲဖြစ်ပါတယ်။



ပုံ (၁၉.၃) - MongoDB Sharding

source: mongodb.org

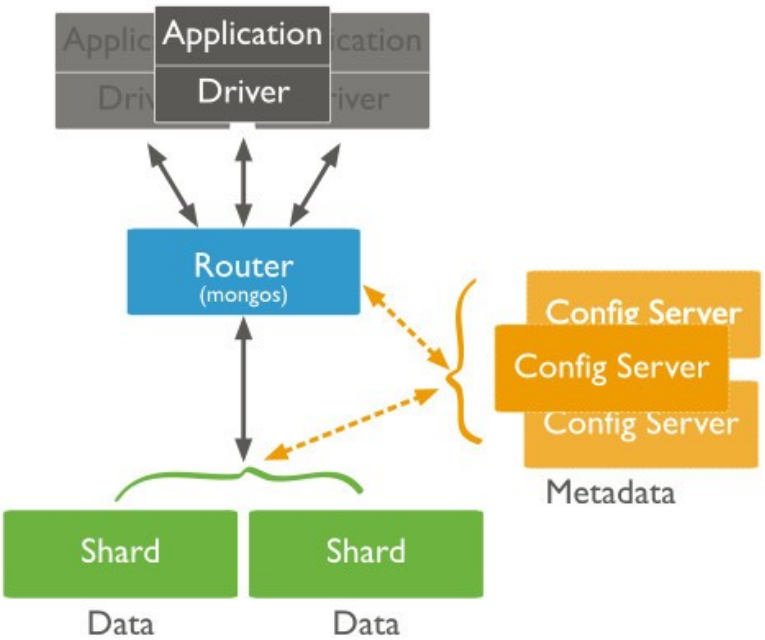
MongoDB ရဲ့ Sharding လုပ်ဆောင်ချက်ကို အသုံးချနိုင်ဖို့အတွက် အစိတ်အပိုင်း (၃) ရပ်လိုအပ်ပါတယ်။ Config Servers, Query Routers နဲ့ Shards တို့ပဲဖြစ်ပါတယ်။ အားလုံးစုပေါင်းထားတဲ့ လုပ်ဆောင်ချက်ကို Sharded Cluster လို့ ခေါ်ပါတယ်။

Config Servers ရဲ့တာဝန်ကတော့ Data တွေကို ဘယ်နည်းဖြန့်ကျက်သိမ်းဆည်းထားတဲ့ဆိုတဲ့ အချက်အလက်မှတ်တမ်း ကို သိမ်းဆည်းဖို့ဖြစ်ပါတယ်။ Sharded Cluster တစ်ခုမှာ Config Server (၃) ခု ပါဝင်ရပါတယ်။

Query Routers ရဲ့ တာဝန်ကတော့ App က ပေးပို့လာတဲ့ Request တွေကိုလက်ခံဖို့ဖြစ်ပါတယ်။ Request တွေ လက်ခံရရှိတဲ့အခါ Query Router က Config Server ထံမှာ ရယူလိုတဲ့ Data တွေ ဘယ်လိုသိမ်းထားသလဲ သွားရောက် လေ့လာပြီး၊ Data တွေရယူကာ App ထံ ပြန်လည်ပေးပို့ပေးမှာဖြစ်ပါတယ်။

Shards ရဲ့တာဝန်ကတော့ Data တွေကို လက်ခံသိမ်းဆည်းဖို့ပဲဖြစ်ပါတယ်။ Shard တစ်ခုဟာ MongoDB Server တစ်ခု ဖြစ်နိုင်သလို၊ Replica Sets တစ်ခုလည်း ဖြစ်နိုင်ပါတယ်။ လက်တွေ့ Production မှာတော့ Replica Sets ကို Shard အဖြစ် အသုံးပြုရမှာဖြစ်ပါတယ်။

Sharded Cluster တစ်ခုရဲ့ ဖွဲ့စည်းပုံကို **ပုံ (၁၉.၄)** မှာ လေ့လာနိုင်ပါတယ်။



ပုံ (၁၉.၄) - MongoDB Sharded Cluster

source: mongodb.org

Sharded Cluster တစ်ခုမှာ တစ်ခုထက်ပိုတဲ့ Query Router တွေ ပါဝင်နိုင်ပါတယ်။ အနည်းဆုံး Query Router တစ်ခု မဖြစ်မနေ ပါဝင်ရပါမယ်။ အချက်အလက်တွေ သိမ်းဆည်းဖို့အတွက် တစ်ခုထက်ပိုတဲ့ Shard တွေပါဝင် နိုင်ပါတယ်။ အနည်းဆုံး Shard (၂) ခုပါဝင်ရပါတယ်။ Config Server ကတော့ အတိအကျ (၃) ခုပါဝင်ရမှာဖြစ် ပါတယ်။

Sharded Cluster သတ်မှတ်ပုံ နမူနာဖော်ပြနိုင်ဖို့အတွက် ကျွန်တော်တို့မှာ အောက်ပါ Domain Name များနဲ့ MongoDB ကို Install လုပ်ထားပြီးသား Database Server များရှိတယ်လို့ မှတ်ယူပေးပါ။

- cfg1.example.com (Config Server 1)
- cfg2.example.com (Config Server 2)
- cfg3.example.com (Config Server 3)
- router.example.com (Query Router)
- rs0 (Replica Stes 1)
- rs1 (Replica Sets 2)

Config Server အတွက် Server (၃) ခု၊ Query Router အတွက် Server တစ်ခုနဲ့၊ Shard အတွက် Replica Sets (၂) ခုတို့ ဖြစ်ပါတယ်။

19.7 – Setting Up Config Servers for Sharded Cluster

ပထမဦးဆုံးအနေနဲ့ Config Server တွေကို အရင် Setup လုပ်ရပါမယ်။ သေချာသွားအောင် Config Server အဖြစ် အသုံးပြုမယ့် Server မှာ MongoDB Service ကို Stop လုပ်ပေးသင့်ပါတယ်။

```
$ sudo service mongodb stop
```

ပြီးတဲ့အခါ Configuration Meta Data တွေသိမ်းဖို့အတွက် Directory တစ်ခုသီးခြားတည်ဆောက်ပေးသင့်ပါတယ်။

```
$ mkdir -p /data/configdb
```

ပြီးတဲ့အခါ MongoDB ကို Install လုပ်စဉ်က တစ်ခါတည်းပါဝင်လာတဲ့ `mongod` ကိုသုံးပြီး Config Server ကို အခုလို စတင်နိုင်ပါတယ်။

```
$ sudo mongod --configsvr --dbpath /data/configdb --port 27019
```

`mongod` Command က အခြေခံအားဖြင့် MongoDB Server ကို စတင်ပေးခြင်းဖြစ်ပါတယ်။ အဲ့ဒီလို စတင်တဲ့အခါ ရိုးရိုး Server အနေနဲ့မဟုတ်ပဲ Config Server အနေနဲ့စတင်ဖို့အတွက် `--configsvr` Option ကို တွဲဖက်ပေးရခြင်း ဖြစ်ပါတယ်။ Meta Data တွေ သိမ်းရမယ့် Location ကိုတော့ `--dbpath` Option နဲ့ တွဲပေးပြီး Config Server Port အဖြစ် 27019 ကို `--port` Option နဲ့ တွဲဖက်ပေးခြင်းဖြစ်ပါတယ်။ 27019 ဟာ MongoDB Config Server အတွက် Default Port ဖြစ်ပါတယ်။

ဒီနည်းအတိုင်း Config Server အဖြစ် အသုံးပြုမယ့် cfg1.example.com, cfg2.example.com နဲ့ cfg3.example.com Server (၃) ခုစလုံးမှာ ဆောင်ရွက်ပေးရမှာပဲဖြစ်ပါတယ်။

19.8 – Setting Up Query Router for Sharded Cluster

ဆက်လက်ပြီး Query Router အဖြစ် အသုံးပြုမယ့် router.example.com Server မှာ MongoDB Install လုပ်စဉ်က ပါဝင်လာတဲ့ mongos ကိုသုံးပြီး Router ကို Run ပေးရမှာဖြစ်ပါတယ်။ ပထမဦးဆုံးအနေနဲ့ သေချာအောင် မူလ MongoDB ကို Stop လုပ်ထား သင့်ပါတယ်။

```
$ sudo service mongodb stop
$ sudo mongos --configdb cfg1.example.com:27019,cfg2.example.com:27019,cfg3.example.com:27019
```

ပြီးတဲ့အခါ `mongos` Command အတွက် `--configdb` Command နဲ့အတူ Config Server စာရင်းကိုတွဲဖက်ပေးရမှာဖြစ်ပါတယ်။ အကယ်၍ Query Router တစ်ခုထက်ပိုသုံးလိုတယ်ဆိုရင်၊ ထပ်မံထည့်သွင်းတဲ့ Query Router Server တွေမှာ `mongos` Command နဲ့အတူ Config Server စာရင်းကို ပထမ Server မှာ ပေးခဲ့တဲ့အတိုင်း ရှေ့နောက်အစီအစဉ် အတိအကျနဲ့ ပေးရမှာဖြစ်ပါတယ်။ ပါဝင်တဲ့ Config Server အရေအတွက် တူညီယုံသာမက၊ ပေးလိုက်တဲ့ ရှေ့နောက်အစီအစဉ်လည်း တူညီဖို့လိုအပ်ပါတယ်။

Query Router Server အတွက် Data Directory သတ်မှတ်ပေးစရာမလိုပါဘူး။ Query Router တစ်ခုကို သီးခြား Server တစ်လုံးနဲ့မထားပဲ Config Server (သို့မဟုတ်) ရိုးရိုး MongoDB Database Server နဲ့လဲ တွဲ Run ထားလို့ရနိုင်ပါတယ်။ ဒါပေမယ့် လက်တွေ့မှာ၊ သီးခြား Server တစ်လုံးနဲ့ထားလိုက်တာက စီမံရပိုမိုလွယ်ကူစေမှာ ဖြစ်ပါတယ်။

19.9 – Adding Replica Stes as Shard to Sharded Cluster

ဆက်လက်ပြီး MongoDB Server (သို့မဟုတ်) Replica Sets တွေကို အချက်အလက်တွေ အမှန်တစ်ကယ် သိမ်းဆည်းရာမှာ အသုံးပြုမယ့် Shard အဖြစ် Sharded Cluster ထဲမှာ ထည့်သွင်းပေးရမှာဖြစ်ပါတယ်။ ပထမဦးဆုံးအနေနဲ့ Query Router ကို Mongo Shell ကနေတစ်ဆင့် ချိတ်ဆက်ပေးရမှာဖြစ်ပါတယ်။

```
$ mongo --host router.example.com --port 27017
```

Query Router ကို Mongo Shell နဲ့ ချိတ်ဆက်ပြီးတဲ့အခါ `sh.addShard()` Function ကိုသုံးပြီး Shard တွေ စတင်ထည့်သွင်းနိုင်ပြီဖြစ်ပါတယ်။

```
> sh.addShard( "rs0/db1.example.com:27017" );
> sh.addShard( "rs1/db5.example.com:27017" );
```

နမူနာမှာ `rs0` Replica Sets နဲ့ `rs1` Replica Sets တို့ကို Sharded Cluster ထဲမှာ ထည့်သွင်းထားပါတယ်။ Replica Sets တစ်ခုမှာ Database Server အများအပြားပါဝင်နိုင်ပေမယ့် အားလုံးကို တစ်ခုချင်းထည့်ပေးစရာမလိုပါဘူး။ Replica Sets ထဲက Server တစ်လုံးကို ပေးလိုက်ရင် ကျန် Server တွေကို MongoDB က အလိုအလျောက် ရှာဖွေထည့်သွင်းပေးသွားမှာဖြစ်ပါတယ်။ ဒါကြောင့် နမူနာမှာ Replica Sets တွေထဲက Server

တစ်ခုစီကိုသာ `sh.addShard()` နဲ့ ထည့်သွင်းပေးထားခြင်းဖြစ်ပါတယ်။

အကယ်၍ Replica Sets မဟုတ်ပဲ ရိုးရိုး MongoDB Server တစ်လုံးကို ထည့်သွင်းချင်တာဆိုရင်တော့ အခုလို ထည့်သွင်းနိုင်ပါတယ်။

```
> sh.addShard("db10.example.com:27017");
```

နောက်ဆုံးက Port နံပါတ် 27017 က MongoDB Default Port ဖြစ်ပါတယ်။ မထည့်လည်းရပေမယ့် သေချာအောင် ထည့်သွင်းပေးထားခြင်းဖြစ်ပါတယ်။

ဆက်လက်ပြီး Shard လုပ်လိုတဲ့ Database ကို အခုလို သတ်မှတ်ပေးရပါမယ်။

```
> sh.enableSharding("mydb");
```

နမူနာအရ `mydb` Database အတွက် Sharding လုပ်ဆောင်ချက်ကို အသုံးပြုပြီး Data တွေကို Sharded Cluster ပေါ်မှာ ဖြန့် သိမ်းပေးသွားတော့မှာဖြစ်ပါတယ်။

ဘယ် Database တွေမှာ Shard လုပ်ဆောင်ချက် သုံးထားသလဲဆိုတာ သိချင်ရင်တော့ `config` Database မှာ ကြည့်နိုင်ပါတယ်။ `config` ဟာ Database Sharding နဲ့ပက်သက်တဲ့ အချက်အလက်တွေ သိမ်းဆည်းဖို့ အတွက် MongoDB အလိုအလျောက် တည်ဆောက်သွားတဲ့ Database တစ်ခုဖြစ်ပါတယ်။ ဥပမာ –

```
> use config;
> db.databases.find();

{ "_id" : "admin", "partitioned" : false, "primary" : "config" }
{ "_id" : "mydb", "partitioned" : true, "primary" : "shard0003" }
```

နမူနာအရ `mydb` Database ကို Partition လုပ်ထားတယ် (Shard လုပ်ထားတယ်) လို့ ဖော်ပြထားတာကို တွေ့ရမှာဖြစ် ပါတယ်။

19.10 – Enable Sharding for Collections

Database မှာ Shard လုပ်ဆောင်ချက် Enable လုပ်ပြီးတဲ့အခါ Collections တွေအတွက်လည်း Shard လုပ်ဆောင်ချက် Enable လုပ်ပေးရဦးမှာပါ။ MongoDB က Collection ထဲကအချက်အလက်တွေကို Sharded Cluster ပေါ်မှာ ဖြန့်သိမ်းတဲ့အခါ Shard Key လို့ခေါ်တဲ့အညွှန်းကိုကြည့်ပြီး အုပ်စုခွဲသိမ်းလေ့ရှိပါတယ်။ ရည်ရွယ်ချက်ကတော့ Data တွေ ရယူတဲ့အခါ၊ မလိုအပ်တဲ့ Shard နှစ်ခုသုံးခုကနေ ခွဲပြီးယူမနေပဲ တစ်ခုတည်းကနေ ရယူနိုင်ဖို့အတွက်ဖြစ်ပါတယ်။

ဥပမာ – `users` Collection အတွက် `city` Field ကို Shard Key အနေနဲ့ သတ်မှတ်ထားမယ်ဆိုရင်၊ မြို့တူတဲ့ သူတွေကို Shard တစ်ခုတည်းမှာ သိမ်းပေးသွားမှာဖြစ်ပါတယ်။ ဒါကြောင့် မြို့တစ်မြို့က User စာရင်းရယူတဲ့ အခါ Shard နှစ်ခုသုံးခုကနေ သွားယူစရာမလိုပဲ တစ်ခုတည်းကနေ ရယူနိုင်မှာဖြစ်ပါတယ်။

```
> sh.shardCollection("mydb.users", { "city": 1 } );
```

နမူနာအရ `city` Field ကို Shard Key အနေနဲ့သုံးပြီး `mydb` Database ရဲ့ `users` Collection ကို Shard လုပ်ပေး သွားမှာဖြစ်ပါတယ်။ ဒီနည်းရဲ့ အားနည်းချက်အနေနဲ့ User ၉၀% လောက်က မြို့တစ်မြို့တည်းကဖြစ်နေ မယ်ဆိုရင် အချက် အလက် ၉၀% ကို Shard တစ်ခုတည်းမှာသွားသိမ်းမှာဖြစ်လို့ Storage အသုံးပြုမှု မမျှတတာ မျိုးဖြစ်နိုင်ပါတယ်။ ဒီလို အခြေအနေမျိုးမှာ Compound Shard Key ကို အသုံးပြုနိုင်ပါတယ်။

```
> sh.shardCollection("mydb.users", { "city": 1, "date_of_birth": 1 } );
```

နမူနာအရ `city` Field ကို Shard Key အနေနဲ့အသုံးပြုပြီး လိုအပ်လို့ Data တွေကို ထပ်မံစိတ်ပိုင်းပြီး အခြား Shard မှာ သွားသိမ်းဖို့လိုမယ်ဆိုရင် `date_of_birth` ကိုသုံးပါလို့ သတ်မှတ်ထားခြင်းဖြစ်ပါတယ်။ ဒါ ကြောင့် မြို့တစ်မြို့တည်း က User တွေကို Shard တစ်ခုတည်းမှာ စုပြုံသိမ်းရလို့ Shard Storage မလောက်တဲ့ အခါ၊ အဲ့ဒီ မြို့က User တွေရဲ့ မွေးနေ့နဲ့ အချက်အလက်တွေကို စိတ်ပိုင်းပြီး အခြား Shard တစ်ခုမှာ ခွဲသိမ်းပေး မှာဖြစ်ပါတယ်။ ဥပမာ – ရန်ကုန် User တွေထဲက ၁၉၈၀ ကနေ ၁၉၉၀ ကြားမွေးသူတွေကို Shard 1 မှာသိမ်းပြီး ၁၉၉၀ နောက်ပိုင်းမွေးသူတွေကို Shard 2 မှာ သိမ်းပေးတဲ့သဘောမျိုး ဖြစ်ပါတယ်။

ဒါကြောင့် MongoDB ရဲ့ လုပ်ဆောင်ချက်မြန်ဆန်စေဖို့အတွက် Shard Key ရွေးချယ်တဲ့အခါ သတိပြုရွေးချယ်ရ မှာဖြစ်ပါတယ်။ အကယ်၍ ဘယ် Field ကို Shard Key အဖြစ် ရွေးချယ်ရမယ်ဆိုတာ မဆုံးဖြတ်နိုင်ရင် `_id` Field ကို အသုံးပြု နိုင်ပါတယ်။ `_id` ဆိုတာ ဘယ်တော့မှ ပြန်ထပ်မှာမဟုတ်လို့ Shard တစ်ခုတည်းမှာ Data တွေ စုပြုံနေတာမျိုး မဖြစ်စေ တော့ပါဘူး။ ဒါပေမယ့် Data တွေရယူတဲ့အခါ Data Set တစ်ခုရဖို့အတွက် Shard နှစ်ခုသုံးခုကနေ ဖြန့်ယူရတာမျိုး ဖြစ်နိုင်တာကိုတော့ နားလည်ထားရမှာဖြစ်ပါတယ်။

```
> sh.shardCollection("mydb.users", { "_id": "hashed" } );
```

နောက်တစ်ချက်အနေနဲ့ သတိပြုသင့်တာကတော့၊ Shard Key အဖြစ် အသုံးပြုမယ့် Field ကို Index လုပ်ထားဖို့ လိုပါ တယ်။ အကယ်၍ Collection က Data တွေ မရှိသေးပဲ အလွတ်ဆိုရင် `sh.shardCollection()` Function က Shard Key အဖြစ်သုံးဖို့ပေးလိုက်တဲ့ Field တွေကို အလိုအလျောက် Index လုပ်ပေးသွားမှာဖြစ်ပါ တယ်။ အကယ်၍ မူလက Data တွေ ရှိနေပြီးဖြစ်တယ်ဆိုရင်တော့ ကိုယ်တိုင် Index လုပ်ပေဖို့ လိုနိုင်ပါတယ်။ Index ပြုလုပ်ပုံကို **အခန်း (၁၁)** မှာ ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။

အထက်ကနမူနာမှာ `{ "_id": 1 }` လို့ မသုံးပဲ `{ "_id": "hashed" }` လို့သုံးထားတာကို သတိပြုမိနိုင်ပါတယ်။ အဓိပ္ပါယ်ကတော့၊ `_id` Field ကို Index လုပ်တဲ့အခါ ရိုးရိုး Index ကိုမသုံးပဲ Hashed Index ကို သုံးမယ်လို့ ဆို

လိုက်ခြင်းဖြစ်ပါတယ်။ Hashed Index ကို ရိုးရိုး Number နဲ့ String တန်ဖိုးမဟုတ်တဲ့တန်ဖိုးတွေ သိမ်းဖို့သုံးတဲ့ Field တွေမှာ သတ်မှတ်သင့်ပါတယ်။ MongoDB က Field ထဲက တန်ဖိုးတွေကို Index မလုပ်မီ၊ Hash Summary ပြောင်းပြီးတော့မှ Index လုပ်ပေးမှာဖြစ်ပါတယ်။ `_id` Field ဟာဆိုရင်လည်း ရိုးရိုးတန်ဖိုးတွေသိမ်းတဲ့ Field မဟုတ်ပဲ `ObjectId()` Function သုံးပြီးသိမ်းပြီး ID တွေကိုသိမ်းတဲ့ Field ဖြစ်တဲ့အတွက် သူ့ကို Shard Key အနေနဲ့သုံးမယ် ဆိုရင် Hashed Index လုပ်ပြီး အသုံးပြုသင့်တဲ့အတွက် { "_id": "hashed" } ကို အသုံးပြုထားခြင်းပဲဖြစ် ပါတယ်။

လက်ရှိ Sharded Cluster Configuration ကို လေ့လာလိုရင် Query Router ကို Mongo Shell နဲ့ချိတ်ဆက်ပြီး အခုလို လေ့လာနိုင်ပါတယ်။

```
> sh.status();

--- Sharding Status ---
  sharding version: { "_id" : 1, "version" : 3 }
  shards:
    {  "_id" : "shard0000",  "host" : "db1.example.com:27017" }
    {  "_id" : "shard0001",  "host" : "db5.example.com:27017" }
  databases:
    {  "_id" : "admin",  "partitioned" : false,  "primary" : "config" }
    {  "_id" : "mydb",  "partitioned" : true,  "primary" : "shard0000" }
        mydb.users
            shard key: { "roll_no" : 1 }
            chunks:
                shard0001    2
                shard0000    2
            { "roll_no" : { "$minKey":1 } } -->> { "roll_no" : "56000" } on : shard0001 { "t":2, "i":0 }
            { "roll_no" : 57500 } -->> { "roll_no" : "58140" } on : shard0001 { "t" : 4, "i" : 0 }
            { "roll_no" : 58140 } -->> { "roll_no" : "59000" } on : shard0000 { "t" : 4, "i" : 1 }
            { "roll_no" : 59000 } -->> { "roll_no" : { "$maxKey":1 } } on : shard0000 { "t":3, "i":3 }
```

`sh.status()` ကို အသုံးပြုရခြင်းဖြစ်ပြီး လက်ရှိ Data တွေကို Cluster ပေါ်မှာ ဘယ်ပုံဘယ်နည်းသိမ်းထားတယ် ဆိုတဲ့ အချက်အလက်တွေကို ဖော်ပြပေးလာမှာပဲဖြစ်ပါတယ်။

19.11 – Removing a Shard from Cluster

Cluster ထဲမှာပါဝင်နေပြီး လက်ရှိလည်း အချက်အလက်တွေ လက်ခံသိမ်းဆည်းနေတဲ့ Shard တစ်ခုကို အကြောင်းအမျိုးမျိုးကြောင့် ပြန်လည်ဖယ်ထုတ်တော့မယ်ဆိုရင်၊ အဲ့ဒီ Shard မှာသိမ်းထားတဲ့ Data တွေကို ကျန် Shard တွေထံပေးပို့တဲ့ လုပ်ငန်းကို အရင်လုပ်ရမှာဖြစ်ပါတယ်။ ပထမဦးဆုံးအနေနဲ့ Shard ရဲ့ ID ကို သိရဖို့အတွက် `sh.status()` ကို အသုံးပြုရပါမယ်။

```
> sh.status();

--- Sharding Status ---
  sharding version: { "_id" : 1, "version" : 3 }
  shards:
    {  "_id" : "shard0000",  "host" : "db1.example.com:27017" }
    {  "_id" : "shard0001",  "host" : "db5.example.com:27017" }
    {  "_id" : "shard0002",  "host" : "db10.example.com:27017" }
  databases:
    {  "_id" : "admin",  "partitioned" : false,  "primary" : "config" }
    {  "_id" : "mydb",  "partitioned" : true,  "primary" : "shard0000" }
… …
```

နမူနာအရ shard0000, shard0001 နဲ့ shard0002 ဆိုတဲ့ Shard သုံးခုရှိနေပါတယ်။ အဲ့ဒီထဲက shard0002 ကို ဖယ်ထုတ်လိုရင် Query Router ပေါ်မှာ အခုလို ဆောင်ရွက်နိုင်ပါတယ်။

```
> use admin;
> db.runCommand( { removeShard: "shard0002" } );

{
    "msg" : "draining started successfully",
    "state" : "started",
    "shard" : "mongodb0",
    "ok" : 1
}
```

`admin` Database ပေါ်မှာ `removeShard` ကို Run ပေးလိုက်ခြင်းဖြစ်ပါတယ်။ နမူနာရလဒ်ရဲ့ `msg` မှာ "draining started successfully" ကိုတွေ့ရမှာဖြစ်ပါတယ်။ Data တွေကို အခြား Shard တွေပေါ်စရွှေ့နေပြီဆိုတဲ့ အဓိပ္ပါယ်ဖြစ်ပါ တယ်။ ရွှေ့တဲ့လုပ်ငန်း ပြီးမပြီးသိရဖို့အတွက် `removeShard` ကို နောက်တစ်ကြိမ် ထပ် Run ကြည့်နိုင်ပါတယ်။

```
> use admin;
> db.runCommand( { removeShard: "shard0002" } );

{
     "msg" : "draining ongoing",
    "state" : "ongoing",
    "remaining" : {
        "chunks" : 42,
        "dbs" : 1
    },
    "ok" : 1
}
```

ဒီတစ်ကြိမ်မှာတော့ `msg` တန်ဖိုးက "drainin ongoing" ဖြစ်သွားပါပြီ။ တနည်းအားဖြင့် Data တွေရွှေ့နေဆဲဖြစ်တယ်ဆို တဲ့အဓိပ္ပါယ်ဖြစ်ပါတယ်။ `remaining.chunks` မှာလည်း တန်ဖိုးတစ်ခုရှိနေတာကို တွေ့ရမှာဖြစ်ပါတယ်။ အခြေခံ အားဖြင့် `remaining.chunks` တန်ဖိုး 0 ဖြစ်သွားရင် Data ရွှေ့တဲ့လုပ်ငန်း ပြီးစီးပြီဖြစ်ပါတယ်။

Database တစ်ခုက Cluster ကနေ ဖယ်ထုတ်လိုတဲ့ Shard ကို Primary Shard အဖြစ်အသုံးပြုထားတယ်ဆိုရင် အပြီးသတ် မဖယ်ထုတ်ခင် အဲ့ဒီပြဿနာကို ဖြေရှင်းပေးရပါဦးမယ်။ Primary ဟုတ်မဟုတ် `sh.status()` ရလဒ်ကို လေ့လာကြည့်ရင် သိရနိုင်ပါတယ်။ `sh.status()` ရလဒ်မှာ အခုလိုပါဝင်နေနိုင်ပါတယ်။

```
{  "_id" : "somedb",  "partitioned" : true,  "primary" : "shard0002" }
```

အဓိပ္ပါယ်က `somedb` ဆိုတဲ့ Database ဟာ shard0002 ကို Primary Shard အနေနဲ့ အသုံးပြုထားတယ်ဆိုတဲ့ အဓိပ္ပါယ်ဖြစ်ပါတယ်။ ဒီလိုအခြေအနေမျိုးမှာ၊ `somedb` ရဲ့ Primary ကို အရင်ပြောင်းပေးဖို့ လိုအပ်ပါတယ်။

```
> use admin;
> db.runCommand( { movePrimary: "somedb", to: "shard0000" });
```

`admin` Database ပေါ်မှာ `somedb` ရဲ့ Primary အဖြစ် shard0000 ကို ပြောင်းသုံးဖို့ သတ်မှတ်ပေးလိုက်ခြင်း ဖြစ်ပါတယ်။ တစ်ခုသတိပြုရမှာက၊ ဒီလုပ်ဆောင်ချက်ကို `remaining.chunks` တန်ဖိုး 0 ဖြစ်ပြီးမှသာ ဆောင်ရွက်ရမှာဖြစ်ပါတယ်။

Data တွေလည်း အခြား Shard ကိုပြောင်းပြီးပြီ။ လက်ရှိ Shard ကို Primary အဖြစ်အသုံးပြုထားတဲ့ Database တွေ ကိုလည်း Primary ပြောင်းပြီးပြီဆိုရင် `removeShard` ကို နောက်တစ်ကြိမ် Run ခြင်းအားဖြင့် Shard ကို Cluster ထဲကနေ အပြီးသတ် ဖယ်ထုတ်နိုင်ပါတယ်။

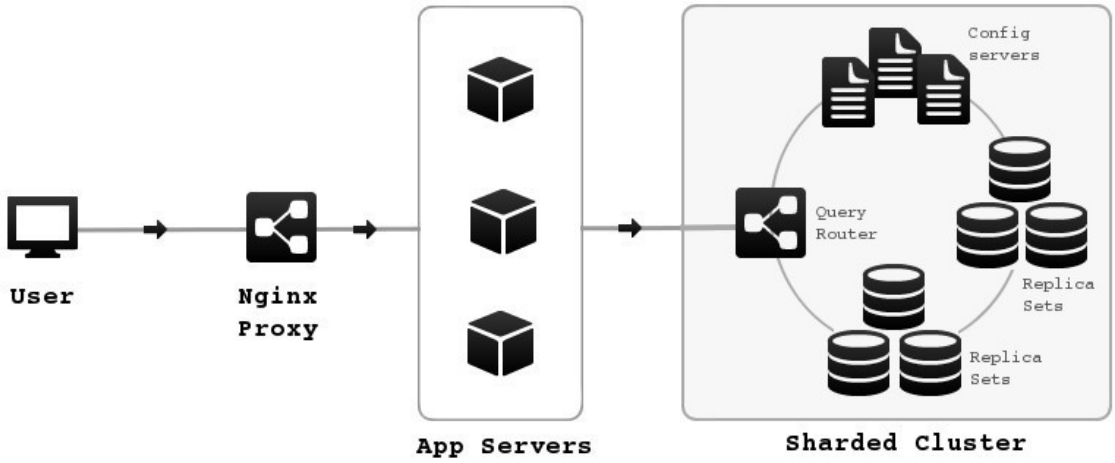
```
> use admin;
> db.runCommand( { removeShard: "shard0002" } );

{
    "msg" : "removeshard completed successfully",
    "state" : "completed",
    "shard" : "shard0002",
    "ok" : 1
}
```

ဒီတစ်ခါရရှိတဲ့ `msg` ကတော့ removeshard completed successfully ဖြစ်ပါတယ်။ ဒါဆိုရင် Shard ကို Cluster ထဲကနေဖယ်ထုတ်ခြင်းလုပ်ငန်း ပြီးစီးသွားပြီပဲဖြစ်ပါတယ်။

Conclusion

ဒီအခန်းရဲ့အစမှာ Database Replication နဲ့ Sharding မပါဝင်သေးတဲ့အခါ ဖြစ်နိုင်ခြေရှိတဲ့ Architecture ပုံသဏ္ဌာန်ကို ဖော်ပြခဲ့ပါတယ်။ အဲ့ဒီ Architecture မှာ Replication နဲ့ Sharding ဖြည့်စွက်လိုက်တဲ့အခါမှာ ရရှိလာနိုင်တဲ့ ပုံသဏ္ဌာန် ကတော့ အခုလိုဖြစ်မှာပါ။



ပုံ (၁၉.၅) - Scalable Architecture with Nginx and Sharded Cluster

ဒီနည်းနဲ့ အသုံးပြုသူများလာလို့ Process Power လိုအပ်လာတဲ့အခါ App Server တွေကို အလွယ်တစ်ကူထပ်ထည့်နိုင်စေမှာဖြစ်သလို၊ Storage Capacity တိုးမြှင့်ဖို့လိုအပ်လာတဲ့အခါ MongoDB Server တွေကို Sharded Cluster ထဲမှာ ထပ်ထည့်ခြင်းအားဖြင့် အလွယ်တစ်ကူ တိုးမြှင့်နိုင်မှာပဲဖြစ်ပါတယ်။ တနည်းအားဖြင့်၊ အသုံးပြုသူ များလာရင်များလာသလောက် Resource ပမာဏကို လိုက်လံတိုးမြှင့်သွားနိုင်တဲ့ Scalable Architecture တစ်ခုကို ရရှိပြီပဲဖြစ်ပါတယ်။

MongoDB Sharded Cluster နဲ့ပက်သက်တဲ့ အကြောင်းအရာအသေးစိတ်ကို အောက်ပါလိပ်စာမှာ ဆက်လက်လေ့လာနိုင်ပါတယ်။

http://docs.mongodb.org/manual/sharding/

Front-end, Back-end, UI Design စသဖြင့် Software Development မှာပါဝင်တဲ့ လုပ်ငန်းအားလုံးကို ဆောင်ရွက်နိုင်စွမ်းရှိပြီး Server Architecture လို ကိစ္စမျိုးကိုပါနားလည်တာဝန်ယူနိုင်သူကို Full-stack Developer လို့ ခေါ်ပါတယ်။

Professional Web Developer Course

HTML5, PHP/MySQL, jQuery/Ajax, Mobile Web စသည့်
Professional Web Developer တစ်ဦး သိရှိထားသင့်သည့်
နည်းပညာများကို စုစည်းသင်ကြားခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

အခန်း(၂၀) – Common Server Architectures

ပုံမှန်အားဖြင့် Software တစ်ခု ရေးသားဖန်တီးရာမှာပါဝင်ရသူတွေကို သက်ဆိုင်ရာတာဝန်အလိုက် Programmer, Designer, Software Engineer, Web Master, Front-end Developer, Back-end Developer, Database Administrator, System Administrator, Network Engineer စသဖြင့်အမျိုးမျိုး ခေါ်ကြပါတယ်။ အခုနောက်ပိုင်းမှာတော့ Full-stack Developer ဆိုတဲ့ အသုံးအနှုန်းတစ်မျိုး ခေတ်စားလာခဲ့ပါတယ်။

Full-stack Developer ဆိုတာ Software စတင်ရေးသားခြင်းကနေ နောက်ဆုံး အများအသုံးပြုနိုင်အောင် Release လုပ်တဲ့အထိ လိုအပ်တဲ့လုပ်ငန်းစဉ်တိုင်းကို တာဝန်ယူအလုပ်လုပ်နိုင်တဲ့ စွယ်စုံရ Developer ကို ခေါ်ခြင်းဖြစ်ပါတယ်။ Code ရေးသားခြင်းကို အဓိကကျွမ်းကျင်သူဖြစ်နိုင်ပေမယ့်၊ လိုအပ်ရင် UI Design တွေ ရေးဆွဲနိုင်သူလည်းဖြစ်ပါတယ်။ Data Structure နဲ့ System Design တွေကိုလည်း ရေးဆွဲနိုင်သူဖြစ်ပါတယ်။ Client App တွေရေးသားနိုင်သလို၊ Back-end Service တွေလည်း ရေးသားနိုင်သူဖြစ်ပါတယ်။ Server Operating System နဲ့ Server Architecture ကို စီမံတပ်ဆင်နိုင်သူဖြစ်ပါတယ်။ ဒီလို Software Development Process မှာ ပါဝင်တဲ့ လုပ်ငန်းတိုင်းကို ကိုယ်တိုင်ဆောင် ရွက်နိုင်သူကို Full-stack Developer လို့ခေါ်တာပါ။ Programming Language အနေနဲ့လည်း နှစ်မျိုး သုံးမျိုး အသုံးပြု ရေးသားနိုင်ပြီး၊ အခြားဆက်စပ်နည်းပညာတွေနဲ့လည်း ထိုက်သင့်သလောက် ရင်းနှီးကျွမ်းဝင် အသုံးချနိုင်သူဖြစ်ပါတယ်။

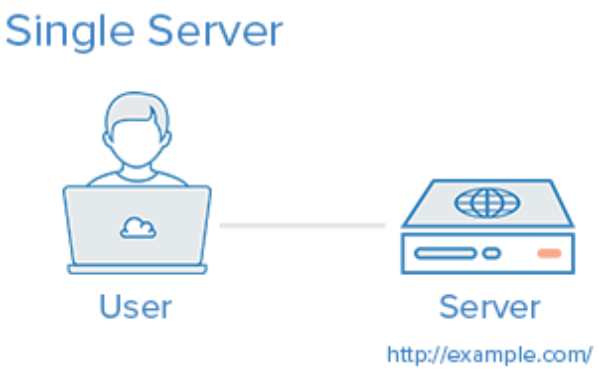
သင်ကြားရေးတဲ့ လေ့လာရေးမှာ Full-stack Developer ဖြစ်အောင်လေ့လာခြင်းနဲ့ သက်ဆိုင်ရာဘာသာရပ်တစ်ခုမှာ အထူးပြုကျွမ်းကျင်တဲ့ Specialist ဖြစ်အောင်လေ့လာခြင်းတို့ကြား၊ တစ်ဦးနဲ့တစ်ဦး အယူအဆမတူ အငြင်းပွားမှုတစ်ချို့ရှိ ကြပါတယ်။ Software Development Team တွေမှာလဲ၊ Specialist တွေကပိုထိရောက်တယ်၊ Full-stack Developer တွေက ပိုထိရောက်တယ်ဆိုတဲ့ အငြင်းအခုံတွေ ရှိကြပါတယ်။ ဒါပေမယ့် အခြေအနေအရပ်ရပ်ကြောင့် ကနေ့ခေတ်မှာ Full-stack Developer တွေ ပိုမိုလိုအပ်လာတယ်ဆိုတဲ့ အချက်ကတော့ ရှောင်လွဲလို့မရပါဘူး။

Engineer တွေ၊ Designer တွေနဲ့ Developer အင်အား၊ ရာထောင်ချီရှိတဲ့ Software ကုမ္ပဏီကြီးတွေမှာ သူ့နေရာနဲ့သူ ကျွမ်းကျင်တဲ့ Specialist တွေ ခန့်ထားပြီး လုပ်ငန်းဆောင်ရွက်နိုင်ပေမယ့်၊ Startup နဲ့ Software Team လေးတွေမှာ တော့ ကဏ္ဍတိုင်းအတွက် Specialist တွေကို ခန့်အပ်တာဝန်ပေးနိုင်မှာမဟုတ်ပါဘူး။ ဒါကြောင့် Developer တစ်ဦးအနေ နဲ့ လုပ်ငန်းတာဝန်နှစ်မျိုးသုံးမျိုးကို တာဝန်ယူရတက်ပါတယ်။ ကျွန်တော်တို့ဆီမှာ ပို ဆိုးပါတယ်။ သက်ဆိုင်ရာကဏ္ဍမှာ ကျွမ်းကျင်တဲ့ Specialist ဆိုတာ အမြဲမရှိနိုင်ပါတယ်။ ၁၀၀% Full-stack Developer မဟုတ်ရင်တောင်မှ၊ Client App ရော၊ Back-end Service ကိုပါ ရေးနိုင်သူတစ်ဦးဖြစ်ဖို့ လိုအပ်တက်ပါတယ်။ ရေးသားထားတဲ့ Software နဲ့ Service တွေကို Host လုပ်မယ့် Server Architecture ကို Developer ကိုယ်တိုင် စီမံဖို့လိုနိုင်ပါတယ်။

ပြီးခဲ့တဲ့အခန်းတွေမှာ Docker, Nginx, Sharded Cluster စတဲ့နည်းပညာတွေကို ဖော်ပြခဲ့ရာမှာလည်း Developer တွေ ကိုယ်တိုင် Server Architecture တွေတည်ဆောက်ရာမှာ အထောက်အကူဖြစ်စေဖို့ဆိုတဲ့ ရည်ရွယ်ချက်နဲ့ပဲ ဖော်ပြခဲ့ခြင်း ဖြစ်ပါတယ်။ ဒါပေမယ့် ဒီနည်းပညာတွေကို အမြဲအသုံးပြုရမယ်ဆိုတဲ့ သဘောတော့လည်း မဟုတ်ပါဘူး။ ကိုယ့်ရဲ့ လိုအပ် ချက်နဲ့ချင့်ချိန်အသုံးပြုရမှာဖြစ်ပါတယ်။ ဒီအခန်းမှာတော့ Server Architecture အမျိုးမျိုးနဲ့ သူတို့ရဲ့ အားသာချက် အားနည်းချက်တွေကို အကျဉ်းချုပ် ဖော်ပြသွားမှာပဲဖြစ်ပါတယ်။

20.1 – Everything on One Server

Server Architecture တွေထဲမှာ အခြေခံအကျဆုံးနဲ့ အရိုးရှင်းဆုံးနည်းစနစ်ကတော့ လိုအပ်တဲ့ Service အားလုံးကို Server တစ်ခုတည်းမှာ စုစည်းထားခြင်းပဲဖြစ်ပါတယ်။ ရှေ့ပိုင်းမှာ အသုံးပြုဖော်ပြခဲ့တဲ့ နည်းပညာတွေအရ ဆိုရင် Nginx, MongoDB နဲ့ ExpressJS API Server အားလုံးကို Server ကွန်ပျူတာ တစ်လုံးတည်းမှာ စုစည်းထည့်သွင်းထားခြင်းပဲ ဖြစ်ပါတယ်။



ပုံ (၂၀.၁) - Single Server Setup

source – digitalocean.com

ဒီနည်းကပေးတဲ့ အားသာချက်ကတော့ Setup လုပ်ရတာ အလွန်ရိုးရှင်းလွယ်ကူခြင်းပဲ ဖြစ်ပါတယ်။ နောက် အားသာချက် တစ်ခုကတော့ ကုန်ကျစရိတ်သက်သာခြင်း ဖြစ်ပါတယ်။ Server ကွန်ပျူတာတစ်လုံးပဲ လိုအပ်မှာဖြစ်လို့ ကိုယ်တိုင် Server Hardware ဝယ်ယူမယ်ဆိုရင်လည်း ကုန်ကျစရိတ်သက်သာသလို၊ VPS (သို့မဟုတ်) အခြား Hosting Service တွေ ဝယ်ယူအသုံးပြုမယ်ဆိုရင်လည်း စရိတ်စကသက်သာမှာဖြစ်ပါတယ်။ ဒါပေမယ့် နောင်အသုံးပြုသူများလာတဲ့အခါ ခံနိုင်ရည် ရှိစေဖို့ရည်ရွယ်ပြီး၊ လက်ရှိမလိုအပ်သေးပေမယ့်၊ လိုတာထက်ပို ကောင်းတဲ့ Hardware Server ကို ကြိုတင်တပ်ဆင်ထား ရတာမျိုးတော့ ဖြစ်နိုင်ပါတယ်။

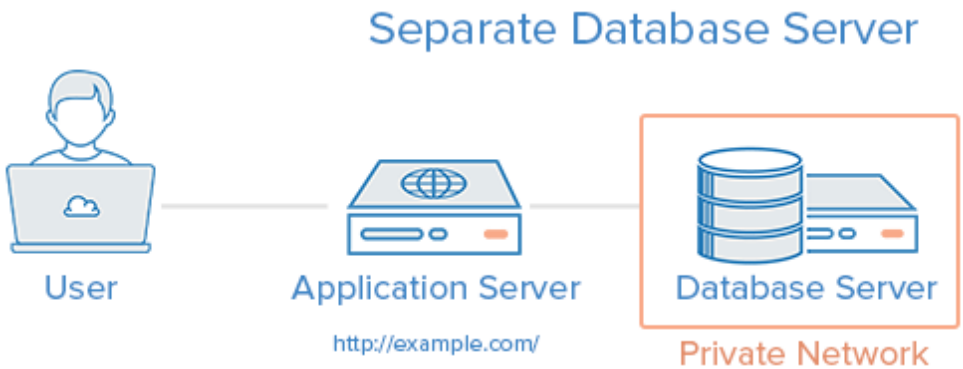
ဒီနည်းရဲ့ အဓိကအကျဆုံး အားနည်းချက်ကတော့ Scalable မဖြစ်ခြင်းပဲဖြစ်ပါတယ်။ နောင်အသုံးပြုသူများလာလို့ RAM (သို့မဟုတ်) Storage Capacity တိုးမြှင့်ဖို့လိုအပ်လာတဲ့အခါ လက်ရှိ Server မှာပဲ RAM နဲ့ HDD တွေ ထပ်တပ် ခြင်းအားဖြင့် လိုအပ်တဲ့ Capacity ရအောင် Upgrade လုပ်ရတော့မှာပါ။ ဒီနည်းကို Scale Up လုပ်တယ်လို့ ခေါ်ပါတယ်။ Vertical Scaling လို့လည်း ခေါ်ပါတယ်။ Scale Up လုပ်တာဟာ အကန့်အသတ်ရှိပါတယ်။ ဥပမာ – Hardware ကန့်သတ်ချက်အရ RAM ပမာဏ 16 GB လောက်ထိပဲ တိုးလို့ရတာမျိုးဖြစ်နိုင်ပါတယ်။ ဒီထက်ပိုလိုလာရင် ထပ်တိုးလို့ ရမှာမဟုတ်တော့ပါဘူး။ အလားတူပဲ Server OS ကန့်သတ်ချက်အရ

HDD ပမာဏ အမြင့်ဆုံး 16TB ထိပဲ လက်ခံ အလုပ်လုပ်ပေးနိုင်တဲ့ အကန့်အသတ်မျိုးတွေ ရှိတက်ပါတယ်။ ဒီထက်ပိုလိုအပ်လာရင် အဆင်ပြေမှာမဟုတ်တော့ပါဘူး။

ဒါကြောင့် ကန့်သတ်ပမာဏတစ်ခုထက် Resource လိုအပ်ချက် အခုမလိုသလို နောင်လည်း လိုလာစရာမရှိဘူးဆိုတဲ့ App မျိုးတွေမှာသာ ဒီနည်းကိုအသုံးပြုသင့်ပါတယ်။ နောင်တစ်ချိန်မှာ Scale လုပ်ဖို့လိုအပ်နိုင်တဲ့ App တွေမှာဆိုရင်တော့ ဒီနည်းကို မသုံးသင့်ပါဘူး။

20.2 – Separate Database Server

နောက်တစ်နည်းကတော့ Database Server ကို သီးခြား Server ကွန်ပျူတာတစ်လုံးနဲ့ ခွဲထုတ်တပ်ဆင်တဲ့နည်းဖြစ်ပါ တယ်။ ကျွန်တော်တို့ ရှေ့ပိုင်းမှာ ဖော်ပြခဲ့တဲ့ နည်းပညာတွေအရဆိုရင် Nginx Web Server နဲ့ ExpressJS API Server ကို Server တစ်လုံးနဲ့ထားပြီး MongoDB ကို သီးခြား Server တစ်လုံးနဲ့ထားရှိရမှာ ဖြစ်ပါတယ်။



ပုံ (၂၀.၂) - Separate Database Server Setup

source – digitalocean.com

ဒီနည်းရဲ့အားသာချက်ကတော့ Resource အသုံးပြုမှုကို Server နှစ်လုံးခွဲပြီး မျှလိုက်တဲ့အတွက် Server တစ်လုံးတည်း အသုံးပြုတာထက် ပိုမိုထိရောက် မြန်ဆန်သွားမှာဖြစ်ပါတယ်။ နောင်လိုအပ်လို့ Scale လုပ်ရမယ်ဆိုရင်လည်း App Server ကို သပ်သပ်၊ Database Server ကိုသပ်သပ် Scale Up လုပ်သွားနိုင်မှာဖြစ်ပါတယ်။ Server နှစ်လုံးဖြစ်သွားတာနဲ့အညီ ရရှိနိုင်တဲ့ အမြင့်ဆုံး Capacity လည်း နှစ်ဆမြင့်တက်သွားမှာပဲဖြစ်ပါတယ်။ ဒါ့အပြင် Data Storage လိုရင် Database Server ကိုသာ Scale Up လုပ်ပြီး၊ App Server ကို မလိုအပ်ရင်ချန်ထားနိုင်တဲ့အားသာချက်မျိုးလည်း ရရှိနိုင်ပါသေး တယ်။

နောက်အားသာချက်တစ်ခုကတော့ App Server ကိုသာ Public Internet ကနေ တိုက်ရိုက်အသုံးပြုခွင့်ပေးပြီး Database Server ကိုတော့ Public Internet ကနေ Access လုပ်ခွင့်မပေးပဲ၊ Private Network ထဲမှာသာထားပြီး၊ App Server ကိုသာ Access လုပ်ခွင့် ပြုထားနိုင်တဲ့အတွက် Database Server လုံခြုံရေးအတွက်လည်း ပိုကောင်းသွားမှာ ဖြစ်ပါတယ်။

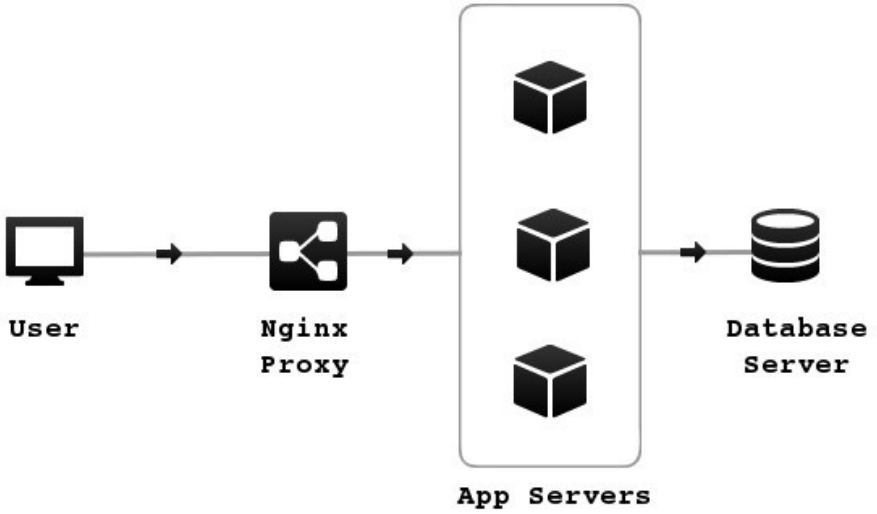
သူလည်းပဲ Setup လုပ်ရတာ ဘာမှမခက်ခဲပါဘူး။ ရိုးရှင်းလွယ်ကူပါတယ်။ ဒါပေမယ့် ဒီနည်းမှာလည်း

Scalability ပိုင်းမှာ အကန့်အသတ်ရှိနေဦးမှာဖြစ်ပါတယ်။ တော်ယုံတန်ယုံ App တွေအတွက် Server တစ်လုံးတည်းထားခြင်းထက် ဒီနည်းနဲ့ Database Server ကို သီးခြားခွဲထားခြင်းက ပိုမိုသင့်တော်မှာဖြစ်ပါတယ်။

20.3 – Load Balancer (Reverse Proxy)

နောက်တစ်နည်းကတော့ Load Balancer အသုံးပြုခြင်းပဲဖြစ်ပါတယ်။ **အခန်း (၁၈)** မှာ Nginx ကို သုံးပြီး Load Balance ပြုလုပ်နိုင်ပုံကို ဖော်ပြခဲ့ပြီးဖြစ်ပါတယ်။ Load Balancer အသုံးပြုရခြင်းရည်ရွယ်ချက်ကတော့ App Server တစ်လုံးထက်ပိုပြီး အသုံးပြုနိုင်ဖို့ဖြစ်ပါတယ်။ ဒီနည်းနဲ့ နောင်လိုအပ်လို့ App Server ရဲ့ စွမ်းဆောင်ရည်မြှင့်တင်ရတော့ မယ်ဆိုရင် လက်ရှိ Server မှာ RAM တွေ၊ HDD တွေ သွားတိုးနေစရာမလိုပဲ၊ နောက်ထပ် Server တစ်လုံးထပ်တိုးလိုက် ခြင်းအားဖြင့် Scale Out ပြုလုပ်သွားနိုင်မှာဖြစ်ပါတယ်။ ဒီနည်းကို Horizontal Scaling လို့လည်း ခေါ်ပါတယ်။

ဒီနည်းရဲ့အားသာချက်ကတော့ Scale လုပ်ရလွယ်ခြင်းဖြစ်ပါတယ်။ Scale လုပ်ရလွယ်တဲ့အတွက်အစပထမပိုင်းမှာ Server Hardware တွေကို လိုတာထက်ပိုပြီး တပ်ဆင်ထားရတာမျိုး လုပ်စရာမလိုတော့ပါဘူး။ နောင်လိုအပ်လာတော့မှ လို သလောက်ကိုသာသီးခြား Server အနေနဲ့ ထပ်တိုးနိုင်တဲ့အတွက် လက်ရှိအမှန်တစ်ကယ်လိုအပ်သလောက်ကိုသာ တပ်ဆင် ထားဖို့လိုတဲ့ အားသာချက်ရှိပါတယ်။



ပုံ (၂၀.၃) - Load Balancer Setup

ဒီနည်းကပေးတဲ့ နောက်ထပ်အားသာချက်ကတော့ **အခန်း (၁၈)** မှာလည်း ဖော်ပြခဲ့ပြီးဖြစ်တဲ့ Security ဖြစ်ပါတယ်။ မလို လားအပ်တဲ့ Request တွေကို App Server ထံရောက်ဖို့မလိုပဲ ကြားခံ Load Balancer Proxy ကနေ စီစစ်နိုင်တဲ့ အား သာချက်ကို ဖြည့်စွက်ရရှိမှာဖြစ်ပါတယ်။ ဒါ့အပြင် Cache လုပ်ဆောင်ချက်အကူအညီနဲ့ တစ်ချို့ Request တွေကို App Server ထံရောက်စရာမလိုပဲ Proxy က အကြောင်းပြန်ပေးအောင် စီမံထားနိုင်ပုံကိုလဲ တွေ့မြင်ခဲ့ရပြီးဖြစ်ပါတယ်။

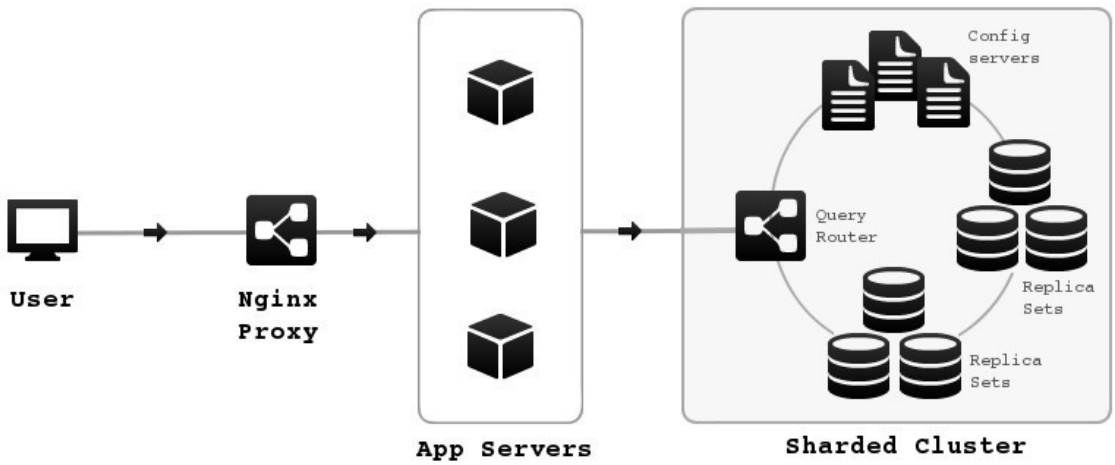
အားနည်းချက်အနေနဲ့ကတော့ တပ်ဆင်စီမံရ အနည်းငယ် ခက်ခဲနိုင်ပါတယ်။ Load Balancer နဲ့ App Server

တွေကြား Session တွေကို ဘယ်လိုစီမံမလဲ၊ SSL Encryption လုပ်ငန်းအဆင်ပြေအောင် ဘယ်လို Setup လုပ်ရမလဲ၊ စတဲ့ ဖြည့်စွက်စဉ်းစားရမှာတွေ၊ ဖြည့်စွက်စီမံရတာတွေ ရှိလာမှာပဲဖြစ်ပါတယ်။ ဒါအပြင် Resource လိုလာတိုင်း Server အသစ်တစ်လုံးထပ်တိုးရတယ်ဆိုတဲ့စရိတ်က လိုအပ်တဲ့ RAM နဲ့ HDD ပဲ ရွေးပြီးတိုးရတဲ့စရိတ်ထက် ပိုများမှာဖြစ်ပါ တယ်။ အကန့်အသတ်မရှိသလောက် Scale လုပ်သွားနိုင်တဲ့ အတွက်ရေရှည်မှာ စရိတ်သက်သာသွား လိမ့်မယ်လို့ ဆိုနိုင် ပေမယ့် အစပိုင်းမှာတော့ အနည်းငယ် ပိုမိုကုန်ကျမှာဖြစ်ပါတယ်။

အသုံးပြုသူပမာဏ ခန့်မှန်းနိုင်ခြင်းမရှိပဲ လိုအပ်သလို Scale လုပ်ဖို့လိုအပ်တဲ့ App တွေအတွက် ဒီနည်းစနစ်က သင့်တော် ပါတယ်။

20.4 – Sharded Cluster

ပြီးခဲ့တဲ့အခန်းမှာ Sharded Cluster အကူအညီနဲ့ Database Server ကိုလည်း Scale လုပ်နိုင်ပုံကို ဖော်ပြခဲ့ပြီးဖြစ်ပါ တယ်။ Load Balancer နဲ့ Shard Cluster လုပ်ဆောင်ချက် အကူအညီနဲ့ အပြည့်အဝ Scalable ဖြစ်တဲ့ ပိုမိုပြည့်စုံတဲ့ Server Architecture ကို တပ်ဆင်ထားနိုင်မှာဖြစ်ပါတယ်။



ပုံ (၂၀.၄) - Load Balancer and Sharded Cluster

ဒီနည်းစနစ်ကတော့ App Server ကိုရော၊ Database Server ကိုပါ Scale Out လုပ်နိုင်မှာ ဖြစ်တဲ့အတွက် ရေရှည်မှာ အပြည့်အဝ Scalable ဖြစ်လိုတယ်ဆိုရင် သုံးရမယ့် နည်းစနစ်ဖြစ်ပါတယ်။ ဒါပေမယ့် တပ်ဆင်စီမံရတာ ရိုးရှင်းလွယ်ကူမှာ မဟုတ်ပါဘူး။

Shard Server တွေ တိုးတဲ့လျှော့တဲ့အခါ၊ Hardware အပြောင်းအလဲပြုလုပ်လိုတဲ့အခါ Data တွေ ဆုံးရှုံးမှုမရှိအောင် Migrate လုပ်တဲ့ လုပ်ငန်းစဉ်ကိုစီမံရပါမယ်။ Shard Server အဖြစ်သုံးထားတဲ့ Replica Sets တစ်ခုလုံး၊ အကြောင်း အမျိုးမျိုးကြောင့် အလုပ်မလုပ်တဲ့အခါ ဘယ်လိုလုပ်မလဲဆိုတာ စဉ်းစားရပါမယ်။ Replica Sets တွေကနေ Data Read လုပ်တဲ့အခါ အပြည့်အဝ Sync မလုပ်ရသေးတဲ့ Secondary Server ကနေ Read လုပ်မိလို့ မပြည့်စုံတဲ့ Data ပြန် ရနေတာမျိုး မဖြစ်အောင် စီမံရပါတော့မယ်။ MongoDB က ဒီလိုပြဿနာတွေ

ကို ပြေလည်အောင် ဖြေရှင်းရမယ့် နည်းစနစ်တွေ ပေးထားပေမယ့်၊ အတိုင်းအတာတစ်ခုထိ ခက်ခဲမှုတွေ ကတော့ ရှိလာမှာပဲဖြစ်ပါတယ်။

နောက်အားနည်းချက် တစ်ခုကတော့ Server တွေပိုလိုတဲ့အတွက် စတင်တပ်ဆင်ခ စရိတ်ကြီးနိုင်ပါတယ်။ မလိုအပ်ပဲနဲ့ တော့ ဒီနည်းကို မသုံးသင့်ပါဘူး။ အသုံးပြုသူ သိန်းသန်းချီပြီး ရှိလာနိုင်စရာရှိတဲ့ App တွေမှာဆိုရင် တော့ ဒီနည်းကိုပဲ အသုံးပြုကြရမှာပါ။

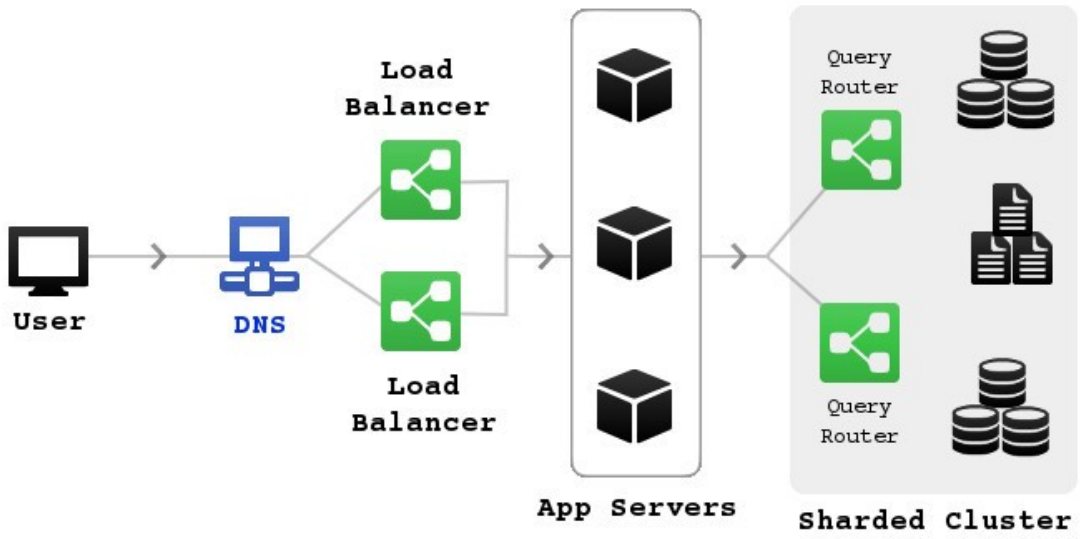
20.5 – Multiple Load Balancers and Query Routers

အသုံးပြုသူသိန်းသန်းချီပြီး ရှိလာတဲ့အခါ ဖြစ်လာနိုင်တဲ့ပြဿနာ ရှိပါသေးတယ်။ အဲ့ဒါကတော့ Load Balancer ကိုသုံးပြီး App ကို Scale လုပ်လို့ရအောင် စီမံထားပေမယ့် Load Balancer ကိုယ်တိုင်က များပြားလွန်းလှတဲ့ Request တွေကို လက်ခံ စီမံနိုင်ခြင်းမရှိတော့တာမျိုး ဖြစ်လာနိုင်ပါတယ်။ ဒီလိုအခြေအနေမှာတော့ Load Balancer တွေကို ထပ်တိုးဖို့ လိုအပ်လာမှာဖြစ်ပါတယ်။ အလားတူပဲ Sharded Cluster မှာလည်း Data Request များလို့ Query Router တစ်ခု တည်းက လက်ခံစီမံနိုင်ခြင်းမရှိတဲ့ အခြေအနေမှာ Query Router တွေ ထပ်တိုးပေးဖို့ လိုအပ်မှာဖြစ်ပါတယ်။

Load Balancer တစ်ခုထက်ပိုအသုံးပြုရတဲ့အခါ သူတို့ရဲ့ရှေ့က ထပ်ဆင့် Load Balancer သဘောမျိုး ဝင်ရောက်လာတာ ကတော့ DNS Server ဖြစ်ပါတယ်။ DNS Server ရဲ့ အခြေခံတာဝန်က Domain Name တွေရဲ့ IP Address ကို Resolve လုပ်ပေးနိုင်ဖို့ ဖြစ်ပါတယ်။ ဒါပေမယ့် Domain Name တစ်ခုအတွက် IP Address နှစ်ခုသုံးခု သတ်မှတ်ထား မယ်ဆိုရင် DNS Server ကလည်း Round Robin Algorithm ကိုသုံးပြီး IP Address တွေကို အလှည့်ကျ ပြန်ပေးမှာ ဖြစ်ပါတယ်။

ဥပမာ – example.com Domain Name အတွက် 123.45.67.89 ဆိုတဲ့ IP Address နဲ့ 123.45.67.99 ဆိုတဲ့ IP Address နှစ်ခုသတ်မှတ်ထားတယ်ဆိုရင်၊ DNS Server က DNS Lookup Request တစ်ခုကို လက်ခံရရှိတဲ့အခါ ပထမ အကြိမ်မှာ 123.45.67.89, 123.45.67.99 လို့ အကြောင်းပြန်ပေးမှာဖြစ်ပြီး နောက်တစ်ကြိမ်မှာ 123.45.67.99, 123.45.67.89 လို့ အကြောင်းပြန်ပေးမှာဖြစ်ပါတယ်။ သက်ဆိုင်ရာ Domain Name အတွက် IP Address စာရင်းကို အကြောင်းပြန်တဲ့အခါ Round Robin နဲ့ အလှည့်ကျစီပြီး ပြန်ပေးမှာဖြစ်ပါတယ်။

ဒါကြောင့် Load Balancer တွေကို ကိုယ်ပိုင် IP Address တွေနဲ့ Setup လုပ်ထားပြီး DNS မှာ အဲ့ဒီ IP Address စာရင်းကိုပေးလိုက်မယ်ဆိုရင် DNS Server က Load Balancer တစ်ခုသဖွယ် ရှေ့ကနေ ဆောင်ရွက်ပေးနေမှာပဲ ဖြစ်ပါ တယ်။



ပုံ (၂၀.၅) - Multiple Load Balancers

ဒီနည်းကတော့ ပိုပြီးပြည့်စုံလာတာနဲ့အမျှ တပ်ဆင်စီမံရတာလည်း ပိုပြီးရှုပ်ထွေးခက်ခဲလာပြီဖြစ်ပါတယ်။ DNS Server ကို Load Balancer အနေနဲ့အသုံးပြုတဲ့အခါ သတိပြုရမယ့်အချက်တစ်ချို့ ရှိပါတယ်။ DNS Server ဟာ IP Address စာရင်းကို ပေးတဲ့အခါ အရှိအတိုင်းပေးမှာဖြစ်ပါတယ်။ သက်ဆိုင်ရာ IP Address မှာ Setup လုပ်ထားတဲ့ Host တွေ အလုပ်မှလုပ်ရဲ့လား စစ်ဆေးနေမှာ မဟုတ်ပါဘူး။ Nginx Load Balancer ကတော့ App Server တစ်ခု Down နေရင် အခြား App Server တစ်ခုကို ပြောင်းလဲအလုပ်လုပ်ပေးနိုင်ပေမယ့်၊ DNS ကတော့ အဲ့ဒီလိုလုပ်ပေးမှာ မဟုတ်ပါဘူး။ သတ်မှတ်ထားတဲ့ IP Address ကိုသာ မူသေပြန်ပေးနေမှာဖြစ်လို့ Load Balancer တွေအားလုံး အလုပ်လုပ်နေကြောင်း သေချာစေဖို့နဲ့ အကြောင်းအမျိုးမျိုးကြောင့် Load Balancer တစ်ခု အလုပ်မလုပ်တဲ့အခါ ဘယ်လိုစီမံမလဲဆိုတဲ့ ကိစ္စမျိုးတွေ ကြိုတင်ပြင်ဆင်ထားရမှာ ဖြစ်ပါတယ်။

DNS ရဲ့နောက်ပြဿနာတစ်ခုကတော့ Cache ဖြစ်ပါတယ်။ Web Browser လို Client တွေရော ကြားခံ DNS Resolver တွေကပါ IP Address စာရင်းကို Cache လုပ်ထားတက်ကြပါတယ်။ ဒီနည်းနဲ့ Domain Name တစ်ခုရဲ့ IP Address ကို သိရဖို့အတွက် ထပ်ခါထပ်ခါ DNS Resolve လုပ်နေစရာမလိုတော့ပဲ စွမ်းဆောင်ရည် ပိုကောင်းစေဖို့ ဖြစ်ပါတယ်။ ဒါပေမယ့် ဒီနည်းကြောင့် DNS Server မှာ IP Address တွေထပ်မံတိုးဖြည့်ခြင်း၊ ပြင်ဆင်ခြင်း လုပ်ငန်းတွေ လုပ်လိုက်ရင် ချက်ခြင်းသက်ရောက်မှာမဟုတ်ပါဘူး။ Browser နဲ့ Resolver တွေက သတ်မှတ်ကာလတစ်ခုအထိ ပြင်ဆင်မှု Update ကို မရယူပဲသူတို့မှာရှိတဲ့ Cache ကိုသာဆက်လက်အသုံးပြုနေမှာဖြစ်ပါတယ်။ ဒီအချက်ကိုလည်း ထည့်သွင်း စဉ်းစားရတော့မှာဖြစ်ပါတယ်။

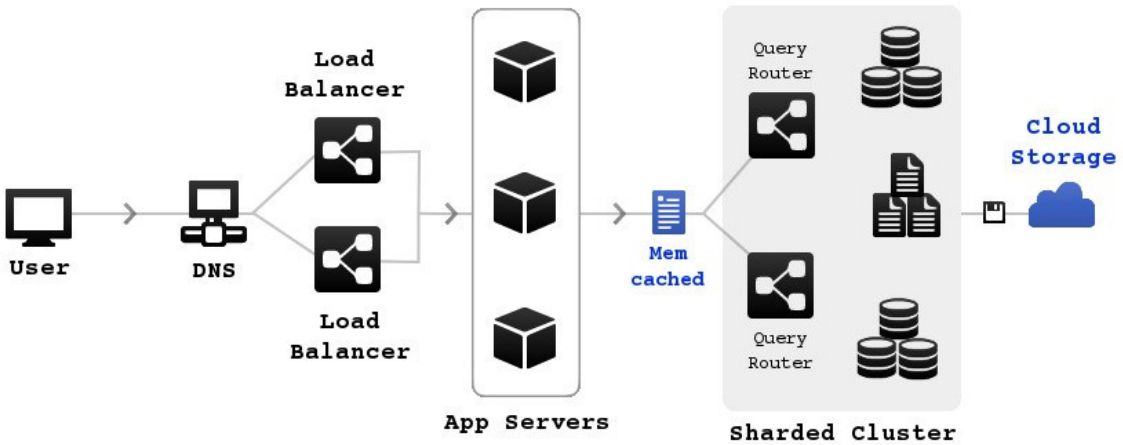
Sharded Cluster မှာ လိုအပ်လို့ Query Router တွေ တိုးတဲ့အခါမှာလည်း အကန့်အသတ်တစ်ခုရှိပါတယ်။ MongoDB ရဲ့ ကန့်သတ်ချက်အရ Client တစ်ခုဟာ Query Router တွေကို အလှည့်ကျသုံးလို့ မရပါဘူး။ Query Router တစ်ခုကိုသာ အမြဲသုံးရမှာဖြစ်ပါတယ်။ ဥပမာ – App Server (၃) ခု ထားတယ်ဆိုရင် App Server A က Query Router 1 ကို အသုံးပြုတယ်ဆိုရင် နောင်ကိုလည်း Query Router 1 ကိုပဲ အသုံးပြုရမှာဖြစ်ပါတယ်။ တနည်းအားဖြင့် Automatic Load Balance လုပ်လို့ အဆင်မပြေတော့ပဲ ဘယ် App Server က ဘယ် Query

Router ကို သုံးရမယ်လို့ ကြိုတင်သတ်မှတ်ထားရမှာဖြစ်ပါတယ်။ ဒါကြောင့် ပြဿနာတစ်စုံတစ်ရာရှိလာတဲ့ အခါ Debug လုပ်အဖြေရှာ ရတာ ထင်သလောက် လွယ်မှာမဟုတ်တော့ပါဘူး။ Code ကြောင့်လည်း ဖြစ်နိုင်ပါ တယ်။ Load Balancer ကြောင့်လည်း ဖြစ်နိုင်ပါတယ်။ Query Router ကြောင့်လည်း ဖြစ်နိုင်ပါတယ်။ Sharded Cluster Configuration ကြောင့်လည်း ဖြစ်နိုင်ပါတယ်။ Architecture ရှုပ်ထွေးလာတာနဲ့အမျှ စီမံရတာလည်း လိုက်လံ ရှုပ်ထွေးခက်ခဲလာမှာပဲ ဖြစ်ပါတယ်။ ဒါကြောင့်လည်း Software Team ကြီးတွေမှာ ဒီလို Architecture ပိုင်း စီမံသူတွေကို သီးခြားခန့်အပ်တာဝန်ပေးထားလေ့ ရှိကြတာပဲ ဖြစ်ပါတယ်။

20.6 – Memcached & Cloud Storage

Server Architecture စဉ်းစားတဲ့အခါ စီမံရလွယ်ကူမှု၊ ကုန်ကျစရိတ်နဲ့၊ ရရှိနိုင်တဲ့ Performance ကို ချိန်ညှိကြရ ပါတယ်။ ပိုကောင်းတဲ့ Performance ကိုလိုချင်တဲ့အခါ စီမံရခက်ခဲလာပြီး ကုန်ကျစရိတ်များလာနိုင်ပါတယ်။ ကုန်ကျစရိတ်သက်သာ ပြီး စီမံရလွယ်ကူတဲ့အခါ အမြင့်ဆုံးစွမ်းဆောင်ရည်ကို ရရှိချင်မှရရှိပါလိမ့်မယ်။

Server Architecture ရဲ့ စွမ်းဆောင်ရည် ပိုကောင်းစေဖို့အတွက် Memcahced ကိုလည်း ထည့်သွင်းဖြည့်စွက်နိုင် ပါသေး တယ်။ Memcached ရဲ့ လုပ်ဆောင်ချက်လိုရင်းကတော့၊ App Server ကို Cache Proxy နဲ့ Cache လုပ် သလိုပဲ Database Server ကို Memcached နဲ့ Cache လုပ်နိုင်ခြင်းဖြစ်ပါတယ်။ အသုံးများတဲ့ Data တွေကို Memcached Server ရဲ့ Memory ပေါ်မှာ တင်ထားလိုက်ခြင်းအားဖြင့် ဆက်သွယ်ရယူရတာ ပိုမြန်သွားသလို၊ အချက်အလက်တစ်ခု လိုချင်တိုင်း Database Server ကို အမြဲတမ်းသွားယူနေစရာမလိုတော့တဲ့အတွက် စွမ်း ဆောင်ရည် ပိုကောင်းသွားမှာပဲ ဖြစ်ပါတယ်။



ပုံ (၂၀.၆) - Memcached and Cloud Backup in Architecture

ပုံ (၂၀.၆) ကိုလေ့လာကြည့်ရင် App Server နဲ့ Sharded Cluster ကြားမှာ Memcached ကို ကြားခံပေးထားတာ ကို တွေ့ရမှာဖြစ်ပါတယ်။

နောက်ထပ်ထပ်မံဖြည့်စွက်သင့်တဲ့အချက်ကတော့ Backup ပဲဖြစ်ပါတယ်။ Backup ဆိုတာ စနစ်ကြီးတစ်ခုလုံး ကို လုပ်စရာ လိုလေ့မရှိပါဘူး။ Database ထဲက Data တွေကိုသာ Backup လုပ်ကြရတာပါ။ Database ထဲက Data တွေကို ကာလ တစ်ခုသတ်မှတ်ပြီး Export ထုတ်ယူကာ အလိုအလျောက် သိမ်းဆည်းအောင် စီမံထားသင့်

ပါတယ်။ ကာလတစ်ခုဆိုတာ တစ်နေ့တစ်ကြိမ်လည်းဖြစ်နိုင်ပါတယ်။ (၄) နာရီခြား တစ်ကြိမ်လည်း ဖြစ်နိုင်ပါတယ်။ ကိုယ့် App ရဲ့ Data အထုတ် အသွင်း ပြုလုပ်မှု Frequency ပေါ်မူတည်ပြီး ပုံမှန် Backup Routine ကာလ တစ်ခုကိုသတ်မှတ်ထားရမှာဖြစ်ပါတယ်။ ဥပမာ - တစ်နေ့တစ်ကြိမ် Backup လုပ်ယူတယ်ဆိုရင် အကြောင်းအမျိုးမျိုးကြောင့် Data တွေပျက်စီးဆုံးရှုံးရတဲ့အခါ နေ့စဉ် Backup Data အမြဲရှိနေမှာဖြစ်လို့ ပြီးခဲ့တဲ့တစ်ရက် အခြေအနေနဲ့ လိုအပ်ရင် Restore ပြန်လုပ်နိုင်မှာဖြစ်ပါတယ်။

Backup နဲ့ပက်သက်ရင် စဉ်းစားရမယ့် အဓိကအချက်ကတော့ Backup ဖိုင်တွေကို ဘယ်လိုသိမ်းမလဲဆိုတဲ့ အချက် ဖြစ်နိုင် ပါတယ်။ Backup ဖိုင်တွေသိမ်းဆည်းဖို့အတွက် NAS, SAN, Tape Backup စတဲ့ Storage Device တွေနဲ့ သိမ်းဆည်း နိုင်ပါတယ်။ ဒါပေမယ့် လက်တွေ့မှာ Backup ဆိုတာ လိုရမယ်ရ သေချာအောင် ယူထားတာ ဖြစ်ပါတယ်။ အမြဲ ပြန်သုံး နေဖို့ လိုအပ်တဲ့အရာမဟုတ်ပါဘူး။ တနည်းအားဖြင့် ပြန်လည်ရယူအသုံးပြုရ လွယ်အောင်ဆိုပြီး ကိုယ့် Architecture ထဲမှာ တွဲသိမ်းနေဖို့ မလိုအပ်ပါဘူး။ နောက်တစ်ချက်က သေချာအောင် Backup လုပ်ပြီး ကိုယ့် Architecture နဲ့ တွဲသိမ်း ထားပါဦးတဲ့၊ Datacenter ကြီးမီးလောင်သွားရင် ဘယ်လိုလုပ်မလဲ။ မဖြစ်နိုင်ဘူးလို့ ပြောလို့ မရပါဘူး၊ သေချာအောင် Backup ယူတယ်ဆိုပြီး၊ ပြဿနာရှိလာလို့ Backup ပါ ရောပါသွားရင်တော့ မဟုတ်သေးပါဘူး။ ဒါကြောင့် Backup ဖိုင်တွေကိုတော့ Google Drive, Amazon S3 စတဲ့ Cloud Storage တွေပေါ်မှာ ခွဲခြားသိမ်းဆည်းထားခြင်းက ပိုပြီး သဘာဝကျမှာဖြစ်ပါတယ်။ ဖိုင်တွေသိမ်းဆည်းဖို့အတွက် ဈေးကြီးတဲ့ SAN Device တွေ ဝယ်စရာမလိုတော့တဲ့အတွက် စရိတ်အရသက်သာသွားမှာဖြစ်သလို၊ အကြောင်းအမျိုးမျိုးကြောင့် Server Architecture ထိခိုက်ပျက်စီး ရင်လည်း Backup ကတော့ သီးခြားကျန်ရှိနေမှာ ဖြစ်ပါတယ်။

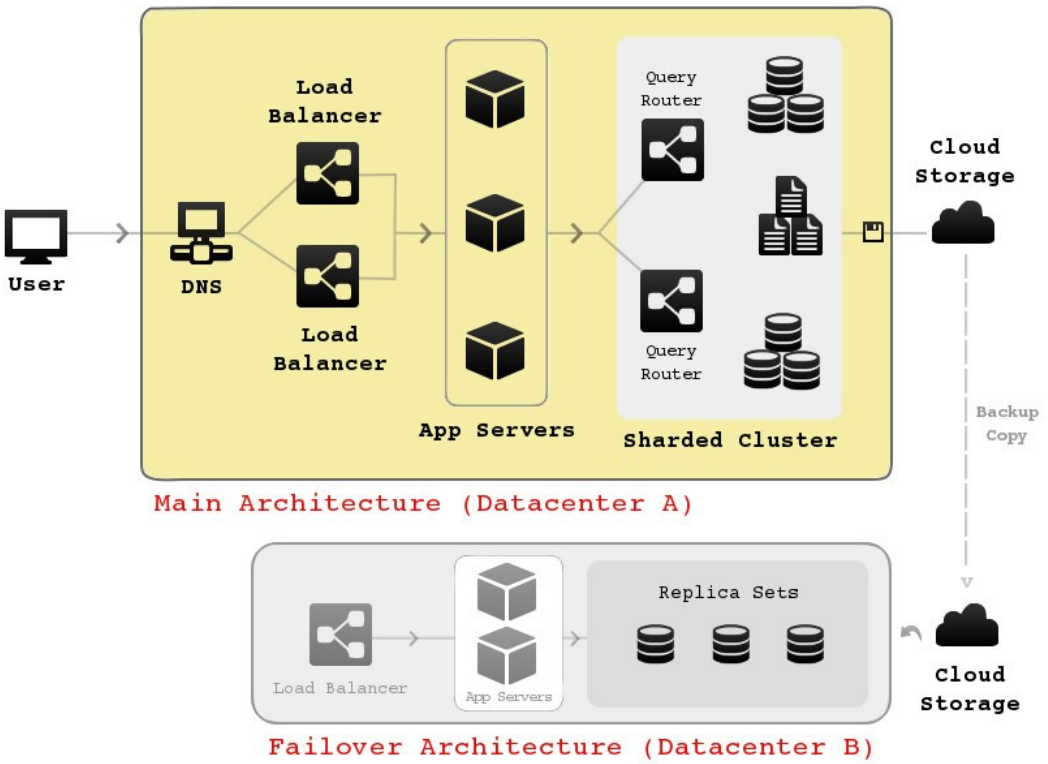
20.7 – Failover Architecture to Embrace Disasters

Data Backup ကို ပုံမှန်ပြုလုပ်ခြင်းအားဖြင့် Data ပျက်စီးဆုံးရှုံးမှုကို ရင်ဆိုင်နိုင်မှာဖြစ်ပေမယ့်၊ အကြောင်းအမျိုးမျိုး ကြောင့် Server Architecture ကြီးတစ်ခုလုံး ပျက်စီးသွားမယ် (သို့မဟုတ်) အလုပ်မလုပ်တော့ဘူးဆိုရင် ဘယ်လိုလုပ်မလဲ ဆိုတာလည်း စဉ်းစားစရာပါ။ ရေးကြီးခြင်း၊ မုန်းတိုင်း၊ ငလျင် စတဲ့ သဘာဝ ဘေးအန္တရာယ်တွေကြောင့်လည်း Server Architecture ကို ထိခိုက်နိုင်သလို၊ မသမာသူများရဲ့ တိုက်ခိုက်မှုကြောင့်လည်း Server Architecture ကို ထိခိုက်ပျက်စီး စေနိုင်ပါတယ်။ ဒီလိုအခြေအနေမျိုး ကြုံလာရင် ရင်ဆိုင်နိုင်ဖို့အတွက် Failover Architecture ကိုလည်း ထည့်သွင်းစဉ်းစား ထားသင့်ပါတယ်။

Failover Architecture ဟာ ဘယ်လိုဖြစ်သင့်တယ်ဆိုတာကို ပုံသေပြောလို့တော့မရတော့ပါဘူး။ အခြေအနေတွေက တစ်ခုနဲ့တစ်ခု တူမှာမဟုတ်ပါဘူး။ အခြေခံ Procedure ကတော့ ဒီလိုပါ။

1. Cloud Storage မှာ Data Backup ကို အမြဲပြုလုပ်ထားသင့်ပါတယ်။
2. အဲ့ဒီ Data Backup ကို အခြား Cloud Storage တစ်ခုမှာ ထပ်ဆင့် Backup ပြုလုပ်ထားသင့်ပါတယ်။
3. သီးခြား Datacenter တစ်ခုမှာ လက်ရှိ Architecture လောက် ပြည့်စုံဖို့မလိုအပ်ပေမယ့် လက်ရှိ Architecture နည်းတူ အလုပ်လုပ်နိုင်တဲ့ Failover Architecture တစ်ခုကို Setup လုပ်ထားသင့်ပါတယ်။
4. Failover Architecture မှာပါဝင်တဲ့ App Server တွေ ဟာ အမြဲအလုပ်လုပ်စရာမလိုပဲ Stand-by အနေအထားနဲ့ အသင့်ရှိနေရမှာ ဖြစ်ပါတယ်။

5. Failover Architecture မှာပါဝင်တဲ့ Database Server ထဲကို ပင်မ Architecture ကနေရရှိလာတဲ့ Data Backup နဲ့ အမြဲ Restore လုပ်ပြီး အသင့်ပြင်ထားသင့်ပါတယ်။
6. ဒီနည်းနဲ့ Failover Architecture ထဲက Database Server မှာ Data တွေ အမြဲအသင့်ရှိနေမှာဖြစ်ပါတယ်။



ပုံ (၂၀.၇) - Failover Architecture

ဒီနည်းနဲ့ အကြောင်းအမျိုးမျိုးကြောင့် ပင်မ Architecture အလုပ်မလုပ်တော့ရင်တော့ Backup Failover Architecture က ဆက်လက်အလုပ်လုပ်ပေးနေနိုင်မှာပဲဖြစ်ပါတယ်။ ဒီနေရာမှာ ထပ်မံစဉ်းစားစရာရှိလာတာကတော့ ပင်မ Architecture ကို ဘယ်ပုံဘယ်နည်း Recover ပြန်လုပ်မလဲဆိုတဲ့ ကိစ္စနဲ့ ပင်မ Architecture ပြန်လည်အလုပ်လုပ်နိုင်တဲ့အခါ Failover နဲ့ ပင်မ Architecture ဘယ်လို Migrate ပြန်လုပ်မလဲဆိုတဲ့ ကိစ္စတွေပါ စဉ်းစားရတော့မှာဖြစ်ပါတယ်။

Conclusion

ကနေ့အချိန်မှာ Full-stack Developer တစ်ဦးဟာ Software ကို ရေးသားဖန်တီးပေးယုံသာမက Server Architecture ကိုလဲ ကိုယ်တိုင်စီမံနိုင်ဖို့ လိုအပ်ပါတယ်။ လက်တွေ့မှာ ကိုယ့်လိုအပ်ချက်နဲ့ ကိုက်ညီအောင် Server Architecture Design ကို ကိုယ်တိုင်ရေးဆွဲသတ်မှတ်ဖို့ လိုနိုင်ပါတယ်။ အဲ့ဒီလို ရေးဆွဲရာမှာ အထောက်အကူဖြစ်စေဖို့အတွက် အသုံးများ ပြီး ဖြစ်နိုင်ခြေရှိတဲ့ Architecture များကို ရွေးထုတ်ဖော်ပြခဲ့ခြင်း

ဖြစ်ပါတယ်။ ဒါ့အပြင် ကျွန်တော်တို့ရေးသားတဲ့ Software ဟာလည်း Server Architecture နဲ့ကိုက်ညီဖို့ လိုအပ်ပါတယ်။ ဥပမာ – Memcached သုံးမယ်ဆိုရင် ကျွန်တော်တို့ Software ကလည်း Memcached Server ကနေ Data ကို ရယူကြည့်ပြီး Cache မရှိမှ အမှန်တစ်ကယ် Database Server ကို ဆက်သွယ်အောင် ရေးသားပေးရမှာဖြစ်ပါတယ်။ တနည်းအားဖြင့် Architecture ကသာ Scalable ဖြစ်ဖို့လိုအပ်ခြင်းမဟုတ်ပဲ၊ ကျွန်တော်တို့ရေးသားတဲ့ Software တွေကလည်း Scalable Architecture ပေါ်မှာ အလုပ်လုပ်နိုင်တဲ့ Scalable Software တွေဖြစ်ဖို့လိုအပ်ပါတယ်။ ဒါကြောင့် Server Architecture နဲ့ ပက်သက်တဲ့ ဗဟု သုတဟာ ကိုယ်တိုင် Server Architecture တွေတပ်ဆင်စီမံတဲ့နေရာမှာသာမက Scalable Software တွေ ရေးသား ဖန်တီးရာမှာလည်း အထောက်အကူဖြစ်စေမှာဖြစ်ကြောင်း ဖော်ပြလိုက်ရပါတယ်။

နိုဂုံးချုပ်

Rockstar Developer

Rockstar Developer ဆိုတာ အဖွဲ့လိုက်ပူးပေါင်းဆောင်ရွက်
နိုင်စွမ်းရှိပြီး၊ ပြုပြင်ထိမ်းသိမ်းရလွယ်ကူတဲ့ Maintainable Code နဲ့
အခြေခံခိုင်မာတဲ့ Scalable Code တွေကိုရေးသားနိုင်တဲ့အပြင်
စဉ်ဆက်မပြတ် လေ့လာသင်ယူနေတဲ့သူဖြစ်ပါတယ်။

Rockstar Developer Course

Project Management, Web Service, Server Architecture
NodeJS အစရှိသည် ဤစာအုပ်ပါ အကြောင်းအရာများကို
စာရေးသူကိုယ်တိုင် သင်ကြားပေးခြင်းဖြစ်သည်။
ဆက်သွယ်ရန် - (၀၉) ၇၃၁ ၆၅၉ ၆၂

http://eimaung.com/courses

နိဂုံးချုပ်

ဒီစာအုပ်ဟာ စာဖက်သူကို သာမာန်ထက် (၁၀) ဆအလုပ်ပိုတွင်တဲ့ 10x Programmer (သို့မဟုတ်) Rockstar Developer တစ်ဦးဖြစ်လာစေရန် တစ်ဖက်တစ်လမ်းကနေ အထောက်အကူဖြစ်စေဖို့ ရည်ရွယ်ရေးသားထားခြင်းဖြစ်ပါတယ်။ သာမာန်ထက် (၁၀) ဆအလုပ်ပိုတွင်တဲ့ Rockstar Developer တစ်ဦးမှာ ရှိလေ့ရှိတဲ့ အရည်အသွေးတွေ အများ ကြီးရှိပါတယ်။ အဲ့ဒီထဲက အဓိက (၅) ချက်ကို ရွေးထုတ်ဖော်ပြရရင်တော့ –

1. အဖွဲ့လိုက်ပူးပေါင်းဆောင်ရွက်နိုင်သည့် အရည်အသွေးရှိခြင်း
2. ဖတ်ရှုနားလည်ရလွယ်ပြီး၊ ပြင်ဆင်ထိမ်းသိမ်းရလွယ်ကူသည့် Code များ ရေးသားနိုင်ခြင်း
3. သဘောသဘာဝပိုင်း ပိုင်နိုင်၍ ရှေ့ရှည်အဆင့်မြှင့်တင်မှုအတွက် အခြေခံခိုင်မာသည့် Code များ ရေးသားနိုင်ခြင်း
4. အလုပ်တွင်စေရန်အတွက်၊ ဆက်စပ်နည်းပညာများကို ထိရောက်အကျိုးရှိအောင် အသုံးချနိုင်ခြင်း
5. အဆက်မပြတ် လေ့လာသင့်ယူတက်သည့် အလေ့အထရှိခြင်း

– တို့ပဲဖြစ်ပါတယ်။

ဒီစာအုပ်မှာ လေ့လာခဲ့တဲ့ Source Code Management System, Issue Tracking System, SCRUM စတဲ့ အကြောင်းအရာတွေကို စာဖက်သူရဲ့ အဖွဲ့လိုက်ပူးပေါင်းဆောင်ရွက်နိုင်မှုကို အထောက်အကူဖြစ်စေလိမ့်မယ်လို့ မျှော်လင့်ပါတယ်။ SOLID, TDD, SOA စတဲ့ သဘောသဘာဝများနဲ့အတူ BackboneJS လို Code Architecture နည်းပညာအကူအညီနဲ့ ပြင်ဆင် ထိမ်းသိမ်းရလွယ်ကူတဲ့ Code များရေးသားရာမှာ အထောက်အကူ ဖြစ်စေလိမ့်မယ်လို့လည်း မျှော်လင့် ပါတယ်။ NodeJS, MongoDB, Nginx စတဲ့ နည်းပညာများကိုလည်း ထည့်သွင်းဖော်ပြခဲ့တဲ့အတွက် ရေရှည်အတွက်အခြေခံခိုင်မာပြီး Scalable ဖြစ်တဲ့ Code များ ရေးသားရာမှာလည်း အထောက်အကူဖြစ်စေလိမ့်မယ်လို့ မျှော်လင့်ပါတယ်။ Yeoman, NPM, Bower, Grunt, Docker စတဲ့ Tool များကိုလည်း ထည့်သွင်းဖော်ပြခဲ့တဲ့အတွက် လုပ်ငန်းစွမ်းဆောင်ရည် ပိုမိုကောင်းမွန်စေဖို့အတွက် အထောက်အကူဖြစ်စေနိုင်တဲ့ စိတ်ကူးအိုင်ဒီယာများလည်း ရရှိလိမ့်မယ်လို့ မျှော်လင့်ပါတယ်။

ဒီစာအုပ်မှာ၊ Rockstar Developer တစ်ဦး သိရှိသင့်တဲ့ သဘောသဘာဝတွေ၊ နည်းပညာတွေနဲ့ နည်းစနစ်တွေကို တက်နိုင်သမျှစုံလင်အောင်စုစည်းဖော်ပြထားပါတယ်။ ဒီလိုပါဝင်တဲ့အကြောင်းအရာများပြားတဲ့ အတွက်ကြောင့် တစ်ချို့နည်းပညာ တွေအကြောင်းကိုဖော်ပြရာမှာ အခြေခံကျပြီးသတိမူသင့်တဲ့အပိုင်းကိုသာ ဦးစားပေးဖော်ပြပြီး တစ်ချို့အသေးစိတ်တွေကို ချန်လှပ်ခဲ့ရတာတွေလည်းရှိပါတယ်။ ဒါပေမယ့် ဒီစာအုပ်က သက်ဆိုင်ရာကဏ္ဍကို ကိုယ်တိုင်ဆက်လက်လေ့လာ သွားရာမှာ လမ်းစရစေဖို့အတွက် အထောက်အကူပြုလိမ့်လို့မျှော်လင့်ရင်း ဒီနေရာမှာတင် နိဂုံးချုပ်လိုက်ရပါတယ်ခင်ဗျာ။

ကျေးဇူးတင်လွှာ

ဤစာအုပ်အတွက်အမှာစာ ရေးသားပေးခဲ့ကြသော၊ ဆရာဦးထွန်းခိုင်နှင့် ကိုသာထက်တို့အား အထူး ကျေးဇူးတင်ရှိပါသည်။ စာမူကြမ်းဖတ်ရှု၍ မှတ်ချက်နှင့် အကြံပြုချက်များပေးခဲ့ကြသော ကိုမိုးလုံးပြည့်လှိုု၊ ကို ဉာဏ်လင်းထွဋ်၊ ကိုနိုင်လင်းအောင်၊ ကိုတင်အောင်လင်း နှင့် ကိုဝင်းထွဋ်ဟန် တို့ကိုလည်း ကျေးဇူးတင်ရှိ ပါသည်။ မအားလပ်သည့်ကြားမှ စာမူကြမ်းဖတ်ရှုပေးခဲ့ ကြသော ကိုသန့်သက်၊ ကိုစွမ်းထက်အောင်၊ ကိုသီ ဟအေးကျော်၊ ကိုရဲလင်းအောင်နှင့် ကိုကောင်းခန့်မင်းထွန်းတို့အားလည်း ကျေးဇူးတင်ရှိပါသည်။ စာအုပ်မ ထွက်မီကပင်၊ ကြိုတင်မှာယူစနစ်ဖြင့် မှာယူအားပေးပြီး ပံ့ပိုးကူညီခဲ့ကြသူများနှင့် စာအုပ်ဖြစ်မြောက်လာရေး အတွက် အဘက်ဘက်မှ ပံ့ပိုးကူညီခဲ့ကြသူများ အားလုံးကို ကျေးဇူးအထူးတင်ရှိပါသည်။

အိမောင်

Fairway Web

စာဖတ်သူများအားလုံး ကိုယ်စိတ်နှစ်ဖြာ ကျမ်းမာချမ်းသာကြပါစေ။

Website: http://rsdbook.com
Author Profile: http://eimaung.com
Contact Email: author@rsdbook.com
Facebook: http://fb.com/RockstarDeveloperBook

၂၀၁၅ ခုနှစ်၊ စက်တင်ဘာလ (၂) ရက်နေ့တွင် ရေးသားပြီးစီးသည်။
၂၀၁၆ ခုနှစ်၊ ဂျူလိုင်လ (၅) ရက်နေ့တွင် PDF Ebook အဖြစ် ပြင်ဆင်ခြင်း ပြီးစီးသည်။